



ORACLE
NetSuite

SuiteCommerce Developer Documentation

2025.2

December 17, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

Commerce Website Developer Documentation Overview	1
Developer Essentials	3
Overview	3
What Can I Customize?	4
Benefits of the Extension Framework	6
What Resources Do I Need?	7
Commerce Migration to SuiteScript 2.0	12
What is the Development Life Cycle?	14
Choose a Tool	18
Set Up the Developer Environment	22
Overview	22
Install JavaScript Utilities	23
Install Node.js	23
Install Gulp.js	24
Developer Tool Roles and Permissions	26
Create a Custom Role with Fetch and Deploy Permissions	26
Assign the Administrator or a Custom Role to an Employee Record	28
Gulp Commands	29
Gulp Command Reference for Theme and Extension Developer Tools	29
Gulp Command Reference for SCA Developer Tools	35
Next Steps for Each Tool	37
Next Steps for Theme Developer Tools	37
Next Steps for Extension Developer Tools	38
Next Steps for SuiteCommerce Advanced Developer Tools	38
Develop Themes	40
Overview	40
Set Up Theme Developer Tools	41
Theme Development Files and Folders	44
Anatomy of the Base Theme	47
Develop Your Theme	50
Create a Custom Theme	50
Fetch Active Theme Files	52
Customize Pre-Existing Theme Files	55
Add a New File to a Theme	55
Build a New Theme	58
Edit the Theme Manifest	63
Test and Deploy Your Theme	68
Test a Theme on a Local Server	68
Deploy a Theme to NetSuite	70
Best Practices for Creating Themes	74
General Best Practices for Themes	74
HTML Best Practices for Themes	75
Sass Best Practices for Themes	77
Managing Theme Assets	78
Commerce Sass Style Definitions	84
Design Hierarchy	84
Style Definitions	85
Style Guide	87
Mobile First	89
Theme Tutorials	91
Theme Tutorial Prerequisites	91
Customize an Existing Theme	91
Do More With Themes	93

Override Active Extension Files	93
Examples of Customizing Extensions Using the Override Method	95
Enable Customization from Site Management Tools	98
Expose Sass Variables for Customization	99
Organize Variables for Display in SMT	101
Create Skins	104
Customize Site Search Elements	107
Include Translations in Themes	109
Develop Extensions	112
Overview	112
Set Up Extension Developer Tools	114
Extension Development Files and Folders	117
Anatomy of an Extension	121
Models in an Extension	123
Views in an Extension	124
APIs for SuiteCommerce Extensions	127
SuiteCommerce Extensibility API	127
Extensibility API Components	128
Instantiating API Components	137
Use SuiteScript With Your Extension	139
Develop Your Extension	141
Get Started with Extensions	141
Create a Baseline Extension	142
Fetch Active Theme and Extension Files	146
Edit and Create Extension Modules	147
Create Additional Modules for an Extension	148
Work with Views in an Extension	149
Create a View	149
Add a Child View	150
Replace a Child View	156
Work with Events in an Extension	157
Subscribe to Events	160
Cancel the Operation for an Event	160
Edit the Extension Manifest	161
Test and Deploy Your Extension	167
Test an Extension on a Local Server	167
Deploy an Extension to NetSuite	168
Best Practices for Developing Commerce Extensions	172
General Best Practices	172
Use Custom Handlebars Helpers	175
Use Multiple Modules in an Extension	178
Extension Tutorials	185
Use the Search Component to Set the 'url' property of Models	185
Capture Form Data on the Login/Register Page	193
Create a Custom Content Type	197
Create Your CCT as an Extension	198
Create the Entry Point JavaScript Files	199
Implement the View File	201
Implement the Template File	205
Implement the Sass Files	206
Test and Deploy the CCT Extension	207
Access Model and Collection Data in a Child View	207
Manage SuiteCommerce Analytics Data Cookie Preferences	215
Do More With Extensions	224
Access Configuration Properties	224

Localize Text in an Extension	225
Extensions for Site Management Tools (SMT)	227
Create Page Types	227
Register Page Types and Templates for SMT	231
Create Custom Content Types for an Extension	239
Modal Dialogs in SCIS	240
Implement Asynchronous and Synchronous Methods	243
Asynchronous Methods	244
Backend Asynchronous and Synchronous Methods	245
Make a View Available Anywhere in Your Templates	246
Manage and Bundle Themes and Extensions	249
Overview	249
Management Resources for Themes and Extensions	250
Deploy to a Sandbox or Custom SSP Application	251
Deploy to a NetSuite Sandbox	251
Deploy to a Custom SSP Application	252
Bundle Themes and Extensions as SuiteApps	257
Bundle Process for SuiteCommerce	257
Declare Target Versions	260
Update Themes and Extensions	261
Developer Tools Reference	264
Overview	264
Token Based Authentication	265
Mixed Domains in a Local Server	267
Secure HTTP (HTTPS) with the Local Server	268
Troubleshooting the Developer Tools	272
Develop SCA Customizations	277
Overview	277
Customize with SCA Tools	278
SCA Framework Technologies	280
TypeScript	280
Model View Controller (MVC) and Backbone.js	281
Asynchronous Module Definitions (AMD) and RequireJS	282
Logic-less Templates and Handlebars.js	283
Templates and the Template Context	283
SCA Module Architecture	287
Dependencies	287
Application Modules	288
Feature Modules	289
SCA Feature Modules	290
Entry Point	290
Routers	290
Views	291
SCA Models, Collections, and Services	295
Frontend Models	295
Asynchronous Data Transactions	296
Collections	298
Services and Backend Models	299
Architecture Overview (Pre-Vinson)	301
Services and Backend Models in Custom Modules	303
Define a JSON Object to Represent a NetSuite Record	303
Create a Backend Model	303
Create a Service to Handle HTTP Requests	306
Set Up SCA Developer Tools	314
Before You Get Started	314

Download SCA Source Files	315
Install Additional Files in the Source Directory	317
Example SCA Customizations	319
Create a Custom Module	319
Modify JSON Configuration Files	325
Extend Frontend Configuration Files	327
Configure Facet Fields	328
Extend the Backend Configuration File	330
Add a Child View to a Composite View	332
Override a Template File	335
Extend a Sass File	338
Add a Sticky Button	340
Customizing the Loading Icon	342
Adding a Custom Web Font	344
Extending Font Awesome	346
Displaying Device-Specific Carousel Images	348
SCA Build Process	352
The distro.json and ns.package.json Files	352
How Files Are Combined	354
Contents of the Deploy and Local Distribution Directories	354
The ssp_libraries.js File	355
Do More With SCA Customizations	356
Create Custom Content Types for SMT	356
Create a Custom Module for Your CCT	356
Create an Entry Point File	357
Create a View File	359
Create a Template File	364
Set Up Your ns.package.json and distro.json Files	365
Customize the Checkout Application	367
Reorder Checkout Modules	368
Add Checkout Modules	369
Add or Remove Checkout Steps	370
Configure Checkout Step Properties	371
Define Checkout Step URLs	372
SCA Reference	373
SCA Source Directory	373
Product Details Page Architecture	376
Changing SCA Developer Tools Connection Information	381
Patches and Upgrades	382
SuiteCommerce Releases and Patches	382
SCA Patches	384
Patches Overview	384
Patch Using Extend Mode	384
Patch Using Override Mode	386
Patch Instructions	388
Update SOAP Library Version for SuiteCommerce Advanced Compatibility	397
Resolve Content-Type Mismatch for Commerce Websites	398
Configuration Changes for Custom Records	408
Changes in Product Reviews and Product List Permissions Levels	411
Add Parameters Required for Shopper Access to Suitelet Internal URLs	465
CMS Page Type Website Records Not Displaying	467
Google Tag Manager Events Not Displaying in Data Layer Upon Checkout	472
Customer not Directed to the Payment Review Page after Checkout using the Checkout Flow	475

Customer not Directed to the Payment Review Page after Checkout after using the Make A Payment Flow	477
Guide to Enable Google Analytics 4	479
Customer Redirected to My Account Overview Instead of Successful Payment Page	498
Hide Add to Cart Button from Facet View	501
Removing Item Redirects to Login Page Under Secure Domain	502
Token-Based Authentication Updates for Developer Tools	503
Domain Bridging Update	508
Address Not Set for Orders for Unfulfillable Items	509
Gift Certificate Error After Return Request Submitted	512
Cannot Disable Default Credit Card	513
Image Tracker Resized After Placing Order	515
Single Secure Domain Requires Login for Pricing	516
Page Not Found Error for SMT Landing Pages	518
CVE-2020-14728: Vulnerability in SuiteCommerce Advanced Services	519
CVE-2020-14729: Vulnerability in SuiteCommerce Advanced Sites	527
Enable Token-Based Authentication	530
Site Pages Not Indexed	550
Deploy to NetSuite Fails with Errors	552
Values Not Stored in Custom Transaction Body Fields	553
Patch Overrides Not Visible in DeployDistribution Folder	555
Shopping User Environment Experiences Degraded Performance	556
beforeShowContent Event Stops Working After Initial Trigger	558
Filter Site Option Does Not Work As Expected	560
Quantity Facet Displays Dollar Sign	562
Custom Overrides Not Applying When Deployed	565
Theme Developer Tool Overrides May Cause Error On Activation	565
Categories Navigate Adapter Event Removed in SMT	566
Categories Do Not Display Properly	569
Incorrect Value for Shipping Estimate Occurs in Shopping Cart	571
Standard Promotion with Inline Discount and Rate as Percentage Not Updating the Amount in Checkout	574
Page With URL Fragments Redirects Too Many Times	576
See Complete List of Stores Link on Store Locator Page Does Not Show Store List	579
Quantity Pricing Displayed in Web Store Even When 'Require Login for Pricing' is Enabled	581
Edited Shipping Address on the Review Your Order Page is Not Showing Changes	584
Custom Page Title in SMT Does Not Display Correctly	586
Currencies Change to the Default in the Shopping Application	589
Order Confirmation Page Not Displayed When Using Single Page Checkout and External Payment	591
Incorrect Discounted Amounts on Checkout Summary	593
DeployDistribution Folder Does Not Include Local Files	596
HTML List Styles Do Not Display	597
Item Search Displays Incorrect Results	605
Category Product Lists Return Page Not Found	607
Cannot Test an Extension on a Local Server	611
Secure Shopping for Site Builder Implementations	613
Cannot Scroll Through Long Menu Lists Using iOS	625
Pages Not Indexed Using Google's Mobile-First Indexing	630
Disabling Display of SuiteCommerce Gift Wrap & Message Extension Transaction Line Fields	632
Users Not Redirected to External Payment System	637
Invoices Do Not Include the Request for a Return Button	640
Unable to Select Item Options Within the SuiteCommerce Configuration Record in NetSuite	644

Incorrect Redirect URL for External Payments	646
npm Error on Implementations	649
Content Appears Incorrectly in a Merchandising Zone	650
Reference My Account Generates Error on Load	653
Error Loading Shopping Page Due to Uncaught TypeError	654
Users Redirected to Checkout Application Instead of Shopping Application	657
Add to Cart Button Does Not Work If Quantity Field Selected	659
URLs with Redundant Facets Generated	661
Content Flickers or Disappears When Browsing the Product Listing Page	665
Enabling Google AdWords Causes Error on Login	667
URL for Commerce Categories Contains Incorrect Delimiters	669
Order Summary for Item-Based Promotions	673
CSS Error Hides First div Element on Product Details Page	675
Invoice Terms Not Included In Order Details	677
Users Required to Re-enter Credit Card Payment Method Details on Payment Page	678
Selected Invoice Not Displayed When Making an Invoice Payment	682
Log In to See Prices Message Appears When Users are Logged In	684
Item Record HTML Meta Data Not Appearing on Page Meta Data	687
Delivery Options Not Appearing After Editing the Cart and Re-entering a Shipping Address ...	689
Order Confirmation and Thank You Page is Blank	692
Matrix Item Options Not Displaying With Google Tag Manager Enabled	695
Delivery Methods Not Appearing in One Page Checkout	698
Mastercard 2-Series BIN Regex Patch	702
Auto-Apply Promotions for Elbrus	710
Change Email Address Patch	722
Duplicate Product Lists in Internet Explorer 11	780
Save for Later Item not Moved to Cart	780
Running Gulp Commands Results in a Syntax Error	783
Missing Promo Code on Return Request	784
Enhanced Page Content Disappears when Resizing the Browser	787
Invoices Page Displays Incorrect Date Sort (pre-Denali)	791
PayPal Payments Cause Error at Checkout	793
Canonical Tags Populated With Relative Paths	795
Shopping Cart Not Scrolling (Mobile)	798
Error When Adding Items to Categories in Site Management Tools	801
Item Search API Response Data not Cached	805
Secure Shopping Domains (Elbrus, Vinson, Mont Blanc, and Denali)	807
Secure Shopping Domain (pre-Denali)	808
PayPal Address not Retained in Customer Record	822
Login Email Address Appears in the Password Reset URL	823
How to Apply .patch Files	827
Migrate SuiteCommerce Advanced	830
Migrate from Aconcagua and Later	830
Migrate From Kilimanjaro and Earlier	833
Upgrade SuiteCommerce to SuiteCommerce Advanced	836

Commerce Website Developer Documentation Overview

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

Whether you're a developer making minor changes to your Commerce website experience, or you need to fully customize your site for a SuiteCommerce Advanced implementation, the following documentation describes the tasks to complete and provides information about the technologies involved.

Available Commerce Developer Documentation Resources

See	To Learn
Developer Essentials	What you can customize, the resources and skills you need to develop customizations, which tool to use, and how to get started with each tool.
Set Up the Developer Environment	How to set up your developer environment to prepare to use the available developer tools.
Develop Themes	How to develop themes to customize the appearance of your Commerce website.
Develop Extensions	How to develop extensions to customize Commerce website functionality.
Manage and Bundle Themes and Extensions	How to manage and maintain your theme or extension during its lifecycle.
Commerce Developer Tools Reference	How to test locally or deploy to NetSuite.
Develop SCA Customizations	How to customize a site that implements the Kilimanjaro release of SuiteCommerce Advanced (SCA) or earlier. You may also need to develop SCA customizations if you are working with a more recent release of SCA and need to access JavaScript, SuiteScript, or configuration objects that are not available using the Extensibility API.
Overview	How to customize your Commerce website's checkout flow using Scriptable Cart when it is not possible to achieve your customization through other means.
Commerce Patches and Upgrades	How updates apply to the Commerce application you are implementing.

Archived Commerce Documentation

For information about releases prior to the Denali version of SuiteCommerce Advanced, see the following documents:

 [SuiteCommerce Advanced & Reference Implementations \(pre-Denali\)](#)

 [Release Notes Prior to SuiteCommerce Advanced Denali](#)

 [Reference Cart & One Page Checkout Guide](#)

Overview

i Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced | SuiteCommerce InStore

Before you start to customize your SuiteCommerce, SuiteCommerce Advanced (SCA), or SuiteCommerce MyAccount (SCMA) implementation, review the following information to learn about the skills and resources you need:

- [What Can I Customize?](#)
- [Benefits of the Extension Framework](#)
- [What Resources Do I Need?](#)
- [What is the Development Life Cycle?](#)
- [Choose a Tool](#)

What Can I Customize?

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced | SuiteCommerce InStore

The term, Commerce website, refers to a site set up for SuiteCommerce, SuiteCommerce Advanced (SCA), or SuiteCommerce MyAccount (SCMA) implementations. The following topics explain how to get started using the elements of the extension framework to customize a Commerce website.

Note: Before you start to customize your Commerce website, be sure you understand the capabilities that are built into Commerce solutions. In many cases, you may be able to configure your site to meet your customer's needs. For more information, see the help topic [Configure Instead of Customize](#).

You can customize your Commerce website as described in the following table.

Customization Option	Description
Website design and appearance	You can alter the layout and style of your website by creating your own custom themes. Using the SuiteCommerce Base Theme as a starting point, you can use the theme developer tools to customize Sass, HTML, and other assets. You can also introduce new images and fonts as assets.
Website functionality	You can add or change the functionality of your website by developing extensions that include JavaScript, SuiteScript, configuration JSON, and other files. You can use the extension developer tools to create a baseline extension and then leverage the SuiteCommerce Extensibility API component classes to build upon that baseline. The Extensibility API lets you extend the core SuiteCommerce application to suit your business needs. Use any of the available component classes to customize the application and add new features or functions.
SCA source code	If you are implementing SCA and your site customization requires access to objects that are not accessible via the Extensibility API, you can use the SCA developer tools to extend SCA source code. If you plan to modify SCA, you should not edit the source code directly. Instead, you should override or extend existing source code to create the custom functionality you need.

Note: Best practice for Commerce website customization is to use the Extensibility API and the theme and extension developer tools to ensure the custom designs and functionality you develop can be easily maintained and implemented across SuiteCommerce and SCA sites and versions of sites. The Extensibility API and the theme and extension developer tools are available with SuiteCommerce and SCA.

Next Steps

To learn more about	See
The extension framework and the ways it streamlines Commerce website customizations	Benefits of the Extension Framework
The skills you need to customize Commerce websites, the developer tool requirements, and supported scripts and languages	What Resources Do I Need?

To learn more about	See
The stages of the development life cycle, tools provided for each stage, and delivery options for themes and extensions	What is the Development Life Cycle?
How to get started with themes or extensions	Choose a Tool

Benefits of the Extension Framework

📌 Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced | SuiteCommerce InStore

The extension framework and Extensibility API were introduced with the Aconcagua (2018.1) release of SuiteCommerce Advanced (SCA). The extension framework includes the Extensibility API, Extension Manager, and the theme and extension developer tools. In addition to the new tools, the extension framework also introduced new formal constructs for Commerce website customizations — themes and extensions.

SuiteCommerce (sometimes referred to as SuiteCommerce Standard) was also introduced with the Aconcagua release. While SuiteCommerce Advanced runs from an unmanaged bundle, allowing changes to source code, SuiteCommerce runs from a managed bundle. This means you cannot modify the SuiteCommerce source code yourself, but you are always running the latest code because NetSuite updates your site automatically.

Prior to the Aconcagua release, customizing an SCA website required techniques such as extending base classes, overriding files, wrapping methods, and even editing SCA source files directly. While providing a great deal of flexibility, this approach to customization created more work for developers such as:

- Tedious and time-consuming procedures to move customizations from one site to another
- Cumbersome processes to include customizations in upgrades from one version of SCA to another
- Developer skills were required to make any changes to website design and functionality, whether minor or significant

Developers needed a better, more reliable way of building customizations and moving them from site to site and version to version. This is why the extension framework was introduced. The extension framework allows new functionality to be introduced and maintained more easily and reliably. The three key elements of the extension framework are:

Extensibility API	A code-level API that provides access to key parts of a Commerce website via standardized objects and methods that remain stable across different sites and different versions.
Extensions	Packaged modules of code that plug into a site after being activated.
Themes	Packaged templates and Sass that can be used to change an entire site's look-and-feel.

With the introduction of these key elements, developers can focus on creating themes and extensions, which are formalized packages that customize Commerce website look and feel and functionality. These packages can be easily versioned, distributed, and activated across different sites and different versions of sites.

The key elements of the extension framework also empower business users. After developers have built and then shipped these theme and extension packages, business users are free to activate them on their site if they like. Business users can then customize and configure the packages (if applicable), roll them back to a previous version, and deactivate them entirely on their own terms.

Regardless of whether you use SuiteCommerce or SCA, the tools and capabilities of the extension framework are available to both products. The crucial difference is that with SCA, you are also able to modify the source code. However, if your SCA customization requires access to objects that are not accessible via the Extensibility API, do not edit the source code directly. Instead, override or extend existing source code to create the custom functionality you need.

What Resources Do I Need?

i Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced | SuiteCommerce InStore

The following topics describe the resources and skills you need to customize Commerce websites. Review the information to ensure you are aware of the requirements and supporting resources as you plan and implement your themes and extensions.

i Note: To build or customize a SuiteCommerce or SuiteCommerce Advanced (SCA) site, you must have a NetSuite account and have SuiteCommerce or SCA and associated bundles provisioned.

- [Required Skills](#)
- [Developer Tool Requirements](#)
- [Supported Scripts, Languages, and APIs](#)
- [Supported Browsers for Commerce Websites](#)
- [Additional Resources](#)

Required Skills

The following skills are needed to develop themes and extensions. The level of experience required depends on the complexity of the changes you want to make and the extent of the features you want to introduce.

Website Customization	Skill	Technologies
Theme	General web development technologies	HTML, Sass/CSS, and Handlebars.js
Extension	General web development technologies	HTML, Sass/CSS, JavaScript, Typescript, REST APIs, model-view libraries (Backbone.js), Node.js/node package manager (npm), and AJAX
	NetSuite technologies	Records, SuiteScript 1.0 and 2.0, services, and searches

i Note: Commerce website setup and maintenance do not require technical skills and can be completed using NetSuite as described in the Store Front topics. Some Commerce website design and layout can be customized using the Site Management Tools (SMT), which do not require programming skills. For more information, see the help topic [Site Management Tools Overview](#). When deciding whether to customize your Commerce website, review [Configure Instead of Customize](#) to be sure you understand all capabilities that are built into SuiteCommerce.

Developer Tool Requirements

SuiteCommerce and SCA include the following tools to help you develop Commerce website customizations:

- Theme developer tools

- Extension developer tools

These tools are essential for several reasons, including:

- They create the local workspace and file structure that are necessary to ensure your theme or extension is compatible with NetSuite.
- They enable you to create, update, and maintain your theme or extension throughout the development lifecycle.

To use the theme and extension developer tools, you need to have the following resources installed in your local environment:

- Node.js JavaScript runtime environment and all of its components, including the Node Package Manager (NPM)
- Gulp.js JavaScript toolkit

For more information about these tools, see [Choose a Tool](#).

Note: If you are developing or customizing an SCA site, the SCA developer tools are also available and they require the same resources as the theme and extension developer tools. However, you should not need to use the SCA developer tools unless you are customizing a site that implements the Kilimanjaro release of SCA or earlier or you need to customize Javascript, SuiteScript, or Configuration objects that are not accessible via the Extensibility API. See [Choose a Tool](#) for more information.

Supported Scripts, Languages, and APIs

You can use standard web development scripts and languages when customizing SuiteCommerce sites, including HTML, CSS, Sass, Handlebars.js, JavaScript, TypeScript, and AJAX. In addition, you need to use NetSuite technologies, like the SuiteScript scripting language, when developing extensions for SuiteCommerce sites.

Note: To ensure compatibility with SEO Page Generator, the JavaScript for your customizations must be valid ECMAScript 5.1. See the [ECMAScript 5.1 Edition Language Specification](#) for more information.

SuiteScript is NetSuite's proprietary scripting language that provides custom objects and methods you can use with JavaScript. The SuiteScript API plays an important role in Commerce site customizations because it enables you to access data stored in NetSuite records. You can also use SuiteScript to store data in NetSuite. To learn more about SuiteScript, see the following topics:

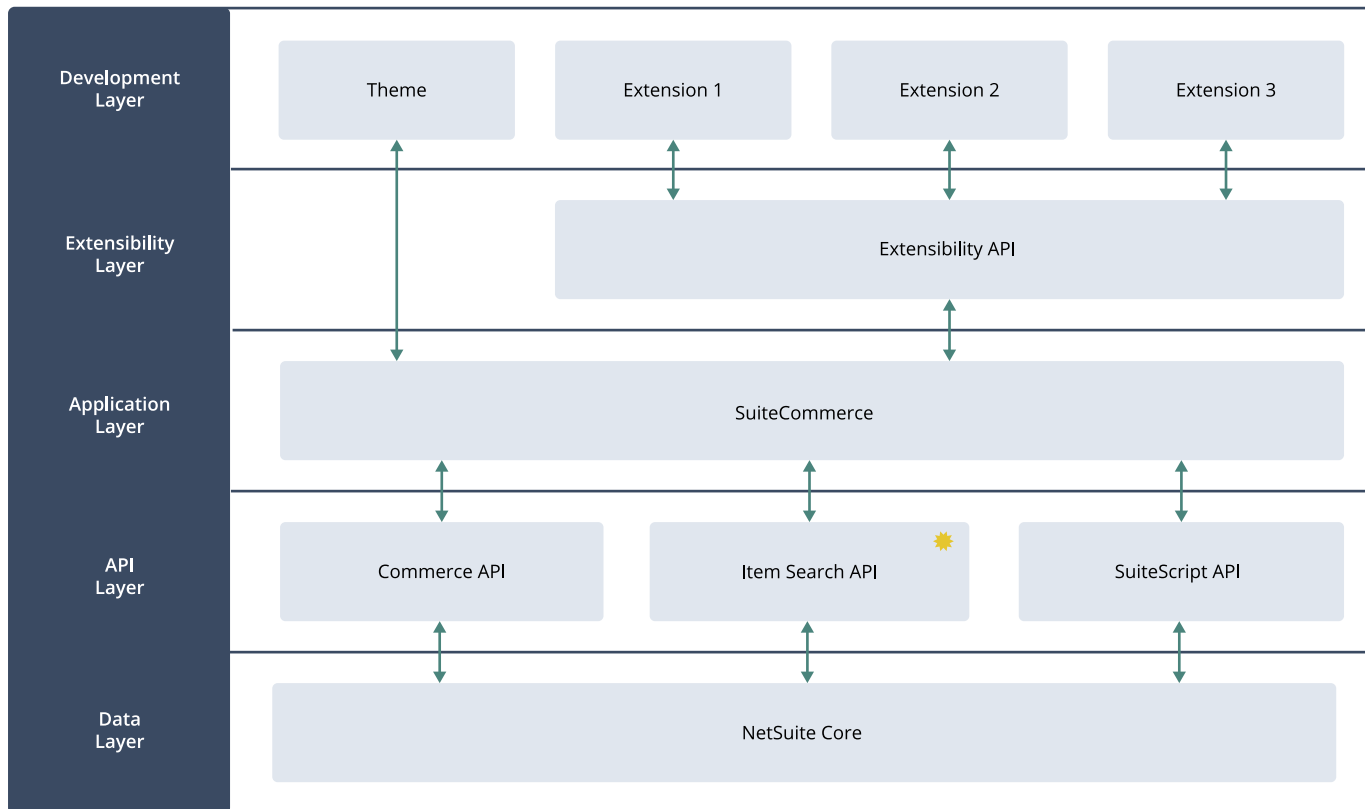
See topic	To learn
SuiteScript Overview	About the differences between SuiteScript versions, including 1.0, 2.0, and 2.1, and to access topics that explain more about what you can do with SuiteScript like using SuiteScript with NetSuite records. Note: SuiteScript 2.1 is not currently supported for Commerce website development. SuiteScript 2.x, 2.0, and 1.0 are supported.
SuiteScript Developer Guide	How to set up your environment for SuiteScript, SuiteScript governance, best practices and more.

See topic	To learn
SuiteScript Records Guide	More about the NetSuite records available to you when using SuiteScript.
SuiteScript 2.x	More about all aspects of SuiteScript 2.0 and 2.1, including API introduction, script types, custom modules.
SuiteScript 2.x API Reference	Details about API objects and modules.
Commerce Migration to SuiteScript 2.0	More about the migration of Commerce website source code to SuiteScript 2.0.

In addition to the SuiteScript API, you can use the following APIs when developing extensions:

API	Description	Can be used with	For more information, see
Extensibility API	Provides classes and methods that enable you to access routers, models, and views in the SuiteCommerce/SCA applications so that you can customize behavior without making changes to the core application code. For example, the Extensibility API enables you to subscribe to events, create views, and add child views. Using the Extensibility API ensures future updates to the SuiteCommerce/SuiteCommerce Advanced applications do not adversely affect your extensions.	■ SuiteCommerce ■ Aconcagua release of SCA and later	■ Overview ■ Extension Tutorials ■ Extensibility API Reference
Commerce API	Exposes shopping objects and methods that you can use to develop Commerce website customizations. For example, with Commerce API methods, you can access customers and orders within your shopping sessions.	■ SuiteCommerce ■ All releases of SCA	■ Commerce API ■ Shopping Objects ■ JSON Object Fields
Item Search API	If you have configured item search settings for your site, you can use the Items Search API to build facet filtering and keyword search on item data. This API supports searching, sorting, faceting, and pagination of item data. Note that API response data can be affected by the search settings you have configured. For example, items returned by the API can be filtered by the facets you define in the search index of your search settings.  Note: SuiteCommerce MyAccount solutions do not use Item Search API.	■ SuiteCommerce ■ All releases of SCA	■ Item Search API ■ Search Settings Overview ■ Sample Item Search Query and Results

The following image illustrates each layer of functionality included with Commerce websites.



☀ Does not apply to SuiteCommerce MyAccount

Supported Browsers for Commerce Websites



Important: This topic only describes browser support for Commerce websites, which is different from the browser support in NetSuite. See the help topic [Supported Browsers for NetSuite](#) for details.

Commerce websites are supported in the following browsers and operating systems:

Browser	Platform
Google Chrome 95 and later	Windows 8.x
	Windows 7
	iOS 13 and later
	Android
	Mac OS 10.15 and later
Microsoft Edge (Chromium-based)	Windows 10
	Windows 8.x
	Windows 7
Mozilla Firefox 95 and later	Windows 10

Browser	Platform
	Windows 8.x
	Windows 7
	Windows Vista
	Mac OS 10.15 and later
Safari 13.1 and later	Mac OS 10.15 and later
	iOS 13 and later

Note the following browser limitations:

- **Access to PayPal Payment Systems** — PayPal requires Transport Security Layer (TLS) 1.2 for all HTTPS connections. If a site user attempts to access PayPal payment systems using a browser that does not have TLS 1.2 enabled, the requested PayPal page does not load. To fix this issue, users should use a browser that supports TLS 1.2 and has TLS 1.2 enabled in browser settings.
- **Internet Explorer 11 and Earlier Not Supported**
 - **SuiteCommerce** — As of the **2022.1** release and later, SuiteCommerce website implementations no longer support the use of Internet Explorer 11 (IE 11) and earlier.
 - **SuiteCommerce MyAccount** — As of the **2022.1** release and later, SuiteCommerce MyAccount website implementations no longer support the use of Internet IE 11 and earlier.
 - **SuiteCommerce Advanced (SCA)** — Previous versions of SCA continue to support IE 11, but IE 11 is not supported for SCA version 2022.1 and later.

Additional Resources

Additional resources are available to support your efforts to customize Commerce websites, including:

Resource	Description
Developer Portal	The SuiteCommerce Developers Portal provides supplemental resources you can explore after setting up the developer tools and mastering the basics of theme and extension development. Resources include sample customizations, new customization ideas, best practices, and design and coding tips to speed up your work.
Training	The following courses are available and should be beneficial to developers who are new to Commerce website configuration and development. Note that you may need to pay additional fees for training, depending on your agreement with NetSuite. ■ SuiteCommerce: Web Store Administration ■ SuiteCommerce: Theme and Extension Development

Commerce Migration to SuiteScript 2.0

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

Commerce website source code is undergoing an initiative to improve the developer experience when developing a SuiteCommerce Advanced site. These improvements will ultimately enable Commerce application developers to interact with core NetSuite records in a unified way, leveraging the benefits of SuiteScript 2.0.

This change is being introduced in phases for SuiteCommerce and SuiteCommerce Advanced. When completed, this approach will:

- Leverage the benefits of SuiteScript 2.0
- Provide developers a single, unified experience to interact with core NetSuite APIs
- Eliminate duplication of business logic within the Commerce API
- Eliminate the need to access the Commerce API

Migration Status

This section explains the current state of the migration to SuiteScript 2.0. Check this topic periodically for status updates and a list of modules that have been migrated.

Migration Updates (by release)

Release	Description of Changes	Modules Migrated to SS 2.0	New Modules
2020.1	■ Continued Module Migration to Unified Developer Experience ■ Continued implementation of My Account backend components (Service Controllers) ■ Updated some modules for SuiteScript 2.0 migration to TypeScript	StoreItem Transaction TransactionHistory	None
2019.2	■ Partial Module Migration to Unified Developer Experience ■ Initial implementation of My Account backend components (Service Controllers) ■ Updated some modules for SuiteScript 2.0 migration to TypeScript ■ Expanded SSP Application support for SuiteScript 2.0. This change impacts all Commerce web store SuiteApp installation and setup procedures. See the help topic Install Your Commerce Website Applications for details.	Case	Subscriptions (Limited Release)
2019.1	Added SSP Application support for SuiteScript 2.0 for implementations utilizing the SuiteTax feature. During this release of SuiteCommerce and SCA, this was required for anyone using the SuiteTax feature.	None	None

 **Note:** Modules that have been migrated to SuiteScript 2.0 are located in the Backend directory of the SCA source code: `<SC_xxx/Backend/SC/` (where xxx refers to the release). If your current SCA implementation includes customizations to these modules, you can review changes to the code base and adjust your customizations as needed to ensure continued functionality. However, if you are developing extensions, attempts to import or include migrated modules in your customization will produce runtime errors.

What is the Development Life Cycle?

📘 Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced | SuiteCommerce InStore

The following topics describe the stages of the life cycle when developing themes and extensions, as well as best practices around versioning, and delivery options for themes and extensions:

- [Theme Development Life Cycle](#)
- [Extension Development Life Cycle](#)
- [Maintaining Themes and Extensions](#)

Theme Development Life Cycle

When you're working with the theme developer tools, the stages of the development life cycle follow this basic flow:

Stage	Activity	Gulp Command to Use	Result
1	Fetch existing theme — You need to download files for the active theme to use as the baseline for your own custom theme. The SuiteCommerce Base Theme is a good starting point.	<code>gulp theme:fetch</code>	Theme developer tools create a Workspace directory and then download theme files into separate subdirectories.
2	Customize existing theme — You can customize the theme by changing templates, styles, or assets. Modify templates, styles, and other assets using your chosen editor in the local Workspace directory created by the theme developer tools. 📘 Note: SuiteScript, JavaScript, and configuration files are not allowed in themes.	—	You revise and store your customized files in the local Workspace directory and subdirectories.
3	Test locally — You should test your theme locally so you can identify any issues and correct them before deployment.	<code>gulp theme:local</code>	Theme developer tools compile all source files into a LocalDistribution/tmp directory. When you start a local server, the local Commerce app/website uses the Sass, template and other files from this directory so you can view and adjust your theme in one or more browsers. Changes you make to source files are immediately visible in your local browser.
4	Deploy theme — You need to deploy your theme to NetSuite so you can activate it for your Commerce website.	<code>gulp theme:deploy</code>	Theme developer tools validate your customizations, copy theme files into a local DeployDistribution directory, and

Stage	Activity	Gulp Command to Use	Result
			then upload these files to a location in your NetSuite File Cabinet.
5	Activate theme — Activating your theme applies it to a Commerce website domain. You can activate your theme using the SuiteCommerce Extension Manager in NetSuite.	—	Your customized theme is active and controls the design and layout of the domain you selected during activation.

When you're ready to get started with theme development, see [Next Steps for Theme Developer Tools](#).

Extension Development Life Cycle

When you create a new extension using the extension developer tools, the life cycle is somewhat different from updating an existing extension. In general, the stages of the extension development life cycle follow this basic flow:

Stage	Activity	Gulp Command to Use	Result
1	Create New Extension Create a baseline extension — Get started by creating the files needed for a basic Hello World extension.	Create New Extension gulp extension:create	Create New Extension Extension developer tools create a Workspace directory and a subdirectory that contains the files and initial module for your extension. This is where you develop your extension, using the starter files created by the developer tools as an example.
	Update Existing Extension Fetch existing extensions — You need to download files for the active extensions to use as the basis for your customizations.	Update Existing Extension gulp extension:fetch	Update Existing Extension Extension developer tools download: - Files for any custom extensions that are activated. - HTML and Sass files for the currently active theme. Do not modify these files. The Sass and HTML files downloaded using this command are for reference only.
2	Develop or revise your extension — Create or update files needed to implement custom features or functionality, including JavaScript, SuiteScript, JSON, HTML templates, Sass files, and more. Use your chosen editor in the local Workspace directory created by the extension developer tools.	—	Develop and store your files in the local Workspace directory and subdirectories.
3	Fetch active theme — You must download files for the active theme before you can deploy an extension or test it on a local server.	gulp extension:theme	Extension developer tools download HTML and Sass files for the active theme. Do not modify these files. The Sass and HTML files downloaded using this command are for reference only.

Stage	Activity	Gulp Command to Use	Result
	The extension developer tools require theme files to compile a local distribution for testing on a local server. The developer tools also use these theme files to run a test compilation before deploying.  Note: If you used the <code>gulp extension:fetch</code> command to update an existing extension in stage 1, you can skip this stage. The <code>fetch</code> command you issued in stage 1 downloads both active theme and extension files.		
4	Test locally — You should test your extension locally so you can identify any issues and correct them before deployment.  Note: If you are developing an extension that includes SuiteScript or JSON files, you must deploy your files to your NetSuite account and activate the extension for these changes to be accessible by the local server.	<code>gulp extension:local</code>	Extension developer tools compile all source files into a <code>LocalDistribution/tmp</code> directory. When you start a local server, the local Commerce app/website uses the Sass, template, and other files from this directory so you can view and adjust your extension in one or more browsers. Changes you make to source files are immediately visible in your local browser.
5	Deploy your extension — You need to deploy your extension to NetSuite so you can activate it for your Commerce website.	<code>gulp extension:deploy</code>	Extension developer tools validate your code, copy your extension files into a local <code>DeployDistribution</code> directory, and update the <code>manifest.json</code> file. The developer tools then upload these files to your NetSuite account, making them available for activation.
6	Activate extension — Activating your extension applies it to a Commerce website domain. You can activate your extension using the SuiteCommerce Extension Manager in NetSuite.	—	Your custom extension is active and its functionality is available in the domain you selected during activation.

When you're ready to get started with extension development, see [Next Steps for Extension Developer Tools](#).

Maintaining Themes and Extensions

When developing and maintaining themes and extensions, keep the following important resources and strategies in mind.

To learn more about these and other resources, see [More Information for Each Tool](#).

Resource	Description
Manifests	The theme and extension developer tools generate a manifest.json file for each theme and extension you develop. Each manifest file includes the information required to compile resources for that particular theme or extension. Manifest files also specify the name, description, version, and other data that is displayed for each theme and extension in the SuiteCommerce Extension Manager in NetSuite. Because manifest files are generated by the theme and extension developer tools, you generally do not need to edit them. One exception is that you may need to add Sass entry points for newly introduced Sass files. And, you can also edit the manifest if you want to omit some local assets from a test or deployment. If you do manually edit a manifest file, be aware that the gulp commands to test and deploy update the extension's manifest.json file and overwrite any manual changes you made. You can preserve your manifest edits by using the <code>--preserve-manifest</code> flag.
Versions	In accordance with standard software practices, you should implement version control for your themes and extensions. When you deploy themes and extensions to NetSuite, you are prompted to provide a version number. This version number is stored in the manifest and when you deploy your theme or extension, NetSuite checks the version to determine if that version already exists. If so, NetSuite overwrites any files that already exist for that version. If not, NetSuite generates new records for your theme or extension. You should implement version control to prevent stable versions of your themes and extensions from being overwritten.
Delivery Options	In addition to deploying to your NetSuite account and activating themes and extensions for specific domains of your Commerce website, you can also bundle your themes and extensions as SuiteApps to make them available for distribution to other NetSuite accounts.

Choose a Tool

📌 Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced | SuiteCommerce InStore

Review the following topics to choose the right tool for the tasks that you want to complete, find the supporting information you need to get started, and also find supporting reference information to tie it all together:

- [Tools Overview](#)
- [Choose a Tool to Suit Your Task](#)
- [More Information for Each Tool](#)

Tools Overview

Developers working with SuiteCommerce and SuiteCommerce Advanced (SCA) should use the theme and extension developer tools to easily version, distribute, and activate customizations across different sites and different versions of sites. Remember that what you develop depends on the type of customization you want to make:

- Extensions — Add new functionality to a Commerce website
- Themes — Control how a site looks and feels

You should use the SCA developer tools only if you are customizing an older version of SCA, or if your SCA implementation requires access to objects not available using the Extensibility API.

Before choosing a tool and starting with development, make sure you understand the essential information in the preceding topics, including:

- [What Can I Customize?](#)
- [What Resources Do I Need?](#)
- [What is the Development Life Cycle?](#)

Choose a Tool to Suit Your Task

The tool you use and whether you develop a theme or extension depends on what you want to customize.

Customization Option	Method	Tool	Available With	Get Started
Website design and appearance	Develop themes	Theme developer tools	SuiteCommerce, SCA, SCMA	Next Steps for Theme Developer Tools
Website functionality	Develop extensions	Extension developer tools	SuiteCommerce, SCA, SCMA, SuiteCommerce InStore (SCIS)	Next Steps for Extension Developer Tools
Website functionality by customizing objects	Extend or override SCA source code	SCA developer tools	SCA	Next Steps for SuiteCommerce

Customization Option	Method	Tool	Available With	Get Started
not available using the Extensibility API		 Note: The SCA developer tools are the only tools available if you are customizing a site that implements the Kilimanjaro release of SCA or earlier.		Advanced Developer Tools

To find detailed reference information for each tool, see [More Information for Each Tool](#).

 **Note:** Always use the theme and extension developer tools if you can, regardless of whether you are working with SuiteCommerce or SCA. Use the SCA developer tools only when absolutely necessary for SCA implementations. Using the theme and extension developer tools makes development and management of your customizations easier and helps ensure compatibility with future releases of SuiteCommerce and SCA.

Next Steps for Theme Developer Tools

To start developing themes, complete the following tasks:

Step/Task	For detailed instructions, see
Set up the developer environment by installing Node.js and Gulp.js	■ Install Node.js ■ Install Gulp.js
Download and install theme developer tools	Set Up Theme Developer Tools
Download files to use as the basis for your own custom theme	■ Anatomy of the Base Theme (recommended if you're new to website development and learning best practices) ■ Fetch Active Theme Files (if you want to customize the currently active theme)
Understand best practices for theme development	Best Practices for Creating Themes
Develop your theme	■ Customize Pre-Existing Theme Files ■ Add a New File to a Theme ■ Theme Tutorials
Edit the theme manifest (only necessary if you need to introduce new HTML, Sass, or other assets)	Theme Manifest
Test your theme locally	Test a Theme on a Local Server
Deploy your theme to NetSuite	Deploy a Theme to NetSuite
Activate your theme	Manage Themes and Extensions
If you want to do more with themes	■ Override Active Extension Files ■ Enable Customization from Site Management Tools

Step/Task	For detailed instructions, see
	■ Customize Site Search Elements ■ Bundle Themes and Extensions as SuiteApps

Next Steps for Extension Developer Tools

To start developing extensions with the extension developer tools, complete the following tasks:

Step/Task	For detailed instructions, see
Set up the developer environment by installing Node.js and Gulp.js	■ Install Node.js ■ Install Gulp.js
Download and install extension developer tools	Set Up Extension Developer Tools
Create or download files to use as the basis for your extension	■ Create a Baseline Extension (recommended if you're new to Commerce website development and want to learn best practices) ■ Fetch Active Theme and Extension Files (if you want to customize currently active extensions)
Add modules to your extension	Create Additional Modules for an Extension (recommended if you're new to Commerce website development and learning best practices)
Develop your extension using available APIs	■ Overview ■ SuiteScript API & Commerce Web Stores ■ Commerce API ■ Item Search API
Edit the extension manifest (only necessary if you need to add new Sass or other assets)	Edit the Extension Manifest
If you haven't already done so, fetch the active theme (theme files are required to test locally and to deploy)	Fetch Active Theme and Extension Files
Test your extension locally	Test an Extension on a Local Server
Deploy your extension to NetSuite	Deploy an Extension to NetSuite
Activate your extension	Manage Themes and Extensions
If you want to do more with extensions	■ Create Custom Content Types for an Extension (to integrate with Site Management Tools) ■ Create Page Types (to integrate with Site Management Tools) ■ Bundle Themes and Extensions as SuiteApps

Next Steps for SuiteCommerce Advanced Developer Tools

To start developing with the SuiteCommerce Advanced (SCA) developer tools, complete the following tasks.

Note: Use the SCA developer tools only if you are customizing a site that implements the Kilimanjaro release of SCA or earlier or if you need to access objects not available using the Extensibility API.

Step/Task	For detailed instructions, see
Set up the developer environment by installing Node.js and Gulp.js	- Install Node.js - Install Gulp.js
Download and install SCA developer tools	- Download SCA Source Files - Install Additional Files in the Source Directory
Understand best practices for customizing SCA	Best Practices for Customizing SCA
Customize by extending or overriding SCA source files	Example SCA Customizations
Understand how SCA source files are organized and compiled	SCA Build Process
Test your SCA customization locally	Test SCA Customizations on a Local Server
Deploy local source files to an SSP application in NetSuite and then deploy the SSP application to your site	Deploy SCA Customizations to NetSuite

More Information for Each Tool

The following topics provide important information for each tool.

Topic	Description
Theme Developer Tools	
Theme Developer Gulp Commands	Describes all gulp commands you can use with the theme developer tools.
Anatomy of the Base Theme	Describes the files and folders in a top-level theme directory and the purpose of each.
Theme Manifest	Describes the mainfiest.json file, which includes all the information required to compile resources for each theme.
Extension Developer Tools	
Extension Developer Gulp Commands	Describes all gulp commands you can use with the extension developer tools.
Anatomy of an Extension	Describes the files and folders in a top-level extensions directory and the purpose of each.
Edit the Extension Manifest	Describes the mainfiest.json file, which includes all the information required to compile resources for each extension.
SCA Developer Tools	
Gulp Command Reference for SCA Developer Tools	Describes all gulp commands you can use with the SCA developer tools.

Overview

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Before you start to customize your SuiteCommerce, SuiteCommerce Advanced (SCA), or SuiteCommerce MyAccount (SCMA) implementation using the theme and extension developer tools, you need to set up your developer environment. The same is true if you plan to develop extensions for SuiteCommerce InStore (SCIS) using the extension developer tools or if you need to customize a Kilimanjaro or earlier release of SCA using the SCA developer tools.

To set up your developer environment, you need to install the following JavaScript utilities:

- [Install Node.js](#)
- [Install Gulp.js](#)

After installing the required JavaScript utilities, make sure you understand the roles and permissions required to use the developer tools as well as the gulp commands that are available for use with each tool. For more information, see:

- [Developer Tool Roles and Permissions](#)
- [Gulp Commands](#)

After installing the required JavaScript utilities and learning more about roles, permissions, and gulp commands, continue to set up your tools and develop your customizations as described in [Next Steps for Each Tool](#).

Install JavaScript Utilities

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

All Commerce developer tools, including theme and extension developer tools and SuiteCommerce Advanced (SCA) developer tools, require Node.js and Gulp.js. To install the required JavaScript utilities, follow these instructions:

- [Install Node.js](#)
- [Install Gulp.js](#)

Install Node.js

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Node.js is the platform required for all Gulp.js tasks required to develop extensions for any Commerce website, including SuiteCommerce InStore (SCIS). Node.js is available at:

<https://nodejs.org/en/download/>

Note: This topic discusses Node.js only as the platform required to develop extensions on Commerce websites. For instructions on developing SuiteCloud projects with Node.js, please see the help topic [SuiteCloud CLI for Node.js](#).

Using the following table for guidance, install the version of Node.js that is supported by your implementation. Use the most current minor release (designated by x in the table) for the version of Node.js you are using.

Commerce Web Store Implementation	Supported Node.js Versions
SuiteCommerce	20.10.0
SuiteCommerce MyAccount	20.10.0
SuiteCommerce Advanced — 2025.2	20.10.0
SuiteCommerce Advanced — 2022.2.1	14.19.0
SuiteCommerce Advanced — 2022.2	12.21.x
SuiteCommerce Advanced — 2022.1	12.21.x
SuiteCommerce Advanced — 2021.2	12.21.x
SuiteCommerce Advanced — 2021.1	12.21.x
SuiteCommerce Advanced — 2020.2	12.16.x
SuiteCommerce Advanced — 2020.1	12.14.x
SuiteCommerce Advanced — 2019.2	10.16.x

Commerce Web Store Implementation	Supported Node.js Versions
SuiteCommerce Advanced — 2019.1	10.15.x
SuiteCommerce Advanced — 2018.2	8.11.4
SuiteCommerce Advanced — Aconcagua	8.9.x LTS
SuiteCommerce Advanced — Kilimanjaro	6.11.x
SuiteCommerce Advanced — Elbrus	4.x.x LTS and Later
SuiteCommerce Advanced — Vinson	4.x.x LTS
SuiteCommerce Advanced — Mont Blanc	4.4.x and 0.12.x
SuiteCommerce Advanced — Denali	0.12.x

When installing Node.js, ensure that you install all Node.js components. This includes the Node Package Manager (NPM), which is used to install Gulp.js and other files required by the developer tools and the build process.

After running the installer, you should see Node.js in the list of available programs on your system. The `npm` command should also be added to the path on your system.

Note: Ensure that you use the correct installer for your platform. You may have to restart your machine for the Node.js commands to be recognized in the path variable on your system.

To verify that Node.js is installed correctly:

1. Depending on your platform open a command line or terminal window.
2. Enter the following command:
`node -v`

If everything is installed correctly, this command outputs the currently installed version of Node.js. There should be no errors or warnings.

After successfully installing Node.js, you are ready to [Install Gulp.js](#).

Install Gulp.js

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

After successfully installing Node.js, the next step in setting up your developer environment is to install Gulp.js. Gulp.js is a third-party, open-source tool that automates many of the tasks related to creating web-based applications. This is required for all Commerce web store implementations.

To Install Gulp.js:

1. Depending on your platform, open a command line or terminal window.
2. Enter the following command:
`npm install --global gulp`

 **Note:** Install Gulp.js using the `--global` flag as shown earlier. On Linux and MacOS, you must run this command using `sudo`. This is required to ensure that Gulp.js is installed correctly.

3. Verify that Gulp.js was installed correctly by entering the following command:

```
gulp -v
```

If installed correctly, this command outputs the currently installed version of Gulp.js. Ensure that this version is 3.9.1 or higher. There should be no errors or warnings.

Permissions

To use Gulp.js to deploy source files to NetSuite, you must use a role with the appropriate permissions. See [Developer Tool Roles and Permissions](#) for more information.

After successfully installing Gulp.js, you are ready to set up one or more of the following:

- [Set Up Theme Developer Tools](#)
- [Set Up Extension Developer Tools](#)
- [Set Up SCA Developer Tools](#)

Developer Tool Roles and Permissions

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Your role, and the permissions associated with it, are important when using the Commerce developer tools. Consider the following requirements and guidelines:

- You must have a role with the appropriate permissions to fetch files from and deploy files to NetSuite using the Commerce developer tools.
- Two-factor authentication (2FA) is required on all NetSuite accounts using NetSuite 2018.2 and later. To support NetSuite 2FA, the Commerce developer tools require token-based authentication for fetch and deploy operations as described in [Token Based Authentication](#).
- To generate a token for authentication, use a role that provides the permissions needed to fetch from and deploy to NetSuite.
- By default, the Administrator and SCDeployer roles provide these permissions. If necessary, NetSuite administrators can create a custom role that includes these permissions.

Note: If you experience issues using a role like SCDeployer to fetch from and deploy to NetSuite, ensure the role has the same permission settings and is added to the same script deployment records as a custom role.

See the following topics for more information.

- [Create a Custom Role with Fetch and Deploy Permissions](#)
- [Assign the Administrator or a Custom Role to an Employee Record](#)

Create a Custom Role with Fetch and Deploy Permissions

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

A NetSuite administrator can create a custom role that includes the permissions needed to fetch from and deploy to NetSuite by:

- Creating the new role in NetSuite
- Editing the script deployment records to include the new role

See the following procedures for detailed steps.

To create a custom role

1. In NetSuite, go to Setup > Users/Roles > Manage Roles > New.
2. Fill in the Role record as described in the [Customizing and Creating Roles](#) topic.
On the **Permission** tab, add the following permissions:

Subtab	Role	Level
Lists	Documents and Files	Full

Subtab	Role	Level
Setup	Custom Record Entries	Full
	Website (External) Publisher	Full
	Allow JS/HTML Uploads	Full
	Custom Lists	Full
	Set Up Web Site	Full
	SuiteScript	View
	SOAP Web Services, REST Web Services	Full
	Access Token Management	Full
	Log in using Access Tokens	Full
	User Access Tokens	Full

3. Save the Role record.
4. Assign the custom role to the Employee record for each user who requires developer access to NetSuite. For more information, see [Assign the Administrator or a Custom Role to an Employee Record](#).
5. To ensure the new custom role can fetch and deploy from NetSuite, you must follow the instructions in the following procedure, [To edit Script Deployment records](#).

To edit Script Deployment records

1. In NetSuite, go to Customization > Scripting > Script Deployments.
2. Set the following filters:
 - **Type:** RESTlet
 - **API Version:** All
3. Add the custom role to the **Roles** list on the **Audience** tab for each of the following Script Deployment records, depending on your implementation:

SuiteCommerce Extension Manager, SuiteCommerce Advanced (Aconcagua release and Later):

Edit each of the following Script Deployment records.

- customdeploy_ext_mech_deployer_rest
- customdeploy_ns_sc_ext_mech_restlets
- customdeploy_ext_mech_web_service_rest
- customdeploy_ns_sca_deployer_20xx_x_x (Where 20xx_x_x equals the version of SuiteCommerce Advanced you are implementing. For example: customdeploy_ns_sca_deployer_2020_1_0)

SuiteCommerce Advanced and Site Builder Extensions (Kilimanjaro release and Earlier):

Edit the script associated with your implementation.

- customdeploy_ns_sca_deployer_kilimanjaro
- customdeploy_ns_sca_deployer_elbrus
- customdeploy_ns_sca_deployer_mont_blanc
- customdeploy_ns_sca_deployer_vinson
- customdeploy_ns_sca_deployer_denali
- customdeploy_ns_sbe_deployer_elbrus
- customdeploy_ns_sbe_deployer_kilimanjaro
- customdeploy_ns_sbe_deployer_mont_blanc
- customdeploy_ns_sbe_deployer_vinson
- customdeploy_ns_sbep_deployer_denali
- customdeploy_ns_sbep_deployer_mont_blanc
- customdeploy_ns_sbep_deployer_vinson
- customdeploy_ns_sbep_deployer_elbrus
- customdeploy_ns_sbep_deployer_kilimanjaro

4. Save the Script Deployment record.

Assign the Administrator or a Custom Role to an Employee Record

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

You must assign the Administrator or a custom role with the correct permissions to the Employee record for each user who requires developer access to NetSuite.

You can assign the necessary role in **Role** field of the **Employee** record (**Access** tab, **Role** subtab). For detailed steps, see the help topic [Assigning Roles to an Employee](#).

Gulp Commands

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

With Gulp.js installed in your developer environment, you can use the commands it provides to create and manage the resources needed for your customizations. The following topics describe the Gulp.js commands available when using the Commerce developer tools:

- [Gulp Command Reference for Theme and Extension Developer Tools](#)
- [Gulp Command Reference for SCA Developer Tools](#)

For more information about how the commands described in these command references play a part in development, see [What is the Development Life Cycle?](#)

Gulp Command Reference for Theme and Extension Developer Tools

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

The following tables list the Gulp.js commands required when using the theme or extension developer tools with your Commerce website:

- [Theme Developer Gulp Commands](#)
- [Extension Developer Gulp Commands](#)

i Note: For a list of commands used with the SuiteCommerce Advanced (SCA) developer tools, see [Gulp Command Reference for SCA Developer Tools](#).

Theme Developer Gulp Commands

Command	Description
gulp theme:fetch	Downloads the active theme and extension files (Sass, HTML, and other assets) for the specified domain. You must have already activated a theme using the Manage Extensions wizard in NetSuite for this command to run. This command places these files in the top-level directory's Theme directory. If this is the first time running this command, the development tools create subdirectories for these files.
gulp theme:fetch --account	Use the --account parameter only for release 2021.2.1 and later accounts that have account-specific domains. Specify your account number with the gulp command to enable access to the NetSuite account. For example, if your account number is 123456, you can enter the following command to successfully authenticate and fetch from your NetSuite account: <pre>gulp theme:fetch --account 123456</pre> For sandbox accounts, specify your account number as shown in this example: <pre>gulp theme:fetch --account 123456-sb1</pre> For more information, see the help topic URLs for Account-Specific Domains.

Command	Description
<code>gulp theme:fetch --to</code>	Fetches a theme from NetSuite as usual, but this command prompts for login credentials.
<code>gulp theme:local</code>	Compiles all Sass and HTML template files for a theme into a functional application. This command also updates the theme manifest.json. After compilation, this command starts a local server. This server watches for changes to Sass and HTML template files. After the server starts, any changes you make to your theme files or extension overrides are automatically recompiled and visible in the browser.
<code>gulp theme:local --preserve-manifest</code>	Performs the same action as <code>gulp theme:local</code> , but it does not update the manifest. Use this command to test your theme locally if you have made any manual changes to the manifest.json file for the theme.
<code>gulp theme:deploy</code>	Compiles Sass, HTML, and asset files into a DeployDistribution folder. This command also updates the theme manifest.json. If you are deploying changes to a published theme, the development tools prompt you for a Vendor Name, Theme Name, Theme Version, Description, and Application when you initially run this command. This command forces you to name a new theme. If you are deploying customizations to a custom theme, this command does not prompt you for this information and gives you the option to create a new theme or update the existing theme. The development tools then create a folder for the theme in the NetSuite File Cabinet and deploy the customized theme's code. In addition to compiling the application, this command creates the .nsdeploy file, if it does not already exist.
<code>gulp theme:deploy --account</code>	Use the <code>--account</code> parameter only for release 2021.2.1 and later accounts that have account-specific domains. Specify your account number with the gulp command to enable access to the NetSuite account. For example, if your account number is 123456, you can enter the following command to successfully authenticate and deploy to your NetSuite account: <pre>gulp theme:deploy --account 123456</pre> For sandbox accounts, specify your account number as shown in this example: <pre>gulp theme:deploy --account 123456-sb1</pre> For more information, see the help topic URLs for Account-Specific Domains.
<code>gulp theme:deploy --preserve-manifest</code>	Performs the same action as <code>gulp theme:deploy</code> , but it does not update the manifest. Use this command to deploy your theme if you have made any manual changes to the manifest.json file for the theme.
<code>gulp theme:deploy --advanced</code>	Deploys as an update of the current custom theme, but resets the prompts regarding the Vendor Name, Theme Name, Theme Version, Description, and Application. This command also rewrites this information in the theme's manifest.json file. This command only takes effect on custom themes.
<code>gulp theme:deploy --create</code>	Creates a new theme instead of updating the existing theme.
<code>gulp theme:deploy --skip-compilation</code>	Deploys the current contents of the DeployDistribution folder without compiling the application. Although the developer tools do not deploy a compiled set of

Command	Description
	files, the default action is to compile files as a final test of your code. Using this command skips this check.
gulp theme:deploy --source templates	Deploys only the HTML template files.
gulp theme:deploy --source sass	Deploys only Sass files.
gulp theme:deploy --source assets	Deploys only theme asset files.
gulp theme:deploy --source skins	Deploys only theme skin preset files.
gulp extension:deploy --source <multiple>	Deploys more than one type of source code when you separate multiple source deployments by commas. For example, <code>gulp extension:deploy --source templates,sass</code>
gulp theme:deploy --to	Deploys to NetSuite as usual, but resets the login credentials, rewriting the .nsdeploy file.
gulp theme:deploy --reactivate	Deploys and reactivates the themes and extensions on your website.
gulp theme:update-manifest	Updates the theme's manifest.json file without requiring a deployment.
gulp validate	Validates the theme's manifest.json file and confirms that the file does not list any files that do not exist in the theme folder.
gulp clear	Removes the DeployDistribution and LocalDistribution directories and the .nsdeploy file.
gulp reactivate	Triggers a reactivation process of all the extensions and themes installed on your website.
gulp reactivate --async	Triggers a reactivation process of all the extensions and themes installed on your website asynchronously.
gulp reactivate --update	Triggers a reactivation process of all the extensions and themes installed on your website, updating each to the latest available version.
gulp reactivate --token	Triggers a reactivation process of all the extensions and themes installed on your website, using a token for authentication. The format is: <key>: <secret>. This parameter requires use of the --account parameter as well.
gulp reactivate --account	Specifies the target account number when triggering the reactivation process. This parameter is required when using the --token parameter.
gulp reactivate --to	Triggers a reactivation process of all the extensions and themes installed on your website as usual, but this command prompts for login credentials.
gulp reactivate --website	Specifies the target website when triggering the reactivation process.
gulp reactivate --domain	Specifies the target domain when triggering the reactivation process.
gulp reactivate --subsidiary	Specifies the target subsidiary when triggering the reactivation process. This parameter only applies to SuiteCommerce InStore.
gulp reactivate --location	Specifies the target location when triggering the reactivation process. This parameter only applies to SuiteCommerce InStore.

Command	Description
gulp reactivate --m	Specifies the molecule to be used when triggering the reactivation process. For example, <code>gulp reactivate --m f</code> uses <code>system.f.netsuite.com</code> .
gulp	Displays a list of all gulp commands.

Extension Developer Gulp Commands

Command	Description
gulp extension:create	Creates an example extension (with one example module) to use as a baseline for development. This command creates all necessary folders to store and maintain your customizations within your top-level extension development directory.
gulp extension:create-module	Creates an additional module within your extension. This command prompts you for information about the module you want to create and the extension within which you want to create it.
gulp extension:fetch	Downloads the active theme and compiles all theme resources (Sass, HTML, and other assets). This command places theme files in the top-level extension development directory's Extras folder. If this is the first time running this command, the development tools create subdirectories for these files.  Note: Any downloaded theme files are only provided for testing your extensions locally. You do not customize any theme files in your top-level extension development directory. If you choose to continue development on a previously deployed, custom extension, this command also downloads these files, placing them in your <code>Workspace/<EXTENSION_FOLDER></code>. The extensions must be activated using the Extension Manager.  Note: This command does not download published extensions. You cannot customize published content.
gulp extension:fetch --account	Use the <code>--account</code> parameter only for release 2021.2.1 and later accounts that have account-specific domains. Specify your account number with the gulp command to enable access to the NetSuite account. For example, if your account number is 123456, you can enter the following command to successfully authenticate and fetch from your NetSuite account: <pre>gulp extension:fetch --account 123456</pre> For sandbox accounts, specify your account number as shown in this example: <pre>gulp extension:fetch --account 123456-sb1</pre> For more information, see the help topic URLs for Account-Specific Domains.
gulp extension:fetch <arg>	Downloads the files of a specific extension, where <code><arg></code> is the name of the extension. This can be helpful if you want to use a previously deployed extension as a baseline for a new one. You can fetch multiple extensions, separated by a comma. For example: <code>gulp extension:fetch --fetch Badges,CartExtension</code>.

Command	Description
	 Note: Extensions must be active to download code or to view them on a local server.
gulp extension:local	Compiles your custom extension files into a functional application and places them in a LocalDistribution folder. After compilation, this command starts a local server. This server watches for changes to any extension files. After the server starts, any changes you make to your extension files are automatically recompiled and visible in the browser.
gulp extension:local --preserve-manifest	Performs the same action as <code>gulp extension:local</code>, but it does not update the manifest. Use this command to test your extension locally if you have made any manual changes to the manifest.json file for the extension.
gulp extension:deploy	Compiles your custom extension files into a functional application and places them into a DeployDistribution folder. If you have more than one extension in your top-level development directory, this command prompts you to declare which extension to deploy. If you have never deployed the extension, this command prompts you for information about the extension. The development tools then create a folder for the extension in the NetSuite File Cabinet and deploy the customized extension code. In addition to compiling the application, this command creates the .nsdeploy file, if it does not already exist.
gulp extension:deploy --account	Use the <code>--account</code> parameter only for release 2021.2.1 and later accounts that have account-specific domains. Specify your account number with the gulp command to enable access to the NetSuite account. For example, if your account number is 123456, you can enter the following command to successfully authenticate and deploy to your NetSuite account: <pre>gulp extension:deploy --account 123456</pre> For sandbox accounts, specify your account number as shown in this example: <pre>gulp extension:deploy --account 123456-sb1</pre> For more information, see the help topic URLs for Account-Specific Domains.
gulp extension:deploy --preserve-manifest	Performs the same action as <code>gulp extension:deploy</code>, but it does not update the manifest. Use this command to deploy your extension if you have made any manual changes to the manifest.json file for the extension.
gulp extension:deploy --advanced	Deploys an update of the extension, but resets the prompts regarding the Vendor Name, Extension Name, Version, Description, Application, etc. This command also rewrites this information in the extension manifest.json file.
gulp extension:deploy --skip-compilation	Deploys the current contents of the DeployDistribution folder without compiling the application. Although the developer tools do not deploy a compiled set of files, the default action is to compile files as a final test of your code. Using this command skips this check.
gulp extension:deploy --source configuration	Compiles and deploys only configuration JSON files.
gulp extension:deploy --source javascript	Compiles and deploys only JavaScript files.

Command	Description
gulp extension:deploy --source ssp-libraries	Compiles and deploys only ssp-libraries.
gulp extension:deploy --source services	Compiles and deploys only services.
gulp extension:deploy --source sass	Compiles and deploys only Sass files.
gulp extension:deploy --source templates	Compiles and deploys only HTML files.
gulp extension:deploy --source assets	Compiles and deploys only assets.
gulp extension:deploy --source <multiple>	Deploys more than one type of source code when you separate multiple source deployments by commas. For example, <code>gulp extension:deploy --source templates,sass</code>
gulp extension:deploy --reactivate	Deploys and reactivates the extensions and themes on your website.
gulp extension:update-manifest	Updates the extension's manifest.json file without requiring a deployment.
gulp validate	Validates the theme's manifest.json file and confirms that the file does not list any files that do not exist in the theme folder.
gulp clear	Removes the DeployDistribution and LocalDistribution directories and the .nsdeploy file.
gulp reactivate	Triggers a reactivation process of all the extensions and themes installed on your website.
gulp reactivate --async	Triggers a reactivation process of all the extensions and themes installed on your website asynchronously.
gulp reactivate --update	Triggers a reactivation process of all the extensions and themes installed on your website, updating each to the latest available version.
gulp reactivate --token	Triggers a reactivation process of all the extensions and themes installed on your website, using a token for authentication. The format is: <key>:<secret>. This parameter requires use of the --account parameter as well.
gulp reactivate --account	Specifies the target account number when triggering the reactivation process. This parameter is required when using the --token parameter.
gulp reactivate --to	Triggers a reactivation process of all the extensions and themes installed on your website as usual, but this command prompts for login credentials.
gulp reactivate --website	Specifies the target website when triggering the reactivation process.
gulp reactivate --domain	Specifies the target domain when triggering the reactivation process.
gulp reactivate --subsidiary	Specifies the target subsidiary when triggering the reactivation process. This parameter only applies to SuiteCommerce InStore.
gulp reactivate --location	Specifies the target location when triggering the reactivation process. This parameter only applies to SuiteCommerce InStore.
gulp reactivate --m	Specifies the molecule to be used when triggering the reactivation process.

Command	Description
	For example, <code>gulp reactivate --m f</code> uses <code>system.f.netsuite.com</code> .
<code>gulp</code>	Displays a list of all gulp commands.

Gulp Command Reference for SCA Developer Tools

① Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

You must run gulp commands at the level of the folder containing the `package.json` file.

The following table lists the most commonly used gulp commands for the SCA developer tools:

Command	Description
<code>gulp</code>	Creates the local distribution directory if it does not exist. This command also compiles the source modules into a deployable application.
<code>gulp deploy</code>	Compiles the application and deploys it to NetSuite. As part of the compilation process, <code>gulp deploy</code> minimizes the application by removing all white space. This command also creates one or more deploy distribution directories, depending on your implementation, if they do not exist. See Contents of the Deploy and Local Distribution Directories for information about the output of this command. In addition to compiling the application, this command creates the <code>.nsdeploy</code> file if it does not exist. If you are using the 2019.2 release of SCA or later, running <code>gulp deploy</code> also compiles all project TypeScript files to JavaScript. TypeScript files have a <code>.ts</code> filename extension.
<code>gulp deploy --account</code>	Use the <code>--account</code> parameter only for release 2021.2.1 and later accounts that have account-specific domains. Specify your account number with the <code>gulp</code> command to enable access to the NetSuite account. For example, if your account number is 123456, you can enter the following command to successfully authenticate and deploy to your NetSuite account: <pre>gulp deploy --account 123456</pre> For sandbox accounts, specify your account number as shown in this example: <pre>gulp deploy --account 123456-sb1</pre> For more information, see the help topic URLs for Account-Specific Domains.
<code>gulp deploy --no-backup</code>	Compiles and deploys the application, but does not upload a backup of the source files.
<code>gulp deploy --nougify</code>	Compiles and deploys the application like <code>gulp deploy</code> , but does not compress the source files. This command is useful for debugging because it produces output files that are easier to read.
<code>gulp deploy --source <source_type></code>	Compiles and deploys only the type of source files specified. Possible options are: ■ javascript ■ ssp-libraries ■ ssp-files ■ services

Command	Description
gulp deploy --to	Enables you to deploy to a different NetSuite account by overriding the settings defined in the .nsdeploy file.
gulp jshint	Runs the JSHint utility on the SuiteCommerce Advanced source files. This utility verifies and validates JavaScript files to help detect possible errors.
gulp local	Compiles the Sass and template files into a functional application, but does not compile the JavaScript files of the application. After compilation, this command starts a local server. This server watches for changes to the SuiteCommerce Advanced source files. After the server starts, any changes you make to a JavaScript file are automatically recompiled and visible in the browser. The server also watches for changes to the Sass and template files. If you are using the 2019.2 release of SCA or later, running <code>gulp local</code> also compiles all project TypeScript files to JavaScript. TypeScript files have a .ts filename extension. See the help topic Test SCA Customizations on a Local Server for more information. This command outputs files to the local distribution directory. See Contents of the Deploy and Local Distribution Directories for information about the output of this command.
gulp clean	Removes the deploy and local distribution directories and the .nsdeploy file.
gulp configuration	Compiles the configuration properties of each source module to generate the configurationManifest.json file in your local distribution folder. Configuration properties for source modules are declared using the configuration property in a module's ns.package.json file.
gulp copy	Copies any files declared using the copy property in a source module's ns.package.json file. For example, if a source module requires specific static files be available in the NetSuite File Cabinet, you can use the copy property in the ns.package.json file to declare the folder that contains those files. This ensures the contents of the declared folder are copied to the output with the internal folder structure maintained.
gulp javascript	Compiles the project's JavaScript output. The JavaScript files compiled are those specified using the javascript property in each source module's ns.package.json file. This command compiles the JavaScript output so that it is suitable for production.
gulp javascript --nougify	Compiles the project's JavaScript output, like <code>gulp javascript</code> , but does not compress the source files. This command is useful for debugging because it produces output files that are easier to read.
gulp ssp-files	Compiles and copies the .ssp files declared using the ssp-files property in each source module's ns.package.json file. (Handlebar syntax can be used in the .ssp files.)
gulp ssp-libraries	Compiles the JavaScript files declared using the ssp-libraries property in each backend module's ns.package.json file. The ssp-libraries section of the distro.json file includes a reference to a list of all backend modules.

Next Steps for Each Tool

① Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Now that your developer environment has the required JavaScript utilities installed, you are ready to download and install the Commerce developer tools and then start developing your customizations as described in the following topics:

- [Next Steps for Theme Developer Tools](#)
- [Next Steps for Extension Developer Tools](#)
- [Next Steps for SuiteCommerce Advanced Developer Tools](#)

If you are unsure which tool best suits your needs, see [Choose a Tool](#) to learn more.

Next Steps for Theme Developer Tools

① Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

To start developing themes, complete the following tasks:

Step/Task	For detailed instructions, see
Download and install theme developer tools	Set Up Theme Developer Tools
Download files to use as the basis for your own custom theme	■ Anatomy of the Base Theme (recommended if you're new to website development and learning best practices) ■ Fetch Active Theme Files (if you want to customize the currently active theme)
Understand best practices for theme development	Best Practices for Creating Themes
Develop your theme	■ Customize Pre-Existing Theme Files ■ Add a New File to a Theme ■ Theme Tutorials
Edit the theme manifest (only necessary if you need to introduce new HTML, Sass, or other assets)	Theme Manifest
Test your theme locally	Test a Theme on a Local Server
Deploy your theme to NetSuite	Deploy a Theme to NetSuite
Activate your theme	Manage Themes and Extensions
If you want to do more with themes	■ Override Active Extension Files ■ Enable Customization from Site Management Tools (to integrate with Site Management Tools) ■ Customize Site Search Elements ■ Bundle Process for SuiteCommerce

Next Steps for Extension Developer Tools

📌 Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

To start developing extensions with the extension developer tools, complete the following tasks:

Step/Task	For detailed instructions, see
Download and install extension developer tools	Set Up Extension Developer Tools
Create or download files to use as the basis for your extension	■ Create a Baseline Extension (recommended if you're new to Commerce website development and want to learn best practices) ■ Fetch Active Theme and Extension Files (if you want to customize currently active extensions)
Add modules to your extension	Create Additional Modules for an Extension (recommended if you're new to Commerce website development and learning best practices)
Develop your extension using available APIs	■ Overview ■ SuiteScript API & Commerce Web Stores ■ Commerce API ■ Item Search API
Edit the extension manifest (only necessary if you need to add new Sass or other assets)	Edit the Extension Manifest
If you haven't already done so, fetch the active theme (theme files are required to test locally and to deploy)	Fetch Active Theme and Extension Files
Test your extension locally	Test an Extension on a Local Server
Deploy your extension to NetSuite	Deploy an Extension to NetSuite
Activate your extension	Manage Themes and Extensions
If you want to do more with extensions	■ Create a Custom Content Type (to integrate with Site Management Tools) ■ Create Page Types (to integrate with Site Management Tools) ■ Bundle Process for SuiteCommerce

Next Steps for SuiteCommerce Advanced Developer Tools

📌 Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

To start developing with the SuiteCommerce Advanced (SCA) developer tools, complete the following tasks.

Note: Use the SCA developer tools only if you are customizing a site that implements the Kilimanjaro release of SCA or earlier or if you need to access objects not available using the Extensibility API.

Step/Task	For detailed instructions, see
Download and install SCA developer tools	■ Download SCA Source Files ■ Install Additional Files in the Source Directory
Understand best practices for customizing SCA	Best Practices for Customizing SCA
Customize by extending or overriding SCA source files	Example SCA Customizations
Understand how SCA source files are organized and compiled	SCA Build Process
Test your SCA customization locally	Test SCA Customizations on a Local Server
Deploy local source files to an SSP application in NetSuite and then deploy the SSP application to your site	Deploy SCA Customizations to NetSuite

Overview

i Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

A theme is a special type of extension that affects a domain's design and appearance (layouts and styles). Themes contain HTML templates, Sass files, and other assets that can be made available as published themes (bundled into a single SuiteApp) or deployed to a NetSuite account by on-site theme developers.

To learn more about customization options and whether developing a custom theme using the theme developer tools is the right option for your needs, see [Developer Essentials](#).

i Note: The only theme available to SuiteCommerce MyAccount sites is the SuiteCommerce Base Theme published by Oracle NetSuite. You can use the Base Theme as a baseline for your own theme to be compatible with SuiteCommerce MyAccount.

Read the following topics to learn how to download the theme developer tools and then start to create your own themes:

- [Set Up Theme Developer Tools](#)
- [Theme Development Files and Folders](#)
- [Anatomy of the Base Theme](#)
- [Develop Your Theme](#)
- [Test and Deploy Your Theme](#)
- [Best Practices for Creating Themes](#)
- [Commerce Sass Style Definitions](#)
- [Theme Tutorials](#)
- [Do More With Themes](#)

Set Up Theme Developer Tools

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Before you can create a theme, you need to download the theme developer tools and extract them to create a top-level theme development workspace. Then you can use the theme developer tools to run Gulp.js commands to fetch files from the server, test your changes locally, and deploy themes to NetSuite. For more information about working with the theme developer tools, see [Theme Development Life Cycle](#).

Important: You can only develop one theme per top-level theme workspace. To develop two or more themes simultaneously, you must set up multiple instances of the theme developer tools, one for each theme.

Note: The theme developer tools are required for all SuiteCommerce sites and any SuiteCommerce Advanced (SCA) sites implementing the Aconcagua release or later. You cannot customize your SCA site's Sass or HTML template files without these tools.

Prerequisites

Before downloading the theme developer tools, complete the following tasks:

- Verify that you have SuiteCommerce or SuiteCommerce Advanced installed as described in [Install Your Commerce Website Applications](#).
- Verify that you have SuiteCommerce Extension Manager installed as described in [Install Your Commerce Website Applications](#).
- Set up your developer environment as described in [Install Node.js](#) and [Install Gulp.js](#).

Download Theme Developer Tools

Locate and download the theme developer tools appropriate for your implementation as described in the following procedure.

To download and extract theme developer tools:

1. Log in to your NetSuite account.
2. In NetSuite, go to Documents > Files > File Cabinet.
3. Navigate to SuiteBundles/Bundle 521562/.
4. Download the .zip file you find there:
ThemeDevelopmentTools-24.1.x.zip (where x equals the latest minor release).

Important: If you are implementing the Aconcagua or the 2018.2 release of SuiteCommerce Advanced, download the ThemeDevelopmentTools-18.2.1.zip file. These are the only tools compatible with your implementation.

5. Extract the .zip file to a location in your local environment. This becomes your root development directory for custom themes.

The .zip file extracts into a directory of the same name, but you can rename this directory to suit your needs.



Important: Do not move, delete, or rename any files or folders within the top-level development directory.

6. Open a command line or terminal window.
7. Access your root development directory created previously.
8. Enter the following command to install required Node.js packages into this directory:

```
npm install
```

This command installs the dependencies required to manage your custom themes. These files are stored in the `node_modules` subdirectory of the root development directory. This command may take several minutes to complete.



Note: If you use a sandbox account, follow the preceding steps to create two separate root development directories—one for production and one for sandbox. Following this best practice helps ensure you fetch from and deploy to the correct account later in the development lifecycle.

Using Theme Developer Tools with Token-Based Authentication

Theme Developer Tools can be used to support token-based authentication (TBA) for your site. To take advantage of this, complete the following steps:



Note: If you completed these steps when setting up the Extension Developer Tools, you do not need to repeat these steps.

1. Create an integration record.
For more information, see the help topic [Create Integration Records for Applications to Use TBA](#).
 - a. In the **Name** field, enter a name for the integration record.
 - b. In the **State** field, select **Enabled**.
 - c. Check the **Token-Based Authentication** box.
 - d. Check the **TBA: Authorization Flow** box.
 - e. In the **Callback URL** field, enter: `http://localhost:7777/tba`.
 - f. Clear the **Authorization Code Grant** box.
 - g. Check the **User Credentials** box.
2. Save the integration. Be sure to copy the consumer key/secret before closing the page.
3. Paste the consumer key/secret in the `.env` file.



Note: This file is inside the root theme developer tools folder that was downloaded when setting up developer tools. For more information about locating this folder, see [Download Theme Developer Tools](#).

Next Steps

After setting up the theme developer tools, you can continue as follows:

- To learn more about how themes work and the files that make up a theme, see [Theme Development Files and Folders](#) and [Anatomy of the Base Theme](#). (Recommended if you're new to website development and learning best practices.)
- If you want to use the SuiteCommerce Base Theme as the starting point for your custom theme, see [Use the SuiteCommerce Base Theme](#).
- To be sure your custom theme works as intended, review [Best Practices for Creating Themes](#).
- If you are ready to begin theme development, see [Develop Your Theme](#) for instructions.

Theme Development Files and Folders

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

When you extract the zip file that contains the theme developer tools as described in [Set Up Theme Developer Tools](#), the result is your top-level theme development directory. Read the following topics to learn more about theme development files and folders:

- [The Theme Development Directory](#)
- [The Theme Workspace](#)
- [The Theme Directory](#)
- [Theme Development Extras Directory](#)

The Theme Development Directory

The top-level theme development directory contains the following files and folders. You name this top-level directory when you extract the theme development tools.

Note: Some of the files and folders listed in the following table are created when you run specific gulp commands during the theme development lifecycle and so are not visible when you initially extract the tools. These cases are noted in the descriptions of the files and folders in the following table.

File/Folder	Description
DeployDistribution/	Created when you run the <code>gulp theme:deploy</code> command, this directory contains all of the files associated with the compiled application. After compilation, Gulp.js deploys the contents of this directory to your NetSuite file cabinet. Do not manually edit the files in this directory.
gulp/	This directory contains all of the files required by Gulp.js. Do not manually edit the files in this directory.
LocalDistribution/	Created when you run the <code>gulp theme:local</code> command, this directory contains all of the files associated with the compiled application used by the local server. When you run <code>gulp theme:local</code> , Gulp.js deploys the contents of this directory to the local Node.js server. Do not manually edit the files in this directory.
node_modules	Created when you run the <code>npm install</code> command, this directory stores the dependencies and other files required by the theme developer tools. Do not manually edit the files in this directory.
ns_npm_repository	Do not manually edit this file.
Workspace/	Created when you initially run the <code>gulp theme:fetch</code> command, this directory contains all downloaded and customized theme and extension HTML, Sass, and asset files. This is the directory where you develop and maintain all of your theme and extension customizations. See The Theme Workspace for more information.
.jshintrc	Do not manually edit this file.
distro.json	Do not manually edit this file.
gulpfile.js	This file contains all the JavaScript code necessary to run Gulp.js.

	Do not manually edit this file.
javascript-libs.js	Do not manually edit this file.
package.json	This file maintains dependencies required to operate the theme development tools. Do not manually edit this file.
version.txt	This file maintains version information for SuiteCommerce or SuiteCommerce Advanced (SCA). Do not manually edit this file.

The Theme Workspace

The Workspace directory is the location within your top-level theme development directory where you create themes. This directory contains a subdirectory for the downloaded theme and an Extras directory to maintain all extension-related source files. When you run the `gulp theme:fetch` command, the theme developer tools delete all Workspace directory subfolders and download the active theme and extension source files, building a new Workspace environment each time.

```

1 <TopLevelDevelopmentDirectory>/
2   Workspace/
3     Extras/
4       <THEME_DIRECTORY>/

```

The Theme Directory

When you access your theme development directory and run the `gulp theme:fetch` command, the development tools create a subdirectory to store the source files for the active theme. The name of this directory matches the name of the theme that is active when you run the `gulp theme:fetch` command. Perform all theme customizations and template overrides here.

For more information about the contents of the theme directory and customizing the active theme, see [Anatomy of the Base Theme](#).

Theme Development Extras Directory

When you access your theme development directory, the Workspace/Extras/Extensions directory contains subdirectories for each extension that was active when you downloaded the current them using the `gulp theme:fetch` command. Each extension subdirectory contains the following files or folders:



Important: Do not manually edit or remove files found in the Extras/Extensions directory. You can only customize extension-related HTML and Sass files using the Override method. See [Override Active Extension Files](#).

File/Folder	Description
assets/	This directory contains any images or fonts associated with the associated extension. These assets include fonts, logos, icons, and other images related to your site that are not managed by NetSuite.
Modules/	This directory contains a module folder that maintains all HTML templates and Sass associated with the extension. These files are provided here for your reference only.
manifest.json	This file lists all extension-related JavaScript, SSP libraries, configuration, HTML templates, Sass, and assets related to the active extensions downloaded when you ran the <code>gulp theme:fetch</code> command. For more information on this file, see Theme Manifest.

Do not manually edit this file.

If your domain does not have any active extensions when you run the `gulp theme:fetch` command, or an active extension does not contain any Sass, HTML, or assets, the development tools do not create a folder for that extension.



Note: The Extras subdirectory also contains an `application_manifest.json` file. This file confirms that you have a valid SSP Application version that supports the themes and extensions. Do not move, delete, or edit this file.

Anatomy of the Base Theme

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

The SuiteCommerce Base Theme provides a basic theme that you can activate and use as a baseline for your own theme development. The SuiteCommerce Base Theme is a managed bundle and it includes all best practices as described in [Best Practices for Creating Themes](#). To use the SuiteCommerce Base Theme, activate it and then fetch its source files. See the help topic [Fetch Active Theme Files](#) for detailed information.

Read the following topics to learn more:

- [Base Theme Directories and Files](#)
- [Use the SuiteCommerce Base Theme](#)

Base Theme Directories and Files

By examining the contents of the SuiteCommerce Base Theme, you can better understand how all the resources within a theme work together. When you fetch the SuiteCommerce Base Theme, the theme developer tools populate your Workspace directory with all of the folders and files that make up this theme. Your local Workspace directory includes:

```

1 | Workspace/
2 |   SuiteCommerceBaseTheme/
3 |     assets/
4 |     Modules/
5 |     Overrides/
6 |     Skins/
7 |     manifest.json

```

The following table provides more information about each of the preceding directories and files.

Folder/File	Description
assets	The assets directory includes two subdirectories, font-awesome and img, which hold the fonts and images that the modules in the Modules folder depend on. Specifically, the font-awesome folder holds the only custom font used in the SuiteCommerce Base Theme and the img folder is a container for all of the site's base imagery (for example, credit card icons, favicon, and so on). When customizing an existing theme or building a new theme, you can organize your theme's fonts and images any way you like within the asset's parent directory. If you want to add a custom font, create a new sub-directory in the assets directory with the same name as the font you want to use. If you prefer to organize the images for your site into separate folders, you can create those folders in the img directory. All files in the asset's parent directory are included in your theme's package when you test locally or deploy.
Modules	The Modules directory holds all the modules that make up the theme. A module defines a specific area of functionality (feature or utility) for your site. Each module includes a Sass and a Templates folder. The Sass folder houses the style sheets for a module while the Templates folder contains all the templates for a module. When customizing an existing theme or developing a new theme, you can directly edit these style and template files to customize the appearance of your site. For example, if you want to change the vertical alignment of the header logo for your site, you can edit the Workspace/ <THEME_DIRECTORY>/Modules/Header@sc-<release>/Sass/_header-logo.scss file.

Folder/File	Description
Overrides	The Overrides directory does not hold files for the theme. Instead, this directory holds the files for theme-specific overrides of any extensions your site has at the time you fetch the theme. Theme-specific overrides are needed when an extension is developed to add new functionality to a site, but the layout and style of that new functionality does not conform to the rest of the site. In such a scenario, the extension developer can create theme-specific overrides for the unique style and layout that is needed. For more information, see Override Active Extension Files.
Skins	The Skins directory holds preset look and feel options that site administrators can select while using the Site Management Tools (SMT). The SuiteCommerce Base Theme includes two skins: skin1.json and skin2.json. A skin.json file contains an object of key-value pairs where the values in the file create a unique look and feel when the skin is selected by an administrator in SMT. For example, if you open the skin1.json file, you can see that it changes the primary text color for buttons to white and the primary hover text color for buttons to white. You can create additional skins by copying an existing object of key-value pairs in a theme, supplying your own values, and then saving the object as a .json file. The .json files that you add to the Skins directory are automatically added to the manifest.json file when you deploy. The name of the skin matches the name of the .json file in the Skins directory. For example, a file named skin900.json produces a skin named skin900, but you can change the name in the manifest file. For more information about themes and SMT, including creating skins, exposing Sass variables for customization, and setting up themes for customization in SMT, see Enable Customization from Site Management Tools.
manifest.json	This JSON file includes all the information required to compile the resources for the theme. It lists all HTML templates, Sass, and other assets for the theme. The manifest.json file is automatically updated when the compilation process runs. You only need to edit this file to introduce new HTML, Sass, or other assets you create as part of your theme customizations. For more information about working with the theme manifest, see the help topic Edit the Theme Manifest.

For more information about creating custom themes, see [Best Practices for Creating Themes](#) and [Theme Tutorials](#).

Use the SuiteCommerce Base Theme

If you want to use the SuiteCommerce Base Theme as the starting point for your theme customizations, install and activate it as follows:

1. Verify the SuiteCommerce Base Theme SuiteApp is installed. The SuiteCommerce Base Theme is a managed SuiteApp and so you should receive updates to this bundle automatically.
 - a. Find the Bundle ID for the SuiteCommerce Base Theme listed in the table in [Install Your Commerce Website Applications](#).
 - b. Go to Customization > SuiteBundler > Search & Install Bundles > List, find the SuiteCommerce Base Theme, and verify the Bundle ID.
2. Activate the SuiteCommerce Base Theme on a domain. See the help topic [Manage Themes and Extensions](#).
3. Use the Theme Developer Tools to fetch the active theme files for the domain. See the help topic [Fetch Active Theme Files](#).

Important Information About the SuiteCommerce Base Theme

- When you deploy a theme based on the SuiteCommerce Base Theme, the theme developer tools prompt you to create a unique name. You cannot overwrite the base theme but you can create a new theme based on it.
- The base theme is a managed bundle. Updates to this theme coincide with major releases of SuiteCommerce, SuiteCommerce MyAccount, and SuiteCommerce Advanced (SCA).
- SuiteCommerce and SuiteCommerce MyAccount updates are all backward compatible with the base theme. Therefore, when your account receives the latest SuiteCommerce bundle update, any themes created with older versions of the base theme are compatible with your SuiteCommerce or SuiteCommerce MyAccount site.
- SCA customers must take care when creating themes using later releases of the SuiteCommerce Base Theme. Each managed update of the base theme is intended to be used with the corresponding core releases of SCA or later. Therefore, a theme created using the SuiteCommerce Base Theme 2019.1 release is compatible with sites implementing the same major release (SCA 2019.1) and later. That same theme might not be compatible on sites implementing an earlier SCA release (2018.2, for example).
- SuiteCommerce Base Theme is the only theme published by Oracle NetSuite available to SuiteCommerce MyAccount sites. You can use the SuiteCommerce Base Theme as a baseline for your own theme to be compatible with SuiteCommerce MyAccount.

Develop Your Theme

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

After successfully installing the theme developer tools, you can start customize existing themes and developing new themes.

There are two ways to create a custom theme:

- By using the create command: This method uses the fallback theme approach that allows one-click updates.
- By fetching active theme files: This method uses the currently active theme as a baseline for the new custom theme.

Read the following topics to learn more about customizing and developing themes:

- [Create a Custom Theme](#)
- [Fetch Active Theme Files](#)
- [Edit the Theme Manifest](#)

Create a Custom Theme

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

There are two ways to create a theme:

- With the create command: This is the preferred method to create a custom theme.
- By fetching active theme files: To know more, see [Fetch the Active Theme](#).

When you use the create command to create a theme, maintaining and updating it is more efficient.

The new theme inherits files from the Base Theme and uses them to create a scaffolding for the new theme. Your custom theme automatically receives new features and updates made to the Base Theme. All you must do to take advantage of the new features is reactivate your theme.

The create command is defined by a new entry in the Themes manifest: "subtype": "fallback". After creating your theme, you can activate it using gulp commands or the extension manager. Then you can customize your theme further by changing the Sass and template files. For areas you customize, your theme pulls from your customized files. For areas you do not customize, your theme pulls from the Base Theme.

When creating a theme using the create command, consider the following:

- You can not add new templates or Sass to your custom theme; you must instead edit the existing files or add this customization using an extension.
- The assets folder is all or nothing. This means that you can either use all the assets from the Base Theme or all the assets from your custom theme. If you define an assets entry on the manifest.json of your custom theme, that is the assets folder that is used.

Prerequisites

Before you can create a custom theme, ensure that you complete the following tasks:

- Download and configure [Set Up Theme Developer Tools](#).
- Install the Base Theme. It doesn't need to be activated. To learn how to install the base theme, see [Anatomy of the Base Theme](#).

To create a custom theme:

1. Open a terminal or command line.
2. Access the top-level theme development directory you created when you downloaded developer tools.
3. Enter the following command:
`gulp theme:create`
4. Follow the prompts to create the scaffolding of your new theme.
5. After the theme has been created, deployed, and activated, you can customize it.

To implement a custom theme

1. Open a terminal or command line.
2. Access the top-level theme development directory you created when you downloaded developer tools.
3. Enter the following command:
`gulp theme:deploy`
4. To activate the new theme, enter the following command:
`gulp reactivate`
For more information, see the help topic [Activating Themes and Extensions](#).
5. In the top-level theme development directory, open the Modules folder:
`Workspace/<THEME-DIRECTORY>/Modules/`
6. In the Modules folder, create the folders that your custom theme will use.
7. In the new folder, create the files that your custom theme will override on the Base Theme.

 **Tip:** When you first deploy the custom theme, Base Theme files and custom theme files are downloaded. Look at and copy the Base Theme folders, and then paste them to the new theme folder. Change the name of the folder for your custom theme, but remember to retain the file names.

 **Note:** The file names must be the same as the file names for the Base Theme. Instead of using the Base Theme files, the custom theme will use the new files with the same name.

8. In the command line, enter the following command:
`gulp theme:local`
You can view the new theme locally and test the changes before deploying and activating the theme.
9. In the command line, enter the following command:
`gulp theme:deploy`
10. In NetSuite, go to **Commerce > Extensions > Extension Manager** and activate your custom theme.

Fetch Active Theme Files

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

To create a theme, you first download files for the currently active theme to use as the baseline for your own custom theme. You can download files of any published theme or any previously deployed custom theme.

Warning: When you fetch a theme, the developer tools download the active theme's development files to your theme directory. This action overwrites any theme files currently stored in your local workspace. This can result in loss of development files if you are not careful.

Prerequisites

Before you can fetch active theme files, you need to:

- [Install Node.js](#)
- [Install Gulp.js](#)
- [Set Up Theme Developer Tools](#)
- Activate a theme for a domain. If this is your first theme, you can activate the SuiteCommerce Base Theme to use as your baseline. The base theme includes many best practices for creating a theme. See the help topic [Activating Themes and Extensions](#) for more information.

To fetch the active theme and extension files:

1. Open a command line or terminal.
2. Access the top-level theme development directory you created when you downloaded the developer tools.
3. Enter the following command:
`gulp theme:fetch`

Note: For release 2021.2.1 and later accounts that have account-specific domains, specify your account number with the gulp command to enable access to the NetSuite account. For more information about using `gulp theme:fetch --account`, see [Theme Developer Gulp Commands](#).

4. When prompted, enter the following information:

Authentication ID	Enter an ID to represent your NetSuite account and role. If you already created an authentication ID in a previous fetch or deploy, the theme developer tools do not prompt you for an ID and continue to use the existing ID for each subsequent fetch or deploy. To fetch from a NetSuite account that is different from the account used to create the existing authentication ID, use the <code>--to</code> parameter. When you use the <code>--to</code> parameter, you can choose a different authentication ID or create a new one. For more information, see Token Based Authentication.
Account and Role	If you are creating a new authentication ID, you need to select the NetSuite account and role to use with that new authentication ID. If using an existing ID, you are not prompted for this information. In NetSuite, select the account that contains the files you want to fetch and the role you want to use for authentication. Click the Allow button to generate the authentication token and secret.

	 Note: You must specify a role that provides the permissions needed to fetch from NetSuite. By default, the Administrator and SCDeployer roles provide these permissions. To learn more, including how to create a custom role that provides the required permissions, see Developer Tool Roles and Permissions .
Choose Your Website	Select your website that includes the domain you are customizing.
Choose Your Domain	Select your domain with the active themes and extensions you want to download.

When you run the `gulp theme: fetch` command, the theme developer tools download all editable theme-related files for the active theme and place them in your theme workspace. For details see [What Happens When You Fetch an Active Theme?](#)

Continue theme development as outlined in [Next Steps](#).

What Happens When You Fetch an Active Theme?

When you run the `gulp theme: fetch` command, the theme developer tools:

- Create the Workspace directory in your theme development directory. If the Workspace directory already exists, the developer tools clear its contents before downloading. This action overwrites any files currently stored in your local workspace with the files being fetched.
- Download all theme-related HTML and Sass files for the active theme and store them in your workspace, by module, within a directory specific to the theme. For example, `Workspace/<THEME_DIRECTORY>/Modules`. The theme directory is intuitively named to match the name of the active theme.
- Download all theme-related assets to `Workspace/<THEME_DIRECTORY>/assets`.
- Download all theme-related skin preset files to `Workspace/<THEME_DIRECTORY>/Skins`.
- If you are downloading a theme that includes previously deployed overrides, the developer tools download these into `../Workspace/<THEME_DIRECTORY>/Overrides`.
- Validate that all files declared in the theme's `manifest.json` have been downloaded correctly. If any problems occurred, the developer tools list all missing files and direct you to execute the `gulp theme: fetch` command again.

If the chosen domain includes any active extensions when you fetch the active theme, the developer tools:

- Download all extension-related HTML and Sass files for the active extensions and store them in your workspace, by module, within directories specific to each active extension. For example, `Workspace/Extras/Extensions/<EXTENSION_DIRECTORY>/Modules`. The extension directories are intuitively named to match the name of each active extension.
- Download all extension-related assets to `../Workspace/Extras/Extensions/<EXTENSION_DIRECTORY>/assets`.

When you fetch the active theme, you are basically downloading the HTML, Sass, and asset files for the active theme to use as a baseline for your own theme development. Because a published theme is protected by copyright, you cannot overwrite any files for that theme. You can, however, use an existing theme as a baseline for your own, which you later deploy as a unique theme. The SuiteCommerce Base Theme SuiteApp provides a good starting point for building your own theme.

 **Note:** Although you cannot overwrite a published theme, you can make any development changes to your own themes.

When you fetch a theme for a domain, the developer tools download the HTML, Sass, and asset files for the active theme plus any HTML, Sass, and asset files of any extensions active at the time you run the `gulp`

theme:fetch command. The local server needs these files to compile your theme when testing locally. You also have the opportunity to override the HTML and Sass for the active extensions with custom changes or add new assets. Any overrides you declare affect the domain once activated. Be aware that this only affects your domain. You are not overwriting any published extension's code.

Example:

Your domain has an active theme, **SuiteCommerceBaseTheme**, and an active extension, **PublishedExtension1**. Your vendor name is **ACME**. You run the `gulp theme:fetch` command and specify your domain.

In this example, your Workspace directory structure should look similar to following:

```
1 | Workspace/
2 |   Extras/
3 |     Extensions/
4 |       ACME/
5 |         PublishedExtension1/
6 |           assets/
7 |           Modules/
8 |             manifest.json
9 |             application_manifest.json
10 | SuiteCommerceBaseTheme/
11 |   assets/
12 |   Modules/
13 |   Overrides/
14 |   Skins/
15 |   manifest.json
```

When the gulp processes are complete, your theme developer environment should look similar to the following:

File Types	Workspace Subdirectory
Theme-related HTML and Sass files	../SuiteCommerceBaseTheme/Modules/
Theme-related asset files	../SuiteCommerceBaseTheme/assets/
Theme-related skin presets	../SuiteCommerceBaseTheme/Skins/
Extension-related HTML and Sass source files (sorted by module)	../Extras/Extensions/ACME/PublishedExtension1/Modules/
Extension-related asset files	../Extras/Extensions/ACME/PublishedExtension1/assets/
Pre-existing extension overrides	../SuiteCommerceBaseTheme/Overrides/

Next Steps

After fetching active theme files, you can customize the files from the active theme, create new files, enable customization from the Site Management Tools, and more as described in the following topics:

- [Customize Pre-Existing Theme Files](#)
- [Add a New File to a Theme](#)
- [Do More With Themes](#)

For guidance about how best to customize your themes, see:

- [Best Practices for Creating Themes](#)

- [Commerce Sass Style Definitions](#)
- [Theme Tutorials](#)

Customize Pre-Existing Theme Files

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

When you download source files for a theme, you gain access to the HTML, Sass, and assets used by that theme. You can customize the HTML and Sass files directly within your Workspace directory. Best practice when making changes to existing Sass variables, is to edit the existing Sass files directly.

To customize a pre-existing theme file:

1. Open the theme directory in your top-level development directory.
For example:
`Workspace/<THEME_DIRECTORY>/`
2. Locate the subdirectory containing the file or files you want to customize.
For example, if you want to customize one or more of the colors for the base theme, locate the `_colors.scss` file in the theme's BaseSassStyles module:
`Workspace/<THEME_DIRECTORY>/Modules/BaseSassStyles/Sass/variables/_colors.scss`
3. Edit the HTML or Sass file within the associated module.
You can edit the file directly. In this example, you want to change the color for success messages to a different shade of green and so you find the `$sc-color-success` variable and change the color value from `#097839` to `#0d852d`.
4. Save the file.
5. Repeat this procedure for any other theme-related HTML or Sass files you intend to customize.
See [Best Practices for Creating Themes](#) to learn more about customizing theme files.

Next Steps

When you are ready, use the theme developer tools to test or deploy your theme. See [Test and Deploy Your Theme](#) for details.

To learn about additional capabilities offered with themes, see [Do More With Themes](#).

For guidance about how best to customize your themes, see:

- [Best Practices for Creating Themes](#)
- [Commerce Sass Style Definitions](#)
- [Theme Tutorials](#)

Add a New File to a Theme

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

When you customize a theme, you can also create new HTML, Sass, or asset files to meet your needs. The following procedure explains how to add a file to your theme and also includes steps for editing the theme's manifest.json file. You need to edit the theme's manifest to ensure the files that you add are

compiled along with all other resources for the theme. For more information about the manifest file, see the help topic [Edit the Theme Manifest](#).



Note: To reference assets within your code, use the HTML and Sass helpers provided. See [Managing Theme Assets](#) for more information.

To create a new file for a theme:

1. Create your new HTML, Sass, or asset file in the correct location, as required.

File Type	Location Within Workspace/<THEME_DIRECTORY>/
HTML	Modules/<MODULE>/Templates
Sass	Modules/BaseSassStyles/Sass/reference-theme
Assets	assets

2. Create each file as required. See [Best Practices for Creating Themes](#) for details.



Important: To avoid file name collisions, do not create new files or folders that share the same name, even if they reside in different locations.

3. Save each new file in the correct location within your directory structure.
For this example, you want to add a new Sass file titled **_newSass.scss**. As a best practice, save this file in the following location:

Workspace/<THEME_DIRECTORY>/Modules/BaseSassStyles/Sass/reference-theme/_newSass.scss

4. Open the theme's manifest.json file.

For example:

Workspace/<THEME_DIRECTORY>/manifest.json

5. Include your new file or files in the appropriate location as required:
 - If adding a template, list the file by application. Include the .tpl file in the files array of the appropriate application object (Shopping, MyAccount, or Checkout). When declaring your new file, include the path to the correct module. The order of your declaration does not matter.

For example:

```

1 //...
2 "templates": {
3   "application": {
4     "shopping": {
5       "files": [
6         //...
7       ]
8     }
9   }
10 //...
```

- If adding a Sass file, list the .scss file in the files array of the sass object. You set up the Sass entry point in a later step. When declaring your new file, include the path to the correct module. Add this line in a location that makes the most semantic sense within the Sass hierarchy. For example:

```

1 //...
2 "sass": {
3   "entry_points": {
4     "shopping": "Modules/Shopping/shopping.scss",
5     "myaccount": "Modules/MyAccount/myaccount.scss",
6     "checkout": "Modules/Checkout/checkout.scss"
```

```

7   }
8   "files": [
9     //...
10  ]
11  }
12  //...

```

- If adding an asset, add your asset as part of the files array of the `img`, `font-awesome`, or `font` object, as appropriate. When declaring your new file, include the path to the correct folder (`img/`, `font-awesome/`, or `font/`). The order of your declaration does not matter. For example:

```

1  //...
2  "assets": {
3    "img": {
4      "files": [
5        //...
6      ]
7    }
8    "font-awesome": {
9      "files": [
10       //...
11     ]
12   }
13 }
14 //...

```

This ensures that the compiler includes your customizations. In this example, you are adding a Sass file. Therefore, you add `_newSass.scss` as a dependency to the files array of the `sass` object.

Your manifest file might look similar to the following example:

```

1  "sass": {
2    "entry_points": {
3      "shopping": "Modules/Shopping/shopping.scss",
4      "myaccount": "Modules/MyAccount/myaccount.scss",
5      "checkout": "Modules/Checkout/checkout.scss"
6    }
7    "files": [
8      //...
9      "Modules/BaseSassStyles@x.y.z/Sass/reference-theme/_newSass.scss",
10     //...

```

6. Save the manifest.json file.
7. If your new file is an asset or template, you have no further action required. However, if your new file is a Sass file, follow these additional steps:
 - a. Identify the application or applications impacted by your new Sass file.
 - b. Edit the application entry point file to include the same dependency you introduced in the manifest file.

Application Impacted	Application Entry Point
Shopping	Modules/Shopping/shopping.scss
My Account	Modules/MyAccount/myaccount.scss
Checkout	Modules/Checkout/checkout.scss

In this example, the Cart module impacts the Shopping application. Therefore, you open the Shopping entry point file and include the dependency. Your Shopping.scss file should look similar to the following example:

```

1  //...
2  @import "../BaseSassStyles@x.y.z/Sass/reference-theme/newSass";
3  //...

```

Just as you did in the manifest.json file, place this file in a location that makes semantic sense within the Sass hierarchy you are customizing.

- c. Save the application Sass file.

Next Steps

When you are ready, use the theme developer tools to test or deploy your theme. See [Test and Deploy Your Theme](#) for details.

To learn about additional capabilities offered with themes, see [Do More With Themes](#).

For guidance about how best to customize your themes, see:

- [Best Practices for Creating Themes](#)
- [Commerce Sass Style Definitions](#)
- [Theme Tutorials](#)

Build a New Theme

i Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

The following tutorial describes how to begin building a new theme using the SuiteCommerce Base Theme as a starting point. If you have a custom theme that you want to use as the basis for a new theme, the steps in this tutorial still apply but some of the customization details will be different.

i Note: Before starting the following procedure, complete the list of tasks in [Theme Tutorial Prerequisites](#).

To build a new theme:

1. From a command line, go to your top-level theme development directory . The top-level theme development directory is the directory where you set up the theme developer tools, for example: ThemeDevelopmentTools-20.2.1.
2. [Create a Record of the New Theme in NetSuite](#).
Now you're ready to start customizing. The following steps describe how to make several small changes, which you can then build on to develop your own custom theme.
3. The first change to make is to [Display a Large Banner Image on the Home Page](#).
4. Next, [Add a Glowing Color Around the Banner Image](#).
5. Now [Import the New Sass File Into Application Entry Points](#).
6. [Add the Exposed Variable to Skin Files](#) so the editable color can be manipulated by site administrators.
7. Before testing out your custom theme, [Update the Theme Version and Deploy](#).
8. [Activate the Current Version of Your Theme](#).
9. [Test Your Theme Customizations](#).

- Now you can build on the changes from the preceding steps to further customize your new theme. Set a new version in the manifest file for each set of changes you deploy and activate in case you want to return to a previous version.

Fetch the Active Theme

- Run the following command to fetch the active theme.

Note: Remember that this command overwrites the files in your workspace so be sure you don't need any of the existing files.

```
1 | gulp theme:fetch
```

When the fetch is complete, your local workspace directory should look like this example:

```
1 | Workspace/
2 |   SuiteCommerceBaseTheme/
3 |     assets/
4 |     Modules/
5 |     Overrides/
6 |     Skins
7 |     manifest.json
```

Create a Record of the New Theme in NetSuite

When building a new theme, you need to create a record of it in NetSuite as described in the following steps.

- Enter the following command from your top-level theme development directory:

```
1 | gulp theme:deploy --create
```

- When prompted, enter information about your theme such as the name you want to give it. For detailed information about each prompt, see [Deploy a Theme to NetSuite](#).

At this stage, you don't want to activate the theme or take any other action in NetSuite because your new theme is just a simple copy of the active theme.

- Notice that in addition to creating a record of the theme in NetSuite, the gulp create command has updated the files in your local workspace directory to match the theme name you specified. For example, if you named your theme My Awesome Theme, your workspace directory looks like this:

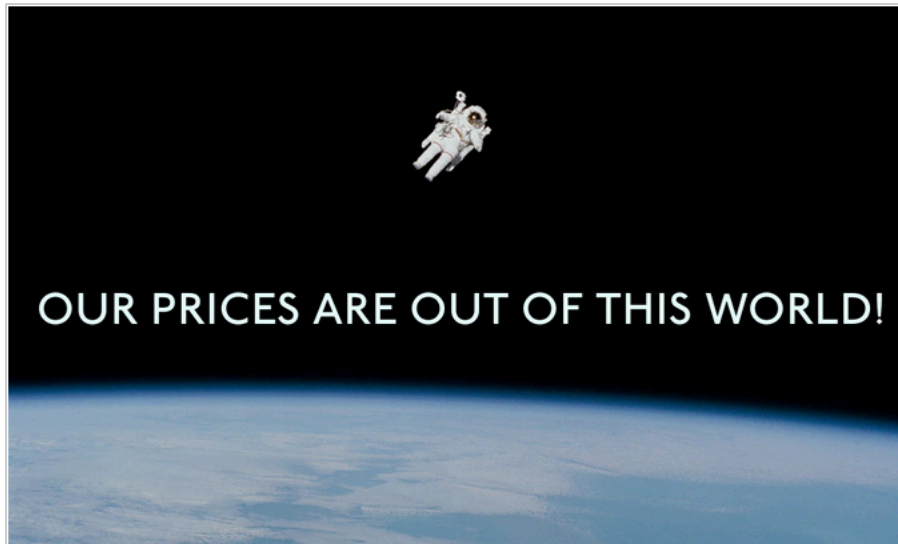
```
1 | Workspace/
2 |   My_Awesome_Theme/
3 |     assets/
4 |     Modules/
5 |     Overrides/
6 |     Skins
7 |     manifest.json
```

Display a Large Banner Image on the Home Page

Follow these steps to display a banner image on the home page:

- Prepare the image you want to use and save it as a .jpg with the following name: prices.jpg.

If you don't have an image, you can use a screen capture of the following image:



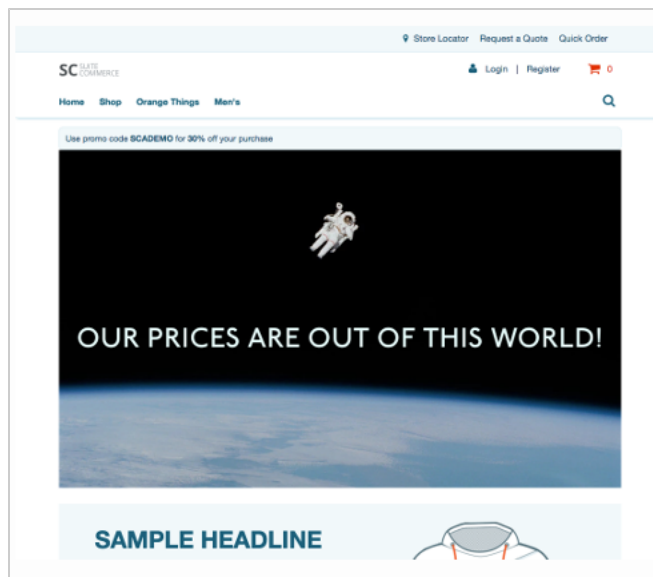
2. Add the prices.jpg file to your local workspace directory in assets/img.
3. Edit the homepage template by opening Modules/Home@sc-~~release~~/Templates/home.tpl and adding the following HTML near the top of the file (for example, above the slider):

```
1 <div class="home-banner-prices">
2   
4 </div>
```

4. Save the changes to the home.tpl file and then run the following command:

```
1 | gulp theme:local
```

Gulp adds the new image file to the manifest and starts up the server. You can open your local homepage in a browser to see the changes. For example, if you used the preceding prices.jpg file for your banner, you should see a homepage like this:



The image is working, but the homepage looks cluttered. The banner top message is too close to the image.

5. Add some styling to increase the space between the image and the banner top message. In your local workspace directory, open the `Modules/Home@sc-<release>/Sass/_home.scss` file and then add the following lines:

```
1 | .home-banner-prices {
2 |     margin-top: $sc-margin-lv4;
3 |     font-size: 0;
4 | }
```

6. Save the changes to the `_home.scss` file. The local server is still running and so it recompiles the site's files when you save. When the recompile is complete, refresh the homepage to see the improvement.

Add a Glowing Color Around the Banner Image

Follow these steps to add the glowing color and to make the color customizable so site administrators can change the color.

1. To make the color customizable, you need to expose a variable for customization. In this case, you need to add the variable to the Sass file and then place a special comment next to it. The comment must contain a properly configured `editable()` function to get picked up by the preprocessor.
2. Following recommended best practices, add a new module for this customization. In your local Workspace/Modules directory, create the following folder and subfolder:

```
1 | GlowModule@1.0.0/
2 |   Sass/
```

3. In the Sass folder, create a file named `_glowy.scss` and add the following code to the file:

```
1 | $color-glowy: #3bfbfd; /* editable({
2 |     "type": "color",
3 |     "label": "Glowy Color",
4 |     "description": "For when you want extra glowy stuff"
5 | })*/
6 |
7 | @keyframes glowy {
8 |     0% {box-shadow: 0 0 -10px $color-glowy;}
9 |     20% {box-shadow: 0 0 100px $color-glowy;}
10 |    40% {box-shadow: 0 0 200px $color-glowy;}
11 |    60% {box-shadow: 0 0 200px $color-glowy;}
12 |    80% {box-shadow: 0 0 100px $color-glowy;}
13 |   100% {box-shadow: 0 0 -10px $color-glowy;}
14 | }
15 |
16 | .glowy {
17 |     animation: glowy 1500ms infinite;
18 | }
```

The first part of the `_glowy.scss` file creates the variable for the editable color. The code sets an initial value for the color, includes an `editable()` state function to establish the variable, and also sets some metadata (type, label, and description) for the variable. Note that while this seems like a regular comment, the `editable()` state function within it makes the variable editable in Site Management Tools (SMT).

The second part of the `_glowy.scss` file is the keyframes of the animation that make the color glow. And, finally, the `.glowy` class is what you will use to attach the animation to an element.

Import the New Sass File Into Application Entry Points

Import the new `_glowy.scss` Sass file into the entry point Sass files for the applications you want to use it in. Doing so ensure that the Sass you have written gets rolled into the master Sass file for the application.

1. In this example, you want to use the new Sass in the Shopping application. In your local workspace directory, open the `Modules/Shopping@sc-<release>/shopping.scss` file and add the following line:

```
1 | @import "../Glowy@custom-1.0.0/Sass/glowy";
```

2. Modify the homepage Sass so your element uses the new styles. In the `Modules/Home@sc-<release>/Sass/_home.scss` file, add the following lines:

```
1 | .home-banner-prices-image {
2 |   @extend .glowy;
3 |   width: 90%;
4 |   margin: 5%;
5 | }
```

3. Save these changes and then start and stop your local server. Because you added new files, they need to be tracked and require a restart. When the local server is back up, refresh your homepage to see the changes.

The banner image should now be outlined in a glowing color.

Add the Exposed Variable to Skin Files

Add your work from the previous steps to a skin so that the editable color can be manipulated by site administrators.

1. In your local workspace directory, open the `Skins/skin1.json` file, scroll to the end of the file, and, after the last entry, add the following line. Be sure you also add a comma to the end of the preceding line.

```
1 | "$color-glowy": "red"
```

2. Now open the `Skins/skin2.json` file, and add the following line to the end of the file. Again, be sure you also add a comma to the end of the preceding line.

```
1 | "$color-glowy": "green"
```

By completing these steps, you have added new values, red and green, that can be activated if an administrator selects them in the Theme Customizer in SMT.

Update the Theme Version and Deploy

Before testing your custom theme, update the theme version in the manifest file to record your work so far, and then deploy the current version of your new custom theme.

1. In your working directory, open the `manifest.json` file at the top level of your custom theme directory.
2. Change the value of the version field to 1.0.1 and then save the file.
3. To deploy your theme, run the following command from the top-level theme development directory. Remember, the top-level theme development directory is the directory where you set up the theme developer tools, for example: `ThemeDevelopmentTools-20.2.1`.


```
1 | gulp theme:deploy
```

4. When prompted, enter information about the theme you are deploying. For detailed information about each prompt, see [Deploy a Theme to NetSuite](#).

Activate the Current Version of Your Theme

Activate the current version of your new custom theme:

1. In NetSuite, go to Commerce > Extensions > Extension Manager.
2. Click **Create Activation** to start a new activation.
3. On the Activate Themes and Extensions page, select the **Themes** tab.
4. Check the **Active** box next to your custom theme and then click **Activate**.

Test Your Theme Customizations

To test your customizations, open your site's homepage and load SMT.

1. You should see that the glowy color around the new banner image is light blue, which is the default you set in the theme.
2. Click the SMT cog icon and select **themes**.
3. Scroll to the bottom of the Theme Customizer, and you should see the new option for **Glowy Color** that you added.
4. Change the value for the **Glowy Color** option and notice that the color changes in real time on your homepage.
5. Because you added new variables for each skin, you can change the skin for your site, here, in the Theme Customizer. If you want to keep one of the skins, save the change and it will take effect on your site right away.

Edit the Theme Manifest

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

Each theme includes a manifest.json file, which is located within the Workspace directory. A theme's manifest is a JSON file that includes all the information required to compile resources for an active theme.

Workspace/<THEME_DIRECTORY>/manifest.json

This file lists all HTML templates, Sass, and assets related to the active theme that you downloaded when you ran the `gulp theme:fetch` command. You only need to edit this file to add any new HTML, Sass, or assets you create as part of your theme customizations. You can edit skin labels as well.

Note: See [Add a New File to a Theme](#) for more examples of how to edit this file when you add any new HTML, Sass, or assets.

This file also manages any extension overrides. However, the theme development tools add the necessary overrides to this manifest when you deploy your customizations. The following topics describes the sections of the theme manifest:

- [Theme Metadata](#)

- [Overrides](#)
- [Templates](#)
- [Sass](#)
- [Skins](#)
- [Assets](#)
- [Record Paths](#)

Theme Metadata

The first entries in the manifest file include metadata about the theme or extension itself. These fields are automatically populated when you initially run the `gulp theme:deploy` command.

```

1 {
2   "name": "StandardTheme",
3   "vendor": "SuiteCommerce",
4   "type": "theme",
5   "target": "SuiteCommerce",
6   "version": "1.2.0",
7   "description": "Standard theme for SuiteCommerce",
8   //...

```

- Name (string) – uniquely identifies the name of the theme. This field is required.
- Vendor (string) – identifies the vendor as a string. This field is required.
- Type (string) – indicates if the type is a **theme**. This field is required.
- Target (comma-separated string) – indicates the Commerce applications supported by the theme. This field is required.
- Version (string) – indicates the version of the theme, such as 1.0.0. This field is required.
- Description (string) – provides a description of the theme as it appears in NetSuite. This field is optional.

Overrides

The override object is only included if you introduce extension overrides. When you use the Override method, the Gulp.js commands detect any HTML or Sass overrides and automatically include them in this file.

For example, if you override the `_error.sass` file of the **Extension1** extension and run the `gulp theme:deploy` command, the theme development tools add the following override your theme's manifest.json file as part of the deployment process:

```

1 //...
2 "override": [
3   {
4     "src": "Overrides/Extension1/Modules/MyExamplePDPEExtension1/Sass/_error.scss",
5     "dst": "Extension1/Modules/MyExamplePDPEExtension1/Sass/_error.scss"
6   },
7   //...

```

Templates

The templates object lists all HTML template files included in the theme by application. The application object includes one object per application (shopping, myaccount, and checkout). Each application lists each file in the files array.

You should manually add any new template files that you introduce as part of your theme customizations to the files array.

```

1 //...
2 "templates": {
3   "application": {
4     "shopping": {
5       "files": [
6         "Modules/AddToCartModifier/Templates/add_to_cart_modifier.tpl"
7       //...
8     ]
9   }
10  "myaccount": {
11    "files": [
12      //...
13    ]
14  }
15  "checkout": {
16    "files": [
17      //...
18    ]
19  }
20 }
21 }
22 //...

```

Sass

The sass object declares the paths to each application entry point and all Sass files to be loaded when you deploy. You should manually add any new Sass files to the files array that you introduce as part of your theme customizations.

Note: When listing a new Sass file, declare each file in an order that makes the most semantic sense within the Sass hierarchy.

```

1 //...
2 "sass": {
3   "entry_points": {
4     "shopping": "Modules/Shopping/shopping.scss",
5     "myaccount": "Modules/MyAccount/myaccount.scss",
6     "checkout": "Modules/Checkout/checkout.scss"
7   }
8   "files": [
9     "Modules/Shopping/shopping.scss",
10    "Modules/MyAccount/myaccount.scss",
11    "Modules/Checkout/checkout.scss",
12    "Modules/twitter-bootstrap-sass@3.3.1/assets/stylesheets/bootstrap/_alerts.scss",
13    //...
14  ]
15 }
16 //...

```

Skins

The skins array is automatically populated when you run either the `gulp theme:local` or `gulp theme:deploy` commands. This array defines an object for each new skin preset file located in the Skins directory when you run these commands. Each skin object includes the following properties:

- **name** – declares the name of the skin as it appears in SMT. As a default, this value equals the file name.
- **file** – declares the name and location of the skin preset file. This name must match the name of the file in the Skins directory.

If necessary, you can edit the manifest.json file to provide a more visually appealing value for the name property.

Example 1. Example

You want to provide four selectable skins for a theme, each with a different color scheme to correspond with the different seasons: winter, spring, summer, and fall. You create four individual skin preset files in your theme's Skins directory: winter_skin.json, spring_skin.json, summer_skin.json, and fall_skin.json.

You use the theme developer tools to run the `theme gulp:local` command. The developer tools automatically edit your theme's manifest.json file to include these skins:

```

1 | //...
2 | , 'skins': [
3 |   {
4 |     'name': 'winter_skin'
5 |     , 'file': 'Skins/winter_skin.json'
6 |   }
7 |   , {
8 |     'name': 'spring_skin'
9 |     , 'file': 'Skins/spring_skin.json'
10 |   }
11 |   , {
12 |     'name': 'summer_skin'
13 |     , 'file': 'Skins/summer_skin.json'
14 |   }
15 |   , {
16 |     'name': 'fall_skin'
17 |     , 'file': 'Skins/fall_skin.json'
18 |   }
19 | ]
20 | //...

```

Later, you decide that you want the SMT Theme Skin Manager to display different names for each skin. You open the manifest.json file and make the following edits to each skin object's name property:

```

1 | //...
2 | , 'skins': [
3 |   {
4 |     'name': 'Winter Color Scheme'
5 |     , 'file': 'Skins/winter_skin.json'
6 |   }
7 |   , {
8 |     'name': 'Spring Color Scheme'
9 |     , 'file': 'Skins/spring_skin.json'
10 |   }
11 |   , {
12 |     'name': 'Summer Color Scheme'
13 |     , 'file': 'Skins/summer_skin.json'
14 |   }
15 |   , {
16 |     'name': 'Fall Color Scheme'
17 |     , 'file': 'Skins/fall_skin.json'
18 |   }
19 | ]
20 | //...

```

Assets

The assets object defines paths to the images and fonts located in the theme directory's assets folder in a files array for each image and font used. These paths are relative to the theme's assets folder path. This is where you add any new asset files introduced as part of your theme customizations.

```

1 | //...

```

```

2  "assets": {
3    "img": {
4      "files": [
5        "img/favicon.ico",
6        "img/image-not-available.png",
7        "img/add-to-cart-logo.png",
8        //...
9      ]
10   },
11   "font-awesome": {
12     "files": [
13       "font-awesome/FontAwesome.otf",
14       "font-awesome/custom/fontawesome-webfont.eot",
15       //...
16     ]
17   },
18   "fonts": {
19     "files": [
20       "fonts/DancingScript-Bold.ttf",
21       "fonts/DancingScript-Regular.ttf",
22       //...
23     ]
24   }
25 }
26 //...

```

Record Paths

The final part of the manifest file lists the path to the theme, the Extension record, and Activation IDs as stored in NetSuite.

```

1  //...
2  "path": "SuiteBundles/Bundle 193239/SuiteCommerce Base Theme",
3  "extension_id": "4535",
4  "activation_id": "59"

```

Next Steps

When you are ready, use the theme developer tools to test or deploy your theme. See [Test and Deploy Your Theme](#) for details.

To learn about additional capabilities offered with themes, see [Do More With Themes](#).

For guidance about how best to customize your themes, see:

- [Best Practices for Creating Themes](#)
- [Commerce Sass Style Definitions](#)
- [Theme Tutorials](#)

Test and Deploy Your Theme

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

After developing your theme, you can test it locally and deploy it as described in the following procedures:

- [Test a Theme on a Local Server](#)
- [Deploy a Theme to NetSuite](#)

Test a Theme on a Local Server

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

You can test your theme customizations on a local server before deploying to NetSuite.

Important: You can use the local server to test frontend changes. You must deploy and activate your theme to view any changes you made to skins or to Sass variables exposed to SMT.

To test your theme on a local server:

1. In your local developer environment, open a command line or terminal and access the top-level development directory.
2. Run the following command:

```
gulp theme:local
```

Warning: Potential data loss. Besides compiling and deploying your theme to a local server, the `gulp theme:local` command updates the `manifest.json` file for the theme. This action overwrites any manual changes you made to this file. To preserve any manual changes to your `manifest.json` file, run the `gulp theme:local --preserve-manifest` command instead. See the help topic [Edit the Theme Manifest](#) for details about this file.

When you run the `gulp theme:local` command, it compiles all theme source files. For details see: [What Happens When You Test a Theme on a Local Server?](#)

3. To view your customized theme in a browser, navigate to the local version of the application using one of the following URLs:
 - `http://<DOMAIN_NAME>/c.<ACCOUNT_ID>/<SSP_APPLICATION>/shopping-local.ssp`
 - `http://<DOMAIN_NAME>/c.<ACCOUNT_ID>/<SSP_APPLICATION>/my_account-local.ssp`
 - `http://<DOMAIN_NAME>/c.<ACCOUNT_ID>/<SSP_APPLICATION>/checkout-local.ssp`

In the above URL patterns, you must replace the variables listed in the following table with values for your specific environment:

Variable	Replacement	Example
<code>DOMAIN_NAME</code>	Replace this value with the domain name configured for your NetSuite website record.	<code>http://www.mysite.com</code>
<code>ACCOUNT_ID</code>	Replace this value with your NetSuite account ID.	<code>c.123456</code>
<code>SSP_APPLICATION</code>	Replace this value with the URL root that you are accessing.	SuiteCommerce example: <code>scs</code>

Variable	Replacement	Example
	For SuiteCommerce implementations, this variable should read scs . For SuiteCommerce Advanced (SCA), this variable equals the URL root of the SCA implementation.	SuiteCommerce Advanced examples: ■ sca-dev-aconcagua ■ sca-dev-2019-2

Complete URLs for SuiteCommerce implementations would look like these examples:

- <http://www.mysite.com/c.123456/scs/shopping-local.ssp>
- http://www.mysite.com/c.123456/scs/my_account-local.ssp

Complete URLs for SuiteCommerce Advanced implementations would look like these examples:

- <http://www.mysite.com/c.123456/sca-dev-aconcagua/shopping-local.ssp>
- <http://www.mysite.com/c.123456/sca-dev-2019-2/shopping-local.ssp>

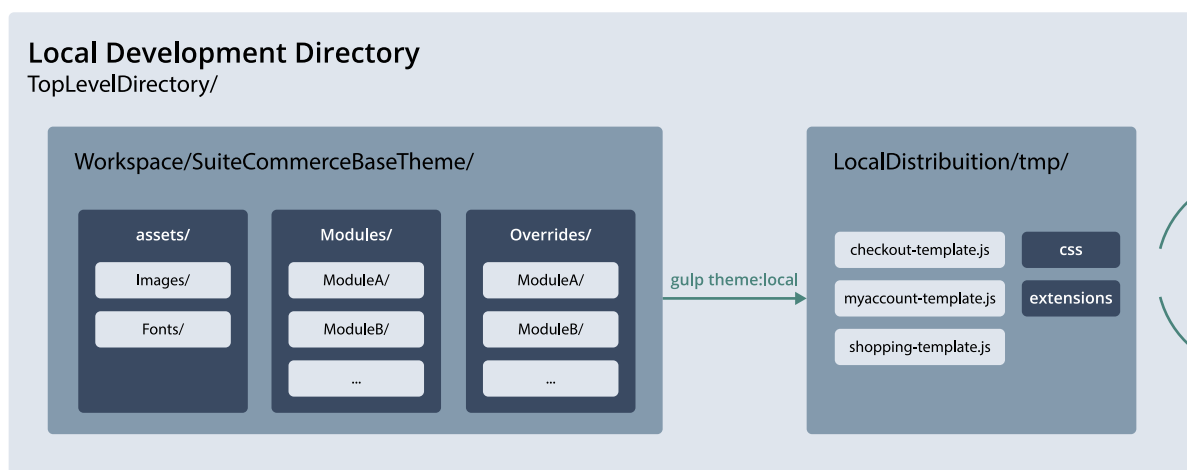


Important: If you add a new file or make changes to any overrides after launching the local server, you must run the `gulp theme:local` command again to include the new changes. Gulp.js does not automatically compile new files or process overrides.

What Happens When You Test a Theme on a Local Server?

The `gulp theme:local` command compiles all source files, including your theme and any extension customizations, into combined files within a `LocalDistribution/tmp` directory. This directory contains the Sass and template files used by the local version of the application.

When you run the command, the local server starts automatically. When the server starts, Gulp.js initializes watch tasks that listen for changes to files in the `Templates` and `Sass` directories. When you save changes to a file, gulp automatically recompiles the source files and updates the `LocalDistribution` directory. Gulp also outputs a message to the console when an update occurs.



Next Steps

After testing your theme locally and verifying its functionality, you can deploy it as described in [Deploy a Theme to NetSuite](#).

Then you must activate the theme to apply it to your website domain. See the help topic [Activating Themes and Extensions](#) for instructions on activating your new theme for the domain or domains of your choice.

Deploy a Theme to NetSuite

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

After testing and verifying your theme locally, deploy your theme to NetSuite so you can activate it for a website domain.

Watch the following help video for information about how to deploy a theme:

 [Deploy a Theme to NetSuite](#)

To deploy your theme to NetSuite:

1. In your local developer environment, open a command line or terminal and access the top-level development directory.
2. Run the following command:

```
gulp theme:deploy
```

Note: For release 2021.2.1 and later accounts that have account-specific domains, specify your account number with the gulp command to enable access to the NetSuite account. For more information about using gulp theme:deploy --account, see [Theme Developer Gulp Commands](#).

Warning: Potential data loss. In addition to compiling and deploying your theme, the gulp theme:deploy command updates the manifest.json file for the theme. This action overwrites any manual changes you made to this file. To preserve any manual changes to your manifest.json file, run the gulp theme:deploy --preserve-manifest command instead. See Theme Manifest for details on this file.

3. When prompted, enter the following information.
Unless otherwise noted, create all entries using only alphanumeric characters without spaces.

Note: These prompts appear the first time you deploy a new theme to NetSuite. To reset the login information and fill in these prompts again, use the gulp theme:deploy --to command.

Prompt	Description
Authentication ID	Enter an ID to represent your NetSuite account and role. If you already created an authentication ID in a previous fetch or deploy operation, the theme developer tools do not prompt you for an ID and continue to use the existing ID for each subsequent fetch or deploy. To deploy to a NetSuite account that is different from the account used to create the existing authentication ID, use the --to parameter. When you use the use the --to parameter, you can choose a different authentication ID or create a new one. For more information, see Token Based Authentication.
Account and Role	If you are creating a new authentication ID, you need to select the NetSuite account and role to use with that new authentication ID. If using an existing ID, you are not prompted for this information.

Prompt	Description
	In NetSuite, select the account you want to deploy to and the role you want to use for authentication. Click the Allow button to generate the authentication token and secret.  Note: You must specify a role that provides the permissions needed to deploy to NetSuite. By default, the Administrator and SCDeployer roles provide these permissions. To learn more, including how to create a custom role that provides the required permissions, see Developer Tool Roles and Permissions.
Website	Choose the website associated with your domain.
Vendor	Enter the name of the vendor building this theme.
Name	Enter a name for your custom theme. This cannot contain special characters and spaces.
Fantasy Name	Provide a name for your theme as you want it to appear in the NetSuite user interface. This can contain special characters and spaces.
Version	Enter a version for your theme.  Important: To take advantage of theme update requirements, assign a version number to your themes. Version numbers should follow this format, where d is a single digit: dd.dd.dd.
Description	Enter a brief description for your theme. The description appears in NetSuite when you select themes to activate.
Supported Products	Select the product or products (SuiteCommerce Online or SCIS) to which you are deploying. An asterisk (*) identifies a selected product. SuiteCommerce Online applies to SuiteCommerce and SuiteCommerce Advanced. ■ Use the spacebar key to select or deselect an option. ■ Use the UP and DOWN arrow keys to scroll through the options. ■ Toggle the A key to select or deselect all options.
Target Version	Using Commerce versioning schema, enter a string to declare the target versions to which this theme applies. The developer tools prompt you for a target version for each product specified earlier. Leaving these prompts blank results in compatibility with all versions of the associated Commerce products. See Declare Target Versions and SuiteScript 2.x Bundle Installation Script Type for more information.

When you run the `gulp theme:deploy` command, the theme developer tools validate your customizations, copy them into a local `DeployDistribution` directory, and update the manifest file with any necessary overrides. The theme developer tools then upload the theme files to your NetSuite account, making them available for activation. For details, see [What Happens When You Deploy a Theme?](#)

Continue theme activation as outlined in [Next Steps](#).

What Happens When You Deploy a Theme?

The deployment process uploads theme files and extension overrides (if applicable) to a location in your NetSuite File Cabinet. This process compiles the theme source files to test them before uploading them to NetSuite.

During the deployment process, the theme developer tools:

- Copy all custom theme development files (modules, assets, and overrides) into the DeployDistribution folder in your local top-level development directory. If this directory does not exist, the developer tools create it.
- Test and validate your customizations.
- Update the manifest.json file to include any overrides.
- Upload the theme source files to your NetSuite File Cabinet as bundled files (maintaining the same organizational structure).

 **Note:** The `gulp theme:deploy` command uploads your theme development files in groups of files called chunks. The default number of files in a single chunk is 80. If needed, you can configure the number of files in each chunk. See [Troubleshooting the Developer Tools](#) for more information.

When the theme source files are uploaded to the File Cabinet, NetSuite creates an Extension record for the custom theme. Like a published theme or extension, the source files have not compiled into usable files, but they are available for activation.

Note: Extension overrides exhibit the following behaviors:

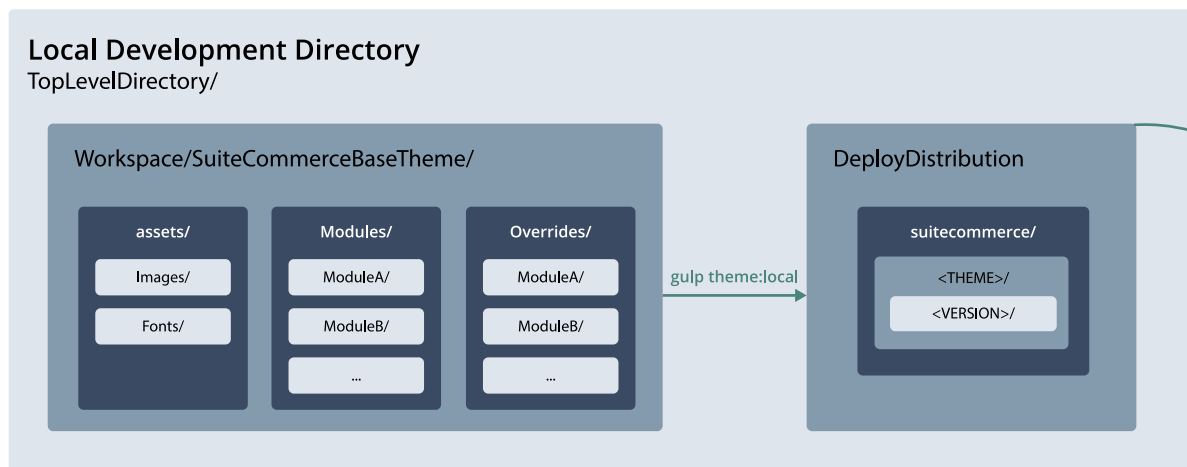
- Your extension overrides always deploy as part of a custom theme. If you activate a custom theme but fail to activate any extension for which the theme includes overrides, the application ignores the overrides and your site functions normally.
- If you deploy a theme with extension overrides and later activate new extensions for the same domain, your deployed customizations do not apply to the new extensions. You must update your theme to include these customizations.

Example 2. Example

You download the source files for the active theme (**SuiteCommerce Base Theme**) and the active extension (**PublishedExtension1**). You follow best practices and procedures to customize your theme files and override your extension files. After testing on a local server, you decide to deploy your customizations.

You run the `gulp theme:deploy` command, and the theme development tools update the manifest file with any overrides, validate your customizations, and copy the contents of your theme directory to your `DeployDistribution` directory. The development tools then upload your development files to a location in your NetSuite File Cabinet. These files are not compiled into any usable runtime files, but they are now available for activation on a site and domain.

When you run the `gulp theme:deploy` command, the development tools prompt you for a new theme name, which you call **MyCustomTheme1**. Finally, the development tools rename your `Workspace/SuiteCommerceBaseTheme` directory to `Workspace/MyCustomTheme1` and update the manifest and configuration file with the new theme name. This prepares the local environment to function with the new name.



Next Steps

After deploying theme files, you must activate the theme to apply it to your website domain. See the help topic [Activating Themes and Extensions](#) for instructions on activating your new theme for the domain or domains of your choice.

Best Practices for Creating Themes

i Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

Before customizing or creating a theme, read and understand the following best practices. These topics provide important steps you must take to ensure that your themes (and extension overrides, if applicable) deploy without error.

- [Create a Custom Theme](#)
- [General Best Practices for Themes](#)
- [HTML Best Practices for Themes](#)
- [Sass Best Practices for Themes](#)
- [Managing Theme Assets](#)

General Best Practices for Themes

i Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

The following list provides some general knowledge and best practices to remember when customizing themes for Commerce websites:

- Use themes to customize HTML templates and Sass for your site.
- If you are using a published theme as a baseline, the developer tools force you to create a new theme. This new theme includes your changes as a custom theme.
- Use the existing folder structure as created when you downloaded themes and extensions to your Workspace directory. If you must add new subdirectories, make sure the paths to any new files are included in the theme manifest and all required entry points.
- Use a development domain to test your changes before applying the updates to your production domain.
- Do not move, delete, or edit files located in your Workspace/Extras directory. The files stored here are for reference and overrides only.
- When you fetch a theme, you also get the HTML and Sass files for any active extensions. You can customize these files to suit your theme using the Override method. See [Override Active Extension Files](#) for details.
- Place any new assets (images or fonts) in the appropriate location within the theme's assets directory.
- Use helpers when referencing any assets within your HTML or Sass customizations and overrides. See [Managing Theme Assets](#) for details.
- Follow the template context when editing HTML template files or creating overrides. See [HTML Best Practices for Themes](#) for details.
- To avoid file name collisions, do not create any new files or folders that share the same name, even if they reside in different locations. The exception to this practice is when working with extension overrides.
- Limit the number of files and folders at the same level to 100 when developing themes and extensions. Some options include introducing images across multiple folders and placing custom modules in a different top-level directory. For more information on file and folder limiting, see the help topic [SOAP Web Services Governance Overview](#).

HTML Best Practices for Themes

i Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

Commerce templates use the Handlebars.js templating engine to keep the HTML (presentation layer) separate from any deeper code logic. These logic-less templates help you build your themes without requiring access to deeper code. Templates define the HTML for specific areas of the user interface, such as a button or Home banner.

When an associated theme is activated, the template engine combines all of these files into a single, finished web page. To achieve this, templates use an easy-to-understand syntax, which is a combination of HTML and Handlebars.js expressions (helpers). These helpers tell the template engine to include the value of variables (properties and objects) when rendering your site or executing functions. These variables are specific to each template, depending on what the template is designed to render. This information is called the template's **context**.

Template Context

Each template relies on its context to know what variables are available (what data the template can render). You cannot customize a template to render information it cannot provide. Therefore, you must operate within this context when making any customizations to pre-existing templates or introducing your own.

Each Commerce template includes a list of context variables within the file itself. The list includes the context objects, types, or any properties as part of an array. This information is nested within comment blocks at the end of each file.

i Note: If you are an extension developer making a template for an extension, the context is defined by your own custom views. See [Work with Views in an Extension](#) for details.

i Note: SuiteCommerce Advanced users have access to the context within the associated view files.

The following example from `case_list.tpl` shows this context notation:

```

1 | ...
2 | {!--
3 | Use the following context variables when customizing this template:
4 |
5 |     pageHeader (String)
6 |     hasCases (Number)
7 |     isLoading (Boolean)
8 |     showPagination (Boolean)
9 |     showCurrentPage (Boolean)
10 |     showBackToAccount (Boolean)
11 |
12 | --}}

```

The following code snippet shows the `hasCases` number and `isLoading` Boolean variables as used in the `case_list.tpl` template code. This code either renders a support case list or renders a **Loading** or **No Cases Found** string, depending on the query results.

```

1 | ...

```

```

2  {{#if hasCases}}
3    <table class="case-list-recordviews-table">
4      <thead class="case-list-content-table">
5
6        ...
7
8      </thead>
9      <tbody data-view="Case.List.Items"></tbody>
10   </table>
11 {{else}}
12   {{#if isLoading}}
13     <p class="case-list-empty">{{translate 'Loading...'}}</p>
14   {{else}}
15     <p class="case-list-empty">{{translate 'No cases were found'}}</p>
16   {{/if}}
17 {{/if}}
18 ...

```

To find the template context in a template file:

1. In an editor, open the source .tpl file for the template you want to customize.
2. At the end of the file, look for the following string, located within comment tags:

```

1  {{!--
2  Use the following context variables when customizing this template:
3  ...

```

3. Note the context variables available to the template and customize accordingly.

To find the context variables using the browser developer tools:

1. Currently, some template files do not include the context. In these cases, you can use the `{{log this}}` helper.

This procedure assumes that you have begun customizing a template.

Open the template file you are customizing in an editor.

Note: The examples in this section use Google's Chrome browser developer tools.

2. Include the following helper as the first line in the template for which you want to reveal the context.

```

1  {{log this}}

```

3. Deploy your customization to your local server. See [Test a Theme on a Local Server](#).
4. Navigate to the page rendering your custom template.
5. Using your browser's developer tools, inspect and refresh the page.
6. The developer tools **Console** tab lists the variables available to the template as the output of the `{{log this}}` helper:

```

▼ Object
  ► bottomBannerImages: Array[3]
  ► carouselImages: Array[3]
  imageHomeSize: "main"
  imageHomeSizeBottom: "main"
  ► __proto__: Object
undefined

```

Sass Best Practices for Themes

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

Commerce themes let you customize the design elements of your web store experience using Sass, or Syntactically Awesome StyleSheets. Sass is a scripting language that is transformed into CSS when you use the theme development tools to compile and deploy your customizations. The SuiteCommerce Designer's Guide provides more information on customizable Sass styles. See [Commerce Sass Style Definitions](#) for details.

Note: All Commerce Sass files are named as partials (with a preceding underscore), such as `_BaseSassStyles.scss`.

The following best practices describe how to implement Sass in your themes:

- [Creating Sass Variables](#)
- [Formatting Sass for SMT](#)
- [Sass Entry Points](#)

Creating Sass Variables

The SuiteCommerce Base Theme SuiteApp includes the BaseSassStyles module. This module contains all Sass variables and metadata (for exposing and organizing variables in the Site Management Tools). The best practice when creating a theme is to use the SuiteCommerce Base Theme as a baseline. See [Anatomy of the Base Theme](#) for more information about this theme.

You can create your own Sass variables. You can introduce variables within new Sass files, as part of a new module, or within the BaseSassStyles module. Observe the following best practices regarding new variables and their metadata:

- If you are defining a new module as part of a theme, define all new variables and any metadata within that module.
- If you are defining a variable for a particular module only, define that variable within the associated module. Avoid introducing global definitions within one module.
- If you are defining a variable that controls multiple properties in different modules (such as a new theme color `$sc-color-theme-250`), define that variable within the BaseSassStyles module.
- If you are adding a new Sass file to your theme, you must declare each file in the theme's manifest.json file and in the applicable entry point file. See [Add a New File to a Theme](#) for details.
- Always test any new variables in your domain, across all applications. This ensuring that any exposed variables display as expected in the Site Management Tools (SMT) Theme Skin Manager.

Formatting Sass for SMT

Some Sass development practices can affect the SMT Theme Skin Manager user experience.

As you build Sass for your theme, you can expose variables to the SMT Theme Skin Manager. When an SMT administrator changes an exposed variable, the application compiles any changes and renders them for preview. This action includes two separate compilations. The first compilation occurs on the frontend within 2 seconds. However, after 5 seconds of user idle time, the application triggers a second compilation on the backend.

Consequently, the following Sass practices result in variables that are only accessible in the backend. As a best practice, avoid using the following practices for any Sass variables exposed to SMT:

- Sass variables that receive user-defined function calls

- If conditional statements
- Mixins

User-Defined Function Calls

Avoid user-defined function calls. For example, assuming that `$sc-primary-color` is also:

```
$sc-color-secondary: myfunc($sc-primary-color) /*editable(.)
```

In this example, you set the result of an exposed variable, such as `$sc-color-secondary`, to a custom function, `myfunc($sc-primary-color)`. Assuming `$sc-primary-color` is also declared as editable, when the SMT administrator changes the primary color value, the primary color value compiles quickly in the frontend. The problem with this approach is that `myfunc($sc-primary-color)` only exists in the backend and takes longer to preview.

If Statements and Mixins

Avoid if constructions using exposed Sass variables. For example:

```
1 | @if $sc-color-secondary == $12345 { background-color: $sc-color-primary;}
2 | @else { background-color: $56789;}
```

The problem with this example is that the frontend does not know the else condition. Passing values within or grouping variables as mixins causes a similar issue. All of these issues are solved during the backend compilation, but this can take a considerable amount of time.

Sass Entry Points

Every theme relies on an **entry point** file to load Sass into the correct application (Shopping, MyAccount, or Checkout). Each entry point declares the Sass files that are part of the application. For example, if a Sass file affects the shopping application only, it needs to load through the `shopping.scss` entry point. If it affects all three applications, it needs to load through all three entry points.

For an example of how to edit an entry point, see [Add a New File to a Theme](#).



Important: All Sass files must also be declared in the theme's manifest.json file. See the help topic [Edit the Theme Manifest](#) for more information.

Each theme includes the following Sass entry points.

Application Impacted	Application Sass File
Shopping	Modules/Shopping/shopping.scss
MyAccount	Modules/MyAccount/myaccount.scss
Checkout	Modules/Checkout/checkout.scss

Managing Theme Assets

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

An asset is an image or font that is not managed by a NetSuite account but still is part of a theme or extension. An example of an asset is a carousel image, an icon, or a new font. This can be either a pre-existing asset or one that you introduce as part of a new theme.

When you run the `gulp theme:fetch` command:

- Assets for the active theme download to your Workspace/<THEME_DIRECTORY>/assets folder. This is the location where you manage all of your assets (new and pre-existing).
- Assets for any active extensions download to your Workspace/Extras/Extensions/<EXTENSION_DIRECTORY>/assets folder. Do not move, delete, edit, or add files in this location.

When you activate your theme, all assets are placed in specific locations in your NetSuite File Cabinet based on parameters you specify when you deployed your theme (vendor name, theme name, and theme version). Later, when you activate a newer version of the same theme, your assets are now located in a different location in your File Cabinet. Your code must adapt to the change in the path. If you use absolute paths, the asset links break.

Important: Do not use absolute paths when referring to assets in your code customizations and overrides. Asset files are managed in different locations in NetSuite based on the theme's vendor, extension name, version, and so on. As a best practice, always use the HTML and Sass helpers when referencing assets to ensure that you maintain dynamic paths without unnecessary code changes.

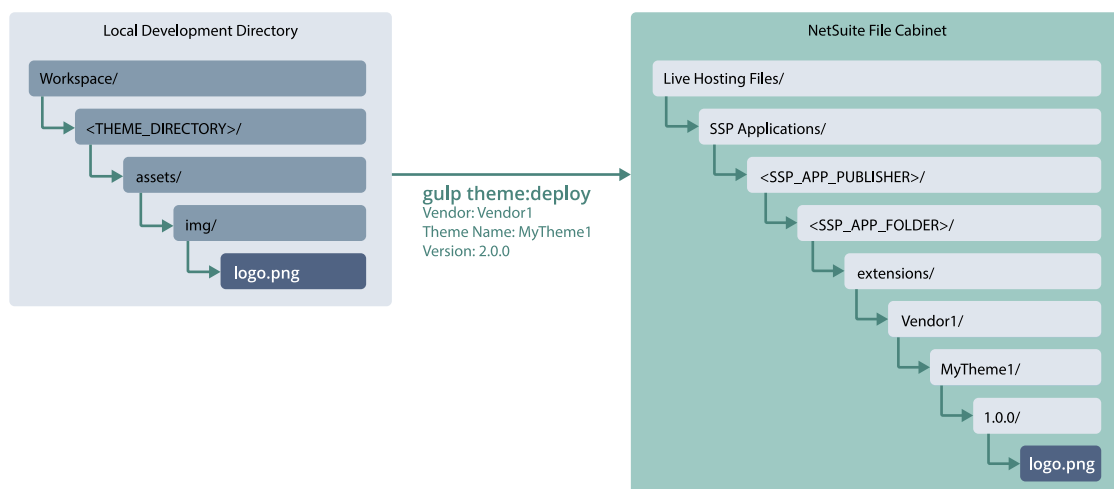
To maintain dynamic paths to your assets, regardless of the vendor, theme, version, and so on, use the following helpers:

- [HTML Helpers](#)
- [Sass Helpers](#)

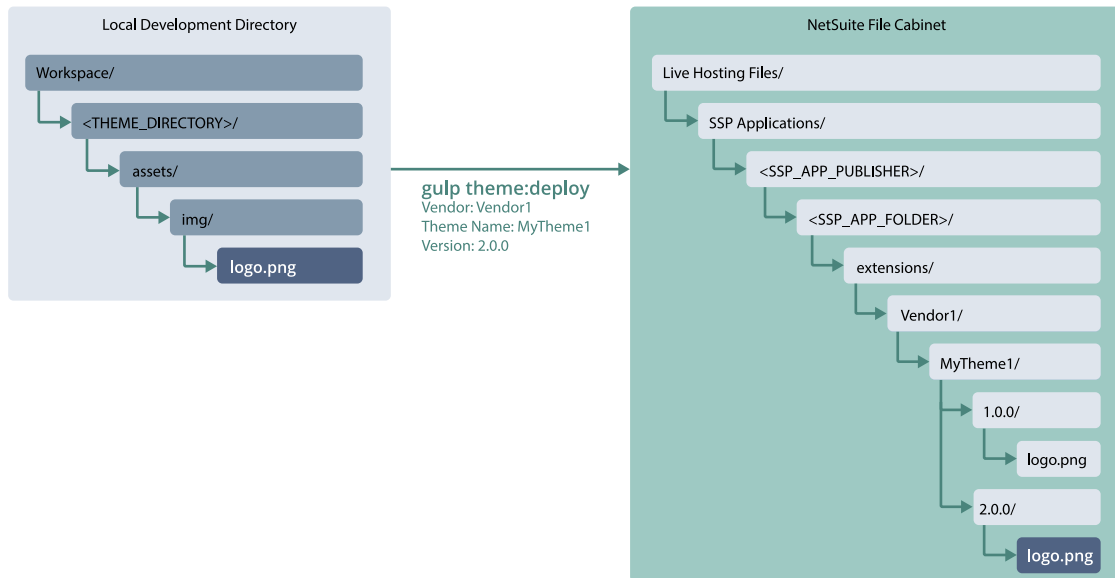
Example:

Using the theme developer tools, you deploy a new theme (**MyTheme1**, Version: **1.0.0**). You then activate that theme for a specific site and domain. The compiler places all assets in your NetSuite File Cabinet. The exact location is specific to the SSP application, vendor, extension, and version that you specified when you deployed your theme.

This location might be similar to the following representation:



Later, you decide to update your theme, giving it a new version, **2.0.0**. When you activate your latest version, the compiler places all assets into a new location:



HTML Helpers

The following Handlebars.js helpers provide a means to maintain dynamic paths to your assets when customizing HTML templates.

- `getExtensionAssetsPath(default_value)`
- `getExtensionAssetsPathWithDefault(config_value, default_value)`
- `getThemeAssetsPath(default_value)`
- `getThemeAssetsPathWithDefault(config_value, default_value)`

getExtensionAssetsPath(default_value)

Use this HTML template helper:

- To access pre-existing assets included with an active extension
- In your extension overrides (templates)

Note: The `default_value` argument is the relative path to the asset.

Example Syntax in a Template File:

The following is an example of how to use this helper in an HTML template:

```
1 | 
```

Example Result:

This helper returns the active extension's asset path (where `<FULL_PATH>` is the SSP Application base path):

```
1 | <FULL_PATH>/extensions/<VENDOR>/<EXTENSION>/<VERSION>/img/logo.png
```

getExtensionAssetsPathWithDefault(config_value, default_value)

Use this HTML template helper:

- To access pre-existing assets included with an active extension
- In your extension overrides (templates)
- When you expect that the asset has a configured path in NetSuite, but want to provide a default path if the configuration does not exist.

Note: The first argument (config_value) is the path as configured in NetSuite. In cases where the asset is not configured, you provide a fallback path (default_value) to be retrieved from the extension's assets.

Example Syntax in a Template File:

The following is an example of how to use this helper in an HTML template:

```
1 | 
```

Example Result:

This helper returns the active extension's asset path. If the first argument is defined, this helper returns the path as configured in NetSuite:

```
1 | <MY_PATH>/img/other_logo.png
```

If the first argument is undefined, this helper uses the second argument to return the correct path of the active extension:

```
1 | <FULL_PATH>/extensions/<VENDOR>/<EXTENSION>/<VERSION>/img/logo.png
```

getThemeAssetsPath(default_value)

Use this HTML template helper:

- To access new and pre-existing assets included in your theme directory
- In your extension overrides (templates)
- In your theme customizations (templates)

Using this helper, default_value is the relative path to the asset.

Example Syntax in Template File:

```
1 | 
```

Example Result:

This helper returns the active theme's asset path (where <FULL_PATH> is the SSP Application base path):

```
1 | <FULL_PATH>/extensions/<VENDOR>/<THEME_NAME>/<VERSION>/img/logo.png
```

getThemeAssetsPathWithDefault(config_value, default_value)

Use this HTML template helper:

- To access new and pre-existing assets included in your theme directory
- In your extension overrides (templates)
- In your theme customizations (templates)
- When you expect that the asset has a configured path in NetSuite, but want to provide a default path if the configuration does not exist.

Using this helper, config_value is the path as configured in NetSuite. In cases where the asset is not configured, you provide a fallback path (default_value) to be retrieved from the extension's assets.

Example Syntax in Template File:

```
1 | 
```

Example Result:

This helper returns the active theme's asset path. If the first argument is defined, this helper returns the path as configured in NetSuite:

```
1 | <MY_PATH>/img/other_logo.png
```

If the first argument is undefined, this helper uses the second argument to return the correct path of the active theme:

```
1 | <FULL_PATH>/extensions/<VENDOR>/<THEME_NAME>/<VERSION>/img/logo.png
```

Sass Helpers

The following helpers provide a means to maintain dynamic paths to your assets when customizing Sass files:

- `getExtensionAssetsPath($asset)`
- `getThemeAssetsPath($asset)`

getExtensionAssetsPath(\$asset)

Use this Sass helper:

- To access pre-existing assets included with an active extension
- In your extension overrides (Sass)

Example Syntax in a Sass File:

```
1 | body {
2 |   background-image: url(getExtensionAssetsPath('img/background-image.png'));
3 | }
```

Example Result:

This helper returns the active extension's asset path.

```
1 | <FULL_PATH>/extensions/<VENDOR>/<EXTENSION>/<VERSION>/img/logo.png
```

getThemeAssetsPath(\$asset)

Use this Sass helper:

- To access new and pre-existing assets included in your theme directory
- In your extension overrides (Sass)
- In your theme customizations (Sass)

Example Syntax in a Sass File:

```
1 | body {
2 |   background-image: url(getThemeAssetsPath('font-awesome'));
3 | }
```

Example Result:

This helper returns the active theme's asset path.

```
1 | <FULL_PATH>/extensions/<VENDOR>/<THEME_NAME>/<VERSION>/font-awesome
```

Commerce Sass Style Definitions

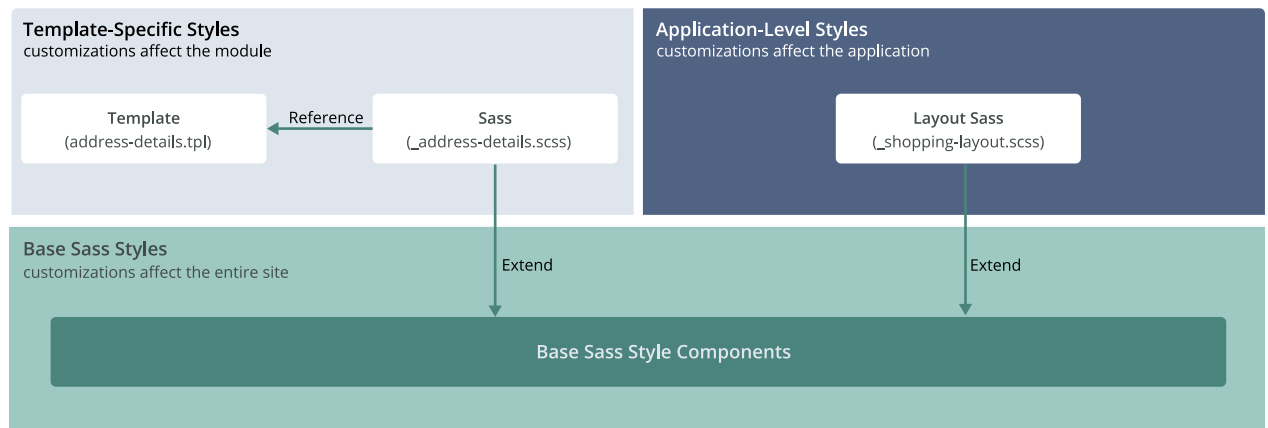
The Commerce design architecture provides an intuitive, robust, and flexible solution that lets you customize the design elements of your web store experience with ease. Commerce web stores use the Sass CSS extension language. With Sass, CSS syntax is fully supported in addition to features such as variables and imports. The following topics provide more information:

- [Design Hierarchy](#)
- [Style Definitions](#)
- [Style Guide](#)
- [Mobile First](#)

Design Hierarchy

Sass styles are organized in the following hierarchy:

- **Base Sass Styles:** These are the basic components designed to be combined and reused across the site.
Template-Specific Styles: Sass styles specific to a page or template are defined here. These are extensions and combinations of the base components.
- **Application-Level Styles:** Within the Base Sass Styles, you can define components that are referenced from a specific application: Shopping, MyAccount, or Checkout. This lets you define unique styling for each application experience.



Base Sass Styles

Base Sass styles are reusable components that can be iteratively combined to create increasingly complex structures such as templates and pages. All basic components are defined within the **BaseSassStyles** folder at `Workspace/<THEME_DIRECTORY>/Modules/BaseSassStyles@sc-<release>`. Components are categorized into three different types:

- **atoms:** basic building blocks to be used within other structures. For example, buttons, messages, and forms.
- **molecules:** simple combinations of atoms. Molecules are built for reuse but serve different purposes depending on the page context.
- **variables:** basic styling rules such as spacing and typography, that are then referenced and extended in atoms and molecules.

Template-Specific Styles

Each module contains a Sass folder that defines styling rules specific to the templates used for that module. Within these Sass files, the Base Sass Style components are used as the starting point and then extended as needed. There is generally a single Sass file which corresponds to each template file within a module.

For example, the Address module has a **address-details.tpl** file in the Templates folder of the module with a corresponding **_address-details.scss** Sass file in the Sass folder of the module. Within the **_address-details.tpl** file, the **address-details-remove-address** class is referenced. This class is defined in the **_address-details.scss** file where base classes are extended to add styles specific for this template.

Class Referenced in Template File:

```
1  {{#if showRemoveButton}}
2    <button class="address-details-remove-address" data-action="remove" data-id="{{internalid}}" {{#if isInvalidAddressToRemove}}disabled{{/if}}>
3      {{translate 'Remove'}}
4    </button>
5  {{/if}}
```

Class Defined in Sass File:

```
1  .address-details-remove-address {
2    @extend .button-edit;
3    margin-right: $sc-base-margin-lvl2;
4    cursor: pointer;
5  }
```

Application-Level Styles

In addition to defining styles specific to a module using template-specific classes, you can define styles to be specific to each application. The Shopping, My Account, and Checkout experiences can each have a unique design.

By default, for each application module, there is a single **layout** Sass file with several application specific style extensions defined:

- Workspace/<THEME_DIRECTORY>/Modules/ShoppingApplication/Sass/_shopping-layout.scss
- Workspace/<THEME_DIRECTORY>/Modules/MyAccountApplication/Sass/_myaccount-layout.scss
- Workspace/<THEME_DIRECTORY>/Modules/MyAccountApplication/Sass/_myaccount-layout-sb.scss
- Workspace/<THEME_DIRECTORY>/Modules/CheckoutApplication/Sass/_checkout-layout.scss

Also, you can define application specific dependencies for base styles or customizations in the theme manifest file. Each file must also be added as a dependency to the appropriate application in the sass object before styles defined in those files can be included when you deploy. See the help topic [Edit the Theme Manifest](#) for details.

Style Definitions

To help you maintain and develop Sass variables, Commerce websites use the following Sass style definitions. These styles provide flexible, comprehensible, and easy-to-maintain variables and their values.

Read the following information to learn how Commerce websites use these style definitions:

- [Colors](#)
- [Typography](#)
- [Spacing](#)

Colors

Colors represent a large number of variables throughout SuiteCommerce and SuiteCommerce Advanced. These are represented as primary colors, secondary colors, themes, messaging, links, and neutral shades.

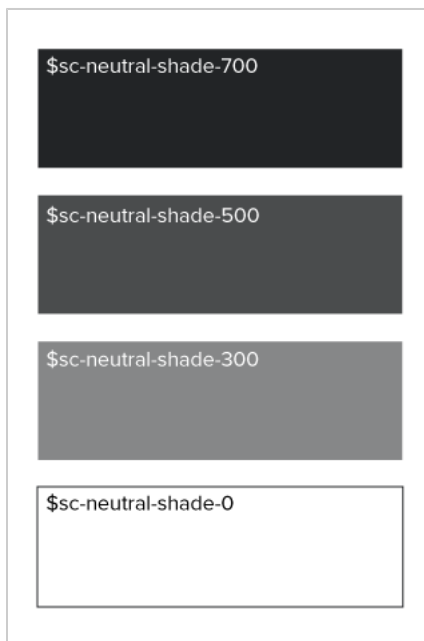
Neutral shades and theme colors are handled differently than other color definitions. Both neutral shades and theme color palettes use a shading scale from 1000 (dark) to 0 (light).

For example, the BaseSassStyle module defines neutral shades in the BaseSassStyles/Sass/variables/_colors.scss file as follows:

```

1 //...
2
3 $sc-neutral-shade: #4D5256;
4 $sc-neutral-shade-700: darken($sc-neutral-shade, 18);
5 $sc-neutral-shade-500: $sc-neutral-shade;
6 $sc-neutral-shade-300: lighten($sc-neutral-shade, 20);
7 $sc-neutral-shade-0: lighten($sc-neutral-shade, 100);
8
9 //...
```

In this example, `sc-neutral-shade` defines the base neutral value `#4D5256`, which is a very dark gray. Commerce websites declare other neutral shades as calculated percentages of the base. Therefore, `sc-neutral-shade-500` has no calculation and matches the base shade. However, `sc-neutral-shade-300` renders as the base shade lightened 20%. As we progress through each neutral shade, shades becomes lighter. You can extend the class definitions in the `_colors.scss` file to introduce intermediate shades as shown in the following image.



The theme color palette handles colors in a similar way. Instead of defining a hexadecimal value, the theme palette defines a previously declared color value. For example, `$sc-color-secondary`.

Typography

Like color variables, Commerce websites declare typography variables in an intuitive, human-readable manner. This relies on the familiar sizing method of extra-small (xs) through triple-extra-large (xxxl). Each

of these sizes is relative to a base font size (\$sc-font-size-base). The source Sass expresses the base font size in pixels and all related font sizes in rem units (relative em). 1rem equals the font size of the base element. Therefore, 1.067rem is a calculation meaning the value for \$sc-font-size-base times 1.067.

In the following example from BaseSassStyles/Sass/variables/_typography.scss, \$sc-font-size-m is equal to 15 pixels (1 x 15px), and \$sc-font-size-l is equal to about 16 pixels (1.067 x 15px). Likewise, \$sc-font-size-xxxl is equal to about 26 pixels (1.73 x 15px).

```

1 //...
2
3 $sc-font-size-base: 15px;
4
5 $sc-font-size-xxs: 0.75rem;
6 $sc-font-size-xs: 0.87rem;
7 $sc-font-size-s: 0.93rem;
8 $sc-font-size-m: 1rem;
9 $sc-font-size-l: 1.067rem;
10 $sc-font-size-xl: 1.2rem;
11 $sc-font-size-xxl: 1.47rem;
12 $sc-font-size-xxxl: 1.73rem;
13
14 //...
```

Spacing

Spacing variables affect the structural height, width, and spacing of elements. These are most commonly referred to as the padding and margins. Like color and typographic variables, spacing variables are declared in an intuitive, human-readable manner. Commerce websites use **levels** to indicate the added or reduced space for a class relative to a base style. Each level designation in the named variable equals the multiplier. That is, lv2 bares a multiplier of 2.

In the following example from BaseSassStyles/Sass/variables/_spacing.scss, \$sc-padding-base is equal to 5 pixels, and \$sc-padding-lv2 is equal to 10 pixels (2 x 5px). Likewise, \$sc-padding-lv8 equals 40 pixels (8 x 5px).

```

1 //...
2
3 $sc-padding-base: 5px;
4 $sc-padding-lv1: $sc-padding-base;
5 $sc-padding-lv2: $sc-padding-base * 2;
6 $sc-padding-lv3: $sc-padding-base * 3;
7 $sc-padding-lv4: $sc-padding-base * 4;
8 $sc-padding-lv5: $sc-padding-base * 5;
9 $sc-padding-lv6: $sc-padding-base * 6;
10 $sc-padding-lv7: $sc-padding-base * 7;
11 $sc-padding-lv8: $sc-padding-base * 8;
12
13 //...
```

Style Guide

i Applies to: SuiteCommerce Advanced



Important: This information applies to the Aconcagua and Kilimanjaro releases of SuiteCommerce Advanced (SCA) only. Recent versions of SCA do not support this functionality.

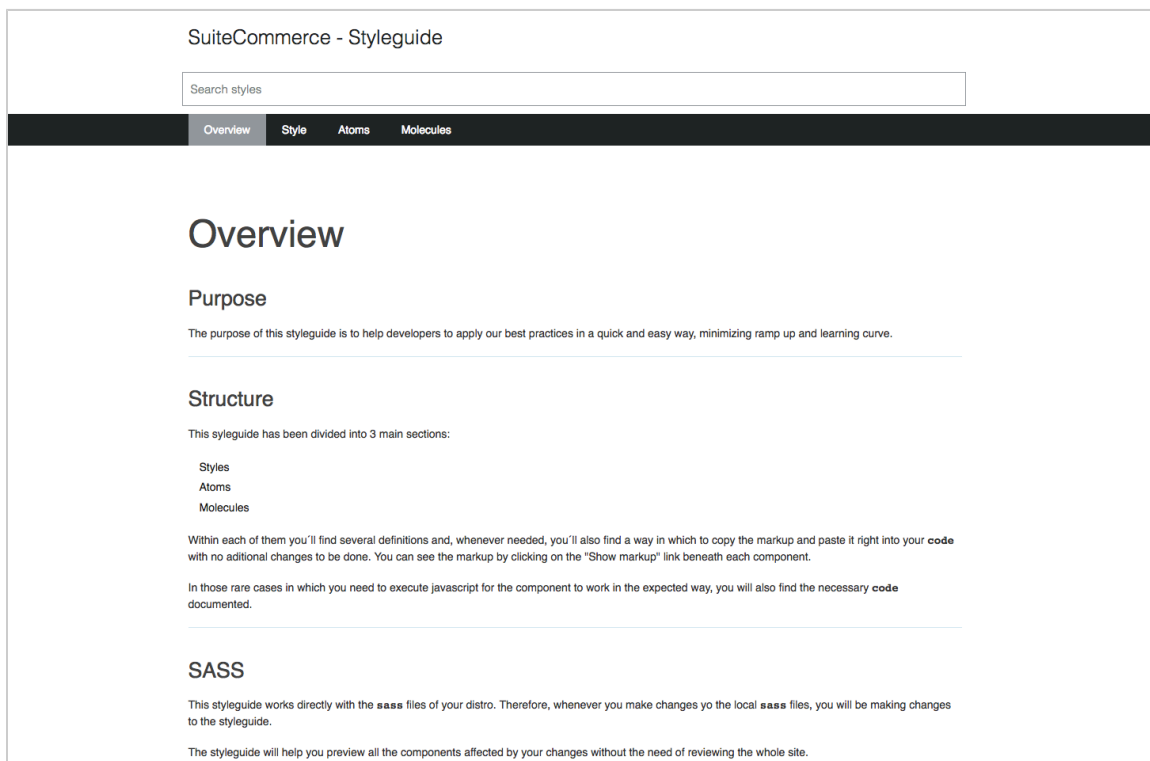
A style guide helps developers and designers use the various style elements defined for a site. Contributors can refer to a style guide to create new pages and elements or customize existing ones. A style guide can also ensure that your best practices for site design are being met by multiple contributors.

You can use the developer tools to create a style guide automatically. To do so, the source BaseSassStyles module uses KSS notation to document and define Sass elements. As a best practice, build any Sass customizations using KSS notation as defined in this section.

To create a style guide using the developer tools:

1. In your local developer environment, open a command line or terminal and access the top-level development directory.
2. Run the following command:
`gulp theme:local styleguide`
3. Point your browser to:
`localhost:3000`

You should see your style guide as shown in the following example:



KSS Notation

Commerce websites use Knyle Style Sheets (KSS) notation to automate the creation of a Sass style guide. KSS is simply a documentation syntax for CSS, written in a human-readable way. A style guide serves as a place to publish KSS documentation and visualize different states of user interface elements defined in your Sass.

Note: KSS does not affect the performance or rendering of CSS in any way. It is meant to document your styles in a human-readable way and automate the creation of a related style guide.

The following elements of this notation appear before each element defined in a Sass file:

- **Title** – defines the element. Best practice is to use an intuitive, human-readable title.
- **Description** – briefly explains the intended purpose and implementation of the element.
- **Markup** – provides the HTML markup used to generate the element.
- **Index** – defines where the element exists within the style guide structure.

The following example comes from the base Sass style atom `_buttons.scss` and highlights the basics of KSS notation as used by Commerce web stores.

```

1 //...
2
3 // Primary buttons
4 (This is an example of the Title element)
5 //
6 // The primary action is generally the action that completes a task or desired action.
7 // Primary buttons use bright and saturated colors to provide stronger visual weight that
8 catches the user's attention to avoid confusion and competition with secondary actions.
9 (This is an example of the Description element)
10 //
11 // Markup:
12 // <button class="button-primary button-medium">Button Primary</button>
13 (This is an example of the Markup element)
14 //
15 // Styleguide 2.1.1
16 (This is an example of the Index element)
17
18 .button-primary {
19   background: $sc-button-primary-background-color;
20   color: $sc-button-primary-text-color;
21   font-weight: $sc-font-weight-normal;
22   border: 1px solid $sc-button-primary-border-color;
23   border-radius: $sc-button-primary-border-radius;
24   text-transform: $sc-button-primary-text-transform;
25   display: inline-block;
26 //...

```

Mobile First

Commerce websites are developed with a Mobile First approach. Mobile First design results in faster, sleeker sites while enabling easier design. Developing sites with the Mobile First approach promotes using only the most important elements on your screen and thereby avoids cluttering the screen with secondary elements not critical to the current task. This approach enhances the user experience by allowing users to quickly find and finish the task at hand whether they are looking for a product to purchase in a B2C environment or reordering in bulk in a B2B environment.

Using the Mobile First approach, element styles are defined for mobile devices and then scaled progressively for tablet and desktop display.

For example, in the following code sample the `.button-medium` style is defined to be `$sc-button-medium-mobile-width`. And then the width is adjusted when the media is of `$screen-sm-min` size (tablet). The **default** media sizes are all defined in Sass variable files.

```

1 .button-medium {
2   width: $sc-button-medium-mobile-width;
3   padding: $sc-button-medium-padding-v $sc-button-medium-padding-h;
4   font-size: $sc-button-medium-font-size;
5   line-height: $sc-button-medium-line-height;
6   letter-spacing: $sc-button-medium-letter-spacing;
7   text-align: center;
8
9   @media(min-width: $screen-sm-min) {
10    width: $sc-button-medium-desktop-width;

```

```
11 | }  
12 | }
```

Theme Tutorials

The following tutorials describe how to work with themes:

- [Theme Tutorial Prerequisites](#)
- [Customize an Existing Theme](#)
- [Build a New Theme](#)

Theme Tutorial Prerequisites

Before you start the following tutorials, complete the following tasks:

- Install and configure SuiteCommerce or SuiteCommerce Advanced (Aconcagua and later versions) as described in [Install Your Commerce Website Applications](#).
- [Install Node.js](#)
- [Install Gulp.js](#)
- [Set Up Theme Developer Tools](#)
- Understand the [Anatomy of the Base Theme](#)
- Install and activate the SuiteCommerce Base Theme as described in [Use the SuiteCommerce Base Theme](#)
- Ensure Site Management Tools (SMT) are enabled as described in the SMT [Site Management Tools Installation and Implementation](#) topic
- Familiarize yourself with the [Best Practices for Creating Themes](#)

Customize an Existing Theme

The following tutorial describes how to customize assets in the SuiteCommerce Base Theme. If you have a custom theme that you want to revise, the steps in this tutorial still apply but some of the customization details will be different.

 **Note:** Before starting the following procedure, complete the list of tasks in [Theme Tutorial Prerequisites](#).

To customize an existing theme:

1. From a command line, go to your top-level theme development directory. The top-level theme development directory is the directory where you set up the theme developer tools, for example: ThemeDevelopmentTools-20.2.1.
2. Run the following command to fetch the active theme. See the help topic [Fetch Active Theme Files](#) for more information.

 **Note:** Remember that this command overwrites the files in your workspace so be sure you don't need any of the existing files.

```
1 | gulp theme:fetch
```

When the fetch is complete, your local workspace directory should look like this example:

```
1 Workspace/
2   SuiteCommerceBaseTheme/
3     assets/
4     Modules/
5     Overrides/
6     Skins
7     manifest.json
```

Note: The following steps assume that you fetch and are working with the SuiteCommerce Base Theme. But, you can also apply the changes in these steps to a different theme.

3. Open the theme directory in your top-level development directory.

For this example:

Workspace/SuiteCommerceBaseTheme/

4. Locate the subdirectory containing the file or files you want to customize.

In this example, you want to customize the top margin of the **Add to Cart** button for this theme. You locate the `_cart-add-to-cart-button.scss` file in the theme's Cart module:

Workspace/<THEME_DIRECTORY>/Modules/Cart/Sass/_cart-add-to-cart-button.scss

5. Edit the `_cart-add-to-cart-button.scss` Sass file.

You can edit this file directly. See [Best Practices for Creating Themes](#) for details. In this example, you edit the `.cart-add-to-cart-button-button` class and change the `margin-top` value from `lv3` to `lv5`.

```
1 //...
2 // New code with new margin-top value:
3 .cart-add-to-cart-button-button{
4   @extend .button-primary;
5   @extend .button-large;
6   width: 100%;
7   font-weight: $sc-font-weight-semibold;
8   margin-top: $sc-margin-lv5;
9 }
10
11 /* Previous code:
12 .cart-add-to-cart-button-button{
13   @extend .button-primary;
14   @extend .button-large;
15   width: 100%;
16   font-weight: $sc-font-weight-semibold;
17   margin-top: $sc-margin-lv3;
18 }*/
19 //...
```

6. Save the file.
7. Repeat this procedure for any other theme-related HTML or Sass files you need to customize.
8. Test your theme customizations locally by running the `gulp theme:local` command as described in [Test a Theme on a Local Server](#).

Do More With Themes

When developing themes, you can do more than make changes to the look and feel of your Commerce website. Read the following topics to learn how to:

- [Override Active Extension Files](#)
- [Enable Customization from Site Management Tools](#)
- [Customize Site Search Elements](#)
- [Include Translations in Themes](#)

Override Active Extension Files

When creating a theme, you can customize the HTML and Sass of any active extensions for a domain and deploy them with your theme customizations.

When you run the `gulp theme:fetch` command, you download the theme Sass, HTML, and asset files. You also download all extension-related HTML and Sass files. The developer tools add these in the appropriate folder within the `Workspace/Extras/extension` directory.

This section explains how to use the override method to customize HTML and Sass files related to extensions.



Important: Extension-related files placed in the Extras directory are provided as a reference only. Never edit these files directly. Instead, use the override method described in this section.

Example

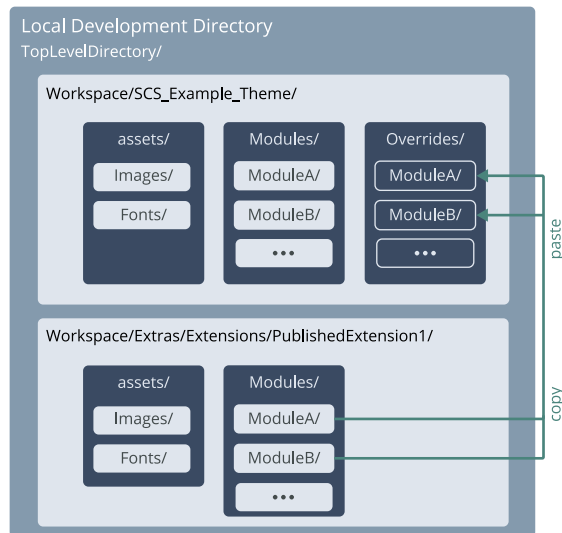
You activate a theme for your domain. You also activate an extension for the same domain. You create your theme to include a new color variable. You want the extension that is active with your theme to use this same variable. However, the extension's Sass does not include the new variable. You must customize the extension's Sass files to include this color variable using the override method. This ensures that the active extension includes your Sass variable, without changing the extension's code. Instead, you override it.

All of your theme customizations and extension overrides are maintained as part of your theme. Therefore, when you later activate your theme and the extension for your domain, the active extension includes your new color variable. This does not change the extension. Instead, this change belongs to the theme.

The Override Method

To customize extension-related HTML and Sass files, you must use the override method. This involves placing copies of source HTML and Sass files from your Extras directory into your theme directory and making your customizations there. When you deploy your changes, the theme development tools detect the new files and initiate an override within the manifest.

Note: When you download an extension's HTML and Sass files, the theme developer tools place them in your Extras/Extensions directory, sorted by extension. These are for reference only. At the same time, the developer tools create an identical directory structure in your theme's Overrides directory.



Important: You cannot override asset files. To introduce new assets for your template customizations or extension overrides, add them as you would for any new files within the template directory, then reference the assets using the HTML and Sass helpers. For more information, see [Add a New File to a Theme](#) and [Managing Theme Assets](#).

To customize an extension using the override method:

1. Locate the source file you want to override.
For example, if you want to override a file named `_Pub-Extend-Error.scss` in **Published Extension 1**, the file you want to override is in the following location:

`Workspace/Extras/Extensions/PublishedExtension1/Modules/PubExtendModule@1.0.0/Sass/_Pub-Extend-Error.scss`

2. Copy the source file to your operating system's clipboard.
3. Paste a copy of the file in the corresponding location within the theme's Overrides directory. For example:

`Workspace/Theme/Overrides/Modules/PubExtendModule@1.0.0/Sass/_Pub-Extend-Error.scss`

Important: Do not rename these files. These files must have the same name for the override method to function correctly.

4. Open your new file in your Overrides directory.
5. Follow best practices to customize your new file. See [Best Practices for Creating Themes](#) for important details on customizing these files.
6. Repeat this procedure for all extension-related files you need to customize.

When you are ready, use the theme developer tools to test or deploy your theme. See [Test and Deploy Your Theme](#) for procedures on how to use these tools.

Examples of Customizing Extensions Using the Override Method

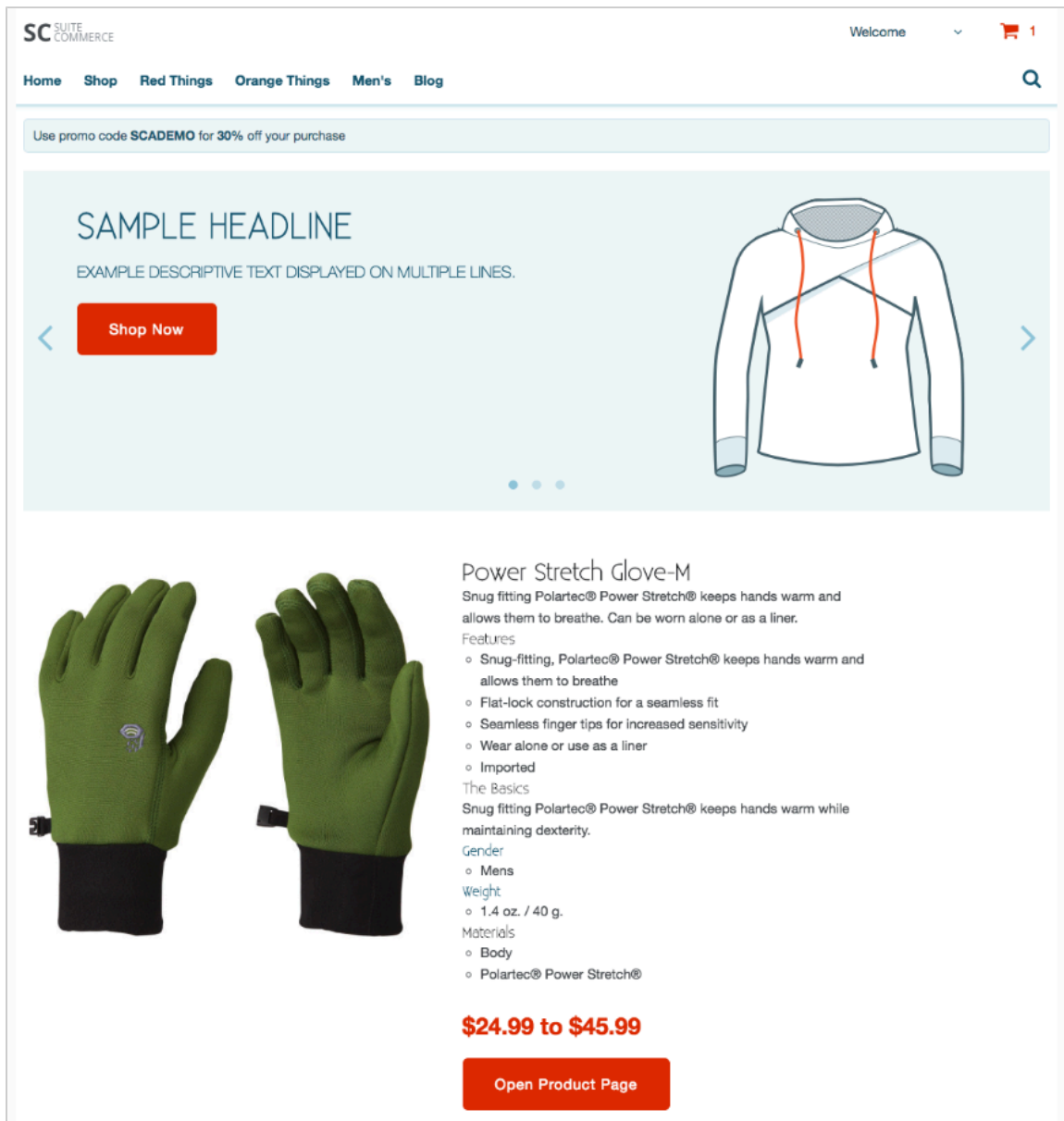
This section details how to use the override method to replace the template and Sass files and get the styling you want for the Featured Product extension and the Featured Category extension. The focus is on these two extensions because they are a great way of adding content to your site. The Featured Product extension lets you highlight a product on your website and the Featured Category extension lets you highlight a commerce category on your website. For more information about these extensions, see the help topics [Featured Product](#) and [Featured Category](#). Each of these extensions are Custom Content Types (CCTs), managed through Site Management Tools (SMT). For more information, see the help topics [Site Management Tools Custom Content Type](#) and [Site Management Tools](#).

To learn more, see:

- [Overriding the Featured Product Extension](#)
- [Overriding the Featured Category Extension](#)

Overriding the Featured Product Extension

The Featured Product extension lets you select a product to display prominently in a CMS area. Standard configurable options include whether to display a products price and availability, adding text over an image, adding button text, and displaying an alternative image. For complex changes, you must edit the template and Sass files. The following image shows the default styling. Note that depending on your site's data, your results may appear differently.



To customize the Featured Product extension using the override method:

1. Go to Workspace/Extras/Extensions/SuiteCommerce/FeaturedProduct/Modules/FeaturedProductCCT/Sass/_featuredproductcct.scss.
2. Copy the source file to your operating system's clipboard.
3. Paste a copy of the file in the corresponding location within the theme's Overrides directory. For example, Workspace/Theme/Overrides/SuiteCommerce/FeaturedProduct/Modules/FeaturedProductCCT/Sass/_featuredproductcct.scss.



Important: Do not rename these files. These files must have the same name for the override method to function correctly.

4. Open your new Sass file from your Overrides directory.
5. Run your local server using `gulp theme:local`.

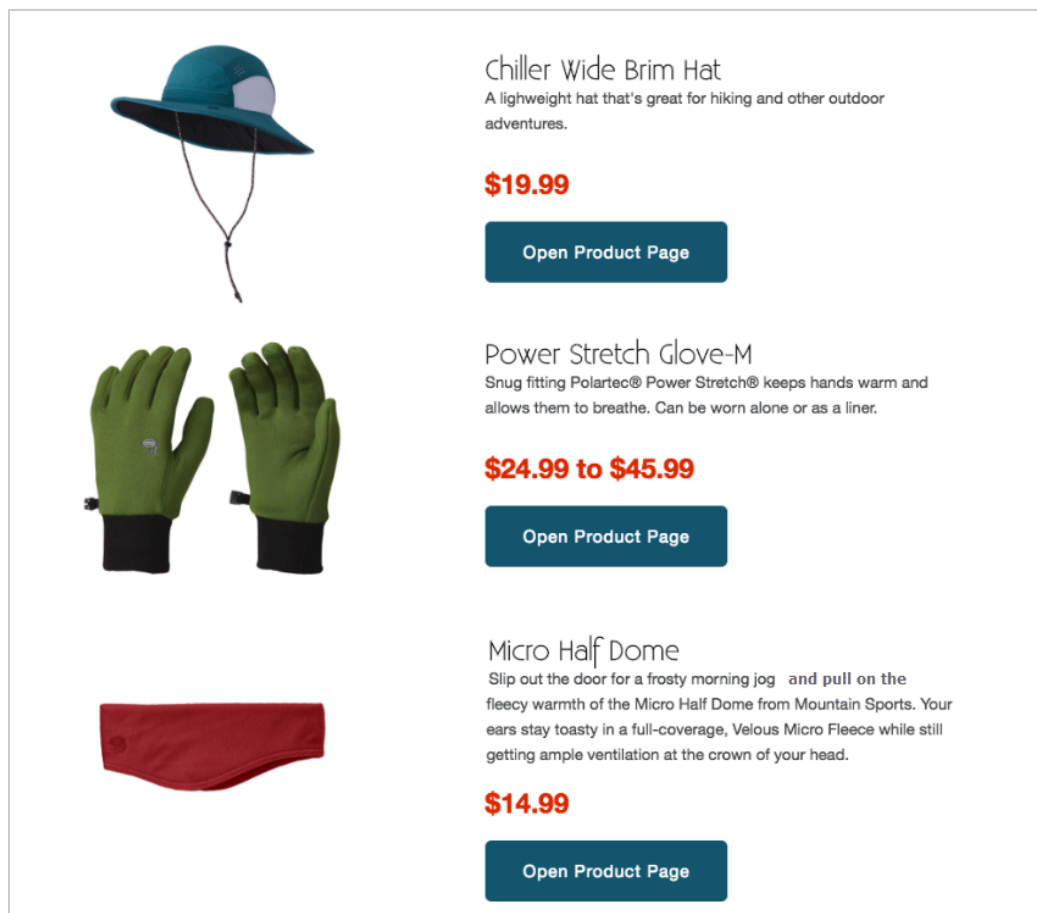
Note: You must place the files in the correct locations before starting the local server, or they will not be watched for changes. You can find which overrides are being watched by looking at the log in your CLI.

- With your server up and running, add the styles you want to the copy of your Sass file. To get you started, the following code shows you how to make the image smaller and how to change the default product details to display a shorter description.

```

1 .home-page
2 .featureproductcct-layout {
3   .pdp-features, .pdp-basics, .pdg-materials {
4     display: none;
5   }
6
7   .product-image {
8     max-height: 250px;
9   }
10 }
```

The following image illustrates how the blog post appears after making these customizations. It also includes additional featured products and a different button style.



- When you are ready, use the theme developer tools to test or deploy your theme. See [Test and Deploy Your Theme](#) for procedures on how to use these tools.

Note: Each instance of a featured product creates an HTTP call to the item's API. To avoid performance issues, do not use this method to add a large set of featured products. Instead, you can use the Featured Category extension. For details, see the help topics [Featured Category](#) and [Overriding the Featured Category Extension](#).

Overriding the Featured Category Extension

The Featured Category extension lets you highlight a commerce category on your website. Clicking on an item takes users to the Product Detail page. This extension is useful when you want to display several items. It reduces the number of API calls to one per category, and gives you control over the items and presentation. The following image depicts the initial configuration. Two lines of items are displayed before linking to the Category Landing page. Standard configurable options include setting the title and button text. You can use the override method to customize how it appears on your site.

To customize the Featured Category extension using the override method:

1. Go to Workspace/Extras/Extensions/NetSuite/FeaturedCategory/Modules/Item/Templates/sc_featuredcategory_item.tpl.
2. Copy the source file to your operating system's clipboard.
3. Paste a copy of the file in the corresponding location within the theme's Overrides directory. For example, Workspace/Theme/Overrides/NetSuite/FeaturedCategory/Modules/Item/Templates/sc_featuredcategory_item.tpl.



Important: Do not rename these files. These files must have the same name for the override method to function correctly.

4. Open your new template file from your Overrides directory.
5. Run your local server using `gulp theme:local`.



Note: You must place the files in the correct locations before starting the local server, or they will not be watched for changes. You can find which overrides are being watched by looking at the log in your CLI.

6. With your server up and running, add the styles you want to the copy of your template file. The following code shows you how to center and enlarge the item price. To remove the item's name, you can hide the line that renders the item's name or remove it entirely.

```
1 .home-page .featurecategorycct-item-price {
2   text-align: center;
3   font-size: $sc-font-size-xl;
4 }
```

The following image illustrates how the blog post appears after making these customizations.

7. When you are ready, use the theme developer tools to test or deploy your theme. See [Test and Deploy Your Theme](#) for procedures on how to use these tools.

Enable Customization from Site Management Tools

Read the following topics to learn how to enable site administrators to customize a Commerce website's look and feel using Site Management Tools (SMT). As a theme developer, you can:

- Expose variables to enable site administrators to customize specific aspects of a site, as described in [Expose Sass Variables for Customization](#).
- Make exposed variables more meaningful to site administrators, as described in [Organize Variables for Display in SMT](#).
- Create preset JSON files to expose a selectable set of predefined settings to site administrators, as outlined in [Create Skins](#).

Before reading the topics in this section, familiarize yourself with the best practices outlined in [Sass Best Practices for Themes](#).

Note: SMT Version 3 includes the Theme Skin Manager. The Theme Skin Manager lets SMT administrators customize styles and layouts of an active theme based on the settings that you define during theme development. SMT administrators can apply changes to individual settings or apply skins to change multiple settings at one time.

Expose Sass Variables for Customization

Applies to: SuiteCommerce | SuiteCommerce Advanced

You control which variables your theme exposes to site administrators by introducing metadata into your Sass files. Expose variables using the `editable()` function, as described in this section.

To expose a variable for customization:

1. Open the Sass file containing the variable you want to expose.
2. Create an inline comment (`//`) or a block comment (`/* ... */`) immediately following the variable declaration.

You can use either comment method. For example:

```
1 | $sc-primary-color: red; //
```

```
1 | $sc-primary-color: red; /* */
```

3. Use the `editable()` function within your comment tags to declare the variable as **editable**.

See [The editable\(\) Function](#) for details. This function is required.

4. Save the file.

If you are creating a new Sass file, you must declare the file within the theme's manifest file and in the appropriate application entry point.

For example, you can define and declare a Sass variable that works in Checkout only. You add the containing Sass file to the theme manifest and the Checkout entry point. Later, when viewing your theme in the SMT Theme Skin Manager, your variable only appears in the SMT user interface when navigating to Checkout. If you want your variable to appear in multiple applications, you must declare the new file within each. Limiting global definitions to the BaseSassStyles module helps ensure functionality of global variables.

See [Add a New File to a Theme](#) for details.

5. Repeat this procedure for every variable you want to expose.

When ready, use the theme developer tools to test or deploy your theme. See [Test and Deploy Your Theme](#) for procedures on how to use these tools. Note that you must also activate your theme to make these changes available for SMT. See the help topic [Manage Themes and Extensions](#) for details.

The editable() Function

The metadata to expose a variable takes the form of an `editable()` function call wrapped inside a comment in your Sass files. This function call accepts a JSON object as its single argument with the following properties:

- `type` – declares the variable type. This property is required.

Possible values include:

- `color` – displays a color picker in SMT.
- `string` – declares a value as a string (font-weight, font-style, thumbnail size, etc.).



Important: SMT only supports `color` and `string` as values for the `type` property. To declare a number value or a font, use a string.

- `label` – provides a formatted string to reference the variable in the SMT user interface. If not defined, SMT displays the Sass variable name as the label.
- `description` – describes the variable in a long-formatted string. This property is optional.
- `options` – provides a list of options as an array for the variable. Use this property to limit possible values using a dropdown list. See [Limit Selections](#). This property is optional.

The following code snippet is an example of how to expose a color variable using inline notation:

```
1 | $sc-primary-color: red; // editable({"type": "color", "label": "Primary Color"})
```

You can also use the `editable()` function with block notation:

```
1 | $sc-primary-color: red; /* editable({
2 |   "type": "color",
3 |   "label": "Primary Color",
4 |   "description": "Used mainly for 'call to action' elements."
5 | })*/
```



Important: The comment must start immediately after the variable declaration. This is essential for the Sass preprocessor to relate the variable name with the declared metadata in the comment.

Limit Selections

You can use the `options` property of the `editable()` function to specify one or more values within a dropdown list in SMT.

For example, you can declare a list of available font weights as selections within a list instead of requiring SMT administrators to manually enter a numeric value as a string.

In this example, each option's `value` property is the value to be processed for `$base-font-weight`, and the `text` property is the corresponding option as viewed in the dropdown list:

```
1 | $base-font-weight: 200; /* editable({
2 |   "type": "string",
3 |   "label": "Font Weight",
4 |   "options": [
5 |     {"value": 200, "text": "light"},
6 |     {"value": 400, "text": "normal"},
7 |     {"value": 800, "text": "bold"}
8 |   ]
9 | })*/
```

```
8 | }) */
```

You can also apply this option to colors. Using the `option` property on a color requires the SMT administrator to choose from a list of possible colors instead of using a color picker. Note that the option value must be a string in this case.

The following example displays a list of color options:

```
1 | $feedback-color: white ; /* editable({
2 |   "type": "color",
3 |   "label": "Feedback Color",
4 |   "options": [
5 |     {"value": "white", "text": "normal"},
6 |     {"value": "black", "text": "contrast"},
7 |     {"value": "#dedede", "text": "low contrast"}
8 |   ]
9 | }) */
```

Organize Variables for Display in SMT

This function lets you declare groups and organize your Sass variables in an intuitive way within the SMT side panel.

When you expose multiple variables in SMT, you should group those variables to make them more meaningful and intuitive to site administrators. Use the `group()` function to create group metadata in any Sass files within your theme as described in the following procedure.

To group variables in the SMT side panel:

1. Open the Sass file that contains the exposed variable you want to group.
2. Use the `group()` function within the comment tags to create an organizational scheme.
See [The `group\(\)` Function](#) for details.
3. Save the file.
If you are creating a new Sass file, you must declare the file within the manifest file and the appropriate application entry point. See [Add a New File to a Theme](#) for details.
4. Repeat this for every new variable you want to expose for your theme.

When you are ready, use the theme developer tools to test or deploy your theme. See [Test and Deploy Your Theme](#) for procedures on how to use these tools. Note that you must also activate your theme to make these changes available for SMT. See the help topic [Manage Themes and Extensions](#) for details.

The `group()` Function

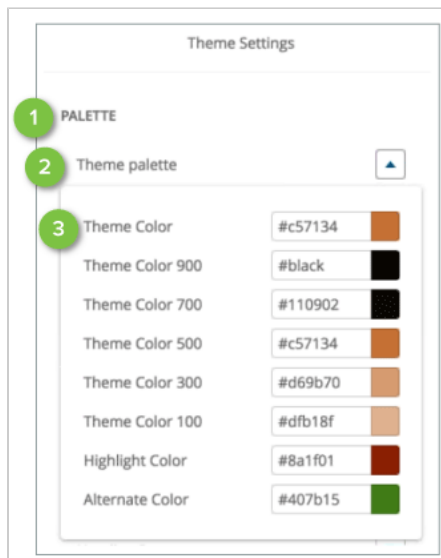
You introduce the `group()` function within comments, as you do with the `editable()` function. However, the `group()` function does not need to be located immediately after a variable declaration within a Sass file.

Note: Functionally, you can add the `group()` function at any point within your theme's Sass files. However, as a best practice, introduce `group()` in the same file that contains the associated variable.

Each `group()` function call accepts a JSON object as its single argument with the following properties:

- `id` – declares the group ID. This is used to define and reference the group.
- `label` – provides a formatted string for the group to be shown in the SMT Theme Skin Manager.
- `description` – describes the group in a long-formatted string. This property is optional.
- `children` – provides an array of variable names or group IDs (for a nested subgroup). If declaring a variable, you must precede the variable with `$`. The SMT user interface mirrors the order of any children declared here.

Note: Any variables or subgroups listed here do not need to be defined before the group definition to be referenced as a group of children.



1. Parent (Group)
2. Child (Subgroup)
3. Child (Variable)

A top-level (parent) group displays as a heading in the SMT side panel, like **PALETTE** in the preceding example. If you add another subgroup, nested children display as expandable/collapsible subgroups, like **Theme palette** in the preceding example. Children of a subgroup are variables exposed for editing. SMT only supports two group levels.

Example

The following code declares one parent group called `Palette-color` and four children to create subgroups. Each of these children is defined later in the code. Any children of the subgroups are declared variables.

```

1  /*
2  group({
3    "id": "palette-color",
4    "label": "Palette",
5    "description": "",
6    "children": [
7      "theme-palette",
8      "neutral-shades",

```



```

9      "text-colors",
10     "container-colors"
11   ]
12 })
13 */
14
15 /*
16 group({
17   "id": "theme-palette",
18   "label": "Theme palette",
19   "description": "",
20   "children": [
21     "$sc-color-theme",
22     "$sc-color-theme-900",
23     "$sc-color-theme-700",
24     "$sc-color-theme-500",
25     "$sc-color-theme-300",
26     "$sc-color-theme-100",
27     "$sc-color-primary",
28     "$sc-color-secondary"
29   ]
30 })
31 */
32
33 /*
34 group({
35   "id": "neutral-shades",
36   "label": "Neutral palette",
37   "description": "",
38   "children": [
39     "$sc-neutral-shade",
40     "$sc-neutral-shade-700",
41     "$sc-neutral-shade-500",
42     "$sc-neutral-shade-300",
43     "$sc-neutral-shade-0"
44   ]
45 })
46 */
47
48 /*
49 group({
50   "id": "text-colors",
51   "label": "Text",
52   "description": "",
53   "children": [
54     "$sc-color-copy",
55     "$sc-color-copy-dark",
56     "$sc-color-copy-light",
57     "$sc-color-link",
58     "$sc-color-link-hover",
59     "$sc-color-link-active"
60   ]
61 })
62 */
63
64 /*
65 group({
66   "id": "container-colors",
67   "label": "Containers",
68   "description": "",
69   "children": [
70     "$sc-color-body-background",
71     "$sc-color-theme-background",
72     "$sc-color-theme-border",
73     "$sc-color-theme-border-light"
74   ]
75 })
76 */

```

The preceding code, after it is deployed and activated for a domain, displays in the SMT side panel as depicted in the following images. The first image shows the parent group, **Palette**, and the four nested subgroups: **Theme palette**, **Neutral shades**, **Text colors**, and **Containers**.

The screenshot shows a 'Theme Settings' dialog box. Under the 'PALETTE' section, there are four subgroups, each with a downward-pointing arrow icon to its right, indicating they are expandable:

- Theme palette
- Neutral shades
- Text colors
- Containers

Children of the parent group are the nested subgroups. Children of each subgroup must be variable names because SMT does not support more than two group levels. These appear as expandable/collapsible groups. SMT administrators can now edit the settings as they please.

The screenshot shows the 'Theme Settings' dialog box with the 'Theme palette' subgroup expanded. It displays a list of color variables, their hex codes, and color swatches:

Variable Name	Hex Code	Color Swatch
Theme Color	#c57134	Orange
Theme Color 900	#black	Black
Theme Color 700	#110902	Dark Brown
Theme Color 500	#c57134	Orange
Theme Color 300	#d69b70	Light Orange
Theme Color 100	#dfb18f	Light Orange
Highlight Color	#8a1f01	Dark Red
Alternate Color	#407b15	Green

Note: The architecture style definitions used to maintain the Sass structures use derived/calculated values. By exposing derived colors, as in this example, any UI selections of the base value automatically results in changes of any derived values. SMT administrators can further edit each derived value within the UI as well, assuming those variables are exposed for editing. See [Style Definitions](#) for more information about how these Sass structures work.

Create Skins

What is a Skin?

Generally speaking, a skin is a selectable set of predefined settings that change the appearance of a theme. One theme can include any number of skins that SMT administrators select to change that

theme's appearance. For example, a theme might include multiple skins with a different color scheme for specific seasons.

- **Theme** – a ready-made package that can be activated for any number of domains. A theme can include any number of skins.
- **Skin** – a set of predefined variables that lets SMT administrators alter a theme in a specific way.

From a development perspective, you define a skin using a JSON file. Each file contains the variables and corresponding values that change when the skin is applied using SMT. You create one file in the theme's Skins directory for each skin you want to make available. When you run the `gulp theme:local` or `gulp theme:deploy` command, the developer tools update the theme's manifest file, automatically adding a skin for each file located in the Skins directory.

When you deploy, and later activate, the theme for a domain, your presets appear as selectable skins in the SMT user interface. When SMT administrators apply your skin, the application compiles the values, and the theme incorporates the changes.



Important: If you apply a skin, then make changes to one or more variables, you will see your changes in the theme preview. However, if you then select a new skin, your changes to the individual variables will be lost.

Important Information

Be aware of the following information when creating skins:

- When developing your skin, include only variables that are exposed for customization. If your skin includes variables that are not editable, they will not change when the skin is applied. See [Expose Sass Variables for Customization](#) for details.
- A theme can include any number of skins or no skins at all. If you do not create any skin preset files, SMT does not provide the option to select a skin.
- Sass variables exposed to SMT are available for customization at any time using the SMT Theme Skin Manager. A theme does not need to include skins for exposed variables to be editable. Likewise, after an SMT administrator selects a skin, they can make their own changes to individual variables.
- When SMT administrators apply changes to exposed variables, those changes apply to the domain, but they do not overwrite the predefined skin or the theme's Sass files. The only way to permanently change a Sass variable or save changes to a skin is to edit the theme's development files or create a new theme using the developer tools.

The Skin Preset File

To create a skin, you need to build a JSON preset file. The JSON preset file contains the variables that you want your skin to change. Each variable listed in the preset file includes the new value to use when the skin is applied.

To create a skin preset file:

1. Expose all Sass variables you want your skin to override. See [Expose Sass Variables for Customization](#) for instructions.
2. Go to your theme's Skins directory.

The Skins directory is located in your theme development directory. For example:

```
1 | <Top-LevelDevelopmentDirectory>/
2 |   Workspace/
3 |     <Theme_Directory>/
4 |       Skins/
```

3. Create a new JSON file.

Name the file in an intuitive manner. Do not include spaces or special characters.

For example:

```
../Workspace/<Theme_Directory>/Skins/winter_skin.json
```

4. Open the JSON file, define the Sass variables you want your skin to change, and include the values your skin changes.

For example, your winter skin might look like the following:

```
1 {
2   "$sc-color-body-background": "white"
3   ,
4   "$sc-button-primary-text-color": "white"
5   ,
6   "$sc-button-primary-hover-text-color": "white"
7   ,
8   "$sc-button-primary-active-text-color": "white"
9   ,
10  "$sc-button-primary-text-transform": "none"
11  ,
12  "$sc-button-secondary-text-color": "white"
13  ,
14  "$sc-button-secondary-hover-text-color": "white"
15  ,
16  "$sc-button-secondary-active-text-color": "white"
17  ,
18  "$sc-button-secondary-text-transform": "none"
19  ,
20  "$sc-button-tertiary-hover-text-color": "white"
21  ,
22  "$sc-button-tertiary-active-text-color": "white"
23  ,
24  "$sc-button-tertiary-text-transform": "none"
25  ,
26  "$sc-button-large-line-height": "1"
27  ,
28  "$sc-button-large-letter-spacing": "1px"
29  ,
30  "$sc-button-large-mobile-width": "100%"
31  ,
32  "$sc-button-large-desktop-width": "auto"
33  ,
34  "$sc-button-medium-line-height": "1"
35  ,
36  "$sc-button-medium-letter-spacing": "0.5px"
37  ,
38  "$sc-button-medium-mobile-width": "100%"
39  ,
40  "$sc-button-medium-desktop-width": "auto"
41  ,
42  "$sc-button-small-line-height": "1"
43  ,
44  "$sc-button-small-letter-spacing": "normal"
45  ,
46  "$sc-button-small-mobile-width": "auto"
47  ,
48  "$sc-button-small-desktop-width": "auto"
49  ,
50  "$sc-body-line-height": "1.6"
51  ,
52  "$sc-h1-line-height": "1.2"
53  ,
54  "$sc-h2-line-height": "1.2"
55  ,
56  "$sc-h3-letter-spacing": "0"
57  ,
58  "$sc-h4-line-height": "1.4"
59  ,
60  "$sc-h5-line-height": "1.4"
61  ,
62  "$sc-h6-line-height": "1.4"
63  ,
64  "$sc-blockquote-line-height": "1.6"
65  ,
66  "$sc-color-primary": "#e23200"
67  ,
68  "$sc-color-secondary": "#15607b"
69  ,
70  "$sc-neutral-shade": "#4d5256"
71  ,
72  "$sc-color-theme": "#97CCDF"
73  ,
74  "$sc-color-link": "#0067b9"
75  ,
76  "$sc-color-link-active": "#0067b9"
77 }
```



Important: When creating a skin preset file, include only variables that are exposed for editing within your theme's Sass files. If you include a variable that is not exposed, that value does not change when the skin is applied to the theme. See [Expose Sass Variables for Customization](#).

5. Save the JSON file.
6. Include your new skin in the extension manifest file. See [Add a Skin to the Theme Manifest](#).

Add a Skin to the Theme Manifest

For a skin to appear in the SMT Theme Skin Manager, it must be declared in the theme's manifest. The developer tools do this automatically when you run either `gulp theme:local` or `gulp theme:deploy`.

Note: You can also manually edit the manifest file, following the example in this section. The theme's manifest.json file is located at `../Workspace/<Theme_Directory>/manifest.json`.

To update the theme manifest file:

1. Perform one of the following Gulp commands:
 - `gulp theme:local` – the developer tools update the manifest and start the local server.
 - `gulp theme:deploy` – the developer tools update the manifest and deploy the theme to NetSuite.

Note: Running `gulp theme:local` updates the manifest, but it does not deploy anything to NetSuite servers. You must deploy your theme files to NetSuite to view your skins in the SMT Theme Customizer.

2. If necessary, you can edit the manifest.json file to provide a more visually appealing value for the name property.

Any edits you make to the manifest.json file do not apply to your domain until you redeploy your theme.

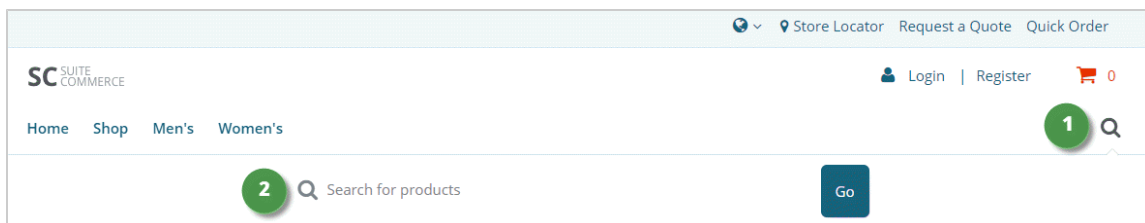
When you are ready, use the theme developer tools to deploy your theme. See [Test and Deploy Your Theme](#) for more information. Note that you must also activate your theme to make these changes available for SMT. See the help topic [Manage Themes and Extensions](#) for details.

See the help topic [Edit the Theme Manifest](#) for more information about this file.

Customize Site Search Elements

This topic applies to SuiteCommerce and the 2018.2 release of SuiteCommerce Advanced only.

Any themes created using these implementations include the ability to customize the Site Search button and input bar. You can customize your theme to locate the Site Search button (magnifying glass button) and input bar in any location within any template or, independently, within different templates.



1. Site Search button
2. Site Search input bar and Go button

Site Search Button

The Site Search button is visible by default in the SuiteCommerce Base Theme. Site users must click this button to activate the Site Search input bar. You can locate this button in any template for desktop or mobile use with the following code:

Note: The code displayed in this topic is backward compatible with Commerce themes created prior to the 2018.2 release.

Desktop

```
1 | <div class="header-menu-search" data-view="SiteSearch.Button"></div>
```

- **Base Theme Module:** Header
- **Base Theme Template:** header_menu.tpl

Mobile

```
1 | <div class="header-menu-searchmobile" data-view="SiteSearch.Button"></div>
```

- **Base Theme Module:** Header
- **Base Theme Template:** header.tpl

Site Search Input Bar

Site users rely on this input bar to search a site. You can locate the Site Search input bar and Go button in any template for desktop or mobile use with the following code:

Site Search

```
1 | <div class="site-search" data-type="site-search">
2 |   <div class="site-search-content">
3 |     <form class="site-search-content-form" method="GET" action="/search" data-action="search">
4 |       <div class="site-search-content-input">
5 |         <div data-view="ItemsSeacher"></div>
6 |         <i class="site-search-input-icon"></i>
7 |         <a class="site-search-input-reset" data-type="search-reset"><i class="site-search-input-reset-icon"></i></a>
8 |       </div>
9 |       <button class="site-search-button-submit" type="submit">{{translate 'Go'}}</button>
10 |     </form>
11 |   </div>
12 | </div>
```

- **Base Theme Module:** SiteSearch
- **Base Theme Template:** site_search.tpl

The Site Search input bar is hidden by default in the SuiteCommerce Base Theme. However, you can customize your templates to make this input bar visible by default. To do so, add the active class to site-search as shown in the following example:

```
1 | <div class="site-search active" data-type="site-search">
```

Go Button

```
1 | <button class="site-search-button-link" data-action="show-itemsearcher" title="{{translate 'Search'}}">
2 |   <i class="site-search-button-icon"></i>
```

```
3 | </button>
```

- **Base Theme Module:** SiteSearch
- **Base Theme Template:** site_search_button.tpl

Include Translations in Themes

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

SuiteCommerce and SuiteCommerce Advanced enable you to provide translations for the templates and text that your extension or theme adds or changes in a site. You can also include translations for any customer-facing text in the Shopping, MyAccount, or Checkout application that is associated with your customization.

Note: Currently, translations included in extensions and themes are not supported in local environments. Due to this limitation, translations that you add to extensions and themes cannot be tested locally.

Before You Begin

Be aware of the following important information:

- Be sure your site is properly configured to support translations. For more information, see the help topic [Set Up NetSuite for Localized Content](#).
- You should have a Workspace directory for your theme or extension. The developer tools create a Workspace directory and a manifest.json file for your theme or extension. For more information, see the help topics [Fetch Active Theme Files](#) and [Create a Baseline Extension](#).
- You should understand how to edit the manifest.json file for your theme or extension. For more information, see [Edit the Theme Manifest](#) and [Edit the Extension Manifest](#).

To include translations in your theme:

1. Manually create a directory named Translations in your theme or extension's Workspace directory as shown in the following example.

Note: You can include translations in themes or extensions and so the steps in this procedure provide information for both.

2. In the Translations directory, include a .json file for each language you want to translate.

You can name the .json file for each language however you like. Note that you must use the same file name when you add the translation file to the manifest.json as described in the follow steps.

```
1 | Workspace/
2 |   MyCoolExtension/
3 |     Modules/
4 |       MyCoolModule/
5 |         Configuration/
6 |         JavaScript/
7 |         Sass/
8 |         SuiteScript/
9 |         SuiteScript2/
10 |        Templates/
11 |        Translations/
12 |          en_US.json
```

```

13 |     es_ES.json
14 |     fr_FR.json

```

3. In each .json file, list a text string for each key you want to translate.
For example: "Home": "Accueil",
4. In the extension or theme manifest.json, add a new section for translations as shown in the following example.
 - In the translations section of the extension or theme manifest, list each application for which you are including translation files: shopping, myaccount, checkout.
 - Specify the locale of each .json language file.
 - List each .json language file from your Translations directory that should apply to each application.

The following example shows the translations section of an extension manifest:

```

1 | "translations": {
2 |   "applications": {
3 |     "shopping": {
4 |       "en_US": "Modules/MyCoolModule/Translations/en_US.json"
5 |     },
6 |     "myaccount": {
7 |       "en_US": "Modules/MyCoolModule/Translations/en_US.json",
8 |       "fr_FR": "Modules/MyCoolModule/Translations/fr_FR.json"
9 |     },
10 |    "checkout": {
11 |      "es_ES": "Modules/MyCoolModule/Translations/es_ES.json"
12 |    }
13 |  }
14 | }

```

Note: When you have multiple languages specified for a single application in an extension or theme's manifest.json file, the language displayed to users is the one set up for the domain in the multi-language configuration. Each configured domain will use a specific translation file. For more information, see the help topic [Configure Hosts for Sites with Multiple Languages](#).

5. Deploy and activate your extension or theme as described in [Deploy an Extension to NetSuite](#) and [Deploy a Theme to NetSuite](#).

When you activate a theme or extension that includes .json files like fr_FR.json and es_ES.json in the preceding example manifest, those language .json files are compiled and uploaded into the NetSuite file cabinet. After activating an extension that includes translations, you can check the Live Hosting Files in the NetSuite file cabinet for the compiled .js language files. The compiled .js language files are stored in the languages directory for extensions.

Note: The first time you activate a theme or extension that includes translations, you must invalidate the cache for each multi-language domain that the theme or extension affects. Invalidating the cache for each affected domain is required only the first time you activate the theme or extension. Invalidate the cache immediately after activating the theme or extension. For more information about invalidating the cache for each domain, see the help topic [Cache Invalidation](#).

Precedence of Translations

The precedence of translations configured for a particular key within a Commerce website are as follows, from highest to lowest:

- The translation set for the key in the Configuration Properties for your site. For more information about these property settings, see the help topic [Translations Subtab](#).
- The translation for the key that you include with your extension or theme as outlined in this topic.
- The translation for the key within the application itself, like the shopping application.
- If the application does not provide a translation, the key itself is used.

Overview

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

Develop extensions to add to or change the functionality of your website. You can use the extension developer tools to create a baseline extension and then leverage the SuiteCommerce Extensibility API component classes to build upon that baseline. The Extensibility API lets you extend the core SuiteCommerce application to suit your business needs. Use any of the available component classes to customize the application and add new features or functions. See the help topic [Extensibility API](#) for more details.

To learn more about customization options and whether developing an extension using the extension developer tools is the right option for your needs, see [Developer Essentials](#).

Read the following topics to learn how to download the extension developer tools and then start to create your own extensions:

- [Set Up Extension Developer Tools](#)
- [Extension Development Files and Folders](#)
- [Anatomy of an Extension](#)
- [APIs for SuiteCommerce Extensions](#)
- [Develop Your Extension](#)
- [Test and Deploy Your Extension](#)
- [Best Practices for Developing Commerce Extensions](#)
- [Extension Tutorials](#)
- [Do More With Extensions](#)



Important: To develop an extension, you must have experience working with JavaScript, HTML, and Sass/CSS. The level of experience required depends on the types of changes you want to make. Advanced JavaScript programming skills, including knowledge of APIs, Backbone.js and jQuery are required.

Benefits of Using Extensions

Extensions introduce added functionality to a Commerce website through any number of JavaScript, SuiteScript, configuration JSON, and other files bundled into a single SuiteApp or deployed to a NetSuite account for later activation using the Manage Extensions wizard (included with the SuiteCommerce Extensions Management SuiteApp). One important benefit of extensions is that they allow non-technical users to extend and update their site by installing and activating any number of pre-developed features from a marketplace.

Extensions provide many other benefits:

- Partners can publish and distribute extensions as bundled SuiteApps.
- In-house site developers working with SuiteCommerce, SuiteCommerce MyAccount, or SuiteCommerce Advanced (SCA) can create and manage their own extensions and activate them for any domains associated with a site.

- Extension developers can access much of the functionality that the NetSuite platform currently provides (SuiteScript, custom records/fields/forms, etc.).
- Extensions eliminate version lock within SCA. This means that users have access to easier upgrades that do not compromise the functionality of features previously activated or enabled for a site.
- Extensions rely on accessing the Extensibility API, which ensures developers maintain backwards compatibility with SuiteCommerce code or with previous releases of SCA.

Set Up Extension Developer Tools

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Before you can create an extension, you must download the extension developer tools and extract them to create a top-level development directory. This is where you maintain all of your extension's JavaScript, SuiteScript, Configuration, HTML, Sass, and assets. You use the tools to run Gulp.js commands to build baseline files for your extension, test your changes locally, and deploy extensions to NetSuite.

Note: These tools are required for all SuiteCommerce sites and any SCA sites implementing the Aconcagua release or later. You build extensions to interact with the Extensibility API to extend the application. See [Extensibility API Components](#) for an explanation of what components are currently accessible using the Extensibility API.

Which Tools Do I Use?

Your extension developer tools can be found in the SuiteBundles folder of your NetSuite File Cabinet, as described in [Download Theme Developer Tools](#). These tools accompany the SuiteCommerce Extension Management bundle and are updated automatically with each new SuiteApp update.

The correct tools you require depend on your implementation:

- **SuiteCommerce and SCA 2019.1 and Later** – Use the latest theme and extension developer tools found in the appropriate SuiteBundles folder in your NetSuite File Cabinet, as detailed in the following procedure.
- **SCA Aconcagua and 2018.2 Release** – These implementations are compatible with the 2018.2 extension developer tools. Use the 2018.2.1 developer tools found in the appropriate SuiteBundles folder in your NetSuite File Cabinet, as detailed in the following procedure. These implementations are not compatible with any other developer tools.

Prerequisites

Before downloading the extension developer tools, complete the following tasks:

- Verify that you have SuiteCommerce or SuiteCommerce Advanced installed as described in [Install Your Commerce Website Applications](#).
- Verify that you have SuiteCommerce Extension Manager installed as described in [Install Your Commerce Website Applications](#).
- Set up your developer environment as described in [Install Node.js](#) and [Install Gulp.js](#).

Download the Extension Developer Tools

To download and extract the extension developer tools:

1. Login to your NetSuite account.
2. In NetSuite, go to Documents > Files > File Cabinet.

3. Go to SuiteBundles/Bundle 521562/.
4. Download the .zip file you find there:

ExtensionDevelopmentTools-24.1.x.zip (where x equals the latest minor release).



Important: If you are implementing the Aconcagua or the 2018.2 release of SuiteCommerce Advanced, download the ExtensionDevelopmentTools-18.2.1.zip file. These are the only tools compatible with your implementation.

5. Extract the .zip file to a location in your local environment. This becomes your root development directory for your custom extensions.

The .zip file extracts into a directory of the same name, but you can rename this directory to suit your needs.



Important: Do not move, delete, or rename any files or folders within the top-level development directory.

6. Open a command line or terminal window.
7. Access your root development directory created previously.
8. Enter the following command to install additional Node.js packages into this directory:

```
npm install
```

This command installs the dependencies required to manage your custom extensions. These files are stored in the node_modules subdirectory of the root development directory. This command may take several minutes to complete.



Note: If you use a sandbox account, follow the preceding steps to create two separate root development directories—one for production and one for sandbox. Following this best practice will help ensure you fetch from and deploy to the correct account later in the development lifecycle.

You are now ready to begin extension development. See [Get Started with Extensions](#) for information about building a baseline extension, testing on a local server, and deploying to a NetSuite account.

If you also intend to create themes, you must [Set Up Theme Developer Tools](#).

Using Extension Developer Tools with Token-Based Authentication

Extension Developer Tools can be used to support token-based authentication (TBA) for your site. To take advantage of this, complete the following steps:



Note: If you completed these steps when setting up the Theme Developer Tools, you do not need to repeat these steps.

1. Create an integration record.

For more information, see the help topic [Create Integration Records for Applications to Use TBA](#).

- a. In the **Name** field, enter a name for the integration record.
- b. In the **State** field, select **Enabled**.
- c. Check the **Token-Based Authentication** box.

- d. Check the **TBA: Authorization Flow** box.
 - e. In the **Callback URL** field, enter: *http://localhost:7777/tba*.
 - f. Clear the **Authorization Code Grant** box.
 - g. Check the **User Credentials** box.
2. Save the integration. Be sure to copy the consumer key/secret before closing the page.
3. Paste the consumer key/secret in the .env file.



Note: This file is inside the root extension developer tools folder that was downloaded when setting up developer tools. For more information about locating this folder, see [Download the Extension Developer Tools](#).

Extension Development Files and Folders

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

When you extract the zip file that contains the extension developer tools as described in [Set Up Extension Developer Tools](#), the result is your top-level extension development directory. Read the following topics to learn more about extension development files and folders:

- [The Top-Level Extension Development Directory](#)
- [The Extensions Workspace Directory](#)
- [The Extras Directory](#)

The Top-Level Extension Development Directory

The top-level extension development directory contains the following files and folders. You name this top-level directory when you extract the extension development tools.

Note: Some of the files and folders listed in the following table are created when you run specific gulp commands during the extension development lifecycle and so are not visible when you initially extract the tools. These cases are noted in the descriptions of the files and folders in the following table.

Important: Do not manually edit any of the individual files in the top-level extensions development directory.

File/Folder	Description
DeployDistribution/	Created the first time you run the <code>gulp extension:deploy</code> command, this directory contains all of the files associated with the compiled application. After compilation, Gulp.js deploys the contents of this directory to your NetSuite file cabinet. Do not manually edit the files in this directory.
gulp/	Created when you extract the extension developer tools, this directory contains all of the files required by Gulp.js. Do not manually edit the files in this directory.
LocalDistribution/	Created the first time you run the <code>gulp extension:local</code> command, this directory contains all of the files associated with the compiled application used by the local server. When you run <code>gulp extension:local</code> , Gulp.js deploys the contents of this directory to the local Node.js server. Do not manually edit the files in this directory.
node_modules	Created when you run the <code>npm install</code> command, this directory stores the dependencies and other files required by the development tools.
ns_npm_repository	Created when you install the extension developer tools, this folder contains important files for the NPM package manager.
Workspace/	Created the first time you run the <code>gulp extension:fetch</code> or <code>gulp extension:create</code> command, this directory maintains all of your extension files under development. This directory also includes an <code>Extras/</code> folder to maintain theme files for local testing. See The Extensions Workspace Directory for detailed information on the contents of this directory.

gulpfile.js	This file contains all the JavaScript code necessary to run Gulp.js.
package.json	This file maintains dependencies required to operate the theme development tools.

The Extensions Workspace Directory

The Workspace directory resides in your top-level extensions development directory. This directory contains a subdirectory for each extension you are creating plus an Extras directory to store the active theme source files.

```

1 <TopLevelDevelopmentDirectory>/
2   Workspace/
3     Extras/
4     <EXTENSION_DIRECTORY>/

```

The Extension Directory

When you run the `gulp extension:create` command, the developer tools create the Workspace directory (unless it already exists) and create a basic extension with source files for you to begin creating your own extension.

Each extension directory contains the following files or folders:

File/Folder	Description
assets/	This directory maintains any images or fonts associated with the extension. These assets include fonts, logos, icons, and other images related to your site that are not managed by NetSuite.
Modules/	This directory contains individual modules as subdirectories associated with the extension. Each module defines a specific area of functionality (feature or utility) for your site and contains the JavaScript, SuiteScript, Configuration, Templates, and Sass files for your extension.
manifest.json	This file maintains all extensible resources for the extension. The development tools automatically edit this file to include any necessary overrides when you run the <code>gulp extension:deploy</code> command. For more information on this file, see Edit the Extension Manifest .

Example

You run the `gulp extension:create` command from your top-level extension development directory. During this task, you name your extension **MyCoolExtension** and choose to create all options for extension files (templates, sass, configuration, etc.). You also choose not to create a Custom Content Type (CCT).

After this command is complete, your Workspace directory contains the following:

```

1 <TopLevelDevelopmentDirectory>/
2   Workspace/
3     MyCoolExtension/
4       assets/
5         fonts/
6         img/
7         services/
8           MyCoolModule.Service.ss
9       Modules/
10        MyCoolModule/
11          Configuration/
12          JavaScript/

```



```

13 |         Sass/
14 |         SuiteScript/
15 |         SuiteScript2/
16 |         Templates/
17 |     manifest.json

```

The MyCoolModule directory contains an example file for each files type. You use these files as an example to build your new extension. For example, the JavaScript folder contains JavaScript collection, model, router, view, and entry point files. For more information about these files, see [What Happens When You Create a Baseline Extension?](#)

The Extras Directory

When you run the `gulp extension:fetch` command, the developer tools create the Workspace directory (unless it already exists) and downloads the source files for the active theme. These files are provided for reference during local testing only. Do not move, delete, add, or edit the files in the Extras subdirectory.

The Extras subdirectory contains the following files or folders:

File/Folder	Description
assets/	This directory contains any images or fonts associated with the active theme. These assets include fonts, logos, icons, and other images related to your site that are not managed by NetSuite.
Modules/	This directory contains a module folder that maintains all HTML templates and Sass associated with the theme. These files are provided here for your reference only.
Overrides/	This directory contains files associated with overrides for the theme. See Override Active Extension Files to learn how to use the Override method to customize HTML and Sass files related to extensions.
manifest.json	This file lists all extension-related HTML templates, Sass, and assets related to the active theme downloaded when you ran the <code>gulp extension:fetch</code> command. Do not manually edit this file.

Note: The Extras subdirectory also contains an `application_manifest.json` file. This file confirms that you have a valid SSP Application version that supports the Themes and Extensions. Do not move, delete, or edit this file.

Example

You run the `gulp extension:fetch` command.

Your domain has an active theme, **ActiveTheme1**, and an active extension, **MyCoolExtension**. You run the `gulp theme:fetch` command and specify your domain. The extension developer tools download all theme-related HTML, Sass, and asset files into the `Extras/ActiveTheme1/` directory.

In this example, your Workspace directory structure should look similar to:

```

1 | <TopLevelDevelopmentDirectory>/
2 |   Workspace/
3 |     Extras/
4 |       ActiveTheme1/
5 |         assets/
6 |         Modules/
7 |         Overrides/
8 |         manifest.json

```

9 | MyCoolExtension/

Anatomy of an Extension

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

SuiteCommerce extensions generally include the following types of files. If you create a baseline extension using the default for each command-line option as described in [Create a Baseline Extension](#), you can view examples of most of these files in your Workspace directory.

File	Description	Example Baseline Extension File*
Entry Point	An essential file in any extension with JavaScript is the entry point file. When an extension is loaded into a site's application, this is the file that is called. In the entry point JavaScript file, you should define a module. Every JavaScript-based file in an extension must be created as a module and SuiteCommerce sites use asynchronous module definition (AMD), which relies on two libraries: RequireJS and almond. Specify these things when you define a module: ■ Module name – The almond library requires all modules to have names. Named modules can be called by other modules. ■ Array of modules this module depends upon – Passed in as an array of names as strings and then named in the callback's parameters. ■ Callback – The code that is returned when the module is called. ■ Return an object – The returned object is the most important part of the module. You should define properties of this object that act as the module's methods.	Acme.MyCoolExtension.MyCoolModule.js This file resides in: Workspace/MyCoolExtension/Modules/MyCoolModule/Javascript
Views	Views are part of the JavaScript layer between the application and the final rendered HTML page. A view is a JavaScript file that provides instructions about how to build the HTML page (along with a template to use). Views are also useful actors in linking the UI of the site and both the JavaScript and data layers. Define views in your extension according to the version of SuiteCommerce used for your website. SuiteCommerce or SCA 2020.2 or later: You should use the SCView class instead of Backbone.View when defining views. SCView is based on Backbone.View, but also includes additional methods specific to SuiteCommerce. See Views in an Extension for more information. SCA 2020.1 or earlier: Define a view that extends Backbone.View. See Work with Views in an Extension for more information about working with views.	MyCoolModule.View.js This file resides in: Workspace/MyCoolExtension/Modules/MyCoolModule/Javascript

File	Description	Example Baseline Extension File*
Templates	Rather than use raw HTML pages, SuiteCommerce extensions use files with a .tpl file extension, as well as syntax provided by the Handlebars framework. Template files are processed each time they are to be rendered by a view, and can be thought of as something between an HTML and a JavaScript file. Although you should not include raw JavaScript in template files, you can use things like predefined variables and helper functions.	acme_mycoolextension_mycoolmodule.tpl This file resides in: Workspace/ MyCoolExtension/Modules/MyCoolModule/ Templates
Models	Just as SuiteCommerce has a special class for dealing with the visual part of our Commerce website application, the view class, it also has a special class for handling data called a model. The model is a data layer and performs many functions related to data, such as synchronizing data between the client (frontend) and the server (backend). Typically, when rendering a view that includes data, you will pass it a model. For more information, see Models in an Extension .	See the information in this table about the Frontend Model and Backend Model.
Frontend Models	The extension developer tools create two initial frontend models for each module in your extension, one model that works with SuiteScript 1.0 services and one that works with SuiteScript 2.0 services. For more information, see What Happens When You Create a Baseline Extension? Define views in your extension according to the version of SuiteCommerce used for your website. SuiteCommerce or SCA 2020.1 or later: Starting with SuiteCommerce release 2020.1, all frontend models use the SCModel class included in the extensibility API. The SCModel class is built on a modified version of Backbone.Model. To use it, you add it as a dependency and then extend it to include our own properties. For information about how to define a model with SCModel, see Models in an Extension. SCA 2019.2 or earlier: Define a model by extending Backbone.Model. See Frontend Models for more information.	SuiteScript 2.0 MyCoolModule.SS2Model.js This file resides in: Workspace/ MyCoolExtension/Modules/MyCoolModule/ Javascript SuiteScript 1.0 MyCoolModule.Model.js This file resides in: Workspace/ MyCoolExtension/Modules/MyCoolModule/ Javascript
SuiteScript Services	SuiteScript is NetSuite's proprietary scripting language that is based on JavaScript. Specifically, it is JavaScript which has access to new APIs, methods, and objects and can only be run on NetSuite servers. A service acts like a RESTful API endpoint. You can define various standard CRUD (create, read, update and delete) method handlers and then pass those on to other bits of code to deal with. When a response is ready, the service will then send it back to the frontend that called it. You can include SuiteScript 1.0 and 2.0 services in your extensions.	SuiteScript 2.0 MyCoolModule.Service.ss This file resides in: Workspace/ MyCoolExtension/Modules/MyCoolModule/ SuiteScript2 SuiteScript 1.0 MyCoolModule.ServiceController.js This file resides in: Workspace/ MyCoolExtension/Modules/MyCoolModule/ SuiteScript

File	Description	Example Baseline Extension File*
Backend Models	Generally speaking, each SuiteScript service you develop should act purely as an API/handler and any required business logic should reside elsewhere. In the context of a service, use a backend model to perform the business logic of a request after a service has handled it. This logic can include functions like reading or writing records. While there is not a backend model class that you extend or call, best practice is to employ backend models to maintain parity between the frontend data layer and the backend data layer.	Extension developer tools do not create a backend model. Store each backend model you create in the same directory as its corresponding service. For example: Workspace/MyCoolExtension/Modules/MyCoolModule/SuiteScript
Extension Manifest	A manifest file contains a map of the structure of your extension, as well as important metadata about what it does and what versions and products it is compatible with. If you use the extension developer tools to create a baseline extension, a manifest file is included automatically.	Workspace/MyCoolExtension/manifest.json

*Visible when you create a baseline extension using the default for each command-line option as described in [Create a Baseline Extension](#).

Models in an Extension

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

A model in an extension lets you define how data is managed in the application, such as how it is validated, saved, or fetched from the server. When a view in an extension requires data, you typically pass a model to the view, which then renders the model data on the website.

To create a model, use the `SCModel` class in the `SCModel` component. `SCModel` is part of the extensibility API since SuiteCommerce 2020.1. If you want to create an extension on a version of SuiteCommerce Advanced prior to the 2020.1 release, use Backbone's `extend()` method – see [Frontend Models](#) for more information.

Note that the `SCModel` class is not a component class like other classes in the extensibility API such as [Layout](#) or [PLP](#). Instead of creating an instance of `SCModel` with the `getComponent()` method, you add `SCModel` as a dependency when defining a model.

The following example shows a basic model that uses the `SCModel` class.

Vendor.MyExtension.Module.Model.js

```

1  define(
2    'Vendor.MyExtension.Module.Model',
3    [
4      'SCModel',
5      'Utils'
6    ],
7    function(SCModelModule, Utils) {
8      'use strict';
9
10     var SCModel = SCModelModule.SCModel;
11
12     function VendorMyExtensionModuleModel () {
13       SCModel.call(this);

```

```

14 |
15 |     this.urlRoot = function() {
16 |         return Utils.getAbsoluteUrl(getExtensionAssetsPath('services/Module.Service.ss'));
17 |     }
18 | }
19 |
20 | VendorMyExtensionModuleModel.prototype = Object.create(SCModel.prototype);
21 |
22 | VendorMyExtensionModuleModel.prototype.constructor = VendorMyExtensionModuleModel;
23 |
24 | return VendorMyExtensionModuleModel;
25 | }
26 | );

```

In the `define()` statement, first specify a name for the module and then an array of the module's dependencies. Because you are creating a model based on `SCModel`, add `SCModel` as a dependency. The `Utils` module provides useful shortcut methods and utilities to work with a SuiteCommerce application. You do not need to add `Backbone` as a dependency even though SuiteCommerce uses `Backbone` as its foundation – `SCModel` replaces the `Model` module in `Backbone`.

In the callback function, create a variable, `SCModel`, to hold the `SCModel` class. You can refer to this variable later in the module.

Next, create a constructor function to define how the model works. You will return this function as the output of the module at the end of the callback function. Within the constructor function, the first thing you must do is call `SCModel`'s constructor with `SCModel.call(this)`.

You can then set the properties of the model. In most cases, you must set the `urlRoot` of the model, which is empty by default. The value of `urlRoot` must be a function that returns a string. You can also define any other properties of the model in the constructor function.

Outside of the constructor function, you need to do some work to ensure your custom model functions correctly as regards its parent model, which is `SCModel`.

```

1 | VendorMyExtensionModuleModel.prototype = Object.create(SCModel.prototype);

```

Set your custom model's prototype to the prototype of `SCModel`. To do this, use `Object.create()` to create a new object, with the object passed in to the method as the prototype of the new object. This method of prototypal inheritance creates a copy of `SCModel`'s prototype and assigns it as the prototype of your custom model.

When you set an object's prototype with `Object.create()`, however, it sets all of the object's properties and methods, including its constructor property. To restore the constructor, reset the constructor property to the custom model's constructor function.

```

1 | VendorMyExtensionModuleModel.prototype.constructor = VendorMyExtensionModuleModel;

```

Finally, return the constructor function `VendorMyExtensionModuleModel` as the output of the module.

Views in an Extension

📌 Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Views in an extension tell the application which template to render, pass context data to the template, and listen for user interface events. In addition, views often contain much of the business logic of an extension. You can pass data, such as a component of the extensibility API, to a view at instantiation.

Use the [SCView](#) class of the SCView component to define a view in an extension. SCView is part of the extensibility API since SuiteCommerce 2020.2.

Note: If you want to create an extension on a version of SuiteCommerce Advanced prior to the 2020.2 release, you must extend the [View](#) module of Backbone.

SCView is an abstract class, which means that instead of instantiating it as you would with other components in the extensibility API, you include it as a dependency when defining your view. You must override the `getContext()` method in your view subclass. `getContext()` is used to set contextual data, which can then be accessed in a template. You can also override the `getEvents()` method if you need to specify event listeners and handlers.

The following example shows the anatomy of a basic view that uses the SCView class.

NetSuite.MyExtension.MainView.js

```

1  define(
2    'NetSuite.MyExtension.MainView',
3    [
4      'SCView',
5      'netsuite_myextension_mainview_tpl'
6    ],
7    function(SCViewComponent, netsuite_myextension_mainview_tpl) {
8      'use strict';
9
10     var SCView = SCViewComponent.SCView;
11
12     function MyExtensionMainView() {
13       SCView.call(this);
14
15       this.template = netsuite_myextension_mainview_tpl;
16     }
17
18     MyExtensionMainView.prototype.getContext() {
19       return {
20         quantityintransit: '10',
21         quantityonhand: '25'
22       };
23     }
24
25     MyExtensionMainView.prototype.getEvents() {
26       'click #getExtendedStockInfo': 'getExtendedStockInfo'
27     }
28
29     // A pseudo function to get additional stock information from another
30     // data source.
31     MyExtensionMainView.prototype.getExtendedStockInfo() = function() {
32       // code
33       // goes
34       // here
35     }
36
37     return MyExtensionMainView;
38   }
39 );

```

To create a view, include the SCView class as a dependency in the module definition, which should conform to the asynchronous module definition (AMD) specification. In the `define()` statement, specify the module's name, any dependencies, and a callback function. In a basic view, the minimum dependencies are typically the SCView class and a template file that contains the HTML output.

In the callback function, create a variable named SCView to hold the SCView class. You can refer to this variable later in the module.

Next create a constructor function to define how the view works. Views usually require a template, so set the value of `this.template` to the template name you passed in to the callback function. If you want to define any other initialization tasks, you can add them within the constructor function.

Because `getContext()` is an abstract method, you must define it and return an object. If your extension does not have context data, return an empty object with `return {}`. You can then access the context data in the template with Handlebars placeholders. For example, to access the value of the `quantityintransit` property, use a placeholder in the following format in the template file:

```
1 | <div>{{quantityintransit}}</div>
```

If you want to register event listeners in the view, override the `getEvents()` method. `getEvents()` must return an object containing the event listeners as properties and the event handlers as the value of those properties. See [SCView](#) in the extensibility API reference for more information.

Define any other methods or properties your view requires on the subclass' prototype property. The preceding example includes a pseudo function to get third-party stock information. You can call this function with `this.getExtendedStockInfo()` elsewhere in the main callback function. If you need to call the function in another scope, for example in the callback function of a promise, call it with `MyExtensionMainView.prototype.getExtendedStockInfo()`.

Return the AMD constructor function at the end of the callback function with `return MyExtensionMainView`.

APIs for SuiteCommerce Extensions

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

When developing extensions for a SuiteCommerce website, you are likely to use one or more APIs to complete your customization. NetSuite and the SuiteCommerce framework provide the following APIs:

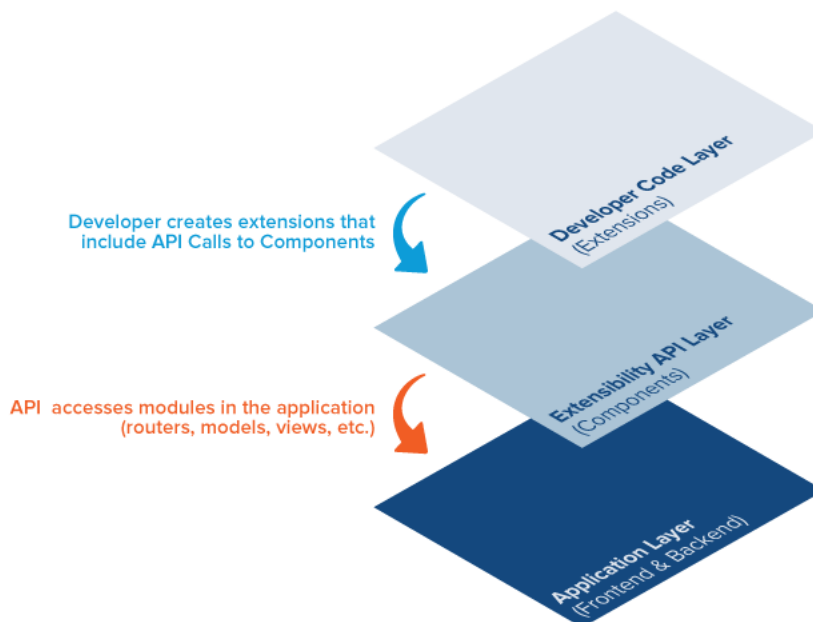
- **SuiteCommerce Extensibility API** – Use the extensibility API to interact with the application layer of a SuiteCommerce website. The extensibility API consists of several components that provide access to different parts of the SuiteCommerce application. For more information, see [SuiteCommerce Extensibility API](#).
- **SuiteScript API** – You can use the SuiteScript API to access custom records and NetSuite records, get and set record data, and perform record actions. SuiteScript 2.x is a JavaScript API that enables you to customize, extend, and process data in NetSuite. For more information, see [Use SuiteScript With Your Extension](#).
- **Item Search API** – The item search API is a REST style API that lets you to retrieve information from item records. It returns the information in JSON format. For more information, see the help topic [Item Search API](#).
- **Commerce API** – The Commerce API exposes a set of shopping objects and functions that you can use to develop web store customizations. For more information, see the help topic [Commerce API](#).

SuiteCommerce Extensibility API

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Note: This information refers to the SuiteCommerce and the Aconcagua release of SuiteCommerce Advanced and later.

SuiteCommerce includes a conceptual layer on top of existing SuiteCommerce architecture. This layer lets extension developers customize a Commerce web store and SuiteCommerce InStore (SCIS) with extensions, so that they don't have to change or customize the underlying application code.



Developers build extensions in their local environment with the extension developer tools. This environment/workspace is represented by the Developer Code Layer in the image above. Each extension can include JavaScript, SuiteScript, or configuration files to do its job. In this code, developers include API calls to components in the Extensibility API. This is represented by the Extensibility API Layer.

The API accesses frontend and backend modules, such as routers, models, and views, in the Application Layer. This way, extensions are separated programmatically from the main application and interact only with the API, not with the application. This makes customization easier and seamless for extension developers and ensures that future updates to the application do not compromise customizations.

See the following topics for more information about the Extensibility API:

- [Extensibility API Components](#) – This topic provides an overview of the Component Classes available for creating extensions.
- [Instantiating API Components](#) – This topic describes how to access specific components, such as Product Details Page or Product List Page.
- [SCIS Extensibility API](#) – This topic provides details about the components available for creating extensions for SCIS.

See the [Overview](#) topic of Extensibility API reference for information about each component and its methods and properties.

Extensibility API Components

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

A component is a feature in a Commerce application that connects with certain parts of the application to accomplish a task. The Extensibility API is organized around these components. Each component has a class and various exposed methods for obtaining or modifying data, views, or template context.



Important: The Backend Extensibility API has been deprecated since the 2020.1 release of SuiteCommerce and SuiteCommerce Advanced. Instead, you should use methods available in the Frontend Extensibility API or SuiteScript 2.0 modules and methods. For more information, see the [Extensibility API Reference](#) and the [SuiteScript 2.x API Introduction](#).

SuiteCommerce, SuiteCommerce Advanced, and SuiteCommerce InStore each have their own set of components. The Extensibility API includes the following components.

- [SuiteCommerce and SuiteCommerce Advanced Components](#)
 - Cart
 - Checkout
 - CMS
 - Environment
 - Layout
 - LoginRegisterPage
 - MyAccountMenu
 - PageType
 - PageTypeBaseView
 - PDP (Product Details Page)
 - PLP (Product List Page)

- SCCollection
- SCCollectionView
- SCFormView
- SCModel
- SCView
- Search
- UserProfile
- **SuiteCommerce InStore (SCIS) Components**
 - Cart
 - SCISLayout

For more information about how to use these components when building extensions, see [Extension Tutorials](#).

SuiteCommerce and SuiteCommerce Advanced Components

The SuiteCommerce components apply to both SuiteCommerce and SuiteCommerce Advanced.

See the full [Overview](#) topic of SuiteCommerce Extensibility API for detailed information about each component.

Cart

This component interacts entirely with the LiveOrder, allowing you to add, update, and remove lines from the order, apply promotions to the order, select a shipping method on the order, associate the shipping and billing address to the order, add payment methods, and so on. This component can access objects available in all three ssp applications: shopping, myaccount and checkout.

ID	Cart
Methods	- addLine() - addLines() - addPayment() - addPromotion() - clearEstimateShipping() - estimateShipping() - getAddresses() - getBillAddress() - getLatestAddition() - getLines() - getPromotions() - getPurchaseOrderNumber() - getShipAddress() - getShipMethod() - getShipMethods()

	getSummary() removeLine() removePromotion() removeShipping() setPurchaseOrderNumber() setShipMethod() setTransactionBodyField() submit() triggerAddToCartConfirmation() updateLine()
Events	beforeAddLine afterAddLine beforeAddPayment afterAddPayment beforeAddPromotion afterAddPromotion beforeClearEstimateShipping afterClearEstimateShipping beforeEstimateShipping afterEstimateShipping beforeRemoveLine afterRemoveLine beforeRemovePromotion afterRemovePromotion beforeSetPurchaseOrderNumber afterSetPurchaseOrderNumber beforeSetShipMethod afterSetShipMethod beforeSetTransactionBodyField afterSetTransactionBodyField beforeSubmit afterSubmit beforeUpdateLine afterUpdateLine

Checkout

This component interacts entirely with the Checkout page and the checkout steps flow. This component lets you modify the steps visually, such as adding, removing, and reordering steps, etc. This component can access objects available in the checkout application only.

ID	Checkout
Methods	getCurrentStep() setCurrentStep() getStepGroupsInfo() getStepsInfo() getCheckoutFlow() addModuleToStep() removeModuleFromStep() addStepsGroup() removeStepsGroup() addStep() removeStep()
Events	beforeAddModuleToStep afterAddModuleToStep beforeSetCurrentStep afterSetCurrentStep beforeAddStep afterAddStep beforeAddStepsGroup afterAddStepsGroup

CMS

This component is created only for the purpose of registering new Custom Content Types. If you want to add views inside of your CCT you can interact with all the rest of the components available. This component can access objects available in all three ssp applications: shopping, myaccount, and checkout.

ID	CMS
Methods	registerCustomContentType()
Events	–

Environment

This component is general to the entire SuiteCommerce Application and provides the utils method and access to Configuration values. This component does not provide interaction with any particular view. This component can access any object available in all three ssp applications: shopping, myaccount and checkout.

ID	Environment
Methods	addTracker() defineSearchMasterOptions()

	getConfig()
	isPageGenerator()
	getSiteSetting()
	getSession()
	setTranslation()
	triggerEvent()
Events	–

Layout

This component lets you modify aspects of SuiteCommerce that do not have a specific component available. For example, no components currently exist that let you add or modify views on the home page, in the header, or in the footer. Using Layout, you can access all the view methods for each view (addChildViews, addChildView, removeChildView, and so on). As more components become available in future SuiteCommerce updates, you can replace the use of Layout for the appropriate (new) component, keeping the same methods.

ID	Layout
Available Since	SuiteCommerce 2018.2
Methods	addChildView() addChildViews() addToViewContextDefinition() addToViewEventsDefinition() closeMessage() registerView() removeChildView() removeToViewContextDefinition() removeToViewEventsDefinition() setChildViewIndex() showContent() showMessage()
Events	–

LoginRegisterPage

The LoginRegisterPage component is used to listen for login and register events on the Log In / Register page in the web store. It also enables you to capture additional form data from the login and registration forms.

ID	LoginRegisterPage
Available Since	SuiteCommerce 2019.1
Methods	–

Events	beforeLogin beforeRegister afterRegister
---------------	--

MyAccountMenu

This component lets you add menu items to the menu on the My Account page. You can add group items and then add subitems to the group items. All group items and subitems have one or more permissions, which lets you restrict which users can view the menu items. Subitems point to a landing page URL that is loaded in the main content area of the page.

ID	MyAccountMenu
Available Since	SuiteCommerce 2019.1
Methods	addGroup() addGroupEntry()
Events	–

PageType

Use the PageType component when you want to register new page types in an extension. A page type record can be created by logging in to NetSuite and creating the record, or by including the page type in the extension manifest file.

ID	PageType
Available Since	SuiteCommerce 2019.1
Methods	registerPageType() registerTemplate()
Events	–

ProductDetailsPage (PDP)

This component interacts entirely with the Product Details page. This accesses objects inside the Product Details View only. This is undefined if invoked from any other part of the application.

ID	PDP
Methods	getAllMatrixChilds() getItemInfo() getPrice() getSelectedMatrixChilds setOption() setQuantity() getStockInfo()

Events	beforeOptionSelection
	afterOptionSelection
	beforeQuantityChange
	afterQuantityChange
	beforeImageChange
	afterImageChange

ProductListPage (PLP)

This component interacts entirely with the Search Page, allows navigating to the next or previous page, set the current page, change the rendering, search for a product, get the last search term, and so on. This component accesses objects inside the Facets Browse View in the shopping application only. This is undefined if invoked from any other part of the application.

ID	PLP
Methods	getPagination()
	setCurrentPage()
	getSorting()
	getAllSorting()
	setSorting()
	getDisplay()
	getAllDisplay()
	setDisplay()
	getPageSize()
	getAllPageSize()
	setPageSize()
	getFilters()
	getAllFilters()
	setFilters()
	getSearchText()
	setSearchText()
	getItemsInfo()
	getCategoryInfo()
	getURL()
Events	–

SCCollection

The SCCollection component is the SuiteCommerce implementation of Backbone.Collection. It inherits all the methods and properties of Backbone.Collection. See the [Backbone.js](#) website for more information about Backbone.Collection.

Available since SuiteCommerce 2020.1.

SCCollectionView

Use the SCCollectionView component when you want to work with a collection of elements in a grid pattern on a web store page. SCCollectionView contains the SCCollectionView abstract class, which you must override with your own child class. Properties and methods in the class let you specify how rows and columns are displayed on the page. See the [extensibility API reference](#) for detailed information.

Available since SuiteCommerce 2020.2.

SCFormView

The SCFormView component enables you to validate form field data, display error messages in a form, and save the data to the associated model. The SCFormView class is an abstract class, which you must override with your own child class.

Available since SuiteCommerce 2020.2.

SCModel

The SCModel component is the SuiteCommerce implementation of Backbone.Model. It inherits all the methods and properties of Backbone.Model. See the [Backbone.js](#) website for more information about Backbone.Model.



Important: In extensions on SuiteCommerce 2020.1 or later, you should use SCModel instead of Backbone.Model.

Available since SuiteCommerce 2020.1.

SCView

The SCView component is the SuiteCommerce implementation of Backbone.View. This component contains an abstract class, SCView, which you must override with your own child class. It provides all methods and properties of Backbone.View, as well as additional methods and properties to work with SuiteCommerce. For example, instead of using the native events property of Backbone.View, you should use the `getEvents()` method of SCView. See the [extensibility API reference](#) for detailed information.



Important: In extensions on SuiteCommerce 2020.2 or later, you should use SCView instead of Backbone.View.

Available since SuiteCommerce 2020.2.

Search

The main purpose of the Search component is to retrieve the URL of the item search API of the SuiteCommerce website. You can then use the URL to search for items or get item information. The returned URL only includes the path and parameters of the item search API for a website; it does not include the URL scheme or domain name. Here is an example of a URL returned by the `getUrl()` method of this component:

```
/api/items?
language=en&country=US&currency=USD&pricelevel=5&use_pcv=F&q=frames&fieldset=relateditems&dimensions=52x30
```

ID	Search
----	--------

Available Since	SuiteCommerce 2021.1
Methods	getUrl()
Events	–

Available since SuiteCommerce 2021.1.

UserProfile

The UserProfile component lets you retrieve information related to the user logged in to the SuiteCommerce website. You can get basic user profile information as well as the user's credit card data.

ID	UserProfile
Available Since	SuiteCommerce 2019.1
Methods	getCookieOptions() getCreditCards() getCustomerSegments() getUserProfile() setCookieOptions()
Events	–

SuiteCommerce InStore (SCIS) Components

The SCIS (SuiteCommerce InStore) components in the extensibility API let you create extensions that add, remove, or update lines in the cart. You can also show modal dialogs to present information to the user, and require the user to confirm or cancel an action.

See the full [extensibility API reference](#) for detailed information about each component.

Cart

The Cart component in SCIS lets you add, remove, and update lines in the cart. It also provides methods to perform other operations in the cart, such as applying promotions and discounts.

ID	Cart
Available Since	SuiteCommerce 2018.2
Methods	addGlobalDiscount() addLine() addLines() addLineDiscount() addPromotion() getLines() getSummary() removeLineDiscount() removePromotion()

	returnLine() setCustomer() unvoidLine() voidLine()
Events	beforeAddGlobalDiscount afterAddGlobalDiscount beforeAddLine afterAddLine beforeAddLineDiscount afterAddLineDiscount beforeRemoveLineDiscount afterRemoveLineDiscount beforeReturnLine afterReturnLine beforeSetCustomer afterSetCustomer beforeUnVoidLine afterUnVoidLine beforeVoidLine afterVoidLine

SCISLayout

The SCISLayout component gives you access to two methods, both of which show a modal dialog to the user, but with different options to dismiss the dialog.

ID	SCISLayout
Available Since	SuiteCommerce 2018.2
Methods	showConfirmationPopup() showMessagePopup()
Events	–

Instantiating API Components

❗ Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

To use an extensibility API component in your extension, you usually have to instantiate the class in the component first. To instantiate the class, pass the ID of the class to the `getComponent()` method on the container object. It is also good practice to check that the component exists before accessing its methods or properties. If the component instantiation fails, `getComponent()` returns `null`.

```
1 | var pdp = container.getComponent('PDP');
2 |
```

```

3 | if (pdp) {
4 |     var stockinfo = pdp.getStockInfo();
5 | }
6 | else {
7 |     console.log('Could not instantiate PDP.');
```

Some components in the extensibility API expose classes that do not need to be instantiated with `getComponent()`. Instead, classes in the following components are added as dependencies when defining modules such as models, views, and collections:

- SCCollection
- SCCollectionView
- SCFormView
- SCModel
- SCView

See the [extensibility API reference](#) for more information about each component.

Access the Frontend Components

To access the frontend components:

1. Access a frontend component in the `mountToApp()` function in the entry point for your extension.

By default, the entry point file is located in the JavaScript subdirectory in your extension code and uses the following naming convention:

```
1 | <Vendor.ExtensionName.ModuleName>.js
```



Important: Some component methods need to be used after the main view of the component has been rendered (after the event `afterShowContent` has been triggered). This applies to the PDP, PLP, and Checkout components.

2. Define the dependencies:

```

1 | define(
2 |     'Vendor.ExtensionName.1.0.0.Extension'
3 |     , [
4 |         'Vendor.ExtensionName.1.0.0.ExtensionName.Model'
5 |         , 'Vendor.ExtensionName.1.0.0.Extension.View'
6 |     ]
7 |     , function (
8 |         ExtensionModel
9 |         , ExtensionView
10 |     )
```



Note: In the above example, the values for `Vendor` and `ExtensionName` depend on the values you enter when you create the baseline extension. The names `ExtensionModel` and `ExtensionName` are example names for your model and view, respectively, that you define when you create the extension code.

3. Instantiate the component with the `getComponent()` method on the container object.

Since you can use the same entry point file for all three applications (MyAccount, Checkout, or Shopping), make sure that your component is not null before using it.

The following example shows how to call the PDP component and test to see that it exists before passing it to the rest of the models and views:

```

1 {
2   'use strict';
3
4   return {
5     mountToApp: function mountToApp (container) {
6
7       var pdp = container.getComponent('PDP');
8
9       if (pdp) {
10        pdp.addChildViews();
11
12        new ExtensionModel({pdp: pdp});
13      }
14    }
15  }
16 });

```

Access the Backend Component

Access `BackEndCartComponent` anywhere in the SuiteScript code, using the `Application.getComponent(component_name)` method.

Currently, only the `BackEndCartComponent` component is available to SuiteScript code.

To access the backend component

1. Define the dependencies:

```

1 define('Vendor.ExtensionName.1.0.0.Extension.File'
2 , [
3   'Application'
4   , 'underscore'
5 ]
6 , function(
7   Application
8   , -
9 )

```

To access the backend component, you must include `Application` as a dependency.

Note: In the above example, the values for `Vendor` and `ExtensionName` depend on the values you entered when you created the baseline extension. The name `Extension` is an example name that you define when you create the extension code.

2. Use the `getComponent()` method to access the `BackEndCartComponent` instance:

```

1 {
2   'use strict';
3
4   // Get the cart component
5   var cart_component = Application.getComponent('Cart');
6 };

```

Use SuiteScript With Your Extension

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

SuiteScript is NetSuite's proprietary scripting language that provides custom objects and methods you can use with JavaScript. The SuiteScript API provides access to NetSuite data. In terms of developing

SuiteCommerce extensions, the SuiteScript API plays an important role in backend models, where NetSuite data is created, read, updated, transformed, and validated. The SuiteScript API lets you access objects such as:

- Transaction objects like the contents of a shopping cart or an order
- Standard, non-transactional records like customer records
- Custom records

Review the following information to learn more about the SuiteScript API:

See topic	To learn
SuiteScript Overview	About the differences between SuiteScript versions, including 1.0, 2.0, and 2.1, and to access topics that explain more about what you can do with SuiteScript like using SuiteScript with NetSuite records.  Note: SuiteScript 2.1 is not currently supported for Commerce website development. SuiteScript 2.x, 2.0, and 1.0 are supported.
SuiteScript Developer Guide	How to set up your environment for SuiteScript, SuiteScript governance, best practices and more.
SuiteScript Records Guide	More about the NetSuite records available to you when using SuiteScript.
SuiteScript 2.x	More about all aspects of SuiteScript 2.0 and 2.1, including API introduction, script types, custom modules.
SuiteScript 2.x API Reference	Details about API objects and modules.

Develop Your Extension

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Extensions require interacting with the Extensibility API. The Extensibility API is available to extension developers for both SuiteCommerce and SuiteCommerce Advanced (SCA). Be aware of the following requirements:

- **SuiteCommerce and SuiteCommerce MyAccount** – The Extensibility API is required to create extensions for SuiteCommerce and SuiteCommerce MyAccount.
- **SuiteCommerce Advanced (Aconcagua and later)** – Creating extensions using the Extensibility API is a best practice for developing Suite Commerce Advanced. However, if your SCA customizations require access to JavaScript, SuiteScript, or configuration objects that are not accessible using this API, you must use the SCA developer tools and customization practices. See [Customize with SCA Tools](#) for details.
- **SuiteCommerce Advanced (Kilimanjaro and earlier)** – The Extensibility API is not available for these implementations of SCA. To take advantage of themes, extensions, and the Extensibility API, you must either migrate your site to the latest release of SCA or use the SCA developer tools and customization practices to customize your site. See [Customize with SCA Tools](#) for details.

Get Started with Extensions

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Before you start developing an extension for SuiteCommerce, make sure you have completed the following tasks:

- Set up the extension developer tools. The developer tools enable you to create a baseline extension, test your extension, deploy it to NetSuite, and more. See [Set Up Extension Developer Tools](#) for more information.
- Familiarized yourself with the SuiteCommerce extensibility API. You can use the extensibility API to access or modify many aspects of the SuiteCommerce application, such as the behavior and content of the product list page, a visitor's cart, or the application settings. See the [SuiteCommerce extensibility API](#) for full details.

Extension Development Steps

The basic steps to create an extension include the following:

1. **Create a baseline extension.** The extension developer tools give you a starting point for extension development. Follow these instructions to create a baseline "Hello World" extension. See [Create a Baseline Extension](#).
2. **Fetch active theme and extension files.** You must download files for the currently active theme before you can deploy an extension or test it on a local server. See [Fetch Active Theme and Extension Files](#).
3. **Add extension functions and features.** You can introduce JavaScript, SuiteScript, Json, HTML templates, and Sass files, as needed. The Extensibility API is available to access frontend and backend modules within the application. See [APIs for SuiteCommerce Extensions](#).

4. **Test on your local machine.** You can test your extension code on a local server for debugging purposes. See [Test an Extension on a Local Server](#).
5. **Deploy your extension to NetSuite.** You can test your extension on a local server. However, you must first deploy your extension files to your account and activate the extension on a domain before testing locally. See [Deploy an Extension to NetSuite](#).
6. **Activate the extension.** To see your extension in action, you must activate it for a domain. See the help topic [Manage Themes and Extensions](#).

Useful Information to Know Before You Begin

Be aware of the following important information:

- The `gulp extension:fetch` command only fetches the following:
 - HTML and Sass files associated with the currently active theme for the specified domain.
 - Extensions development files for custom extensions currently active for the specified domain. You cannot download files for published extensions.
- Login parameters, including your NetSuite email, account, role, and domain information, are stored in the `.nsdeploy` file when you create an extension. To reset your login and deployment information, delete the `.nsdeploy` file. This file is located here:


```
<LOCAL_SOURCEFILES_ROOT>/gulp/config/.nsdeploy
```
- Extension information, including the extension name, fantasy name, version, and description, are stored in the extension's `Manifest.json` file. The manifest file is located here:


```
<LOCAL_SOURCEFILES_ROOT>/Workspace/<EXTENSION_DIRECTORY>/manifest.json
```

Create a Baseline Extension

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

A baseline extension is a set of files, installed locally in your extension development directory. You use these files as a basis to build any extension for your Commerce web store or SCIS implementation.

When you run the `gulp extension:create` command, the developer tools create this baseline extension in a `Workspace` directory within your top-level extension development directory. The baseline extension starts out with one module that you design, based on your needs. It can include any combination of the following file types within one new module:

- Templates
- Sass
- Configuration
- JavaScript
- SuiteScript 1.0
- SuiteScript 2.0

i Note: Only 2020.1 Extension Developer Tools and later include the SuiteScript 2.0 file type as an option.

To create a baseline extension:

1. Open a command line or terminal.

2. Access the top-level extension development directory you created when you installed the extension developer tools.
3. Enter the following command:

```
gulp extension:create
```
4. When prompted, enter the information for your baseline extension.



Important: Note the following requirements and behavior:

- Unless otherwise directed, create all entries using only alphanumeric characters without spaces.
- Pressing **Enter** results in the default value.

Set Your Extension Fantasy Name	Provide a name for your extension as you want it to appear in the NetSuite user interface. This can contain special characters and spaces. For example: My Cool Extension!
Set Your Extension Name	Provide a name for your extension's files and folders within your development environment.
Set the Vendor Name	Enter your business, vendor, or partner name.
Assign a Version Number	Add a version number for your extension.  Important: To take advantage of extension update requirements, assign a version number to your extensions. Version numbers should follow this format, where d is a single digit: dd.dd.dd.
Set the Initial Module Name	Name the module to be created as part of the baseline extension. Each extension needs at least one module to contain your extension files.
Set a Description for Your Extension	Provide some text to describe your extension. This string can contain spaces and special characters.
This Extension Supports	Select one of the following options: ■ SuiteCommerce Online – To create an extension for SuiteCommerce or SuiteCommerce Advanced. ■ SuiteCommerce InStore – To create an extension for SuiteCommerce InStore. An asterisk (*) identifies the selected option. Use arrow keys to scroll. Press the spacebar to select an option, press <a> key to select all, or <i> key to invert your selections.
This Extension Will Be Applied To	Select one or more applications that your extension requires. Options include: ■ Shopping ■ My Account ■ Checkout
Target Version	Using Commerce versioning schema, enter a string to declare the target versions to which this extension applies. The developer tools prompt you for a target version for each product specified earlier. Leaving these prompts blank results in compatibility with all versions of the associated Commerce products. See Declare Target Versions and SuiteScript 2.x Bundle Installation Script Type for more information.
For This Extension You Will Be Using	Select one or more file types that your extension should include. Your file options are:

	■ JavaScript ■ SuiteScript ■ Templates ■ Sass ■ Configuration ■ SuiteScript2 The developer tools create a baseline Hello World extension to get you started. For more information, see What Happens When You Create a Baseline Extension?  Note: Only 2020.1 Extension Developer Tools and later include the SuiteScript 2.0 file type as an option.
--	---

After following these prompts, you can optionally build on this baseline by adding additional modules. Consider the following topics:

- [Create Additional Modules for an Extension](#)
- [Create Custom Content Types for an Extension](#)

When you have completed building baseline extension files in your developer environment, you are ready to develop your extension. See [Develop Extensions](#) for details.

What Happens When You Create a Baseline Extension?

When you run the `gulp extension:create` command, the extension developer tools:

- Create the Workspace directory in your top-level development directory, unless it already exists.
- Create a subdirectory with the same name you set for the extension. This subdirectory houses all of the assets and the initial module for the extension. This is where you develop your extension, using the starter files created by the developer tools as an example.
- Create a `manifest.json` file. This file includes all the information required to compile resources for your extension.

For example, if you run the `gulp extension:create` command and create an extension named **MyCoolExtension** with the following options:

- Set a Vendor/Business name of Acme
- Set the initial module name to **MyCoolModule**
- Include all available file types

In this example, your Workspace directory structure should look similar to:

```

1 Workspace/
2   MyCoolExtension/
3     assets/
4       fonts/
5       img/
6       services/
7     Modules/
8       MyCoolModule/
9         Configuration/
10        JavaScript/
11        Sass/
12        SuiteScript/
13        SuiteScript2/
14        Templates/
15    manifest.json

```

For a detailed explanation of the files in a baseline extension, see [Anatomy of an Extension](#).

Starter Files

Building on the preceding example, the developer tools generate Configuration, JavaScript, Sass, SuiteScript, SuiteScript 2, and Template files to provide an example Hello World extension. If you use these starter files as the basis for your own extension, you may need to make specific adjustments to the following files generated in the JavaScript directory, depending on the version of SuiteCommerce or SuiteCommerce Advanced you are using:

- `Acme.MyCoolExtension.MyCoolModule.Model.js` — This file is the frontend model for the extension's initial module. If you are developing for SuiteCommerce or SuiteCommerce Advanced 2020.1 or later and want to include SuiteScript 2.0 services in your extension, set the `getAbsoluteUrl` method as described in [Using SuiteScript 2.0 Services](#).
- `MyCoolModule.Model.View.js` — This file is the view for the example extension. Views listen to the user interface events and coordinate with the data required by the associated module to handle the presentation layer of your extension.

If you are developing for SuiteCommerce Advanced 2020.1.0 or earlier, refer to [Work with Views in an Extension](#) for more information about working with views.

If you are developing for SuiteCommerce or SuiteCommerce Advanced 2020.2.0 or later, the generated example view does not use currently recommended API methods. Starting with the 2020.2 release, you should use `SCView` in your extensions instead of `Backbone.View`. `SCView` extends from `Backbone.View`, inheriting all its methods and properties. `SCView` also provides additional methods for Commerce site development. For more information, see the [extensibility API reference](#).

Using SuiteScript 2.0 Services

When you run the `gulp extension:create` command, the extension developer tools create a frontend model in the JavaScript directory for your extension's initial module. For this example, the model name would be: `Acme.MyCoolExtension.MyCoolModule.Model.js`.

This model includes logic to issue requests via entry points to a backend service. During extension development, the location of extension files after activation is unknown. So, to successfully issue service requests, the model uses the `getAbsoluteUrl` method to generate the absolute URL to the service. This is the service URL that will exist when the extension is deployed to NetSuite and activated. The `getAbsoluteUrl` method includes two parameters:

- The relative path to the service, which is returned by the `getExtensionAssetsPath` helper.
- A boolean that is set to `True` to generate an absolute URL for a SuiteScript 2.0 service; or set to `False` to generate the URL to a SuiteScript 1.0 backend service. Note that leaving the boolean empty is the same as a `False` setting.



Important: The extension developer tools create one initial frontend model for each module in your extension. You need to create additional models if your extension module includes multiple services. Create a separate model for each service.

To create additional models, you can copy and rename the initial model created by the extension developer tools. Then make sure each model includes the correct values for the `getAbsoluteUrl` method parameters as shown in the following examples:

SuiteScript 1.0 service

```
1 //@property {String} urlRoot
2   urlRoot: Utils.getAbsoluteUrl(
3     getExtensionAssetsPath("services/MyCoolModule.Service.ss")
```

```
4 |    );
```

SuiteScript 2.0 service

```
1 //@property {String} urlRoot
2   urlRoot: Utils.getAbsoluteUrl(
3     getExtensionAssetsPath(
4       "Modules/MyCoolModule/SuiteScript2/MyCoolModule.Service.ss"
5     ),
6     true
7   );
```

Fetch Active Theme and Extension Files

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

If creating an extension for SuiteCommerce or SCA, you must first download files for the currently active theme before you can deploy an extension or test it on a local server. You do this by running the following command:

```
gulp extension:fetch
```

i Note: SCIS extension developers can disregard this section.

This command:

- Downloads HTML and Sass files for the currently active theme. Do not modify these files. The Sass and HTML files downloaded using this command are for reference only.
- Downloads files for any custom extensions that are currently activated.

The developer tools require theme files (HTML and Sass) to compile a local distribution for testing an extension on a local server. The developer tools also use these files to run a test compilation before deploying. When you fetch the active theme, you provide the necessary files in your local extension workspace. Later, when you test locally or deploy your extension, the developer tools can compile the code without error.

If you want to skip this step, you can run the `gulp extension:deploy --skip-compilation` command to deploy your extension files without running the compilation. For more Gulp.js commands used with extensions, see [Extension Developer Gulp Commands](#).

The `gulp extension:fetch` command also fetches files for any custom extensions. When you fetch files for an active extension, you download these development files to your local extension workspace. You can now customize these files further development and testing.

⚠ Warning: When you fetch custom extension files, the developer tools download extension files that are activated for the specified domain. This action overwrites any extension files currently stored in your local workspace. This can result in loss of development files if you are not careful. To ensure that you do not lose any work under development, do not run the `gulp extension:fetch` command until you have deployed your extension work to NetSuite.

i Note: This command does not download files for published extensions. A published extension is an extension installed into your account as part of a published SuiteApp. You cannot edit published content due to copyright restrictions.

Fetch the active files:

1. In your local developer environment, open a command line or terminal and access the top-level extension development directory.
2. Run the following command:
`gulp extension:fetch`

Note: For release 2021.2.1 and later accounts that have account-specific domains, specify your account number with the `gulp` command to enable access to the NetSuite account. For more information about using `gulp extension:fetch --account`, see [Extension Developer Gulp Commands](#).

3. When prompted, enter the following information:

Authentication ID	Enter an ID to represent your NetSuite account and role. If you have already created an authentication ID in a previous fetch or deploy operation, the extension developer tools do not prompt you for an ID and will continue to use the existing ID for each subsequent fetch or deploy. To fetch from a NetSuite account that is different from the account used to create the existing authentication ID, use the <code>--to</code> parameter. When you use the <code>--to</code> parameter, you can choose a different authentication ID or create a new one. For more information, see Token Based Authentication.
Account and Role	If you are creating a new authentication ID, you need to select the NetSuite account and role to use with that new authentication ID. If using an existing ID, you are not prompted for this information. In NetSuite, select the account that contains the theme or extensions you want to fetch and the role you want to use for authentication. Click the Allow button to generate the authentication token and secret. Note: You must select a role that provides the permissions needed to fetch from NetSuite. By default, the Administrator and SCDeployer roles provide these permissions. To learn more, including how to create a custom role that provides the required permissions, see Developer Tool Roles and Permissions.
Choose Your Website	Select your website with the domain you want to access.
Choose Your Domain	Select your domain with the active theme or extensions you want to download.
Extensions to Fetch	If you have any active extensions for the domain selected, you can download the working files of any selected extensions. This is optional. Content downloads to your extension's Extras folder.

The `gulp extension:fetch` command creates a theme subdirectory in your Workspace/Extras directory. These files are for use by the developer tools only. Do not add, edit, delete, or move any files in this location.

You are now ready to perform either of the following tasks:

Edit and Create Extension Modules

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

When building an extension, you should develop each area of functionality as a module and then store each module in the Modules folder of the extension's Workspace directory as described in [Extension Development Files and Folders](#).

When you follow the steps described in [Create a Baseline Extension](#), the extension developer tools create an initial module for your extension. You can edit the starter files included in the initial module to develop the custom functionality you need. For more information about the initial module and the files it contains, see: [Anatomy of an Extension](#) and [What Happens When You Create a Baseline Extension?](#)

You can use the extension developer tools to create additional modules for your extension. For detailed instructions, see [Create Additional Modules for an Extension](#).

Create Additional Modules for an Extension

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

The `gulp extension:create` command builds an extension with one baseline module. However, if your extension requires more than one module, the developer tools let you create more using the `-module` attribute.

To add an additional module to your baseline extension:

1. Create a baseline extension, if you have not done so already. See [Create a Baseline Extension](#) for details.
2. Open a command line or terminal.
3. Access the top-level extension development directory you created when you installed the extension developer tools.
4. Enter the following command:
`gulp extension:create-module`
5. When prompted, enter the following information:



Important: Unless otherwise directed, create all entries using only alphanumeric characters without spaces.

Set the Module Name	Name the module to be created as part of the baseline extension. Use only alphanumeric characters without spaces.
For This Module You Will Be Using	Select one or more file types your extension requires. Your file options are: ■ JavaScript ■ SuiteScript ■ Templates ■ Sass ■ Configuration ■ SuiteScript2 The developer tools create a baseline of these files to get you started. i Note: Only 2020.1 Extension Developer Tools and later include the SuiteScript 2.0 file type as an option.
Select extension to add module	Select the name of the extension to contain this module. This option only appears if your Workspace directory contains more than one extension.

6. Repeat these steps to add any additional modules as needed.



Important: NetSuite imposes certain record limits to control the consumption of web services. To avoid errors related to record limits when deploying themes and extensions, limit the number of files and folders at the same level to 100 when developing themes and extensions. Some options include introducing images across multiple folders and placing custom modules in a different top-level directory. For more information on record limiting, see the help topic [SOAP Web Services Governance Overview](#).

When you have completed building baseline extension files in your developer environment, you are ready to develop your extension. See [Develop Extensions](#) for details.

Work with Views in an Extension

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Views are JavaScript files that provide instructions about how to build an HTML page, including instructions for which template to use. Views also handle data transfer to and from templates. For more information about views and templates, see [Anatomy of an Extension](#).

When developing an extension, you can create views or add child views to a parent view to add additional functionality. When creating a view, you should use one of the existing base classes in the extensibility API. This section describes how to use the Extensibility API to complete the following tasks for views in an extension:

- [Create a View](#) – explains how to implement a new view for your extension.
- [Add a Child View](#) – explains how to add a custom child view to a parent view in the Product Details Page, Product List Page, Cart, Layout, or Checkout components using the Extensibility API.
- [Replace a Child View](#) – explains how to replace an existing view with a custom view in an extension.

For more information on view architecture, see the help topic [Views](#).

Create a View

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Views listen to the user interface events and coordinate with the data required by the associated module to handle the presentation layer of the application. A view also includes the `getContext()` method, which defines the variables available to templates using the view.

Extensions might require a new view for an existing page or create new experience altogether. This section explains how to build a view that declares a template and a context.



Important: If you are developing views for release 2020.2 or later, do not use `Backbone.View` as outlined in the following procedure. For release 2020.2 and later, you should use `SCView` as described in the [extensibility API reference](#).

To create a view:

1. Create a new JavaScript file within the JavaScript directory of your extension module.
2. Define the view and its associated template.

```

1 | define('MyCoolView'
2 | , [
3 |   'Backbone'
4 |   , 'my_cool_tpl'
5 | ]
6 | , function (
7 |   Backbone
8 |   , my_cool_tpl
9 | )

```

3. Extend Backbone.View, declaring your template and your context.

```

1 | {
2 |   return Backbone.View.extend({
3 |
4 |     template: my_cool_tpl,
5 |
6 |     getContext: function () {
7 |       return {
8 |         name: 'John'
9 |       };
10 |    }
11 |  });
12 | });

```

4. Save your view.
5. Open the entry point JavaScript file for the module containing your new view.
6. Declare your view as a dependency.

For example:

```

1 | define('MyCoolModule'
2 | , [
3 |   'MyCoolView'
4 |   ...
5 | ]
6 | , function (
7 |   MyCoolView
8 |   ...
9 | )

```

7. Save your entry point file.
8. Declare your new view file to the javascript object of the appropriate application entry points in the extension manifest. See [Edit the Extension Manifest](#) for details on this file.
9. Build any associated templates following the best practices and instructions detailed in [HTML Best Practices for Themes](#).

Add a Child View

❗ Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

A child view is simply a view within a parent view.

For example, ProductDetails.QuickView.View is a parent view of ProductDetails.Options.Selector.View.

You can add child views with short or verbose syntax, in extensions that modify functionality in the Product Details Page, Product List Page, Cart, Layout, or Checkout components using the Extensibility API. Use short or verbose syntax depending on where in the parent view that you want to place your child view.

Note: This topic only shows how to add a new child view using the Extensibility API in an extension entry point. When you add a child view, you must also define the view and the template for the view. For more information about these tasks, see [Views](#) and [Logic-less Templates and Handlebars.js](#).

In addition, if you add a child view to a view rendered as a Backbone.Collection, the same child view is added to all the views that belong to that collection.

Before You Begin

Before you add a child view, complete the following steps:

- [Choose Short or Verbose Syntax](#) – you can use the `addChildView()` (short) or `addChildViews()` (verbose) methods to add a child view.
- [Find the ID of the Placeholder](#) – find the placeholder name for where you want to place the child view. This placeholder can be either:
 - `data-view` – identifies where to add a child view.
 - `data-cms-area` – identifies where to drop a CCT using Site Management Tools and also where to add a child view.

If adding a child view in a `data-cms-area`, add the prefix `cms:` to the value of the `data-cms-area`. For example:

```
1 | pdp.addChildView('cms:header_banner_top', function() { ... });
```

Choose Short or Verbose Syntax

Use the [Short Syntax](#) to add your child view to the default view of the component and append the child view to the existing views inside of a specific placeholder. Use the [Verbose Syntax](#) to select the main view id of the component to which you want to add your view and set the order of the child views.

Short Syntax

Use this method to add your view to the default main view id for the component and append the view to the existing views inside if a specific placeholder.

With this method, you only need to specify the following information:

- Component name.
- ID of the placeholder. If your placeholder is a `data-view`, the ID is the value of that attribute. If the placeholder is a `data-cms-area`, the ID is the value of that attribute (with the prefix `cms:`).
- Name of the constructor function.

To add a child view using the short syntax and the `addChildView()` method, use the following syntax:

```
1 | var <component variable> = container.getComponent(<component name>);
2 | if(<component variable>)
3 | {
4 |   cart.addChildView('<placeholder ID>', function <constructor name>()
5 |   {
6 |     return new <view name>();
7 |   });
8 | }
```

The following list describes the variables in the syntax above:

- `<component variable>`. This JavaScript variable that represents an instance of the component to which you want to add the view, for example, `cart`.
- `<component name>`. This is the name of the component to which you want to add the view. For example, `Cart`.
- `<placeholder id>`. This specifies the placeholder to which you want to append the child view.
- `<constructor function>`. This is the constructor function.
- `<view name>`. This is a reference to the view (defined in the `define` portion of the view).

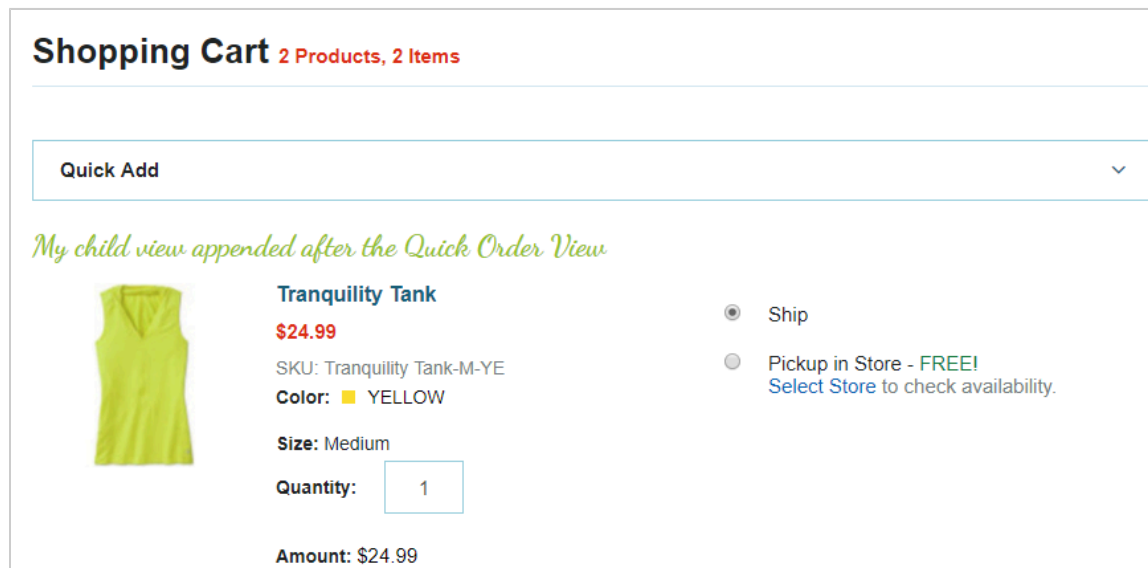
Example

Using the short syntax, you want to append a view to a placeholder inside the default view of the `Cart` component. Using this syntax and filling in the variables for this example, the following snippet appends a child view to the `Quick.Order` placeholder in the `Cart` component:

```
1 var cart = container.getComponent('Cart');
2 if(cart)
3 {
4   cart.addChildView('Quick.Order', function myViewConstructor()
5   {
6     return new ExtensionView();
7   });
8 }
```

After Adding a Child View

The following image depicts the `CartDetailedView` after deploying your change:



Verbose Syntax

Use this method to select the main view id of the component that you want to add your view and the order that the new view appears inside the specific placeholder for the main view id.

With this method, you specify the following properties:

- Main view id of the component to which you want to add the view
- The id of the placeholder where you want to add your component

This placeholder can be either:

- data-view – identifies where to add a child view.
- data-cms-area – identifies where to drop a CCT using Site Management Tools and also where to add a child view.
- The index that specifies the order to add the view inside the placeholder
- An ID for the child view

To add a child view using the verbose syntax with the `addChildViews()` method, use the following syntax:

```

1 var <component variable> = container.getComponent('<component name>');
2 if(<component variable>)
3 {
4   cart.addChildViews(
5     <main view id>,
6     {
7       '<placeholder id>': {
8         '<new view id>': {
9           childViewIndex: <index>
10          ,   childViewConstructor: function ()
11            {
12              return new <constructor method>();
13            }
14          }
15        }
16      }
17    );
18  }

```

The following list describes the variables in the above syntax:

- <component variable>. This JavaScript variable that represents an instance of the component to which you want to add the view, for example, `cart`.
- <component name>. This specifies the name of the component to which you want to add the view. For example, `Cart`.
- <main view ID>. This is the main view id for the component to which you want to add the view. For example, `cart.CART_MINI_VIEW`.
- <placeholder id>. This specifies the placeholder ID to where you want to add a child view.
- <index>. This specifies the indicator for the location to add the view. Use one of the following values:
 - Use a value of less than 10 to add the view before the placeholder. For example, 5.
 - Greater than 10. Add the view after the placeholder.

`childViewConstructor` is a constructor function that returns a `View`.

Example

Using the verbose syntax, you want to add a child view to the Product Details Quick View (`PDP_QUICK_VIEW`), but not in the Full View (`PDP_FULL_VIEW`).

```

1 var cart = container.getComponent('Cart');
2 if(cart)
3 {
4   cart.addChildViews(
5     cart.CART_MINI_VIEW,
6     {
7       'Header.MiniCartItemCell': {
8         'MyCustomView': {
9           childViewIndex: 5
10          ,   childViewConstructor: function ()
11            {

```

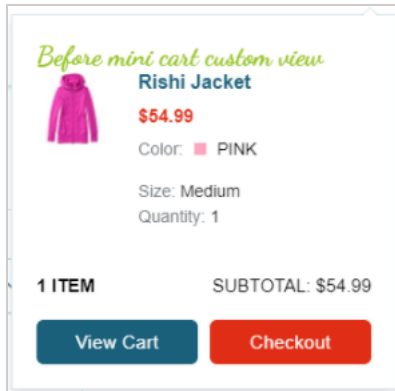
```

12     return new ExtensionView();
13   }
14 }
15 }
16 }
17 );
18 }

```

After Adding a Child View

The following image shows the new `ExtensionView` that appears before the default view inside the `Header.MiniCartItemCell`



Find the ID of the Placeholder

Find the placeholder ID where you want to place the view. The placeholder can be either:

- `data-view` – identifies where to add a child view. Example: `Facets.BrowseView`
- `data-cms-area` – identifies where to add a Custom Content Type as a child view using Site Management Tools. Example: `header_banner_top`

Use one of the following procedures to find the placeholder ID:

- [Code from a Template Method](#)
- [Browser Code Inspector Method](#)



Note: the `data-view` and `data-cms-area` are both are properties of a `div` element.

Code from a Template Method

When you download the base theme before you start developing extensions, the `gulp theme:fetch` command creates the `Extras/<theme name>/Modules` directory inside the workspace development directory.

The `Modules/` directory contains the templates and Sass files for each module in the application.

You can use these templates to find the correct `data-view` placeholder to which you want to add the child view.

For example, you want to add a child view to the **Product Options** area in the PDP Component. The following example is an excerpt from the `product_details_full.tpl` with a `data-view` named `Product.Options`. This file is located in the `product_details` module of the SuiteCommerce Base Theme source.

```

1 | // ...

```

```

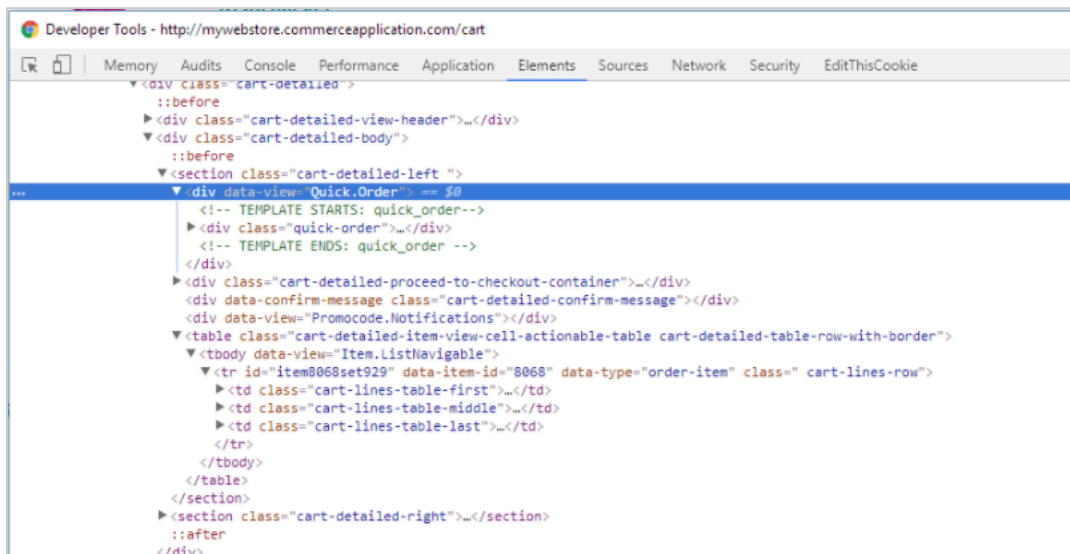
2 <div class="product-details-full-main">
3   {{#if isItemProperlyConfigured}}
4     <form id="product-details-full-form" data-action="submit-form" method="POST">
5
6       <section class="product-details-full-info">
7         <div id="banner-summary-bottom" class="product-details-full-banner-summary-bottom"></div>
8       </section>
9
10      <section data-view="Product.Options"></section>
11
12      <div data-view="Product.Sku"></div>
13      // ...

```

Browser Code Inspector Method

Use the code inspector in your browser to locate the data-view attribute in the area of the user interface to which you want to add the child view.

The following example shows the Google Chrome Developer Tools with the Quick-Order data-view attribute identified:



To add a child view in an extension:

1. Open the entry point file for the extension.

Although not always necessary, a common practice is to declare the child view in the `mountToApp()` function in the entry point.

Note: If you have not already done so, add the child view as a dependency.

2. Select the correct component and get an instance of the component.

For example, to add a child view in the Cart component, get an instance of the Cart component with the following code in your entry point file:

```

1 return {
2   mountToApp: function mountToApp (container)
3   {
4     var cart = container.getComponent('Cart');
5   }
6 }

```

For more information about accessing a component, see [Instantiating API Components](#).

3. Find the main view id for the component to which you want to add the view.

For example, you might want to only add the view into the Full View for the Product Details Page. The following table lists the available view IDs for each component:

Component	Main View ID	Name
Product Details Page (PDP)	PDP_FULL_VIEW (default)	ProductDetails.Full.View
	PDP_QUICK_VIEW	ProductDetails.QuickView.View
Product List Page (PLP)	PLP_VIEW	Facets.Browse.View
Cart	CART_VIEW (default)	Cart.Detailed.View
	CART_MINI_VIEW	Header.MiniCart.View
Checkout	WIZARD_VIEW	Wizard.View

Note: To add a view into any other area of the user interface, use the `LayoutComponent` component. If you add a child view with the `Layout` component in a specific placeholder id, it will be shown every time when that placeholder id appears in any page.

4. If you want to add the child view to the default main view id, add the child view with the [Short Syntax](#). If you want to choose the main view id, add the child view with the [Verbose Syntax](#).

Update JSON-LD After Adding or Modifying Views

If you have configured SuiteCommerce to use JSON-LD as your preferred structured data markup, you might need to update JSON-LD for the view. Data in the JSON-LD object, embedded in the `<head>` tag of a product details page, should be consistent with information displayed in the view. If your extension adds information to a view (or removes information), use the `modifyViewJsonLd()` method of the `Visual` component in the extensibility API to update the JSON-LD. See the [extensibility API reference](#) for details about `modifyViewJsonLd()`.

For more information about configuring JSON-LD in SuiteCommerce, see the help topic [Structured Data Markup](#).

Replace a Child View

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

If you want to make significant changes to any child view in one of the default templates, best practice is to replace the existing view with a custom view that contains the required functionality. You must add a child view in the same placeholder with the same ID of the view that you want to replace.

Note: The developer is responsible for maintaining the view functionality when migrating to new SuiteCommerce releases to ensure that the new SuiteCommerce functionality does not break the custom view code.

To replace a child view:

1. Obtain the main view id and the placeholder ID for the view that you want to replace. See [Before You Begin](#).

2. In the entry point file for the extension, use the `addChildViews()` method to replace the view. See [Verbose Syntax](#)

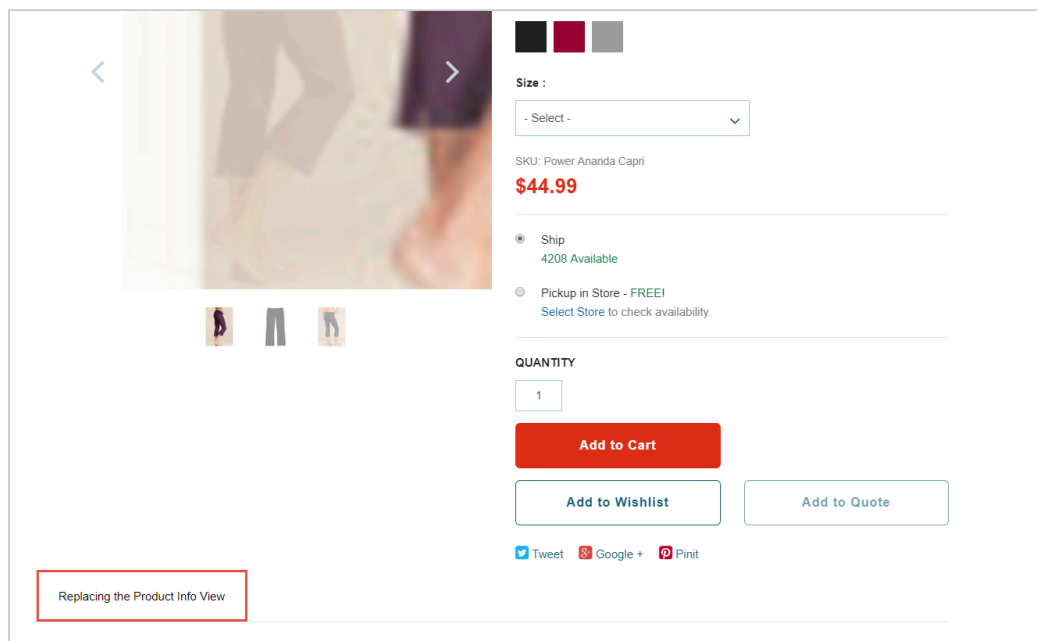
The following example shows how to replace the `Product.Information` view on the Product Details Page with the `ExtensionView` view.

```

1 | var pdp = container.getComponent('PDP');
2 | if(pdp)
3 | {
4 |     pdp.addChildViews(
5 |         pdp.PDP_FULL_VIEW
6 |         , {
7 |             'Product.Information': {
8 |                 'Product.Information': {
9 |                     childViewIndex: 10
10 |                     , childViewConstructor: function(){
11 |                         return new ExtensionView();
12 |                     }
13 |                 }
14 |             }
15 |         }
16 |     );
17 | }

```

The following image shows the result of replacing the view. The change is indicated by a red rectangle:



Work with Events in an Extension

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

In the Extensibility API, events correspond to operations that occur in the following components:

- `ProductListPageComponent`
- `ProductDetailsComponent`

- CartComponent and BackendCartComponent
- CheckoutComponent

Each operation triggers two events: an event that occurs before the operation starts and an event that occurs after the operation completes. You can subscribe to these events and then direct an extension to act on the results of the operation. Each component includes listeners that inherit from the `CancelableEvents` class; if you subscribe to the before event, you can cancel the operation.

Events can trigger due to user action or another process programmatically triggering the event. For example, if a user adds an item to the cart or an extension calls the `CartComponent.addLine(line)` method, both the `beforeAddLine` and `afterAddLine` events trigger. However, if you cancel the operation, the `afterAddLine` event does not occur.

The following lists the events for each component:

ProductListPageComponent

beforeOptionSelection
afterOptionSelection
beforeQuantityChange
afterQuantityChange
beforeImageChange
afterImageChange

ProductDetailsComponent

beforeOptionSelection
afterOptionSelection
beforeQuantityChange
afterQuantityChange
beforeImageChange
afterImageChange

CartComponent

afterAddLine
beforeAddLine
afterAddPayment
beforeAddPayment
afterAddPromotion
beforeAddPromotion
afterClearEstimateShipping
beforeClearEstimateShipping

afterEstimateShipping
beforeEstimateShipping
afterRemoveLine
beforeRemoveLine
afterRemovePromotion
beforeRemovePromotion
afterSubmit
beforeSubmit
afterUpdateLine
beforeUpdateLine

BackendCartComponent

afterAddLine
beforeAddLine
afterAddPayment
beforeAddPayment
afterAddPromotion
beforeAddPromotion
afterClearEstimateShipping
beforeClearEstimateShipping
afterEstimateShipping
beforeEstimateShipping
afterRemoveLine
beforeRemoveLine
afterRemovePromotion
beforeRemovePromotion
afterSubmit
beforeSubmit
afterUpdateLine
beforeUpdateLine

CheckoutComponent

afterAddModuleToStep
beforeAddModuleToStep

afterAddStep
 beforeAddStep
 afterAddStepsGroup
 beforeAddStepsGroup
 afterSetCurrentStep
 beforeSetCurrentStep

Subscribe to Events

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Use the `<component>.on(event, event_handler)` method for a component to create a listener and subscribe to an event.

Each listener takes two arguments:

- `event` – declares the name of the event to listen to.
- `event_handler` – declares the function to process the results of the operation that triggered the event. In general, each event handler receives a single argument.

The following code example shows how to subscribe to the `afterOptionSelection` event for the PDP and then log the result to the browser console:

```
1 var pdp = container.getComponent('PDP');
2
3 // ...
4
5 pdp.on('afterOptionSelection', function(event) {
6   console.log('option ' + event.optionCartId + ' was selected with value ' + event.value)
7 });
```

Cancel the Operation for an Event

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Because the components inherit the `CancelableEvents` class, you can direct an extension to cancel the operation associated with an event by throwing an error or returning a rejected Deferred promise. Subscribe to the event. When the event occurs, cancel the operation for the event.

The following code example shows how to cancel an operation in the PDP by throwing an error if a product option is selected and `someCondition` is true:

```
1 pdp_component.on('beforeOptionSelection', function() {
2   if(someCondition)
3     throw new Error('Canceling the select option operation.');
```

The following code example shows how to cancel an operation in the PDP by returning a rejected promise if a product option is selected and `someCondition` is true:

```

1 | pdp_component.on('beforeOptionSelection', function() {
2 |   if(someCondition)
3 |     return jQuery.Deferred().reject(new Error('Canceling the select option operation.'));
4 | });

```

Note: If you cancel the operation associated with an event, the after event does not occur.

Edit the Extension Manifest

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Your extension's Workspace directory includes a manifest.json file, which includes all the information required to compile resources for the extension.

../Workspace/<EXTENSION_DIRECTORY>/manifest.json

This file is auto-generated when you execute the `gulp extension:create` command and includes all JavaScript, JSON, SuiteScript, HTML templates, Sass, and assets required to compile and activate your extension.

Manual Edits

This file lists all JavaScript, JSON, SuiteScript, HTML templates, Sass, and assets related to your extension. Although this file is automatically generated, you may need to update it manually if adding Sass entry points for any newly introduced Sass files or changing CCT labels.

Warning: Potential data loss. Besides compiling and deploying your extension to a local server, the `gulp extension:local` and `gulp extension:deploy` commands update the extension's manifest.json file and overwrites any manual changes you made to this file.

To preserve manual changes to manifest.json use the following commands to test your extension locally or deploy to NetSuite:

- `gulp extension:local --preserve-manifest`
- `gulp extension:deploy --preserve-manifest`

Extension Metadata

The first entries in the manifest file include metadata about the extension itself. These fields are automatically populated when you initially run the `gulp extension:create` command.

```

1 | {
2 |   "name": "MyCoolExtension",
3 |   "fantasyName": "My Cool Extension!",
4 |   "vendor": "Acme",
5 |   "type": "extension",
6 |   "target": "SCA,SCS",
7 |   "target_version": {
8 |     "SCA": "19.2.0",
9 |     "SCS": "19.2.0"
10 |  },
11 |   "version": "1.0.0",
12 |   "description": "My cool extension does magic!",

```

```
13 | //...
```

- **name** (string) – uniquely identifies the name of the extension. This field is required. This must be alphanumeric characters without spaces.
- **fantasyName** (string) – identifies the label for the extension. This field can include special characters.
- **vendor** (string) – identifies the vendor as a string. This field is required.
- **type** (string) – indicates the type as an **extension**. This field is required.
- **target** (comma-separated string) – indicates the SuiteCommerce or SCA applications supported by the extension. This field is required.
- **target_version** (array) – indicates the SuiteCommerce or SCA application versions supported by the extension. This field is required.
- **version** (string) – indicates the version of the extension, such as 1.0.0. This field is required.
- **description** (string) – provides a description of the extension as it appears in NetSuite. This field is optional.



Important: To take advantage of extension update requirements, version numbers should follow this format, where d is a single digit: dd.dd.dd.

Assets

The **assets** object defines paths to the images and fonts located in the extension directory's **assets** folder. This defines an array for each image and font used. These paths are relative to the extension's **assets** folder path. Extensions treat services as assets. These are created on NetSuite servers when you activate/deploy your extension.

If the extension developer tools detect a file name with the pattern **XYZ.ServiceController.js**, they create the service (.ss) file and add a reference in the manifest. Later, when you activate the extension, the .ss file deploys to the backend as an asset.

```
1 | //...
2 | "assets": {
3 |   "img": {
4 |     "files": []
5 |   },
6 |   "fonts": {
7 |     "files": []
8 |   },
9 |   "services": {
10 |    "files": [
11 |      "services/MyCoolModule.Service.ss",
12 |      "services/AdditionalCoolModule.Service.ss"
13 |    ]
14 |  }
15 | },
16 | //...
```

Configuration

The **configuration** object defines paths to the JSON files in your extension directory's **Configuration** folder.

```
1 | //...
2 | "configuration": {
3 |   "files": [
```

```

4     "Modules/MyCoolModule/Configuration/MyCoolModule.json",
5     "Modules/AdditionalCoolModule/Configuration/AdditionalCoolModule.json"
6   ],
7 },
8 //...

```

Templates

The `templates` object lists all HTML template files included in the extension by application. The `application` object includes one object per application (shopping, myaccount, and checkout). Each application lists each file in the `files` array.

```

1 //...
2 "templates": {
3   "application": {
4     "shopping": {
5       "files": [
6         "Modules/MyCoolModule/Templates/acme_mycoolextension_mycoolmodule_list.tpl",
7         "Modules/MyCoolModule/Templates/acme_mycoolextension_mycoolmodule_edit.tpl",
8         "Modules/AdditionalCoolModule/Templates/acme_mycoolextension_additionalcoolmodule_list.tpl",
9         "Modules/AdditionalCoolModule/Templates/acme_mycoolextension_additionalcoolmodule_edit.tpl"
10      ]
11    },
12    "myaccount": {
13      "files": [
14        "Modules/MyCoolModule/Templates/acme_mycoolextension_mycoolmodule_list.tpl",
15        "Modules/MyCoolModule/Templates/acme_mycoolextension_mycoolmodule_edit.tpl",
16        "Modules/AdditionalCoolModule/Templates/acme_mycoolextension_additionalcoolmodule_list.tpl",
17        "Modules/AdditionalCoolModule/Templates/acme_mycoolextension_additionalcoolmodule_edit.tpl"
18      ]
19    },
20    "checkout": {
21      "files": [
22        "Modules/MyCoolModule/Templates/acme_mycoolextension_mycoolmodule_list.tpl",
23        "Modules/MyCoolModule/Templates/acme_mycoolextension_mycoolmodule_edit.tpl",
24        "Modules/AdditionalCoolModule/Templates/acme_mycoolextension_additionalcoolmodule_list.tpl",
25        "Modules/AdditionalCoolModule/Templates/acme_mycoolextension_additionalcoolmodule_edit.tpl"
26      ]
27    }
28  },
29 },
30 //...

```

Sass

The `sass` object declares the paths to each application entry point and all Sass files to be loaded when you deploy. If you introduce a new Sass file into your extension workspace, you must manually add it to the `files` array within the `sass` object.

Note: If manually listing Sass files, declare each file in an order that makes the most semantic sense within the Sass hierarchy. If the developer tools automatically added a Sass file, the entry is always at the end of the `files` array. However, your dependencies may require that this order be changed. Check the order in which a file was added to the array because the developer tools always add a file to the end of the array.

```

1 //...
2 "sass": {
3   "entry_points": {
4     "shopping": "Modules/MyCoolModule/Sass/_mycoolextension-mycoolmodule.scss",
5     "myaccount": "Modules/MyCoolModule/Sass/_mycoolextension-mycoolmodule.scss",
6     "checkout": "Modules/MyCoolModule/Sass/_mycoolextension-mycoolmodule.scss"

```

```

7     },
8     "files": [
9         "Modules/MyCoolModule/Sass/_mycoolextension-mycoolmodule.scss",
10        "Modules/MyCoolModule/Sass/_mycoolmodule.scss",
11        "Modules/AdditionalCoolModule/Sass/_mycoolextension-additionalcoolmodule.scss",
12        "Modules/AdditionalCoolModule/Sass/_additionalcoolmodule.scss"
13        "Modules/AdditionalCoolModule/Sass/sass-extension_myNewSassFile.scss"
14    ]
15 },
16 //...

```

JavaScript

The javascript object declares the paths to each application entry point and all JavaScript files to be loaded when you deploy.

```

1 //...
2 "javascript": {
3     "entry_points": {
4         "shopping": "Modules/MyCoolModule/JavaScript/Acme.MyCoolExtension.MyCoolModule.js",
5         "myaccount": "Modules/MyCoolModule/JavaScript/Acme.MyCoolExtension.MyCoolModule.js",
6         "checkout": "Modules/MyCoolModule/JavaScript/Acme.MyCoolExtension.MyCoolModule.js"
7     },
8     "application": {
9         "shopping": {
10            "files": [
11                "Modules/MyCoolModule/JavaScript/Acme.MyCoolExtension.MyCoolModule.js",
12                "Modules/MyCoolModule/JavaScript/MyCoolModule.Router.js",
13                "Modules/MyCoolModule/JavaScript/MyCoolModule.List.View.js",
14                "Modules/MyCoolModule/JavaScript/MyCoolModule.Edit.View.js",
15                "Modules/MyCoolModule/JavaScript/MyCoolModule.Collection.js",
16                "Modules/MyCoolModule/JavaScript/MyCoolModule.Model.js",
17                "Modules/AdditionalCoolModule/JavaScript/Acme.MyCoolExtension.AdditionalCoolModule.js",
18                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Router.js",
19                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.List.View.js",
20                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Edit.View.js",
21                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Collection.js",
22                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Model.js"
23            ]
24        },
25        "myaccount": {
26            "files": [
27                "Modules/MyCoolModule/JavaScript/Acme.MyCoolExtension.MyCoolModule.js",
28                "Modules/MyCoolModule/JavaScript/MyCoolModule.Router.js",
29                "Modules/MyCoolModule/JavaScript/MyCoolModule.List.View.js",
30                "Modules/MyCoolModule/JavaScript/MyCoolModule.Edit.View.js",
31                "Modules/MyCoolModule/JavaScript/MyCoolModule.Collection.js",
32                "Modules/MyCoolModule/JavaScript/MyCoolModule.Model.js",
33                "Modules/AdditionalCoolModule/JavaScript/Acme.MyCoolExtension.AdditionalCoolModule.js",
34                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Router.js",
35                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.List.View.js",
36                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Edit.View.js",
37                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Collection.js",
38                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Model.js"
39            ]
40        },
41        "checkout": {
42            "files": [
43                "Modules/MyCoolModule/JavaScript/Acme.MyCoolExtension.MyCoolModule.js",
44                "Modules/MyCoolModule/JavaScript/MyCoolModule.Router.js",
45                "Modules/MyCoolModule/JavaScript/MyCoolModule.List.View.js",
46                "Modules/MyCoolModule/JavaScript/MyCoolModule.Edit.View.js",
47                "Modules/MyCoolModule/JavaScript/MyCoolModule.Collection.js",
48                "Modules/MyCoolModule/JavaScript/MyCoolModule.Model.js",
49                "Modules/AdditionalCoolModule/JavaScript/Acme.MyCoolExtension.AdditionalCoolModule.js",
50                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Router.js",
51                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.List.View.js",
52                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Edit.View.js",
53                "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Collection.js",

```

```

54 |         "Modules/AdditionalCoolModule/JavaScript/AdditionalCoolModule.Model.js"
55 |     ]
56 | }
57 | }
58 | },
59 | //...

```

Ssp-libraries

The ssp-libraries object declares the paths to all SuiteScript files to be loaded when you deploy.

```

1 | //...
2 | "ssp-libraries": {
3 |     "entry_point": "Modules/MyCoolModule/SuiteScript/Acme.MyCoolExtension.MyCoolModule.js",
4 |     "files": [
5 |         "Modules/MyCoolModule/SuiteScript/Acme.MyCoolExtension.MyCoolModule.js",
6 |         "Modules/MyCoolModule/SuiteScript/MyCoolModule.ServiceController.js",
7 |         "Modules/MyCoolModule/SuiteScript/MyCoolModule.Model.js",
8 |         "Modules/AdditionalCoolModule/SuiteScript/Acme.MyCoolExtension.AdditionalCoolModule.js",
9 |         "Modules/AdditionalCoolModule/SuiteScript/AdditionalCoolModule.ServiceController.js",
10 |        "Modules/AdditionalCoolModule/SuiteScript/AdditionalCoolModule.Model.js"
11 |     ]
12 | }
13 | //...

```

Suitescript 2

The suitescript2 object declares the paths to SuiteScript 2.0 services to be loaded when you deploy. This object defines an array that can include .js or .ss files.

```

1 | //...
2 | "suitescript2": {
3 |     "files": [
4 |         "Modules/MyCoolModule/SuiteScript2/MyCoolModule.Service.ss"
5 |     ]
6 | }
7 | //...

```

CCT

If your extension is set up to include a CCT, your manifest's metadata includes specific information regarding your CCT.

The cct array declares the specific information required to build a CCT in NetSuite. This includes the following attributes:

- label (required) – lists the name you gave your CCT.
- icon – lists the path of the icon to be displayed in SMT for the CCT.
- settings_record (required) – lists the **ID** of the custom record you associate with this CCT.
- registercct_id (required) – lists the **Name** field of the CMS Content Type Record for your CCT. This is also the value of the id property within the registerCustomContentType() method of your CCT module's entry point JavaScript file.
- description – lists the description you gave your CCT.

```

1 | //...

```

```

2  "cct": [
3    {
4      "label": "This is My CCT!",
5      "icon": "img/cct_acme_mycct_icon.svg",
6      "settings_record": "customrecord_cct_acme_mycct",
7      "registercct_id": "cct_acme_mycct",
8      "description": "My cool CCT does magic!"
9    },
10   {
11     "label": "This is My Second CCT",
12     "icon": "img/cct_acme_mycct2_icon.svg",
13     "settings_record": "customrecord_cct_acme_mycct2",
14     "registercct_id": "cct_acme_mycct2",
15     "description": "My cool CCT does magic!"
16   }
17 ]
18 }

```

Page

If your extension is set up to include a new CMS Page Type, you must manually add a page type definition within your extension's manifest.json. When an Extension SuiteApp is installed into a NetSuite account, this entry ensures that the CMS Page Type record is created in NetSuite. This is required if creating a new Page Type for use with Site Management Tools (SMT).

The page object includes the types array, which introduces each new page type included in the extension. All fields correlate to the CMS Page Type record in NetSuite:

- **name** (required) – Sets the name of the Page Type. This becomes the name of the CMS Page Type record. This must match the name attribute when registering your Page Type with `PageTypeComponent.registerPageType()` in your extension. See [Register Page Types and Templates for SMT](#) for details. This is required.
- **displayName** (required) – This string sets the label displayed in SMT for the Page Type when creating a new page.
- **description** – (optional) This string sets the description of the Page Type.
- **baseUriPath** (optional) – This string sets the base URL path for all the pages of this page type. For example, if you have a page type called **blog** and a base URL path of **blog**, all blog pages are accessed by `mysite.com/blog/[page url]`.
- **settingsRecord** (optional) – This string sets the ID of any custom record you associate with this page type, inheriting the custom record settings when you add or edit the page type.

```

1  //...
2  "page": {
3    "types": [
4      {
5        "name": "blog",
6        "displayName": "Blog",
7        "description": "This is a blog page type",
8        "baseUriPath": "blog",
9        "settingsRecord": "custrec_page_type_test"
10     },
11     {
12       "name": "other-page-type",
13       "displayName": "My other page type",
14       "description": "This is another page type",
15     }
16   ]
17 }
18 //...

```


Test and Deploy Your Extension

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

After activating your extension for a domain in NetSuite, you can test your extension code on a local server for debugging purposes. When you are satisfied with your extension code, you can then deploy it to your website using the extension developer tools.

- Test an extension – Test your extension locally with the gulp command `gulp extension:local`. This command automatically recompiles the extension files with a watch task when you change files in your extension development directory.
- Deploy an extension – Upload your extension's source files to your NetSuite account with the gulp command `gulp extension:deploy`. See [Deploy an Extension to NetSuite](#) for more information

For a list of all gulp commands available with the extension developer tools, see [Gulp Command Reference for Theme and Extension Developer Tools](#).

Test an Extension on a Local Server

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

You can test your extension customizations on a local server before deploying to NetSuite. The local server is installed as part of the Node.js installation and uses the Express web framework.

```
gulp extension:local
```

This command compiles all source files into combined files within a LocalDistribution/tmp directory.

When the server starts, Gulp.js initializes watch tasks that listen for changes to files in the JavaScript, Templates, or Sass directories. When you save changes to a file, gulp automatically recompiles the source files and updates the LocalDistribution directory. Gulp also outputs a message to the console when an update occurs.



Important: Typically, you test your code locally before deploying to a live site in a NetSuite account. However, if you are developing an extension that includes SuiteScript or configuration (JSON) files, you must deploy your files to your account and activate the extension for these changes to be accessible by the local server. SuiteScript includes services, which do not exist in your account's backend until you deploy them. Likewise, changes to configuration JSON files do not apply to a domain until deployed. See the help topic [Manage Themes and Extensions](#).



Note: If you modify any manifest files, you must restart your local server to see changes.

To test your extension on a local server:

1. If you have not already done so, fetch the active theme.
This is required before you can test an extension locally. See [Fetch Active Theme and Extension Files](#) for details.
2. Open a command line or terminal and access the top-level development directory. This is the same directory created when you extracted the Extension Developer Tools.
3. Run the following command:
`gulp extension:local`

If this is the first time you are running `gulp extension:local` in this directory, this command creates a subdirectory called `LocalDistribution`. It then compiles the source files and outputs them to this directory.

 **Warning:** Potential data loss. Besides compiling and deploying your extension to a local server, the `gulp extension:local` command updates the `manifest.json` file for the extension. This action overwrites any manual changes you made to this file. To preserve any manual changes to your `manifest.json` file, run the `gulp extension:local --preserve-manifest` command instead. See [Edit the Extension Manifest](#) for details on this file.

4. Navigate to the local version of the application using one of the following URLs:
 - **Shopping:** `http://<DOMAIN_NAME>/c.<ACCOUNT_ID>/<SSP_APPLICATION>/shopping-local.ssp`
 - **My Account:** `http://<DOMAIN_NAME>/c.<ACCOUNT_ID>/SSP_APPLICATION/my_account-local.ssp`
 - **Checkout:** `http://<DOMAIN_NAME>/c.<ACCOUNT_ID>/SSP_APPLICATION/checkout-local.ssp`

In the above URL patterns, you must replace the following variables with values for your specific environment:

<i>DOMAIN_NAME</i>	Replace this value with the domain name configured for your NetSuite website record.	For example: <code>http://www.mysite.com</code>
<i>ACCOUNT_ID</i>	Replace this value with your NetSuite account ID.	For example: <code>c.123456</code>
<i>SSP_APPLICATION</i>	Replace this value with the URL root that you are accessing. For SuiteCommerce implementations, this variable should read scs . For SuiteCommerce Advanced (SCA), this variable equals the URL root of the SCA implementation.	SuiteCommerce example: <code>scs</code> SuiteCommerce Advanced examples: ■ <code>sca-dev-aconcagua</code> ■ <code>sca-dev-2019-2-0</code>

Complete URLs for SuiteCommerce implementations would look like these examples:

- `http://www.mysite.com/c.123456/scs/shopping-local.ssp`
- `http://www.mysite.com/c.123456/scs/my_account-local.ssp`

Complete URLs for SuiteCommerce Advanced implementations would look like these examples:

- `http://www.mysite.com/c.123456/sca-dev-aconcagua/shopping-local.ssp`
- `http://www.mysite.com/c.123456/sca-dev-2019-2-0/shopping-local.ssp`

 **Note:** When accessing the secure checkout domain using HTTPS on the local server, you must use a different URL. See [Secure HTTP \(HTTPS\) with the Local Server](#) for more information.

Deploy an Extension to NetSuite

 **Applies to:** SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

The term **deploy** refers to what happens when you use Gulp.js to upload source files and any custom changes to a hosted site. To do this, you use the following command:

```
gulp extension:deploy
```

This command validates your code, copies them into a local DeployDistribution directory, and updates the manifest.json. The developer tools then upload these files to your NetSuite account, making them available for activation.

Note: For more Gulp.js commands used with extensions, see [Extension Developer Gulp Commands](#).

To deploy your extension to NetSuite:

1. If you have not already done so, fetch the active theme.

This is required before you can deploy an extension. See [Fetch Active Theme and Extension Files](#) for details.

2. In your local developer environment, open a command line or terminal and access the top-level development directory.
3. Run the following command:

```
gulp extension:deploy
```

Note: For release 2021.2.1 and later accounts that have account-specific domains, specify your account number with the gulp command to enable access to the NetSuite account. For more information about using `gulp extension:deploy --account`, see [Extension Developer Gulp Commands](#).

Warning: Potential data loss. Besides compiling and deploying your extension to NetSuite, the `gulp extension:deploy` command updates the manifest.json file for the extension. This action overwrites any manual changes you made to this file. To preserve any manual changes to your manifest.json file, run the `gulp extension:deploy --preserve-manifest` command instead. See [Edit the Extension Manifest](#) for details on this file.

4. When prompted, enter the following information.
 - Unless otherwise directed, create all entries using only alphanumeric characters without spaces.
 - These prompts appear the first time you deploy a new extension to NetSuite. To provide new or changed information, use the `gulp extension:deploy --to` command to deploy your changes.

Select Extension	If you have more than one extension in your developer environment, choose the extension you are deploying.
Authentication ID	Enter an ID to represent your NetSuite account and role. If you have already created an authentication ID in a previous fetch or deploy operation, the extension developer tools do not prompt you for an ID and will continue to use the existing ID for each subsequent fetch or deploy. To deploy to a NetSuite account that is different from the account used to create the existing authentication ID, use the <code>--to</code> parameter. When you use the <code>--to</code> parameter, you can choose a different authentication ID or create a new one. For more information, see Token Based Authentication.
Account and Role	If you are creating a new authentication ID, you need to select the NetSuite account and role to use with that new authentication ID. If using an existing ID, you are not prompted for this information. In NetSuite, select the account you want to deploy to and the role you want to use for authentication. Click the Allow button to generate the authentication token and secret.

	 Note: You must specify a role that provides the permissions needed to deploy to NetSuite. By default, the Administrator and SCDeployer roles provide these permissions. To learn more, including how to create a custom role that provides the required permissions, see Developer Tool Roles and Permissions .
Choose Your Website	Choose your site from the list.
Vendor	Enter your partner or business vendor name.
Name	Provide a name for your extension within your development environment.
Fantasy Name	Provide a name for your extension as you want it to appear in the NetSuite user interface. This can contain special characters and spaces.
Version	Add a version number for your extension.  Important: To take advantage of extension update requirements, assign a version number to your extensions. Version numbers should follow this format, where d is a single digit: dd.dd.dd.
Description	Provide some text to describe your extension. This string can contain special characters and spaces.
Select Supported Products	Select one of the following options: ■ SuiteCommerce Online – To create an extension for SuiteCommerce or SuiteCommerce Advanced. ■ SuiteCommerce InStore – To create an extension for SuiteCommerce InStore. An asterisk (*) identifies a selected option. Press the spacebar to select. Use arrow keys to scroll. Press the A key to select all. Press the I key to invert your selections.
Target Version	Using Commerce versioning schema, enter a string to declare the target versions to which this extension applies. The developer tools prompt you for a target version for each product specified earlier. Leaving these prompts blank results in compatibility with all versions of the associated Commerce products. See Declare Target Versions and SuiteScript 2.x Bundle Installation Script Type for more information.

5. Activate your new extension for the domain or domains of your choice. See the help topic [Manage Themes and Extensions](#) for instructions.

 **Important:** Any changes to your extension source code do not apply to your site until you activate the extension for a specific domain using the Manage Extensions wizard in NetSuite. The deploy process includes a notation reminding you of the domain and theme name to set during this process.

What Happens When You Deploy an Extension?

The deployment process is specific to uploading custom extensions to a location in your NetSuite file cabinet. This process does not compile any files or associate any files with a site or domain. To do that, you must activate a theme and include your extensions as necessary.

During the deployment process:

- The development tools copy all of your custom extension development files (modules and assets) into the DeployDistribution folder in your top-level development directory. If this directory does not exist, the development tools create it.

- The extension development tools validate your code.
- The extension development tools upload these files to your NetSuite file cabinet (maintaining the same organizational structure). These are simply your development files. Nothing is compiled or activated at this time.

 **Note:** The `gulp extension:deploy` command uploads your extension development files in groups of files called chunks. A single chunk, by default, contains 80 files. If needed, you can change the number of files in each chunk. See [Troubleshooting the Developer Tools](#) for more information.

Best Practices for Developing Commerce Extensions

The following topics describe general best practices to take into account when creating an extension for a SuiteCommerce or SuiteCommerce Advanced (SCA) website.

- [General Best Practices](#)
- [Use Custom Handlebars Helpers](#)
- [Use Multiple Modules in an Extension](#)

General Best Practices

When developing customizations for your Commerce websites, follow the best practices outlined in these sections:

- [Use Configuration Instead of Customization](#)
- [Use the Extensibility API Whenever Possible](#)
- [Use the SuiteCommerce Implementations of BackboneJS Modules](#)
- [Do Not Extend Extensions](#)
- [Working with Core Modules](#)
- [Get Configuration Values with the Environment Component](#)
- [Specify a Target Version for Extensions](#)

Use Configuration Instead of Customization

Before starting development work on an extension, consider whether SuiteCommerce already offers what you want to achieve with a custom extension. Instead of creating an extension, you may be able to configure your NetSuite account and the built-in features of the SuiteCommerce application. By configuring instead of customizing, you may benefit from a faster implementation time, tried and tested code, and continued support of the feature or functionality. Some of the ways in which you can configure SuiteCommerce include:

- Enabling and configuring built-in SuiteCommerce features such as personalized catalog views, commerce categories, and checkout flows.
- Changing settings on the configuration record and the website setup record to modify the look and feel of the website or the backend functionality.
- Using Site Management Tools (SMT) to configure the features and extensions that are enabled in the NetSuite account.

For more information, see the help topic [Configure Instead of Customize](#).

Use the Extensibility API Whenever Possible

Extensions in both SuiteCommerce and SuiteCommerce Advanced have access to the extensibility API. The extensibility API provides a stable layer between your custom extension and the core code of the SuiteCommerce application. The components in the extensibility API have methods and properties that are documented in the API reference and are managed by Oracle NetSuite to ensure maximum compatibility and consistency across SuiteCommerce sites and versions.

Although SuiteCommerce Advanced gives you access to the SuiteCommerce source code, you should still use the extensibility API whenever possible. Because SuiteCommerce Advanced is an unmanaged bundle (it is not upgraded automatically by Oracle NetSuite at the time of each release), you may not have access to all components and their methods and events. Check the [Overview](#) topic of SuiteCommerce Extensibility API for information about the components that are available in each release.

Extensibility API Components

Components in the extensibility API contain methods and events that enable an extension to interact with the website, as well as the underlying SuiteCommerce environment and configuration. Some components are general components that provide the foundation of an extension such as SCView or SCModel, whereas other components let you perform actions in a specific context. For example, you can use the PDP (product details page) component to get the stock information of an item. You can also listen for certain events such as when an option is selected or a quantity is changed on the page (see [Work with Events in an Extension](#) for more information).

Most components in the extensibility API typically provide:

- **Methods** – Getter and setter methods let you fetch and save data.
- **Events** – Many components provide ‘before’ and ‘after’ cancelable events that let you interrupt an action depending on the logic or criteria of your extension. For example, you could validate a promo code before it is applied to items in the cart.

The following example shows how the `on` method is used to listen for the `beforeQuantityChange` cancelable event. When a website visitor changes the quantity of an item on the PDP, the `beforeQuantityChange` event is fired. As one of the parameters to `on`, you specify a handler that is called when the event occurs. In this case, the handler checks if the quantity is a number and displays a browser alert if `isNaN()` returns false. A return value of false prevents the quantity from being changed.

```
1 var pdp = container.getComponent('PDP');
2
3 pdp.on('beforeQuantityChange', function() {
4   if (!_.isNaN(newquantity)) {
5     alert('That is not a valid quantity.');
```

Use the SuiteCommerce Implementations of BackboneJS Modules

The [Backbone.js](#) library is one of the core technologies on which a SuiteCommerce website relies. The extensibility API contains several components, however, that are intended to replace the usage of the Backbone modules. The SuiteCommerce components provide classes, methods, and events that are specifically designed for SuiteCommerce websites. More information about each component is available in the [extensibility API reference](#).

Component Usage	Available Since	Core Backbone Module
SCModel – SCModel lets you define a model for your extension.	SuiteCommerce 2020.1	Backbone.Model
SCView – Use SCView to define a view that renders content on a part of the website.	SuiteCommerce 2020.2	Backbone.View

Component Usage	Available Since	Core Backbone Module
SCFormView – SCFormView provides functions to validate form data, save form data, and display error messages related to form input.	SuiteCommerce 2020.2	Backbone.FormView
SCCollection – Use SCCollection to hold a set of models.	SuiteCommerce 2020.1	Backbone.Collection
SCCollectionView – SCCollectionView lets you display a collection (list) of items, usually in a grid pattern.	SuiteCommerce 2020.2	Backbone.CollectionView

Do Not Extend Extensions

You should avoid extending extensions created by Oracle NetSuite or other third-parties. You should also avoid extending the extensibility API components. Reasons to not extend extensions include:

- By extending an extension, you are effectively depending on an environment where a site administrator can disable any extension at any time in the NetSuite account.
- The developer of an extension may change its behavior, methods, or return values without warning. If your extension relies on the behavior or methods in another extension, your extension may stop working.

If you do want to extend an extension, however, you could consider the following approaches:

- If you own the extension you want to extend, consider creating an API within the extension code that can be called by the new extension.
- If you do not own the extension, consider creating an adapter for it and then let the new extension communicate with it through the adapter.

Working with Core Modules

When developing extensions for SuiteCommerce websites, you should avoid involving core modules to ensure your extensions function as expected when new versions of the SuiteCommerce bundle are pushed to your account. Read the following guidelines to learn more about practices to avoid.

Note: For SuiteCommerce Advanced (SCA) sites, you have more flexibility when it comes to working with SCA core modules in your extensions. However, the following guidelines apply to SCA sites, too, if you want to minimize the issues you may encounter when you migrate to a new SCA version or when you apply your extension to another website.

When working with core modules in your extensions, follow these guidelines:

- Do not include a core module in a `require()` or `define()` statement.
- Do not extend (`extend()`) a core module to add or overwrite a method or property.
- Do not wrap (`_.wrap()`) a core module's method to change its output.
- Do not modify a core module or object's prototype. For example, you should:
 - Use `addChildView()` to add a child view rather than `SomeView.prototype.childViews.MyChildView`.
 - Use `addToViewContextDefinition()` to add a new property to a view's context object, rather than `SomeView.prototype.getContext().MyNewProperty`.

The following guidelines apply to working with core modules in any production code:

- To access a core module or object's private method, use `application.getLayout()`. Do not use `application._layoutInstance`.

- Do not access the SC global variable.
- Do not edit core modules directly; use overrides instead.

Get Configuration Values with the Environment Component

Use the [Environment](#) component to get configuration values of the website instead of the SC.Configuration module or the SC global variable. For example, to determine the current add to cart behavior of the website, you should get the addToCartBehavior configuration value.

```
1 var env = container.getComponent('Environment');
2 var addToCartBehavior = env.getConfig('addToCartBehavior');
```

Specify a Target Version for Extensions

You should specify version compatibility in your extension to ensure that it can only be installed and activated on a compatible SuiteCommerce website. If the extension is not compatible with the website, it is hidden on the Activate Themes and Extensions page in the NetSuite account. To specify version compatibility, use the target_version key in the extension manifest file.

```
1 "target_version": {
2   "SCA": ">=21.2.0",
3   "SCS": ">=21.2.0"
4 }
```

See [Edit the Extension Manifest](#) for more information about manifest metadata.

When creating an extension for SuiteCommerce Advanced, you should also check the [extensibility API](#) to determine if a component or method is available in a particular version of SCA. In addition, when you instantiate a component, you can verify that it is available by checking its return value. If a component is not available, its return value is null.

```
1 var userProfile = container.getComponent('UserProfile');
2
3 if (userProfile) {
4   // Component is available
5 }
```

Use Custom Handlebars Helpers

The SuiteCommerce templating engine is powered by the Handlebars templating language. One of the benefits of Handlebars is that it enables you to include helper functions that perform simple logic in your templates.

Handlebars helpers are JavaScript functions. You give a function a name and a callback, and it processes your data as you want. Handlebars include a number of built-in helpers, which you can read more about in the [Handlebars documentation](#). SuiteCommerce includes proprietary helpers that are required for many features to function. If you are working with a SuiteCommerce Advanced (SCA) implementation, you can find these proprietary helpers in the HandlebarsExtras module in the SCA source files.

This topic describes how you can write custom helpers that go beyond the built-in helpers offered by Handlebars and the proprietary ones available with SuiteCommerce bundles.

Read the following sections for more information and examples:

- [Considerations](#)

- [Create a Simple Custom Helper](#)
- [Create a Block Helper](#)
- [Use Helpers for Debugging](#)

Considerations

Before adding custom helpers, consider whether helpers are the best way to accomplish your goal. One of the primary objectives of SuiteCommerce website architecture is to keep the templates (presentation layer) separate from any deeper code logic. If you're writing a pure Handlebars helper, ask yourself whether it should be a helper. If the necessary logic can reside in a view or a model, it likely should.

Another thing to keep in mind is whether you need a particular transformation to occur in the template. By creating a helper, you make the function you write available globally across templates. If a helper is only going to be used in one place, you might think about modifying the associated view's context object so that the transformation occurs there instead.

Create a Simple Custom Helper

The `registerHelper()` method on the Handlebars module is the main mechanism needed to create a helper and it operates like this:

```
Handlebars.registerHelper('helperName', function(param1, param2, etc)
{
  // Your code
  // return your value
});
```

To create a helper in an existing extension, follow these steps:

1. In your extension, create a new module directory named, `CustomHandlebarsHelpers`. Include a JavaScript folder within this new module directory so that your new extension files resemble this example:

```
Workspace/
  MyExtension
    Modules/
      CustomHandlebarsHelpers/
        JavaScript/
```

2. In the JavaScript folder shown in the preceding example, create a JavaScript file named `CustomHandlebarsHelpers.js`.
3. In the `CustomHandlebarsHelpers.js` file, add `Handlebars` as a dependency.
4. In the `CustomHandlebarsHelpers.js` file, use `Handlebars.registerHelper()` to define a new helper.
 - a. Give your helper a unique name.
 - b. Define the parameters for the helper, which are the values you expect to pass to it.
 - c. In the callback, include the code to perform the transformation.
 - d. Return the value.
5. When complete, your code should resemble the following sample:

```
define('CustomHandlebarsHelpers', [
  'Handlebars'
], function (
  Handlebars
){
  'use strict';
```

```
Handlebars.registerHelper('isTheBest', function (name) {
    return name + ' is the best!';
});
```

Or this example if you use TypeScript:

```
///
import Handlebars = require('../../../../Commons/Utilities/JavaScript/Handlebars');
Handlebars.registerHelper('isTheBest', function (name) {
    return name + ' is the best!';
});
```



Note: The path to Handlebars depends on where your extension resides. Also, remember that your custom module does not need to return an object that includes a `mountToApp` property for Handlebars. You are calling the Handlebars module directly and using a built-in method to modify the Handlebars object.

6. Edit the `manifest.json` file to include the new `CustomHandlebarsHelpers.js` file. For more information, see [Edit the Extension Manifest](#).

Test the Simple Helper

The best way to test a simple helper is in your browser's developer console. First, start your local server and then, navigate to the local version of your Commerce website. For step-by-step instructions, see [Test an Extension on a Local Server](#).

When you have your local Commerce website running in a browser, open the developer console. Access the simple helper you created by entering code like the following:

```
var Handlebars = require('Handlebars');
Handlebars.helpers.isTheBest('Pizza');
```

If the helper is working properly, the developer console returns:

```
> "Pizza is the best!"
```

If the helper you develop is more complex, you should test it in a template.

Create a Block Helper

Custom block helpers let you define your own iterators and conditionals. The following example shows how to create a custom helper to determine if a passed string starts with a particular set of characters.

Consider the following sample helper code:

```
Handlebars.registerHelper('startsWith', function (prefix, text, options) {
    if (text.indexOf(prefix) === 0) {
        return options.fn(this);
    }
    return options.inverse(this);
});
```

This helper includes a custom if/else block. This sample completes the following actions:

- Registers the `startsWith` helper
- Passes three parameters:
 - The prefix characters you want to check the passed string for

- The text string you want to check
 - A hash of options, which Handlebars handles for you, including determining the output to show in the template
- When invoked, this code uses `indexOf` to see if the starting point of the prefix is at the start of the main string:
 - If it is there, returns the template code for the truthy part of the condition
 - If it is not there, returns the template code for the falsy part of the condition

Test the Block Helper

Due to the complexity of this helper, you should test in a template. After adding the helper to a template, deploy the extension that contains the template and then activate the extension as described in [Deploy an Extension to NetSuite](#). After you deploy the extension, you can fetch it and test it locally, as described in [Test an Extension on a Local Server](#)

When you have your local Commerce website running in a browser, open the developer console. Test the block helper you created by entering code like the following:

```
{{#startsWith 'Ste' 'Steve'}}
  <p>Steve? That's a great name!</p>
{{else}}
  <p>Well, that name is pretty good, but it's not Steve</p>
{{/startsWith}}
```

If you are working with a template that has access to the current user's profile data, you could replace the 'Steve' parameter with something like, `model.fullName`.

Refer to the Handlebars documentation for more details about [Block Helpers](#).

Use Helpers for Debugging

You can use helpers when debugging to print the content of objects to the developer console or to the page.

More recent versions of SuiteCommerce and SCA have a built-in helper called, `log`. You can use the `log` helper as shown in the following samples:

- To log the view's context object: `{{log this}}`

 **Note:** In the preceding sample, this is contextual and so it will change value if, for example, you use it in an iterator or a block.

- To log the model: `{{log model}}`

Also keep in mind that the templates provided in recent versions of SuiteCommerce and SCA list the properties that developers have sent to the templates. This means you can find the property names at the bottom of each template.

Use Multiple Modules in an Extension

When developing extensions, there are cases where it is best to use multiple modules to organize the files and assets for your customization. For example, it may be easier to manage and maintain your extension if you organize the files for each separate function that the extension performs into separate modules. And, multiple modules are ideal for scenarios where you want to run different code in different parts of the site, but you want to contain all the code within the same extension.

Note: The examples in this topic assume that you are familiar with extension files and folders and have some experience developing and testing extensions. If not, refer to these topics for more information, [Extension Development Files and Folders](#), [Anatomy of an Extension](#), [Get Started with Extensions](#).

These are the basic steps to include multiple modules in your extension:

1. Add an additional module to the Modules folder.
2. Add needed files to the new module. An additional entry point file is not necessary.

Note: If you do add another entry point file with a `mountToApp()` function, it is not automatically called when the module is loaded.

3. Add the files from the second module as dependencies to the files in your first module, as required.
4. Call the methods from the files within the additional module like you normally do with the methods from the original module.

Read the following sections to learn more:

- [Folder Structure](#)
- [Entry Points](#)
- [Calling Files in Other Modules](#)
- [Asset Organization and File Reuse](#)

Folder Structure

As described in [Extension Development Files and Folders](#), the Workspace directory contains a subdirectory for each extension you are creating. And a Modules folder is included in the subdirectory for each extension. The Modules folder usually contains one module/folder that is also named after your extension, like so:

```
<TopLevelDevelopmentDirectory>
  Workspace
    MyCoolExtension
      assets
      Modules
        MyCoolModule
      manifest.json
```

However, the name of the folder for each module is flexible and you are not required to name each module after your extension. This topic guides you through several examples that are based upon the following directory structure:

```
<TopLevelDevelopmentDirectory>
  Workspace
    MyCoolExtension
      assets
      Modules
        ModuleOne
          JavaScript
            ModuleOne.js
        ModuleTwo
          JavaScript
            ModuleTwo.js
      manifest.json
```

To create entry point files, add code like the following to the ModuleOne.js and ModuleTwo.js files:

```
define('ModuleOne'
, [
]
, function
(
)
{
'use strict';
return {
  mountToApp: function ()
  {
    console.log('ModuleOne loaded!');
  }
}
})
```



Note: Be sure to change ModuleOne in the preceding code sample to ModuleTwo for the ModuleTwo.js file.

Entry Points

When developing customizations for the Kilimanjaro (2017.2) and earlier releases of SCA, you needed to create an entry point for each module. However, with the introduction of the extension framework and extensions in the Aconcagua release, this is no longer necessary. When working with extensions, there is typically one entry point for the entire extension as illustrated in [Single Entry Point for an Extension](#). The only exception is when you want different entry point files for the different applications on your site as shown in [Different Entry Points for Site Applications](#).

Example 3. Single Entry Point for an Extension

If you created the files and folders described in [Folder Structure](#), you can walk through this example to see how a single entry point works.

In the extension's manifest.json file, add the path to the ModuleOne.js file as the entry point for each application your extension will be applied to. For example, if your extension only applies to the checkout application, set the entry point as shown in the following example:

```
"javascript": {
  "entry_points": {
    "shopping": "Modules/ModuleOne/JavaScript/ModuleOne.js"
```

Now if you run the gulp command to update the extension manifest:

```
gulp extension:update-manifest
```

And then run a local test of your extension with this gulp command:

```
gulp extension:local
```

When you access your local site and check your browser's developer console, you see this message:

```
ModuleOne loaded!
```

This result is expected because, although the extension's manifest file has been updated and includes the ModuleTwo file in its list of files, you have not called the second module.

When your extension is loaded, only the entry point file in the manifest is called. Therefore, if you want additional code to be called, you need to add calls to that entry point file's `mountToApp()` method. To learn more, see [Calling Files in Other Modules](#).

Example 4. Different Entry Points for Site Applications

If you created the files and folders described in [Folder Structure](#), you can walk through this example to see how to set up different entry points for the applications on your site.

If you need to have different entry points for the applications on your site, you can modify the `entry_points` object in your extension manifest as shown in the following example:

```
"javascript": {
  "entry_points": {
    "shopping": "Modules/ModuleOne/JavaScript/ModuleOne.js",
    "myaccount": "Modules/ModuleTwo/JavaScript/ModuleTwo.js",
    "checkout": "Modules/ModuleTwo/JavaScript/ModuleTwo.js"
  },
}
```

If you modify your extension's `manifest.json` file, as shown in the preceding code sample, and save it, you can restart your local server to test this modification. Now when you access the checkout or customer account areas of your local site, the following message displays in your browser's developer console:

```
ModuleTwo loaded!
```

This approach works well when you want to run different code in different parts of the site, but you want to contain all the code within the same extension. This approach also helps you manage dependencies.

Calling Files in Other Modules

If you created the files and folders described in [Folder Structure](#), you can walk through this example to see how to call files in other modules.

Reset the `manifest.json` file so that all three applications use the same entry point as shown in the following code sample:

```
"javascript": {
  "entry_points": {
    "shopping": "Modules/ModuleOne/JavaScript/ModuleOne.js",
    "myaccount": "Modules/ModuleOne/JavaScript/ModuleOne.js",
    "checkout": "Modules/ModuleOne/JavaScript/ModuleOne.js"
  },
}
```

If you modify `ModuleOne.js` to add `ModuleTwo` as a dependency, you can then call it from within `ModuleOne.js`. For example, you can call `ModuleTwo`'s `mountToApp()` method as shown in the following code sample:

```
define('ModuleOne'
, [
  'ModuleTwo'
]
, function
(
  ModuleTwo
)
{
  'use strict';
  return {
    mountToApp: function ()
    {
      console.log('ModuleOne loaded!');
      ModuleTwo.mountToApp();
    }
  }
}
```

```
})
```

If you save these changes and then restart your local server to test these modifications, the following message displays in your browser's developer console:

```
ModuleOne loaded!
ModuleTwo loaded!
```

This output verifies that calling the file in the second module is working as expected.

Calling files in other modules is not limited to entry point files. This approach works for other types of files in your extension, too. You can reference any file, like views or models, if the file has been properly defined and made available to the application. So, instead of having an entry point file in your second module, you could have a view, model, or any other type of supported file instead. As shown in the preceding code sample, you can simply add it as a dependency and call one of its methods.

Asset Organization and File Reuse

If you want to organize the files in your extension's top-level assets directory to group them by module, you can. The following sample directory structure shows how you may organize your images:

```
assets
  fonts
  img
    ModuleOne
      img.jpg
    ModuleTwo
      img.jpg
  services
```

With your image files organized in this way, you could write code to:

- Load the ModuleOne image file when the ModuleOne entry point file is loaded.
- Load the ModuleTwo image file when the ModuleTwo entry point file is loaded.

All you need to do is create a dynamic path to the image based on the entry point loaded.

For example, you may want text and a small image to appear in your site's header, but you want the text and image to be different, depending on which application is in use.

The following example shows how to make this scenario work by reusing the view and template files from ModuleOne in ModuleTwo. The only thing that is different is that you duplicate the ModuleOne entry point file and change its name (and a value in the file that captures its name).

First you edit the manifest file so that ModuleTwo's entry point is loaded when the shopper accesses the checkout application:

```
"javascript": {
  "entry_points": {
    "checkout": "Modules/ModuleTwo/JavaScript/ModuleTwo.js",
  },
}
```

The ModuleOne entry point file looks like this:

```
define('ModuleOne'
, [
  'ModuleOne.View'
])
```



```

, function
(
  ModuleOneView
)
{
  'use strict';
  return {
    name: 'ModuleOne'
  , mountToApp: function (container)
    {
      console.log(this.name + ' loaded!');
      var Layout = container.getComponent('Layout');
      var self = this;
      Layout.addChildView('cms:header_banner_top', function ()
      {
        return new ModuleOneView({moduleName: self.name});
      });
    }
  }
}
})

```

The ModuleTwo entry point file looks identical, except for the name in the opening of the define statement and the value of name. The ModuleTwo entry point still adds the same view and also dynamically passes the module name to the view constructor:

```

define('ModuleTwo'
, [
  'ModuleOne.View'
]
, function
(
  ModuleOneView
)
{
  'use strict';
  return {
    name: 'ModuleTwo'
  , mountToApp: function (container)
    {
      console.log(this.name + ' loaded!');
      var Layout = container.getComponent('Layout');
      var self = this;
      Layout.addChildView('cms:header_banner_top', function ()
      {
        return new ModuleOneView({moduleName: self.name});
      });
    }
  }
}
})

```

The view for this example scenario looks like this:

```

define('ModuleOne.View'
, [
  'Backbone'
, 'module_one_tpl'
]
, function
(
  Backbone
, module_one_tpl
)
{
  'use strict';
  return Backbone.View.extend({
    template: module_one_tpl
  , getContext: function ()
    {
      return {
        message: 'This is ' + this.options.moduleName
      }
    }
  });
}

```

```

        , image: 'img/' + this.options.moduleName + '/img1.jpg'
      }
    }
  })
})

```

When the view is created, it receives whatever options the constructor is given and attaches them to its own options property. For this case, you pass it the name of the module, which you are using to build the path for the image URL, as well as a message to be passed to the template.

The template for this example scenario is simple:

```

<p>{{message}}</p>
<p></p>

```

If you save these changes and then restart your local server to test these modifications:

- When the ModuleOne entry point is loaded, the image.jpg stored in assets/img/ModuleOne is displayed
- When the ModuleTwo entry point is loaded, the image.jpg stored in assets/img/ModuleTwo is displayed

Both the view and template are completely reusable in this context, and you can build them out however you want. If you need to diverge and create a customization that is better suited to another scenario, you could modify the view and templates with more conditionals, or you could simply go back to having separate files. It's really up to you. The main point of this example is to illustrate that you can use cross-module files in your extensions.

Extension Tutorials

📘 Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Use the guides in the following table to learn about specific aspects of developing SuiteCommerce extensions. The guides provide step-by-step instructions and code samples that show how to develop an extension that accomplishes a defined task.

Guide	Extensibility API Components Used
Use the Search Component to Set the 'url' property of Models	■ PLP ■ SCCollection ■ SCModel ■ SCView ■ Search
Capture Form Data on the Login/Register Page	■ LoginRegisterPage ■ SCView
Create a Custom Content Type	■ CMS
Access Model and Collection Data in a Child View	■ PLP ■ SCView
Manage SuiteCommerce Analytics Data Cookie Preferences	■ Layout ■ SCView ■ UserProfile

Use the Search Component to Set the 'url' property of Models

📘 Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

The [Search](#) component of the extensibility API (introduced with SuiteCommerce 2021.1) enables you to obtain the URL of the item search API of a SuiteCommerce website.

Basic Usage

The Search component includes the `getUrl()` method, which you can use to get the URL of the item search API of your SuiteCommerce website. When you call `getUrl()`, it returns a string that corresponds to the end fragment (everything after the domain name part of the URL) of the item search URL. You can append search parameters to the URL by passing in an object in which the object properties are the search parameters. `getUrl()` returns the item search URL with the

Here is an example of how to call `getUrl()` with a search parameter to find items that contain the string "shirt". You typically call this method in the `mountToApp()` method of the `entrypoint` module in your extension.

```
1 var search = container.getComponent('Search');
2 var searchParams = {q: "shirt"};
3
4 var searchURL = search.getUrl(searchParams);
```

The URL returned with the above search parameters might look like the following:

`api/items?language=en&country=US¤cy=USD&pricelevel=5&use_pcv=F&q=shirt`

Note that the returned URL includes data from the site visitor's session, as well as the appended search parameters (`q=shirt`). The default parameters from the user session are:

- language
- country
- currency
- pricelevel
- use_pcv

Using the Search Component with Models and Collections

Basic usage of the `getUrl()` method of the Search component allows you to get the item search API URL. You could then use that URL to make AJAX requests. However, a more integrated approach in which you can use this component is to set the returned URL as the value of the `url` property of a model or a collection of models.

The `url` and `urlRoot` properties of the `SCCollection` and `SCModel` components point to service files. You can set the item search URL as the value of these properties, simplifying the integration of item data in your extensions.

Example – Displaying Specific Items on the Product List Page

The following example explains how to use the Search component to find specific items, assign the item search URL to models in a collection, and use a collection view to display the items at the top of the product list page.

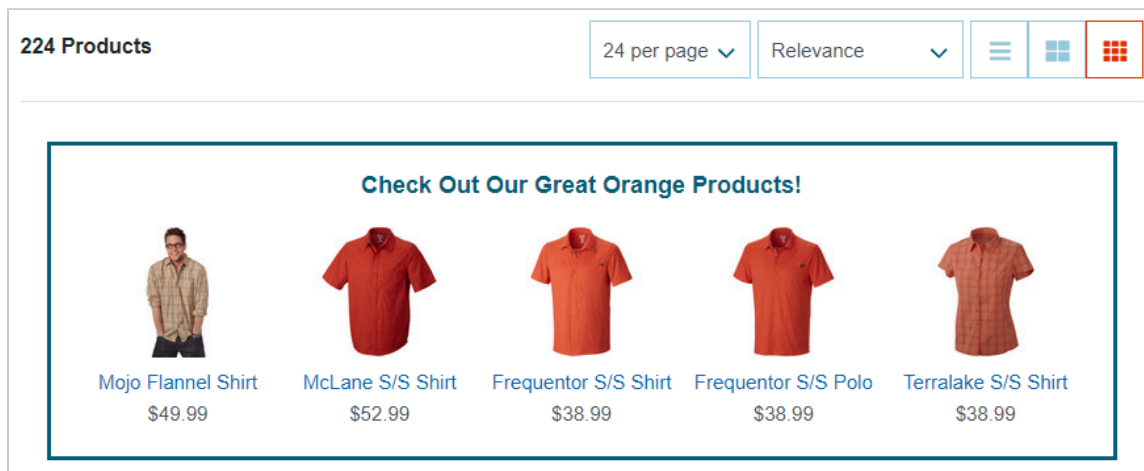
The steps to fully implement the Search component in this example include the following:

- Create the **entrypoint file** – The entrypoint file is created as part of the baseline extension with the extension developer tools.
- Create a **collection view** – Create a collection view based on the `SCCollectionView` component. A collection view simplifies displaying models in a grid or layout pattern.
- Create a **collection** – You will use the Search component in this file to specify the search parameters and retrieve the item search URL.
- Create a **cell view** – A cell view is required for each model in the collection.

- Create the **templates** – You must create templates for both the collection view and cell view. The collection view template is like a container that includes a placeholder in which the cell views are inserted. The cell view template holds the HTML for each item displayed on the website.
- Create a **Sass** file – Define the styles to match the look and feel of your website.

To get started with the extension, create a baseline extension with the extension developer tools – see [Create a Baseline Extension](#). A baseline extension creates the basic folder structure of an extension and provides a set of starter files, including the entrypoint file, model files, and view files.

At completing the steps in this example, you should be able to load the extension on your website and view the five items returned by the item search API.



Create the EntryPoint File

Every SuiteCommerce extension has an entrypoint file, which is the first file that is loaded by the application. You only need to specify the collection view as a dependency; the collection view has other dependencies including the collection, and the cell view in which each item will be displayed.

In the `mountToApp()` method, create an instance of the [PLP](#) and [Search](#) components. Then, after checking that both components were correctly instantiated, use the `addChildViews()` method to insert the collection view into the `Facets.Items` element.

By specifying a low `childViewIndex` for the inserted view, it will appear at the top of the facets layout. Finally, in the `childViewConstructor` callback, pass in the container object to be able to access the extensibility API components later in the collection view.

JavaScript/MyCompany.OrangeZoneExtension.js

```

1  define(
2    MyCompany.OrangeZoneExtension',
3    [
4      MyCompany.OrangeZoneExtension.Collection.View'
5    ],
6    function(MyCompanyOrangeZoneExtensionCollectionView) {
7      'use strict';
8
9      return {
10         mountToApp: function mountToApp (container) {
11
12           var PLP = container.getComponent('PLP');
```

```

13     var Search = container.getComponent('Search');
14
15     if (PLP && Search) {
16         PLP.addChildViews(PLP.PLP_VIEW, {
17             'Facets.Items': {
18                 'MyCompany.OrangeZoneExtension.CollectionView': {
19                     childViewIndex: 1,
20                     childViewConstructor: function () {
21                         return new MyCompanyOrangeZoneExtensionCollectionView({
22                             application: container
23                         });
24                     }
25                 }
26             }
27         });
28     }
29 }
30 }

```

Create a Collection View

Define a custom collection view with the [SCCollectionView](#) component. In this extension, the collection view is what will be rendered on the website. Two important aspects of a collection view are the methods [getCellViewsPerRow](#) and [getCellViewInstance](#), which are used to set the number of cells per row and specify the view used for each cell respectively.

JavaScript/MyCompany.OrangeZoneExtension.Collection.View.js

```

1  define(
2      'MyCompany.OrangeZoneExtension.Collection.View',
3      [
4          'SCCollectionView',
5          'MyCompany.OrangeZone.Collection',
6          'MyCompany.OrangeZone.Cell.View',
7          'mycompany_orangezone_collection.tpl'
8      ],
9      function (SCCollectionViewModule, MyCompanyOrangeZoneCollection, MyCompanyOrangeZoneCellView, mycompany_orangezone_collection_tpl) {
10         'use strict'
11
12         var SCCollectionView = SCCollectionViewModule.SCCollectionView;
13
14         function MyCompanyOrangeZoneExtensionCollectionView (options) {
15             SCCollectionView.call(this, options.collection);
16
17             this.application = options.application;
18             this.collection = new MyCompanyOrangeZoneCollection([], options);
19             this.template = mycompany_orangezone_collection_tpl;
20         }
21
22         MyCompanyOrangeZoneExtensionCollectionView.prototype = Object.create(SCCollectionView.prototype);
23         MyCompanyOrangeZoneExtensionCollectionView.prototype.constructor = MyCompanyOrangeZoneExtensionCollectionView;
24
25         MyCompanyOrangeZoneExtensionCollectionView.prototype.getCellViewInstance = function (model) {
26             return new MyCompanyOrangeZoneCellView({
27                 application: this.application,
28                 model: model
29             });
30         }
31
32         MyCompanyOrangeZoneExtensionCollectionView.prototype.getCellViewsPerRow = function () {
33             return 5;
34         }
35
36         MyCompanyOrangeZoneExtensionCollectionView.prototype.render = function () {
37             var self = this;
38             this.collection.fetch().done(function () {
39                 SCCollectionView.prototype.render.call(self);

```

```

40     })
41   }
42
43   MyCompanyOrangeZoneCollectionView.prototype.getContext = function () {
44     return {}
45   }
46
47   return MyCompanyOrangeZoneCollectionView;
48 }

```

First, declare the dependencies for the collection view, including `SCCollectionView` and the collection itself. Then define the constructor function of the collection view – when the collection view is instantiated, the application calls the constructor function first.

In the constructor function, call the constructor function of `SCCollectionView` to get `SCCollectionView`'s properties. Set some properties of the extension's collection view, including the collection for the view and the template.

After the constructor, create a copy of the parent class's prototype and apply it to our custom class and then set the constructor to point to our newly created constructor. Next, override `getCellViewInstance()` to specify which view to use for each model in the collection. In this case, the view is the cell view called `MyCompanyOrangeZoneExtensionCellView` (see [Create the Cell View](#)) to which you pass the application and model objects. Then, to specify the number of cells to display in each row, override `cellViewsPerRow()` and return 5.

Because the view requires data before it is rendered, override the `render()` method to make sure the data is available. Inside the `render()` method, call `fetch()` on the collection to get the item information and then use `done()` on the returned promise to call the parent class' `render()` method.

Call the `getContext()` method as the final step in the collection view subclass. `getContext()` is used to pass data to the template, but because the collection view template does not require any data, you return an empty object – this is because `getContext()` is a required property when subclassing `SCCollectionView`.

Create the Collection

A collection holds the array of models. In this example, each model corresponds to an item returned from the item search API. The collection is required by the collection view defined previously.

JavaScript/MyCompany.OrangeZone.Collection.js

```

1  define(
2    'MyCompany.OrangeZone.Collection',
3    [
4      'SCCollectionView',
5      'SCModel',
6      'underscore'
7    ],
8    function(
9      SCCollectionViewModule,
10     SCModelModule,
11     -
12   ) {
13     'use strict';
14
15     var SCCollectionView = SCCollectionViewModule.SCCollection;
16     var SCModel = SCModelModule.SCModel;
17
18     function MyCompanyOrangeZoneCollection(models, options) {
19       SCCollectionView.call(this, models, options);
20
21       this.model = SCModel;
22     }

```

```

23     var searchParams = {
24         'custitem31': 'orange',
25         'limit': '5',
26         'fields': 'displayname,itemimages_detail,onlinecustomerprice_formatted,urlcomponent'
27     }
28
29     var searchUrl = options.application.getComponent('Search').getUrl(searchParams);
30
31     this.url = function url () {
32         return searchUrl
33     }
34
35     this.parse = function parse(response) {
36         return _.compact(response.items) || []
37     }
38 }
39
40 MyCompanyOrangeZoneCollection.prototype = Object.create(SCCollection.prototype);
41 MyCompanyOrangeZoneCollection.prototype.constructor = MyCompanyOrangeZoneCollection;
42
43 return MyCompanyOrangeZoneCollection;
44 }
45 )

```

`MyCompanyOrangeZoneCollection()` on line 18 is the constructor function of the class. As with all collections that extend `SCCollection`, you must first call the parent constructor function with `call()`. You then declare the collection's model.

To specify the items you want to find with the item search API, create an object that contains the search parameters. `searchParams` contains three properties: `custitem31` refers to a field that specifies an item's color, `limit` restricts the number of items to retrieve, and `fields` specifies the item fields to return.

Note: The properties and field names in this example are specific to a particular NetSuite account. Check the properties and field names of your account and change them accordingly to suit your needs.

Pass the search parameters object into the `getUrl()` method and assign the returned URL as the value of the `url` property of the collection. When the collection view calls `fetch()` on this collection, it will use the assigned URL.

The `parse()` function is called when a response is returned from the `fetch` and is a processor that determines how to handle the returned results. You don't usually need to override this function, but because the item search API returns its items as a property of the response object (rather than as the response object itself), we need to pluck out those results and set the array of results as the collection.

The remainder of the class is concerned with setting the prototype object to a copy of the parent class', and then setting the constructor property to that of the newly created one.

Create the Cell View

The cell view is used to display each item in the collection. The main purpose of defining the cell view is often to determine the context data that will be passed to the cell view template. However, depending on the response from the item search API, you may need to perform logic on the data to ensure it is valid and correctly formatted.

In this example, the cell view class includes a custom function (`getThumbnail()`) to determine which item image to show. When you get item information with the item search API, it returns the item images in a nested property called `itemimages_detail`. The `getThumbnail()` function gets the appropriate image information from the property. Note that image information returned for your SuiteCommerce website may differ depending on the account settings and the website configuration.

JavaScript/MyCompany.OrangeZoneExtension.Cell.View.js

```

1  define('MyCompany.OrangeZoneExtension.Cell.View',
2  [
3    'SCView',
4    'Utils',
5    'mycompany_orangezoneextension_cell_tpl'
6  ],
7  function(
8    SCViewModule,
9    Utils,
10   mycompany_orangezoneextension_cell_tpl
11 ) {
12   'use strict';
13
14   var SCView = SCViewModule.SCView;
15
16   function MyCompanyOrangeZoneExtensionCellView(options) {
17     SCView.call(this, options);
18
19     this.model = options.model;
20     this.template = mycompany_orangezoneextension_cell_tpl;
21     this.Environment = options.application.getComponent('Environment');
22
23     var self = this;
24
25     this.getThumbnail = function getThumbnail(item) {
26       var images = item.get('itemimages_detail');
27       var separator = 'media';
28
29       var color = 'orange';
30
31       var thumbnail = {
32         alt: item.get('displayname'),
33         url: Utils.getThemeAbsolutePathOfNonManagedResources(
34           'img/no_image_available.jpeg',
35           self.Environment.getConfig('imageNotAvailable')
36         )
37       };
38
39       if (images && images[separator] && images[separator][color]) {
40         var image = images[separator][color].urls[0];
41
42         thumbnail.url = image.url || thumbnail.url;
43         thumbnail.alt = image.altimagetext || thumbnail.alt;
44       }
45
46       return thumbnail;
47     }
48   }
49
50   MyCompanyOrangeZoneCellView.prototype = Object.create(SCView.prototype);
51   MyCompanyOrangeZoneCellView.prototype.constructor = MyCompanyOrangeZoneCellView;
52
53   MyCompanyOrangeZoneCellView.prototype.getContext = function () {
54     return {
55       name: this.model.get('displayname'),
56       price: this.model.get('onlinecustomerprice_formatted'),
57       thumbnail: this.getThumbnail(this.model),
58       url: this.model.get('urlcomponent')
59     }
60   }
61
62   return MyCompanyOrangeZoneCellView;
63 }
64 );

```

The cell view is based on the [SCView](#) abstract class. Include SCView as a dependency and then in the constructor function call the constructor function of the parent class. After declaring the model and template, define a custom function to parse the raw image information and return an object that contains nicely formatted image information.

To pass the required context information to the cell view template, use the `getContext()` method of `SCView`. `getContext()` must return an object that contains the item information as property/value pairs. Note that the `thumbnail` property calls the custom function to get a valid image.

Create Templates for the Collection View and Cell View

The collection view template is a simple template that holds the content of the collection view. It must contain a HTML element that has a data attribute called `data-type` with the value `backbone.collection.view.rows`. The application will render the rows inside the element.

Templates/mycompany_orangezoneextension_collection.tpl

```
1 <div class="mycompany-orangezone-collection">
2   <h3 class="mycompany-orangezone-heading">{{translate 'Check Out Our Great Orange Products!'}}</h3>
3   <div data-type="backbone.collection.view.rows"></div>
4 </div>
```

The cell view is rendered once for each item in the collection. The template for the cell view references several context properties defined previously in the cell view. Note that because the size of the view is expected to be relatively small, you can wrap the image URL in the `resizeImage` custom Handlebars helper, which modifies the image URL to include additional parameters for the image resizing service.

Templates/mycompany_orangezoneextension_cell.tpl

```
1 <div class="mycompany-orangezone-item">
2   <a href="{{url}}"></a>
3   <a href="{{url}}" class="mycompany-orangezone-item-name">{{name}}</a>
4   <p>{{price}}</p>
5 </div>
```

Create the Sass File

If you want to adjust the styling of your extension, you can modify the SCSS file that was created with the baseline extension. The most notable aspects of the styling in this example include a thick border around the content and a media query that only shows the item name on devices with larger screens.

Sass/mycompany-orangezone.scss

```
1 .mycompany-orangezoneextension-collection {
2   border: 3px solid $sc-color-secondary;
3   margin: 0 $sc-margin-lv4 $sc-margin-lv4 $sc-margin-lv4;
4   padding: $sc-padding-lv4;
5   text-align: center;
6 }
7
8 .mycompany-orangezoneextension-heading {
9   margin-bottom: $sc-margin-lv4;
10 }
11
12 .mycompany-orangezoneextension-item {
13   box-sizing: border-box;
14   display: inline-block;
15   max-width: 20%;
16 }
17
18 .mycompany-orangezoneextension-item-name {
19   display: none;
20
21   @media (min-width: $screen-xs-min) {
22     display: inline-block;
23   }
24 }
```

24 | }

Test the Extension

You should now be able to test the extension locally by running the gulp command `gulp extension:local`.

Capture Form Data on the Login/Register Page

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

On a SuiteCommerce or SuiteCommerce Advanced website, the default registration form contains a set of basic fields to collect the minimum required user information: name, surname, email address, and password. If you want to collect additional information, you can create an extension that uses the **LoginRegisterPage** component.

The LoginRegisterPage component is available since SuiteCommerce 2019.1. It enables you to add custom fields to the login and registration forms and save the user input in NetSuite. It also provides custom events in the login/register flow that you can listen for – you can use these events to perform pre- or post-processing when either of the forms is submitted.

Note: Instead of creating a custom extension, you may want to use the Advanced Sign Up extension by Oracle NetSuite. It lets you add custom fields to the Login/Register page on your website, as well as configure how the customer record is created and approved. See the help topic [Advanced Sign Up](#) for more information.

Overview of How to Capture Form Data

This topic describes how to create an extension based on a scenario in which a business wants to collect additional information from users before they register or log in on your website. You will see how to add an additional field to the registration form to obtain a company's tax registration number. Because the business only sells to other businesses, you perform some basic validation to make sure the field is not blank. You will also see how to set up the custom fields in NetSuite to store the data.

Steps to Capture Form Data

Follow these steps to create an extension that captures form data on the Login/Register Page:

1. **Create custom fields in NetSuite** – The custom fields store the data entered by users on the Login/Register page.
2. **Set up your extension files** or create a baseline extension – see [Create a Baseline Extension](#).
3. **Update the entry point file** – Instantiate the LoginRegisterPage component and add a child view.
4. **Update the view file** – Define a view for the form and create a template with the input fields.
5. **Add input validation** – Use the beforeRegister event provided by the LoginRegisterPage component to perform input validation before form submittal.

Create a Custom Field

An essential part of capturing form data on your website is storing the data in NetSuite. When a user registers on your website, the users details are saved on a lead record. If you want to obtain additional information about users, usually when they register or sign up on your website, you need to create custom fields in NetSuite to store the data.

When creating a custom field, you must specify an identifier (ID) for the field. You will then reference that field ID as the name property of a form element in the view template. All custom field IDs start with the prefix "custerity" – the string you type as the field ID is then appended to the prefix, for example, "custerity_myfield".

Take note of the field ID – you will need to reference the ID in the template file that contains the HTML for the custom fields on the registration form.

To create a custom field in NetSuite

1. Go to Customization > Lists, Records, and Fields > Entity Fields > New.
2. In the **Label** field, enter the name of the field as you want it to appear on the customer record in NetSuite.
3. In the **ID** field, enter an identifier for the field.



Note: Use an underscore as the first character of the ID to make the final ID easier to read. For example, if you enter "_comptaxnumber", the final ID of the field will be "custerity_comptaxnumber".

4. On the Applies To tab, check the **Customer** and the **Web Site** boxes.
5. Click **Save**.

See the help topic [Creating a Custom Field](#) for more information about custom fields.

Update the Entry Point File

If you created a baseline extension, you already have a starting point for developing your extension. In the main module file of the baseline extension, create an instance of the LoginRegisterPage component in the mountToApp() function (see [Instantiating API Components](#) for more information about creating an instance of a component). The LoginRegisterPage component inherits several methods from its parent component, so after checking that the component is not null, add a child view to the Register page with addChildView().

When adding a child view, you must specify the ID of the view placeholder element on the form to which you want to add the child view. The base theme (2019.1 and later) includes empty placeholder elements at the bottom of the login and registration forms. If you are using an earlier theme or another theme, you must add the elements to the theme. The view placeholder for the registration form is <div data-view="Register.CustomFields"></div>, so pass that in as the first argument to addChildView(). The second argument to addChildView() is a function that returns an instance of the view.

NetSuite.CompanyTaxNumber.Main.js

```
1 define(
2   'Vendor.CompanyTaxNumber.MainModule',
3   [Vendor.CompanyTaxNumber.MainModule.View],
4   function(CompanyTaxNumberView) {
5     'use strict';
6
7     return {
```

```

8      mountToApp: function mountToApp(container) {
9          var loginRegisterPageComponent = container.getComponent('LoginRegisterPage');
10
11          if (loginRegisterPageComponent) {
12              loginRegisterPageComponent.addChildView('Register.CustomFields', function() {
13                  return new CompanyTaxNumberView(loginRegisterPageComponent);
14              });
15          }
16      }
17  }
18 }
19 );

```

When returning an instance of the view, pass in the LoginRegisterPage component to the view constructor. You will need access to the component in the view to be able to listen for specific events on the Login/Register page.

```

1 | return new CompanyTaxNumberView(loginRegisterPageComponent);

```

Create a View

To capture user input, add a view to the Login/Register page that contains whatever form elements you want to use to capture data, such as text boxes or checkboxes. When the entry point file is called, the view is inserted at placeholders at the bottom of the login and register forms.

Define your view using the SCView class from the extensibility API – see [SCView](#) for an example of how to define a view with SCView. Note that in versions of SuiteCommerce Advanced prior to release 20.2, you must create a view with the native [View](#) module in Backbone.

The following example is a view for the registration form on the Login/Register page. The view has several dependencies including SCView and the template rendered by the view. The constructor function for the view accepts one argument which should be an instance of the LoginRegisterPage component. It also adds an event listener (beforeRegister) that can be used to validate user input.

NetSuite.CompanyTaxNumber.Main.View.js

```

1  define(
2      'NetSuite.CompanyTaxNumber.Main.View',
3      [
4          'SCView',
5          'jQuery',
6          'netsuite_companytaxnumber_main.tpl'
7      ],
8      function(SCViewComponent, jQuery, netsuite_companytaxnumber_tpl) {
9          'use strict';
10
11          var SCView = SCViewComponent.SCView;
12
13          function CompanyTaxNumberMainView(loginRegisterPageComponent) {
14              SCView.call(this);
15
16              this.template = netsuite_companytaxnumber_tpl;
17
18              loginRegisterPageComponent.on('beforeRegister', function(formFields) {
19                  if (formFields.custentity_comptaxnumber == '') {
20                      alert('You must enter your company tax number. ');
21                      return jQuery.Deferred().reject();
22                  }
23              });
24          }
25
26          CompanyTaxNumberMainView.prototype = Object.create(SCView.prototype);
27          CompanyTaxNumberMainView.prototype.constructor = CompanyTaxNumberView;
28

```

```

29 | CompanyTaxNumberMainView.prototype.getContext = function() {
30 |     return {};
31 | }
32 |
33 | return CompanyTaxNumberMainView;
34 | }
35 | );

```

In addition to defining the view, you must also update the template file for the view. The template includes a div that contains a text input field. Note that the name attribute of the input field must be exactly the same as the ID of the custom field record you created in the NetSuite account. The CSS class names are the same as those used for other fields on the form to match the overall look and feel of the form.

netsuite_newcompanytaxnumber_main.tpl

```

1 | <p class="login-register-register-form-description">
2 |     We only serve business customers who are currently registered for tax in Canada, USA, and France.
3 | </p>
4 | <div class="login-register-register-form-controls-group" data-validation="control-group">
5 |     <label class="login-register-register-form-label" for="company-tax-number"> Company Tax Number <small class="login-register-regis
6 |     ter-form-required">*</small></label>
7 |     <div class="login-register-register-form-controls" data-validation="control">
8 |         <input type="text" name="custentity_comptaxnumber" id="company-tax-number" class="login-register-register-form-input">
9 |     </div>
10 | </div>

```

Listen for Login/Register Events To Validate Data

The LoginRegisterPage component provides three custom events:

- beforeLogin
- beforeRegister
- afterRegister

All 'before' events are cancelable events that happen just before the event. To validate user input on the registration form, use the beforeRegister event to ensure validation is performed before data is sent to the server. To use the beforeRegister event, call the on() method of the LoginRegisterPage component. The on() method allows you to attach an event handler to any of the component's events.

The beforeRegister event returns an object that contains the form data, which you can then pass as an argument to the event handler.

Additionally, because .on returns a deferred object, you must return a rejected promise in the event handler if validation fails. Returning anything else, such as null or an empty string allows the registration process to continue.

```

1 | loginRegisterPageComponent.on('beforeRegister', function(formFields) {
2 |     if (formFields.custentity_comptaxnumber == '') {
3 |         alert('You must enter your company tax number.');
```

In the example above, a browser alert is shown with the validation error message. You may want to experiment with other ways to show the validation message, for example, by showing a modal dialog, or by displaying the message next to the form field in the same format as other field validation messages. For example, you could append a <p> element to the form field with the appropriate data-* attribute from the base theme.

```
1 | jQuery('#company-tax-number').after('<p data-validation-error="block">You must enter your company tax number.</p>');
```

Test the Extension and Check Form Data Is Saved

You can test your extension at any time by starting a local server – see [Test an Extension on a Local Server](#). If you enter all required fields and validation for your custom field does not fail, you should be able to view the data you entered in the custom field in the NetSuite account.

1. Go to Lists > Relationships > Leads.
2. Click **View** next to the lead name you entered on the registration form.
3. Select the **Custom** tab to view the custom field and the data you entered.
If the custom field is not visible, you may need to customize the form to show the field. See the help topic [Custom Forms](#) for more information.
4. Verify that the data you entered on the form is in the custom field.

Create a Custom Content Type

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

As an extension developer, you can create Site Management Tools (SMT) custom content types (CCT) to provide custom website applications or features that website administrators can enable and manage through the SMT user interface. As a best practice, you should build your CCT as an extension using the extension developer tools.

In this tutorial, follow the steps in each listed procedure to develop a CCT named SC.CCT.ImageViewer. This CCT introduces a simple image viewer that retrieves data from either a custom record (as set up in SMT) or from certain data previously made available in the browser.

Implementing CCTs for your SuiteCommerce website requires two separate, but related activities:

- Create a CCT as an extension for SuiteCommerce. You must first create an extension with a CCT to contain the JavaScript, HTML, Sass, and other resources required. See the checklist below for guided steps.
- Set up custom records in NetSuite for your CCT using SMT. This requires creating the custom records, fields, and content type record for your custom content type, and then enabling the CCT in SMT.

Follow these steps to implement a CCT as an extension for your SuiteCommerce site.

Step	Description
Create Your CCT as an Extension	Use the extension development tools to create a baseline extension that includes a baseline module and then add a CCT baseline module as a separate module.
Create the Entry Point JavaScript Files	Create the entry point JavaScript files that are necessary to mount the extension to the application.
Implement the View File	Define what the CCT loads when it is dragged into the application.
Implement the Template File	Create the template file required to render the data defined by the view.
Implement the Sass Files	Use the Sass files auto-generated by the extension development tools to style the HTML defined by the template file.

Step	Description
Test and Deploy the CCT Extension	Test and deploy the extension.

You must also set up your custom records in NetSuite for your custom content type using SMT. To do this, follow the instructions in [Site Management Tools Custom Content Type](#).



Important: You must have SMT enabled in your NetSuite account to implement CCTs. See the help topic [Site Management Tools](#) for information on how to set up SMT.

Create Your CCT as an Extension

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

The sample ImageViewer extension includes a baseline module named `ImageViewerModule`. The CCT is a second module for the ImageViewer extension, called `ImageViewerCCT`.

To create an extension for a CCT with the extension developer tools, you create a baseline extension that includes a baseline module. You then add a baseline CCT as a separate module for the extension.

To create your CCT as an extension:

1. Create a baseline extension. See [Create a Baseline Extension](#).

As part of the ImageViewer example, use the following values when prompted:

Property	Value
Fantasy name	Image Viewer!
Extension name	ImageViewer
Vendor name	NetSuite
Version	1.0.0
Description	The best image viewer of the market!
Applications	Shopping, My Account, Checkout
File Types to Include	Sass, JavaScript
Initial Module Name	ImageViewerModule

2. Add a CCT to your baseline extension. [Create Custom Content Types for an Extension](#).

As part of the ImageViewer example, use the following values when prompted:

Property	Value
Label of the CCT	Image Viewer
Name of the CCT	ImageViewerCCT
Description for the CCT	Drag n' drop it anywhere to show the items image or a custom image
File types to include	Templates, Sass, JavaScript
Extension to add CCT	ImageViewer

After adding the CCT, your workspace includes the following files and folders:

File Name	Location	Description
ImageViewerModule.js	ImageViewer/Modules/ImageViewerModule/JavaScript	Entry point for the baseline extension. See Create the Entry Point JavaScript Files .
ImageViewerCCT.js	ImageViewer/Modules/ImageViewerCCT/JavaScript	Entry point for the CCT module. See Create the Entry Point JavaScript Files .
ImageViewerCCT.View.js	ImageViewer/Modules/ImageViewerCCT/JavaScript	View file for the CCT. See Implement the View File .
NetSuite_imageviewer_imageviewercct.tpl	ImageViewer/Modules/ImageViewerCCT/Templates	Template file for the CCT. See Implement the Template File .
_imageviewercct.scss _imageviewermodule.scss	ImageViewer/Modules/ImageViewerCCT/Sass ImageViewer/Modules/ImageViewerModule/Sass	Sass files for the baseline extension module and the CCT module. See Implement the Sass Files .
ImageViewer/manifest.json		Manifest file for the baseline extension and CCT module. The extension developer tools automatically generate all the required information in this file. You do not need to edit it. For more information about this file, see Edit the Extension Manifest .

Best Practices

Follow these best practices when creating a CCT:

- The above files (with the exception of the Sass files) are the minimum required for an extension with a CCT. Depending on your extension functionality, you may also need to create SuiteScript directories and SuiteScript files for the extension module or the CCT module.
- The extension development tools generate a baseline extension that you can use as a basis to build any extension for your SuiteCommerce site. For the example CCT in this topic, not all of the generated files are used. In addition, the files initially contain code that you overwrite with your extension-specific code. For more information about baseline extensions, see [Create a Baseline Extension](#) and [What Happens When You Create a Baseline Extension?](#)
- You can use a similar process as described in this topic to create a single extension that references multiple CCT modules. For more information, see [Create Additional Modules for an Extension](#).

Create the Entry Point JavaScript Files

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Each module in an extension contains an entry point. This includes the initial module built with the baseline extension and any added CCT modules. The entry point is required to mount the extension to the application. The entry point provides the connection between the CCT and SMT. For more information on the architecture and purpose of this file, see [Entry Point](#).

Each entry point includes a `mountToApp()` function. In an extension that implements a CCT, the `mountToApp()` function in the extensions entry point calls the `mountToApp()` function in the CCT module's entry point.

In addition, the `mountToApp()` function in the CCT module entry point calls the `getComponent('CMS').registerCustomContentType()` method. This method passes values for the `id` and `view` variables defined by the CCT module to the CMS Component.

The following diagram describes the file structure of the baseline extension and the CCT module files:

1	ImageViewer/	[Extension]
2	Modules/	
3	ImageViewer/	[Extension Module]
4	JavaScript/	
5	ImageViewerModule.js	[Extension Entry Point]
6	ImageViewerCCT/	[CCT Module]
7	JavaScript/	
8	ImageViewerCCT.js	[Extension Entry Point]

To define the entry points for the extension baseline module and the CCT module:

1. In the extension baseline module, define the dependencies and implement the `mountToApp()` function. The `mountToApp()` function in the extension baseline module calls the `mountToApp()` function for the CCT module.

For example, to define the entry point for `ImageViewerModule` in the example `ImageViewer` extension:

- a. Open the file `Modules/ImageViewerModule/JavaScript/ImageViewerModule.js`.
- b. Add the dependency on the CCT module.

For example, the following code in the `ImageViewerModule` entry point defines a dependency on the `ImageViewerCCT` module:

```
1 define(
2   'Netsuite.ImageViewer.ImageViewModule'
3   , [
4     'Netsuite.ImageViewer.ImageViewCCT'
5   ]
6   , function (
7     ImageViewerCCT
8   )
```

- c. Define the `mountToApp()` function.

For example, in the following code for the `ImageViewerModule` entry point, the `mountToApp()` function calls the `mountToApp()` function for the CCT module, `ImageViewerCCT`:

```
1 {
2   'use strict';
3
4   return {
5     mountToApp: function mountToApp (container)
6     {
7       ImageViewerCCT.mountToApp(container);
8     }
9   };
10  });
```

2. Save the JavaScript file.
3. Create the entry point for the CCT module by defining the dependencies and implementing the `mountToApp()` function.

For example, to define the entry point for `ImageViewerCCT` module in the example extension:

- a. Open the file `Modules/ImageViewerCCT/JavaScript/ImageViewerCCT.js`.
- b. Add the dependency on `ImageViewerCCT.View`.

```

1 define(
2   'NetSuite.ImageViewer.ImageViewerCCT'
3 ,
4   [
5     'NetSuite.ImageViewer.ImageViewerCCT.View'
6   ]
7 , function (
8   ImageViewerCCTView
9 )

```

- c. Define the `mountToApp()` function.

For example, for the `ImageViewer CCT`, the `mountToApp()` function may look similar to the following:

```

1 {
2   'use strict';
3
4   return {
5     mountToApp: function mountToApp (container)
6     {
7       container.getComponent('CMS').registerCustomContentType({
8         id: 'cct_netsuite_imageviewercct'
9         , view: ImageViewerCCTView
10        , options: {
11          container: container
12        }
13      });
14    }
15  };
16 }

```

The following table describes the values for the `id`, `view`, and `options` variables in the `registerCustomContentType()` method for the CCT module entry point:

Variable	Description
<code>id</code>	The <code>id</code> variable must be lowercase. It must match the Name field of the CMS Content Type record in NetSuite, that you create later.
<code>view</code>	The view <code>ImageViewerCCTView</code> is used to render the CCT.
<code>options</code>	The <code>options</code> variable registers additional data that the view constructor will receive. Options are passed as the arguments when initializing the <code>ImageViewCCTView</code> view.

4. Save the JavaScript file.
5. To continue creating an extension for a CCT, see [Implement the View File](#)

Implement the View File

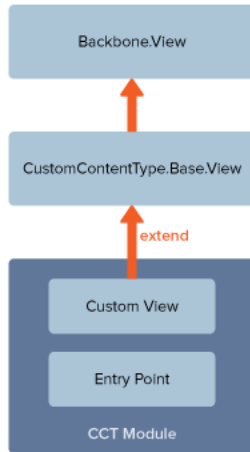
Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

This view defines what the CCT loads when it is dragged into the application.

To implement a view file:

1. Review the dependencies for the view.

The view requires `CustomContentType.Base.View` extends `Backbone.View`, initializes the CCT settings, and includes the base CCT class from which all custom CCT modules extend.



For example, your dependencies might look similar to the following:

```

1  define('NetSuite.ImageViewer.ImageViewerCCT.View'
2  , [
3    'CustomContentType.Base.View'
4    , 'Utils'
5
6    , 'netsuite_imageviewer_imageviewercct.tpl'
7
8    , 'jQuery'
9  ]
10 , function (
11   CustomContentTypeBaseView
12   , Utils
13
14   , netsuite_imageviewer_imageviewercct_tpl
15
16   , jQuery
17 )
  
```

2. Note the template that includes the HTML required to render the data defined by the view:

```

1  |  template: netsuite_imageviewer_imageviewercct_tpl
  
```

For more information about the templates for the CCT module, see [Implement the Template File](#).

3. Define the `initialize()` function.

This function is executed when the view is instantiated. The options variable is used to initialize the view constructor.

```

1  |  , initialize: function initialize (options)
2  |  |  {
3  |  |  |  if (options)
4  |  |  |  |  {
5  |  |  |  |  |  this.container = options.container;
6  |  |  |  |  }
7  |  |  |  }
8  |  |  |  this._initialize();
9  |  |  }
  
```

4. Note the `install()` function.

```

1  |  , install: function (settings, context_data)
2  |  |  {
3  |  |  |  this._install(settings, context_data);
  
```

```

4
5      // call some ajax here
6
7      var promise = jQuery.Deferred();
8      return promise.resolve();
9  }

```

Note: Until the promise resolves, you cannot edit the settings of this CCT.

5. Define the contextDataRequest variable.

The CCT might require access to the data, provided by the contextData objects. In this case, the ImageViewer CCT requires access to item data. These objects contain all the context data available by default, depending on where you dropped the CCT.

```

1 | , contextDataRequest: ['item']

```

6. Define validateContextDataRequest.

You must override validateContextDataRequest to return true in case the data does not exist in the area where you drop the CCT in the SMT admin.

```

1 | , validateContextDataRequest: function validateContextDataRequest()
2 | {
3 |     return true;
4 | }

```

7. Define the getContext() method.

The templates receives a single object returned by the getContext() method. This can be a property defined in each associated Custom Record within a settings object. The settings object contains each **Field ID** for the custom record. You access the values for the fields in the getContext() function.

The following table lists the fields in the custom record for the example ImageViewer CCT:

Field	Description
custrecord_cct_ns_ivcct_valign	Sets the vertical alignment of a text object.
custrecord_cct_ns_ivcct_text	Declares some text to display.
custrecord_cct_ns_ivcct_imageurl	Declares the URL to an image.

```

1 | // ...
2 | var texts = []
3 | , imageUrl = ''
4 | , valign = this.valign[this.settings.custrecord_cct_ns_ivcct_valign] || this.valign['3'];
5 |
6 | if (this.contextData.item)
7 | {
8 |     var item = this.contextData.item();
9 |
10 |     texts = [item.keyMapping_name];
11 |
12 |     var thumbnail = item.keyMapping_thumbnail;
13 |     imageUrl = (thumbnail && thumbnail.url) ? thumbnail.url : thumbnail;
14 | }
15 |
16 | var set_text = Utils.trim(this.settings.custrecord_cct_ns_ivcct_text)
17 | , set_texts = set_text ? set_text.split('\n') : []
18 | , set_imageUrl = Utils.trim(this.settings.custrecord_cct_ns_ivcct_imageurl);
19 |
20 | texts = set_texts.length ? set_texts : texts;
21 | imageUrl = set_imageUrl ? set_imageUrl : imageUrl;
22 |

```

```
23 | // ...
```

For the example ImageViewer CCT, your view might look similar to the following:

```
1  define('NetSuite.ImageViewer.ImageViewerCCT.View'
2  , [
3    'CustomContentType.Base.View'
4    , 'Utils'
5
6    , 'netsuite_imageviewer_imageviewercct.tpl'
7
8    , 'jQuery'
9  ]
10 , function (
11   CustomContentTypeBaseView
12   , Utils
13
14   , netsuite_imageviewer_imageviewercct_tpl
15
16   , jQuery
17 )
18 {
19   'use strict';
20
21   return CustomContentTypeBaseView.extend({
22
23     template: netsuite_imageviewer_imageviewercct_tpl
24
25     , initialize: function initialize (options)
26     {
27       if (options)
28       {
29         this.container = options.container;
30       }
31
32       this._initialize();
33     }
34
35     , install: function (settings, context_data)
36     {
37       this._install(settings, context_data);
38
39       // call some ajax here
40
41       var promise = jQuery.Deferred();
42       return promise.resolve();
43     }
44
45     , contextDataRequest: ['item']
46
47     , validateContextDataRequest: function validateContextDataRequest()
48     {
49       return true;
50     }
51
52     , valign: {
53       '1': 'top'
54       , '2': 'center'
55       , '3': 'bottom'
56     }
57
58     , getContext: function getContext()
59     {
60       var texts = []
61       , imageUrl = ''
62       , valign = this.valign[this.settings.custrecord_cct_ns_ivcct_valign] || this.valign['3'];
63
64       if (this.contextData.item)
65       {
66         var item = this.contextData.item();
67
68         texts = [item.keyMapping_name];
```

```

69     var thumbnail = item.keyMapping_thumbnail;
70     imageUrl = (thumbnail && thumbnail.url) ? thumbnail.url : thumbnail;
71 }
72
73 var set_text = Utils.trim(this.settings.custrecord_cct_ns_ivcct_text)
74 , set_texts = set_text ? set_text.split('\n') : []
75 , set_imageUrl = Utils.trim(this.settings.custrecord_cct_ns_ivcct_imageurl);
76
77 texts = set_texts.length ? set_texts : texts;
78 imageUrl = set_imageUrl ? set_imageUrl : imageUrl;
79
80 return {
81     hasText: !!texts.length
82     , texts: texts
83     , hasImage: !!imageUrl
84     , imageUrl: imageUrl
85     , valign: valign
86 };
87
88 }
89 });
90 });
91 });

```

8. Save the view file.

Note: The above example uses the valign object to map the values of the customlist_cct_ns_ivcct_valign list in the custom record for the CCT.

Implement the Template File

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

The template is necessary to render the data defined by the view.

To implement the template file:

1. In the ImageViewer/Modules/ImageViewerCCT/Templates directory, open NetSuite_imageviewer_imageviewercct.tpl.
2. Create the HTML required to render the data defined by the view.

Based on the example ImageViewerCCT, your template might look similar to the following:

```

1  <div class="imageviewercct">
2    <div class="imageviewercct-container">
3      {{#if hasImage}}
4        
5      {{/if}}
6      {{#if hasText}}
7        <div class="imageviewercct-container-caption imageviewercct-container-caption-{{valign}}">
8          {{#each texts}}
9            <h2 class="imageviewercct-container-caption-title">{{this}}</h2>
10           {{/each}}
11         </div>
12       {{/if}}
13     </div>
14   </div>

```

3. Save the template.

Implement the Sass Files

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Depending on the options that you choose, the extension development tools can generate Sass files when you create a baseline extension, add a module, or add a custom content type module to an extension. Use the Sass file to style the HTML included in the template file.

The tools generate the Sass file in the `Sass/` directory for the module and use the following naming convention:

```
1 | _<module_name>.scss
```

For more information, see [Sass Best Practices for Themes](#).

To implement the Sass files for the CCT:

1. Open the Sass file for the CCT module.

For example, In the `ImageViewer/Module/ImageViewerCCT/Sass` directory, open `_imageviewercct.scss`.

2. Add the Sass code.

Based on the example `ImageViewer CCT`, your Sass styles in the module directory might look similar to the following:

```
1 .imageviewercct {
2   position: relative;
3 }
4
5 .imageviewercct-container{
6   overflow:hidden;
7   height:100%;
8   text-align:center;
9 }
10
11 .imageviewercct-container-image{
12   width: 100%;
13 }
14
15 .imageviewercct-container-caption-top {
16   top:0px;
17   margin-top: 25px;
18 }
19
20 .imageviewercct-container-caption-center {
21   top:50%;
22   transform: translateY(-50%);
23 }
24
25 .imageviewercct-container-caption-bottom {
26   bottom:0px;
27   margin-bottom: 25px;
28 }
29
30 .imageviewercct-container-caption{
31   position:absolute;
32   width:100%;
33   text-align:center;
34   padding: 0px 25px;
35 }
36 h2.imageviewercct-container-caption-title{
```



```

37     text-shadow: 1px 1px 1px #000,
38                 -1px -1px 1px #000,
39                 -1px 1px 1px #000,
40                 1px -1px 1px #000;
41     font-size:$sc-font-size-m * 2.66;
42     line-height: $sc-font-size-m * 2.66;
43     color:$sc-color-primary;
44 }
45 }

```

3. Save the Sass file.

Test and Deploy the CCT Extension

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

After you create the extension for your CCT, you can test your extension in your local environment or deploy the extension to your NetSuite account. See one of the following topics:

To test or deploy your extension:

1. If you have not already done so, set up your NetSuite records. Complete the tasks defined in [Site Management Tools Custom Content Type](#).
The parameters required for setup are listed in cct array within the extension manifest. See [Edit the Extension Manifest](#) for details. These attributes are also listed in the terminal window when you first create the CCT using the developer tools.
2. Test or deploy your extension. See one of the following topics:
 - [Test an Extension on a Local Server](#)
 - [Deploy an Extension to NetSuite](#)

Access Model and Collection Data in a Child View

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

This tutorial describes how to add the detailed description of each item to a search results page. The detailed description of an item contains HTML that is typically only displayed on the product details page (PDP). But, in this tutorial, you learn how to develop an extension that displays item descriptions when a shopper is viewing items on the product list page (PLP).

Note: The procedures in this tutorial require extensibility API classes that are only available with SuiteCommerce and SuiteCommerce Advanced 2020.2 and later releases. For more information, see the [extensibility API reference](#).

Read the following sections to prepare for and then complete the tutorial:

- [contextData and contextDataRequest Properties](#)
- [Practices to Avoid](#)
- [Prerequisites](#)

- To develop an extension to access model and collection data in a child view

contextData and contextDataRequest Properties

The key capabilities you are using in this tutorial are the `contextData` and `contextDataRequest` properties:

- `contextData` is a property of every view, but it is only populated with data in some of those views.
- `contextDataRequest` is a property available to views that are children of views with `contextData`. You can call `contextDataRequest` by passing in an array of types of data you want returned.

Views can return four types of objects and each view may support multiple object types. The object types that a view can return is specific to each view class. However, most views do not support requests.

When requests are supported and a data object is returned, its values are specific to the view instance. For example, if you are working on a product details page (PDP), `contextData` potentially supports requests for the product and item model. Whether `contextData` supports requests for both the product and item model, or just one of those models, depends on the particular product page the user is viewing.

The following table lists the parent view classes that have `contextData` properties and also lists the types of data that they can provide to child views.

contextData Object	Class Availability	Information Returned
category	Facets.Browse.View	The category model
item	ItemRelations.RelatedItem.View Facets.ItemCell.View ProductDetails.Base.View	The item model
itemlist	Facets.Browse.View	An object that contains the item collection
product	ProductDetails.Base.View	The parent model for a product, which also contains the item model

Keep in mind that the availability shown in the preceding table cascades, which means that you do not have to add your child view directly into the classes listed. If your child view has a parent, grandparent, and so on, that is listed in the preceding table, `contextDataRequest` will be available.



Important: Remember that `contextData` is only populated with data in the view classes listed in the preceding table and in the child views of the listed view classes.

Practices to Avoid

When working with current releases of SuiteCommerce and SuiteCommerce Advanced, avoid the following practices when developing extensions.

Practice	Description
Do not add a child view by extending the <code>childViews</code> property.	In older SCA versions, it was common to extend the <code>childViews</code> property of a view and then add your view as an additional property. This practice works for older SCA versions that do not support the extensibility API, but it is not recommended for current SCA versions as it does not produce a sustainable and stable extension. This approach also requires adding the view as a dependency to your file, which is not permitted for core modules. For

Practice	Description
	more information, see Extensibility Layer Safe Mode in the Extensions Subtab configuration property reference.
Avoid customizing the template in the theme	For sites that use a managed theme, you cannot customize any of the theme's files. In general, you should not customize a theme's template files to implement new pieces of functionality. When developing an extension like the one in this tutorial, the goal is to add new information into your site's pages while leaving existing information intact. There is no reason to modify any existing code to facilitate that, including templates. Developing an extension like the one in this tutorial is the best way to ensure your customization remains stable across different sites and different versions.

Prerequisites

Before you can complete this tutorial, you need to:

- Install and configure SuiteCommerce or SuiteCommerce Advanced (2020.2 and later versions) as described in [Install Your Commerce Website Applications](#)
- [Install Node.js](#)
- [Install Gulp.js](#)
- [Set Up Extension Developer Tools](#)
- Understand the [Anatomy of an Extension](#)
- Set up items for your Commerce website as described in [Web Store Items](#)

To develop an extension to access model and collection data in a child view

1. [Expose the Field to the Relevant Field Set.](#)
2. [Create the Extension Framework.](#)
3. Use the PLP component's `addChildViews()` method in your entry point file to target an existing view into which you can inject your customized view. See [Create the Entry Point File](#) for instructions.
4. Create a custom view using `SCView` that:
 - a. Requests `item` context data from the parent view.
 - b. Accesses the `contextData` object with `contextDataRequest`, passing the required field through to the template context.
 - c. Conditionally renders the child view if, and only if, the shopper is viewing the page using the List display type.

See [Create the View File](#) for instructions.
5. [Create the Template](#) that renders the detailed description, which can include HTML.
6. [Test the Extension.](#)
7. [Troubleshoot the Extension.](#)

Expose the Field to the Relevant Field Set

Before you start to work with the code for this extension, you need to have the field you want included in the items API response. For this tutorial scenario, you are dealing with search results on a product list page (PLP) that use the (default) search field set. So, add a standard field called **Detailed Description** (`storedetaileddescription`) to the default search field set. To learn more about modifying field sets, see the help topic [Define Field Sets](#).

After saving the field set and record, make sure you rebuild the search index.

Note: The code samples provided in this tutorial assume you are working with the **Detailed Description** (storedetaileddescription) field described in the preceding paragraph. If you choose to work with a different standard or custom field, make sure you alter the code samples accordingly.

Create the Extension Framework

You can create an extension manually or by using the `gulp extension:create` command as described in [Create a Baseline Extension](#). For this tutorial, create your extension manually as described in the following steps.

1. Access the top-level extension development directory you created when you installed the extension developer tools.
2. In the top-level extension development directory, unless it already exists, create a new directory named `Workspace`.
3. In the `Workspace` directory, set up the folders for this extension so that they look like this:

```
Workspace/
  CDRExample
    Modules/
      CDRExample/
        JavaScript/
          Templates/
```

4. In the `Workspace/CDRExample` directory, create a `manifest.json` file and place the following code in that file.

```
{
  "name": "CDRExample",
  "fantasyName": "Context Data Request Example",
  "vendor": "ExampleVendor",
  "version": "1.0.0",
  "type": "extension",
  "target": "SCA,SCS",
  "target_version": {
    "SCA": ">=20.2.0",
    "SCS": ">=20.2.0"
  },
  "description": "An example extension to demonstrate contextDataRequest",
  "skins": [],
  "templates": {
    "application": {
      "shopping": {
        "files": [
          "Modules/CDRExample/Templates/cdr_example.tpl"
        ]
      }
    }
  },
  "javascript": {
    "entry_points": {
      "shopping": "Modules/CDRExample/JavaScript/CDRExample.js"
    },
    "application": {
      "shopping": {
        "files": [
          "Modules/CDRExample/JavaScript/CDRExample.js",
          "Modules/CDRExample/JavaScript/CDRExample.View.js"
        ]
      }
    }
  }
}
```

```

    }
  },
  "local_folder": "Workspace\\CDRExample"
}

```

Create the Entry Point File

To show the **Detailed Description** (storedetaileddescription) field, you need to render a view and template that have access to that information. To create that view, first you need to create an entry point file.

In the Workspace/CDRExample/Modules/CDRExample/JavaScript directory, create an entry point file named CDRExample.js that contains the following code.

```

define('CDRExample', [
  'CDRExample.View'
], function (
  CDRExampleView
) {
  'use strict';
  return {
    mountToApp: function mountToApp (container) {
      var PLP = container.getComponent('PLP');
      if (PLP) {
        PLP.addChildViews(PLP.PLP_VIEW, {
          'ItemViews.Price': {
            'CDRExample.View': {
              childViewIndex: 0,
              childViewConstructor: function () {
                return new CDRExampleView({PLP: PLP})
              }
            }
          }
        })
      }
    }
  }
})

```

In the preceding code, notice that:

- You are accessing the PLP (Product List Page) component. This is necessary because:
 - You want to use its addChildViews() method.
 - You want to access information it has on the current display type.
- You add the child view using verbose syntax, addChildViews(), which accepts two view injection targets instead of the usual one.
 - The reason for this is you need to be as specific as possible. For example, there could be other instances of ItemViews.Price throughout your Commerce website, and you do not necessarily want to inject your new child view into those.
 - Therefore, you use the verbose method, addChildViews(), attached to a specific visual component (the PLP component) and pass in a reference of the main PLP view, which keeps things in scope. For more information about verbose syntax, see [Add a Child View](#).

Create the View File

The view is where most of the work of the extension takes place. For this tutorial, use the SCView module in the view file that you create.

Note: If you are developing for SuiteCommerce or SuiteCommerce Advanced 2020.2.0 or later, the view generated when you use the `gulp extension:create` command does not implement currently recommended API methods. Starting with the 2020.2 release, you should use `SCView` in your extensions instead of `Backbone.View`. `SCView` extends from `Backbone.View`, inheriting all its methods and properties. `SCView` also provides additional methods for Commerce site development. For more information, see the [extensibility API reference](#).

In the `Workspace/CDRExample/Modules/CDRExample/JavaScript` directory, create a file named `CDRExample.View.js` that contains the following JavaScript:

```
define('CDRExample.View', [
  'SCView',
  'cdr_example_tpl'
], function (
  SCViewModule,
  cdr_example_tpl
) {
  'use strict';
  var SCView = SCViewModule.SCView;
  function CDRExampleView (options) {
    SCView.call(this, options);
    this.template = cdr_example_tpl;
    this.contextDataRequest = ['item'];
    this.displayType = options.PLP.getDisplay().id;
  }
  CDRExampleView.prototype = Object.create(SCView.prototype);
  CDRExampleView.prototype.constructor = CDRExampleView;
  CDRExampleView.prototype.render = function () {
    if (this.displayType == 'list') {
      SCView.prototype.render.call(this);
    }
  }
  CDRExampleView.prototype.getContext = function () {
    return {
      storedetaileddescription: this.contextData.item().storedetaileddescription
    }
  }
  return CDRExampleView
});
```

In the preceding JavaScript, notice the following details about the `CDRExampleView` function:

- You are making a `contextDataRequest` for item data.
- You are creating a custom `displayType` property that uses the PLP component's `getDisplay()` method, which returns a data object about the user's current display type.

After setting the view's prototype and setting the constructor function, you override the view's `render` property with a custom function.

Important: The `render` property on `SCView` is protected, which means you are allowed to override it. Before overriding a property, check the API documentation to determine if it is permitted. Never override private properties in your customizations.

The override you are putting in place for `render` calls the parent class's `render` property if, and only if, the display type ID that is passed in is `list`. This mechanism prevents the view from being rendered when other display types, like `grid` or `table`, are active.

Finally, in the `getContext` property, you create a function that defines the context object that is sent to the template. You can see that after implementing `contextDataRequest` in the constructor, it is here in this part of the code where you finally access the `contextData`.

The `item()` object is essentially a copy of the item model, which allows you to get the `displayType` property you want from it.

Create the Template

In the `CDRExample/Modules/CDRExampleModule/Templates` directory, create a template file named `cdr_example.tpl`. Put the following line in the file and then save it.

```
{{storedetaileddescription}}
```

Notice that the preceding value is wrapped in triple curly braces—a feature of Handlebars that prevents HTML escaping.

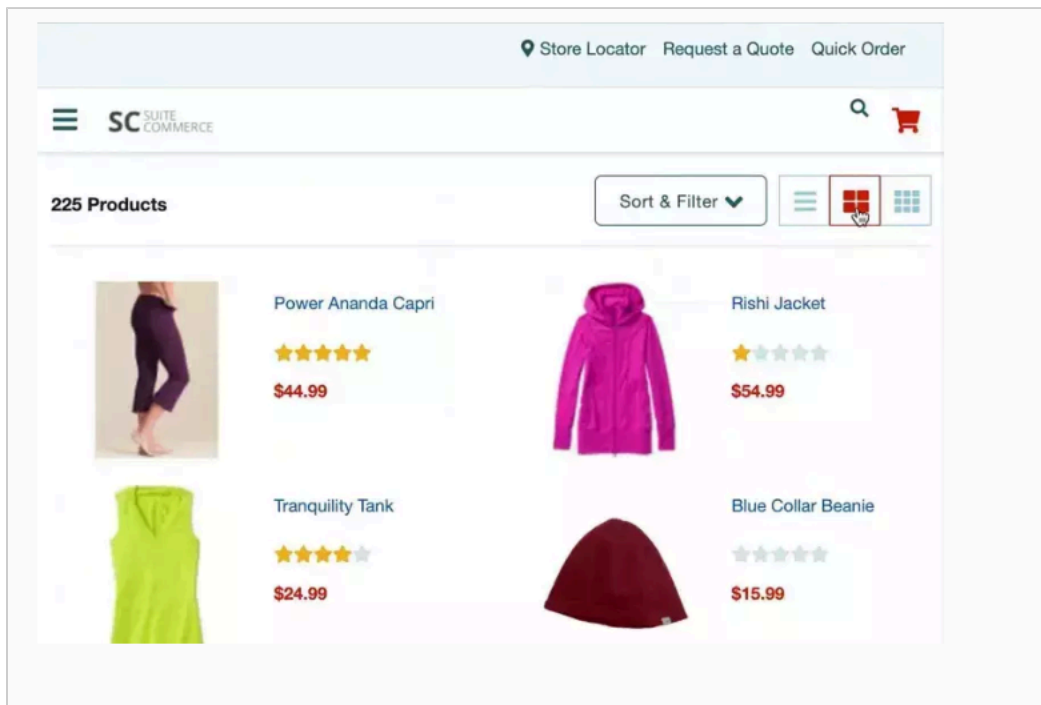
In cases where you are certain that the HTML you are passing into the template is safe (that is, valid HTML from a trusted, secure source), you can tell Handlebars to interpret the stringified value as raw HTML. This has the effect of injecting the HTML directly into the template's source.

Because you expect this field to always be HTML served straight from the item record stored in your NetSuite account, you can use the triple curly braces to prevent HTML escaping.

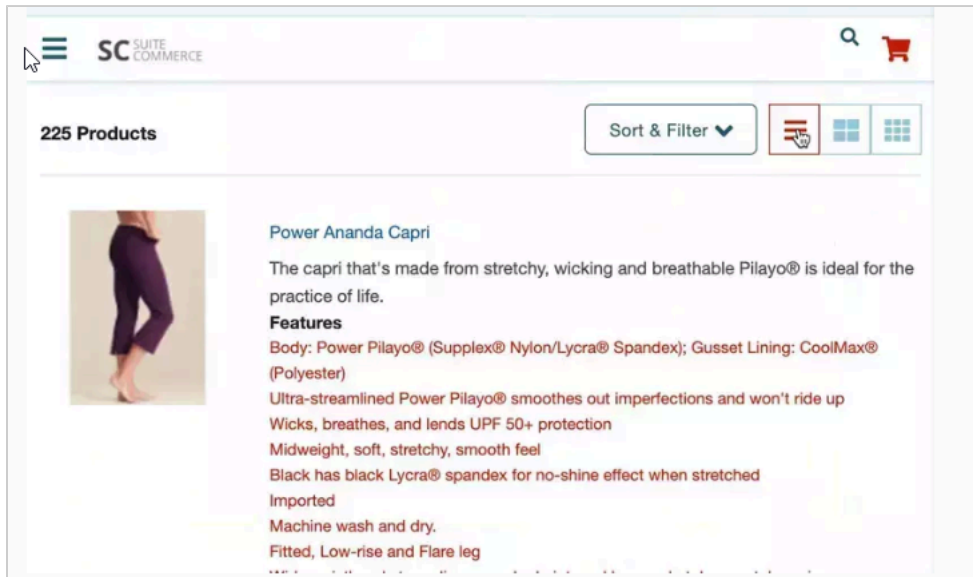
Test the Extension

Start your local server and then navigate to the local version of your Commerce website. For step-by-step instructions, see [Test an Extension on a Local Server](#).

Visit a product list page and change the display type. When you view the grid and table displays, there is no change, as shown in the following screen shot.



However, when you view the list display type, you should see the detailed description for each listed product:



Troubleshoot the Extension

If you encounter these errors, double-check the code for your extension as described in the following table.

Error	Potential Issue
contextData is undefined	Make sure you've added contextData to one of the views listed in the table in contextData and contextDataRequest Properties (or one of their child views). The contextData object is only available in specific scenarios and not every view will expose data this way.
Detailed Description (storedetaileddescription) field is undefined	Check the data received from the items API to make sure the storedetaileddescription field is there. If not, double-check these settings: ■ You added the field to the field set ■ You saved both the field set row and form, and also the website setup record ■ The data exists in the item record ■ You ran a search re-index If all the settings are correct and the storedetaileddescription field is not included in the data received from the items API, perform a cache invalidation request for the relevant URL path. If necessary, you may need to invalidate the entire domain's cache. If you get this error and the storedetaileddescription field is included in the data received from the items API, log the object to the console and check for typos in the field name.

Manage SuiteCommerce Analytics Data Cookie Preferences

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

The SuiteCommerce Analytics Data feature lets you track and measure user behavior and navigation on a SuiteCommerce website. As well as enabling the feature in NetSuite, you also need to install the SuiteCommerce cookie consent extension, which provides a way for shoppers to manage their consent about the use of tracking and analytics cookies on a SuiteCommerce website. If you prefer a different user experience or want to introduce additional functionality, you can create a custom extension that utilizes methods available in the extensibility API.

Note: The API methods of the UserProfile component allow you to save and fetch cookie preferences from NetSuite. Use other JavaScript browser methods to set the cookie in the user's browser, for example, `document.cookie`.

Prerequisites

- SuiteCommerce Analytics Data is installed in your account. Analytical data related to shopper navigation and behavior on your SuiteCommerce website is managed with the SuiteCommerce Data Analytics feature. See the help topic [SuiteCommerce Analytics Data Overview](#).
- The Application Performance Management for SuiteCommerce (SC APM) SuiteApp is installed in your account. APM lets you measure application performance and speed, as well as register application actions in NetSuite. See the help topic [Application Performance Management \(APM\) for Commerce](#).

Create Extension Files

Create a baseline extension to get the basic extension files and folder structure, and then create the following files.

```

1 MyCookiesExtension
2   \Modules
3     \MainModule
4       \JavaScript
5         NetSuite.MyCookiesExtension.MainModule.js
6         NetSuite.MyCookiesExtension.Banner.View.js
7         NetSuite.MyCookiesExtension.Footer.View.js
8       \Templates
9         netsuite_mycookiesextension_banner.tpl
10        netsuite_mycookiesextension_footer.tpl

```

The following tables describes the purpose of each file in the extension.

File	Description
NetSuite.MyCookiesExtension.MainModule.js	This file is entry point file for the extension. It instantiates the required SuiteCommerce API components and loads the header and footer views.
NetSuite.MyCookiesExtension.Banner.View.js	A view that shows a form allowing website visitors to set their cookie preferences. It appears above the website header.

File	Description
NetSuite.MyCookiesExtension.Footer.View.js	A view that shows a link that allows a website visitor to show the cookie banner. It appears in the footer placeholder of the website.
netsuite_mycookiesextension_banner.tpl	The template file for the banner view.
netsuite_mycookiesextension_footer.tpl	The template file for the footer view.

Update the Entry Point File

The entry point file contains the code that runs when the extension module is first mounted to the application. So, instantiate the Layout component in the entry point file to be able to add the banner and footer views to the page. You should also instantiate the UserProfile component and pass it to the instance of the banner view in an object.

When you add a view to a page with `addChildView()`, you must specify where on the page you want the view to be displayed. In this case, the banner view will appear at the top of the page at the Header placeholder. The footer view will appear at the bottom of the page at the Footer placeholder.

NetSuite.MyCookiesExtension.MainModule.js

```

1  define(
2    'NetSuite.MyCookiesExtension.MainModule',
3    [
4      'NetSuite.MyCookiesExtension.Banner.View',
5      'NetSuite.MyCookiesExtension.Footer.View'
6    ],
7    function (MyCookiesExtensionBannerView, MyCookiesExtensionFooterView) {
8      'use strict';
9
10     return {
11       mountToApp: function mountToApp (container)
12       {
13         var layout = container.getComponent('Layout');
14         var userProfile = container.getComponent('UserProfile');
15
16         if (layout) {
17           layout.addChildView('Header', function() {
18             return new MyCookiesExtensionBannerView({userProfile: userProfile});
19           });
20
21           layout.addChildView('Footer', function() {
22             return new MyCookiesExtensionFooterView();
23           });
24         }
25       }
26     };
27   });

```

Create a View for the Cookie Banner in the Header

The banner view contains the bulk of the logic that lets website visitors accept or reject the SuiteCommerce analytics cookie. To capture the user's preferences, create a form that lets the shopper select which cookies they want to accept or reject. The form should let users accept or reject different types of cookies, for example, marketing cookies, performance cookies, or analytical cookies. The example code in this topic lets a website visitor accept or reject the analytical cookie only (the essential cookies cannot be rejected).

You can use the [SCView](#) class to define a view for your extension. In the view, add a reference to the template that displays a form. You can use a simple form with checkboxes or a more complex form with additional functionality or features.

The following example shows a truncated version of the view file with the truncated parts replaced with `// <...>`. You can read about the truncated parts below in other sections of this topic.

NetSuite.MyCookiesExtension.Banner.View.js

```

1  define (
2    'NetSuite.MyCookiesExtension.Banner.View',
3    [
4      'SCView',
5      'jQuery',
6      'netsuite_mycookiesextension_banner.tpl'
7    ],
8    function(SCViewComponent, jQuery, netsuite_mycookiesextension_banner_tpl) {
9      'use strict';
10
11      var SCView = SCViewComponent.SCView;
12
13      function MyCookiesExtensionBannerView(params) {
14        SCView.call(this);
15
16        this.template = netsuite_mycookiesextension_banner_tpl;
17
18        this.userProfile = params.userProfile;
19        var userProfile = this.userProfile;
20
21        // Initialisation tasks. See below.
22        // <...>
23      }
24
25      MyCookiesExtensionBannerView.prototype = Object.create(SCView.prototype);
26
27      MyCookiesExtensionBannerView.prototype.constructor = MyCookiesExtensionBannerView;
28
29      MyCookiesExtensionBannerView.prototype.getContext = function() {
30        return;
31      }
32
33      MyCookiesExtensionBannerView.prototype.getEvents = function() {
34        return {
35          'click #myCookiesExtensionSavePreferences': 'savePrefs',
36          'click #myCookiesExtensionClosePreferences': 'closePrefs',
37        }
38      }
39
40      // Gets cookie preferences from the banner form. See below.
41      MyCookiesExtensionBannerView.prototype.savePrefs = function() {
42        // <...>
43      }
44
45      // Gets the value of the analytics cookie from the browser. See below.
46      MyCookiesExtensionBannerView.prototype.getBrowserConsentCookie = function() {
47        // <...>
48      }
49
50      // Populates the cookie consent form.
51      MyCookiesExtensionBannerView.prototype.populateCookiesConsentForm = function(consentCookieSettings) {
52        jQuery(function() {
53
54
55      MyCookiesExtensionBannerView.prototype.closePrefs = function() {
56        jQuery('#myCookiesExtensionBanner').hide();
57        if (consentCookieSettings.ANALYTICAL == true) {
58          jQuery('#myCookiesExtensionPreferenceAnalyticalCookies').prop('checked', true);
59        }
60      });
61    }
62
63    return MyCookiesExtensionBannerView;

```

```

64     }
65 };

```

Define Initialization Tasks

Add initialization tasks in the constructor function of the view. The first task is to get the browser consent cookie. If the cookie does not exist, show the cookie consent form. If the cookie does exist, check whether the browser cookie is newer than the server cookie. If the browser cookie is newer, update the server cookie and then update the browser cookie. If the browser cookie is not newer, update the browser cookie with the server cookie.

```

1  var browserConsentCookie = this.getBrowserConsentCookie() || null;
2
3  if (browserConsentCookie) {
4
5      // Unencode cookie
6      var consentCookieData = JSON.parse(window.atob(browserConsentCookie));
7
8      var consentCookieTimestamp = consentCookieData.timestamp;
9      var consentCookieSettings = consentCookieData.consentSettings;
10     var consentCookieId = consentCookieData.id;
11
12     // Populate cookies preferences form
13     this.populateCookiesConsentForm(consentCookieSettings);
14
15     // Check if user is logged in.
16     userProfile.getUserProfile().then(function(profile) {
17
18         if (profile.isloggedin) {
19             userProfile.getCookieOptions().then(function(cookieOptions) {
20
21                 // If the browser cookie preferences timestamp is newer than the server cookie preferences
22                 // timestamp, we need to update the server cookie preferences (and the browser
23                 // cookie preferences thereafter).
24                 if (consentCookieTimestamp > cookieOptions.consentLastUpdate) {
25                     userProfile.setCookieOptions(consentCookieSettings).then(function(result) {
26
27                         // Get server cookie preferences we just set so that we can update
28                         // the browser cookie accordingly.
29                         userProfile.getCookieOptions().then(function(cookieOptions) {
30
31                             var browserCookieOptions = {};
32
33                             browserCookieOptions.id = consentCookieId;
34                             browserCookieOptions.timestamp = cookieOptions.consentLastUpdate;
35                             browserCookieOptions.consentSettings = {};
36
37                             Object.keys(cookieOptions).forEach(function(k, i) {
38                                 if (k != 'consentLastUpdate') {
39                                     browserCookieOptions.consentSettings[k] = i == 1 ? true : false;
40                                 }
41                             });
42
43                             var browserCookieValue = window.btoa(JSON.stringify(browserCookieOptions));
44                             document.cookie = "trackingConsentCookie=" + browserCookieValue;
45                         });
46
47                     }).catch(function(result) {
48                         console.log(result);
49                     });
50                 }
51             } else {
52                 // If the server cookie preferences are newer, update the browser cookie.
53                 MyCookiesExtensionBannerView.prototype.setBrowserConsentCookie(cookieOptions);
54                 MyCookiesExtensionBannerView.prototype.populateCookiesConsentForm();
55             }
56         });
57     }
58     else {

```

```

59     console.log('User not logged in. Nothing to do. :-(');
60   }
61   });
62 }
63 else {
64   jQuery(function() {
65     jQuery('#myCookiesExtensionBanner').show();
66   });
67 }

```

Get or Set the Browser Cookie

When the extension loads, check if the browser consent cookie specific to SuiteCommerce Analytics Data exists and return its value with the `getBrowserConsentCookie()` function. The consent cookie name should be 'trackingConsentCookie' and its value should be a Base64-encoded string.

The following example shows one way to **get** the browser cookie.

```

1  MyCookiesExtensionBannerView.prototype.getBrowserConsentCookie = function() {
2    var cookies = document.cookie.split(';');
3    var browserConsentCookie = null;
4
5    var browserConsentCookie = cookies.find(function(element) {
6      if (element.trim().startsWith('trackingConsentCookie')) {
7        return element;
8      }
9    });
10
11    if (browserConsentCookie) {
12      var consentCookieValueStart = browserConsentCookie.indexOf('=');
13      browserConsentCookie = browserConsentCookie.substring(consentCookieValueStart + 1);
14
15      if (browserConsentCookie.length > 0) {
16        return browserConsentCookie;
17      }
18    }
19  }

```

The following example shows one way to **set** the browser cookie.

```

1  MyCookiesExtensionBannerView.prototype.setBrowserConsentCookie = function(cookieOptions) {
2    var browserCookieOptions = {};
3
4    browserCookieOptions.id = consentCookieId;
5    browserCookieOptions.timestamp = cookieOptions.consentLastUpdate;
6    browserCookieOptions.consentSettings = {};
7
8    Object.keys(cookieOptions).forEach(function(k, i) {
9      if (k !== 'consentLastUpdate') {
10        browserCookieOptions.consentSettings[k] = i == 1 ? true : false;
11      }
12    })
13
14    var browserCookieValue = window.btoa(JSON.stringify(browserCookieOptions));
15
16    document.cookie = "trackingConsentCookie=" + browserCookieValue
17  }

```

Get and Format Consent Cookie Preferences

When the website visitor clicks the Save button, get the visitor's preferences from the form and create an object that contains the preferences in the correct format. The `savePrefs()` function (see below) gets the form values in the banner, creates a cookie preferences object, and encodes the object to a Base64 string. The cookie preferences object must be in the following format:

```

1 {
2   id: <id>,
3   timestamp: <timestamp>,
4   consentsettings: {<preferenceName>: <preferenceValue>, <...>}
5 }

```

- **id** - A unique ID as a string. You can generate an ID by concatenating the value of timestamp and the browser user agent.
- **timestamp** - A timestamp generated by `Date.now()`.
- **consentsettings** - An object that contains a list of key/value pairs, each of which corresponds to a cookie preference name `<preferenceName>`, and its value `<preferenceValue>`. The value must be a boolean - true or false.

You can include any number of cookie preferences in the `consentsettings` property of the object. It is important, however, that the name of the cookie preference for analytics tracking is 'ANALYTICAL'. Using any other name for the analytical cookie will be ignored. Additionally, the SuiteCommerce Analytics Data feature only tracks user behavior and navigation if the value of ANALYTICAL is true.

Then create a Base64-encoded string from the object and save the preferences with the `setCookiePreferences()` function. Hide the banner to indicate the website visitor's preferences have been saved. The visitor can show the banner again by clicking the link in the footer.

```

1 MyCookiesExtensionBannerView.prototype.savePrefs = function() {
2
3   var cookieConsentPreferences = {};
4   var cookieOptions = {};
5
6   var now = Date.now();
7   var browserUserAgent = navigator.userAgent.replaceAll(' ', '');
8
9   // Get form values
10  cookieConsentPreferences.ANALYTICAL = jQuery('#myCookiesExtensionPreferenceAnalyticalCookies').is(':checked') ? true : false;
11  cookieConsentPreferences.ESSENTIAL = true;
12
13  cookieOptions.id = now + browserUserAgent;
14  cookieOptions.timestamp = now;
15  cookieOptions.consentSettings = cookieConsentPreferences;
16
17  var browserCookieValue = window.btoa(JSON.stringify(cookieOptions));
18
19  // Try to set cookie options. If the user is logged in, cookies can be set on the server side.
20
21  if (this.userProfile) {
22    this.setCookiePreferences(cookieOptions, browserCookieValue);
23  }
24
25  jQuery('#myCookiesExtensionBanner').hide();
26 }

```

Persist Cookie Preferences in NetSuite

To save the visitor's preferences in NetSuite, create a separate function in the banner view called `setCookiePreferences()` with two arguments. The first argument you pass to the function is an object that contains the visitor's preferences in the format required to save the object on the server side. The second argument is the Base64-encoded string of the visitor's preferences that will be saved on the client side in a cookie.

In the `setCookiePreferences()` function, use the `setCookieOptions()` method of the `UserProfile` component to save the visitor's consent preferences. The method returns a promise, which if resolved, indicates that the consent preferences were saved successfully.

```

1 MyCookiesExtensionBannerView.prototype.setCookiePreferences = function(cookieOptions, browserCookieValue) {
2   var nlrum = window.NLRUM || null;
3   var actionId = null;
4
5   if (nlrum) {
6     actionId = nlrum.actionStart('Accept Privacy Consent');
7   }
8
9   this.userProfile.setCookieOptions(cookieOptions.consentSettings).then(function(result) {
10    console.log('setCookieOptions was successful.');
```

```

11  }).catch(function() {
12    console.log('setCookieOptions failed.');
```

```

13  }).always(function() {
14    document.cookie = "trackingConsentCookie=" + browserCookieValue;
15
16    if (nlrum) {
17
18      function getCookieOptions() {
19        var browserConsentCookie = MyCookiesExtensionBannerView.prototype.getBrowserConsentCookie() || null;
20        var consentCookieData = JSON.parse(window.atob(browserConsentCookie));
21
22        if (consentCookieData) {
23          return {
24            id: consentCookieData.id,
25            timestamp: consentCookieData.timestamp,
26            consentSettings: formatCookieOptions(consentCookieData.consentSettings)
27          };
28        }
29        return null;
30      }
31
32      function formatCookieOptions(cookieOptions){
33        let options = [];
34
35        Object.keys(cookieOptions).forEach(
36          (option) => {
37            options.push(option + '|' + (cookieOptions[option] ? 'T' : 'F'));
38          }
39        )
40
41        return options.join(',');
42      }
43
44      var actionData = {
45        siteUrl: window.location.hostname,
46        shopperAnalyticsConsent: getCookieOptions()
47      };
48
49      nlrum.actionEnd(actionId, actionData);
50    }
51  });

```

The `setCookieOptions()` method only works if the website is on a secure connection (HTTPS) and if the user is logged in. If either of the conditions is not met, the method fails and returns a rejected promise. Even if the consent preferences cannot be saved in NetSuite, the `setCookiePreferences()` function sets the cookie on the client side. When the user logs in successfully the next time, the user's preferences will be saved in NetSuite, provided the cookie still exists.

When saving cookie consent preferences in NetSuite, you can also log the action in Application Performance Management (APM) for Commerce. Use the `window.NLRUM` object to log the action. You must first create an action ID by calling `actionStart()` with the string 'Accept Privacy Consent' as its argument. Using any other string will be ignored by APM. Then call `actionEnd()`, passing in the action ID and the visitor's cookie preferences in the required format.

```

1 {
2   siteUrl: shop.netsuite.com,
3   shopperAnalyticsConsent: {
4     id: "1648808771790Mozilla/5.0(WindowsNT10.0;Win64;x64;rv:98.0)Gecko/20100101Firefox/98.0",
5     timestamp: 1648808771790,

```

```

6 |     consentSettings: "ESSENTIAL|T,ANALYTICAL|T"
7 |   }
8 | };

```

See the help topic [Application Performance Management \(APM\) for Commerce](#) for more information about installing and using APM.

Finally, the banner view requires a template to render the cookie preferences form.

netsuite_mycookiesextension_banner.tpl

```

1 | <div id="myCookiesExtensionBanner" class="" style="display: none; margin-left: 20px; padding: 10px;">
2 |   <div>We use cookies! </div>
3 |   <form>
4 |     <input type="checkbox" id="myCookiesExtensionPreferenceEssentialCookies" name="ESSENTIAL" checked="" disabled=""> <b>Essen
5 |     tial</b> - These cookies are required for the website to function correctly.<br>
6 |     <input type="checkbox" id="myCookiesExtensionPreferenceAnalyticalCookies" name="ANALYTICAL"> <b>Analytical</b> - We use analyti
7 |     cal cookies to measure your interaction on the website when you are logged in so that we can improve your shopping experience. For
8 |     more information, see our <a href="#">Cookies Policy</a>.<br>
9 |     <input type="button" id="myCookiesExtensionSavePreferences" style="background-color: rgb(21,111,144); color: white; font-
10 |     weight: bold; float: left; width: 100px" value="Save"> <input type="button" id="myCookiesExtensionClosePreferences" style="back
11 |     ground-color: white; color: rgb(21,111,144); display: inline; font-weight: bold; margin-left: 16px; width: 100px" value="Close">
12 |   </form>
13 | </div>

```

Create a View for the Footer

The view for the footer is a relatively simple view that shows a link in the footer area of the website. Clicking the link shows the cookies banner in the header and scrolls the page to the top. To add an event listener for the click event on the link, use `getEvents()` to define the event listener and the event handler. The event handler is a short function that uses jQuery to show the banner's div element.

```

1 | define('NetSuite.MyCookiesExtension.Footer.View',
2 | [
3 |   'SCView',
4 |   'jQuery',
5 |   'netsuite_mycookiesextension_footer.tpl'
6 | ],
7 | function (SCViewComponent, jQuery, netsuite_mycookiesextension_footer_tpl) {
8 |   'use strict';
9 |
10 |   var SCView = SCViewComponent.SCView;
11 |
12 |   function MyCookiesExtensionFooterView() {
13 |     SCView.call(this);
14 |
15 |     this.template = netsuite_mycookiesextension_footer_tpl;
16 |   }
17 |
18 |   MyCookiesExtensionFooterView.prototype = Object.create(SCView.prototype);
19 |
20 |   MyCookiesExtensionFooterView.prototype.constructor = MyCookiesExtensionFooterView;
21 |
22 |   MyCookiesExtensionFooterView.prototype.getContext = function() {
23 |     return;
24 |   }
25 |
26 |   MyCookiesExtensionFooterView.prototype.getEvents = function() {
27 |     return {
28 |       'click #myCookiesExtensionCookiesLink': 'showCookiesBanner'
29 |     }
30 |   }
31 |
32 |   MyCookiesExtensionFooterView.prototype.showCookiesBanner = function() {
33 |     jQuery('#myCookiesExtensionBanner').show();
34 |     window.scrollTo(0,0);

```



```
35     }  
36  
37     return MyCookiesExtensionFooterView;  
38 }  
39 );
```

The footer view requires a template to show a link that displays the consent form.

netsuite_mycookiesextension_footer.tpl

```
1 <div id="myCookiesExtensionFooter" class="" style="display: block; margin-left: 20px; padding: 10px;">  
2   <a href="#" id="myCookiesExtensionCookiesLink">Cookies Policy </a></div>  
3 </div>
```

Related Topics

- [Develop Extensions](#)
- [Extension Tutorials](#)

Do More With Extensions

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

As you develop your extension, you may want to expand the capabilities of the extension, or include additional functionality that enhances the behavior or usage of the extension. Explore the following topics for more information.

- [Access Configuration Properties](#)
- [Localize Text in an Extension](#)
- [Extensions for Site Management Tools \(SMT\)](#)
- [Modal Dialogs in SCIS](#)
- [Implement Asynchronous and Synchronous Methods](#)
- [Make a View Available Anywhere in Your Templates](#)

SuiteCommerce and SuiteCommerce Advanced enable you to provide translations for the templates and text that your extension or theme adds or changes in a site. You can also include translations for any customer-facing text in the Shopping, MyAccount, or Checkout application that is associated with your customization. For more information, see [Include Translations in Themes](#).

Access Configuration Properties

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

The SuiteCommerce Configuration is a record in your NetSuite account that lets you configure properties without customizing source code. You access the record through NetSuite to configure common properties and modify application behavior for a specified domain. For more information about this record, see the help topic [Configure Properties](#).

Note: The SuiteCommerce Configuration record is part of the SuiteCommerce Configuration SuiteApp.

You can build your extension to access any configuration property value for the associated domain using the `getConfig(key)` method of `EnvironmentComponent`. The key parameter is the ID of a property in the configuration record. The method returns a string, array, object, or Boolean, depending on the configuration property. For more information about the configuration properties and their respective IDs, see the help topic [Configuration Properties Reference](#).

The following example shows how to use the `getConfig(key)` method to access the value of the `checkoutApp.skipLogin` property:

```
1 var environmentComponent = container.getComponent('Environment');
2 var skipLogin = environmentComponent.getConfig('checkoutApp.skipLogin');
```

In the above example, the `skipLogin` variable can be true or false, depending on whether the **Skip Checkout Login** property is enabled for the domain.

Guidelines for getConfig(key)

Read and understand the following guidelines when working with the getConfig(key) method:

Dot-Notation

You can use dot-notation to access the hierarchy of configuration properties. For example, using getConfig('checkoutApp') will return a JavaScript object that contains the skipLogin property value.

You can also access the hierarchy of configuration properties by string value or by name. All three of the following API calls return the value of the skipLogin property:

```
1 getConfig('checkoutApp.skipLogin')
2 getConfig('checkoutApp').skipLogin
3 getConfig().checkoutApp.skipLogin
```

Read-Only Values

The getConfig() method returns a copy of the configuration or configuration property. As a result, values for configuration properties are read-only. To make changes to a configuration property, you must change the property using the SuiteCommerce Configuration record. See the help topic [Configuration](#) for details.

Avoid using getConfig() without a key

Although it is possible to use getConfig() without specifying a key, it is not recommended. If you do not specify a key, the application attempts to return the entire configuration object. On some pages in the application, this may return a very large object, which will result in an internal error (InternalError: too much recursion) or a range error (RangeError: Maximum call stack size exceeded).

Localize Text in an Extension

📌 Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

This section describes how to configure and manage localized text in an extension. An extension can include dictionary files with localized text, one file per language. When a user sets a language for a website, the application replaces string literals specified in the extension code with language-specific strings, which appear on the site.

Use the Extensibility API to replace string literals with the localized version in JavaScript files on the frontend and in SuiteScript files on the backend. Use the Handlebars translate custom helper to replace text with the localized version in template files. In addition, the Extensibility API includes a method to add additional translated string literals to the language-specific dictionary files.

translate() Method Syntax

The translate() method has the following syntax:

```
1 | _('<string {0} {1}>').translate(0, 1)
```

In the above syntax, string is the string value to translate and 0 and 1 are optional parameters for values within string. Parameter values are optional but you can include an unlimited number, as required.

Custom Handlebars Helper

Use the custom Handlebars translate helper in the template file.

The translate custom helper uses the `_ .translate()` function in `HandlebarsExtra.js`, which is located in the JavaScript subdirectory of the `HandlebarsExtras` application module.

To configure and manage localized text in an extension:

1. Enable your site for localized content and set up the desired languages. For more information, see the help topic [Set Up NetSuite for Localized Content](#).
2. For printing localized strings from JavaScript or SuiteScript files, use the `_ .translate()` method as shown below:

1. Locate the desired text to translate.

For example, you want to translate the string `Extension` in the following code example:

```
1 | return Backbone.View.extend({
2 |   template: mine_extension_module_list_tpl
3 |   , title: 'Extension'
4 |   , events: {
5 |     'click [data-action="remove"]': 'removeModule'
6 |   }
7 | ...
```

2. Use the `translate()` method for the string `Extension`:

```
1 | return Backbone.View.extend({
2 |   template: mine_extension_module_list_tpl
3 |   , title: _('Extension').translate()
4 |   , events: {
5 |     'click [data-action="remove"]': 'removeModule'
6 |   }
7 | ...
```

3. To translate the text in a template file, use the `translate` custom helper.

For example, a template file contains the following code with the string `Task name`:

```
1 | ...
2 | <span>
3 |   {{'Task name'}}
4 | </span>
5 | ...
```

The following code example shows the template file with the `translate` custom helper:

```
1 | ...
2 | <span>
3 |   {{translate 'Task name'}}
4 | </span>
5 | ...
```

Similar to the `translate()` method, you can use parameters in the `translate` custom helper. For example:

```
1 | {{translate 'There are ${0} apples in the tree' appleCount}}
```

3. To add additional localization values in the extension code, use the `EnvironmentComponent.setTranslation(locale, keys)` method in the Extensibility API.

The following example shows how to use the `setTranslation()` method to add translation key values to the `es_ES` locale:

```
1 | environmentComponent.setTranslation('es_ES', [{key: 'Faster than light', value: 'Más rápido que la luz'}])
```

 **Note:** You can use this method in any JavaScript or SuiteScript file.

Extensions for Site Management Tools (SMT)

 **Applies to:** SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Read the following topics to learn how to enable site administrators to customize a SuiteCommerce website using Site Management Tools (SMT). As an extension developer, you can develop extensions:

- To expose new page types to site administrators or let them customize existing page types by introducing new layouts they can choose from. For more information, see [Create Page Types](#).
- To register new page types so site administrators can define each new page as a specific type. For more information, see [Register Page Types and Templates for SMT](#).

Create Page Types

 **Applies to:** SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Page types let Site Management Tools (SMT) administrators introduce a variety of pages and layouts to a website. When SMT admins create new pages in SMT, they define it as a specific page type. This can be either the default landing page or any newly created page type introduced through an extension. SMT admins can also change the layout of existing pages. When SMT admins edit an existing core page, they create an enhanced page and can select from a list of different layouts, based on the layout options defined by the extension.

However, for these elements to be available in SMT, extension developers must first set up an extension to register page types and templates using the Extensibility API.

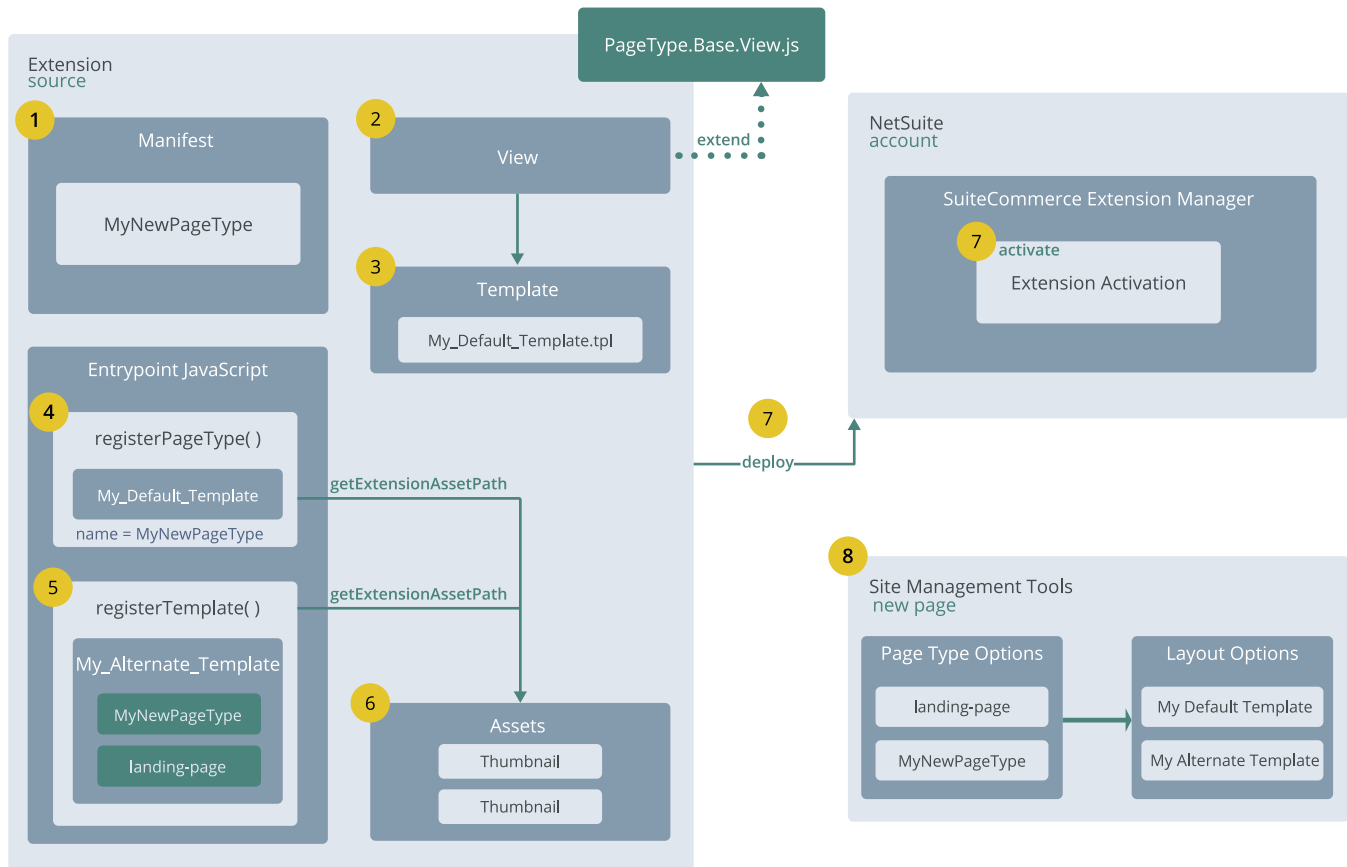
 **Note:** Page types are associated with CMS Page Type records. For more information on these records in NetSuite, see the help topic [CMS Page Type Record](#). For more information on creating new pages using SMT, see the help topic [Site Management Tools Page Layout Selector](#).

The following steps explain how to set up a Commerce extension to:

- Create a new CMS Page Type record in NetSuite

- Register new page types to be available when creating new pages for a Commerce website using SMT
- Create templates and define them as either a default layout for a page type or as alternative layout options

Note: These steps assume that you have already started building an extension using the extension developer tools. See [Extension Development Steps](#) for a checklist of steps required to build an extension.



Step	Description	More Information
1	Set up your extension to create a new CMS Page Type record in NetSuite. This requires manually editing your extension's manifest.json file. During testing and development, you need to create this CMS Page Type record manually. Note: Your extension automatically creates the CMS Page Type record when it is installed into an account from a bundle. During early testing, however, your extension is not yet part of a bundle, so you must manually create it.	Add a CMS Page Type Record
2	Create a view for your new page type. This JavaScript file is essential to render the instance of the page type. This file must extend PageType.Base.View.js.	Create a View to Render a Registered Page Type

Step	Description	More Information
3	Create a template to act as the default layout for your registered page type.	Site Management Tools Templates and Areas
4	Register your new page type using the Extensibility API.	Register a Page Type
5	Register alternative layouts using the Extensibility API. Every page type includes a default layout. However, you can register other templates to be available as alternative layout options for a new or existing page types.	Register a Template
6	Create template thumbnails as assets. Each template file corresponds to a different layout option, selectable in SMT when creating a new page. Each template includes a thumbnail preview image that you include with the extension.	Create Layout Thumbnails
7	Deploy your changes. You can test your extension on a local server. However, you must first deploy your extension files to your account and activate the extension on a domain to test the functionality in SMT.  Important: You must activate the extension before any deployed changes take effect. See the help topic Manage Themes and Extensions for details.	Deploy an Extension to NetSuite.
8	Log into SMT and use the Layout Selector when creating a new page or editing an enhanced page.	Site Management Tools Page Layout Selector

Add a CMS Page Type Record

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

You set up your extension's manifest.json file to automatically create a new CSM Page Type record. When a user installs your extension from a bundle, NetSuite automatically creates the record based on the parameters you set here.

To set up your extension to create a CMS Page Type record:

1. Open your extension's manifest.json file.
This is located in your Workspace/[extension] folder.
2. Add the page object and types array.
Create a new object in the types array for each new page type you want your extension to create. See [Page](#) in the [Edit the Extension Manifest](#) topic for details.
3. Save your manifest.json file.

Page Types in a Development Environment

During early testing and development, no bundle exists for your extension. You can test an extension by deploying it to an account. However, since the bundle installation is what triggers the CMS Page Type

record creation, you must manually create the record in NetSuite to test your extension. This is not the typical user experience, but it is necessary to build the mechanism during development.

Observe these minimum requirements for testing a CMS Page Type record with your extension:

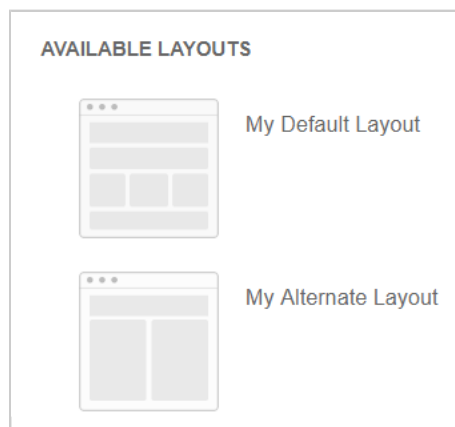
- The **Name** and **Display Name** fields are mandatory.
- The **Name** field must match the name parameter in the `registerPageType()` method in your extension's entrypoint JavaScript file. No spaces or special characters are allowed in this field. See [Register a Page Type](#) for details.
- To appear in SMT, the **CMS Creatable** field must be set to **Yes**.

To create a CMS Page Type record manually, see the help topic [CMS Page Type Record](#).

Create Layout Thumbnails

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Part of setting up a layout in your extension includes specifying a layout thumbnail. This image acts as a small preview or representation of the associated layout. Here is an example of thumbnails in use in Site Management Tools (SMT):



You can download a collection of stock images to use for your thumbnails or as a starting point when developing your own: [Stock SMT Layout Thumbnails](#).

If you create your own thumbnails, follow these best practices when creating thumbnails:

- When possible, use the stock thumbnails provided as a starting point.
- Format thumbnail images as SVG files. Other image types are supported, but SVGs provide more versatility and scalability with typically smaller file sizes.
- Limit file sizes to NetSuite file size limitations. See the help topic [Best Practices for Preparing Files for Upload to the File Cabinet](#) for details.
- Create thumbnail images with either of the following pixel dimensions:
 - 84 x 84 px
 - 180 x 180 px
 - 245 x 245 px

- Introduce thumbnail images within your extension as assets. This requires editing your extension's manifest.json file and using HTML helpers to register them. See the following help topics for details:
 - [Managing Theme Assets](#)
 - [Edit the Extension Manifest](#)
- Include a reference to these thumbnails when you register your page types or layouts. See [Register Page Types and Templates for SMT](#) and [HTML Helpers](#) for help.

Register Page Types and Templates for SMT

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

When site administrators create a new page in Site Management Tools (SMT), they define it as a specific page type. Site administrators can assign the Landing Page page type or any new page type, if registered using a SuiteCommerce extension.

As the extension developer, you use the Extensibility API to register new page types for this purpose. You can also register templates to either be the default layout for your new page type or as an alternative layout option for any new or pre-existing page type. When your extension is installed and activated, these page types and templates appear as selectable options when creating new pages or enhancing existing pages in SMT.

i Note: This topic is one step of a larger process, as explained in [Create Page Types](#).

For example, you create a new blog extension. Within the extension, you register a new Blog page type with a default template (layout). You also register two different templates as alternatives for your blog page type. Later, after installing and activating your extension, **Blog** is now available as a page type when creating pages in SMT. After creating a new blog page, site administrators can keep the default layout registered for that page, or they can choose from one of the two predefined alternative layouts.

This tutorial explains how to:

Step	Description
Create a View to Render a Registered Page Type	This topic explains a few important details about any view required to render your new page type.
Register a Page Type	This topic explains how to access the Extensibility API to register a page type and declare a default template.
Register a Template	This topic explains how to access the Extensibility API to register a template and make it available as an alternative layout for one or more page types.

Create a View to Render a Registered Page Type

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

When you register a page type with the PageType Component, you must also create a view to render the data when a user navigates to an instance of that page type. When creating this view, follow these best practices:

- Any view handling requests for a registered page type must extend `PageType.Base.View.js`.
- Any view handling requests for a registered page type must be declared in the `registerPageType()` method registering the associated page type
- If you are creating a view to render a registered page type, you can use the `beforeShowContent()` to control when the page type is rendered. This is part of `PageType.Base.View.js`. When a route of this page type is changed, an instance of this class is created, waiting until the promise returns before resolving. This lets you get any required data while retrieving the CMS data at the same time.

See [PageType](#) for details on this method.

To set up a view to render a page type:

- Open the view file or create a new one. This file should be located in the extension's JavaScript directory.
- Declare `PageType.Base.View` as a dependency and add it to the module definition function.
- Declare the view's template file and add it to the module definition function.

The following example adds `My_New_Default_Template.tpl` as a dependency:

- Declare any third-party libraries as needed.

The following example declares `MyPageType.View.js` and `My_New_Default_Template.tpl` as dependencies:

```
1 define('MyPageType.View'
2 , [
3   'PageType.Base.View'
4   , 'My_New_Default_Template.tpl'
5 ]
6 , function (
7   PageType
8   , My_New_Default_Template_tpl
9 )
10 {
11   //...
```

- Set up the view to extend `PageType.Base.View`.

```
1 define('MyPageType.View'
2 , [
3   'PageType.Base.View'
4   , 'My_New_Default_Template.tpl'
5 ]
6 , function (
7   PageType
8   , My_New_Default_Template_tpl
9 )
10 {
11   'use strict';
12
13   return PageType.PageTypeBaseView.extend({
14     //...
```

- Include the template that your associated page type is using as the default layout.

This template is the same file declared when you register your page type in the extension's entry point JavaScript file (`registerPageType()` method). See [Register a Page Type](#) for details.

```
1 define('MyPageType.View'
2 , [
3   'PageType.Base.View'
4   , 'My_New_Default_Template.tpl'
```

```

5   , 'jQuery'
6   ]
7   , function (
8       PageType
9       , My_New_Default_Template_tpl
10      , jQuery
11    )
12  {
13    'use strict';
14
15    return PageType.PageTypeBaseView.extend({
16
17      template: My_New_Default_Template_tpl
18
19      , initialize: function (options) {
20        //...

```

7. Complete the view file as necessary to meet your needs.

Register a Page Type

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

For a new page type to function with SMT, register it as part of your extension using the Extensibility API. Call the following method, which is part of PageType component:

registerPageType()

See [PageType](#) for details on this component.

Note: Page Type registration is only required for new CMS Page Type records introduced along with your extension. See [Add a CMS Page Type Record](#) for details.

To register a page type:

1. Open your extension's entry point JavaScript file.
2. Add the view rendering your new page type as a dependency and add it to the module definition function.
3. Add the Utils library as a dependency and add it to the module definition function.

This is required to use the HTML helper needed to add a thumbnail image as an asset. The following example adds MyPageType.View and the Utils library as dependencies.

```

1  define(
2    'NewExtension.MyPageType'
3    , [
4      'MyExtension.View'
5      , 'MyPageType.View'
6      , 'Utils'
7    ]
8    , function (
9      MyExtensionView
10     , MyPageTypeView
11     , Utils
12   )
13   {
14     //...

```

4. In the `mountToApp()` method, register the page type by calling the `PageType` Component and the `registerPageType()` method.

See [PageType](#) for details on this method.

Your code should look similar to the following example:

```

1 //...
2 return {
3   mountToApp: function mountToApp (container)
4   {
5
6     //...
7
8     var pageType = container.getComponent('PageType');
9
10    pageType.registerPageType({
11      'name': 'MyNewPageType'
12    , 'view': MyPageTypeView
13    , 'options': {'myOptionProperty': 'myOptionValue'}
14    , 'defaultTemplate': {
15      'name': 'My_New_Default_Template.tpl'
16      , 'displayName': 'My Default Layout'
17      , 'thumbnail': Utils.getAbsoluteUrl(getExtensionAssetsPath('img/default123.png'))
18    }
19    });
20 //...

```

- **name** – This string must match the name field in the corresponding CMS Page Type record and within the page object in your extension's manifest.json file. No spaces or special characters are allowed.
- **view** – This string references the module's view JavaScript file that defines the template you declare as the default. This view must extend `PageType.Base.View.js`.
See [Create a View to Render a Registered Page Type](#) for details on creating this file.
- **options** – This object registers additional data that the view constructor will receive. Options are passed as the arguments when initializing the view.
- **defaultTemplate** – This object defines the template file used as the default layout for the registered page type. This accepts the following parameters:
 - **name** – This string declares the template file used as the default for the page type.
 - **displayName** – This string specifies the user interface label for the default template as rendered in SMT.
 - **thumbnail** – This string specifies the location of the thumbnail image for the layout, as rendered in SMT.

You introduce this image as an extension asset. Use the `getAbsoluteUrl` and `getExtensionAssetsPath` helpers to define the path to the thumbnail. See [HTML Helpers](#) for details on referencing this file. See [Create Layout Thumbnails](#) for more details on how to create this image.

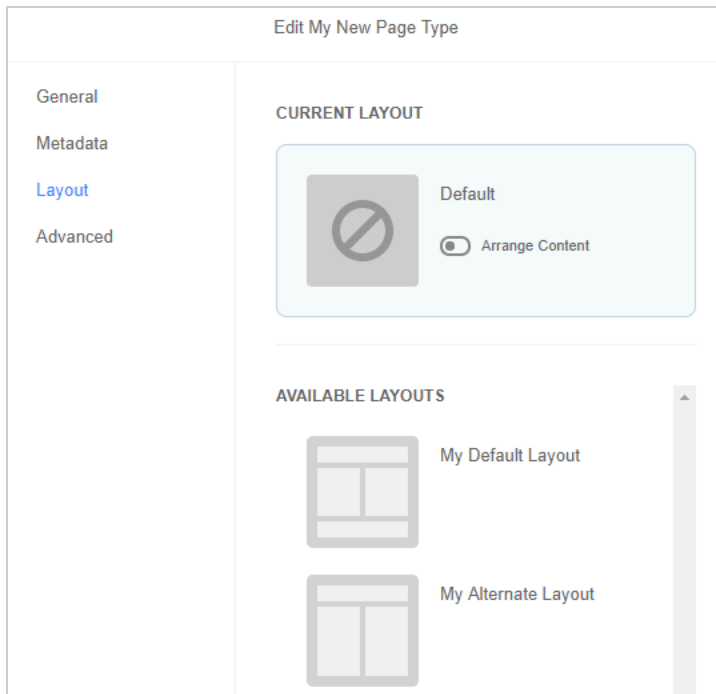
 **Note:** Registered templates must exist as part of the extension.

5. Save your entry point JavaScript file.

After registering your new page type, you must also include it in your extension's manifest.json file. This sets up your extension to create the associated CMS Page Type record in NetSuite. However, you must create CMS Page Type records manually during testing. See [Add a CMS Page Type Record](#) for details.

In the example code snippet above, you register a new page type, **MyNewPageType**, declaring `my_new_page_type_template.tpl` as the default layout. In SMT, this layout displays `default-thumbnail.png` as the thumbnail image with the label: **My Default Layout**. The view initializes the template and extends

PageType.Base.View.js. In SMT, you see the default layout and thumbnail for the page type as shown in the example below.



Register a Template

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Extension developers can register a template to be available as an alternative layout for any page type in SMT using the Extensibility API. To do this, call the following method, which is part of the PageType component:

`registerTemplate()`

Extension developers can specify templates to be alternative layouts for one or more page types. When an SMT admin creates a new page or enhances a pre-existing page, they can choose the default layout or select from a list of alternative layouts. These layouts are defined by the active extension.

To register a template as an alternative layout:

1. Open your extension's entry point JavaScript file.
2. In the `mountToApp()` method, register an alternative template using the PageType Component and the `registerTemplate()` method.

Your code should look similar to the following example:

```
1 //...
2 return {
3   mountToApp: function mountToApp (container)
```

```

4      {
5
6      //...
7
8      var pageType = container.getComponent('PageType');
9
10     pageType.registerPageType({
11         //...
12         // Optional page type registration code.
13         //...
14     }
15     });
16
17     pageType.registerTemplate({
18         pageTypes: ['MyNewPageType', 'cms-landing-page']
19     ,   template: {
20         name: 'My_New_Alternate_Template.tpl'
21         ,   displayName: 'My Alternate Layout'
22         ,   thumbnail: Utils.getAbsoluteUrl(getExtensionAssetsPath('img/two_column123.png'))
23     }
24     });
25     });
26 }
27 };
28 //...

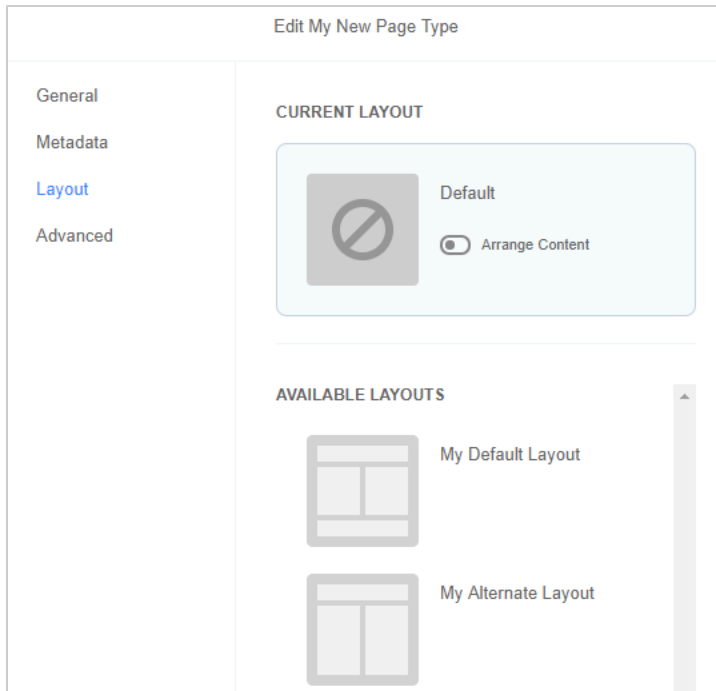
```

- **pageTypes** – This array declares the page types that accept the template as an alternative layout. The page types listed here can be any currently supported or newly registered page types. For a list of supported, core page types, see [Supported Page Types](#).
- **template** – This object defines the template and accepts the following parameters:
 - **name** – This string declares the template file name to be used as an alternative layout for the listed pageTypes.
 - **displayName** – This string specifies the user interface label for the layout, as rendered in SMT.
 - **thumbnail** – This string specifies the location of the thumbnail image for the layout, as rendered in SMT. This must be an absolute URL. As a best practice, introduce thumbnails as assets and use the `getAbsoluteUrl` and `getExtensionAssetsPath` helpers to define the path. See [HTML Helpers](#) for details.

3. Save your entry point JavaScript file.

Note: Registered templates must exist as part of the extension. For more information on creating templates to take advantage of SMT content areas, see the help topic [Site Management Tools Templates and Areas](#).

In this example code snippet, SMT provides `my-new-alternate-template.tpl` as an optional layout when an SMT admin creates or edits a page with **MyNewPageType** or **cms-landing-page** as the page type. This layout displays `alternate-thumbnail.png` as the thumbnail image with the label: **My Alternate Layout**. In SMT, you see the default layout and thumbnail for the page type as shown in the example below.



Supported Page Types

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

When editing a new page or enhancing an existing page (such as the Home Page, Order Details page, etc.), SMT admins can change the page layout, assuming the active extension allows it. To do so, the extension must register a template as a selectable alternative for the page type associated with the page being edited.

As the extension developer, you can register as an alternative layout for one or more page types. These can be any newly registered page type or a pre-existing, core page type (as listed below). To make a template available as an alternative layout for a pre-existing page, include the appropriate page type to the pageTypes array when you register your template. See [Register a Template](#) for details.

For example, to create an alternative layout for the Order Details enhanced page, add OrderDetail to the pageTypes array when you register the template.

The following lists includes the core page types supported by SuiteCommerce. These are available as CMS Page Type records with the latest version of SuiteCommerce or SuiteCommerce Advanced. However, to make these page types available for SMT, you must edit the associated record to make it CMS Creatable. See the help topic [CMS Page Types](#) for details.

AccountBalance

AddressBook

AddressEdit

Cart.Detailed.View

CasesDetail

CasesList

CreditCardDetails
CreditCardsList
CreditMemoDetail
DepositApplicationDetail
DepositDetail
EmailPreferences
facet-browse
forgot-password
home-page
InvoiceDetail
InvoiceDetail
landing-page
login
login-register
MyAccountOverview
NewCase
OpenInvoicesHistory
OrderDetail
OrderDetail
PaidInvoicesHistory
PasswordUpdate
PaymentDetail
PrintStatement
ProductDetails.Full.View
ProductDetails.QuickView.View
product-review-form
ProfileInfo
PurchaseDetail
PurchaseHistory
QuotesDetail
QuotesHistory
register
ReorderHistory

reset-password
returnAuthorizationConfirmation
returnAuthorizationFrom
ReturnsDetail
ReturnsHistory
TransactionHistory
WishlistDetail
WishlistList

Create Custom Content Types for an Extension

 **Applies to:** SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

When you create your baseline extension files, the developer tools provide a method to build your extension as a CCT. The developer tools provide on-screen instructions to assist you with developing your CCT. This requires:

- Creating a baseline extension
- Adding one or more baseline CCTs
- Customizing your CCT and preparing it for use with Site Management Tools

 **Note:** To implement a CCT using Site Management Tools, you must configure the **CMS Adapter Version** to 3 in the SuiteCommerce Configuration record. This property is located in the **Integrations** tab and **Site Management Tools** subtab.

To add a CCT to your baseline extension:

1. Open a command line or terminal.
2. Access the top-level extension development directory you created when you installed the extension developer tools.
3. Create a baseline extension, if you have not done so already. See [Create a Baseline Extension](#) for details.
4. Enter the following command:
gulp extension:create-cct
5. When prompted, enter the following information:

 **Important:** Unless otherwise directed, create all entries using only alphanumeric characters without spaces.

Set the Label of the CCT	Name the CCT to be added to your baseline extension. This can contain alphanumeric and special characters.
Set the Name of the CCT	Provide a name for your CCT as it appears in your development files and folders. Use only alphanumeric characters without spaces.

Set a Description for your CCT	Provide some text to describe your CCT.
For This CCT You Will Be Using	Select one or more file types your CCT requires (Templates, Sass, Configuration, JavaScript, SuiteScript). An asterisk (*) identifies a selected application.
Select Extension to Add CCT	Select the name of the extension to contain this module. This option only appears if your Workspace directory contains more than one extension.

6. Repeat these steps to add any additional CCTs as needed.

After you have successfully created your baseline CCT, you must set it up for use with Site Management Tools and customize your code to meet your needs as described in [To Build a CCT](#).

To Build a CCT:

1. Open the extension's manifest.json file. This can be found in Workspace/ <EXTENSION_DIRECTORY>/.

This file lists the following as part of the manifest metadata. You need this information when setting up your records in NetSuite.

icon	Equals a default icon that is visible in SMT Admin. As a default, the developer tools provide one to help you get started.
settings_record	Equals the ID of the custom record you associate with this CCT.
registercct_id	Equals the Name field of the CMS Content Type Record for your CCT. This is also the value of the id property within the registerCustomContentType() method of your CCT module's entry point JavaScript file.

2. Prepare your CCT for use with SMT.

Follow the instructions listed in the help topic [Site Management Tools Custom Content Type](#). Build your custom record and your CMS Content Type Record using the parameters explained in Step 1.

3. Develop your CCT module.

Follow the instructions listed in the help topic [Create a Custom Content Type](#). Ensure that your entry point JavaScript file incorporates your CMS Content Type record name, as described in Step 1.

4. Deploy and activate your extension. Then log into SMT to test confirm that your CCT was added correctly. See the help topic [Site Management Tools](#) for details on using SMT.

- See [Deploy an Extension to NetSuite](#) for instructions on deploying your extension.
- See the help topic [Manage Themes and Extensions](#) for instructions on activating your extension.
- [Overview](#)

Modal Dialogs in SCIS

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

The SCIS extensibility API enables you to create extensions for SCIS. It lets you show modal dialogs in response to events that occur in the cart, such as adding an item to the cart, applying a discount, or voiding a line.

The steps to create an SCIS extension are similar to the steps to create an extension for SuiteCommerce or SuiteCommerce Advanced:

1. Download and install the extension developer tools.
2. Create a baseline extension. Select SuiteCommerce InStore in the supported extension step. See [Create a Baseline Extension](#) for more information.
3. Add your extension code.
4. Test the extension locally, and then deploy.

When you create a baseline extension for SCIS, the developer tools generates the directory structure for the extension, as well as the JavaScript file for the initial module. You can find the initial module file in the directory structure at `[workspace folder]/[extension name]/Modules/JavaScript`.

The initial module file contains some basic scaffolding code to get started. The module name is generated from the information you entered when creating the baseline extension with gulp. The `define()` function lets you specify the name of the module, any dependencies, and a return function. Because the initial module has no dependencies, the second argument is an empty array `[]`. In the return function, the `mountToApp()` method initializes the module and returns data. The code for your extension goes in the `mountToApp()` method.

```

1  define(
2    'Company.SCISExtensionName.MainModule',
3    [],
4    function() {
5      'use strict',
6      return {
7        mountToApp: function mountToApp(container) {
8
9        }
10     }
11   }
12 );

```

Modal Dialog Types

Show a modal dialog when you want to show important information to the user or you want the user to confirm an action. Two types of modal dialog are available:

- Information dialog (or information window) – Shows a modal dialog with one button and space for a text message. The user must click the button to close the dialog and continue with the operation.
- Confirmation dialog (or option window)– Shows a modal dialog with two buttons and space for a text message. Use this type of dialog when you want the user to choose between one of two actions, usually to confirm or cancel an operation. Separate actions are defined for each button.

Modal dialogs have a predefined look and feel. You cannot modify the look and feel of a dialog, however, you can specify content of the dialog, such as the text in the dialog and whether a toast-style notification strip is shown in the dialog. You can also change the name of the buttons as required.

Define Modal Dialog Options

To create a modal dialog, you get an instance of the SCIS layout component and call the `showConfirmationPopup()` method with an object as its argument. The object contains a set of properties that let you specify the content of the dialog, such as the title, a message, and the text labels for the two buttons. The following example uses all available properties for the options of a confirmation dialog. The same properties are available to create an information dialog, except the `cancelationButtonText` property.

```

1 var layout = SC.Application('SCIS').getComponent('SCISLayout');
2
3 var confirmationPopupOptionsAgeRequirements = {
4   title: 'Age Verification',
5   toastType: 'warning',
6   toastContent: 'Verify customer is +18 years old.',
7   messageContent: 'Sale of this item prohibited to minors. Tap Age Confirmed only if the customer is +18 years old.',
8   confirmationButtonText: 'Age Confirmed',
9   cancelationButtonText: 'Remove Item'
10 }

```

Show the Modal Dialog After an Event Occurs

In our example, we want the modal dialog to appear before an item is added to the cart. Get an instance of the cart component to listen for the `beforeAddLine` event, which occurs just before the item is added. Use the `on()` method of the cart component to add an event listener for the `beforeAddLine` event.

The `on()` method accepts an event name and an event handler as its arguments. Contextual data is passed to the event handler as an object. In the case of the `beforeAddLine` event, properties of the object include the item's internal ID. Use dot notation to get the internal ID of the item.

```

1 var cart = SC.Application('SCIS').getComponent('Cart');
2
3 cart.on('beforeAddLine', function(data) {
4   if (data.line.internalid == '10101') {
5     layout.showConfirmationPopup(confirmationPopupOptionsAgeRequirements).then(function(result){
6       if (result.action === 'cancel') {
7         throw Error('Item not added to cart.');

```

If the internal ID of the item added to the cart matches a specified number (in this example, 10101), show the confirmation dialog with `showConfirmationPopup()`. Since this method returns a promise, use `then()` to attach a done callback. In the callback, check the value returned by `showConfirmationPopup()`. The value returned will be either 'cancel' or 'OK', depending on whether the cancelation or confirmation button was clicked. Finally, if the value returned from the modal dialog is 'cancel', throw `Error()` stops execution of the script, which prevents the item being added to the cart.

The following example shows the full sample code for a modal dialog in SCIS.

```

1 define(
2   'Company.SCISExtension.MainModule',
3   [],
4   function () {
5     'use strict';
6
7     return {
8       mountToApp: function mountToApp (container) {
9         var layout = SC.Application('SCIS').getComponent('SCISLayout');
10        var cart = SC.Application('SCIS').getComponent('Cart');
11
12        confirmationPopupOptionsAgeRequirements = {
13          title: 'Age Verification',
14          toastType: 'warning',
15          toastContent: 'Verify customer is +18 years old.',
16          messageContent: 'Sale of this item prohibited to minors. Tap Age Confirmed only if the customer is +18 years
17          old.',
18          confirmationButtonText: 'Age Confirmed' ,
19          cancelationButtonText: 'Remove Item'
20        }

```

```

21 |         cart.on('beforeAddLine', function(data) {
22 |             if (data.line.internalid == '10101') {
23 |                 layout.showConfirmationPopup(confirmationPopupOptionsAgeRequirements).then(function(result){
24 |                     if (result === 'cancel') {
25 |                         throw Error('Item not added to cart.');
```

Implement Asynchronous and Synchronous Methods

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

The Extensibility API handles tasks synchronously in the frontend and both synchronously and asynchronously in the backend (the backend includes two versions of each method). Asynchronous methods use the jQuery Deferred Object API. This API provides an easy way to register callbacks and monitor the success or failure of data transactions.

All asynchronous methods return a promise through the deferred object. A promise represents the eventual result of any asynchronous process. Any method that returns a deferred object is an asynchronous method.

The extensibility API implements the deferred object using the `jQuery.deferred()` method. Whenever an asynchronous operation runs, it returns a deferred promise.

This utility object allows you to relay the success or failure state of the operation and continue with your desired flow of execution.

Returning the Deferred object gives you the flexibility to use the promises returned by the default Deferred object of the Extensibility API or the `jQuery.deferred()` method. For more information about these promises, see <https://api.jquery.com/category/deferred-object> and www.promisejs.org.

i Note: The implementation of deferred is similar to standard promises, except that it uses `.fail()` instead of `.catch()`.

jQuery.deferred()

To use the promises returned by the jQuery library, you must instantiate `jQuery.deferred()` in your extension code.

For example, you might use `jQuery.Deferred().resolve()` to return a resolved promise in your code, especially in cases when you cannot use Backbone to implement asynchronous behavior.

The following example shows the instantiation in a JavaScript (frontend) or SuiteScript (backend) file:

```

1 | define('MyExtension', ['jQuery'], function(jQuery) {
2 |     var promise = jQuery.deferred();
```

```
3 | })
```

Asynchronous Methods

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Both the frontend and backend components provide methods that return a deferred promise.

For example, the `setOption(cart_option_id, value)` method of the `ProductDetailsComponent` (PDP) lets you set options for the currently displayed product. It returns a type of `Deferred.<Boolean>` that represents the eventual result of the set option operation, which is either `true` or `false`.

The following example show the usage of the `setOption(cart_option_id, value)` method. You can direct the extension to set the option and then check the value of the result after the promise returned by the method resolves:

```
1 | pdp.setOption(optionId, newValue)
2 | .then(
3 |   function(result) {
4 |     // check if result is false or true
5 |   }
6 | , function(error){
7 |   console.log(error);
8 | }
9 | );
```

Note: You can also implement similar functionality to make a call to an external, third-party validator to check the returned value and take different actions depending on the returned value.

You can also return a single promise for multiple method calls, where the promise does not resolve until both methods return a value.

In the following example, you call `container.getComponent('Cart')`, which gives you all methods associated with the `Cart` component. You then call both the `cart.getLines()` and `cart.getPromotions()` methods of `CartComponent`. Because this is asynchronous, the returned promise does not resolve until both methods complete and return a value:

```
1 | var cart = container.getComponent('Cart');
2 |
3 | // ...
4 |
5 | jQuery.when([cart.getLines(), cart.getPromotions()])
6 | .then(function (result)
7 | {
8 |   var lines = result[0]
9 |   , promotions = result[1];
10 | })
11 | .catch(function(error)
12 | {
13 |   console.log(error);
14 | });
```

The following example shows the above functionality for the `cart.getLines()` and `cart.getPromotions()` methods of `CartComponent` using a promise:

```
1 | var cart = container.getComponent('Cart');
2 |
```

```

3 // ...
4
5 Promise.all([cart.getLines(), cart.getPromotions()])
6 .then(function (result)
7 {
8     var lines = result[0]
9     , ship_methods = result[1];
10 })
11 .catch(function(error)
12 {
13     console.log(error);
14 });

```

Backend Asynchronous and Synchronous Methods

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

Backend components include both asynchronous and synchronous methods. Each asynchronous method also includes a synchronous version with the word Sync appended to the asynchronous method. For example, the asynchronous method: `getLines()` includes a synchronous version: `getLinesSync()`.

You can use asynchronous methods to reuse the frontend extension code for the `CartComponent` on the backend with the `BackendCartComponent`. The backend methods use the same signature as the frontend methods.

The following example shows the use of both asynchronous and synchronous versions of the `cart.getLines()` method in a SuiteScript file:

Note: In this example, `MyExtension`, `Vendor`, and `ExtensionName` are defined when you create a baseline extension using the developer tools.

```

1 define('Vendor.ExtensionName.1.0.0.MyExtension'
2 , [
3     'Application'
4 ]
5 , function(
6     Application
7 )
8 {
9     'use strict';
10
11     return {
12
13         myMethod: function()
14         {
15             var cart = Application.getComponent('Cart');
16
17             // asynchronous API
18             cart.getLines()
19             .then(function(result)
20             {
21                 nlapiLogExecution('DEBUG', 'Lines in the cart', JSON.stringify(result));
22             })
23             .catch(function(error)
24             {
25                 nlapiLogExecution('ERROR', 'Error getting lines in the cart', JSON.stringify(error));
26             });
27
28             // synchronous API
29             var lines = cart.getLinesSync()
30         }
31     };
32

```

33 | });

Note: The asynchronous methods in the backend currently use SuiteScript 1.0, which does not support asynchronous operations. As a result, although the asynchronous methods return a promise, the actual operation of the method is synchronous.

Make a View Available Anywhere in Your Templates

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

The `registerView()` method allows you to register a view to a component, making the view available anywhere that component is available. This functionality makes it easier to use simple views in any template in your theme. You should not use the `registerView()` method for every view, but instead use it:

- For views that are self contained and do not rely on context or model data.
- With designers in mind so that after you create a view, a designer is free to use it in any template they need.

The `registerView()` method is used in SuiteCommerce and SuiteCommerce Advanced (SCA) to allow the site search elements, the search button, and the keyword input box, to be used in more than one template. This is implemented in the source code for SCA, for example, by using `registerView()` to register the views for the site search elements with the layout component. Because the layout component is available globally, you can include the site search elements in your templates by adding `data-view="SiteSearch"` and `data-view="SiteSearch.Button"` where you want those elements to appear. This is one example of the flexibility that the `registerView()` method provides. If you want to learn more about the search elements, see [Customize Site Search Elements](#).

How to Use `registerView()`

You can add simple views using an entry point file in an extension. For example, if you want a "Hello World!" message to be available anywhere, include the following code in your entry point file:

```

1  define('HelloWorld'
2  , [
3    'HelloWorld.View.js'
4  ]
5  , function
6  (
7    HelloWorldView
8  )
9  {
10   'use strict';
11
12   return {
13     mountToApp: function mountToApp (container)
14     {
15       var Layout = container.getComponent('Layout');
16
17       if (Layout)
18       {
19         Layout.registerView('HelloWorld', function HelloWorld ()
20         {
21           return new HelloWorldView
22         });
23       }
24     }

```



```

25 | }
26 | })

```

Create a view that includes the following code:

```

1 | define('HelloWorld.View'
2 | , [
3 |   'helloworld.tpl'
4 | ]
5 | , function
6 | (
7 |   helloworld_tpl
8 | )
9 | {
10 |   'use strict'
11 |
12 |   return Backbone.View.extend({
13 |     template: helloworld_tpl
14 |   })
15 | });

```

Create a template that contains the following HTML:

```

1 | <p>Hello World!</p>

```

Now you can edit any template to include the message:

```

1 | <div data-view="HelloWorld"></div>

```

If you want to add some complexity, you can pass options in the constructor, such as the container object.

Why Use `registerView()` Instead of `addChildView()`?

The main benefit of the `registerView()` method is that it lets you separate the JavaScript for your Commerce website from your site's templates.

For example, if a developer creates a widget using `addChildView()` and writes the widget so that it renders on the home page, a designer can add the new widget to a home page template so that it is displayed on that page.

At a later date, if the business decides the widget needs to move to a different location, the developer must change the JavaScript so that it renders elsewhere and the designer must update the theme. And, both the JavaScript and the theme must be uploaded and activated again. However, if the developer uses `registerView()` instead, the designer is the only person who must make a change to use the widget in a different location.

For developers who make their SuiteCommerce extensions available to external customers, the `registerView()` method allows you to ship extensions that are flexible. Your customers are not limited to using the functionality provided by your extension in specific places that you define. Plus, your customers do not need to make updates to their JavaScript files to use the functionality. Instead, all your customers need to do is update their theme's templates.

The `registerView()` method is not a replacement for `addChildView()`, but it does complement `addChildView()` by providing more flexibility. You should use the two methods as follows:

- `addChildView()` — You know exactly where this child view should go and it is unlikely to move because it is, for example, context-specific.
- `registerView()` — This view could be used in several places and it is generic enough to work almost anywhere.

Can I Use Model Data with `registerView()`?

What if you use a service to retrieve data from NetSuite and you want to include that data in a view that is available globally? Will this approach work? This approach will work only if that service has been called at the time the view is rendered.

Typically, you use the `registerView()` method in an entry point file. Entry point files are called when the modules are first loaded. Models, however, are typically created only when a particular route has been taken, for example, when a shopper visits a specific page. So, if a user accesses that page and at the point of creating the model, you register the view, then you can pass the data along.

It is possible to delay the rendering of a view by performing the fetch in the view file and then only triggering `showContent()` when its promise resolves. This approach would work, too.

Keep in mind that although these approaches will work, they do not fall within the use cases the `registerView()` method was originally designed to support.

Overview

You can manage and maintain your theme or extension during its lifecycle. The following topics provide more information.

- [Management Resources for Themes and Extensions](#)
- [Deploy to a Sandbox or Custom SSP Application](#)
- [Bundle Themes and Extensions as SuiteApps](#)

Management Resources for Themes and Extensions

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

When developing themes and extensions, you can use the following tools to help maintain and manage those resources.

Note: The tools described in the following table are available for SuiteCommerce and the Aconcagua release of SuiteCommerce Advanced and later.

Tool or Resource	Description	For more information, see
Extension Manager	The Extension Manager helps you manage themes and extensions for each domain linked to your Commerce website. For example, you can use the Extension Manager to activate or deactivate available themes and extensions.	Manage Themes and Extensions Extension Manager Overview
SuiteBundler	SuiteBundler allows you to package groups of objects for distribution to other accounts. These packages are called bundles, or SuiteApps. NetSuite provides an assistant called the Bundle Builder that steps you through the process of creating bundles.	SuiteBundler Overview
SuiteCloud Development Framework (SDF)	SDF is a development framework that you can use to create SDF SuiteApps.	SuiteCloud Development Framework
NetSuite Sandbox	A NetSuite sandbox is an isolated test account where you can develop and test your customizations without affecting your production account. You can have one or more sandboxes provisioned for each production account.	NetSuite Sandbox About Sandbox Accounts

Deploy to a Sandbox or Custom SSP Application

The steps in the following procedures describe how to deploy to a sandbox account or to a custom SSP application.

- [Deploy to a NetSuite Sandbox](#)
- [Deploy to a Custom SSP Application](#)

Deploy to a NetSuite Sandbox

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Commerce websites also let you deploy to a sandbox account. See the help topics [NetSuite Sandbox](#) and [NetSuite Sandbox FAQ](#) for more information about sandbox accounts.

i Note: Before deploying to your sandbox account, you should verify that the Commerce bundles are installed. You may also need to delete the .nsdeploy file if you previously deployed to an account other than the sandbox account. Deleting this file enables you to enter the login credentials for the sandbox account.

To deploy to a sandbox account:

1. Go to your command line or terminal window.
2. Enter the appropriate command for your environment from the top-level directory of the source files (the same directory used during the tools installation).
 - Theme developer tools: `gulp theme:deploy`
 - Extension developer tools: `gulp extension:deploy`
 - Core SuiteCommerce Advanced developer tools: `gulp deploy`

If this is the first time you are deploying, the tools create a sub directory called DeployDistribution. The tools then compile the source files and output them to this directory.

i Note: If you have already created an authentication ID in a previous fetch or deploy operation, the developer tools do not prompt you for an ID and will continue to use the existing ID for each subsequent fetch or deploy. If you want to choose a different authentication ID or create a new one, use the `--to` parameter when you deploy. For more information, see [Token Based Authentication](#).

3. When prompted, enter the authentication ID that you use for your sandbox account.
4. In NetSuite, select the sandbox account you want to deploy to and the role you want to use for authentication. Click the **Allow** button to generate the authentication token and secret.

i Note: You must specify a role that provides the permissions needed to deploy to NetSuite. By default, the Administrator and SCDeployer roles provide these permissions. To learn more, including how to create a custom role that provides the required permissions, see [Developer Tool Roles and Permissions](#).

5. When prompted, navigate to Web Site Hosting Files for your Commerce installation:
 - SuiteCommerce: Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - SCS

- SuiteCommerce Advanced: Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - SCA <version> Development

After you enter your connection settings, the contents of the DeployDistribution folder on your local system are uploaded to the NetSuite file cabinet of your sandbox account. This process may take a few minutes. Wait for the process to complete before proceeding.

Deploy to a Custom SSP Application

i Applies to: SuiteCommerce Advanced

During installation of SuiteCommerce Advanced (SCA), the SSP applications included in the SCA bundle are automatically configured in your NetSuite account. Generally, you deploy customizations to the Development SSP application associated with your SCA release, as described in the [Deploy SCA Customizations to NetSuite](#) topic.

SSP Application Support for SuiteScript 2.0

As of the 2019.1 release, SCA supports SuiteScript 2.0. The SSP applications for SCA 2019.1 and later releases include resources to run services implemented using SuiteScript 2.0. These resources can be found in the Development2 and Source2 directories.

You can also deploy local files to a **custom** SSP application. Custom SSP applications enable you to test local customizations of SCA on the domain and site you specify. When you create custom SSP applications, NetSuite generates the directory structure in its file cabinet to which you can deploy your local files.



Important: If you are implementing the 2019.1 release of SCA or later and want to use custom SSPs for testing, you must create two custom SSP applications. You need one custom application to support the creation and deployment of SuiteScript 1.0 resources and one to support SuiteScript 2.0. In the following procedure, the custom application you create for SuiteScript 2.0 is referred to as **SSPv2**.

To deploy to custom SSP application, complete the following steps:

- [Step 1: Create a Custom SSP Application](#)
 - [Create a custom SSP application for SuiteScript 1.0](#)
(all implementations of SCA)
 - [Create a custom SSP application for SuiteScript 2.0](#)
(2019.1 and later implementations of SCA)
- [Step 2: Deploy Local Files](#)
(all implementations of SCA)
- [Step 3: Configure SSP Applications](#)
(all implementations of SCA)



Note: Before deploying to a Custom SSP Application, the SCA bundle must be installed in your account. You cannot deploy local files into an account without the bundle installed.

Step 1: Create a Custom SSP Application

If you are implementing the 2019.1 release of SCA or later and want to use custom SSPs for testing, you must create two custom SSP applications. You need one custom application to support the creation and deployment of SuiteScript 1.0 resources and one to support SuiteScript 2.0.

The NetSuite SSP application record defines the folder in the NetSuite file cabinet where website customization assets are stored. If you are unfamiliar with SSP applications in NetSuite, review [SSP Application Overview](#) in the NetSuite Help Center before proceeding.



Important: If you are updating a site that has been previously deployed to a custom SSP application, you do not need to create a new SSP application. To deploy to the existing SSP application, you need to know the Application Folder and the Application Name. You can find this information on the SSP Application Record for the site you are working on.

The following procedure applies to all implementations of SCA.

Create a custom SSP application for SuiteScript 1.0

1. Go to Commerce > Hosting > SSP Applications > New.
2. To create the record for your SuiteScript 1.0 assets, follow the steps outlined in the NetSuite Help topic [Create a SuiteScript 1.0 SSP Application Record](#) to provide the following information:
 - If you are implementing SCA 2019.1 or later, when naming your SSP application be sure to append v1 to the end. For example: MyCustomSSPv1
 - Specify a version of 1.x.
 - Create a new application folder and provide the HTML Hosting Root, Application Publisher, and Application Name for your application.

Creating an SSP application results in a directory structure in the NetSuite File Cabinet with the following path: **<HTML Hosting Root>: /SSP Applications/<Application Publisher>/<Application Name>**. For example, if you specify the following information for the application folder when creating an SSP application:

- **HTML Hosting Root:** Live Hosting Files
- **Application Publisher:** ProDev
- **Application Name:** ProDevSite

The resulting directory structure in the NetSuite File Cabinet is: Web Site Hosting Files/Live Hosting Files/SSP Applications/ProDev/ProDevSite

Create a custom SSP application for SuiteScript 2.0

1. This procedure applies only to 2019.1 implementations of SCA and later.
The NetSuite SSPv2 application record defines the folder in the NetSuite file cabinet where website customization assets that support SuiteScript 2.0 are stored. If you are implementing the 2019.1 release of SCA or later, you must create this additional SSP application for SuiteScript 2.0 assets.
In NetSuite, go to Commerce > Hosting > SSP Applications > New.
2. To create the record for your SuiteScript 2.0 assets, follow the steps outlined in the NetSuite Help topic [Create a SuiteScript 2.0 SSP Application Record](#) to provide the following information:
 - Be sure that the name you assign to the SSP application is the same as the name used for the SuiteScript 1.0 application, but append v2 to the end. For example: MyCustomSSPv2
 - Specify a version of 2.x.
 - Create a new application folder and provide the same HTML Hosting Root and Application Publisher that you used in [Create a custom SSP application for SuiteScript 1.0](#). Also use the same Application Name, but add the number 2 to the end. For example: ProDevSite2

- Be sure the URL root is the same as the root used for the SuiteScript 1.0 application, but append `_ss2` to the end. For example, if the URL root for your SuiteScript 1.0 application is `ProDev`, the root for your SuiteScript 2.0 application should be `ProDev_ss2`.

Creating an SSP application with a version of 2.x results in a directory structure in the NetSuite File Cabinet with the following path: **<HTML Hosting Root>: /SSP Applications/<Application Publisher>/<Application Name>**. For example, if you specify the following information for the application folder when creating an SSP application with a version of 2.x:

- **HTML Hosting Root:** Live Hosting Files
- **Application Publisher:** ProDev
- **Application Name:** ProDevSite2

The resulting directory structure in the NetSuite File Cabinet is: Web Site Hosting Files/Live Hosting Files/SSP Applications/ProDev/ProDevSite2

Step 2: Deploy Local Files

After creating your SSP application, use the core SCA developer tools to deploy local files to your NetSuite file cabinet. You need the following information to complete these steps:

- An authentication ID to connect with NetSuite.
- Details from the SSP application record created in [Create a custom SSP application for SuiteScript 1.0](#): HTML Hosting Root, Application Publisher, Application Name

To deploy local files using Gulp:

1. Return to your command line or terminal window.
2. From the SCA source directory (the directory that contains the package.json file), enter the following command.

```
gulp deploy
```

If this is the first time you are running `gulp deploy` in this directory, this command creates one or more deploy distribution directories depending on your implementation.

Note: If you have already created an authentication ID in a previous fetch or deploy operation, the developer tools do not prompt you for an ID and will continue to use the existing ID for each subsequent fetch or deploy. If you want to choose a different authentication ID or create a new one, use the `--to` parameter when you deploy. For more information, see [Token Based Authentication](#).

3. In NetSuite, select the account you want to deploy to and the role you want to use for authentication. Click the **Allow** button to generate the authentication token and secret.

Note: You must specify a role that provides the permissions needed to deploy to NetSuite. By default, the Administrator and SCDeployer roles provide these permissions. To learn more, including how to create a custom role that provides the required permissions, see [Developer Tool Roles and Permissions](#).

4. When prompted, provide the following details from your SSP application record:

✓ **Tip:** These values correspond to the path to your SSP application in the NetSuite File Cabinet: <HTML Hosting Root>: /SSP Applications/<Application Publisher>/<Application Name>

- Hosting Files folder: This is the HTML Hosting Root of your SSP application.
- Application Publisher: This is the Publisher of your SSP application.
- SSP Application: This is the Name of your SSP application.

After all of the connection settings are entered, files from your local system are pushed to the NetSuite file cabinet. This process may take a few minutes. Wait for the process to complete before proceeding.

Note: The first time you run the `gulp deploy` command, the connection settings are stored in the `.nsdeploy` file in the root directory of your source SCA files. During subsequent deployments, login credentials are not required. If you need to change the SSP application you are deploying to, you can manually edit the `.nsdeploy` file with the updated information. For details, see [Changing SCA Developer Tools Connection Information](#). Also, during the initial deployment to NetSuite, the Gulp.js commands create a manifest file within the NetSuite file cabinet. This file contains the list of files uploaded and a hash of its content. On subsequent deployments, the Gulp.js commands use the manifest file to determine new or changed files. Only those files are updated during deployment to the NetSuite file cabinet.

Step 3: Configure SSP Applications

After completing the preceding steps, all necessary files should be available to the SSP applications within NetSuite. In this final step, complete some configuration tasks to enable your website to use the customized SCA application you uploaded.

1. Go to Commerce > Hosting > SSP Applications and then click **Edit** next to the SSP application created in [Create a custom SSP application for SuiteScript 1.0](#).
2. In the Libraries subtab under Scripts, click **Add**.
3. Find and add the `ssp_libraries.js` and `ssp_libraries_ext.js` files to the libraries for the SSP application.

These files were pushed to the NetSuite file cabinet with the `gulp deploy` command. When you add these files to the SSP library, they are stored in the root directory of your SSP application at <HTML Hosting Root>:/SSP Applications/<Application Publisher>/<Application Name>.

Note: Depending on the number of files in your file cabinet you may need to scroll through a long list. However, do not rely on the search. The files available may not be refreshed, resulting in a no matches found error even though the files do exist.

4. In the Touch Points tab, define the touch points for this SSP application.
Touch points are the entry points for your web store. For each touch point defined here, your website links to an SSP application page.
 - a. In the Name field, select the desired touch point.
 - b. In the Entry Page field, select the `.ssp` file that should be the starting point for this touch point.
 - c. Click **Add**.

For example, if you want your web store to use the Shopping application when a user goes to your Home page, select the **View Homepage** Touch Point and set the Entry page as the `shopping.ssp` file.

5. Click **Save** on the record.

You are returned to the list of available SSP applications.

6. Link the SSP application created in [Create a custom SSP application for SuiteScript 1.0](#) to your domain.
 - a. Click **View** next to your SSPv1 application.
 - b. Click **Link to Domain**.
 - c. In the Domain dropdown field, select the domain you want to link this application to.
 - d. Click **Save**.
7. If you are implementing SCA 2019.1 or later and are using a single domain for shopping and checkout, follow these steps to deploy the SSP application created in [Create a custom SSP application for SuiteScript 2.0](#):
 - a. Click **View** next to your SSPv2 application.
 - b. Click **Deploy to Domain**.
 - c. In the Domain dropdown field, select the domain for your site.
 - d. Click **Save**.
8. If you are implementing SCA 2019.1 or later and are *not* using a single domain for shopping and checkout, follow these steps to deploy the SSP application created in [Create a custom SSP application for SuiteScript 2.0](#):
 - a. Click **View** next to your SSPv2 application.
 - b. Click **Deploy to Site**.
 - c. In the **Deploy to All Shopping Domains on Site** dropdown field, select the site associated with your shopping domains.
 - d. In the **Deploy to Checkout Domain on Site** field, select the site associated with your checkout domain.



Note: If you want to deploy the SSP application to the checkout domain of one site and the shopping domains of another, you can select different site names from the lists. Alternatively, if you want to deploy to just the checkout domain or just shopping domains, select - Do Not Deploy - for the domain type to which you do not want to deploy.

- e. Click **Save**.

After completing the steps in this procedure, you can view the site by navigating to the domain defined for that site.

Bundle Themes and Extensions as SuiteApps

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

Commerce websites let you bundle themes and extensions as SuiteApps, making them available for distribution to other NetSuite accounts. Although some steps to bundle a SuiteApp are already documented in the help topics, you need to be aware of some settings and processes specific to bundling Commerce themes and extensions.

Before You Begin

Before you bundle a Commerce theme or extension, be aware of the following:

- These procedures assume that you have created or updated a theme or extension, thoroughly tested it on a local server or on a test domain, and that you have deployed the files to a NetSuite account. Refer to the following help topics for more information about building themes and extensions:
 - [Develop Themes](#) – explains how to create themes.
 - [Develop Extensions](#) – explains how to create extensions.
- You must deploy themes and extensions to the NetSuite Account where you are creating the SuiteApp.
- The SuiteCommerce Extensions Management SuiteApp provides a default installation script. Use this baseline file to create unique installation scripts for each SuiteApp you build.
- You must designate a version when creating a theme or extension. This allows SuiteApps to be updated over time. The developer tools prompt you for this information when you deploy your files. Theme and extension version numbers should follow this format, where d is a single digit: dd.dd.dd

Bundle Process for SuiteCommerce

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

To bundle Commerce themes and extensions, follow these steps:

- [Step 1: Prepare Themes and Extensions for Bundling](#)
- [Step 2: Create a Unique Installation Script for Your Theme or Extension SuiteApp](#)
- [Step 3: Create the SuiteApp for Your Themes or Extensions](#)

Step 1: Prepare Themes and Extensions for Bundling

The first step is to prepare any folders containing your themes or extensions to enable bundling.

To prepare your theme or extension for bundling:

1. In your NetSuite File Cabinet, navigate to the folder containing your theme or extension. This folder is the one containing your theme and extension files, including the manifest.json file. For example:
 SuiteScript/Deploy_Extensions/<VendorName>/<ThemeOrExtensionName>@<Version>/

2. Click **edit** next to the extension folder you are including in the bundle.
3. In the Document Folder record, check the **Available for SuiteBundles** option.
4. Click **Save**.
5. Repeat this for every theme and extension you want to include in a SuiteApp.

You are now ready to create an installation script.

Step 2: Create a Unique Installation Script for Your Theme or Extension SuiteApp

Before bundling your themes or extensions as a SuiteApp, you must complete the following steps:

1. Create an installation script file.
2. Create an Installation Script record.
3. Deploy your installation script.

A bundle installation script is essential to allow SuiteApp installation, update, and uninstallation processes to run properly. The SuiteCommerce Extension Management SuiteApp comes with a default bundle installation script that you can use, titled: **ExtensionBundle.js**. As a best practice, copy the default file to create a unique script for each SuiteApp you want to build.



Important: Never edit the default ExtensionBundle.js file directly. As a best practice, create a unique installation script file for each theme or extension SuiteApp you want to build, even if you do not include any customizations. Store your installation scripts in a location associated with your theme and extension files.

This default script includes the necessary functions to install, update, or uninstall the SuiteApp and its contents. This includes the following trigger functions:

- beforeInstall
- afterInstall
- afterUpdate
- beforeUninstall



Note: You only need one installation script per SuiteApp. The script applies to any included themes and extensions.

To create an installation script file:

1. In the NetSuite File Cabinet, browse to **SuiteBundles/Bundle xxxxxx**, where xxxxxx equals the Bundle ID of the SuiteCommerce Extension Management SuiteApp.
See the help topic [Install Your Commerce Website Applications](#) for the latest Bundle ID of the SuiteCommerce Extension Management SuiteApp.
2. Make a copy of the default ExtensionBundle.js file:
 - a. Click **Copy Files**.
 - b. Expand the **Filters** area.
 - c. In the **Copy To** list, select a location in the File Cabinet to store your copy.
 - d. Check the box next to ExtensionBundle.js.

- e. Click **Copy**.
- f. Rename your new file as needed. (Optional)

To rename a file, navigate to its location and edit the file record.

3. Edit your new installation script file to meet your needs (optional).

The default script can handle most requirements. However, if you have custom needs or want to make special validations or operations in NetSuite records, you can edit the script to meet your needs. For specific information and instructions regarding installation script files, see the help topic [SuiteScript 2.x Bundle Installation Script Type](#).



Important: If you need to edit ExtensionBundle.js, maintain the validations and operations that came with default file.



Note: The ExtensionBundle.js file that comes with the 18.2 release of the SuiteCommerce Extension Management SuiteApp support updates. However, if you previously created an extension as a SuiteApp prior to the 18.2 release, we recommend recreating the SuiteApp with the latest update to take advantage of the version management functionality. This includes creating a new installation script with SuiteScript 2.0.

To create an Installation Script record:

1. Go to Customization > Scripting > Scripts > New.
2. In the Script File field, click **List**.
3. In the **-All-** list, scroll to select the location of your installation script (this is your copy of ExtensionBundle.js).
4. Select your installation script from the list.
5. Click **Create Script Record**.
6. From the Select Script Type list, select **Bundle Installation**.
7. In the new Script record, name your bundle and provide a description (optional).
8. In the **Scripts** tab, confirm that the following script fields populated correctly.
 - **Before Install Function:** beforeInstall
 - **After Install Function:** afterInstall
 - **After Update Function:** afterUpdate
 - **Before Uninstall Function:** beforeUninstall
9. Click **Save**.

To deploy your installation script:

1. Create a Script Deployment. See the help topic [Methods of Deploying a Script](#).
2. Name your script and provide an ID, beginning with underscore (_).
3. Set the **Status** field to **Released**.



Important: Failure to set this field to **Released** results in a failed installation.

4. Choose the **Log Level** to meet your needs.
 5. When you have finished, click **Save**.
- You are now ready to create your SuiteApp.

Step 3: Create the SuiteApp for Your Themes or Extensions

After you have successfully created and deployed your installation script, you are ready to build your SuiteApp. Follow the instructions detailed in [Creating a Bundle with the Bundle Builder](#).

Observe the following information when building a SuiteApp for Commerce themes and extensions:

- When choosing the installation script, select the script you created earlier.
- You cannot use the same Installation Script record across multiple SuiteApps when bundling Commerce themes or extensions. You must select a unique Installation Script record for each SuiteApp.
- When prompted to select objects for the SuiteApp, browse to **File Cabinet/Folders** and choose the objects associated with each theme or extension you are including in your SuiteApp. Look for the objects that match your deployed themes and extensions.
- You can create theme and extension SuiteApps as managed or unmanaged bundles.
- You can bundle any number of themes and extensions within one SuiteApp.

Declare Target Versions

i Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

When you run the theme or extension developer tools, you specify the Commerce product for which your theme or extension applies. However, you must also specify which releases of those products are compatible with your theme or extension. You do this by declaring a **target version**.

Target versions are required when creating themes or extensions for the following Commerce products:

- Commerce Web Stores: Includes SuiteCommerce (SC), SuiteCommerce MyAccount (SCMA), and SuiteCommerce Advanced (SCA). The theme and extension developer tools both refer to this combination as **SuiteCommerce Online**.
- SuiteCommerce InStore (SCIS) (Extensions only)

Target versions prevent site managers from activating or updating incompatible themes and extensions on their site because the Extension Manager:

- Only lists themes and extensions that are compatible with the declared target version of the associated Commerce product.
- Excludes incompatible updates to active themes and extensions.

To declare a target version:

1. Access the theme or extension developer tools, depending on what you are creating. For details, see [Develop Themes](#) and [Develop Extensions](#).
2. Follow the on-screen prompts.
The developer tools prompt you for a target version when using the following commands:
 - gulp theme:deploy
 - gulp extension:create
 - gulp extension:deploy
3. When prompted to enter a target version, enter a string to declare compatible versions. This string requires a three-part version number scheme, as described in [Patches and Upgrades](#). The

developer tools prompt you for a target version for each product specified earlier. Leaving these prompts blank results in compatibility with all versions of the associated Commerce products.

You can set the range of compatible versions using standard SEMVER range specifiers, such as the tilde (~), greater-than and equal (>=), and caret (^).

Note: For more information on SEMVER schema, see <https://semver.org/>.

4. Leave this prompt empty to specify that the theme or extension is available for all versions of the associated Commerce product. This applies for legacy and new themes and extensions, where the target version was not provided.

The following table highlights a few useful examples.

Prefix	String Example	Description	Compatibility Examples
None	None	Leaving the target version prompt blank results in compatibility with all versions of the associated product.	All releases
None	19.1.0	Listing the version with no symbol prefix results in compatibility with the specified release only.	2019.1.0
>=	>=19.1.0	Greater Than and Equal This prefix results in compatibility with the specified release and all future releases.	2019.1.0 2019.1.1 2019.2.0 2020.1.0
^	^19.1.0	Caret This prefix results in compatibility with the specified SC/SCA/SCIS release and all future releases within a given year, both major and minor.	2019.1.0 2019.1.1 2019.2.0
~	~19.1.0	Tilde This prefix results in compatibility with the specified major release including all future minor releases.	2019.1.0 2019.1.1

Update Themes and Extensions

Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced

This topic explains how to update Commerce themes and extensions that are part of a SuiteApp.

Before You Begin

Before releasing an updated Commerce theme or extension, be aware of the following best practices:

- Business users can choose whether or not to activate an updated theme or extension. Therefore, always develop your theme and extension updates to be backwards compatible with previous versions.
For example: NetSuite records are maintained within a NetSuite account. If your extension update requires altering a custom record, you could break backwards compatibility with previous versions of the extension.

- Always test theme and extension functionality and backwards compatibility on a development or sandbox account before releasing the bundle.
- If you bundled a theme or extension prior to the 18.2 release of the SuiteCommerce Extensions Management SuiteApp, we recommend rebuilding the installation script using SuiteScript 2.0 the next time you update a theme or extension included in the bundle.
- You must increment the version of at least one theme or extension when updating a SuiteApp. If the SuiteApp does not include a theme or extension with a later version, the update fails.
- The SuiteApp version has no programmatic connection with the versioning system of themes and extensions.
- If your theme or extension update requires customizing the installation script, you can edit the existing script. You do not need to build a new script.
- These procedures only apply to themes and extensions that are being released as part of a bundled SuiteApp. Updates to custom themes or extensions (as developed by internal site developers) are made using the developer tools.

Update Themes and Extensions Within a SuiteApp

To update a theme or extension that is part of a SuiteApp:

1. Update your theme or extension as required. See [Develop Themes](#) and [Develop Extensions](#) for more information.
2. Deploy your updated theme or extension to the NetSuite account used to create the original SuiteApp.

Increment each version using a Major.Minor.Patch numbering scheme. Theme and extension version numbers should follow this format, where d is a single digit: dd.dd.dd. Failure to use this format results in failed SuiteApp updates.

3. Confirm that the manifest.json file associated with the updated theme or extension reflects the correct version.
 - a. In NetSuite, navigate to the location where you deployed your theme or extension update. For example:
SuiteScript/Deploy_Extensions/<VendorName>/<ThemeOrExtensionName>@<Version>/
 - b. Open the associated manifest.json file and confirm the version.

The manifest.json file should reflect the latest version in SemVer notation. For example, if you deployed an update to MyCoolExtension 1.0.0 to version 2.0.0, your manifest.json file should look similar to the following example:

```

1 {
2   "fantasyName": "My Cool Extension!",
3   "name": "MyCoolExtension",
4   "vendor": "Acme",
5   "type": "extension",
6   "target": "SuiteCommerce",
7   "version": "2.0.0",
8   "description": "This is my cool update!",
9   //...

```

- c. If the manifest does not reflect the correct version, redeploy or edit the manifest file in your file cabinet.
4. If your theme or extension update requires changes to the installation script, edit the installation script as needed.

See the help topic [SuiteScript 2.x Bundle Installation Script Type](#) for details.

5. Edit the SuiteApp using SuiteBundler in NetSuite. See the help topic [Creating a Bundle with the Bundle Builder](#) for details.

Overview

i Applies to: SuiteCommerce | SuiteCommerce MyAccount | SuiteCommerce Advanced | SuiteCommerce InStore

The following topics provide reference information for using the Commerce developer tools. Some of these topics provide important details for testing locally or deploying to NetSuite.

- [Token Based Authentication](#)
- [Mixed Domains in a Local Server](#)
- [Secure HTTP \(HTTPS\) with the Local Server](#)
- [Troubleshooting the Developer Tools](#)

Token Based Authentication

 **Applies to:** SuiteCommerce | SuiteCommerce Advanced

When fetching or deploying using the theme and extension developer tools, users must authenticate with NetSuite. In previous releases, developer tool users were able to authenticate by providing their NetSuite user name and password. Now two-factor authentication (2FA) is mandatory on all NetSuite accounts using NetSuite 2018.2 and later. To support NetSuite 2FA, the developer command-line tools require token-based authentication for fetch and deploy operations. You can generate and manage tokens as described in the following topics:

- [Generate Tokens for Authentication](#)
- [Manage Authentication Tokens](#)

Generate Tokens for Authentication

Before you can generate a token for authentication, you must:

- Have the Token Based Authentication feature enabled in the NetSuite account if you are working on a production account. For more information, see the help topic [Enable the Token-based Authentication Feature](#).
- Have a NetSuite role that provides the permissions needed to fetch from and deploy to NetSuite. The Administrator role provide these permissions by default. For more information, see [Developer Tool Roles and Permissions](#).

 **Note:** For release 2021.2.1 and later accounts that have account-specific domains, you must specify your account number with the gulp command to fetch or deploy. Keep this in mind when reviewing the following procedures. For more information about using the --account parameter with fetch and deploy commands, see [Gulp Command Reference for Theme and Extension Developer Tools](#).

To generate an initial token for authentication:

1. The first time you use a gulp command to fetch or deploy, the Commerce developer tools prompt for an authentication ID. You can enter any alphanumeric string for the authentication ID, which represents your NetSuite account and role.
2. If you are already logged into NetSuite, you are prompted to select the NetSuite account and role that you want to use for the current fetch/deploy operation. If you are not logged into NetSuite, you are prompted to do so.
3. In NetSuite, select the **Allow** button to generate the token and associated secret that is required for access.
4. In NetSuite, when you see a message that the authentication process will continue in your CLI application, you can close the NetSuite window and the fetch/deploy operation continues and completes in the Commerce developer tool.

 **Note:** After generating an initial authentication token, each subsequent fetch or deploy operation uses this token unless you specify the --to parameter as described in the following procedure.

To generate a new token for authentication:

1. You can generate a new token for authentication by specifying the `--to` parameter when you fetch or deploy. For example: `gulp extension:deploy --to`
2. The developer tools give you the option to choose a saved token or to generate a new token.
 - The saved tokens you have previously used to authenticate are listed by authentication ID and account.
 - To generate a new token, choose the **New token** option.
3. The Commerce developer tools prompt for an authentication ID. You can enter any alphanumeric string for the authentication ID, which represents your NetSuite account and role.
4. If you are already logged into NetSuite, you are prompted to select the NetSuite account and role that you want to use for the current fetch/deploy operation. If you are not logged into NetSuite, you are prompted to do so.
5. In NetSuite, select the **Allow** button to generate the token and associated secret that is required for access.
6. In NetSuite, when you see a message that the authentication process will continue in your CLI application, you can close the NetSuite window and the fetch/deploy operation continues and completes in the Commerce developer tool.

Manage Authentication Tokens

If you need to view, edit, or revoke previously generated tokens, you can do so in NetSuite. To learn more, see the help topic [Manage TBA Tokens in the NetSuite UI](#).

Tokens that you use to connect to NetSuite from the developer tools are saved in a local file called `.nstba`. On Windows, the `.nstba` file is stored in the `C:\Users\user_name` directory. You can edit the `.nstba` file to manage the saved tokens that are listed in the developer tools when you specify the `--to` parameter for a fetch or deploy operation.

Mixed Domains in a Local Server

 **Applies to:** SuiteCommerce | SuiteCommerce Advanced

When redirecting between domains while testing on your local server, the default behavior is for the application to load the **production** version of the initial page in the new domain.

For example, suppose you start in the **local** version of the Shopping domain using the shopping-local.ssp URL. On login, you are redirected to the **production** version of the Checkout domain at the my-account.ssp URL. To return to the local version of that domain you must manually edit the URL.

Change: `https://checkout.netsuite.com/c.xxxxxx/sca-dev-denali/my_account.ssp?n=3`

To: `https://checkout.netsuite.com/c.xxxxxx/sca-dev-denali/my_account-local.ssp?n=3`

Also, in some browsers, when you manually update this URL you may encounter blank page loads with console errors similar to the following:

```
1 | Mixed Content: The page at 'https://checkout.netsuite.com/c.xxxxxx/sca-dev-denali/my_account-local.ssp?n=3' was loaded over HTTPS, but requested an insecure stylesheet 'http://localhost:7777/css/myaccount.css'. This request has been blocked; the content must be served over HTTPS.
```

For example, Chrome returns this error to protect you from a site that is not secure. If this happens in Chrome, in the address bar you will see a shield. Click on the shield and select **Load unsafe scripts**. The page then loads normally.

 **Note:** You can also launch the Chrome browser with the flag `--allow-running-insecure-content` or you can install the certificates in your local web server.

Secure HTTP (HTTPS) with the Local Server

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

The `gulp local` command starts two instances of the local server at the following URLs:

- `http://localhost:7777`
- `https://localhost:7778`

The instance running at port 7778 provides a secure domain using HTTPS. This enables you to test the application using secure domains. However, before using secure domains on the local server, you must perform the following:

Task	Follow These Instructions
Generate the required SSL certificates and private keys.	Generate SSL Certificates and Private Keys
Configure the KEYPEM and CERTPEM environment variables.	Configure the KEYPEM and CERTPEM Environment Variables
Install the certificates on your system.	Install the Generated Certificates
Modify the <code>distro.json</code> file.	Modify the <code>distro.json</code> File
Modify the root URL in the application's <code>local.ssp</code> file: <code>shopping-local.ssp</code> , <code>my_account-local.ssp</code> , or <code>checkout-local.ssp</code>	Modify the Root URL of the SSP Application
Access the local server using a secure URL.	Access the Local Server Using a Secure URL

Generate SSL Certificates and Private Keys

To access a secure domain via HTTPS when running the local server, you must use an SSL certificate and a private key. Since the local server is intended for testing and not a production environment, you can create a self-signed certificate locally and do not need to use a third-party certificate provider.

To generate an SSL Certificate and a Private Key

1. Download and Install OpenSSL.
2. Generate an RSA private key.
 - a. Run the following command:


```
openssl genrsa -des3 -out ca.key 1024
```
 - b. Enter and confirm a password for the certificate.

You will use this password in the remaining procedures for creating a certificate and private key.

This command outputs the RSA private key in a file called `ca.key`.
3. Create a new SSL certificate.
 - a. Run the following command:


```
openssl req -new -sha256 -key ca.key -out ca.csr
```

This command uses the RSA private key created in the previous step.

- b. Accept the default value for the localhost field. The other fields are not required to create the certificate used by the local server when running HTTPS.

This command outputs the SSL certificate in a file called `ca.csr`.

4. Create a self-signed certificate:

```
openssl x509 -req -days 3600 -in ca.csr -out ca.crt -signkey ca.key
```

If you are prompted to enter a password, use the password you entered when generating the RSA key.

5. Create a server key:

```
openssl genrsa -des3 -out server.key 1024
```

This command outputs the server private key to a file called `server.key`.

6. Create a certificate signing request (CSR):

```
openssl req -new -sha256 -key server.key -out server.csr
```

This command outputs the CSR to a file called `server.csr`.

7. Remove the password from the server certificate.

This step is optional. If you encounter problems with the password, you can remove it from the certificate.

- a. Copy the `server.key` file to `server.key.org`.
- b. Run the following command to generate a new `server.key` file that has no password:

```
openssl rsa -in server.key.org -out server.key
```

This command creates a new private key called `server.key`. The local server uses this file when creating a secure domain. Therefore, you should move it to a permanent location.

8. Create a self-signed server certificate:

```
openssl x509 -req -sha256 -days 3600 -in server.csr -signkey server.key -out server.crt
```

This command creates a new server certificate called `server.crt`. The local server uses this file when creating a secure domain. Therefore, you should move it to a permanent location.

Configure the KEYPEM and CERTPEM Environment Variables

After generating a server certificate and private key, you must define environment variables that point to these files.

Using the method for setting environment variables for your operating system, create the following:

Note: You must set these environment variables before running the local server.

KEYPEM	<path_to_file>/server.key
CERTPEM	<path_to_file>/server.crt

On Windows, for example, you can set these environment variables as in the following example:

```
1 | set KEYPEM=c:\OpenSSL-Win64\server.key
```

```
2 | set CERTPEM=c:\OpenSSL-Win64\server.crt
```

Install the Generated Certificates

After generating the SSL and server certificates, you must enable them to work with your web browser. On Windows, you can use the Certificate Import Wizard.

To install generated certificates:

1. Run the `server.crt` file you generated using OpenSSL.
2. Click **Install Certificate**.
3. Click **Next**.
4. Choose **Place all certificates in the following store**, then click **Browse**.
5. Choose **Trusted Root Certification Authorities**, then click **OK**.
6. Click **Next**.
7. Verify that your settings are correct, then click **Finish**.
8. Click **Yes** to verify that you want to install the certificate on your system.

After installing the server certificate, you should repeat these procedures to install the `ca.crt` file generated in a previous step.

Modify the `distro.json` File

To access a secure domain on the local sever, you must ensure that the `https` object exists in the `local` object of `taskConfig`. After adding the `https` object, add an entry for the HTTPS port, certificate and key.

Your `distro.json` file should look similar to the following:

```
1 | "tasksConfig": {
2 |   "local": {
3 |     "http": {
4 |       "port": 7777
5 |     },
6 |     "lessSourceMap": false,
7 |     "jsRequire": true,
8 |     "https": {
9 |       "port": 7778,
10 |      "key": "KEYPEM",
11 |      "cert": "CERTPEM"
12 |     }
13 |   },
14 |   ...
```



Note: You must set the key and cert properties as shown above. The local server uses these values to determine the environment variables used to local the certificate and key required to use HTTPS.

Modify the Root URL of the SSP Application

To use HTTPS with the local server, you must change the value of the `ROOT` variable in the SSP application.

To modify the root URL of the SSP application:

1. Open the application's `index-local.ssp` file.

This file is located in `<SCA_Source_Root>`. For example, the `index-local.ssp` file for the Shopping application resides in this directory: `/Modules/suitecommerce/ShoppingApplication@x.y.z/Internal`



Note: For SCA 2019.2 and later releases, `index-local.ssp` files reside in the SCA Modules stored in the Advanced directory. For example: `/Advanced/ShoppingApplication/Internals`

2. Change the value of the `ROOT` variable:

```
1 | var ROOT = 'https://localhost:7778/'
```

3. Compile and deploy the application using the following command:

```
gulp deploy
```

Because this procedure changes a backend file, you must deploy the files to NetSuite. In the process of compiling the application, this command creates the `local.ssp` file based on the `index-local.ssp` file that you modified in the preceding steps.

Access the Local Server Using a Secure URL

To access the local server using the local server, you must use the URL of your secure domain.

To access the local server using a secure domain:

1. Run the following command:

```
gulp local
```

2. Access the secure domain of the local server using a URL of the following form:

```
https://mysite.com/c.<account_id>/<SSP_application>/shopping-local.ssp
```

For example, your URL should look similar to the following to access the Shopping application:

```
https://mysite.com/c.123456/sca-dev-montblanc/shopping-local.ssp
```

Troubleshooting the Developer Tools

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

The following are known problems or errors that you may encounter when installing or running the developer tools. You can go to the section that describes the error to learn more:

- [startDeploy Errored: Cannot Read Property 'files' of Undefined at deployData.forEach.deploy](#)
- [Invalid Assignment Left-Hand Side](#)
- [TypeError: out.writeUInt32LE is not a function](#)
- [Error INVALID_LOGIN_ATTEMPT Invalid login attempt](#)
- [Error INSUFFICIENT_PERMISSION You do not have privileges to view this page](#)
- [Sass-Related Error](#)
- [Gulp.js Version Mismatch](#)
- [Warning Messages Related to GIT](#)
- [ENOSPC Error When Running Local Server](#)
- [EMFILE Error When Running a Local Server](#)
- [Sass Compilation Errors when Running Gulp](#)
- [Source Code Error When Running Gulp](#)

startDeploy Errored: Cannot Read Property 'files' of Undefined at deployData.forEach.deploy

The `gulp theme:deploy` and `gulp extension:deploy` commands use SuiteScript Restlet service requests to upload your theme and extension development files to the NetSuite file cabinet. And, SuiteScript's governance model limits the number of units each Restlet request can use. For more information, see the help topic [SuiteScript Governance and Limits](#).

The `chunk_size` property in the `<Top-LevelDevelopmentDirectory>/gulp/config/config.json` file controls how many files are uploaded per Restlet request. Each Restlet request has a fixed number of units that it can consume per the SuiteScript governance model. The number of units required to upload each file to the NetSuite file cabinet varies based on the number of nested folders within your theme or extension development files. More complex directory structures consume more units.

If you receive this error for a theme or extension that has a complex directory structure, reducing the value of the `chunk_size` property from the default setting of 80 may resolve the issue.

Note: You can also adjust the `chunk_size` property if you need to speed up deployment. Increasing the number of files uploaded per request should speed up the process because the deployment will require fewer requests to the Restlet.

Error Messages When Running the Developer Tools

You may encounter the following errors when you deploy themes, extensions, and other customizations to NetSuite.

Invalid Assignment Left-Hand Side

This error may occur with Vinson implementations of SuiteCommerce Advanced (SCA) when deploying or fetching using the SCA developer tools. This error occurs when the required version of Node.js for your implementation is not compatible with the xmlbuilder version that is installed. Complete the following steps to correct this issue.

1. Within the developer environment path, go to:
./node_modules/xml2js
2. Run this command: `npm install xmlbuilder@11.0.0`
3. To ensure xmlbuilder@11.0.0 has overwritten the previous version, run: `npm list xmlbuilder`
4. If the preceding command returns any version other than xmlbuilder@11.0.0, you need to install version 11.0.0 in the directory where the version is not correct.

For example, if the preceding command returns version information that shows a version other than xmlbuilder@11.0.0 within one of the folders in the node_modules directory, go to that folder and run this command: `npm install xmlbuilder@11.0.0`
5. To ensure xmlbuilder@11.0.0 has overwritten the previous version, run: `npm list xmlbuilder`

TypeError: out.writeUInt32LE is not a function

This error may occur with Vinson implementations of SuiteCommerce Advanced (SCA) when deploying or fetching using the SCA developer tools. This error occurs when the required version of Node.js for your implementation is not compatible with installed versions of the font converter tools, ttf2eot and ttf2woff. Complete the following steps to correct this issue.

From the root directory of SCA:

1. Go to: `node_modules/gulp-ttf2woff`
2. Run this command: `npm install ttf2woff@2.0.2`
3. Then go to: `node_modules/gulp-ttf2eot`
4. Run this command: `npm install ttf2eot@2.0.0`

Error INVALID_LOGIN_ATTEMPT Invalid login attempt

This error occurs when deploying or fetching using a role that requires 2FA. You must specify a role that provides the permissions needed to deploy to NetSuite. By default, the Administrator and SCDeployer roles provide these permissions. To learn more, including how to create a custom role that provides the required permissions, see [Developer Tool Roles and Permissions](#).

Error INSUFFICIENT_PERMISSION You do not have privileges to view this page

This error occurs when you add a role that has appropriate fetch/deploy permissions to an Employee Record, but the **Audience** in the Restlet Script Deployment is not added correctly for the role.

See [Create a Custom Role with Fetch and Deploy Permissions](#) for more information on setting this up correctly.

Sass-Related Error

When running gulp with an unsupported version of Node.js you may encounter a Sass-related error similar to the following:

```

1 gyp ERR! build error
2 gyp ERR! stack Error: `make` failed with exit code: 2
3 gyp ERR! stack   at ChildProcess.onExit
4 (/home/sg/netsuite/ml/node_modules/gulp-sass/node_modules/node-sass/node_modules/pangyp/lib/build.js:272:23)
5 gyp ERR! stack   at emitTwo (events.js:87:13)
6 gyp ERR! stack   at ChildProcess.emit (events.js:172:7)
7 gyp ERR! stack   at Process.ChildProcess._handle.onexit
8 (internal/child_process.js:200:12)
9 gyp ERR! System Linux 3.13.0-37-generic
10 gyp ERR! command "/usr/local/bin/node"
11 "/home/sg/netsuite/ml/node_modules/gulp-sass/node_modules/node-sass/node_modules/pangyp/bin/node-gyp"
12 "rebuild"
13 gyp ERR! cwd
14 /home/sg/netsuite/ml/node_modules/gulp-sass/node_modules/node-sass
15 gyp ERR! node -v v4.0.0
16 gyp ERR! pangyp -v v2.3.2
17 gyp ERR! not ok
18 Build failed

```

Resolution: Ensure that you are using Node.js version 0.12.x or older.

Gulp.js Version Mismatch

When running gulp, you may see a warning message related to a version mismatch. This is due to a mismatch between the version installed globally on your system and the version used by the developer tools.

Resolution: This warning is expected and does not cause problems with the developer tools.

Warning Messages Related to GIT

This error is caused by the Sass compiler which expects a GIT server to be installed.

Resolution: This warning is expected and does not cause problems with the developer tools.

ENOSPC Error When Running Local Server

On UNIX systems, the `gulp watch` command (used when running a local server) may return this exception. This is caused when the gulp process exceeds the limit of files that can be watched by the system.

Resolution: Enter the following command to resolve this issue:

```
echo fs.inotify.max_user_watches=524288 | sudo tee -a /etc/sysctl.conf && sudo sysctl -p
```

EMFILE Error When Running a Local Server

On UNIX systems, the `gulp local` command may return this exception if the gulp process exceeds the limit of files that can simultaneously be opened.

Resolution: Enter the following command to increase the system limit on the number of files that can simultaneously be opened.

```
ulimit -n 2048
```

Sass Compilation Errors when Running Gulp

This information refers to the Mont Blanc release of SuiteCommerce Advanced and later.

Changes to the developer tools source code may be required to mitigate errors in Sass compilation when deploying.

SCA uses gulp-sass for compiling .scss files. Node-sass (v3.5), which is a dependency of gulp-sass, has caused some Sass code that previously compiled fine to throw fatal errors, particularly in Bootstrap. These errors are returned when running the `gulp local` command to start your local server or when deploying with `gulp deploy`.

The errors returned begin as follows:

```
1 | formatted Error: You may not @extend an outer selector from within @media you may only @extend selectors within the same directive.
   | . [etc]
```

To correct the compilation errors:

1. Open the command line in the project <Source Code> directory.
2. Execute the following commands to remove the 3.5 dependency and install the expected version.

```
npm cache clear
npm uninstall gulp-sass node-sass
npm config set save-exact true
npm install --save node-sass@3.4.1 gulp-sass@2.1.0
```

 **Note:** This will modify your package.json. Preserve these changes.

3. Run `gulp local` to validate that the error is resolved.

If you have also installed gulp-sass globally, you might need to uninstall the globals.

To uninstall gulp-sass globals:

1. Open the command line in the top-level directory of your Mont Blanc source code.
2. If you are a **Windows** user, execute the following command as either Administrator or as a User:

```
npm uninstall -g node-sass gulp-sass
```

3. If you are a **Linux / Mac** user, execute the following command as Administrator:

```
npm uninstall -g node-sass gulp-sass sudo
npm uninstall -g node-sass gulp-sass
```

Source Code Error When Running Gulp

This information refers to the Mont Blanc release of SuiteCommerce Advanced or later.

When deploying source files, you might encounter the following error:

```
1 | SOURCE CODE ERROR. Error: You may not @extend an outer selector from within @media. You may only @extend selectors within the same directive.
```

If this occurs, a mismatch of node-sass versions exists between the package.json files located in the top-level directory and the node_modules/gulp-sass directory. To fix this error, perform the following.

To correct the Source Code Error:

1. Open the command line in the top-level directory of your source code.
2. Execute the following commands:

Windows Users

```
1 | npm install -g npm
2 | npm cache clear
3 | npm uninstall gulp-sass
4 | npm uninstall node-sass
5 | npm config set save-exact true
6 | npm install node-sass@3.4.1 --save
7 | npm install --save gulp-sass@2.1.0
```

Linux/Mac Users

```
1 | sudo npm install -g npm
2 | npm cache clear
3 | npm uninstall gulp-sass
4 | npm uninstall node-sass
5 | npm config set save-exact true
6 | npm install node-sass@3.4.1 --save
7 | npm install --save gulp-sass@2.1.0
```



Note: This modifies the package.json file. Preserve these changes.

3. Run `gulp local` to validate that the error is resolved.

If you have also installed gulp-sass globally, you might need to uninstall the globals.

To uninstall gulp-sass globals:

1. Open the command line in the top-level directory of your source code.
2. If you are a **Windows** user, execute the following command as either Administrator or as a User:


```
npm uninstall -g node-sass gulp-sass
```
3. If you are a **Linux / Mac** user, execute the following command as Administrator:


```
npm uninstall -g node-sass gulp-sass
sudo npm uninstall -g node-sass gulp-sass
```

Overview

 **Applies to:** SuiteCommerce Advanced

If you need to customize JavaScript, SuiteScript, or configuration objects that are not available through the Extensibility API, you can use the SCA developer tools to override or extend existing SCA source code to create the custom functionality you want. Do not edit SCA source code directly. Direct edits to source code are difficult to maintain and also make upgrades to newer versions of SCA a cumbersome and time-consuming process.

You should use the SCA developer tools when:

- You are customizing a site that implements the Aconcagua release of SCA or later and need to access JavaScript, SuiteScript, or configuration objects that are not available using the Extensibility API. See [Extensibility API Components](#) for a list of components exposed using the Extensibility API.
- You are customizing a site that implements the Kilimanjaro release of SCA or earlier.



Important: If you are implementing the Aconcagua Release of SCA or later, use themes and extensions to customize your site. To customize HTML or Sass files for these implementations, you must use the theme developer tools. See [Develop Themes](#) and [Develop Extensions](#) for details on using these tools. To learn more about why you should use the theme and extension developer tools, see [Benefits of the Extension Framework](#).

To learn more about using the SCA developer tools, see [Customize with SCA Tools](#).

Customize with SCA Tools

Applies to: SuiteCommerce Advanced

If your SCA version is Kilimanjaro or earlier, or the requirements of your customization cannot be met using the Extensibility API, you can use the SCA tools to develop your customization. But, before proceeding with the SCA tools, be sure to verify that your project requires it.

When you customize with the SCA tools, be sure to follow best practices such as extending JavaScript and JavaScript configuration files, overriding template, language, and other resource files, and organizing the source code for custom modules as recommended. For more information see the help topic [Best Practices for Customizing SCA](#).

To start using the SCA tools, review the following topics:

- [What Resources Do I Need?](#)
- [Next Steps](#)

What Resources Do I Need?

The resources and skills you need to customize Commerce websites with the SCA tools are described in the following table.

Requirement	Description
Skills	You must have experience working with JavaScript, TypeScript, HTML, and CSS. The level of experience required depends on the types of changes you want to make. The core functionality of SCA can be augmented or modified, but advanced JavaScript programming skills, including knowledge of Backbone.js and jQuery, are required.
Knowledge	Before you start to develop SCA customizations, be sure to learn more about: ■ Technologies like Model View Controller (MVC) and Asynchronous Module Definitions (AMD) described in SCA Framework Technologies ■ SCA modules, including Application, Framework, Feature, and Utility modules described in SCA Module Architecture ■ Frontend and backend models and services described in SCA Models, Collections, and Services ■ Requirements for custom modules described in Services and Backend Models in Custom Modules
SCA Tools	To use the SCA tools, you need to have the following resources installed in your local environment: ■ Node.js JavaScript runtime environment and all of its components, including the Node Package Manager (NPM) ■ Gulp.js JavaScript toolkit See Set Up SCA Developer Tools for more information.
Supported Scripts, Languages, and APIs	You can use standard web development scripts and languages when customizing SCA sites, including HTML, CSS, Sass, Handlebars.js, JavaScript, TypeScript, and AJAX. Available APIs include: ■ The Commerce API, which provides access to shopping objects and methods. For example, with Commerce API methods, you can access customers and orders within your shopping sessions. For more information, see the help topic Commerce API.

Requirement	Description
	■ If you have configured item search settings for your site, you can use the Items Search API to build facet filtering and keyword search on item data. For more information, see the help topic Item Search API. ■ SuiteScript is NetSuite's proprietary scripting language that provides custom objects and methods you can use with JavaScript. The SuiteScript API plays an important role in SCA site customizations because it enables you to access data stored in NetSuite records. You can also use SuiteScript to store data in NetSuite. To learn more, see the help topic SuiteScript Overview.
Supported Browsers	For more information, see Supported Browsers for Commerce Websites .
Additional Resources	The SuiteCommerce Developers Portal provides supplemental resources you can explore after setting up the SCA tools and mastering the basics of SCA customization.

Next Steps

To start developing with the SCA developer tools, complete the following tasks listed in this table.

Note: Remember, you should use the SCA developer tools only if you are customizing a site that implements the Kilimanjaro release of SCA or earlier, or if you need to access objects not available using the Extensibility API.

Step/Task	For detailed instructions, see
Set up the developer environment by installing Node.js and Gulp.js	■ Install Node.js ■ Install Gulp.js
Download and install SCA developer tools	■ Download SCA Source Files ■ Install Additional Files in the Source Directory
Understand best practices for customizing SCA	Best Practices for Customizing SCA
Customize by extending or overriding SCA source files	Example SCA Customizations
Understand how SCA source files are organized and compiled	SCA Build Process
Test your SCA customization locally	Test SCA Customizations on a Local Server
Deploy local source files to an SSP application in NetSuite and then deploy the SSP application to your site	Deploy SCA Customizations to NetSuite

SCA Framework Technologies

Applies to: SuiteCommerce Advanced

SuiteCommerce Advanced (SCA) source files are organized in modules. To learn how these modules work together to create Commerce web store applications, you must understand the underlying technologies that the application is built on. To implement these technologies, SCA uses several third-party libraries that are common in web-based ecommerce applications.

Note: These topics refer to SCA implementations. SuiteCommerce source also follows this framework; however, you create themes and extensions to customize SuiteCommerce. Extensions interact with the Extensibility API, and are handled using specific developer tools and practices. For information, see [Benefits of the Extension Framework](#), [Develop Themes](#), and [Develop Extensions](#).

Depending on your implementation of SCA, the application relies on either JavaScript or TypeScript to enhance large project development efficiency and scalability. The 2019.2 release of SCA and later uses TypeScript, while earlier releases use JavaScript.

To discover more about these technologies, review the following sections:

- [TypeScript](#)
- [Model View Controller \(MVC\) and Backbone.js](#)
- [Asynchronous Module Definitions \(AMD\) and RequireJS](#)
- [Logic-less Templates and Handlebars.js](#)

TypeScript

Applies to: SuiteCommerce Advanced

SuiteCommerce Advanced (SCA) source code is being migrated to TypeScript in phases. TypeScript is an open source superset of JavaScript designed to optimize development efficiency. To learn more about TypeScript, see [Typescriptlang.org](https://typescriptlang.org).

Important: Do not edit SCA source code directly. Use the SCA developer tools to override or extend existing SCA source code to create the custom functionality you want. For more information, see the help topic [Best Practices for Customizing SCA](#).

The code migration to TypeScript is ongoing and includes changes such as:

- Changing filename extension from .js to .ts
- Transforming module system from Asynchronous Module Definition (AMD) to a TypeScript compatible (common JavaScript) system
- Implementing base class for Models
- Implementing base class for Collection
- Migrating Utils.ts
- Migrating Case.Model
- Migrating Case.Collection

TypeScript code compiles to common JavaScript. The `gulp local` and `gulp deploy` commands automatically compile all SCA TypeScript files to JavaScript.

Model View Controller (MVC) and Backbone.js

i Applies to: SuiteCommerce Advanced

Source files of each module follow the Model View Controller (MVC) architectural pattern. One of the core principles of the MVC pattern is to separate the presentation layer (view) of an application from the data (model) used by the application. All interaction between views and models are handled by the controller.

To implement the MVC pattern, SuiteCommerce Advanced (SCA) uses the Backbone.js libraries. Backbone.js is an open-source JavaScript library that provides a framework for developing web applications.

Structure of a Backbone.js Application

Backbone.js is based on the MVC pattern. Modules that define interfaces or handle data use Backbone.js to define routers, views, and models/collections. The Case module, for example, implements each of these:

- **Routers:** map URLs to client-side pages to methods that define actions and events. In feature modules, these methods initialize the views, models, and collections that handle the user interface and create frontend objects to handle data.

Each router contains a `routes` object that defines the mapping between URLs and methods.

- **Views:** contain methods that define the behavior of the user interface. Views are commonly associated with a model or collection. When a view is initialized, its corresponding model is usually passed, ensuring that the view has access to the data within the model. Most views define listeners that notify the view when an element within the model is modified. These listeners enable a specific part of the application to be updated independently without having to reload the page.

With the Elbrus release of SCA and later, all views are composite views by default and extend `Backbone.CompositeView.js`.

Views do not define the HTML or CSS of a feature or component. Instead, they specify the template that defines the HTML code for the component or feature. The templating engine handles the rendering and updating of the HTML.

- **Models and Collections:** define objects that contain data used by the application. They also define the methods that create, update, or delete data.

Backbone.js Module Example

For example, the Case module defines a router (`Case.Router.js`) that contains the following:

```
1 routes:
2 {
3   'cases': 'showCasesList'
4   ,
5   'cases?:options': 'showCasesList'
6   ,
7   'cases/:id': 'showCase'
8   ,
9   'newcase': 'createNewCase'
```

10 | }

When a user clicks on an individual case from the list of support cases, instead of sending an HTTP request back to the server, the router calls the `showCaseList` method based on the value of the partial URL (cases). This method performs two main tasks:

- Initializes the model that contains data for a specific support case (`Case.Model`).
- Initializes the view that displays the data (`Case.Detail.View`)

`Case.Model` defines the data object that contains information about an individual support case. `Case.Model` extends the `Backbone.Model` by defining additional methods for performing frontend validation. When the router initializes `Case.Detail.View`, it passes the instance of `Case.Model`.

SCA Backbone.js Implementation

The Backbone.js libraries provide a general framework for how a web application is structured and its general functionality. However, much of the specific functionality of the application must be implemented. SCA stores Backbone.js files in the following directories:

- **Modules/third_parties/backbone.js:** contains the core Backbone.js libraries. In general, you should not modify the files in this directory.
- **Modules/suitecommerce/BackboneExtras:** contains extensions of core Backbone.js functionality, including `Backbone.View` and `Backbone.Model`. If you need to make changes to the Backbone.js framework, you may need to modify the files in this directory.

Note: In SCA 2020.1 and later, you should use the SuiteCommerce extensibility API's `SCCollection` and `SCModel` components instead of the native `Backbone.Collection` and `Backbone.Model`. In SCA2020.2 and later, you should use the SuiteCommerce extensibility API's `SCView` component instead of the native `Backbone.View`. `SCCollection`, `SCModel`, and `SCView` are components of the SuiteCommerce extensibility API that replace `Backbone.Collection`, `Backbone.Model`, and `Backbone.View`, respectively. See the [extensibility API reference](#) for more information.

Asynchronous Module Definitions (AMD) and RequireJS

Applies to: SuiteCommerce Advanced

Although Backbone.js provides a general framework for web applications, it does not specify how application code should be organized. To organize code into modules, SuiteCommerce Advanced implements another design pattern called Asynchronous Module Definitions (AMD).

- SuiteCommerce Advanced 2019.2 and later uses ECMAScript 2015 standard JavaScript in TypeScript files to define modules and dependencies. These TypeScript files are compiled to JavaScript when executing `gulp local` and `gulp deploy` commands. During this compile, Asynchronous Module Definitions (AMD) design pattern code is produced.
- SuiteCommerce Advanced 2019.1 and earlier source code is provided as JavaScript files and implements the Asynchronous Module Definitions (AMD) design pattern.

See [Dependencies](#) for more information about how modules implement the AMD design pattern.

Logic-less Templates and Handlebars.js

i Applies to: SuiteCommerce Advanced

Another advantage of the Backbone.js libraries is that they provide open support for web templates and templating engines. Web templates contain the raw HTML of the user interface. Each module that contains Backbone.js views uses a template to define the corresponding HTML. These files are stored in the Templates directory.

Templates define the HTML for discrete areas or features of the user interface. The template engine combines all of the template files into a single, finished web page.

SuiteCommerce Advanced (SCA) uses the Handlebars.js library to implement templates and the template engine. One advantage of Handlebars.js is that it provides logic-less templates. Consequently most of the business logic of the application is handled outside of the template. Within the Backbone.js framework, this logic is handled by the view. Using logic-less templates means that the HTML within the template is much easier to understand.

Although Handlebars.js is considered a logic-less template, it does provide basic logical constructs. These are defined by Handlebars.js helpers. Helpers are primarily used to evaluate values of placeholders and display the appropriate HTML. Placeholders are like variables that contain information that is added to the HTML when the template is generated. This information is passed to the template by the getContext method of a view.

The following code sample from the case_list.tpl template file shows how these are used to display the user interface according to the current state of the application.

```

1 <div class="case-list-results-container">
2   {{#if hasCases}}
3     <table class="recordviews-table">
4       ...
5       ...
6     </table>
7   {{else}}
8     {{#if isLoading}}
9       <p class="case-list-empty">{{translate 'Loading...'}}</p>
10    {{else}}
11      <p class="case-list-empty">{{translate 'No cases were found'}}</p>
12    {{/if}}
13  {{/if}}
14
```

In this example, the Handlebars.js engine evaluates the hasCases placeholder. This placeholder corresponds to a property in the object passed to the template engine from the getContext method of the view. hasCases is a Boolean property. If its value is true, the template engine outputs a table. If its value is false, the template engines check the value of the isLoading property which is also passed from the getContext method. The template engine displays a message based on the value of this property.

In the above example, the if statement is an example of a default Handlebars.js helper. SCA provides additional helpers that you can use in your templates. For more information, see [Custom Helpers](#).

Templates and the Template Context

i Applies to: SuiteCommerce Advanced

SuiteCommerce Advanced (SCA) uses templates to define the HTML code used to display features of the application. Templates perform the following primary tasks:

- Define the raw HTML code that displays the user interface of a browser-based application.
- Contain placeholders that are replaced with data passed to the template.

Templates are compiled into functional JavaScript and HTML by a templating engine. SCA uses the Handlebars.js library to define templates and the template engine. The template engine performs the following tasks:

- Transforms the raw template file into a JavaScript function.
- Replaces placeholders based on data passed to the template

Logic-less Templates

Logic-less templates ensure that HTML code is separated from application code. All application logic is handled in the view. For example, when the application needs to access data stored in a model, a view handles that interaction. Logic-less templates make the application more modular and ensure that the source code is easier to maintain. More importantly, these templates ensure template customizations are not broken when upgrading to a new SCA version.

Template Compilation

The source files for each template contain the raw HTML as well as Handlebars placeholders. When you compile the application, one of the tasks the `gulp template` command performs is to pre-compile each of the template source files into a JavaScript function. The `gulp deploy` and `gulp local` commands call the `gulp template` command which, in turn, calls the Handlebars compiler.

The result of the `gulp template` command is that the template is transformed into a JavaScript function that can be called by passing a context object. This function is called when the view is rendered. Within the template file, Handlebars.js placeholders are transformed into JavaScript functions. For example, the following `if` construct:

```
1 | {{#if showLanguages}}
```

is transformed into the following code when the template is compiled:

```
1 | if(context.showLanguages){}
```

Handlebars.js placeholders that do not contain logic are also transformed into JavaScript functions. For example the following HTML code and placeholder:

```
1 | <p>my name is {{userName}}</p>
```

is transformed into the following when the template is compiled:

```
1 | function(context){var s = '<p>myname is'; s += context.userName; s += '</p>'; return s; }
```

Interaction Between Templates and Views

Since templates do not contain application code and contain minimal logic, the data they require must be passed to them. When calling, the template method passes the context object. Templates only contain logic for performing loops and conditionals. Views contain all of the logic required to display the contents of a template. Views also contain all of the application logic. To display a template, the view calls the template function from the `view.render` method. For example, when the application is compiled and

running, a view's render method would contain code similar to the following example to access the values contained in the context object:

```
1 | var context = this.getContext() body.innerHTML = this.template(context)
```

In this example, the view passes the context object to the template function. The object passed to the template is the object returned by the getContext method. In SCA, almost every view contains a getContext method. In general, this method is responsible for returning a context object that contains all of the data required by the template. This object may contain properties that contain data obtained from the model, configuration properties, or other properties the view requires.

Template Context

The template's context object returned by the getContext method can be seen as a contract between the view and the template. The purpose of this contract is to ensure that any customizations you make to the template are not overridden or broken when upgrading to a new version of SCA. Properties within the context object are not removed or modified in future versions. New properties may be added to the context object, but it will always be backward compatible with previous versions.

To ensure that this contract is maintained, the template only has access to this context object. The view is responsible for providing the context object and defining and respecting the contract in future versions.

Another area where the template context is important is within composite views. For example, you can customize a child view to use a custom template instead of the default template. When customizing an existing template or creating a new one, if you access the properties contained in the context object, these customizations will be preserved after an upgrade.

Custom Helpers

The Handlebars.js library defines helpers that perform functionality beyond that performed by the default placeholders. Handlebars.js defines multiple default helpers.

In addition to these default helpers, SCA defines **custom** helpers that perform various tasks. These helpers are defined in the HandlebarsExtras.js file located in the JavaScript subdirectory of the HandlebarsExtras application module. The following helpers are defined:

- **translate**: returns a safe string that contains a localized version of the string by calling the `_.translate` function. Returning a safe string ensures that any HTML formatting or other text is preserved.
- **formatCurrency**: returns a formatted currency value by calling the `formatCurrency` function.
- **highlightKeyword**: returns a highlighted keyword by calling the `_.highlightKeyword` function. For example, the `site_search_item.tpl` template file uses this helper to highlight search results within a page using the following:

```
1 | {{highlightKeyword model._name query}}
```

- **displayMessage**: creates an instance of a `GlobalViewsMessageView` and renders the view.
- **objectToAttributes**: returns the attributes of an object, causing them to be displayed in the template. This helper calls the `_.objectToAttributes` method.
- **each**: defines a custom each helper that supports a Backbone.js collection. This helper iterates through each item of the collection.
- **resizeImage**: builds URL images using the `imageSizeMapping` utility. This helper passes a URL and an object containing the dimensions of the image to this utility. It returns a normalized URL image based on these values.

- `fixUrl`: returns a valid URL by calling the `_.fixUrl` utility.
- `trimHtml`: returns a trimmed HTML string based on the length passed to the helper. This helper calls the `jQuery.trim` method. The string returned can contain HTML elements.
- `breaklines`: places break tags (`<br/>`) instead of new lines provided by the backend. Use this in Quotes, Returns, Case, and Review Order messages.
- `ifEquals`: creates an equality condition to a Handlebars `{{if}}` helper. The following example depicts use for a custom transaction body field:

```
1 | {{#ifEquals model.options.custbody_my_transaction_body_field_2 'T'}}
2 |   This is true
3 | {{else}}
4 |   This is false
5 | {{/ifEquals}}
```


SCA Module Architecture

i Applies to: SuiteCommerce Advanced

The source files for SuiteCommerce Advanced (SCA) are organized in multiple modules. Each module defines a specific area of functionality which generally falls in one of the following categories:

- **Application modules:** define high-level collections of features that perform similar types of functions. SCA includes three separate applications:
 - Shopping
 - Checkout
 - My Account
- **Feature modules:** define specific areas of functionality within SCA. The Case module, for example, contains the code and style sheets that implement the Case Management feature.
- **Framework modules:** define the general logic and structure of SCA and define how modules and applications work together. The application modules extend these modules to inherit the basic framework.
- **Utility modules:** provide functionality that is used by multiple modules. The GlobalViews module, for example, defines views that are reused in multiple areas of SCA.

The following sections provide more information about SCA modules and source files:

- [Dependencies](#)
- [Application Modules](#)
- [Feature Modules](#)

Dependencies

i Applies to: SuiteCommerce Advanced

Every JavaScript file in the SCA source contains the same general structure that corresponds to Asynchronous Module Definition (AMD) API. RequireJS uses the AMD API to asynchronously load modules within the application. The following code sample shows the general structure of each file:

```

1  define('<module_name>'
2
3  [ '<dependency_1>'
4
5    '<dependency_2>'
6    ...
7  ]
8
9  function (
10    <module_1>
11    , <module_2>
12    ...
13  )
14
15 )

```

```
16 | {  
17 |     <module code>  
18 | }
```

All files within a module contain one define function which occurs at the beginning of the file and uses the following syntax:

```
1 | define(module_id, [dependencies], function)
```

- **module_id** – a string defining the name of the module.
- **dependencies** – an array containing the names of the dependencies. In SCA, this is generally the name of a module, a component within a module, or a core dependency.
- **function** – defines a function that instantiates the module.

By following this structure consistently, the developer tools can compile the application into a single JavaScript file by calculating the cascading dependencies starting with the root level application modules down to each lower-level feature and utility modules.

Application Modules

Applies to: SuiteCommerce Advanced

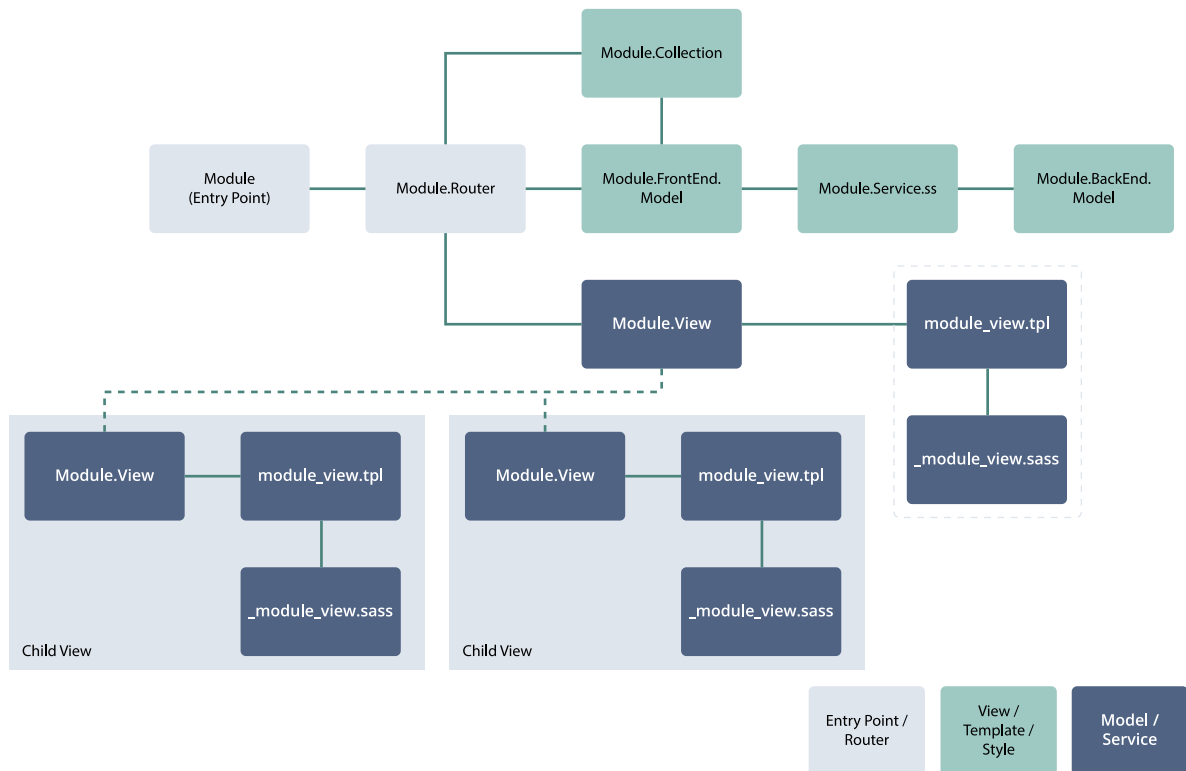
SuiteCommerce Advanced (SCA) is composed of three separate applications: Checkout, My Account, and Shopping. Each of these applications contains its own top-level modules that define the dependencies and general layout. These modules are contained in the following directories:

Application	Directory	Entry Point
My Account	MyAccountApplication	SC.MyAccount.Starter.ts
	MyAccountApplication.SCA	
Checkout	CheckoutApplication	SC.Checkout.Starter.ts
Shopping	Shopping Application	SC.Shopping.Starter.ts

Each application module contains a starter file which defines the starting node or entry point for each application. Starter files for recent SCA versions are TypeScript files with a .ts extension. Earlier SCA versions have JavaScript starter files with a .js extension. Each starter file lists all of the dependencies for each application. Any dependencies not listed in the starter file will not be included in the application.

 **Note:** When creating a new module, you must add it to the starter file for each application that uses the new module. See [Create a Custom Module](#) for more information.

The following image shows the hierarchical relationship between the top-level application modules and the lower-level feature modules.



Feature Modules

Applies to: SuiteCommerce Advanced

Most of the modules within SuiteCommerce Advanced (SCA) are feature modules, which means they define the behavior for a specific feature or related functionality. In contrast to the application and framework modules which define the underlying infrastructure and logic of an application, feature modules perform the following:

- Define the user interface for a feature
- Define the low-level application behavior
- Define data models
- Handle data transfer between the application and NetSuite

For more detailed information about feature modules and how they work, see [SCA Feature Modules](#).

SCA Feature Modules

Applies to: SuiteCommerce Advanced

SuiteCommerce Advanced (SCA) feature modules define the behavior for a specific feature or related functionality. Feature modules generally conform to the Model-View-Present (MVP) design paradigm prescribed by Backbone.js. Therefore, that they implement some combination of the following:

- **Entry Point:** defines how the module interfaces with the top-level application.
- **Routers:** direct URLs to client-side methods.
- **Views:** listen to the user interface events and coordinate with the data required by the associated module.
- **Models, Collections, and Services:** retrieve, create, and update data on the frontend and backend.

The following sections provide more information about the components of a feature module:

- [Entry Point](#)
- [Routers](#)
- [Views](#)
- [SCA Models, Collections, and Services](#)

Entry Point

Applies to: SuiteCommerce Advanced

Most feature modules contain an entry point which defines how the module interfaces with the top-level application module. The entry point of a module is defined in a file called, <module_name>.js.

All modules that are dependencies of the application define a method called `mountToApp`. The `ApplicationSkeleton` module calls this method for each module listed as a dependency in the starter file of the application. For most modules, the `mountToApp` method initializes and returns the router for the module, making the module available to the higher-level application context.

For modules that are included in the My Account application, the entry point also returns a `MenuObject` item. This object contains information about the module that the `SC.MyAccount` node uses to create and display the menu items that appear in My Account.

Routers

Applies to: SuiteCommerce Advanced

Within a single-page application, routers direct URLs to client-side methods rather than passing an HTTP request to the server. All routers contain a `routes` property that maps between a partial URL and a method defined within the router. Methods within the router perform different functions including initializing views, models, and collections.

For example, the `Address.Router` contains the following `routes` property:

```
1 routes: {
2   'addressbook': 'addressBook'
3   , 'addressbook/new': 'newAddress'
4   , 'addressbook/:id': 'addressDetailed'
5 }
```

When a user clicks on the Address Book menu item in the My Account application, the router of the Address module intercepts the partial URL (addressbook) and calls the `addressBook` method. The `addressBook` method is responsible for initializing the collection containing the list of addresses and initializing the view that displays them. Depending on the module, routers may initialize properties and variables required by the module. They may also contain methods that define the behavior and logic of the module.

Views

Applies to: SuiteCommerce Advanced

Within the Backbone.js framework, views listen to the user interface events and coordinate with the data required by the associated module. In some modules, when a router initializes a view, the router also passes a model or collection containing the data required. In other modules, the view includes all necessary models or collections as dependencies.

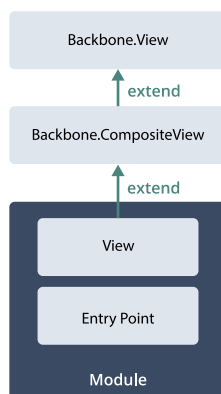
View Architecture

Note: The information in this section applies to the Elbrus release of SCA and later. However, if you are customizing SCA 2020.2 and later, do not use `Backbone.View` as described in the following sections and examples. For release 2020.2 and later, you should use `SCView` as described in the [extensibility API reference](#). `SCView` is the SuiteCommerce implementation of `Backbone.View`.

All `Backbone.Views` are `Backbone.CompositeViews` by default. Composite views are views that contain child views. Composite views further modularize the application, so some types of views can be used in multiple contexts. For example, some modules define child views that are extensions of `Backbone.View`. Therefore, those child views are also `Backbone.CompositeViews`, which ensures that some types of data are displayed in a consistent way across the application.

All views extend `Backbone.CompositeView` by default. Any custom views should use code similar code to this example:

```
1  ...js
2  var MyView = Backbone.View.extend({
3    initialize: function()
4    {
5      //every view now is a composite view!
6    }
7  });
8  ...
```



A parent view must declare its child views. To do this, each Parent View declares the `childViews` property, which is an object containing all the Container names from the View (the data-views). Each container has an object with the Child View's name and an object for each View with the following information:

- `childViewIndex` – this is the order in which the children render in that container.
- `childViewConstructor` – This can be either functions or `Backbone.View` subclasses. Each defined function must return an instance of a `Backbone.View`.

The following example declares the `childViews` property, containing the Buttons container. The Buttons container declares the `Whishlist` and `SaveForLater` objects. You can include multiple views in one container.

```

1  ...
2  childViews: {
3    'Buttons': {
4      'Whishlist': {
5        'childViewIndex': 10
6        , 'childViewConstructor': function() {
7          return new WhishlistView();
8        }
9      }
10     'SaveForLater': {
11       'childViewIndex': 20
12       , 'childViewConstructor': SomeView
13     }
14   }
15 }
16 ...

```



Note: You can add new child views to a view class using the `addChildViews( )` method. You can also add new child views to an instance of a view using the `addChildViews( )` method or by passing the `childViews` property in the options when initializing it.

To add a plugin before or after you initialize the View, add the plugin to the View class and execute code in those moments (`beforeInitialize` and `afterInitialize`), as shown:

```

1  ...
2  Backbone.View.beforeInitialize.install({
3    name: 'compositeView'
4    , priority: 1
5    , execute: function ()
6    {
7      var childViews = _.extend({}, this.childViews, this.constructor.childViews);
8
9      this.childViewInstances = {};
10
11     this.addChildViews(childViews);
12
13     if (this.options)
14     {
15       if (this.options.extraChildViews)
16       {
17         console.warn('DEPRECATED: "options.extraChildViews" is deprecated. Use "options.childViews" instead');
18         //Add extra child views from view's initialization options
19
20         this.addChildViews(this.options.extraChildViews);
21       }
22
23       if (this.options.childViews)
24       {
25         //Add child views from view's initialization options
26         this.addChildViews(this.options.childViews);
27       }
28     }
29   }
30 }

```

```

29     }
30   });
31   ...

```

Backward Compatibility

To make this change backward compatible with earlier releases of SCA, Backbone.CompositeView.js includes a CompositeView.add() method as a no operation (noop) method. This prevents any errors when calling it from any custom code in implementations prior to the Elbrus release. SCA supports the previous format as shown:

```

1  ...
2  childViews: {
3    'Promocode': function()
4    {
5      return new PromocodeView({model: this.model.get('promocode')});
6    }
7  }, 'SomeWidget': SomeWidget
8  }
9  ...

```

In this case, the container and the view have the same name, and you can combine the old and the new format in the childViews property.

```

1  ...
2  childViews: {
3    Promocode':
4    {
5      'Promocode':
6      {
7        childViewIndex: 10
8        , childViewConstructor: function(options)
9        {
10         return new PromocodeView({model: this.model.get('promocode')});
11       }
12     }
13   }
14 }
15 ...

```

View Architecture (Vinson Release and Earlier)

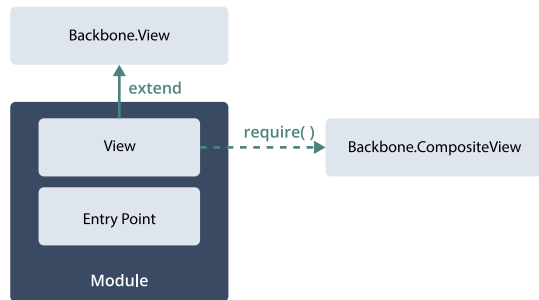
This section applies to the Vinson release of SCA and earlier.

In the Vinson release of SCA and earlier, each view must declare itself as a composite view using the require() method and call CompositeView.add(this) within the initialize() method. Any custom views should use code similar to this example:

```

1  ...
2  var CompositeView = require('Backbone.CompositeView');
3  var MyView = new backbone.View.extend({
4    initialize: function()
5    {
6      CompositeView.add(this);
7      //now this view is a composite view!
8    }
9  });
10 ...

```



Rendering Data From the View

The process of rendering the HTML is handled by the templating engine and the Backbone.js framework. To work within this framework, each view defines the following:

- A template file included as a dependency within the define method of the view.
- A template property defined in the body of the view.
- A getContext method that returns all of the data attributes required by the template.

When you compile SCA using the developer tools, Gulp.js uses these elements to compile the templates into the application. See [Logic-less Templates and Handlebars.js](#) for more information about templates and the template engine.

If implementing the Elbrus release of SCA or later, a data view acts as a container for multiple child views. Any parent view must declare a placeholder element in its associated template to render the children within it as depicted in the following example:

```

1 | ...
2 | html
3 |   <div data-view="Buttons"></div>
4 | ...
  
```


SCA Models, Collections, and Services

Applies to: SuiteCommerce Advanced

SuiteCommerce Advanced (SCA) works with data stored in NetSuite. During a shopping session, for example, SCA may perform the following data-related actions:

- Read data from NetSuite records. For example, when a user views details about an item, this information is retrieved from the NetSuite item record.
- Update data based on user input. For example, when a user places an order, information about the order is stored in a NetSuite order record.
- Receive new data based on user input. For example, when a user adds a new address to the Address Book, the NetSuite address record is updated.

To retrieve, create, and update data, SCA processes the data on the frontend and backend as follows:

- Frontend: defines how data is presented to the user and how the user inputs data. This data is handled by frontend models and collections.
- Backend: defines how SCA transfers data to and from NetSuite. This transfer is handled by services and backend models.

Read the following sections to learn more:

- [Frontend Models](#)
- [Collections](#)
- [Services and Backend Models](#)
- [Services and Backend Models in Custom Modules](#)

Frontend Models

Applies to: SuiteCommerce Advanced

Frontend models contain the data that SuiteCommerce Advanced (SCA) uses. Frontend models are part of the Backbone.js framework that defines the data component within the MVC paradigm. Each frontend model is an extension of Backbone.Model which is defined in BackboneExtras. Backbone.Model defines inheritable methods that are available to all models defined in SCA. If necessary, a model can override these core methods. For example, many modules override the `initialize` method to set values of properties that are specific to that model.

Note: If you are customizing SCA 2020.1 and later, do not use Backbone.Model as described in the following sections and examples. For release 2020.1 and later, you should use SCModel as described in the [extensibility API reference](#). SCModel is the SuiteCommerce implementation of Backbone.Model and it inherits all methods and properties of Backbone.Model

Most feature modules define models that hold the data they require. Models in SCA are defined in the following ways:

- Models that are specific to features are defined by returning an object containing the model. These models are defined using the following syntax:

```
1 | return Backbone.Model.extend
```

- Models that are used by the wider application context. These models are created and their data populated when they are initialized. These models are defined in the following manner:

```
1 | ProfileModel = Backbone.Model.extend
```

By defining a model this way, it can be accessed from models and views that are defined outside of the current module. In this example, the ProfileModel can be accessed by other modules that need to access information stored in the user's profile. Most models that are created this way are defined as singletons. Therefore there can only be one instance defined per user session.

When a router or view initializes a model, it may pass properties that the model requires. These can be properties that the frontend model uses internally or properties that are passed to the backend model.

In general, frontend models contain code that performs the following tasks:

- Sets initial values of properties
- Defines model-specific methods
- Overrides default methods
- Performs frontend validation of user-submitted data

All frontend modules define the following properties:

- `urlRoot`: specifies the backend service that is used to handle HTTP requests. This property is a string containing a partial URL.
- `validation`: specifies an object that defines the properties that are validated when a user submits form data.

Asynchronous Data Transactions

i Applies to: SuiteCommerce Advanced

When working with data stored in NetSuite or, internally, within the application, SCA often needs to perform tasks asynchronously. Handling tasks asynchronously improves the performance of SCA implementations.

To handle tasks asynchronously, SCA uses the jQuery Deferred Object API. This API provides an easy way of registering callbacks and monitoring the success or failure of data transactions. See [jQuery API documentation](#) for information about the methods provided by the Deferred Object API.

i Note: Although there are other JavaScript mechanisms for performing tasks asynchronously, NetSuite recommends that you use jQuery Deferred Objects when customizing or extending SCA.

There are two main contexts where SCA uses the Deferred Object API:

- When performing AJAX calls using the `save()`, `fetch()`, and `destroy()` methods of a model or collection
- When performing tasks asynchronously using promises

See the following sections for more information:

- [Using the Deferred Object in AJAX Calls](#)
- [Using Promises in Asynchronous Data Transfers](#)
- [Stopping AJAX Requests](#)

Using the Deferred Object in AJAX Calls

SuiteCommerce Advanced (SCA) frequently interacts with data stored in NetSuite. To perform a data transaction, the code within a router or view calls a method on the frontend model. This method then makes an AJAX call that sends an HTTP request to the service.

In general, this AJAX call needs to be asynchronous since other application code may need to continue processing while the data is loading. The primary reason for using asynchronous AJAX calls is that each browser window operates within a single thread. A synchronous AJAX call will freeze the browser until the AJAX call is complete.

One frequent use of the deferred object is to wait until the AJAX call is complete to perform other tasks.

The following example from `Case.Detail.View` shows how the `done` method is used when saving data to NetSuite:

```
1 this.model.save().done(function()
2 {
3     self.showContent();
4     self.showConfirmationMessage(_('Case successfully closed').translate());
5     jQuery('#reply').val('');
6 });
```

In this example, the `done` method accepts a single function as an argument. This function calls the `showContent()` method of the view and performs other tasks, which defers calling the `showContent()` method until the data transaction initiated by the `save` method is completed.

Using Promises in Asynchronous Data Transfers

SuiteCommerce Advanced (SCA) also uses the Deferred Object API in other contexts where tasks need to be performed asynchronously. In this case, a module explicitly creates an instance of the `jQuery.Deferred` object. After creating this object the module must ensure that the deferred object is ultimately resolved. This resolution is handled using promises.

Promises are part of the jQuery Deferred Object that enable asynchronous data transfers. Promises prevent other methods from interfering with the progress or status of the request. The following code example from the `ProductList.CartSaveForLater.View` module shows how a deferred object is created and ultimately resolved:

```
1 addItemToList: function (product)
2 {
3     var defer = jQuery.Deferred();
4
5     if (this.validateGiftCertificate(product))
6     {
7         var self = this;
8
9
10        this.application.ProductListModule.Utils.getSavedForLaterProductList().done(function(pl_json)
11        {
12            if (!pl_json.internalid)
13            {
14                var pl_model = new ProductListModel(pl_json);
15
16                pl_model.save().done(function (pl_json)
17                {
18                    self.doAddItemToList(pl_json.internalid, product, defer);
19                });
20            }
21            else
22            {
23                self.doAddItemToList(pl_json.internalid, product, defer);
```

```

24         }
25     });
26 }
27 else
28 {
29     defer.resolve();
30 }
31
32 return defer.promise();
33 }

```

Stopping AJAX Requests

SuiteCommerce Advanced (SCA) includes an `AjaxRequestsKiller` utility module that you can use to stop execution of AJAX requests. You can enhance performance by stopping AJAX requests that are no longer needed. This module is primarily used in two contexts:

- URL changes:** When a URL change is detected, this module stops execution of all pending AJAX requests that were initiated by another router or view. Some AJAX requests are initiated without a URL change, for example, the `collection.fetch` and `model.fetch` methods. In such cases, you should pass the `killerID` property to the method as shown in the following example:

```

1 | model.fetch({
2 |     killerID: AjaxRequestsKiller.getKillerId()
3 | })

```

- Site Search:** The `SiteSearch` module uses the `AjaxRequestsKiller` module to delete unnecessary AJAX calls when using type-ahead functionality. As a user enters a character in the search field, the `SiteSearch` module sends an AJAX request to retrieve information from the Item Search API. As the user continues entering characters, additional AJAX requests are sent. Using the `AjaxRequestsKiller` improves performance by deleting old requests.

When the value of the `killerID` property is reset, this module abandons the AJAX request associated with the old value of the `killerID` property. It also sets the `preventDefault` property of the request to `true` to avoid sending an error to the `ErrorHandling` module.

This module defines the following properties and methods:

- getKillerId (method):** returns a unique ID that identifies the AJAX request. Each time the URL is reset, a new ID is generated.
- getLambsToBeKilled (method):** returns an array of AJAX request IDs to be halted.
- mountToApp (method):** loads the module into the application context. This method performs the following tasks:
 - Sets the value of the `killerID` property
 - Pushes the `killerID` property to an array
 - When the `killerID` property is reset, calls the `abort` method on the AJAX request

Collections

Applies to: SuiteCommerce Advanced

Collections are sets of models. Define the data object that contains the set of models. Define methods that perform actions when the data in a model changes. They can also perform actions on all models within the collection.

SuiteCommerce Advanced often uses collections when displaying lists of items. The Case module, for example, uses Case.Collection to load a list of support cases assigned to a user and Case.List.View uses the collection to display the list.

Note: In SuiteCommerce 2020.1 and later, you should use the SuiteCommerce SCCollection component instead of Backbone.Collection. SCCollection is a component of the SuiteCommerce extensibility API. It inherits all methods and properties of Backbone.Collection. See the [extensibility API reference](#) for more information.

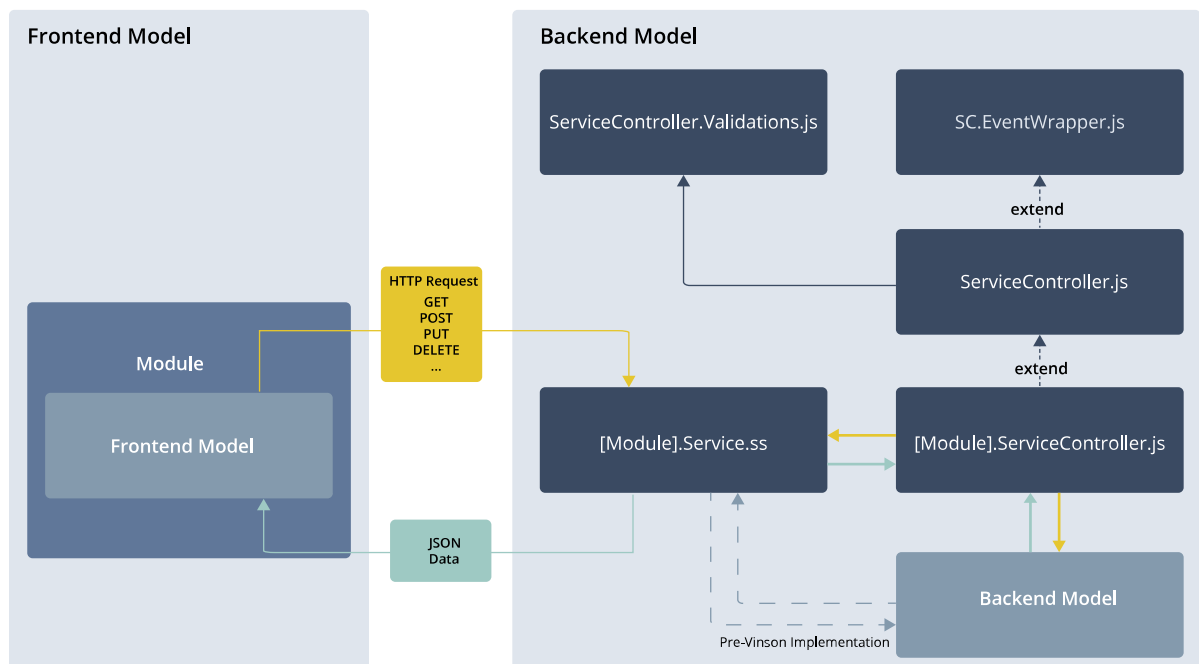
Services and Backend Models

Applies to: SuiteCommerce Advanced

To handle data transactions with NetSuite, SuiteCommerce Advanced (SCA) provides multiple endpoints that enable access to data stored in NetSuite records. How SCA handles these RESTful services depends on your implementation. For details about the services architecture for earlier versions of SCA, see [Architecture Overview \(Pre-Vinson\)](#).

This section explains the service and backend model architecture for the Vinson release of SCA and later. Services handle communication between the frontend SCA application and the backend models that connect to NetSuite. With the Vinson release, these services (.ss files) are auto-generated and call Service Controllers. These Service Controllers and other files maintain code common to all services within a centralized location, as described in the detailed descriptions that follow.

Note: The Vinson release of SCA is backward-compatible with the pre-Vinson services. To enable backward-compatibility, you must edit the associated module's ns.package.json files. See [Using Pre-Vinson Services with Vinson Release or Later](#) for more information.



Note: The files and functionality described in this topic apply to the SCA components that are coded in SuiteScript 1.0. The following content does not apply to the components that have been migrated to SuiteScript 2.0. When migration to SuiteScript 2.0 nears completion, more information will be provided for the migrated components. See [Commerce Migration to SuiteScript 2.0](#) for more information about the migration progress.

The following components make up the backend service architecture. The following sections provide more detail about each component:

- [SC.EventWrapper](#)
- [ServiceController](#)
- [\[Module\].ServiceController](#)
- [\[Module\].Service.ss](#)
- [Backend Model](#)
- [ServiceController.Validations.js](#)

SC.EventWrapper

This file contains centralized, extensible Before and After Event listening capabilities plus extension capabilities for use by all service methods. `SC.EventWrapper.js` is located in the `SspLibraries` module.

ServiceController

This file contains the common logic used to operate any service. `ServiceController` contains the `handle()` method, which executes the method related to the HTTP requests identified in any `[Module].ServiceController`. `ServiceController.js` is located in the `SspLibraries` module.

`ServiceController` contains the required try/catch block to ensure that any errors are passed back to the frontend application and are handled by the `ErrorManagement` module. The `ServiceController` module also extends `SC.EventWrapper` and references validation methods declared in `ServiceController.Validations`.

Note: As of the 2019.2 release of SCA, all instances that extend `ServiceController` are located in one of two places. For modules utilizing SuiteScript 2.0, these instances can be found in `SC_xxx/Backend/SC/`. If customizing using SuiteScript 1.0, these can be found in the SuiteScript folder of the module during development. When you deploy to the server, the specific `[Module].ServiceController` code writes to the `ssp_libraries.js` file.

[Module].ServiceController

One `[Module].ServiceController` exists per service. This file processes requests for a specific service and extends code within `ServiceController`. Each `[Module].ServiceController` executes the proper functions based on the HTTP actions (GET, POST, PUT, DELETE) initiated by the frontend framework, which are passed through the associated `.ss` file. `[Module].ServiceController` can also call methods on any necessary backend models.

Each `[Module].ServiceController` can include an options object, which details any permissions and validations needed for the HTTP methods. Each service requires a unique `[Module].ServiceController`.

[Module].Service.ss

This file handles communication between the frontend SCA application and the associated controller ([Module].ServiceController). The .ss file is automatically generated in the services folder of the local distribution. After you deploy your site, the .ss file exists in the services folder of the proper SSP Application. One .ss file exists per service.

Each [Module].Service.ss file is also responsible for passing data from the backend model to the frontend model or collection that initiated the HTTP request. This data is passed as a JSON object from the backend model to the .ss file through the associated [Module].ServiceController.

The following example shows the auto-generated .ss file for the Account Login service. This file only calls the [Model].ServiceController associated with the service as defined in the ns.package.json file.

```

1 //Autogenerated service function.
2 function service(request, response)
3 {
4   'use strict';
5   require('Account.Login.ServiceController').handle(request,response);
6 }
```



Note: When you deploy your site, the autogenerated-services object of the associated module's ns.package.json file signals the developer tools to auto-generate each .ss file and associate it with a specific service controller. Failure to include this code will result in an error and the .ss file will not generate. If you want to use pre-existing (pre-Vinson) services with a Vinson implementation, you must register the service in the services section of the associated module's ns.package.json file. See [Using Pre-Vinson Services with Vinson Release or Later](#) for more information.

Backend Model

The backend model contains methods that access NetSuite records by calling the SuiteScript API. The Backend Model architecture and logic have not changed with the Vinson Release of SCA.

ServiceController.Validations.js

This file contains the extensible validation methods for requests common to all services. This file is located in the SspLibraries module.

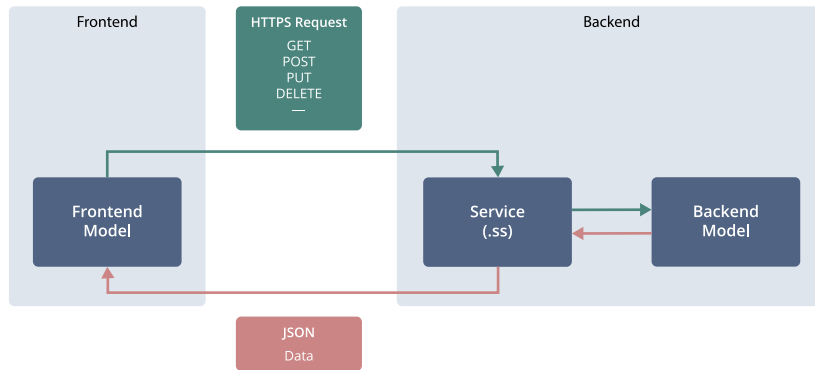
Each feature or module in SCA generally defines the services and backend models needed to access the required data. The Cart module, for example, defines the services and backend models for accessing items currently in the cart. Each module's [Module].ServiceController files and backend models are stored in the JavaScript directory of each module.

When you deploy SCA, the gulp tasks auto-generate the required .ss files and deploy all of the services to the SSP application. The gulp tasks also add backend models to the ssp_libraries.js file. For more information, see [The ssp_libraries.js File](#).

Architecture Overview (Pre-Vinson)

Applies to: SuiteCommerce Advanced

To handle data transactions with NetSuite, SuiteCommerce Advanced (SCA) provides multiple endpoints that enable access to data stored in NetSuite records. These endpoints run on NetSuite and are implemented as RESTful APIs.



Each endpoint has the following components. The following sections provide more detail about each component:

- [Service \(request handler\)](#)
- [Backend Model](#)

Service (request handler)

The service (.ss file) handles communication between the frontend SCA application and the backend models that connect to NetSuite. The service receives an HTTP request initiated by the frontend framework, usually a model or collection. Services handle standard HTTP request methods (GET, POST, PUT, DELETE). Based on the HTTP action specified in the request, the service calls methods on the backend model.

For example, the `ProductLists.Service.ss` service returns information about product lists. Since this is a RESTful service, you can access product list information using HTTP methods as follows:

- HTTP POST — create a new product list.
- HTTP PUT — update a product list.
- HTTP DELETE — delete a product list.
- HTTP GET — perform a search on a product list. This is accomplished by passing search filters as parameters to the service.

Services are also responsible for passing data from the backend model to the frontend model or collection that initiated the HTTP request. This data is passed as a JSON object.

Backend Model

The backend model contains methods that access NetSuite records by calling the SuiteScript API. See the help topic [SuiteScript API & Commerce Web Stores](#) for more information.

Each model defines methods that correspond to the HTTP actions handled by the service. For example, if a model is handling a GET action to retrieve data from NetSuite, it must also format the corresponding JSON object that is returned to the frontend framework.

Each feature or module in SCA generally defines the services and backend models needed to access the required data. The Cart module, for example, defines the services and backend models for accessing items currently in the cart. Services and backend models are stored in the JavaScript directory of each module.

When you deploy SCA, the gulp tasks deploy all of the services to the SSP application. The gulp tasks also add backend models to the `ssp_libraries.js` file. For more information, see [The `ssp\_libraries.js` File](#).

Services and Backend Models in Custom Modules

Applies to: SuiteCommerce Advanced

If you need to create a custom module to access NetSuite records, read the following topics to learn how:

- [Define a JSON Object to Represent a NetSuite Record](#)
- [Create a Backend Model](#)
- [Create a Service to Handle HTTP Requests](#)

Define a JSON Object to Represent a NetSuite Record

Applies to: SuiteCommerce Advanced

To use the data in a NetSuite record, you must determine how to represent the record as a JSON object. For example, a custom record containing a set of questions and answers may have the following structure:

- **record name** —`customrecord_q_and_a`
- **custom field** —`custrecord_q_and_a_question`
- **custom field** —`custrecord_q_and_a_answer`

A JSON object representing this custom record would look like the following:

```
1 | {
2 |   "question": "A String with the question"
3 |   , "answer": "A String with the answer"
4 |   , "createdAt": "2016-03-31"
5 | }
```

This example only requires name/value pairs representing each custom field and its value. However, depending on the complexity of the record you need to represent, a JSON object can be more complex. For example, a JSON object can contain arrays or nested objects.

After defining the formal structure of a JSON object, you can define the backend model that accesses the NetSuite record. See [Create a Backend Model](#) for more information.

Create a Backend Model

Applies to: SuiteCommerce Advanced

Backend models handle data transactions with NetSuite using SuiteScript.

Note: The following code samples use SuiteScript 1.0 functions and objects only. These examples do not include SuiteScript 2.0 capabilities.

```
1 | define(
```

```

2 | 'QuestionsAndAnswers.Model'
3 | , ['SC.Model', 'Application']
4 | , function (SCModel, Application)
5 | {
6 |     'use strict';
7 |
8 |     return SCModel.extend({
9 |
10 |         name: 'QuestionsAndAnswers'
11 |
12 |     , validation: {
13 |         question: {required: true, msg: 'The question is required'}}
14 |     }
15 |
16 |     search: function(page)
17 |     {
18 |         var filters = [
19 |             new nlobjSearchFilter('custrecord_q_and_a_answer', null, 'isnot', '')
20 |         ]
21 |         , columns = [
22 |             new nlobjSearchColumn('custrecord_q_and_a_question')
23 |             , new nlobjSearchColumn('custrecord_q_and_a_answer')
24 |             , new nlobjSearchColumn('created')
25 |         ];
26 |
27 |         return Application.getPaginatedSearchResults({
28 |             record_type: 'customrecord_q_and_a'
29 |             , filters: filters
30 |             , columns: columns
31 |             , page: parseInt(page, 10) || 1
32 |         });
33 |     }
34 |
35 |     create: function(data)
36 |     {
37 |
38 |         this.validate(data);
39 |
40 |         var question = nlapiCreateRecord('customrecord_q_and_a');
41 |         question.setFieldValue('custrecord_q_and_a_question', data.question);
42 |         var question_id = nlapiSubmitRecord(question);
43 |
44 |         return data;
45 |     }
46 | });
47 | });

```

This example demonstrates the following tasks that are common to all backend models. The following sections provide more detail about each task:

- [Define Dependencies](#)
- [Extend from SC.Model](#)
- [Define the Validation Object](#)
- [Define Methods to Handle HTTP Actions](#)

Define Dependencies

Like frontend modules, backend models use RequireJS to define dependencies. See [Dependencies](#) for more information.

All backend models must include the following dependencies:

- **SC.Model** — defines the base class for backend models. All backend models should extend this module.
- **Application** — provides useful methods, including server-side search helpers.

Extend from SC.Model

All backend models must return an object that is extended from SC.Model.

```
1 | return SCModel.extend()
```

SC.Model defines the base model which defines core functionality, including validation.

Define the Validation Object

The validation object defines any validation requirements for the JSON object.

```
1 | validation: {
2 |     question: {required: true, msg: 'The question is required'}
3 | }
```

The Backbone.Validation module uses this object to perform validation. Any validation errors are returned to the frontend application through the service. The SC.Model module includes Backbone.Validation as a dependency.

In the above example, the `required` attribute indicates that the `question` property of the JSON object must have a value. If this property is undefined, the backend model returns the message specified by the `msg` property.

Define Methods to Handle HTTP Actions

A backend model must define a method to handle each of the HTTP actions supported by a service. These methods frequently use the SuiteScript API to handle data transactions with NetSuite. For example, the `search` method in the above example shows how a backend model retrieves data from a NetSuite record:

```
1 | search: function(page)
2 | {
3 |     var filters = [
4 |         new nlobjSearchFilter('custrecord_q_and_a_answer', null, 'isnot', '')
5 |     ]
6 |     , columns = [
7 |         new nlobjSearchColumn('custrecord_q_and_a_queston')
8 |         , new nlobjSearchColumn('custrecord_q_and_a_answer')
9 |         , new nlobjSearchColumn('created')
10 |    ];
11 |
12 |    return Application.getPaginatedSearchResults({
13 |        record_type: 'customrecord_q_and_a'
14 |        , filters: filters
15 |        , columns: columns
16 |        , page: parseInt(page, 10) || 1
17 |    });
18 | }
```

In this example, the `search` method performs the following:

- Creates an array containing search filters. Each search filter is defined by calling the `nlobjSearchFilter` method. Since this model is only retrieving data for one record, only one search filter is required.
- Creates an array for each column of the NetSuite record. In this example, each column corresponds to a property in the JSON object.
- Calls the `getPaginatedSearchResults` of the `Application` module. This method returns a JSON object based on the parameters it receives. This JSON object contains all of the `customrecord_q_and_a` records in NetSuite.

Create a Service to Handle HTTP Requests

Applies to: SuiteCommerce Advanced

Services handle HTTP requests sent by the frontend application to access backend models, which connect to NetSuite. Different releases of SuiteCommerce Advanced (SCA) handle services with significant differences. This section explains how to create a service when implementing the Vinson release of SCA or later.

- 2019.2 implementations and later provide two options: SuiteScript 2.0 or SuiteScript 1.0. How you go about this depends on the services you are creating. These implementations require Service Controllers.

Note: The following code samples use SuiteScript 1.0 functions and objects only. These examples do not include SuiteScript 2.0 capabilities.

- SCA implementations of Vinson through 2019.1 require only SuiteScript 1.0. These implementations require Service Controllers.

Note: If you are using custom services that pre-date the Vinson release of SCA, you can still use them without refactoring your code. SCA offers backward compatibility with any services of implementations prior to Vinson. See [Using Pre-Vinson Services with Vinson Release or Later](#) for more information.

Much of the logic required to operate a service is contained in a centralized location within the source files provided. However, you must complete the following steps to create a custom service:

- [Step 1: Create a Custom \[Module\].ServiceController](#)
- [Step 2: Set Up the Dev Tools](#)
- [Step 3: Set Up Custom Validations \(Optional\)](#)

Step 1: Create a Custom [Module].ServiceController

Each service requires a unique [Module].ServiceController. You create this file, where [Module] is the module using the service. To create this custom ServiceController, complete the following tasks:

- Create a new [Module].ServiceController file
- Define the custom [Module].ServiceController and dependencies
- Extend ServiceController
- Add custom Validation Options (Optional)
- Add HTTP methods (as required for the service)

Create a new [Module].ServiceController file

- Create a new file titled [Module].ServiceController.js, where [Module] is the module associated with the service.
- Place this service controller file in the module's SuiteScript directory.

Define the custom [Module].ServiceController and dependencies

- In your new [Module].ServiceController.js file, define the specific ServiceController and any dependencies.

2. Include ServiceController as a dependency. How you do this depends on your implementation.

The following code defines the Account Login ServiceController and enables the service to access methods defined in ServiceController and Account.Model.

```

1 // Account.Login.ServiceController.js - Service to handle the login of a user into the system
2 define(
3   'Account.Login.ServiceController'
4   , [
5     'ServiceController'
6     , 'Account.Model'
7   ]
8   , function(
9     ServiceController
10    , AccountModel
11  )

```

Extend ServiceController

1. [Module].ServiceController must extend ServiceController and include the name property. This property is used when listening to an event. The value of the name property should match the name of [Module].ServiceController.
2. Use the following example as a guide. The example shows this extension with the name property.

```

1 {
2   'use strict';
3   return ServiceController.extend({
4     name: 'Account.Login.ServiceController'

```

Add custom Validation Options (Optional)

1. If you need to define any validation options for the service, include these within return ServiceController.extend.
2. Use the following example as a guide. This example defines a requirement for the HTTP protocol to be secure before operating this service.

```

1 , options: {
2   common: {
3     requireSecure: true
4   }
5 }

```

Add HTTP methods

1. Within the object returned, define the functions required to handle the HTTP methods for the service. This code reads the HTTP method sent from the frontend (POST, DELETE, GET or PUT). For your [Module].ServiceController to function, you must declare each method in lowercase and it must match the name of the HTTP method being used.
2. Use the following example as a guide. This example defines what the service does on a POST request.

```

1 , post: function()
2 {
3   return AccountModel.login(this.data.email, this.data.password, this.data.redirect)
4 }

```

Using these examples, your custom [Module].ServiceController might look something like this:

```

1 // Account.Login.ServiceController.js - Service to handle the login of a user into the system
2 define(
3   'Account.Login.ServiceController'
4   , [
5     'ServiceController'
6     , 'Account.Model'
7   ]
8   , function(
9     ServiceController
10    , AccountModel
11  )
12  {
13    'use strict';
14    return ServiceController.extend({
15      name: 'Account.Login.ServiceController'
16      , options: {
17        common: {
18          requireSecure: true
19        }
20      }
21      , post: function()
22      {
23        return AccountModel.login(this.data.email, this.data.password, this.data.redirect)
24      }
25    });
26  }
27 );

```

Step 2: Set Up the Dev Tools

If implementing the Vinson release of SCA or later, the dev tools automatically generate the required .ss service file when you execute `gulp deploy`. However, you must also complete the following tasks to successfully deploy your new services:

- Include your new service controller in the `ssp_libraries.js` file
- Auto-generate your `[Module].Service.ss` file

Include your new service controller in the `ssp_libraries.js` file

1. You must edit the `distro.json` file to include your new `[Module].ServiceController` in the `ssp_libraries.js` file when you deploy your site.
Open the `distro.json` file in the root directory of the SCA source code.
2. Create a new dependency within the `Ssp-libraries` object for your new custom Service Controller. For example, `Account.Login.ServiceController`:

```

1 "ssp-libraries": {
2   "entryPoint": "SCA",
3   "dependencies": [
4     "Account.Login.ServiceController",
5     ...
6   ],
7 }
8 }

```

3. Save the `distro.json` file.

Auto-generate your `[Module].Service.ss` file

1. You must edit the `ns.package.json` file of the module associated with the service. This file signals the dev tools to auto-generate your `[Module].Service.ss` file and associate it with a specific service controller.

Open the `ns.package.json` file located in the module containing your new, custom Service Controller.

2. In the autogenerated-services object, define the new `.ss` file and associate it with the appropriate Service Controller.

```
1 | , "autogenerated-services":
2 | {
3 |   "Account.Login.Service.ss" : "Account.Login.ServiceController"
4 | }
```



Note: The autogenerated-services object may not exist in your implementation. If not, create the object and add the `.ss` files and Service Controller associations. Each `.ss` file must associate with one Service Controller.

3. Save the `ns.package.json` file.
4. Use the dev tools to deploy your services to your site.

The dev tools will create `Account.Login.Service.ss` and associate it with `Account.Login.ServiceController.js`.

Step 3: Set Up Custom Validations (Optional)

As an option, you can extend `ServiceController.Validations` to create any custom validations for your services beyond those included by default. To set up custom validations, complete the following tasks:

- Create a custom Validations file and add custom validation logic.
- Edit the `distro.json` file to include this new file as a dependency.

The following procedures provide step-by-step instructions for these tasks:

- [Create a custom Validations file](#)
- [Set up the dependency in `distro.json`](#)

Create a custom Validations file

1. In the `SspLibraries` folder, create a new file titled `ServiceController.Extend.Validations.js`.
2. Define this file and extend `ServiceController.Validations`.
3. Add your custom validation logic.

The following example depicts a typical custom validations file. This file extends `ServiceController.Validations` and contains custom logic to check validations.

```
1 | define(
2 |   'ServiceController.Extend.Validations'
3 |   , [
4 |     'underscore'
5 |     , 'ServiceController.Validations'
6 |   ]
7 |   , function (
8 |     _
9 |     , ServiceControllerValidations
10 |   )
11 |   {
12 |
13 |     'use strict';
14 |
15 |   }
```

```

16     _._extend(ServiceControllerValidations, {
17         checkValidationGeneric: function()
18         {
19             // validation logic here...
20         }
21     });
22 });

```

4. Set up the options object in any [Module].ServiceController using these validations. For example:

```

1     , options: {
2         common: {
3             checkValidationGeneric: true
4         }
5     }

```

Set up the dependency in distro.json

1. Open the distro.json file in the root directory of the SCA source code.
2. Create a new dependency within the Ssp-libraries object for your new custom Service Controller. For example:

```

1     "ssp-libraries": {
2         "entryPoint": "SCA",
3         "dependencies": [
4             "ServiceController.Extend.Validations",
5             ...
6         ],
7     },
8 }

```

3. Save the distro.json file and deploy using the dev tools.

Create a Service to Handle HTTP Requests (Pre-Vinson)

i Applies to: SuiteCommerce Advanced

The following example shows the required components of a service in pre-Vinson implementations of SuiteCommerce Advanced (SCA).

```

1 function service (request)
2 {
3     'use strict';
4
5     var Application = require('Application');
6
7     try
8     {
9         var method = request.getMethod();
10
11         var QuestionsAndAnswersModel = require('QuestionsAndAnswers.Model');
12
13         switch (method)
14         {
15             case 'GET':
16
17                 var page = request.getParameter('page');
18                 , result = QuestionsAndAnswersModel.search(page);
19
20                 Application.sendContent(result,{'cache': response.CACHE_DURATION_LONG});
21                 break;
22
23             case 'POST':

```



```

24
25     var data = JSON.parse(request.getBody() || '{}')
26     ,   result = QuestionsAndAnswersModel.create(data)
27
28     Application.sendContent(result, {'status': 201});
29
30     break;
31
32     default:
33
34     Application.sendError(methodNotAllowedError);
35   }
36 }
37 catch (e)
38 {
39   Application.sendError(e);
40 }
41 }

```

To create a custom service for a pre-Vinson implementation of SCA, you must perform the following:

- Step 1: Create a Reference to the Application Module
- Step 2: Define a try/catch Block
- Step 3: Create a Reference to the Backend Model
- Step 4: Define a Switch Statement to Handle the HTTP Request

Step 1: Create a Reference to the Application Module

The following code enables the service to access methods defined in the Application module.

```

1 | var Application = require('Application');

```

The Application module contains HTTP methods that the service uses to send data to the frontend application and return any errors. This module is defined in the `ssp_libraries.js` file. See [The ssp_libraries.js File](#).

Note: Services in SCA do not include the HTTP response as a parameter. Services return a JSON object using the `sendContent` method defined in the Application module.

Step 2: Define a try/catch Block

The body of a service must be included within a try/catch block. This ensures that any errors that occur are handled correctly. These errors are passed back to the frontend application and are handled by the ErrorManagement module.

Step 3: Create a Reference to the Backend Model

```

1 | var QuestionsAndAnswersModel = require('QuestionsAndAnswers.Model');

```

This statement enables the service to call methods defined in the backend model.

Step 4: Define a Switch Statement to Handle the HTTP Request

The main task of a service is to specify how to handle the HTTP action specified by the request. This is generally performed by a switch statement which contains a case statement for each HTTP method supported by the service.

```

1 switch (method)
2 {
3     case 'GET':
4
5         var page = request.getParameter('page');
6         , result = QuestionsAndAnswersModel.search(page);
7
8         Application.sendContent(result, {'cache': response.CACHE_DURATION_LONG});
9
10        break;
11
12    case 'POST':
13
14        var data = JSON.parse(request.getBody() || '{}');
15        , result = ProductReview.create(data)
16
17        Application.sendContent(result, {'status': 201});
18
19        break;
20
21    default:
22
23        Application.sendError(methodNotAllowedError);

```

The switch statement also contains a default statement to handle errors related to the HTTP request. This statement returns an error which is sent back to the frontend application and handled by the ErrorManagement module.

Using Pre-Vinson Services with Vinson Release or Later

Applies to: SuiteCommerce Advanced

Services included with SuiteCommerce Advanced (SCA) prior to the Vinson release contain all the logic necessary to function with the Vinson release and later.

The Vinson release of SCA ports existing, unmodified services of previous releases. Therefore, any unmodified services using pre-Vinson architecture will function with Vinson without requiring any changes to the code. However, if you made any customizations to pre-Vinson services, you can still use your existing customizations with the Vinson release of SCA. This requires that you edit the appropriate `ns.package.json` file to use the existing service (.ss file).

With the Vinson release of SCA, each module's `ns.package.json` file contains the autogenerated-services object. This object includes a list of .ss files to automatically generate when you deploy your site plus their associated Service Controllers. The following example shows the `ns.package.json` file for the Account module:

```

1 {
2   "gulp": {
3     "javascript": [
4       "JavaScript/*.js"
5     ]
6   },
7   "ssp-libraries": [
8     "SuiteScript/*.js"
9   ],
10  "autogenerated-services":
11  {
12    "Account.ForgotPassword.Service.ss" : "Account.ForgotPassword.ServiceController",
13    "Account.Login.Service.ss" : "Account.Login.ServiceController",
14    "Account.Register.Service.ss" : "Account.Register.ServiceController",
15    "Account.RegisterAsGuest.Service.ss" : "Account.RegisterAsGuest.ServiceController",
16    "Account.ResetPassword.Service.ss" : "Account.ResetPassword.ServiceController"
17  }
18 }

```

If you have customized any services using pre-Vinson architecture, you can add code to the services array of the appropriate ns.package.json file, forcing the SCA developer tools to use existing .ss files of the same name instead of automatically generating new ones.

To use pre-Vinson services with Vinson release or later:

1. Open the ns.package.json file located in the module containing the service file you want to use.
2. Add the following code to the file:

```
1 | , "services": [  
2 |   "SuiteScript/*.ss"  
3 | ]
```



Note: If a custom service (.ss file) exists in the module that bears the same name as one listed in the autogenerated-services object, this code prevents the SCA developer tools from automatically generating that .ss file. The dev tools will display a warning, informing you that the service will not be automatically generated because of the custom service with the same name. This warning is displayed as information only and does not require any action.

3. Save the ns.package.json file.
4. Use the SCA developer tools to deploy your services to your site.

Set Up SCA Developer Tools

Applies to: SuiteCommerce Advanced

You should use the SCA developer tools only if you are:

- Customizing a site that implements the Kilimanjaro release of SCA or earlier.
- Customizing a site that implements the Aconcagua release of SCA or later and need to access JavaScript, SuiteScript, or configuration objects that are not available using the Extensibility API.



Important: If you are implementing the Aconcagua Release of SCA or later, the best practice is to use themes and extensions to customize your site. To learn more about why you should use the theme and extension developer tools, see [Benefits of the Extension Framework](#).

The SCA developer tools enable you to customize the application from your local development environment and then deploy your customized application to NetSuite or a local environment for testing. The SCA developer tools are based on open-source software and are integrated within your existing development environment. These tools enable you to do the following:

- Store code locally within a version control system.
- Create and edit code locally in your preferred text editor or IDE.
- Compile the application locally.
- Deploy the application to NetSuite or a local server environment for testing. You can also deploy directly to your production or sandbox accounts in NetSuite.

Read the following instructions to learn how to set up the SCA developer tools:

- [Before You Get Started](#)
- [Download SCA Source Files](#)
- [Install Additional Files in the Source Directory](#)

Before You Get Started

Applies to: SuiteCommerce Advanced

Before you can set up and use the SCA developer tools, you need to complete these tasks:

Task	Description
Make sure the SCA SuiteApp is installed in your account.	SuiteApp installation is generally performed by your system or NetSuite administrator. This process installs all of the required application files in NetSuite. It also installs a downloadable zip file containing the source code for SCA.
Make sure your role enables you to deploy source files to NetSuite.	To use Gulp.js to deploy source files to NetSuite, you must use either the System Administrator role, the Store Manager role, or a custom role with the following permissions set to Full: ■ Documents and Files ■ Website (External) publisher ■ Web Services

Task	Description
Set up your developer environment.	To use the SCA developer tools, you first need to install the Node.js and Gulp.js utilities. For detailed instructions, see Set Up the Developer Environment .

After completing these tasks, you can continue setting up as described in [Download SCA Source Files](#).

Download SCA Source Files

 **Applies to:** SuiteCommerce Advanced

To begin locally editing and customizing SuiteCommerce Advanced (SCA), you must first download the source files from the NetSuite File Cabinet.



Important: If you are implementing the Aconcagua Release of SCA or later, the best practice is to use themes and extensions to customize your site. However, if your customizations require access to objects not available using the Extensibility API, you can use the SCA developer tools. You can run the SCA developer tools in parallel with the theme and extension developer tools.



Note: The following information and procedure are valid for SCA 2019.1 and later. If you have SCA release 2018.2 or earlier installed, refer to [Download SCA Source Files for Release 2018.2 and Earlier](#) for instructions specific to your release.

Your File Cabinet includes the following implementations:

- **Source** – This implementation contains all of the original installation files. It is locked to prevent changes. The Source implementation can be used for creating diffs against your customizations to troubleshoot issues.
- **Development** – The Development implementation is where you begin your work.
In the `_Sources` directory of the Development implementation, there is a zipped source file available. Download this file to your local development environment to begin working on your customizations and to create distribution files. When you install the SCA SuiteApp, appropriate default touch points are automatically deployed to this implementation. After deployment to a website, your site is available at the domain defined for that site. Refer to [Deploy SCA Customizations to NetSuite](#) for details.
- **Source2** – This implementation contains the original installation files for the SCA source modules that have been migrated to SuiteScript 2.0. For more information, see [Commerce Migration to SuiteScript 2.0](#).
- **Development2** – The Development2 implementation is where touch points for customizations to SuiteScript 2.0 modules are automatically deployed. For more information, see [Commerce Migration to SuiteScript 2.0](#).



Important: Unlike earlier releases, Aconcagua and later releases of SCA do not contain HTML and Sass files. So for Aconcagua and later releases of SCA, you must use the theme developer tools to customize the look and feel of your site. See [Develop Themes](#) for details. The theme developer tools can be used simultaneously with the SCA developer tools.

To download the SCA source files:

1. Log into your NetSuite account where SCA is installed.
2. Access the File Cabinet at Documents > Files > File Cabinet.

3. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > [SSP Application] > Development > _Sources.
The SSP Application for SCA is named: NetSuite Inc. - SCA <release_name>.
4. Click the name of the .zip file to download it.
5. Extract the .zip file to the directory where you want to store your source files.
Rename the top-level directory or accept the default name. However, do not change the names of subfolders, files, or their structure within this directory.
6. To complete your development environment set up, follow the procedure in [Install Additional Files in the Source Directory](#).

After completing these steps, you can begin editing or customizing SCA. If necessary, you can add these files to a version control system.

Note the following:

- The source files for SuiteCommerce Advanced 2019.1 and earlier are JavaScript files with .js filename extensions. The source files for SuiteCommerce Advanced 2019.2 and later are TypeScript files with .ts filename extensions. For more information about TypeScript, see [TypeScript](#).
- Generally, you only need to download the source files the first time you set up your local development environment. You may also need to download the source files to ensure you have the correct base version or the most recent version after a bundle update.

Download SCA Source Files for Release 2018.2 and Earlier

The following information and procedure are valid for SuiteCommerce Advanced (SCA) release 2018.2 and earlier.

Your File Cabinet includes the following implementations:

- **Source** – This implementation contains all of the original installation files. It is locked to prevent changes. In the _Sources directory of the Source implementation, there is a zipped source file available. Download this file to your local development environment to begin working on your customizations and to create distribution files. The Source implementation can also be used in the future for creating diffs against your customizations to troubleshoot issues.
- **Development** – The Development implementation is where you deploy your customizations. When you install the SCA SuiteApp, appropriate default touch points are automatically deployed to this implementation. Refer to [Deploy SCA Customizations to NetSuite](#) for details.



Important: Unlike earlier releases, Aconcagua and later releases of SCA do not contain HTML and Sass files. So for Aconcagua and later releases of SCA, you must use the theme developer tools to customize the look and feel of your site. See [Develop Themes](#) for details. The theme developer tools can be used simultaneously with the SCA developer tools.

To download SCA source files for release 2018.2 and earlier:

1. Log into your NetSuite account where SCA is installed.
2. Access the File Cabinet at Documents > Files > File Cabinet.
3. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > [SSP Application] > Source > _Sources.
The SSP Application for SCA is named: NetSuite Inc. - SCA <release_name>.
4. Click the name of the .zip file to download it.

5. Extract the .zip file to the directory where you want to store your source files.
Rename the top-level directory or accept the default name. However, do not change the names of subfolders, files, or their structure within this directory.
6. Install additional files to complete your development environment set up as described in [Install Additional Files in the Source Directory](#).

After completing these steps, you can begin editing or customizing SCA. If necessary, you can add these files to a version control system.

Install Additional Files in the Source Directory

Applies to: SuiteCommerce Advanced



Important: These topics refer to customizations of the Kilimanjaro release of SuiteCommerce Advanced (SCA) or earlier. If you are implementing the Aconcagua Release of SCA or later, the best practice is to use themes and extensions to customize your site unless your customizations require access to objects not available using the Extensibility API.

After downloading and extracting the SCA source directory, complete the following steps to finish installing the developer tools.



Note: You must run all commands in the SCA directory where the package.json file is located.

- If implementing the 2020.1 release or later, this directory is the SC_xxx directory within your root source directory. For example: SC_20.1
- If implementing the 2019.2 release, this directory is the *_Live directory within your root source directory. For example: SC_19.2_Live
- If implementing the 2019.1 release or earlier, this directory is the top-level root directory. For example: SuiteCommerce Advanced <release_name>

To install additional required files:

1. Open a command line or terminal window.
2. Access the local SCA source directory that contains the package.json file.
3. Install additional Node.js packages by running the following command:

```
npm install
```

This command may take several minutes to install the dependencies required by the developer tools.
4. Run Gulp.js using the following command:

```
gulp
```

Running Gulp.js creates the local distribution directory and compiles SCA source modules into a deployable application.

Note the following:

- If you encounter Sass Compilation Errors when running Gulp.js, you need to manually correct the compilation errors. For more information, see [Troubleshooting the Developer Tools](#).
- The first time you run the gulp command, the developer tools create a local distribution folder that contains the combined SCA application. After the command completes successfully, you see the new folder at the following location:
 - If implementing the 2020.1 release or later, the LocalDistributionAdvanced folder is created in the SC_xxx directory within your root source directory. For example: SC_20.1/LocalDistributionAdvanced
 - If implementing the 2019.2 release, the LocalDistributionAdvanced folder is created in the *_Live directory. For example: SC_19.2_Live/LocalDistributionAdvanced
 - If implementing the 2019.1 release or earlier, the LocalDistribution folder is created at the top level of the SCA source directory. For example: SuiteCommerce Advanced <release_name>/LocalDistribution
- Each time you download a new version of the SCA source files, you must perform this procedure in each new source directory.
- If you have multiple copies of the SCA source directory in your development or test environment, you must run this procedure in each of those directories.
- If you notice any NPM vulnerabilities related to an NPM audit, these vulnerabilities will not impact source code in the SCA application.

Example SCA Customizations

 **Applies to:** SuiteCommerce Advanced



Important: You should use the SCA developer tools only if you are customizing the Kilimanjaro release of SuiteCommerce Advanced (SCA) or earlier. If you are implementing the Aconcagua Release of SCA or later, the best practice is to use themes and extensions to customize your site unless your customization requires access to objects not available using the Extensibility API.

These examples show basic methods for extending SuiteCommerce Advanced (SCA) and creating your own custom modules.

Note the following information about the provided examples:

- The module source files for SCA 2019.1 and earlier are JavaScript files with .js filename extensions. The module source files for SCA 2019.2 and later are TypeScript files with .ts filename extensions. For more information about TypeScript, see [TypeScript](#).
- Sample scripts included in the customization examples use SuiteScript 1.0 functions and objects only. These examples do not include SuiteScript 2.0 capabilities.

To learn more about customizing SCA, review the following examples:

- [Create a Custom Module](#)
- [Modify JSON Configuration Files](#)
- [Extend Frontend Configuration Files](#)
- [Configure Facet Fields](#)
- [Extend the Backend Configuration File](#)
- [Add a Child View to a Composite View](#)
- [Override a Template File](#)
- [Extend a Sass File](#)
- [Add a Sticky Button](#)
- [Customizing the Loading Icon](#)
- [Adding a Custom Web Font](#)
- [Extending Font Awesome](#)
- [Displaying Device-Specific Carousel Images](#)

Create a Custom Module

 **Applies to:** SuiteCommerce Advanced



Important: This example applies to customizations of the Kilimanjaro release of SCA or earlier. If you are implementing the Aconcagua Release of SCA or later, the best practice is to use themes and extensions to customize your site unless your customizations require access to objects not available using the Extensibility API.

SuiteCommerce Advanced (SCA) is designed so that you can extend it by creating new modules to add functionality specific to your website. When creating a new module, be aware of the following notes and requirements:

- The module source files for SCA 2019.1 and earlier are JavaScript files with .js filename extensions. The module source files for SCA 2019.2 and later are TypeScript files with .ts filename extensions. For more information about TypeScript, see [TypeScript](#).
- The top level directory of your module should have a directory of the format: module_name@x.y.z where module_name is the name of the module and x.y.z is the version number. See the help topic [Organize Source Code for Custom Modules](#).
- The directory containing your custom module must reside in the correct directory, as follows:
 - For 2019.2 and later, SCA source code modules are stored in three sub-directories: Advanced, Backend, and Commons. Create another sub-directory at the same directory level to store all custom module code. Examples: SC_20.1/extensions, SC_19.2_Live/extensions.
 - For 2019.1 and earlier, the directory containing your custom module must be in the Modules directory. Modules shipped with SCA are stored in two subdirectories, suitecommerce and third_parties. Create a third subdirectory to store all custom module code. Example: Modules/extensions.
 - In this example, the directory where the custom module is stored is named extensions. However, you can name this directory any intuitive name as required.
- The directory for your new custom module must contain an ns.package.json file to define all of the module dependencies required by the developer tools. For more information, see [The distro.json and ns.package.json Files](#).
- The module name must be unique. You cannot have duplicate module names, even when they reside in different folders.
- You must update the distro.json file to ensure that your custom module is loaded into the application.

To add a custom module:

1. Create a custom module directory within your custom directory with the format of ModuleName@version.

Examples:

- If implementing 2020.1 and later, create this directory in the extensions directory. For example: SC_20.1/extensions/MyCustomModule@1.0.0.
- If implementing 2019.2, create this directory in the extensions directory. For example: SC_19.2_Live/extensions/MyCustomModule@1.0.0.
- If implementing 2019.1 and earlier, create this directory in the Modules/extensions directory. For example: Modules/extensions/MyCustomModule@1.0.0.



Important: You cannot use existing module names, even when those modules reside in different folders.

2. Create the subdirectories within the module.

The exact subdirectory structure depends on the requirements of your module and your implementation of SCA. You should use the same directory structure as existing application modules. Typically, you will need JavaScript, Sass, SuiteScript, and Templates subdirectories.

3. Create the necessary files for your module.

This is where all of your module's logic is contained. These files are either JavaScript (.js) or TypeScript (.ts), depending on your implementation of SCA. At a minimum, you will need something like the following:

Note: This procedure does not include examples of backend models or services files which are often necessary for backend integration with NetSuite.

- **Entry point** – this file acts as an entry point to the module and must return an object containing the `mountToApp` property that receives the application as a parameter.

JavaScript example:

```

1 //MyNewModule.js
2 define('MyNewModule'
3 , [
4   'MyNewModule.Router'
5 ]
6 , function (
7   Router
8 )
9 {
10   'use strict';
11
12   return {
13     mountToApp: function (application)
14     {
15       // Initializes the router
16       return new Router(application);
17     }
18   };
19 });

```

TypeScript example:

```

1 ///
2
3 import Router = require('./Router');
4
5 export function mountToApp(application)
6 {
7   new Router(application);
8 }

```

- **Router** – if you are on SuiteCommerce 2019.1 or later, use extensions or a page type component. If you are on SuiteCommerce 2018.2 or earlier, this JavaScript file extends the Backbone router to map URL patterns to the views defined within the module.

```

1 //MyNewModule.Router.js
2 define('MyNewModule.Router'
3 , [
4   'MyNewModule.View'
5   , 'Backbone'
6 ]
7 , function (
8   MyNewModuleView
9   , Backbone
10 )
11 {
12   'use strict';
13
14   //@class Address.Router @extend Backbone.Router
15   return Backbone.Router.extend({
16
17     routes: {
18       'my-new-module': 'MyNewModuleRouter'
19     }
20
21   , initialize: function (application)
22     {
23       this.application = application;

```

```

24     }
25
26     // list myNewRouter output
27     , MyNewModuleRouter: function ()
28     {
29         var view = new MyNewModuleView({
30             application: this.application
31         })
32         view.showContent();
33     }
34 });
35 });

```

- **View** – this view is called from the router. Often, a view contains a `getContext` function that describes all of the variables that are passed from the view to the template.

JavaScript example:

```

1 //MyNewModule.View.js
2 define(
3     'MyNewModule.View'
4     , [
5         'my_new_module.tpl'
6         , 'Backbone'
7         , 'jQuery'
8     ]
9     , function (
10         MyNewModuleTemplate
11         , Backbone
12         , jQuery
13     )
14     {
15         'use strict';
16
17         //@class Address.List.View List profile's addresses @extend Backbone.View
18         return Backbone.View.extend({
19
20             template: MyNewModuleTemplate
21
22             , events: {
23                 'click [data-action="test"]': 'testAction'
24             }
25
26             , testAction: function ()
27             {
28                 alert("This is a test action")
29             }
30
31             , getContext: function ()
32             {
33                 return {
34                     //@property {String} myNewModuleContextVar
35                     myNewModuleContextVar: 'myVariable'
36                 };
37             }
38         });
39 });

```

TypeScript example:

```

1 ///
2
3 import my_new_module.tpl = require('../../my_new_module.tpl');
4 import Backbone = require('../../Commons/Utilities/JavaScript/backbone.custom');
5 import jQuery = require('../../Commons/Utilities/JavaScript/jQuery');
6
7 //@class Address.List.View List profile's addresses @extend Backbone.View
8
9 let BackboneViewExtend: any = Backbone.View.extend({
10
11     template: MyNewModuleTemplate,

```

```

12
13     events: {
14         'click [data-action="test"]': 'testAction'
15     },
16
17     testAction: function ()
18     {
19         alert("This is a test action")
20     },
21
22     getContext: function ()
23     {
24         return {
25             //@property {String} myNewModuleContextVar
26             myNewModuleContextVar: 'myVariable'
27         };
28     }
29 });
30
31 export = BackboneViewExtend;

```

- **Template** – this file contains the markup used in combination with the views. Note that the SCA implementation leverages the Handlebars templating process.

```

1 | <h2>This is my template file for my new module. It takes variables, {{myNewModuleContextVar}}, passed from the View.</h2>

```



Important: Template file names must be unique. You cannot have duplicate template file names even when they belong to different modules.

4. If your custom module contains any configurable properties, you must include a Configuration subdirectory and place your Configuration Modifications within it. See [Modify JSON Configuration Files](#) for details.



Note: This step only applies to Vinson implementations of SCA and later.

5. Create a new **ns.package.json** file within the new custom module's directory.
 - If implementing 2020.1 and later, consider this example: SC_20.1/extensions/MyCustomModule@1.0.0/ns.package.json
 - If implementing 2019.2, consider this example: SC_19.2_Live/extensions/MyCustomModule@1.0.0/ns.package.json
 - If implementing 2019.1 and earlier, consider this example: Modules/extensions/MyCustomModule@1.0.0/ns.package.json

This file must contain entries for all of the directories required by the module. Use the following one of the following examples as a template.

This file must contain entries for all of the directories required by the module. This file is either JavaScript (.js) or TypeScript (.ts), depending on your implementation of SCA.

JavaScript example:

```

1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   , "templates": [
7 |     "Templates/*.tpl"
8 |   ]
9 |   , "ssp-libraries": [
10 |    "SuiteScript/*.js"
11 |   ]

```

```

12 |     , "services.new": [
13 |         "SuiteScript/*.Service.ss"
14 |     ]
15 |     , "sass": [
16 |         "Sass/**/*.scss"
17 |     ]
18 | }
19 | }

```

TypeScript example:

```

1 | {
2 |     "gulp": {
3 |         "javascript": [
4 |             "JavaScript/*.js",
5 |             "JavaScript/*.ts"
6 |         ]
7 |     , "templates": [
8 |         "Templates/*.tpl"
9 |     ]
10 |    , "ssp-libraries": [
11 |        "SuiteScript/*.js"
12 |    ]
13 |    , "services.new": [
14 |        "SuiteScript/*.Service.ss"
15 |    ]
16 |    , "sass": [
17 |        "Sass/**/*.scss"
18 |    ]
19 | }
20 | }

```

For Vinson implementations of SCA and later, you must also include the following lines within the gulp object of the ns.package.json file:

```

1 | , "configuration": [
2 |     "Configuration/*.json"
3 | ]

```

6. Update the distro.json file.

- If implementing 2019.2 and later, this file is in the Advanced directory. Examples: SC_20.1/Advanced/distro.json, SC_19.2_Live/Advanced/distro.json.
- If implementing 2019.1 and earlier, this file is in the root directory of the SCA source code.

Update this file with the following two changes:

- Add an entry for the new module in the modules object.
 - Example if implementing 2019.2 and later:

```

1 | {
2 |     "name": "SuiteCommerce Advanced",
3 |     "version": "2.0",
4 |     "modules": {
5 |         "../extensions/myNewModule": "1.0.0",
6 |         "../Commons/Address",
7 |         "../AjaxRequestKiller",
8 |         "../Commons/ApplicationSkeleton",
9 |         ...
10 |    }

```

- Example if implementing 2019.1 and earlier:

```

1 | {
2 |     "name": "SuiteCommerce Advanced 1.0.0",
3 |     "version": "1.0.0",
4 |     "modules": {

```

```

5 | "suitecommerce/Account": "1.0.0",
6 | "extensions/myNewModule": "1.0.0",
7 | "suitecommerce/AjaxRequestsKiller": "1.0.0",
8 | "suitecommerce/ApplicationSkeleton": "1.0.0",
9 | ...
10 | }

```

- Define any application dependencies in the javascript object.

For example, if the module is required in the Shop Flow application, add the module to the SC.Shopping.Starter entrypoint.

```

1 | "javascript": [
2 |   {
3 |     "entryPoint": "SC.Shopping.Starter",
4 |     "exportFile": "shopping.js",
5 |     "dependencies": [
6 |       "Backbone.View.Plugins",
7 |       "jQuery.html",
8 |       "ItemDetails",
9 |       "myNewModule",
10 |     ],
11 |     "UrlHelper",
12 |     "CMSadapter"
13 |   ],

```

7. View your changes.



Important: If you are on SuiteCommerce 2019.2 or later, all TypeScript (.ts) files are compiled to JavaScript (.js) files when executing the `gulp local` and `gulp deploy` commands.

After creating your new module, you can test it by viewing your changes in the application. If you are running a local server, you can view your changes by reloading your website. See the help topic [Test SCA Customizations on a Local Server](#) for more information.

If you are viewing your site in NetSuite, you can deploy your changes using the developer tools. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

Modify JSON Configuration Files

Applies to: SuiteCommerce Advanced

This example applies to Vinson implementations of SuiteCommerce Advanced (SCA) and later. For details about configuring SCA for Vinson and earlier releases, see the help topic [Extend JavaScript Configuration Files](#).

The following procedure explains how to use the Configuration Modification method to change existing configurable properties and associated metadata. When you deploy this modification to NetSuite, the SuiteCommerce Configuration record's user interface will reflect any changes specified. Configuration Modification requires knowledge of JSONPath query schema, see <https://github.com/s3u/JSONPath> for more information.



Important: Before you modify configuration files, you should be aware of these requirements and limitations:

- Never alter or delete the configurationManifest.json file. To make changes to property metadata, customize the individual JSON files only. When you deploy your site using the SCA developer tools, these individual files and customizations concatenate into and update the configurationManifest.json file. For detailed information about how JSON configuration files affect the configurationManifest.json and the SuiteCommerce Configuration record, see the help topic [Configure Properties](#).
- Making changes to core source files or changing vital functionality of the application can make migrating to future releases difficult. Before making changes to SCA, see the help topic [Best Practices for Customizing SCA](#).

To modify the SuiteCommerce Configuration record's user interface:

1. Create the directory structure for a custom module.

In this example, the name of the module being customized is the RecentlyViewedItems module. Following best practices, the example custom directory is named RecentlyViewedItemsExtension@1.0.0.

2. Create a Configuration subdirectory in your new module directory.
3. Create a new JSON file in the Configuration subdirectory. In this example, the name of the JSON file being customized is SearchResultsPerPage.json. Following best practices, the custom JSON file is named SearchResultsPerPageModification.json.
4. Add modification code to your custom JSON file.

See the help topics [Configuration Modification Schema](#) and [Use Case Examples](#) for specific information and examples on how to structure this file.

5. Create and edit an ns.package.json file in the root directory of your custom module.

Add the following code to this file:

```

1  {
2    "gulp": {
3      "configuration": [
4        "Configuration/*.json"
5      ]
6    }
7  }
```

6. Update the distro.json file.

- If implementing 2019.2 and later, this file is in the Advanced directory. Examples: SC_20.1/Advanced/distro.json, SC_19.2_Live/Advanced/distro.json.
- If implementing 2019.1 and earlier, this file is in the root directory of the SCA source code.

Add the name of your custom module to the list of modules defined in the modules object.

```

1  ...
2  "modules": {
3    "extensions/RecentlyViewedItemsExtension": "1.0.0",
4    "suitecommerce/Account": "2.2.0",
5    ...
6  }
```


7. Deploy your changes to NetSuite and access the SuiteCommerce Configuration record in NetSuite to view your changes. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.

Note: You must deploy your customizations to NetSuite using the SCA developer tools to apply modifications to the SuiteCommerce Configuration record. You can confirm changes to the configurationManifest.json code on a local server, but modifications will not take effect until you deploy to NetSuite.

Extend Frontend Configuration Files

Applies to: SuiteCommerce Advanced

This example applies to pre-Vinson implementations of SuiteCommerce Advanced (SCA) only. For details about configuring SCA for Vinson or later releases, see [Modify JSON Configuration Files](#).

Important: Making changes to core source files or changing vital functionality of the application can make migrating to future releases difficult. Before making changes to SCA, see the help topic [Best Practices for Customizing SCA](#).

SCA enables you to configure the behavior of the frontend application by modifying configuration properties and objects. These properties are:

- Stored in configuration files that define the objects that are loaded when the application starts.
- Accessible to all modules within the application.

See the help topic [Configure Properties](#) for more information about these files and the properties they contain.

To redefine these properties for your installation, you must create a custom module that includes the original configuration file as a dependency.

To extend a frontend configuration file:

1. Create the directory structure for your custom module.
In this example, the name of the custom module is Configuration, so the module directory would be Configurator@1.0.0.
2. Create the JavaScript subdirectory in your new module directory.
3. Create a new JavaScript file in the JavaScript directory of your custom module.
In this example, the name of the JavaScript file is Configurator.js
4. Add the mountToApp method to your custom JavaScript file.
For example, the code you add should look similar to:

```

1 | define('Configurator'
2 | , [
3 |   'SC.Configuration'
4 | ]
5 | , function (
6 |   Configuration
7 | )
8 | {
9 |   'use strict';
10 |   return {
11 |     mountToApp: function ()
12 |     {
13 |       //Add your custom properties here.
14 |     }
15 |   };

```

```
16 | });
```

This code performs the following tasks:

- Lists the dependencies required. When customizing a configuration file, you must include the object it returns as a dependency.
 - Defines the `mountToApp` method. This method is required to load your custom module into the application. This method also contains the custom properties you are configuring.
5. Add your custom properties to the block within the `mountToApp` method as shown in the example above.

The following example shows how to redefine different configuration properties:

```
1 | Configuration.imageNotAvailable = 'http://www.tnstate.edu/sociology/images/Image%20Not%20Available.jpg';
2 |
3 | Configuration.addToCartBehavior = 'goToCart';
4 |
5 | Configuration.facetDelimiters.betweenFacetNameAndValue = '%';
6 |
7 | Configuration.productReviews.loginRequired = true;
8 |
9 | Configuration.typeahead.maxResults = 4;
```

6. Create and edit the `ns.package.json` in the root directory of your custom module.

For example, the code you add should look similar to:

```
1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   }
7 | }
```

7. Update the `distro.json` file in the root directory of the SCA source directory. You must update this file in two places:
 - Add the name of your custom module to the list of modules defined in the `modules` object.
 - Add the name of your custom module to the dependencies of the application whose configuration file you are customizing. To ensure that your customized configuration properties are available to all modules, place your module at the top of this list.
8. View your changes.

If you are running a local server, you can view your changes by reloading your website. See the help topic [Test SCA Customizations on a Local Server](#) for more information.

If you are viewing your website in NetSuite, you can deploy your changes using the SCA developer tools. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

Configure Facet Fields

i Applies to: SuiteCommerce Advanced

This example applies to pre-Vinson implementations of SuiteCommerce Advanced (SCA) only. For details about configuring SCA for Vinson or later releases, see [Modify JSON Configuration Files](#).

You can customize facets by extending a frontend configuration file. Specifically, the `SC.Shopping.Configuration.js` file contains the facets, which contains an object for each facet field. You can customize these objects to determine how each associated facet field displays and behaves on your website.

To configure facet fields:

1. Configure how facet fields display on your site.

For example, suppose you want to configure the color facet field to display on your website in the following ways:

- Display the facet title as **Item Color**
- Appear expanded when loaded
- Be collapsible
- Appear first in the list of facets on your website

Your customized object might look like this:

```

1  , facets: [
2    {
3      id: 'color'
4      , name: _('Item Color').translate()
5      , priority: 1
6      , behavior: 'multi'
7      , template: facets_faceted_navigation_item_color_tpl
8      , collapsed: false
9      , uncollapsible: false
10     , showHeading: true
11     , colors: colors
12     , titleSeparator: ', '
13   }

```

2. Modify URL components for each facet field.

- a. Find the URL Component of the facet field you want to define.

- In NetSuite, go to Commerce > Marketing > SEO > URL Components for Facets.
- Note the URL Component of each facet you want to configure.

See the help topic [URL Components for Facets](#) for detailed information. After indexing Facet Fields in your Commerce site record, you can modify the URL components for each facet field.

- b. Create a custom module that includes the SC.Shopping.Configuration object as a dependency.

Note: Do not edit the original SC.Shopping.Configuration.js source file directly. See [Extend Frontend Configuration Files](#) for information and best practices on customizing JavaScript within frontend configuration files.

- c. Extend the facets , creating an object for each facet field you want to configure. Set the configuration properties as necessary.

See the help topic [Facets Subtab](#) for detailed information about each property.

3. Customize color palettes for facets.

The SC.Shopping.Configuration.js file contains the colors object. This object maps a color label to its hexadecimal value. You can customize these color values by extending the colors object.

- a. Extend the colors object within item options or facet object's colors property.
- b. Add any hexadecimal color values. The property's name must match the corresponding color in the colors object.

For example, the colors object lists default name/value pairs:

```

1  var colors = {
2    'black': '#212121'

```

```

3      , 'gray': '#9c9c9c'
4      , 'grey': '#9c9c9c'
5      , 'white': '#fff'
6      , 'brown': '#804d3b'
7      , 'beige': '#eedcbe'
8      , 'blue': '#0f5da3'
9      , 'light-blue': '#8fdeec'
10     , 'purple': '#9b4a97'
11     , 'lilac': '#ceadd0'
12     , 'red': '#f63440'
13     , 'pink': '#ffa5c1'
14     , 'orange': '#ff5200'
15     , 'peach': '#ffcc8c'
16     , 'yellow': '#ffde00'
17     , 'light-yellow': '#ffee7a'
18     , 'green': '#00af43'
19     , 'lime': '#c3d600'
20     , 'teal': '#00ab95'
21     , 'aqua': '#28e1c5'
22     , 'burgandy': '#9c0633'
23     , 'navy': '#002d5d'
24     }

```

For example, if you want to use a different color for blue and red in your site's faceted navigation, you can specify, #0000FF, for blue and, #f63440, for red. As a best practice, do not edit the default colors object. Instead, extend the object in the colors property of the facet as shown in the following example:

```

1      , {
2        id: 'color'
3        , name: _('Item Color').translate()
4        , priority: 1
5        , behavior: 'multi'
6        , template: facets_faceted_navigation_item_color_tpl
7        , collapsed: false
8        , uncollapsible: false
9        , showHeading: true
10       , colors: _.extend(colors, {'blue': '#0000FF', 'red': '#f63440'})
11       , titleSeparator: ', '
12     }

```

4. Configure facet delimiters.

The facetDelimiters object is defined in the global SC.Configuration.js file. This object specifies the characters within the URL that appear between facets, facet names, and their values. To configure facet delimiters:

- a. Create a custom module that includes the SC.Shopping.Configuration object as a dependency.

Note: Do not edit the original SC.Shopping.Configuration.js source file directly. See [Extend Frontend Configuration Files](#) for information and best practices on customizing JavaScript within frontend configuration files.

- b. Extend the facetDelimiters object and set the configuration properties as necessary. See the help topic [Facets Delimiters Subtab](#) for detailed information about each property.

Extend the Backend Configuration File

i Applies to: SuiteCommerce Advanced

This example applies to pre-Vinson implementations of SuiteCommerce Advanced (SCA) only. For details about configuring SCA for Vinson or later releases, see [Modify JSON Configuration Files](#).



Important: Making changes to core JavaScript source files or changing vital functionality of the application can make migrating to future releases difficult. Before making changes to SCA, see the help topic [Best Practices for Customizing SCA](#).

SCA uses a backend configuration file to modify the behavior of NetSuite features. See [Extend the Backend Configuration File](#) for more information.

During deployment, the backend configuration file is combined and deployed as a SuiteScript library in NetSuite. Like other JavaScript customizations, best practice is to create a custom module that redefines the configuration properties that you want to modify. However, since this file is deployed as a SuiteScript library, you only add your custom module to the `distro.json` file as a dependency. You do not need to define the `mountToApp` method.

To extend the backend configuration file:

1. Create the directory structure for your custom module.

In this example, the name of the custom module is `BackendConfigurator`, so the module directory would be `BackendConfigurator@1.0.0`.

2. Create the SuiteScript subdirectory in your new module directory.

Because the backend configuration file is a SuiteScript file, this directory is required.

3. Create a new JavaScript file in the SuiteScript directory of your custom module.

In this example, this file is named `BackendConfigurator.js`.

4. Add code to redefine the backend configuration properties you want to change.

For example, the code you add should look similar to:

```
1  define('BackendConfigurator'
2  , [
3    'Configuration'
4  ]
5  , function (
6    config
7  )
8  {
9    'use strict';
10   //disable CMS
11   config.useCMS = false;
12 });
```

This sample code performs the following:

- Lists the dependencies required by the custom module. When customizing the backend configuration file, the only required dependency is the `Configuration` object. This object is defined in the `Configuration.js` file located in the `Modules/Ssplibraries` directory of the SCA source directory.
 - Redefines a single configuration property. In this case, the custom module is setting the `useCMS` property to `false`.
5. Create and edit the `ns.package.json` file in the root directory of your custom module.

Your `ns.package.json` file should contain code similar to:

```
1  {
2    "gulp": {
3      "ssp-libraries": [
4        "SuiteScript/*.js"
5      ]
6    }
7  }
```

```

6 | }
7 | }

```

6. Edit the `distro.json` file.

You must add your custom module to the `distro.json` file for it to be loaded into the application. Because this module is extending an SSP library, you must add it in the following locations:

- Add the name of your custom module to the `modules` . To ensure that your customizations are not overwritten by other modules, add the custom module to the top of the .
- Add the name of your custom module to the dependencies of the `ssp-libraries` object.

7. View your changes.

Because the backend configuration file is a SuiteScript file and stored as an SSP library, you must deploy your custom module directly to NetSuite to view your changes. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

Add a Child View to a Composite View

i Applies to: SuiteCommerce Advanced

Adding a child view to a parent view is a common way of extending the functionality of SuiteCommerce Advanced (SCA). For example, you can easily add a message to a page by adding a `GlobalViews.Message.View` view as a child view. Adding a child view requires making two types of changes to the SCA source code:

- Extend the `childViews` object of a view. Because this requires a change to a JavaScript or TypeScript source file, you should create a custom module that uses prototyping to add the child view.

The module source files for SCA 2019.1 and earlier are JavaScript files with `.js` filename extensions. The module source files for SCA 2019.2 and later are TypeScript files with `.ts` filename extensions. For more information about TypeScript, see [TypeScript](#).

- Override the template file. In addition to adding the view to the `childViews` object, you must also edit the HTML template to implement the child view in a page.

i Note: Because a template file can only be overridden one time, you may want to define the override in a different custom module created specifically for template overrides when making your own customizations. See [Override a Template File](#) for more information.

To extend the `childViews` object of a view:

1. Create the directory structure for your custom module.
 - a. Create a directory called `extensions`.
 - If implementing 2019.2 and later, create this directory in the Advanced directory. Examples: `SC_20.1/Advanced/extensions`, `SC_19.2_Live/Advanced/extensions`.
 - If implementing 2019.1 and earlier, create this directory in the Modules directory. For example: `Modules/extensions`
 - b. In the `extensions` directory, create a subdirectory called `HeaderExtension@1.0.0`.
 - c. In the `HeaderExtension` directory, create a subdirectory called `JavaScript`.
 - d. Also in the `HeaderExtension` directory, create another subdirectory called `Templates`.

When creating customizations, you should create a directory structure as described in [Organize Source Code for Custom Modules](#).

2. Create a new file.

- If implementing 2019.2 or later, name this file HeaderExtension.ts.
- If implementing 2019.1 or earlier, name this file HeaderExtension.js.

3. Define your custom module and dependencies by adding the following code to this file.

This code defines the dependencies required by your custom module. If implementing SuiteCommerce 2019.1 or earlier, see [Asynchronous Module Definitions \(AMD\) and RequireJS](#) for information about defining dependencies within a module. This module includes the following views as dependencies:

- Header.View – is required to extend the childViews object.
- GlobalViews.Message.View – is required to add a message view to the application header.

JavaScript example:

```

1 | define('HeaderExtension'
2 | , [
3 |     'underscore'
4 |     , 'Header.View'
5 |     , 'GlobalViews.Message.View'
6 | ]
7 | , function (
8 |     _
9 |     , HeaderView
10 |    , GlobalViewsMessageView
11 | )
12 | {
13 |     'use strict';
14 |
15 |     //Additional code goes here.
16 |
17 | });

```

TypeScript example:

```

1 | ///
2 |
3 | ///
4 |
5 | import * as _ from 'underscore';
6 | import HeaderView = require('../Header/JavaScript/Header.View');
7 | import GlobalViewsMessagesView = require('../GlobalViews.Messages.View');
8 |
9 |     //Additional code goes here.;

```

4. Add the mountToApp method to the Header.Extension.js file (or the Header.Extension.ts file) as shown in the following sample:

```

1 | return {
2 |
3 |     mountToApp: function (application)
4 |     {
5 |
6 |         HeaderView.prototype.childViews.HeaderExtension = function()
7 |         {
8 |             return new GlobalViewsMessageView({
9 |                 message: 'Hello World! - This is an Example of a GlobalMessageView!'
10 |                 , type: 'success'
11 |                 , closable: true
12 |             });
13 |         }
14 |     }

```

```
15 | }
```

This code performs the following:

- Specifies a return statement that returns the `mountToApp` method. This method is required to load a module into the application.
- Extends the `childViews` object of the `Header.View` module using JavaScript prototyping.

5. Copy the original template file to your custom module.

- a. Copy the `header.tpl` file to the `Templates` directory of your custom module.
 - If implementing 2019.2 and later, copy the `header.tpl` file from the `Advanced/Header/Templates` directory.
 - If implementing 2019.1 and earlier, copy the `header.tpl` file from the `Modules/suitecommerce/Header/Templates` directory.
- b. Edit the custom template file by adding the following HTML code:

```
1 | <div data-view="HeaderExtension"></div>
```

Add the HTML code at a place in the template where it will be displayed in the Header view. For example, you can add it directly above the `<div class="header-menu-cart">`.

6. Create the `ns.package.json` file

- a. Create a file called `ns.package.json` in the `HeaderExtension` directory.
- b. Add the following code to the `ns.package.json` file.
 - If implementing 2019.2 and later:

```
1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*"
5 |     ]
6 |   },
7 |   "templates": [
8 |     "Templates/*"
9 |   ],
10 |   "overrides": {
11 |     "../Header/Templates/header.tpl": "Templates/header.tpl"
12 |   }
13 | }
```

- If implementing 2019.1 and earlier:

```
1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*"
5 |     ]
6 |   },
7 |   "templates": [
8 |     "Templates/*"
9 |   ],
10 |   "overrides": {
11 |     "suitecommerce/Header@1.1.0/Templates/header.tpl": "Templates/header.tpl"
12 |   }
13 | }
```

7. Add an entry for your module to the `distro.json` file.

- If implementing 2019.2 and later, the `distro.json` file is located in the `Advanced` directory of the SCA source code. Examples: `SC_20.1/Advanced/distro.json`, `SC_19.2_Live/Advanced/distro.json`

- If implementing 2019.1 and earlier, the `distro.json` file is located in the root directory of the SCA source code.

You must add your custom view to the `javascript` object within the `distro.json` file. Your code should look similar to:

```

1  "javascript": [
2    {
3      "entryPoint": "SC.Shopping.Starter",
4      "exportFile": "shopping.js",
5      "dependencies": [
6        "Header.Extension.View",
7        "Backbone.View.Plugins",
8        "jQuery.html",
9        "ItemDetails",
10       ...

```

In this example, you are only customizing a single file, so only add the `Header.Extension` module to the `javascript` object. In cases where you are customizing or overriding an entire application module, you may need to add the application module name here.

8. View your changes.

If you are running a local server, you can view your changes by reloading your website. See the help topic [Test SCA Customizations on a Local Server](#) for more information.

If you are viewing your site in NetSuite, you can deploy your changes using the SCA developer tools. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

Override a Template File

 **Applies to:** SuiteCommerce Advanced

Define a Template File Override



Important: Making changes to core JavaScript source files or changing vital functionality of the application can make migrating to future releases difficult. Before making changes to SuiteCommerce Advanced (SCA), see the help topic [Best Practices for Customizing SCA](#).

This example describes how to override a template file if you are on the Kilimanjaro release of SCA or earlier. If implementing the Aconcagua release of SCA or later, the best practice is to use themes and extensions to customize your website. See [Develop Themes](#) and [Develop Extensions](#) for more information.

When customizing SCA, you may need to change the HTML code of the application. To add, remove, or change the HTML, you must customize the template file for the module that corresponds to the feature you want to change. To customize a template, you must use the override method provided by the SCA developer tools. Because there is no mechanism for overriding or extending a specific part of a template, you must override the entire file.

This example describes how to override a template file to add additional text to the product description page. You can use this example as a guide when customizing your own templates.

To override a template:

1. Create a directory to store your template. Where you place this directory depends on your implementation.
 - If implementing 2019.2 and later, create this directory within the SCA source code root directory. Examples: SC_20.1/extensions, SC_19.2_Live/extensions.
 - If implementing 2019.1 and earlier, create this directory within the Modules directory. For example: Modules/extensions.
 - a. In the extensions directory, create a subdirectory called ItemDetailsExtension@1.0.0.
When creating a new custom module, the module name must be unique. You must not have duplicate module names, even if those modules reside in different folders.
 - b. In the ItemDetailsExtension@1.0.0 directory, create a subdirectory called Templates.
 - c. If required, create another subdirectory called Sass.

You should create a directory structure similar to that described in [Organize Source Code for Custom Modules](#).

2. Copy the original template file to your custom directory. The string x.y.z corresponds to the version of the ItemDetails module in your version of SCA.
 - If implementing 2019.2 and later, copy the item_detail.tpl file from /ItemDetails@x.y.z/Templates to the Templates directory of your custom application module.
 - If implementing 2019.1 and earlier, copy the item_detail.tpl file from Modules/suite_commerce/ItemDetails@x.y.z/Templates to the Templates directory of your custom application module.
3. Create the ns.package.json file.
You must create this file in the ItemDetailsExtension@1.0.0 directory.
4. Add the required code to the ns.package.json file.
 - If implementing 2019.2 and later, the contents of your ns.package.json file should be similar to:

```

1  {
2    "gulp": {
3      "templates": [
4        "Templates/*"
5      ]
6    },
7    "sass": [
8      "Sass/**/*.*scss"
9    ]
10   },
11   "overrides": {
12     "Advanced/ItemDetails@x.y.z/Templates/item_details.tpl": "Templates/item_details.tpl"
13   }

```

- If implementing 2019.1 and earlier, the contents of your ns.package.json file should be similar to:

```

1  {
2    "gulp": {
3      "templates": [
4        "Templates/*"
5      ]
6    },
7    "sass": [
8      "Sass/**/*.*scss"
9    ]
10   },
11   "overrides": {
12     "suitecommerce/ItemDetails@x.y.z/Templates/item_details.tpl": "Templates/item_details.tpl"
13   }

```

Note: You must replace the string x.y.z in the preceding code samples with the version of the ItemDetails module in your version of SCA. SuiteCommerce 2019.2 and later do not include the @x.y.z directory name suffix.

The first section of the preceding code samples defines the required objects used by gulp to include your customized template and Sass files when combining the application.

The second section contains the override directive, which maps the original file in the SCA source distribution directory to your custom template file. This mapping tells the gulp.js combiner to override a specific file.

5. Edit your custom template.

This procedure shows the basic steps required to modify a template. However, you can fully customize your templates as required. See [Commerce Sass Style Definitions](#) for more information.

1. Open the item_details.tpl file that you copied in a previous step.
2. In your editor, search for the string **Add to Cart**.
This string defines the label for the add to cart button.
3. Replace this string with the string 'Add to Your Cart.'

6. Create a custom Sass file, if required.

See [Extend a Sass File](#) for more information.

7. Update the distro.json file.

You must add your custom module to the distro.json file to ensure that the gulp tasks include your custom template. If you are customizing Sass files, you must also add a reference to them in the appropriate location in the distro.json file. See [Extend a Sass File](#) for more information.

In the following code samples, the ItemDetailsExtension module is added at the beginning of the list of modules. However, you can add the module anywhere in the modules object. If a module contains only customized template or Sass files, the order of precedence in this list does not matter.

- If implementing 2019.2 and later, add an entry for the new module in the list of modules in the modules object as shown in the following sample:

```

1 {
2   "name": "SuiteCommerce Advanced Live",
3   "version": "2.0",
4   "isSCA": true,
5   "buildToolsVersion": "Live",
6   "folders": {
7     "thirdPartyModules": "../third_parties",
8     "distribution": "../LocalDistribution",
9     "deploy": "../DeployDistribution"
10  },
11  "modules": {
12    "../extensions/ItemDetailsExtension": "1.0.0",
13    "../Commons/Address",
14    "../Commons/AjaxRequestsKiller",
15    ...

```

- If implementing 2019.1 and earlier, add an entry for the new module in the list of modules in the modules object as shown in the following sample:

```

1 {
2   "name": "SuiteCommerce Advanced Mont Blanc",
3   "version": "2.0",
4   "buildToolsVersion": "1.1.0",
5   "folders": {
6     "modules": "Modules",

```

```

7      "suitecommerceModules": "Modules/suitecommerce",
8      "thirdPartyModules": "Modules/third_parties",
9      "distribution": "LocalDistribution",
10     "deploy": "DeployDistribution"
11   },
12   "modules": {
13     "extensions/ItemDetailsExtension": "1.0.0",
14     "suitecommerce/Account": "2.1.0",
15     ...

```

8. View your changes.

If you are running a local server, you can view your changes by reloading your website. See the help topic [Test SCA Customizations on a Local Server](#) for more information.

If you are viewing your site in NetSuite, you can deploy your changes using the developer tools. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

When viewing your changes, you should see new button label and the item details and item image should be reversed.

Extend a Sass File

 **Applies to:** SuiteCommerce Advanced

 [Extend CSS in SCA](#)



Important: Making changes to core JavaScript source files or changing vital functionality of the application can make migrating to future releases difficult. Before making changes to SuiteCommerce Advanced (SCA), see the help topic [Best Practices for Customizing SCA](#).

This topic describes how to extend a Sass file if you are on the Kilimanjaro release of SCA or earlier. If implementing the Aconcagua release of SCA or later, the best practice is to use themes and extensions to customize your website. See [Develop Themes](#) and [Develop Extensions](#) for more information.

SCA enables you to customize your website to easily apply global style changes while supporting migration to later versions. When customizing styles, you create a custom module that overrides specific Sass variables already defined in the BaseSassStyles module. You then redefine application dependencies so that base Sass styles import your customizations in the correct order.

This example shows how to create a custom module to change the default background and foreground colors.

To extend a Sass file:

1. Create the directory structure to store your custom module.
 - a. Create a directory called extensions. Where you place this depends on your implementation.
 - If implementing 2019.2 and later, create the extensions directory within the SC_xxx directory within your root source directory. Examples: SC_20.1/extensions, SC_19.2_Live/extensions.
 - If implementing 2019.1 and earlier, create the extensions directory within the Modules directory. For example: Modules/extensions.
 - b. In the extensions directory, create a subdirectory called CustomSass@1.0.0.
When creating a new custom module, the module name must be unique. You must not have duplicate module names, even if those modules reside in different folders.
 - c. In the CustomSass@1.0.0 directory, create a subdirectory called Sass.

In general, when creating custom modules, you should create a directory structure similar to that described in the procedure above. See the help topic [Organize Source Code for Custom Modules](#) for more information.

2. Create a Sass file.

- a. In the Sass directory, create a new file called `_custom-sass.scss`.
- b. Add Sass variable definitions to this file.

These variable definitions override the base Sass variables defined in the `BaseSassStyles` module. See [Design Hierarchy](#) for more information.

```

1 | $sc-color-primary: #0000ff; // originally #f15c28;
2 | $sc-color-secondary: #00ff00; //originally #5B7F8C;
3 | $sc-color-theme: #00ff00; //originally #5B7F8C;
4 | $sc-color-link: #00ff00; //originally #2f9ac3;
5 | $sc-color-theme-light: #00aa00; // originally #9cb6bf
6 | $sc-color-theme-background: #000000;
7 |
8 | $sc-color-dark-copy: #e7d13b; // originally #1f2223;
9 | $sc-color-copy: #ede39f; //originally #404040;
10 | body {
11 |     background-color: $sc-color-theme-background;
12 | }
```

3. Create the `ns.package.json` file.

- a. Create a file called `ns.package.json` in the `CustomSass@1.0.0` directory.
- b. Add the following code to the `ns.package.json` file.

```

1 | {
2 |   "gulp": {
3 |     "sass": [
4 |       "Sass/**/*.scss"
5 |     ]
6 |   }
7 | }
```

4. Update the `distro.json` file.

- a. Add an entry for your `CustomSass@1.0.0` module to the `distro.json` file.
 - If implementing 2019.2 and later, the `distro.json` file is located in the `Advanced` directory of the SCA source code. Examples: `SC_20.1/Advanced/distro.json`, `SC_19.2_Live/Advanced/distro.json`

```

1 | {
2 |   "name": "SuiteCommerce Advanced Live",
3 |   "version": "2.0",
4 |   "isSCA": true,
5 |   "buildToolsVersion": "Live",
6 |   "folders": {
7 |     "thirdPartyModules": "../third_parties",
8 |     "distribution": "../LocalDistribution",
9 |     "deploy": "../DeployDistribution"
10 |   },
11 |   "modules": {
12 |     "extensions/CustomSass": "1.0.0",
13 |     "Account",
14 |     "Balance",
15 |     ...

```

- If implementing 2019.1 and earlier, the `distro.json` file is located in the root directory of your SCA source code.

```

1 | {

```

```

2  "name": "SuiteCommerce Advanced Mont Blanc",
3  "version": "2.0",
4  "buildToolsVersion": "1.1.0",
5  "folders": {
6    "modules": "Modules",
7    "suitecommerceModules": "Modules/suitecommerce",
8    "thirdPartyModules": "Modules/third_parties",
9    "distribution": "LocalDistribution",
10   "deploy": "DeployDistribution"
11 },
12 "modules": {
13   "extensions/CustomSass": "1.0.0",
14   "suitecommerce/Account": "2.1.0",
15   ...

```

- b. Split the BaseSassStyle entry in the sass object and add your custom module between them. This ensures that the Sass variables are loaded in the correct order as follows:

1. The main-variables Sass file.
2. The custom Sass variables defined in the CustomSass module.
3. Additional Sass files of the BaseSassStyles module. Including these last ensures that the additional Sass files import the custom variable definitions.

```

1  {
2    "module": "BaseSassStyles",
3    "include": [
4      "main-variables"
5    ]
6  },
7
8  "CustomSass",
9
10 {
11   "module": "BaseSassStyles",
12   "include": [
13     "main-atoms",
14     "main-molecules",
15     "bootstrap-overrides"
16   ]
17 }
18
19 },

```



Note: You must define these dependencies for each application whose Sass variable you want to change.

5. View your changes.

If you are running a local server, you can view your changes by reloading your website. See the help topic [Test SCA Customizations on a Local Server](#) for more information.

If you are viewing your site in NetSuite, you can deploy your changes using the developer tools. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

Add a Sticky Button

Applies to: SuiteCommerce Advanced

Sticky buttons are anchor elements that remain at a fixed location on a page even as a user scrolls through it. This feature is particularly useful on pages that have a lot of content placed vertically and require a strategically placed, call-to-action button that directs users to the next page.

An example of this is the **Proceed to Checkout** button that is displayed as soon as an item is added to the shopping cart and remains there as the user scrolls through the page to look for other items to add to the cart.

The **Proceed to Checkout** button remains in place even as the user is scrolling the page and viewing other items.

To add this type of interaction, you can implement the use of the `jQueryExtras/jquery.scStickyButton.js` plugin - an additional module provided by Oracle NetSuite. The module is available as part of the SuiteCommerce Advanced source code. Download the file and extract `jquery.scStickyButton.js` to the JavaScript subdirectory of your custom module: `Workspace/<extension_name>/Modules/<module_name>/JavaScript`. For example, `Workspace/MyExtension/Modules/Main/JavaScript`.

Note: The `jQueryExtras/jquery.scStickyButton.js` plugin is used only when the user's screen width is less than 768 pixels. On larger screens, the buttons remains stationary at its original position on the page.

To add a sticky button to your web page:

1. Update the view
 - a. Add `jquery.scStickyButton.js` as a dependency to your view:

```
1 define('MyCompany.MyExtension.MyModule.View',
2 [
3     'mycompany_myextension_mymodule.tpl',
4     'MyCompany.MyExtension.MyModule.SS2Model',
5     'Backbone',
6     'jquery.scStickyButton'
7 ],
8 function (
9     mycompany_myextension_mymodule_tpl,
10    MyModuleSS2Model,
11    Backbone,
12    jqueryscStickButton
13 ) {
14     // ...
15 }
16 );
```

- b. Create a listener to initialize the plugin where the main layout view is rendered by defining the `showContent()` method as follows:

```
1 showContent: function showContent () {
2     var self = this;
3     this.application.getLayout().showContent(this).done(function () {
4         self.initPlugins();
5     });
6 }
```

- c. Define the `initPlugins()` method to make a button element sticky. This method uses jQuery to search the view's rendered HTML for elements with the `sticky` attribute and calls `scStickyButton()` on those elements. The `scStickyButton()` method is the initialization method in `jquery.scStickyButton` that converts elements into sticky buttons. Define the `initPlugins()` method as follows:

```
1 initPlugins: function initPlugins () {
2     this.$el.find('[data-action="sticky"]').scStickyButton();
3 }
```

2. To update the template:

- a. Define the element that you want to stick to the top of your view by adding a custom data attribute to an HTML element:

```
1 | <a class="some-button-class" data-action="sticky" href="some-path" data-hashtag="#">Sticky button!</a>
```

- b. The data-action="sticky" attribute identifies the button element as a sticky button. You can also add other attributes, if required.
3. Test the extension:
 - a. Log in to your local server and check that the sticky button is displayed at the top of the page as you scroll down.

Customizing the Loading Icon

Applies to: SuiteCommerce Advanced

The loading icon is displayed when data is sent to and from your site. On desktop devices, the default behavior tracks the cursor around the page and is hidden when the transfer is complete. On mobile devices, there is no cursor loading icon, therefore no loading icon is shown during loading.

This section outlines two ways you can customize the loading icon. You can replace the loading icon image without changing its behavior, or you can fully customize the user experience. For instance, you may want to add loading bars or a full-screen loading page.

To find out more, explore these topics:

- [Replacing the Loading Icon Image](#)
- [Replacing the Loading Icon with Custom Loading Behavior](#)
- [Adding Cursor Tracking](#)

Replacing the Loading Icon Image

Note: The following procedure does not disable the JavaScript associated with the loading icon.

This section outlines how to replace the loading icon image, without changing its existing behavior. The loading icon is an image asset included with your site's source code. To replace the loading icon image, you should change it in the source code with an alternative image that has the same name as the default. This is typically named, ajax-loader.gif. After you deploy your site's source code, the new loading icon appears on your site.

Depending on the type of site and what version you are using, you can find the source code in the following locations:

- For sites using the Kilimanjaro release of SuiteCommerce Advanced or earlier, the icon is found in the site's source code at Modules\suitecommerce\GlobalImages\images.
- For sites using the Aconcagua release of SuiteCommerce Advanced or later, the icon is found in the site's theme's source at assets\img.

Note: You should avoid directly replacing the loading icon image file in the File Cabinet because it will be overwritten the next time your site's source or theme's source is deployed to your account.

After deploying the new image, you should perform a cache invalidation request on the file location and clear your browser's cache. The loading icon will be served from one of the following file locations depending on which version of SuiteCommerce you use:

- [https://\[domain\]/scs/extensions/SC/Threads/3.1.0/img/ajax-loader.gif](https://[domain]/scs/extensions/SC/Threads/3.1.0/img/ajax-loader.gif)
- [https://\[domain\]/sca-dev-kilimanjaro/img/ajax-loader.gif](https://[domain]/sca-dev-kilimanjaro/img/ajax-loader.gif)

For more information about cache invalidation requests, see the help topic [Cache Invalidation](#).

Replacing the Loading Icon with Custom Loading Behavior



Important: As a developer you may want to refer to `jQuery.ajaxSetup` (2020.1 release of SuiteCommerce and SuiteCommerce Advanced or earlier) or `jQuery.Loader` (2020.2 release of SuiteCommerce and SuiteCommerce Advanced or later) when creating extensions or custom modules. However, to avoid negatively impacting your site's performance, you should not directly customize these modules.

By default all HTTP requests are tracked using `jQuery.ajax()`. This includes page loads, requests to the item search API, NetSuite services, and other customizations that use this method. Changing the loading icon's behavior involves unbinding the default event handlers and binding new ones. The code uses these event handlers to show and hide the icon.

When implementing custom loading behavior, keep the following principles in mind:

- Inject an element with the `loadingIndicator` ID that is hidden by default.
- Add event listeners to `ajaxStart()` and `ajaxStop()`, which determine when to show and hide the loading icon.

To replace the loading icon with custom loading behavior:

1. Run the following code in a module/extension entry point file to unbind the existing event handlers that are used to show and hide the existing loading icon. This requires an extension or custom module. Referencing the `loadingIndicatorShow` and `loadingIndicatorHide` functions in `jQuery's` `unbind()` method removes them from the `ajaxStart` and `ajaxStop` events.

```
1 | jQuery(document).unbind('ajaxStart', SC.loadingIndicatorShow);
2 | jQuery(document).unbind('ajaxStop', SC.loadingIndicatorHide);
3 | jQuery('#loadingIndicator').remove();
```

2. After the default event listeners are unbound, you can add your own event listeners for your custom loading behavior. Either create your own or use a third-party provider. Whichever you choose, you need to bind two callbacks to `jQuery's` Ajax events. Use the following code as an example. The exact structure or calls will depend on how your loading indicator's code works.

```
1 | define('CustomLoadingIndicator', [
2 |     'jQuery',
3 |     'LoadingIndicatorLibraryFile'
4 | ], function (CustomLoadingIndicator (
5 |     jQuery,
6 |     LoadingIndicatorLibraryFile
7 | ) {
8 |     'use strict';
9 |
10 |    // Dummy code to load third-party loading indicator file
11 |    LoadingIndicatorLibraryFile.mountToApp();
12 |
13 |    //Code to create new event listeners
14 |    jQuery(document)
15 |        .ajaxStart(LoadingIndicatorLibraryFile.loadingIndicatorShow())
16 |        .ajaxStop(LoadingIndicatorLibraryFile.loadingIndicatorHide());
17 |
18 |    });
```

Note: LoadingIndicatorLibraryFile, refers to an arbitrary file that contains the custom loading indicator code, such as that provided from a third-party. If you are writing your own code, ignore this and place the code into the entry point file.

3. View your changes.

If you are running a local server, you can view your changes by reloading your website. See the help topic [Test SCA Customizations on a Local Server](#) for more information.

If you are viewing your site in NetSuite, you can deploy your changes using the developer tools. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

Adding Cursor Tracking

If you decide to implement cursor tracking, you should optimize your code for performance by using Underscore's `throttle()` method. To avoid potential performance issues, include a function/callback, which fires one time every x number of milliseconds. If a function/callback is not included each movement of the mouse triggers your code.

In the following example, the event handler is triggered every 50 milliseconds.

```
1 jQuery(document.body).on({
2     mousemove: _.throttle(function (event) {
3         //Your cursor tracking code
4     }, 50);
5 });
```

To implement your own cursor tracking, you can bind a `mousemove` event handler to `document.body` using jQuery. For more information, see [jQuery API documentation](#). For an example of how to do this, see the `jQuery.ajaxSetup` or `jQuery.Loader` modules in your site's source code.

Adding a Custom Web Font

Applies to: SuiteCommerce Advanced

When customizing your Commerce website, you may want to add a custom web font to comply with corporate branding guidelines or to make your site stand out. The easiest and best way to apply a web font to your site is with the `@font-face` rule. This rule lets you define new font families and built-in provisions for different font file types, font weights, and font styles. After your web font is defined, you can use it in your styling declaration. The following `@font-face` example tells the application there's a new font family. It includes the name of the font, where the font is located, and to apply the font to all text within the page's body.

```
1 @font-face {
2     font-family: "My New Cool Font";
3     src: url("path/to/My-New-Cool-Font.woff");
4 }
5 body {
6     font-family: "My New Cool Font";
7 }
```

Best Practices for Adding a Custom Web Font

- You either need to own the font or have a font license. The specifics vary, so make sure you are compliant. Note that free fonts may have restrictions on how they can be used.

- A copy of the font is distributed to visitors to your site. If you are using a corporate font or another proprietary font, you may want to add a layer of protection to the font file. To do this, use a third-party service that is designed to deliver fonts remotely using JavaScript.
- Using a custom web font impacts site performance. It's an extra resource that must be loaded. It adds at least one extra HTTP request and additional CSS. Typically impact is minimal, however, it's important to remember small performance hits can quickly add up. When adding a custom web font, balance the aesthetic needs of the site with the impact it has on performance.
- You can host your font remotely or with your source code as part of your customization. When you have a copy of the font and make it available as part of your customization, you have control of what is being sent. If you are hosting your font remotely, see [Hosting a Font Remotely](#).
- The `@font-face` rule loads synchronously, meaning it blocks the page's loading/rendering from completing until it finishes loading. If the requested font file takes an unusual amount of time to load, it can block the text from rendering and block visitors from using your site. If you are concerned about performance, you can load the font asynchronously and apply it after the initial render using JavaScript.

To add a custom web font:

1. Store your custom web font. Where you store it depends on your implementation.

If you are using the Aconcagua release of SuiteCommerce or SuiteCommerce Advanced or later, store your font file in the assets folder of a theme or extension. Note that you can organize your assets folder with subfolders. For example, you may want to include a fonts subfolder in your assets folder. If you are adding subfolders, ensure you update the path in your code.

If you are using the Kilimanjaro release of SuiteCommerce or SuiteCommerce Advanced or earlier, create a fonts folder in a module's folder and store your font file there. For example, `Modules/Fon`ts.

2. Reference the path to your font file path. How you reference the path depends on your implementation. If you are hosting your font remotely, see [Hosting a Font Remotely](#).

If you are using the Aconcagua release of SuiteCommerce or SuiteCommerce Advanced or later, you should dynamically generate the URL using the available built-in helpers. Depending on whether you are referencing the assets folder of an extension or a theme, use `getExtensionAssetPath()` or `getThemeAssetPath()`.

Note: The built-in helpers are also available to use within your template files and `getExtensionAssetsPath()` is also available in your extension's JavaScript files. You can use them to reference other non-font asset files like services, images, and library JavaScript files.

Assuming you are referencing the assets folder of a theme, your code will look similar to this:

```
1 | @font-face {
2 |   font-family: "My New Cool Font";
3 |   src: url(getThemeAssetsPath("My-New-Cool-Font.woff"));
4 | }
```

If you are using the Kilimanjaro release of SuiteCommerce or SuiteCommerce Advanced or earlier, use standard syntax to refer to the path relatively. Assuming the font you are using is in a fonts subfolder in the Sass file you are currently editing, you would reference the file as follows:

```
1 | @font-face {
2 |   font-family: "My New Cool Font";
3 |   src: url("../Fonts/My-New-Cool-Font.woff");
4 | }
```

If you need to reference a font file in another module, use more instances of `../`.

```

1 | @font-face {
2 |     font-family: "My New Cool Font";
3 |     src: url("../MyOtherModule/Fonts/My-New-Cool-Font.woff");
4 | }

```

Update the module's `ns.package.json` file to include the fonts folder.

```

1 | {
2 |   "gulp": {
3 |     "fonts": [
4 |       "Fonts/*"
5 |     ],
6 |     "sass": [
7 |       "Sass/*.scss"
8 |     ]
9 |   }
10 | }

```

Note: Depending on how your code is organized, you may want to have a module to load your custom font. You must update the `distro.json` file to include a reference to the module in the modules object, and a reference to it in the relevant applications' CSS dependencies.

3. View your changes.

If you are running a local server, you can view your changes by reloading your website. See the help topic [Test SCA Customizations on a Local Server](#) for more information.

If you are viewing your site in NetSuite, you can deploy your changes using the developer tools. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

Hosting a Font Remotely

A common way of serving font files is from a third-party CDN that specializes in serving font files. If you are using a third-party server or service to deliver your web font, you need to use the `@font-face` rule in your Sass file, specifying the URL for the font file to the remote server.

```

1 | @font-face {
2 |     font-family: "My New Cool Font";
3 |     src: url("https://www.mycoolwebserver.com/My-New-Cool-Font.woff");
4 | }

```

However, depending on the type of third-party server or service you are using, you may not be able to link directly to your desired fonts. If this is the case, you can use `@import` to fetch stylesheets from the remote server or service.

```

1 | @import url("//fonts.thirdpartyserver.com/css?family=My+New+Cool+Font");

```

Extending Font Awesome

Applies to: SuiteCommerce Advanced

Font Awesome is a web font included with your Commerce site, which provides a library of icons that you can use on your site. Typically, icons are used as a substitute for text, as a reinforcement for text, and to

provide visual hints. For instance, you may want to include an icon with error messages to make it stand out. By default, Font Awesome includes a large library of icons, however it is possible to add your own.

To add your own icons, there are two approaches you can take:

- If you are delivering a customization through an extension, you can create a custom font that includes your new icons. How you create a new icon font depends on the third-party service you are using. After you have uploaded your icons and created your new font, follow the steps outlined in [Adding a Custom Web Font](#).
- If you are adding only a few icons, you can replace Font Awesome with your own version that includes your new icons. For more information, see [Modifying Font Awesome](#).

Note: If you are using the Aconcagua release of SuiteCommerce or SuiteCommerce Advanced or later, Font Awesome font files must be replaced. If you are using the Kilimanjaro release of SuiteCommerce or SuiteCommerce Advanced or earlier, Font Awesome files must be overridden.

Modifying Font Awesome

This section details how you can use the existing Font Awesome font as a basis for a new font, which includes your new icons.

Note: Before adding your own icons, check the [Font Awesome Library](#) to see if there is already an icon that applies. Be sure to check the version that applies to the version in your code.

To modify Font Awesome:

1. Create a folder for your new font called custom. The location of the folder depends on which version you are using.
 - If you are using the Kilimanjaro release of SuiteCommerce or SuiteCommerce Advanced or earlier, go to Modules/third_parties/font-awesome@x.x.x.
 - If you are using the Aconcagua release of SuiteCommerce or SuiteCommerce Advanced or later, in your site's theme's source code, go to assets/font-awesome.
2. Update the Sass file so the custom path points to the location of your new font files. By pointing to the location of your new font files, you avoid changing the structure of the code. This keeps the original font files in case you need them.

In your site's theme's source code under Modules > font-awesome > x.x.x-custom > scss/_path.scss, find the `@font-face` declaration for the font that points to `#{$fa-font-custom-path}`. The value for this variable is declared in `_variables.scss` and is defined as follows:

```
1 | $fa-font-path: getThemeAssetsPath("font-awesome");
2 | $fa-font-custom-path: $fa-font-path;
```

Update this path to point to your new font folder.

```
1 | $fa-font-path: getThemeAssetsPath("font-awesome/custom");
```

3. Go to the font generating service of your choice. Upload the existing SVG version of the Font Awesome font file you want to add your new icons to. For example, fontawesome-webfont.svg. The location of the font file depends on which version you are using.
 - If you are using the Kilimanjaro release of SuiteCommerce or SuiteCommerce Advanced or earlier, the font files are found in your source code under Modules/third_parties/font-awesome@x.x.x-custom.

- If you are using the Aconcagua release of SuiteCommerce or SuiteCommerce Advanced or later, the font files are found in your site's theme's source code under Workspace > assets > fontawesome.
- 4. Follow the steps of the font generating service to add your new icons to the file and generate your font.
- 5. Download your new font into your custom folder. Before downloading, keep the following in mind:
 - Change the default Unicode references assigned to your new icons. Each icon needs to be treated as its own character with an associated keystroke. Each variable has a name beginning with \$fa-var- and a value that begins with \f. It is important to maintain this consistency without overwriting existing unicode references.
 - If required, make sure the file names match the originals. If they do not match, you must update the file names in the @font-face declaration.
- 6. Create the underlying variables and the base class for your new icons.

You can directly edit `_variables.scss` and `_icons.scss`, or you can keep them separate. If you choose to keep your new icons separate, you may want to create a separate module or keep them with other site-specific customizations. Whichever way you choose, your new declarations will look similar to the example below.

```

1 //Variable declaration
2 $fa-var-brandlogo: "\f400";
3
4 //Icon declaration
5 #{ $fa-css-prefix }-brandlogo:before {content: $fa-var-brandlogo; }
6
7 //Base class declaration
8 .brand-logo-icon {
9   @extend .fa;
10  @extend %fa-brandlogo;
11 }
12
13 //Example instance within my code
14 .header-logo-icon {
15   @extend .brand-logo-icon;
16   font-size: 3em;
17   padding: $sc-small-margin 0;
18 }
19

```

- 7. View your changes.

If you are running a local server, you can view your changes by reloading your website. See the help topic [Test SCA Customizations on a Local Server](#) for more information.

If you are viewing your site in NetSuite, you can deploy your changes using the developer tools. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

Displaying Device-Specific Carousel Images

i Applies to: SuiteCommerce Advanced

Carousel images show a series of images that usually link to various categories or pages on your website. Depending on the customer, you may want to show different images based on what device they are using. The example customization in this section, demonstrates how to customize the home page carousel to show different images based on whether the user is using a mobile, tablet, or desktop device. The example customization includes utility functions built into SuiteCommerce that you can use to test the viewport width and the conditional statements that show or hide content based on the device.

Note: The example code in this section is delivered through the extension framework, which is available to sites running SuiteCommerce or the Aconcagua release of SuiteCommerce Advanced or later. However, the underlying mechanisms are available to the Kilimanjaro release of SuiteCommerce Advanced or earlier.

Note: The example in this section should be used as a guide only and should not be used as a direct template.

Mechanisms for Displaying Device-Specific Carousel Images

Before diving into the code, here are some key points to keep in mind.

- Built-in utility functions that perform device detection are available in the `Utils.js` module.
 - `getDeviceType(widthToCheck)`: If an integer value is specified for this method, the value you passed is evaluated. Otherwise, a string of either desktop, tablet, or mobile is returned depending on the current viewport width.
 - `isPhoneDevice()`, `isTabletDevice()`, `isDesktopDevice()`: Each of these methods runs the device type checker function, evaluates the string it returns, and then returns a value.
- When generating paths for new images, create two additional configuration objects and add them to the Configuration record. For details, see the help topic [SuiteScript 2.x Global Objects](#).
- To replace the existing image path so the correct image is displayed, use the `addToViewContextDefinition()` method, available in the extensibility API. This method is primarily used to add new properties to a view's `getContext()` object, but it can also be used to replace existing ones.

Using these mechanisms, you can create something that:

- Detects the user's device type.
- Gets the partial image paths from the Configuration record and processes them, so that the full URL paths required to load the images can be generated.
- Swaps out the property in the home view's context object, so that the correct image is displayed when the template is rendered.

To display device-specific carousel images:

1. Create the entry point file.

Use the `Example.DeviceSpecificCarousel.DeviceSpecificCarousel.js` file, shown below, as a guide. When the module loads, the following variables are declared:

- Two extensibility API components
- A flag that tests whether the user is using a mobile or tablet device
- A reference to `this` to call the new function `mapCarouselImages()`

```

1 | define('Example.DeviceSpecificCarousel.DeviceSpecificCarousel'
2 | , [
3 |   'Utils'
4 |   , 'underscore'
5 | ]
6 | , function
7 | (
8 |   Utils

```

```

9      , -
10    }
11  {
12    'use strict';
13
14    return {
15      mountToApp: function mountToApp (container)
16      {
17        var Layout = container.getComponent('Layout')
18        , Environment = container.getComponent('Environment')
19        , replaceCarouselImages = Utils.isPhoneDevice() || Utils.isTabletDevice()
20        , self = this;
21
22        //Determines which images you want to replace the existing images with.
23
24        if (Layout && Environment && replaceCarouselImages)
25        {
26          var carouselImages =
27            Utils.isPhoneDevice() ? Environment.getConfig('home.carouselImagesMobile')
28            : Utils.isTabletDevice() ? Environment.getConfig('home.carouselImagesTablet')
29            : [];
30
31          //Checks if the replacement carousel images have been provided. If they have been, the updated array is passed to the
          Home.View context object.
32
33          if (carouselImages.length)
34          {
35            Layout.addToViewContextDefinition('Home.View', 'carouselImages', 'array', function ()
36            {
37              return self.mapCarouselImages(carouselImages);
38            });
39          }
40        }
41      }
42
43      //A utility function used to generate the correct URLs for each banner image.
44
45      , mapCarouselImages: function mapCarouselImages (urlArray)
46      {
47        return _.map(urlArray, function (url)
48        {
49          return _.getAbsoluteUrlOfNonManagedResources(url)
50        });
51      }
52    }
53  });

```

2. Declare properties in the configuration file.

Use the Example.DeviceSpecificCarousel.DeviceSpecificCarousel.json file, shown below, as a guide. In this example, new properties are declared to add to an existing tab or group. Its structure mimics that of the configuration file of the existing carousel images.

```

1  {
2    "properties":
3    {
4      "home.carouselImagesMobile":
5      {
6        "group": "layout"
7        , "type": "array"
8        , "title": "Carousel Images Mobile"
9        , "description": "Carousel Image URLs for Mobile Devices"
10       , "items":
11       {
12         "type": "string"
13         , "title": "URL"
14       }
15       , "default": []
16     }
17     , "home.carouselImagesTablet":
18     {
19       "group": "layout"

```



```

20     , "type": "array"
21     , "title": "Carousel Images Tablet"
22     , "description": "Carousel Image URLs for Tablet Devices"
23     , "items":
24     {
25         "type": "string"
26         , "title": "URL"
27     }
28     , "default": []
29     }
30 }
31 }

```

After the above customization is deployed, the two subtabs are displayed in the Layout tab on the Configuration record.

3. Deploy your customization. For details, see the help topic [Deploy SCA Customizations to NetSuite](#).
4. Upload and add the images you want to use for tablet and mobile devices.
 - a. Go to Documents > Files > File Cabinet.
 - b. Select the **img** folder.
 - c. Click the **Add File** button and select your images.
5. Go to Commerce > Websites > Configuration.
6. Select the website and domain, and click **Configure**.
7. Select the **Layout** tab and add the images to their respective arrays. For example, **Layout > Carousel Images Tablet**, and **Layout > Carousel Images Mobile**.
8. Click **Save**.
9. Test your changes.

Resize your site's viewport and refresh the page. The different images are displayed if the viewport width is within of one of the device types.

Note: If you prefer not to use your own code for the carousel slider, you can use the SC Slideshow (Bundle ID 250352) or the SC Photo Gallery (Bundle ID 249422). For more information, see the help topic [Installing a Bundle](#).

SCA Build Process

i Applies to: SuiteCommerce Advanced

To understand how the Suite Commerce Advanced (SCA) developer tools work and why they are necessary, it is important to understand how the SCA source files are organized and how the application is compiled. Read the following topics to learn more:

- [The distro.json and ns.package.json Files](#)
- [How Files Are Combined](#)
- [Contents of the Deploy and Local Distribution Directories](#)

For details about the SCA source directory structure, see [SCA Source Directory](#).

The distro.json and ns.package.json Files

i Applies to: SuiteCommerce Advanced

To determine which files to include with the Distribution folder of SCA, Gulp.js uses two configuration files:

- `distro.json`
- `ns.package.json`

The `distro.json` file resides in the following location, depending on your implementation:

- If implementing the 2020.1 release and later, this file is located within the `SC_xxx/Advanced` directory within your root source directory. For example: `SC_20.1/Advanced`
- If implementing 2019.2, this file is located within the `*_Live/Advanced/` directory within your root source directory. For example: `SC_19.2_Live/Advanced/`
- If implementing 2019.1 and earlier, this file is located within the top level of the SCA directory.

The `distro.json` file defines the modules that are included in SCA. It also specifies additional configuration information.

Gulp.js parses this file to determine which files within the Modules directory to combine or copy into the Distribution folder. The `distro.json` file contains the following parameters:

- **name:** specifies the name of the application. By default this value is SuiteCommerce Advanced <version>.
- **version:** specifies the application version.
- **modules:** lists the modules SCA uses. Each module is listed on a separate line and uses the following format:

```
"<module_name>": "<module_version>"
```

By default the SCA modules are listed in alphabetical order because there are no dependencies between modules. However, when customizing SCA, your modules may have dependencies on other modules. When dependencies exist, you must ensure that any required modules are listed before other modules.

- **taskConfig:** defines the gulp tasks required by the build process. In general, you do not need to edit these tasks.

- **copy:** specifies the source and target paths when copying files to the Distribution directory.

Although the distro.json file defines the modules that are included in SCA, the ns.package.json file determines which files within an individual module are included. Each module included in the distro.json file must have a corresponding ns.package.json file. The ns.package.json file:

- Must be at the top level of the module folder
- Uses the structure shown in the following examples

Example 5. JavaScript Example

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5      ]
6    ,   "templates": [
7        "Templates/*.txt"
8      ]
9    ,   "ssp-libraries": [
10       "SuiteScript/*.js"
11     ]
12    ,   "services.new": [
13       "SuiteScript/*.Service.ss"
14     ]
15   }
16 }
```

Example 6. TypeScript Example

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5        "JavaScript/*.ts"
6      ]
7    ,   "templates": [
8        "Templates/*.txt"
9      ]
10   ,   "ssp-libraries": [
11       "SuiteScript/*.js"
12     ]
13   ,   "services.new": [
14       "SuiteScript/*.Service.ss"
15     ]
16   }
17 }
```

In the preceding JavaScript example, there is a mapping between file types and partial paths to the location of the files. In general, Gulp.js uses wildcards to specify the contents of a directory. For example, `JavaScript/*.js` includes all files with the `.js` extension that are in the JavaScript folder. However, when creating your own modules, you can point to specific files within a directory, for example:

```

1  ...
2  "javascript": [
3    "JavaScript/MyJavaScriptFile.js"
4  ]
5  ...
```

The types shown in the preceding examples (javascript, templates, services, etc.) correspond to a specific Gulp task. Gulp tasks determine how certain file types are handled. For example, the javascript Gulp task causes all JavaScript files within a distribution to be compiled into a single file. Other Gulp tasks are responsible for copying files from the Modules directory to the Distribution directory.

If you are using the 2019.2 release of SCA or later, many of the source files are written in TypeScript with .ts filename extensions. In this case, the ns.package.json file uses JavaScript/*.ts to include .ts files as JavaScript. For example:

```

1  ...
2  "javascript": [
3      "JavaScript/*.js",
4      "JavaScript/*.ts"
5  ]
6  ...

```

As a best practice, use only the default Gulp tasks when creating or editing a module. To use a custom file type within the ns.package.json file, you must create your own custom Gulp task.

Gulp tasks are stored in the following location:

SCA/gulp/tasks

Note: This directory is created when you install Gulp.js. It is not included in the downloaded version of the SCA source directory. See [Set Up the Developer Environment](#) for more information.

How Files Are Combined

Applies to: SuiteCommerce Advanced

The SuiteCommerce Advanced (SCA) source code is divided into multiple modules. These modules contain multiple subdirectories which, in turn, contain individual files, including JavaScript and Sass files. If you are implementing SCA 2019.2 or later, there are also TypeScript files which are identified with a .ts filename extension. The contents of each of these files contains a logical structure where each element is separated by white space.

Although this organization makes the source files easy to read and understand, it is inefficient when passing these files across a network to a web browser. Most modern web applications use some method of combining multiple files spread across multiple directories into a smaller number of combined files. These files are more efficient for a web browser to load because fewer requests are required and a smaller total number of bytes are transferred.

To combine these files, SCA uses Gulp.js to perform the following tasks:

- Determine which files need to be included in the web application.
- Combine all of the JavaScript, Sass, and SSP library files into single files.
- Copy any additional resources required by the application. For example, Gulp.js copies any image files, SSP files, or services.

All resources required by SCA, including combined and copied files, are stored in the **Distribution** directory. This directory is deployed to NetSuite. See [Contents of the Deploy and Local Distribution Directories](#) for information about the output of the gulp tasks.

Contents of the Deploy and Local Distribution Directories

Applies to: SuiteCommerce Advanced

Gulp commands automatically generate directories as follows:

- `gulp deploy`: creates one or more deploy distribution directories depending on your implementation.
 - If you are implementing SCA 2019.2 or later, this command creates the `DeployDistributionAdvanced` and `DeployDistributionAdvancedSS2` directories. The `DeployDistributionAdvanced` directory contains the combined source files that are deployed to NetSuite. The `DeployDistributionAdvancedSS2` directory contains the SuiteScript 2.0 services for deployment.
 - If you are implementing SCA 2019.1 or earlier, this command creates the `DeployDistribution` directory. This directory contains the combined source files that are deployed to NetSuite.
- `gulp local`: creates a local distribution directory.
 - If you are implementing SCA 2019.2 or later, this command creates the `LocalDistibutionAdvanced` directory. This directory contains files that are used by the local server.
 - If you are implementing SCA 2019.1 or earlier, this command creates the `LocalDistribution` directory. This directory contains files that are used by the local server.

Do not directly edit any of the files in the deploy or local distribution directories. However, you may need to view the contents of these directories when troubleshooting.

The `ssp_libraries.js` File

Applies to: SuiteCommerce Advanced

The `ssp_libraries.js` file contains the server-side JavaScript code used by SuiteCommerce Advanced (SCA). When you deploy SCA to NetSuite, the `gulp` task installs this file in the SSP Application.

When compiling the application, the `gulp` tasks generate the `ssp_libraries.js` file by combining the following:

- Backend models — includes all of the backend models defined in the application modules.
- `SspLibraries` module — contains JavaScript files that provide server-side methods and utilities used by the backend models. This module includes the following files:
 - `Application.js` — defines functions for interacting with SuiteScript and the Commerce API. These functions obtain context and environment information from NetSuite. This file also provides methods for sending HTTP responses and errors when returning paginated results.
 - `Configuration.js` — defines backend configuration for SCA. For more information, see the help topic [Backend Configuration](#).
 - `Console.js` — creates the server-side console used to access the SSP application.
 - `Events.js` — defines the core utilities and base classes to create high-level backend entity models used by services.
 - `Models.Init.js` — defines global variables available to all backend models.
 - `SC.Models.js` — defines the base class used by backend models.
 - `Utils.js` — defines global utility methods that perform server-side tasks, including searching records and formatting currencies.

Do More With SCA Customizations

Applies to: SuiteCommerce Advanced

When customizing SuiteCommerce Advanced (SCA), you can do more than change the behavior and design of your Commerce website. For example, you can enable site administrators to customize Commerce website functionality using the Site Management Tools (SMT). Read the following topics to learn more:

- [Create Custom Content Types for SMT](#)
- [Customize the Checkout Application](#)

Create Custom Content Types for SMT

Applies to: SuiteCommerce Advanced

A custom content type (CCT) lets you create custom functionality as content that you can dynamically manage using the Site Management Tools (SMT). Implementing CCTs for SCA requires two separate, but related, activities.

If you are implementing the Kilimanjaro release of SCA, you must first create a custom SCA module to contain the JavaScript, HTML, Sass, and resources required for your CCT. This topic explains how to accomplish this. The second activity involves setting up custom records in NetSuite for your custom content type using SMT.

See the help topic [Site Management Tools Custom Content Type](#) for details on setting up your CCT using Site Management Tools.



Important: If you are customizing the Aconcagua release of SCA or later, you must create CCTs using extensions. See the help topic [Create a Custom Content Type](#).



Note: You must have SMT enabled in your NetSuite account to implement CCTs on your SCA website. See the help topic [Site Management Tools](#) for information on how to set up SMT.

This procedure explains how to:

1. [Create a Custom Module for Your CCT.](#)
2. [Create an Entry Point File.](#)
3. [Create a View File.](#)
4. [Create a Template File.](#)
5. [Set Up Your ns.package.json and distro.json Files](#)
6. Deploy your code to your NetSuite account.

Create a Custom Module for Your CCT

Applies to: SuiteCommerce Advanced

Your CCT module must include the following files at a minimum:

- **Entry point** – this JavaScript file includes the `mountToApp()` method, which links your custom module to a CMS content type record, making it available for inclusion in your website. If you are implementing SCA 2019.2 or later, this can be a TypeScript or JavaScript file.
- **View** – this JavaScript file listens for and interprets events and defines the variables for use by the template. These variables are the various fields within your CMS content type record. If you are implementing SCA 2019.2 or later, this can be a TypeScript or JavaScript file.
- **HTML template** – this file uses Handlebars.js helpers and HTML to render the content to your site.

The following procedures use a fictitious CCT named **SC.CCT.ImageViewer** as an example. This CCT introduces a simple image viewer that gets data from either a custom record (as set up in SMT) or from certain data previously made available in the browser.

Note: You can create a custom CCT module to retrieve, create, and update data stored in NetSuite records using services and models. To do this, you must implement these files as you would any custom module. See [SCA Feature Modules](#) for details.

To create a CCT Module:

1. Create a custom module to contain your CCT files.
As a best practice, use the following format when naming your CCT Module: `SC.CCT.cctName@x.y.z`, where `cctName` is the name of your CCT and `x.y.z` is the version number.
For example:
`SC.CCT.ImageViewer@0.0.1`
2. In your new module, create the following subdirectories at a minimum:
 - **JavaScript** – contains the entry point JavaScript or TypeScript file and all views.
 - **Templates** – contains the HTML template that will render your CCT.

Note: If your CCT introduces any new Sass, SuiteScript, or Configuration files, add subdirectories for these files as well.

Create an Entry Point File

Applies to: SuiteCommerce Advanced

The entry point file is necessary to mount your module to the application. This provides the connection between SuiteCommerce Advanced (SCA) and Site Management Tools (SMT). For more information about the architecture and purpose of this file, see [Entry Point](#).

Note: If you are implementing SCA 2019.2 or later, the entry point file can be a TypeScript or JavaScript file. For more information about TypeScript support, see [TypeScript](#).

To create the entry point file:

1. In your CCT module's JavaScript directory, create a new JavaScript (.js) file.
2. Name this file to intuitively relate to your module.
JavaScript example:
`../SC.CCT.ImageViewer@0.0.1/JavaScript/SC.CCT.ImageViewer.js`
TypeScript example:

```
../SC.CCT.ImageViewer@0.0.1/JavaScript/SC.CCT.ImageViewer.ts
```

3. Define your entry point dependencies. This includes the view that you create later.

JavaScript example:

```
1 define(
2   'SC.CCT.ImageViewer'
3   , [
4     'SC.CCT.ImageViewer.View'
5   ]
6   , function (
7     SCCCTImageViewerView
8   )
```

TypeScript example:

```
1 ///
2
3 import SCCCTImageViewerView = require('./SC.CCT.ImageViewer.View');
```

4. Create the mountToApp() method.

This method is required to initialize your CCT module, making it available to the application. Wrapped inside the mountToApp method is `getComponent('CMS').registerCustomContentType()`, which passes the id and view variables to SMT:

- The id variable loads the CMS content type record, connecting your module to the CCT and making your CCT available in SMT.



Important: The value of the id variable must match the **Name** field of the CMS Content Type record, and it must be all lowercase. See the help topic [CMS Content Type Record](#) for details.

- The view variable initializes the view.

In the following examples, the id variable is 'sc_cct_imageviewer', which matches the CMS content type record's **Name** field.

JavaScript example:

```
1 {
2   'use strict';
3
4   //@class SC.CCT.ImageViewer
5   return {
6     mountToApp: function mountToApp (application)
7     {
8       application.getComponent('CMS').registerCustomContentType({
9         id: 'sc_cct_imageviewer'
10        , view: SCCCTImageViewerView
11      });
12    }
13  };
14  });
```

TypeScript example:

```
1 {
2   'use strict';
3
4   //@class SC.CCT.ImageViewer
5   export function mountToApp(application){
6     application.getComponent('CMS').registerCustomContentType({
7       id: 'sc_cct_imageviewer'
8     , view: SCCCTImageViewerView
```



```

9      });
10     }
11 };

```

5. Save the file.



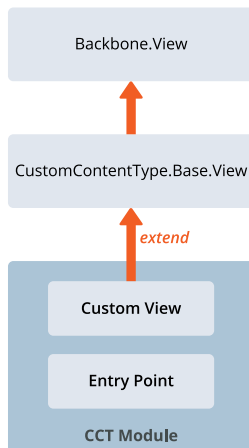
Important: Your SCA source code includes a module named `SC.CCT.Html`. This module connects your SCA application to the four core CCTs that are included with the SMT SuiteApp. Do not customize or alter the contents of this module.

Create a View File

Applies to: SuiteCommerce Advanced

Although the entry point loads your custom module as a CCT in SMT, your CCT still has no data to work with. The view file is necessary to access data, listen to and interpret user events, and specify the context for the template to render the data.

Included with your SCA source files, the `CustomContentType` module introduces the `CustomContentType.Base.View.js`. This file extends `Backbone.View.js`, initializes the CCT settings, and includes the base CCT class from which all custom CCT modules extend. You create your custom view to extend `CustomContentType.Base.View.js`.



Accessing Data

Each view can access SCA data about an item, itemlist, product, or category (available to the DOM through certain SCA modules). Your view can also return the values for custom record fields linked to the CMS Content Type record.



Important: Each view requires the `getContext()` method to expose this data to your templates.

Accessing SCA Data

If your CCT requires access to data not associated with a custom record's fields, you can use the `contextDataRequest` array property to access some SCA objects. If your CCT requires access to this data, you can set up your view to consume the information that the following `contextData` objects provide, assuming the information is previously provided in the DOM at the location that you place the CCT.

contextData pre-Vinson Object	View		Information Returned
	JavaScript File (SCA 2019.1 and Earlier)	TypeScript File (SCA 2019.2 and Later)	
category	Facets.Browse.View.js	Facets.Browse.View.ts	Returns the data of the category you are navigating
item	ProductDetails.Base.View.js	ProductDetails.Base.View.ts	Returns the data of the item in the Product Details page
itemlist	Facets.Browse.View.js	Facets.Browse.View.ts	Returns the current list of items in the search page
product	ProductDetails.Base.View.js	ProductDetails.Base.View.ts	Returns the data of the product in the Product Details page



Note: By default, when you add SCA content into an area using SMT Admin, the application runs the `validateContextDataRequest` method to check that you have all the requested contexts. In some cases, the `contextData` object might request information that is optional, and the data might not exist. In these cases, the `validateContextDataRequest` method fails because the data request returns empty. To account for this, set up your view to override the `validateContextDataRequest` method to always return true.

Example:

The example `ImageViewer` CCT requests the name of the `item` object. Your code might look similar to:

```

1 //...
2 ,   contextDataRequest: ['item']
3 ,   validateContextDataRequest: function()
4 {
5     return true;
6 }
7 ,   getContext: function getContext()
8 {
9     //...
10    if (this.contextData.item)
11    {
12        var item = this.contextData.item();
13        //...
14    }
15    //...
16 }
17 //...
```

Accessing Custom Record Fields

By default, the template receives a context object for each property defined in each associated custom record within a settings object. This is where you define the fields, by **Field ID**.

Example:

The example `ImageViewer` CCT uses a custom record with the following fields:

- `custrecord_sc_cct_iv_valign` – sets the vertical alignment of a text object.
- `custrecord_sc_cct_iv_text` – declares some text to display.
- `custrecord_sc_cct_iv_imageurl` – declares the URL to an image.

Within the `getContext()` method, you declare specific fields associated with your custom record as settings. Assuming the `custrecord_sc_cct_iv_valign` field requires a custom list of options, your code might look similar to:

```

1 //...
2 ,   getContext: function()
3   {
4     var texts = []
5     ,   imageUrl = ''
6     ,   valign = this.valign[this.settings.custrecord_sc_cct_iv_valign] || this.valign['3'];
7
8     var set_text = Utils.trim(this.settings.custrecord_sc_cct_iv_text)
9     ,   set_texts = set_text ? set_text.split('\n') : []
10    ,   set_imageUrl = Utils.trim(this.settings.custrecord_sc_cct_iv_imageurl);
11
12    texts = set_texts.length ? set_texts : texts;
13    imageUrl = set_imageUrl ? set_imageUrl : imageUrl;
14 //...

```

To create the View:

1. In your CCT module's JavaScript directory, create a new JavaScript file. (This can be a TypeScript (.ts) file if you are implementing SCA 2019.2 or later.)
2. Name this file to intuitively relate to your module as a view.

Example:

```
../SC.CCT.ImageViewer@0.0.1/JavaScript/SC.CCT.ImageViewer.View.js
```

3. Define the view's dependencies.

For the example `ImageViewer` CCT, your code might look like this:

JavaScript example:

```

1 define(
2   'SC.CCT.ImageViewer.View'
3   ,   [
4     'CustomContentType.Base.View'
5     ,   'SC.Configuration'
6
7     ,   'sc_cct_imageviewer_tpl'
8
9     ,   'Utils'
10    ,   'jQuery'
11  ]
12  ,   function (
13    CustomContentTypeBaseView
14    ,   Configuration
15
16    ,   sc_cct_imageviewer_tpl
17
18    ,   Utils
19    ,   jQuery
20  )
21  {

```

TypeScript example:

```
1 | // amd-module name="SC.CCT.ImageViewer.View"/>
```

```

2
3 import CustomContentTypeBaseView = require('../../CustomContentType.Base.View');
4 import Configuration = require('../../Configuration');
5 import sc_cct_imageviewer_tpl = require('../Templates/sc_cct_imageviewer.tpl');
6 import Utils = require('../../Commons/Utilities/Utils')
7 import jQuery = require('../../Commons/Utilities/jQuery')

```

4. Build your view according to your CCT's requirements.
5. Save the view file.

If you are implementing SCA 2019.2 or later and using TypeScript, your example ImageViewer CCT file might look similar to:

```

1  /// amd-module name="SC.CCT.ImageViewer.View"/>
2
3  import CustomContentTypeBaseView = require('../../CustomContentType.Base.View');
4  import Configuration = require('../../SC.Configuration');
5  import sc_cct_imageviewer_tpl = require('../Templates/sc_cct_imageviewer.tpl');
6  import Utils = require('../../Commons/Utilities/Utils.ts')
7  import jQuery = require('../../Commons/Utilities/jQuery')
8
9
10 const cctImageViewerView: any = CustomContentTypeBaseView.extend({
11     template: sc_cct_imageviewer_tpl
12 ,   install: function (settings, context_data)
13     {
14         this._install(settings, context_data);
15         var promise = jQuery.Deferred();
16         // The setTimeout method emulates an ajax call when the CCT is executed for the first time.
17
18         setTimeout(function()
19             {
20                 promise.resolve();
21             }, 4000);
22
23         return promise;
24     }
25 ,   contextDataRequest: ['item']
26 ,   validateContextDataRequest: function()
27     {
28         return true;
29     }
30 ,   valign: {
31     '1': 'top'
32     , '2': 'center'
33     , '3': 'bottom'
34     }
35 ,   getContext: function()
36     {
37         var texts = []
38         , imageUrl = ''
39         , valign = this.valign[this.settings.custrecord_sc_cct_iv_valign] || this.valign['3'];
40
41         if (this.contextData.item)
42         {
43             var item = this.contextData.item();
44             texts = [item.get('_name')];
45             var thumbnail = item.get('_thumbnail');
46             imageUrl = thumbnail.url ? thumbnail.url : thumbnail;
47         }
48
49         var set_text = Utils.trim(this.settings.custrecord_sc_cct_iv_text)
50         , set_texts = set_text ? set_text.split('\n') : []
51         , set_imageUrl = Utils.trim(this.settings.custrecord_sc_cct_iv_imageurl);
52
53         texts = set_texts.length ? set_texts : texts;
54

```

```

58     imageUrl = set_imageUrl ? set_imageUrl : imageUrl;
59
60     return {
61         hasText: !!texts.length
62         , texts: texts
63         , hasImage: !!imageUrl
64         , imageUrl: imageUrl
65         , valign: valign
66     };
67 }
68 });
69 export = cctImageViewerView;

```

If you are on SuiteCommerce Advanced 2019.1 or earlier, your example ImageViewer CCT file might look similar to:

```

1  define(
2      'SC.CCT.ImageView.View'
3      , [
4          'CustomContentType.Base.View'
5          , 'SC.Configuration'
6
7          , 'sc_cct_imageviewer_tpl'
8
9          , 'Utils'
10         , 'jQuery'
11     ]
12     , function (
13         CustomContentTypeBaseView
14         , Configuration
15
16         , sc_cct_imageviewer_tpl
17
18         , Utils
19         , jQuery
20     )
21     {
22         'use strict';
23
24         return CustomContentTypeBaseView.extend({
25
26             template: sc_cct_imageviewer_tpl
27
28             , install: function (settings, context_data)
29             {
30                 this._install(settings, context_data);
31
32                 var promise = jQuery.Deferred();
33
34                 // The setTimeout method emulates an ajax call when the CCT is executed for the first time.
35                 setTimeout(function()
36                 {
37                     promise.resolve();
38                 }, 4000);
39
40                 return promise;
41             }
42
43             , contextDataRequest: ['item']
44
45             , validateContextDataRequest: function()
46             {
47                 return true;
48             }
49
50             , valign: {
51                 '1': 'top'
52                 , '2': 'center'
53                 , '3': 'bottom'
54             }
55
56             , getContext: function()

```

```

57     {
58         var texts = []
59         ,   imageUrl = ''
60         ,   valign = this.valign[this.settings.custrecord_sc_cct_iv_valign] || this.valign['3'];
61
62         if (this.contextData.item)
63         {
64             var item = this.contextData.item();
65
66             texts = [item.get('_name')];
67
68             var thumbnail = item.get('_thumbnail');
69             imageUrl = thumbnail.url ? thumbnail.url : thumbnail;
70         }
71
72         var set_text = Utils.trim(this.settings.custrecord_sc_cct_iv_text)
73         ,   set_texts = set_text ? set_text.split('\n') : []
74         ,   set_imageUrl = Utils.trim(this.settings.custrecord_sc_cct_iv_imageurl);
75
76         texts = set_texts.length ? set_texts : texts;
77         imageUrl = set_imageUrl ? set_imageUrl : imageUrl;
78
79
80         return {
81             hasText: !!texts.length
82             ,   texts: texts
83             ,   hasImage: !!imageUrl
84             ,   imageUrl: imageUrl
85             ,   valign: valign
86         };
87     }
88 });
89 });

```

Create a Template File

i Applies to: SuiteCommerce Advanced

A template file is necessary to render the data defined by the view.

To create a template:

1. In your CCT module's Templates directory, create a new .tpl file.
2. Name this file to intuitively relate to your module as a template.

For example:

```
../SC.CCT.ImageViewer@0.0.1/Templates/sc_cct_imageviewer.tpl
```

3. Create the HTML required to render the data defined by the view.

For the example ImageViewer CCT, your template might look similar to:

```

1 <div class="sc-cct-imageviewer">
2   <div class="sc-cct-imageviewer-slider-container">
3     <div class="sc-cct-imageviewer-image-slider">
4       <ul data-sc-cct-imageviewer class="sc-cct-imageviewer-image-slider-list">
5         <li>
6           <div class="sc-cct-imageviewer-slide-main-container">
7             {{#if hasImage}}
8               
9             {{/if}}
10            {{#if hasText}}
11              <div class="sc-cct-imageviewer-slide-caption sc-cct-imageviewer-slide-caption-{{valign}}">

```

```

12         {{#each texts}}
13             <h2 class="sc-cct-imageviewer-slide-caption-title">{{this}}</h2>
14         {{/each}}
15     </div>
16     {{/if}}
17 </div>
18 </li>
19 </ul>
20 </div>
21 </div>
22 </div>

```

4. Save the template.

Set Up Your ns.package.json and distro.json Files

i Applies to: SuiteCommerce Advanced

To set up your ns.package.json file:

1. Open your root CCT module directory.
2. Create a new file in the custom module and name it ns.package.json.

For example:

../SC.CCT.ImageViewer@0.0.1/ns.package.json

3. Build the ns.package.json file using the following code:

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*"
5      ]
6    , "templates": [
7      "Templates/*"
8    ]
9  }
10 }

```

i Note: If your CCT includes any custom Sass, SuiteScript, services, or configuration files, you must account for these files here as well.

To set up your distro.json file:

1. Open the distro.json file.
 - If implementing SCA 2019.2 or later, the distro.json file is located in the Advanced directory of the SCA source code. Examples: SC_20.1/Advanced/distro.json, SC_19.2_Live/Advanced/distro.json
 - If implementing SCA 2019.1 or earlier, this file is located in the top-level directory of your SCA development directory.
2. Add an entry for the new CCT module in the modules object to ensure that the Gulp tasks include your code when you deploy to NetSuite. It should look similar to the following examples.
 - If implementing SCA 2019.2 or later, it should look similar to:

```

1 {
2   "name": "SuiteCommerce Advanced Live",
3   "version": "2.0",
4   "modules": {
5     //...
6     "../Advanced/SC.CCT.Html": "1.0.0",
7     "../Advanced/SC.CCT.ImageViewer": "0.0.1",
8     //...
9   }

```

- If implementing SCA 2019.1 or earlier, it should look similar to:

```

1 {
2   "name": "SuiteCommerce Advanced 1.0.0",
3   "version": "1.0.0",
4   "modules": {
5     //...
6     "suitecommerce/SC.CCT.Html": "1.0.0",
7     "suitecommerce/SC.CCT.ImageViewer": "0.0.1",
8     //...
9   }

```

3. Define any application dependencies for the preferred application (Shopping, My Account, or Checkout), within the javascript object.

For the example ImageViewer CCT, you add the module to the SC.Shopping.Starter entry point.

```

1 //...
2 "javascript": [
3   {
4     "entryPoint": "SC.Shopping.Starter",
5     "exportFile": "shopping.js",
6     "dependencies": [
7       "Backbone.View.Plugins",
8       "jQuery.html",
9       "ItemDetails",
10      //...
11      "SC.CCT.Html",
12      "SC.CCT.ImageViewer"
13    ],
14  }

```

4. If your CCT includes any Sass, include the module definition in the dependencies array of the preferred application (Shopping, My Account, or Checkout) within the sass object.

For the example ImageViewer CCT, your code might look similar to:

```

1 //...
2 "sass": {
3   //...
4   "applications": [
5     {
6       "name": "Shopping",
7       "exportFile": "shopping.css",
8       "dependencies": [
9         {
10          //...
11          "PickupInStore",
12          "Location.SCA",
13          "SC.CCT.ImageViewer"
14        }
15      ]
16    }
17  ]

```

5. Save the distro.json file.
6. If you have not already set your NetSuite records, complete the tasks as defined in [Site Management Tools Custom Content Type](#).



Important: Creating a custom module is only one step in the process. You must still perform the steps necessary to set up your custom module for use in SMT.

7. Test your custom module in your local environment or deploy to your NetSuite account. See the help topic [Develop Your SCA Customization](#) for details.

Customize the Checkout Application

Applies to: SuiteCommerce Advanced



Important: You should use the SCA developer tools to create custom modules as described in the following procedures only if you are customizing the Kilimanjaro release of SuiteCommerce Advanced (SCA) or earlier. If you are implementing the Aconcagua Release of SCA or later, the best practice is to use the theme and extension developer tools to customize your site. To learn more about why you should use the theme and extension developer tools, see [Benefits of the Extension Framework](#).

Much of the Commerce Checkout application can be customized or configured by editing standard configuration properties and by using existing extensions or creating new extensions. See the help topic [Checkout](#) for detailed information and procedures on how to do this.

If, however, you need to implement an advanced customization for an SCA solution, you can create custom modules and, using best practices, modify the checkout experience as described in the following procedures. To understand best practices, see the help topic [Best Practices for Customizing SCA](#).

By default, several modules are included in each checkout configuration file. Available modules include:

- `OrderWizard.Module.Address.Billing`: lets the user add, edit, remove, or choose a billing address.
- `OrderWizard.Module.Address.Shipping`: lets the user add, edit, remove, or choose a shipping address.
- `OrderWizard.Module.Confirmation`: provides a confirmation message after an order is submitted.
- `OrderWizard.Module.PaymentMethod.Creditcard`: lets the user add, edit, remove, or choose a credit card as a payment method.
- `OrderWizard.Module.PaymentMethod.GiftCertificates`: lets the user add a gift certificate code as a payment method.
- `OrderWizard.Module.PaymentMethod.Invoice`: lets the user select an invoice option as a payment method.
- `OrderWizard.Module.PaymentMethod.PayPal`: lets the user select a PayPal account as a payment method.
- `OrderWizard.Module.PaymentMethod.Selector`: lets the user select from a list of payment methods.
- `OrderWizard.Module.PaymentMethod.PurchaseNumber`: displays a Purchase Order Number field to allow users to search by PO Number. This field will appear independently of the Payment Method selected by the user.
- `OrderWizard.Module.RegisterEmail`: gives guests the option of registering an email address.
- `OrderWizard.Module.RegisterGuest`: lets the user register with the site as a customer.
- `OrderWizard.Module.Shipmethod`: lets the user select a delivery method.

- `OrderWizard.Module.ShowPayments`: displays the payment type selected. This is typical of a review page.
- `OrderWizard.Module.ShowShipments`: displays the shipment method selected if `MultiShipTo` is enabled. This is typical of a review page.
- `OrderWizard.Module.TermsAndConditions`: creates a Terms and Conditions checkbox during the Review step. If this property is not set, the default is false.

To customize the checkout flow:

1. Create a custom module that includes the `SC.Checkout.Configuration` object as a dependency.

Note: Do not edit the `SC.Checkout.Configuration.js` file directly. See the help topic [Develop Your SCA Customization](#) for information and best practices on customizing JavaScript.

Note: Do not edit the `SC.Checkout.Configuration.js` file (or `SC.Checkout.Configuration.ts` file if you are implementing SCA 2019.2 or later) directly. See the help topic [Develop Your SCA Customization](#) for information and best practices on customizing JavaScript.

2. Redefine the dependency `SC.Checkout.Configuration.Steps.Standard` to the preferred checkout flow as defined in the following table:

Checkout Flow	Dependency	Associated File	
		JavaScript (SCA 2019.1 and Earlier)	TypeScript (SCA 2019.2 and Later)
Standard (Default)	<code>SC.Checkout.Configuration.Steps.Standard</code>	<code>SC.Checkout.Configuration.Steps.Standard.js</code>	<code>SC.Checkout.Configuration.Steps.Standard.ts</code>
One Page Checkout	<code>SC.Checkout.Configuration.Steps.OPC</code>	<code>SC.Checkout.Configuration.Steps.OPC.js</code>	<code>SC.Checkout.Configuration.Steps.OPC.ts</code>
Billing First	<code>SC.Checkout.Configuration.Steps.BillingFirst</code>	<code>SC.Checkout.Configuration.Steps.BillingFirst.js</code>	<code>SC.Checkout.Configuration.Steps.BillingFirst.ts</code>

3. Extend the checkout flow's associated .js file to customize the checkout configuration.

You can customize checkout in following ways:

- [Reorder Checkout Modules](#)
- [Add Checkout Modules](#)
- [Add or Remove Checkout Steps](#)
- [Configure Checkout Step Properties](#)
- [Define Checkout Step URLs](#)

Note: The three checkout configurations shipped with SCA are fully supported by NetSuite. You can create completely customized experiences, but note that advanced customization can create flows that do not make intuitive sense to the user. NetSuite cannot guarantee the outcome of advanced customizations.

Reorder Checkout Modules

Applies to: SuiteCommerce Advanced

You can reorder modules within a checkout step by repositioning the module within the code.



Important: If reordering modules, ensure that you do not disrupt any flow between modules. For example, you want to place both the shipping address and delivery method modules within the same step (Shipping Address). However, the delivery methods available depend on the shipping address. This creates a flow issue. To maintain flow consistency in this case, the shipping address module must always precede the delivery method module in a separate step. Be aware of flow inconsistencies that may exist within steps or groups when customizing modules.

Add Checkout Modules

Applies to: SuiteCommerce Advanced

You can add modules to steps in multiple-step groups. For example, a guest shopper is prompted to register in the final confirmation step by default. By placing the OrderWizard.Module.RegisterGuest module in multiple steps, you can offer the registration at many points during the checkout process.

Each module is designed to use components and naming conventions consistent between the modules. When creating custom modules, we suggest that you follow similar conventions. For example, the RegisterEmail module contains the following components that are common among most other modules:

- **Module definition:** named from the most generic component (OrderWizard) to the most specific component (RegisterEmail).

```
1 | define('OrderWizard.Module.RegisterEmail', ['Wizard.Module'], function (WizardModule)
```

- **Associated Template:** this defines the HTML layout for the rendered output. Templates are found in the templates folder of each specific module.

```
1 | template: 'order_wizard_registeremail_module'
```

- **Render function:** the wizard object renders each step sequentially as it is defined in the code. The `_render();` function is responsible for displaying the template in the layout.

```
1 | , render: function()
2 | {
3 |     this.profile = this.wizard.options.profile;
4 |
5 |     // if the user is logged we dont render the module
6 |     if (this.profile.get('isGuest') !== 'T')
7 |     {
8 |         this.trigger('ready', true);
9 |     }
10 |    else
11 |    {
12 |        this._render();
13 |        if (this.profile.get('email') && this.wizard.isPaypalComplete())
14 |        {
15 |            this.trigger('ready', true);
16 |        }
17 |    }
18 | }
```

- **Submit function:** defines the logic performed when the Continue button is clicked.

```
1 | submit: function()
2 | {
3 |
4 |     var email = this.$('input[name=email]').val()
5 |     , newsletter = this.$('input[name=sign-up-newsletter]').is(':checked')
6 |     , self = this;
```

```

7
8      // if the user is not guest or not change the current values we just resolve the promise
9      if (this.profile.get('isGuest') !== 'T' || (email === this.profile.get('email') && this.profile.get('emailsub
scribe') === (newsletter ? 'T' : 'F') ))
10     {
11         return this.isValid();
12     }
13
14     this.profile.set('email', email);
15     this.profile.set('confirm_email', email);
16
17     return this.isValid().then(function() {
18         self.profile.set('emailsubscribe', newsletter);
19         return self.profile.save();
20     });
21
22 }

```

- **isValid function:** defines error checking for entries.

```

1  isValid: function()
2  {
3      var promise = jQuery.Deferred();
4
5      if (this.wizard.options.profile.get('isGuest') !== 'T')
6      {
7          return promise.resolve();
8      }
9
10     if (!Backbone.Validation.patterns.email.test(this.wizard.options.profile.get('email')))
11     {
12         return promise.reject(_('Email is required').translate());
13     }
14
15     return promise.resolve();
16 }

```

Add or Remove Checkout Steps

Applies to: SuiteCommerce Advanced

Each checkout flow configuration file corresponds to a configured checkout flow option. Each of these files contain multiple groups to achieve a specific task for that flow, such as adding shipping information. These groups contain one or more checkout steps to achieve that task. Each checkout step provides one or more wizard modules to guide the user through the checkout process.

Groups define the breadcrumbs available at the top of the page. Checkout steps then provide the modules that guide the shopper through the checkout process. You can add or remove checkout steps to any named group array by adding or removing steps as objects. You can also customize each step by editing properties.

For example, the following code sample in the `SC.Checkout.Configuration.Steps.Standard.js` file defines the Shipping Address group. This group contains two checkout steps: Choose Shipping Address and Enter Shipping Address. Each checkout step contains three modules. You can customize the Shipping Address group by modifying each step's code.

```

1  {
2      name: _('Shipping Address').translate()
3      , steps: [
4
5          {
6              name: _('Choose Shipping Address').translate()
7              , url: 'shipping/address'

```

```

8      , isActive: function ()
9      {
10         return !this.wizard.isMultiShipTo();
11     }
12     , modules: [
13         OrderWizardModuleMultiShipToEnableLink
14         , OrderWizardModuleAddressShipping
15         , [OrderWizardModuleCartSummary, cart_summary_options]
16     ]
17     }
18     , {
19         name: _('Enter Shipping Address').translate()
20         , url: 'shipping/selectAddress'
21         , isActive: function ()
22         {
23             return this.wizard.isMultiShipTo();
24         }
25         , modules: [
26             [OrderWizardModuleMultiShipToEnableLink, {exclude_on_skip_step: true}]
27             , [OrderWizardModuleMultiShipToSelectAddressesShipping, {edit_addresses_url: 'shipping/selectAddress' }]
28             , [OrderWizardModuleCartSummary, cart_summary_options]
29         ]
30     }
31 ]
32 }

```

Configure Checkout Step Properties

i Applies to: SuiteCommerce Advanced

Use the following properties when configuring each checkout step. Set these properties in the applicable Checkout Steps configuration file.

- **name** (string): specifies the literal name for the step as it appears in the breadcrumb.
- **getName** (function): specifies the dynamic name for the step as it appears in the breadcrumb. If this property is not defined, the default is the name property.
- **url** (string): specifies the url for the step. This property is required and must be unique among all steps.
- **continueButtonLabel** (string): specifies the label of the **Continue** button for the step. If not specified, the default is Continue.
- **hideBackButton** (boolean): specifies if the **Back** button appears on the site or not. If set to true, the button is hidden for this step. If not specified, the default is false.
- **hideContinueButton** (boolean): specifies if the **Continue** button appears on the site or not. If set to true, the button is hidden for this step. If not specified, the default is false.
- **hideSecondContinueButtonOnPhone** (boolean): specifies if the second **Continue** button displays on a smart phone or not. If set to true, the button is hidden for this step. If not specified, the default is false. Use this if there are too many **Continue** buttons in a step, such as the top, bottom and summary buttons).
- **hideSummaryItems** (boolean): specifies if the cart summary's items are shown on this step or not. If set to true, the item is hidden for this step. If not specified, the default is false.
- **hideBreadcrumb** (boolean): specifies if the page's breadcrumb is shown on this step or not. If set to true, the breadcrumb is hidden for this step. If not specified, the default is false.
- **headerView** (string): specifies the main site header defined for the step. If not defined, the simplified header is used. You can define the normal 'header' or a custom one.
- **footerView** (string): specifies the main site footer defined for the step. If not defined, the simplified footer is used. You can define the normal 'footer' or a custom one.

- `bottomMessage` (string): defines a message to display at the bottom of the step above the **Continue** and **Back** buttons. If not specified, the default is no message.
- `isActive` (function): specifies if the entire step is active (shown) or not, based on one or more conditions. The default implementation returns true. It is not required that all modules are active.
- `save` (function): is a custom save function that finishes the wizard and saves the configuration. When used within this module, this property calls the submit function.
- `present` (function): lets you customize the step to perform some action, such as trigger Google Analytics tracking.
- `modules` (array): defines an array of modules used for the step.

Define Checkout Step URLs

 **Applies to:** SuiteCommerce Advanced



Important: Each step must have a unique URL. This is important for maintaining site navigation, code statelessness, and for effective use of reporting, so you can perform analytics based off of URLs using tools such as Google Analytics.

Set the URL property of each step to define the unique URL of the step.

SCA Reference

i Applies to: SuiteCommerce Advanced

The following topics provide more detail about the SuiteCommerce Advanced (SCA) source files, architecture, and developer tools.

- [SCA Source Directory](#)
- [Product Details Page Architecture](#)
- [Changing SCA Developer Tools Connection Information](#)

For reference information about SCA gulp commands, see [Gulp Command Reference for SCA Developer Tools](#).

SCA Source Directory

i Applies to: SuiteCommerce Advanced

You can download the SuiteCommerce Advanced (SCA) source files directly from the file cabinet in NetSuite. The structure of these source files depends on the version of your implementation. The tables in the following topics describe the files and folders you see when you extract your source files from the NetSuite file cabinet and then set up the SCA developer tools:

- [SCA 2019.2 and Later Source Directory](#)
- [SCA 2019.1 and Earlier Source Directory](#)

i Note: From the SuiteCommerce Advanced release 2024.2 and later, source files will be set by default to inactive in the NetSuite File Cabinet. All files will still work as expected but you cannot view them unless you check the **Show Inactives** box.

SCA 2019.2 and Later Source Directory

For SCA 2019.2 and later releases, when you download and extract the source files as described in [Download SCA Source Files](#), the default name for the top-level directory is SuiteCommerce Advanced xx.x (where xx.x refers to the release, such as 20.1). The following directories and sub-directories reside in the top-level SuiteCommerce Advanced directory:

```

1 | SC_xx.x_Live
2 |   Advanced
3 |   Backend
4 |   Commons
5 |   gulp
6 |   DeployDistributionAdvanced
7 |   DeployDistributionAdvancedSS2
8 |   LocalDistributionAdvanced
9 |   LocalDistributionAdvancedSS2
10 | node_modules

```

The following table describes the preceding directories as well as some important files:

File / Folder	Description
SC_xx.x_Live	This folder contains the workspace for SuiteCommerce Advanced source code. This folder also contains administrative files necessary to run npm and gulp commands. You execute npm and node commands from this folder location. xx.x refers to the release. For example, SC_20.1_Live.
Advanced	This folder contains the source code for all of the modules that are used exclusively by SCA. This folder also contains the distro.json file, which lists all of the modules used by the SCA application and the name and version number of the distribution.
Backend	This folder contains models and service controllers for modules that have been migrated to SuiteScript 2.0.
Commons	This folder contains the source code for all of all modules that are common to both SCA and SuiteCommerce InStore.
gulp	This folder contains all of the files required by Gulp.js. You should not edit the files within this directory.
DeployDistributionAdvanced	This folder contains all of the files associated with the compiled application and any that rely on SuiteScript 1.0. This directory is created when you run the gulp deploy command. When you run the command, the compiled application files are output to this directory. After compilation, the contents of this directory are deployed to your NetSuite account. See Contents of the Deploy and Local Distribution Directories for information on the contents of this directory.  Important: Do not manually edit the files in this directory. It is created and updated when you run the gulp command.
DeployDistributionAdvancedSS2	This folder contains all of the files associated with the compiled application that rely on SuiteScript 2.0. This directory is created when you run the gulp deploy command. See Contents of the Deploy and Local Distribution Directories for information on the contents of this directory.  Important: Do not manually edit the files in this directory. It is created and updated when you run the gulp command.
LocalDistributionAdvanced	Created when you run the gulp local command, this directory contains all of the files associated with the compiled application used by the local server. When you run this command, Gulp.js outputs the compiled application files to this directory. After compilation, the contents of this directory are deployed to the local Node.js server. See Contents of the Deploy and Local Distribution Directories for information on the contents of this directory.

	 Important: Do not manually edit the files in this directory. It is created and updated when you run the gulp command.
LocalDistributionAdvancedSS2	Created when you run the <code>gulp local</code> command, this directory contains all of the files associated with the compiled application that rely on SuiteScript 2.0 and are used by the local server. When you run this command Gulp.js outputs the compiled application files to this directory. After compilation, the contents of this directory are deployed to the local Node.js server. See Contents of the Deploy and Local Distribution Directories for information on the contents of this directory.  Important: Do not manually edit the files in this directory. It is created and updated when you run the gulp command.
node_modules	This folder stores the dependencies and other files required by Node.js. This directory is created when you run the <code>npm install</code> command during SCA developer tools set up.
gulpfile.js	Contains all the JavaScript code for Gulp.js. You should only edit this file if you need to add a new task to Gulp.js .
.nsdeploy	This file defines how Gulp.js connects to your NetSuite account during deployment. This file is created the first time you run the <code>gulp deploy</code> command.
distro.json	This file, located in the <code>SC_xx.x_Live</code> directory, lists all of the modules used by the SCA application. It also specifies the name and version number of the distribution.

SCA 2019.1 and Earlier Source Directory

The source directory for SCA 2019.1 and earlier releases includes the following folders:

```

1 SuiteCommerce Advanced Version
2   gulp
3   Modules
4   node_modules
5   LocalDistribution
6   DeployDistribution

```

The following table describes the preceding directories as well as some important files:

File / Folder	Description
SuiteCommerce Advanced Version	This folder contains the workspace for SuiteCommerce Advanced source code. This folder also contains administrative files necessary to run npm and gulp commands. You execute npm and node commands from this folder location. <i>Version</i> refers to the release. For example: SuiteCommerce Advanced 2019.1 or SuiteCommerce Advanced Aconcagua
gulp	Contains all of the files required by Gulp.js. This file is created when Gulp.js is installed. You should not edit the files within this directory.
Modules	This folder contains the source code for all of the modules that are used exclusively by SCA.

node_modules	Stores the dependencies and other files required by Node.js. This directory is created when you run the <code>npm install</code> command as part of the SCA developer tools set up.
LocalDistribution	Created when you run the <code>gulp local</code> command. This directory contains all of the files associated with the compiled application used by the local server. When you run this command, Gulp.js outputs the compiled application files to this directory. After compilation, the contents of this directory are deployed to the local Node.js server. See Contents of the Deploy and Local Distribution Directories for information on the contents of this directory.  Important: Do not manually edit the files in this directory. It is created and updated when you run the <code>gulp</code> command.
DeployDistribution	Contains all of the files associated with the compiled application. This file is created when you run the <code>gulp deploy</code> command. When you run this command, the compiled application files are output to this directory. After compilation, the contents of this directory are deployed to your NetSuite account. See Contents of the Deploy and Local Distribution Directories for information on the contents of this directory.  Important: Do not manually edit the files in this directory. It is created and updated when you run the <code>gulp</code> command.
.nsdeploy	Defines how Gulp.js connects to your NetSuite account during deployment. This file is created the first time you run the <code>gulp deploy</code> command.
distro.json	Lists all of the modules used by the SuiteCommerce Advanced application. It also specifies the name and version number of the distribution.
gulpfile.js	Contains all the JavaScript code for Gulp.js. You should only edit this file if you need to add a new task to Gulp.js.

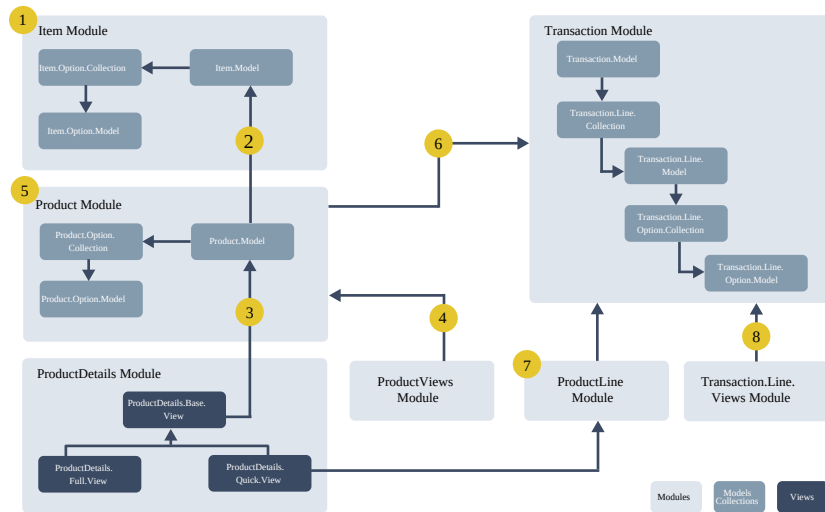
Product Details Page Architecture

 **Applies to:** SuiteCommerce Advanced

This section applies to the Elbrus release of SuiteCommerce Advanced (SCA) and later. The following architecture is not backward compatible with previous versions of SCA.

The product details page (PDP) provides shoppers with detailed information about a product and lets shoppers add items to their cart. To ensure a positive user experience, the PDP needs to create intuitive interactions and return information from the server quickly and efficiently. The architecture of the PDP is designed to address these challenges. Communication of item details is accomplished through the Items and Product modules. This unifies the data structure governing order management.

The following diagram details the architecture behind the PDP and how user selections interact with NetSuite.



1. Logic in the Item module retrieves read-only data from NetSuite. `Item.Model` retrieves data about an item through the Search API. This is a client-side, read-only instance of an item record in NetSuite. `Item.Option` receives read-only information about an option associated with the item. `Item.Option.Collection` is a collection of these possible options.

Note: `Item.Model` is the read-only data from NetSuite used by `Product.Model` to validate the user selection. Providing a read-only version of the Item ensures that the original backend data cannot be accidentally overridden.

2. `Product.Model` contains an `Item.Model` associated with an item to display in the PDP. `Product.Option.Model` contains editable instances of the related `Item.Option` models, and `Product.Option.Collection` is a collection of these options.
3. The views and child views in the `ProductDetails` module use the data in the `Product` module to render the PDP. `ProductDetails.Base.View` contains the abstract view from which the Full PDP and Quick View views inherit.
4. The views and child views in the `ProductViews` module use the data in the `Product` module to render price and item options in the PDP.

Note: To enhance extensibility, SCA uses two different item option templates, one for the PDP and another for search results (Facets).

5. When the user interacts with the PDP to select item options, `Product.Model` validates the data against the `Item.Model` that each `Product.Model` contains. This is why the `Product.Model` contains an `Item.Model` and why `Item.Model` is a client-side representation of the backend record.
6. When the user chooses valid options and clicks Add To Cart, the following conversions occur:
 - `Product.Option.Model` is converted to `Transaction.Line.Option.Model`
 - `Product.Option.Collection` is converted to `Transaction.Line.Option.Collection`
 - `Product.Model` is converted to `Transaction.Line.Model`

`Transaction.Model` contains a collection of transaction lines, which each include an item and selected options. `Transaction.Line.Option.Model` contains one of the selected options, and `Transaction.Line.Option.Collection` is a collection of all selected options.

7. The `ProductLine` module contains views that are shared between the PDP, Checkout, and My Account applications.

8. The Transaction module uses views in the Transaction.Line.Views module to render transaction details in the cart and in other modules.

Note: Item.KeyMapping.js is part of the Item Module. This file contains mapping definitions of what is returned by the Search API. For example, if you want to set the name of items for data stored in a custom field instead of the default **Display Name** field, you extend the mapping in the Item.KeyMapping file, as opposed to customizing each instance across the entire code base.

Configuration

To configure the Product Details Page, you must have the SuiteCommerce Configuration bundle installed. Refer to the correct section for details on configurable properties as described below.

To configure the Product Details Page:

1. Select the domain to configure at Commerce > Websites > Configuration.
2. In the SuiteCommerce Configuration record, navigate to the Shopping Catalog tab and the appropriate subtab:
 - [Item Options Templates Subtab](#) – configure how item options appear in the Product Details Page and in My Account transactions pages.
 - [Product Details Information Subtab](#) – configure extra fields to appear on your Product Details Page.
 - [Multi-Image Option Subtab](#) – configure multiple images in the Product Details Page based on multiple option selections.
3. Save the SuiteCommerce Configuration record to apply changes to your site.

Code Examples

The following examples show how to work with the PDP:

- [Add an Item to the Cart](#)
- [Validate an Item for Purchase](#)
- [Add Item to Wishlist](#)
- [Get Options](#)

Add an Item to the Cart

The following example of a custom model adds an item to the cart. Product.Model contains the item through the fetch method. Based on the internal ID of the item, this code then sets the quantity and item options and adds the product to the cart through the LiveOrderModel.

```

1  define('AddToCart',
2  [
3      'Product.Model'
4      , 'LiveOrder.Model'
5  ],
6  function(
7      ProductModel
8      , LiveOrderModel
9  )

```

```

10 {
11   var my_item_internal_id = '40'
12   ,   quantity = 10
13   ,   product = new ProductModel()
14   // load the item.
15   product.getItem().fetch({
16     data: {id: my_item_internal_id}
17   }).then(function () {
18     // set quantity
19     product.set('quantity', quantity);
20     // set options
21     product.setOption('option_id', 'value');
22     // add to cart
23     LiveOrderModel.getInstance().addProduct(product);
24   });
25 });

```

Validate an Item for Purchase

All shopping, wishlist, and quote item validation is centralized in the Product.Model. You can validate per field and developers can override the default validation per field.

The following customization uses a generic areAttributesValid method to validate per field. In this case it uses options and quantity, as opposed to validating the entire set. This method uses a pre-set default validation to validate the specified fields. The generateOptionalExtraValidation method overrides the default validation for any fields, in this case, quantity. If the values are not valid, the code introduces an alert message. Otherwise, the code adds the product to the cart.

Note: The default validation is set in Product.Model.js.

```

1  define('ValidateAddToCart'
2  , [
3    'LiveOrder.Model'
4  ]
5  , function (
6    LiveOrderModel
7  )
8  {
9    function generateOptionalExtraValidation() {
10     return {
11       'quantity': function () {}
12     }
13   }
14   var product = obtainSomeProduct(/*...*/);
15   var cart = LiveOrderModel.getInstance();
16   if (!product.areAttributesValid(['options', 'quantity'], this.generateOptionalExtraValidation()))
17   {
18     alert('Invalid Product!')
19   }
20   else
21   {
22     cart.addProduct(product);
23   }
24 });

```

Add Item to Wishlist

The following customization gets new item data from the product and the correct product list. It then adds the product line to the list and alerts the user after successful validation.

```

1  define('ValidateForWishList'

```

```

2 , [
3   'ProductList.Item.Model'
4 ]
5 , function (
6   ProductListItemModel
7 )
8 {
9   var getNewItemData = function (product, productList)
10   {
11     var product_list_line = ProductListItemModel.createFromProduct(product);
12     product_list_line.set('productList', {
13       id: productList.get('internalid')
14       , owner: productList.get('owner').id
15     });
16     return product_list_line;
17   };
18
19   var doAddProductToList = function (product, productList, dontShowMessage)
20   {
21     var product_list_line_to_save = getNewItemData(product, productList);
22     product_list_line_to_save.save(null, {
23       validate: false
24       , success: function (product_list_line_to_save_saved)
25       {
26         alert('Product Added!');
27       }
28     });
29   };
30   var product = getSomeProduct(/*...*/)
31   , product_list = getSomeProduct(/*...*/);
32   doAddProductToList(product, product_list);
33 });

```

Get Options

The following customization returns all valid options for an item or matrix item. Only valid options will render as selectable options in the browser.

```

1  define('ReadProductOptions'
2  , [
3    'underscore'
4  ]
5  , function (
6    -
7  )
8  {
9    var showFirstOptionWithValidValues = function (product)
10    {
11      //Collection of product.options
12      var options_to_render = product.get('options');
13      options_to_render.each(function (option)
14      {
15        //each option is a Model
16        if (option.get('isMatrixDimension'))
17        {
18          var valid_values_for_selected_option = product.getValidValuesForOption(option);
19          _.each(option.get('values'), function (value)
20          {
21            value.set('isAvailable', _.contains(valid_values_for_selected_option, value.get('label')));
22          });
23        }
24      });
25      var first_valid_option = options_to_render.find(function (option)
26      {
27        return _.some(option.get('values'), function (value) { return value.get('isAvailable'); });
28      });
29      var selected_option = product.get('options').findWhere({cartOptionId: first_valid_option.get('cartOptionId')});
30      //Note that we are always backbone Model and the value of the set option contains all details of the set value (label and internalid)

```

```

31 |         alert('First Option with value values: ' + first_valid_option.get('cartOptionId') + ' set value: ' + selected_op
tion.get('value').label)
32 |     };
33 |     showFirstOptionWithValidValues(getSomeIProduct(/*...*/));
34 | });

```

Changing SCA Developer Tools Connection Information

Applies to: SuiteCommerce Advanced



Important: The information in this topic applies to the Kilimanjaro release of SuiteCommerce Advanced (SCA) or earlier. If you are implementing the Aconcagua release of SCA or later, the best practice is to use the theme and extension developer tools to customize your site. See [Choose a Tool](#) for more information.

When deploying to NetSuite using the SCA developer tools and the `gulp deploy` command, Gulp.js uses the `.nsdeploy` file to determine how to connect to NetSuite. When you run `gulp deploy`, the script automatically configures the `.nsdeploy` file based on the information you provide when running `gulp deploy`. However, if there are changes to your account information or target folder, you may need to edit this file.

To edit the `.nsdeploy` file:

1. Open the `.nsdeploy` file in a text editor.
2. Edit the following parameters to reflect your account and system information:



Note: The `authID`, described in the following list is needed for token-based authentication to NetSuite. For more information, see [Token Based Authentication](#).

- **authID:** the authentication ID that represents the NetSuite account you want to deploy to and the role you want to use for authentication.
- **account:** the account number where you want to deploy.
- **hostname:** the hostname where your account is located.
- **target_folder:** the ID of the SSP application folder.

SuiteCommerce Releases and Patches

SuiteCommerce, SuiteCommerce MyAccount, and SuiteCommerce Advanced (SCA) releases follow the same general release schedule. How these updates apply to your implementation depends on the Commerce application you are implementing:

SuiteCommerce and SuiteCommerce MyAccount

SuiteCommerce and SuiteCommerce MyAccount are delivered as part of the same **managed bundle**. Therefore, all major and minor release updates apply to your account automatically. Unless otherwise noted in the release notes, these updates require no further action.

SuiteCommerce Advanced

SCA implementations are delivered as **unmanaged bundles**. Therefore, you must manually migrate your account to the latest major or minor release to take advantage of any related features and updates.

- To migrate your SCA implementation to the latest release, see [Migrate SuiteCommerce Advanced](#) for details.
- If you are interested in migrating to SCA from a SuiteCommerce implementation, see [Upgrade SuiteCommerce to SuiteCommerce Advanced](#).

All major and minor releases include specific release notes, indicating the new features or updates that apply. Releases prior to 2019.2 include release notes for major releases only. See the help topic [SuiteCommerce, SuiteCommerce MyAccount, and SuiteCommerce Advanced Release Notes](#) for details.

Familiarize yourself with the following topics:

- [Types of Releases](#)
- [What are SCA Patches?](#)

Types of Releases

What is a Major Release?

Each release includes a version number using a nomenclature that is based on the semantic versioning schema, or SEMVER. An example of release version nomenclature is **2019.2.1**, as depicted in the following image.



Major releases include significant product enhancements and new features or updates to both SuiteCommerce and SuiteCommerce Advanced applications. These releases occur approximately twice annually and are designated by the year of release. For example, 2019.1 and 2019.2.

Earlier Releases

Prior to the 2018.2 release, major releases of SuiteCommerce and SuiteCommerce Advanced were released using names of well-known mountains, not numbers. These were released as the following (listed in reverse-chronological order):

- Aconcagua
- Kilimanjaro (SCA Only)
- Elbrus (SCA Only)
- Vinson (SCA Only)
- Mont Blanc (SCA Only)
- Denali (SCA Only)

What is a Minor Release?

Minor releases include periodic updates to core source code for SuiteCommerce, SuiteCommerce MyAccount, and SuiteCommerce Advanced. These releases occur periodically within a major release cycle. Release notes explain any updates that have occurred. For example, 2019.2.1.

What are SCA Patches?

Patches apply to the Kilimanjaro release of SuiteCommerce Advanced and earlier only.

Patches are documented instructions detailing how to implement a code update by manually customizing core SuiteCommerce Advanced source code. Patches are only available for SuiteCommerce Advanced implementations of Kilimanjaro and earlier.

If you are implementing the Kilimanjaro release of SCA or earlier and you do not want to migrate to the latest release of SCA, we provide patching instructions for some issues you may encounter with your implementation.

For detailed instructions on patching SCA implementations (Kilimanjaro and earlier) see [SCA Patches](#).

Note: The Aconcagua release of SCA and later does not require patching. SuiteCommerce and SuiteCommerce MyAccount implementations receive updates automatically and, therefore, also do not require patching. The best practice to update SCA (Aconcagua and later) is to migrate to the latest major or minor release.

SCA Patches

Patches are documented instructions detailing how to implement a code update by manually customizing core SuiteCommerce Advanced source code. If you are implementing the Kilimanjaro release of SCA or earlier and you do not want to migrate to the latest release of SCA, we provide patching instructions for some issues you may encounter with your implementation.

Note: Implementations of SuiteCommerce and the Aconcagua release of SCA and later do not require patching. SuiteCommerce implementations receive updates automatically. The best practice to update SCA (Aconcagua and later) is to migrate to the latest major or minor release.

When implementing patches for SuiteCommerce Advanced, refer to the following resources:

- [Patch Instructions](#) provides links to the procedures to follow to resolve particular issues.
- [Patches Overview](#) describes the modes available for patching. Note that review of the detailed steps for each mode is not required before getting started. The individual patch instructions tell you which mode to use and also provide a link to the detailed steps.

Patches Overview

We provide two primary ways to patch SCA:

- [Patch Using Extend Mode](#)
- [Patch Using Override Mode](#)

Note: To apply patches to SuiteCommerce Advanced, you must have experience working with JavaScript, TypeScript, HTML, and CSS. Advanced JavaScript programming skills may be required if customizations in your implementation are affected by patch updates.

The preferred mode when implementing patches is **extend mode**. With extend mode, you use the JavaScript prototype function to extend a specific property, object, or method. Extend mode enables you to target the exact item you need to modify so that you don't have to extend an entire model or view. This mode also helps ensure the customization you implement for a patch continues to work when you migrate to a newer version of SuiteCommerce Advanced.

You can patch using **override mode** if you need to modify a file in a way that cannot be accomplished using extend mode. With override mode, you create a custom module to override a particular file. However, NetSuite best practice is to use extend mode for patches and to use override mode only when necessary.

Patch Using Extend Mode

If you want to implement a patch that corrects an issue by extending a property, object, or method, follow the steps in this procedure. You can access the instructions for each patch from a list provided in [Patch Instructions](#).

Note: Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Develop An Extension

1. Create an **extensions** directory where you can store your custom module. Depending on your implementation, this directory might already exist.
 - If you're patching SCA 2019.1 and earlier, create the extensions directory in the Modules directory.

- If you're patching SCA 2019.2 and later, create the extensions directory in the directory where the module that contains the file you want to extend resides.
- 2. Within the **extensions** directory, create a subdirectory with a name similar to the module you need to customize.
A good pattern to follow is: `.../extensions/<module_name>Extension@<extension_version>`
For example, If you need to patch the CMSadapter module, name your subdirectory: `CMSadapterExtension@1.0.0`.
- 3. Open the custom module subdirectory from the preceding step and create a new subdirectory with a name that reflects the type of file you need to patch. Possible types include: JavaScript, SuiteScript, Configuration, Sass, Template.
For example: `.../extensions/CMSadapterExtension@1.0.0/JavaScript`
- 4. In the subdirectory from the preceding step, create a file that includes the code needed for your extension.
Name this file according to best practices.
For example: `CMSadapter.Component.Extension.js`
- 5. Open the file and extend the property, object, or method by copy-pasting the code sample included in your patch instructions.
- 6. Save the file.

Step 2: Prepare the Developer Tools For Your Patch

1. Open the custom module directory you created in the first part of this procedure. For example, if you are extending the CMSadapter module, open the `CMSadapterExtension@1.0.0` directory.
2. Create a file in the custom module directory named `ns.package.json`.
For example: `.../extensions/CMSadapterExtension@1.0.0/ns.package.json`
3. Paste the code provided in your patch instructions into the new `ns.package.json` file.
The provided code is specific to the file type for your extension.
4. In the top level of the SuiteCommerce Advanced directory, open the `distro.json` file.

Note: For SuiteCommerce Advanced 2019.2 and later, the `distro.json` file is located in the Advanced directory. For example: `SuiteCommerce Advanced 2019.2/SC_19.2_Live/Advanced/distro.json`.

5. Add your extension to the `modules` object in the `distro.json` file.
Add your extension to the list of existing values that follow the `"modules"`: key as shown in the following example.

Note: Values for keys in the `modules` object are release-specific and so the following example may not look exactly like the object in your `distro.json` file. Be sure to add your module to the list of existing values exactly as shown in the instructions for your patch

```

1 {
2   "name": "SuiteCommerce Advanced Kilimanjaro",
3   "version": "2.0",
4   "isSCA": true,
5   "buildToolsVersion": "1.3.1",
6   "folders": {
7     "modules": "Modules",
8     "suitecommerceModules": "Modules/suitecommerce",
9     "extensionsModules": "Modules/extensions",
10    "thirdPartyModules": "Modules/third_parties",
11    "distribution": "LocalDistribution",

```

```

12     "deploy": "DeployDistribution"
13   },
14   "modules": {
15     "extensions/CMSadapterExtension": "1.0.0",
16     "extensions/MyExampleCartExtension1": "1.0.0",
17     ...

```

This step ensures that the Gulp tasks include your extension when you deploy. In the preceding example, extensions have been added at the beginning of the list of modules. However, you can add your extension anywhere in the list of existing values. The order of precedence in this list does not matter.

6. You may need to add your custom extension as a dependency to the starters of the required applications.

See your patch instructions for details.

7. Save the distro.json file.

Step 3: Test and Deploy Your Patch

How you test and deploy depends on your patch.

- If your patch extends a SuiteScript file, deploy the source code to NetSuite to test the functionality. See the help topic [Deploy SCA Customizations to NetSuite](#) for details. SuiteScript files run on the server and must be deployed to NetSuite to take effect.
- For extensions to file types other than SuiteScript, you can test your source code on a local server or deploy to your NetSuite account. See the help topic [Test SCA Customizations on a Local Server](#) or [Deploy SCA Customizations to NetSuite](#) for details.

Patch Using Override Mode

If you want to implement a patch that corrects an issue by overriding a method, follow the steps in this procedure. Remember to use override mode for patches only when you need to modify a file in a way that cannot be accomplished using extend mode. You can access the instructions for each patch from a list provided in [SCA Patches](#).

 **Note:** Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Create the Override File

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
 - If you're patching SCA 2019.1 and earlier, create the extensions directory in the Modules directory.
 - If you're patching SCA 2019.2 and later, create the extensions directory in the directory where the module that contains the file you want to override resides.
2. Within the **extensions** directory, create a subdirectory with a name similar to the module you need to patch.
 A good pattern to follow is: `.../extensions/<module_name>Extension@<extension_version>`
 For example, if you need to patch the CMSadapter module, name your subdirectory: `CMSadapterExtension@1.0.0`.
3. Open the custom module subdirectory from the preceding step and create a new subdirectory with a name that reflects the type of file you need to override. Possible types include: JavaScript, SuiteScript, Templates.
 For example: `.../extensions/CMSadapterExtension@1.0.0/JavaScript`

4. Copy the source file that you want to override and paste it into the JavaScript, Suitescript, or Templates subdirectory from the preceding step.

For example, to override the MSadapter.Component.js file in SCA 2018.2, you would:

Copy This File:	Place the Copy Here:
Modules/suitecommerce/CMSadapter@X.Y.Z/JavaScript/ CMSadapter.Component.js	Modules/extensions/CMSadapterExtension@1.0.0/ JavaScript

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

5. Paste the code provided in your patch instructions into your copy of the source file that you want to override. Make sure you replace the specific blocks or lines of code as instructed.
6. Save the file.

Step 2: Prepare the Developer Tools For Your Patch

1. Open the custom module directory you created in the first part of this procedure. For example, if you are patching the CMSadapter module, open the CMSadapterExtension@1.0.0 directory.
2. Create a file in the custom module directory named ns.package.json.
For example: `.../extensions/CMSadapterExtension@1.0.0/ns.package.json`
3. Paste the code provided in your patch instructions into the new ns.package.json file.
The code provided for the ns.package.json file is specific to the file that you want to override.
4. In the top level of the SuiteCommerce Advanced directory, open the distro.json file.

Note: For SuiteCommerce Advanced 2019.2 and later, the distro.json file is located in the Advanced directory. For example: SuiteCommerce Advanced 2019.2/SC_19.2_Live/Advanced/distro.json.

5. Add your custom module to the modules object in the distro.json file.

Add your custom module to the list of existing values that follow the "modules": key as shown in the following example.

Note: Values for keys in the modules object are release-specific and so the following example may not look exactly like the object in your distro.json file. Be sure to add your module to the list of existing values exactly as shown in the instructions for your patch.

```

1  {
2      "name": "SuiteCommerce Advanced Kilimanjaro",
3      "version": "2.0",
4      "isSCA": true,
5      "buildToolsVersion": "1.3.1",
6      "folders": {
7          "modules": "Modules",
8          "suitecommerceModules": "Modules/suitecommerce",
9          "extensionsModules": "Modules/extensions",
10         "thirdPartyModules": "Modules/third_parties",
11         "distribution": "LocalDistribution",
12         "deploy": "DeployDistribution"
13     },
14     "modules": {
15         "extensions/CMSadapterExtension": "1.0.0",
16         "extensions/MyExampleCartExtension1": "1.0.0",
17         ...

```

This step ensures that the Gulp tasks include your module when you deploy. In the preceding example, custom modules have been added at the beginning of the list of modules. However, you

can add your custom module anywhere in the list of existing values. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Patch

How you test and deploy depends on your patch.

- If your patch overrides a SuiteScript file, deploy the source code to NetSuite to test the functionality. See the help topic [Deploy SCA Customizations to NetSuite](#) for details. SuiteScript files run on the server and must be deployed to NetSuite to take effect.
- For overrides to JavaScript files, you can test your source code on a local server or deploy to your NetSuite account. See the help topic [Test SCA Customizations on a Local Server](#) or [Deploy SCA Customizations to NetSuite](#) for details.

Patch Instructions

The following table lists available patches for SuiteCommerce Advanced and Site Builder Extensions and Extensions Premium. Click on the issue title for information about how to implement the patch.

Issue	Release	Description
Update SOAP Library Version for SuiteCommerce Advanced Compatibility	SCA Denali – 2025.1	These patch instructions describe how to update the SOAP library version in your SCA environment to avoid issues with your developer tools.
Resolve Content-Type Mismatch for Commerce Websites	SuiteCommerce SCA Elbrus – 2023.2 SiteBuilder Extensions Premium Elbrus and SiteBuilder Extensions Premium Kilimanjaro	These patch instructions describe the steps to take to avoid any loss of website functionality after a change to the behavior of Content-Type headers is made on April 21, 2025.
Configuration Changes for Custom Records	SCA Vinson, Mont Blanc, and Denali SCA Reference My Account 1.02 – 1.06 ShopFlow 1.02 – 1.07	These patch instructions describe how to take steps to ensure the correct custom record behavior is implemented for your website
Changes in Product Reviews and Product List Permissions Levels	SCA 2024.2 and earlier Reference ShopFlow Reference My Account	These patch instructions describe how to update permissions to provide a higher level of security for the custom records created for Reference Product Reviews and Reference Product List.
Add Parameters Required for Shopper Access to Suitelet Internal URLs	SCA 2018.2 – 2019.2	These patch instructions describe the steps to add two parameters required to access the Suitelet's internal URL for the Shopper role.
CMS Page Type Website Records Not Displaying	SCA 2019.1 – 2023.2	These patch instructions describe how to fix the problem that caused CMS Page Type website records not to display correctly in some Commerce websites.
Google Tag Manager Events Not Displaying in Data Layer Upon Checkout	SCA 2018.2 – 2022.2	This section provides a sample of modified data layer code that will correctly track Google Analytics events information for implementations of SCA 2018.2 –

Issue	Release	Description
		2022.2 using Google Analytics 4 with the Google Tag Manager Commerce Extension.
Customer not Directed to the Payment Review Page after Checkout using the Checkout Flow	SCA 2021.2.3 SCA 2021.2 SCA 2020.1.12 SCA 2020.1 SCA 2018.2 SCA Kilimanjaro SiteBuilder Extensions Kilimanjaro SiteBuilder Extensions Premium Kilimanjaro	These patch instructions describe how to fix the problem that caused customers to not be directed to the payment review page after using the Checkout flow on the web site.
Customer not Directed to the Payment Review Page after Checkout after using the Make A Payment Flow	SCA 2021.2.3 SCA 2021.2 SCA 2020.1.12 SCA 2020.1 SCA 2018.2 SCA Kilimanjaro SiteBuilder Extensions Kilimanjaro SiteBuilder Extensions Premium Kilimanjaro	These patch instructions describe how to fix the problem that caused customers to not be directed to the payment review page after using the Make a Payment flow on the web site.
Guide to Enable Google Analytics 4	SCA versions Denali to 2022.1	This guide provides instructions to implement Google Analytics 4 (GA4) on pre-2022.2 implementations of SCA.  Note: Do not implement GA4 if you use GA4 with Google Tag Manager (GTM) because this causes duplicate tracking results. See the help topic Google Tag Manager for information about GTM.
Customer Redirected to My Account Overview Instead of Successful Payment Page	SCA 2021.2.3 SCA 2021.2 SCA 2020.1 SCA 2020.1.12 SCA 2018.2 SCA Kilimanjaro SiteBuilder Extensions Kilimanjaro SiteBuilder Extensions Premium Kilimanjaro	These patch instructions describe how to correct a problem that caused customers to be redirected to the My Account Overview page after they paid an invoice instead of the Successful Payment page.

Issue	Release	Description
Hide Add to Cart Button from Facet View	SCA Vinson SCA Elbrus SCA Kilimanjaro SCA Aconcagua	These patch instructions describe how to correct a problem so that the Add to Cart button is hidden from guest shoppers and only displayed to logged-in shoppers.
Removing Item Redirects to Login Page Under Secure Domain	Site Builder Reference Checkout 2.05	These patch instructions describe how to correct a problem that caused shoppers to be redirected to the login page after they removed an item from their shopping cart.
Token-Based Authentication Updates for Developer Tools	SCA 2020.2 SCA 2020.1 SCA 2019.2 SCA 2019.1 SCA 2018.2 SCA Aconcagua SCA and Site Builder Extensions Kilimanjaro SCA and Site Builder Extensions Elbrus SCA and Site Builder Extensions Mont Blanc SCA and Site Builder Extensions Vinson SCA and Site Builder Extensions Denali	These patch instructions describe how to update the developer tools to continue to support secure connection to NetSuite using token-based authentication.
Domain Bridging Update	Kilimanjaro Elbrus Vinson	These patch instructions describe how to update domain bridging which was causing an error to be returned on the Checkout page when the Encrypted Payload Domain Bridging feature is active.
Address Not Set for Orders for Unfulfillable Items	Kilimanjaro	These patch instructions describe how to correct an issue that prevents the default address from being set for orders for unfulfillable (service or downloadable) items.
Gift Certificate Error After Return Request Submitted	Elbrus	These patch instructions describe how to correct an issue that prevents a return authorization request from being submitted if the invoice status is set to Approved and there is a gift certificate included in the order.
Cannot Disable Default Credit Card	SCA 2018.2 SCA Aconcagua Kilimanjaro	These patch instructions describe how to correct an issue with the Make this my default credit card option on Commerce websites.
Image Tracker Resized After Placing Order	Elbrus	These patch instructions describe how to correct an issue with the Google AdWords image tracker image being resized.

Issue	Release	Description
Single Secure Domain Requires Login for Pricing	Elbrus	These patch instructions describe how to correct an issue encountered by users logging in to a Commerce website using a single secure domain when the Required Login for Pricing feature is enabled.
Page Not Found Error for SMT Landing Pages	Mont Blanc	These patch instructions describe how to correct an issue with SMT landing pages.
Enable Token-Based Authentication	SCA 2020.2 SCA 2020.1 SCA 2019.2 SCA 2019.1 SCA 2018.2 SCA Aconcagua SCA and Site Builder Extensions Kilimanjaro SCA and Site Builder Extensions Elbrus SCA and Site Builder Extensions Mont Blanc SCA and Site Builder Extensions Vinson SCA and Site Builder Extensions Denali	These patch instructions describe how to prepare your current installation of SCA or Site Builder Extensions for token-based authentication.
CVE-2020-14728: Vulnerability in SuiteCommerce Advanced Services	2019.2 2019.1 2018.2 Aconcagua Kilimanjaro Elbrus Mont Blanc Vinson	These patch instructions describe how to protect your site from the risk of Cross-site Scripting (XSS).
CVE-2020-14729: Vulnerability in SuiteCommerce Advanced Sites	SCA sites prior to 2020.1.4	This patch describes how to add security headers to minimize the risk of interface redressing, also known as clickjacking.
Enable Token-Based Authentication	SCA sites prior to 2021.1	These patch instructions describe how to prepare your current installation of SuiteCommerce Advanced for token-based authentication.
Site Pages Not Indexed	2019.1 2018.2 Aconcagua Kilimanjaro Elbrus	This patch describes how to correct an issue in some implementations of SCA where prerender of site pages fails, causing those pages to not be indexed.

Issue	Release	Description
	Mont Blanc Vinson	
Deploy to NetSuite Fails with Errors	2020.1 2019.2	This patch corrects an issue that causes deployments to NetSuite to fail with errors.
Values Not Stored in Custom Transaction Body Fields	Elbrus	These patch instructions describe how to correct an issue with custom transaction body fields. Specifically, text entered into custom transaction body fields during the checkout flow does not display in the Custom subtab of the resulting sales order as expected.
Patch Overrides Not Visible in DeployDistribution Folder	2018.2	This patch corrects an issue that causes patch overrides to not be visible in the DeployDistribution folder despite successful deployment.
Shopping User Environment Experiences Degraded Performance	2019.2 2019.1 2018.2 Aconcagua	This patch corrects an issue in some implementations of SuiteCommerce Advanced where shopping.user.environment.ssp experiences degraded performance.
beforeShowContent Event Stops Working After Initial Trigger	2019.2 2019.1	This patch corrects an issue in some implementations of SCA 2019.1 and 2019.2 where the beforeShowContentEvent stops working after the initial trigger in these components: ProductListPageComponent and ProductDetailsComponent.
Filter Site Option Does Not Work As Expected	Vinson	This patch corrects an issue in some Vinson implementations where site filtering fails to filter out orders for other web stores from the same account.
Quantity Facet Displays Dollar Sign	Kilimanjaro Vinson Elbrus	This patch corrects an issue in some implementations where adjusting the filter for a quantity facet on your site displays a numerical value that includes a dollar sign.
Custom Overrides Not Applying When Deployed	2018.2	This patch corrects an issue where overrides to core SCA source files do not work as expected.
Theme Developer Tool Overrides May Cause Error On Activation	2018.2	This patch corrects an issue where overrides implemented using the 2018.2 theme developer tools cause an error on activation.
Categories Navigate Adapter Event Removed in SMT	2018.2	This patch corrects an error where an adapter event was removed in SMT. This error prevents SMT admins from previewing current and future dated categories.
Categories Do Not Display Properly	Kilimanjaro	This patch corrects a problem where a child category does not display properly.
Incorrect Value for Shipping Estimate Occurs in Shopping Cart	Elbrus	This patch corrects a problem in some Elbrus implementations where the shipping estimate contains incorrect values before the actual value is shown.
Standard Promotion with Inline Discount and Rate as	Vinson Mont Blanc	This patch corrects a problem in some Vinson, Mont Blanc, and Denali implementations where the amount in checkout does not update when applying a standard

Issue	Release	Description
Percentage Not Updating the Amount in Checkout	Denali	promotion with an inline discount and setting rate as percentage.
Page With URL Fragments Redirects Too Many Times	Elbrus Kilimanjaro	This patch corrects a problem where a page with one or more URL fragments that return an ERR_TOO_MANY_REDIRECTS instead of showing a 404 error or the Product Details page.
See Complete List of Stores Link on Store Locator Page Does Not Show Store List	Kilimanjaro Vinson Elbrus	This patch corrects a problem where the See Complete List of Stores link on the Store Locator page reloads the page but does not show the complete list of stores.
Quantity Pricing Displayed in Web Store Even When 'Require Login for Pricing' is Enabled	Kilimanjaro Vinson Elbrus	This patch corrects a problem where quantity pricing displays in the web store even when Require Login for Pricing is enabled.
Edited Shipping Address on the Review Your Order Page is Not Showing Changes	Aconcagua Kilimanjaro	This patch corrects a problem where the Review Your Order page is not showing changes when user edits the shipping address.
Custom Page Title in SMT Does Not Display Correctly	Aconcagua	This patch corrects a problem where creating a title for a custom facets page with Site Management Tools does not display the correct title.
Currencies Change to the Default in the Shopping Application	Kilimanjaro	This patch corrects a problem where customers using multi-subsidiaries experience currencies not changing in the webstore, which continues to use the default currency.
Order Confirmation Page Not Displayed When Using Single Page Checkout and External Payment	Elbrus Vinson	This patch corrects a problem where the order confirmation page is not displayed when using single page checkout and an external payment method.
Incorrect Discounted Amounts on Checkout Summary	Aconcagua Kilimanjaro Elbrus	This patch corrects a problem where Checkout Summary pages display incorrect discounted sub-total amounts and inconsistent strike-through and original amounts.
DeployDistribution Folder Does Not Include Local Files	Aconcagua R2	This patch corrects a problem preventing local files to deploy to the DeployDistribution folder.
HTML List Styles Do Not Display	Aconcagua Kilimanjaro Elbrus Vinson Some SuiteCommerce Themes	This patch corrects a problem where editing item descriptions on an Item record using and styles does not display as expected in the webstore.
Item Search Displays Incorrect Results	Elbrus	This patch corrects a problem where the application returns incorrect results when entering more than one search term in the item search criteria.
Category Product Lists Return Page Not Found	2018.2	This patch corrects a problem in some implementations that results in a Page Not Found

Issue	Release	Description
		error, instead of returning Commerce Categories based on filtered facet values.
Cannot Test an Extension on a Local Server	2018.2	This patch corrects a problem in some implementations that prevents developers from testing extensions locally.
Secure Shopping for Site Builder Implementations	Site Builder Extensions Premium Vinson Site Builder Extensions Vinson Reference Checkout 2.05 Reference Checkout 2.04 Reference My Account Premium 1.06 Reference My Account 1.06 Reference My Account Premium 1.05 Reference My Account 1.05	This patch allows some Site Builder customers to secure the shopping portion of a web store under an HTTPS domain.
Cannot Scroll Through Long Menu Lists Using iOS	Kilimanjaro, Elbrus, Vinson, Mont Blanc, and Denali	This patch corrects a potential issue where iOS users cannot scroll through a long list of menu items when the list reaches beyond the device screen.
Pages Not Indexed Using Google's Mobile-First Indexing	2018.2, Aconcagua, Kilimanjaro, Elbrus, Vinson, Mont Blanc, and Denali	This patch corrects a problem in which search engines discard content and do not render the page correctly using mobile-first indexing.
Disabling Display of SuiteCommerce Gift Wrap & Message Extension Transaction Line Fields	SuiteCommerce Site Builder Websites Reference ShopFlow Reference Checkout Reference My Account	This section provides procedures for disabling the display of Gift Wrap & Message extension transaction line item fields.
Users Not Redirected to External Payment System	Vinson	This patch corrects an error where SuiteCommerce Advanced does not correctly send a user to an external payment system.
Invoices Do Not Include the Request for a Return Button	Vinson	For Reference My Account v1.06, this patch adds the Request for a Return Button to Invoice records in My Account.
Unable to Select Item Options Within the SuiteCommerce Configuration Record in NetSuite	Elbrus	This patch corrects an issue where site administrators cannot choose item options when configuring a domain using the SuiteCommerce Configuration record in NetSuite.
Incorrect Redirect URL for External Payments	Elbrus Vinson	This patch corrects an error where the page for an external payment system fails to properly redirect back to the SuiteCommerce site from which it was launched.

Issue	Release	Description
npm Error on Implementations	Denali R2	This patch corrects an issue where <code>npm install</code> fails to run on Denali R2.
Content Appears Incorrectly in a Merchandising Zone	Elbrus Kilimanjaro	This patch corrects an error when content incorrectly shows up in a merchandizing zone.
Reference My Account Generates Error on Load	Reference My Account version 1.04	This patch corrects an error where My Account fails to load if there are 1000+ invoices associated for that account.
Error Loading Shopping Page Due to Uncaught TypeError	Elbrus	This patch corrects an error where SuiteCommerce could not correctly parse the percent sign (%) in a URL.
Users Redirected to Checkout Application Instead of Shopping Application	Elbrus and Kilimanjaro	This patch corrects an issue where users in a password protected site were redirected to the Checkout application instead of the Shopping application.
Add to Cart Button Does Not Work If Quantity Field Selected	Elbrus	This patch corrects an issue where the Add to Cart button requires two clicks if the focus is on the Quantity field.
URLs with Redundant Facets Generated	Elbrus	This patch corrects an issue where URLs contain redundant facets.
Content Flickers or Disappears When Browsing the Product Listing Page	Kilimanjaro and earlier	This patch corrects an error that occurs when the product listing page flickers or disappears, for content added in SMT.
Enabling Google AdWords Causes Error on Login	Vinson	This patch contains a fix for an error that occurs when Google AdWords is enabled.
URL for Commerce Categories Contains Incorrect Delimiters	Elbrus and earlier	This patch contains a fix for URLs to commerce categories that contain incorrect delimiters.
Order Summary for Item-Based Promotions	Elbrus and earlier	This patch corrects an issue where the item totals for item-based promotions in the Order Summary are incorrect.
CSS Error Hides First div Element on Product Details Page	Mont Blanc and Vinson	This patch contains a fix for a CSS error that hides the first div element on the product details page. This div typically includes the product description.
Invoice Terms Not Included In Order Details	pre-Denali	This patch corrects an issue in Reference My Account v1.05 where the order in My Account does not include the invoice terms.
Users Required to Re-enter Credit Card Payment Method Details on Payment Page	Elbrus and Kilimanjaro	This patch corrects an issue where users are required to re-enter credit card payment method details to successfully complete their order.
Selected Invoice Not Displayed When Making an Invoice Payment	Mont Blanc	This patch corrects an issue where an invoice selected for payment does not appear on the Make a Payment page.
Log In to See Prices Message Appears When Users are Logged In	Elbrus	This patch corrects an issue where logged-in users receive a message directing them to log in to see prices.
Item Record HTML Meta Data Not Appearing on Page Meta Data	Elbrus	This patch corrects an issue where the value of an Item record's Meta Tag HTML field does not appear in the page's meta data.

Issue	Release	Description
Delivery Options Not Appearing After Editing the Cart and Re-entering a Shipping Address	SiteBuilder Extensions — Elbrus	This patch corrects an issue on Site Builder Extensions where delivery methods do not appear after editing cart and re-entering the same shipping address.
Order Confirmation and Thank You Page is Blank	Mont Blanc and Vinson	This patch corrects an issue where a Thank You page is not displayed after an order is complete.
Matrix Item Options Not Displaying With Google Tag Manager Enabled	Elbrus	This patch corrects an issue where not all selected options of a matrix item appear in the Product Details Page. This issue only applies to sites implementing Google Tag Manager.
Delivery Methods Not Appearing in One Page Checkout	Mont Blanc, Vinson, Elbrus	This patch corrects an issue where not all delivery methods appear in the One Page Checkout flow after adding a zip code and checking out as a guest.
Mastercard 2-Series BIN Regex Patch	Denali, Mont Blanc, Vinson	This patch is required to include the Mastercard 2-Series BIN regex value for payment method configuration as of the Elbrus release of SuiteCommerce Advanced.
Auto-Apply Promotions for Elbrus	Elbrus	This patch is required for Elbrus to take advantage of the auto-apply promotions features available as of the Kilimanjaro release of SuiteCommerce Advanced.
Change Email Address Patch	Denali, Mont Blanc, Vinson, Elbrus	This patch is required to take advantage of the change email address feature available as of the Kilimanjaro release of SuiteCommerce Advanced.
Duplicate Product Lists in Internet Explorer 11	pre-Denali	Provides instructions on how to modify the ProductList.js file in ShopFlow 1.07 to correct an issue where IE 11 caching causes duplicate Product Lists to display in My Account.
Save for Later Item not Moved to Cart	Mont Blanc	This patch corrects an issue where an error is returned when users set an item as Save for Later and then return to move that item to the cart.
Running Gulp Commands Results in a Syntax Error	Vinson and earlier	This patch corrects an issue where running gulp commands from a command line or terminal results in a syntax error.
Missing Promo Code on Return Request	Mont Blanc	This patch corrects an issue where promo codes applied to the original sales order are not included in the calculations for a return request.
Enhanced Page Content Disappears when Resizing the Browser	Denali, Mont Blanc, Vinson, Elbrus	This patch corrects an issue where enhanced pages do not maintain content when dynamically resizing the browser window.
Invoices Page Displays Incorrect Date Sort (pre-Denali)	pre-Denali	This patch corrects an issue in Reference My Account v1.05, where invoices are displayed out of order.
PayPal Payments Cause Error at Checkout	Mont Blanc	This patch extends the past() method in the OrderWizard.Module.PaymentMethod.PayPal.js file to correct cases where an error message is displayed when orders are placed using PayPal.
Canonical Tags Populated With Relative Paths	Vinson	This patch modifies commerce category canonical URLs to use absolute paths.

Issue	Release	Description
Shopping Cart Not Scrolling (Mobile)	Vinson and earlier	This patch corrects a situation where mobile users cannot scroll their Cart after removing an item from the Saved for Later product list. Follow these instructions to extend the ProductList.DetailsLater.View.js file and override the existing Query.scPush.js file with a new one (provided).
Error When Adding Items to Categories in Site Management Tools	Vinson	This patch contains a fix for an error on a category or subcategory that contains more than 10 items in Site Management Tools.
Item Search API Response Data not Cached	Vinson	This patch extends the <code>getSearchApiParams()</code> method in the Session module to include the <code>pricelevel</code> parameter. This is required to enable caching of the Item Search API Response data. SearchApiCdnCache.zip
Secure Shopping Domains (Elbrus, Vinson, Mont Blanc, and Denali)	Elbrus, Vinson, Mont Blanc, and Denali	This patch enables you to configure your site to maintain a secure browser. The patch for the Denali release of SuiteCommerce Advanced also includes a fix to maintain the identity of the user and cart contents as described in Serversync Touchpoint.  Note: The original patches provided for this fix have been updated. If you have previously applied a patch for SSL, only the diff between the previous patch and the current one should be applied. Vinson-ssl-V2.patch MontBlanc-ssl-V2.patch Denali-ssl-V2.patch Elbrus-ssl.patch
Secure Shopping Domain (pre-Denali)	pre-Denali	This section guides you through changes required for you to configure your pre-Denali site to maintain a secure browser.
PayPal Address not Retained in Customer Record.	Vinson and earlier	Provides instructions to retain PayPal Address details in the NetSuite customer record.
Login Email Address Appears in the Password Reset URL	Elbrus	This patch corrects a security issue in which the password reset email message includes the original login email address in the password reset URL.

Update SOAP Library Version for SuiteCommerce Advanced Compatibility

 **Applies to:** SuiteCommerce Advanced

Before NetSuite 2026.1 is released, you need to update the SOAP library version in your SuiteCommerce Advanced (SCA) environment to avoid issues with your developer tools.

These patch instructions describe how to update an attribute in your SOAP library. If you don't make this update, you may experience disruptions when you deploy changes with gulp commands in the SCA developer tools.



Important: This change won't affect live SuiteCommerce or SCA websites, or extensions. It only affects deployment processes with SCA developer tools.

To avoid problems with your developer tools, make this change after you've updated to NetSuite 2025.2 and before NetSuite 2026.1 is released.

To update the version attribute in your SOAP library:

1. In your local workspace, open the source code folder for the SCA developer tools.
2. Go to the correct folder depending on your SCA version:
 - For SCA 2019.1 and earlier (not including Denali):
`ns_npm_repository\suitetalk4node\src\`
 - For SCA Denali:
`gulp\ns-deploy\suitetalk4node\src`
 - For SCA 2019.2 to SCA 2024.2:
`Commons\ns_npm_repository\suitetalk4node\src\`
 - For SCA 2025.1:
`node_modules\@sc-utils\suitetalk\src`
3. Edit the `tool.js` file in a text editor.
4. Find the existing **nsVersion** attribute and change the value to **2024_2**.
 For example, change the **nsVersion** attribute from **2020_2** to **2024_2**.
5. Save and close the file.
6. Test your environment to ensure that everything is working correctly.

Resolve Content-Type Mismatch for Commerce Websites

A change to the behavior of Content-Type response headers may affect some SuiteScript Server Pages (SSPs) on SuiteCommerce, SuiteCommerce Advanced (SCA), and SiteBuilder Extensions (SBE) websites.

When responding to the client, SSPs are now required to clearly state the type of content that they are sending. In most cases for SuiteCommerce, SCA, and SBE, the SSPs need to specify that they are sending JavaScript code.

Patch instructions vary depending on what Commerce products and versions you are using. Typically, SuiteCommerce customers do not need to take any action as your NetSuite SSPs have been automatically updated. However, patching is required for the following implementations:

- SCA Elbrus to 2023.2
- SBE Elbrus and Kilimanjaro
- Any SuiteCommerce extensions that use a custom SSP

After the change is made, any unpatched code will likely stop functioning correctly as the browser will interpret responses without an explicit content stated as text. This will result in an error message like the following:

Refused to execute script from 'https://www.example.com/secure/customFieldsMetadata.ssp?t=123456789' because its MIME type ('text/plain') is not executable, and strict MIME type checking is enabled.

Be sure to follow the instructions for your the Commerce product and version you have implemented.

Implementation	Patch Instructions
SuiteCommerce with custom extensions	Resolve Content-Type Mismatch for SuiteCommerce with Extensions
SuiteCommerce without custom extensions	There is nothing you need to do. The SuiteCommerce bundle will update automatically.
SCA 2019.2 to 2023.2	Resolve Content-Type Mismatch for SCA 2019.2 to 2023.2
SCA 2019.1	Resolve Content-Type Mismatch for SCA 2019.1
SCA 2018.2	Resolve Content-Type Mismatch for SCA 2018.2
SCA Elbrus to Aconcagua SiteBuilder Extensions Elbrus and SiteBuilder Extensions Kilimanjaro (Premium and non-Premium versions)	Resolve Content-Type Mismatch for SCA Elbrus to Aconcagua, SB Extensions Elbrus and Kilimanjaro

Resolve Content-Type Mismatch for SuiteCommerce with Extensions

Note: These patch instructions only apply if you have developed custom extensions for your SuiteCommerce website that include custom SSPs. If you have an implementation of SuiteCommerce without custom extensions, there is nothing you need to do.

If your SuiteCommerce website uses a custom extension with a custom SSP, follow these instructions to confirm your SSPs are correctly set up.

1. Review your custom SSPs to confirm that the Content-Type set in the response header matches the content returned.
2. If no Content-Type is set, you will need to define the correct one for your custom SSP.
3. Make sure the services you created to handle http requests follow best practices. See the help topic [Create and Use SSP Applications](#)

If your services are not following best practice, you should extend ServiceController. See [Create a Service to Handle HTTP Requests](#).

Resolve Content-Type Mismatch for SCA 2019.2 to 2023.2

To implement this patch, create a custom module to override the customFieldsMetadata.ssp SuiteScript file, which is part of the CustomFields module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Create the Override File for customFieldsMetadata.ssp

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Commons/extensions/CustomFieldsExtension/SuiteScript/customFieldsMetadata.ssp
2. In the new customFieldsMetadata.ssp file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | response.write('SC.ENVIRONMENT.customFieldsMetadata=' + JSON.stringify(metadata));
```

Replace it with the following code:

```
1 | response.setContentType('JAVASCRIPT');
2 | response.write('SC.ENVIRONMENT.customFieldsMetadata=' + JSON.stringify(metadata));
```

Step 2: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new ns.package.json file that you created in the Commons directory: /Commons/extensions/CustomFieldsExtension/ns.package.json

```
1 | {
2 |   "gulp": {
3 |     "ssp-files": {
4 |       "SuiteScript/customFieldsMetadata.ssp": "customFieldsMetadata.ssp"
5 |     }
6 |   },
7 |   "overrides": {
8 |     "Commons/CustomFields/SuiteScript/customFieldsMetadata.ssp" : "SuiteScript/customFieldsMetadata.ssp"
9 |   }
10 | }
```

2. Open the distro.json file in the Advanced directory and add your custom module to the modules array as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the "modules" key.

```
1 |   "modules": [
2 |     "../Commons/extensions/CustomFieldsExtension",
3 |     ...]
```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 4: Check Custom SSPs

In addition to the customFieldsMetadata.ssp file, your custom SSPs may also be affected. Follow these instructions to confirm they are correctly set up.

1. Review your custom SSPs to confirm that the Content-Type set in the response header matches the content returned.
2. If no Content-Type is set, you will need to define the correct one for your custom SSP.

3. Make sure the services you created to handle http requests follow best practices. See the help topic [Create and Use SSP Applications](#).
If your services are not following best practice, you should extend ServiceController. See [Create a Service to Handle HTTP Requests](#).

Resolve Content-Type Mismatch for SCA 2019.1

To implement this patch, create a custom module to override the `customFieldsMetadata.ssp` SuiteScript file, which is part of the CustomFields module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Create the Override File for `customFieldsMetadata.ssp`

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/CustomFieldsExtension@1.0.0/SuiteScript/customFieldsMetadata.ssp`
2. In the new `customFieldsMetadata.ssp` file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | response.write('SC.ENVIRONMENT.customFieldsMetadata=' + JSON.stringify(metadata));
```

Replace it with the following code:

```
1 | response.setContentType('JAVASCRIPT');
2 | response.write('SC.ENVIRONMENT.customFieldsMetadata=' + JSON.stringify(metadata));
```

Step 2: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the Modules directory: `/Modules/extensions/CustomFieldsExtension@1.0.0/ns.package.json`

```
1 | {
2 |   "gulp": {
3 |     "ssp-files": {
4 |       "SuiteScript/customFieldsMetadata.ssp": "customFieldsMetadata.ssp"
5 |     }
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/CustomFields@X.Y.Z/SuiteScript/customFieldsMetadata.ssp" : "SuiteScript/customFieldsMetadata.ssp"
9 |   }
10 | }
```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and add your custom module to the modules object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```

1 | "modules": {
2 |   "extensions/CustomFieldsExtension": "1.0.0",
3 |   ...
4 | }

```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 4: Check Custom SSPs

In addition to the `customFieldsMetadata.ssp` file, your custom SSPs may also be affected. Follow these instructions to confirm they are correctly set up.

1. Review your custom SSPs to confirm that the Content-Type set in the response header matches the content returned.
2. If no Content-Type is set, you will need to define the correct one for your custom SSP.
3. Make sure the services you created to handle http requests follow best practices. See the help topic [Create and Use SSP Applications](#)

If your services are not following best practice, you should extend `ServiceController`. See [Create a Service to Handle HTTP Requests](#).

Resolve Content-Type Mismatch for SCA 2018.2

To implement this patch, create a custom module to override the following files:

- `customFieldsMetadata.ssp` SuiteScript file, which is part of the CustomFields module.
- `checkout.environment.shortcache.ssp` SuiteScript file, which is part of the CheckoutApplication module.
- `myaccount.environment.shortcache.ssp` SuiteScript file, which is part of the MyAccountApplication module.
- `shopping.environment.shortcache.ssp` SuiteScript file, which is part of the ShoppingApplication module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Create the Override File for `customFieldsMetadata.ssp`

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/CustomFieldsExtension@1.0.0/SuiteScript/customFieldsMetadata.ssp`
2. In the new `customFieldsMetadata.ssp` file, find and replace the following line of code with the provided code samples.

Find the following code:

```

1 | response.write('SC.ENVIRONMENT.customFieldsMetadata=' + JSON.stringify(metadata));

```

Replace it with the following code:

```
1 | response.setContentType('JAVASCRIPT');
2 | response.write('SC.ENVIRONMENT.customFieldsMetadata=' + JSON.stringify(metadata));
```

Step 2: Create the Override File for checkout.environment.shortcache.ssp

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/CheckoutApplicationExtension@1.0.0/SuiteScript/checkout.environment.shortcache.ssp
2. In the new checkout.environment.shortcache.ssp file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | response.setCDNCacheable(response.CACHE_DURATION_SHORT);
```

Replace it with the following code:

```
1 | response.setCDNCacheable(response.CACHE_DURATION_SHORT);
2 | response.setContentType('JAVASCRIPT');
```

Step 3: Create the Override File for myaccount.environment.shortcache.ssp

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/MyAccountApplicationExtension@1.0.0/SuiteScript/myaccount.environment.shortcache.ssp
2. In the new myaccount.environment.shortcache.ssp file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | response.setCDNCacheable(response.CACHE_DURATION_SHORT);
```

Replace it with the following code:

```
1 | response.setCDNCacheable(response.CACHE_DURATION_SHORT);
2 | response.setContentType('JAVASCRIPT');
```

Step 4: Create the Override File for shopping.environment.shortcache.ssp

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ShoppingApplicationExtension@1.0.0/SuiteScript/shopping.environment.shortcache.ssp
2. In the new shopping.environment.shortcache.ssp file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | response.setCDNCacheable(response.CACHE_DURATION_SHORT);
```

Replace it with the following code:

```
1 response.setCDNCacheable(response.CACHE_DURATION_SHORT);
2 response.setContentType('JAVASCRIPT');
```

Step 5: Prepare the Developer Tools for Your Patch – customFieldsMetadata.ssp

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the Modules directory: `/Modules/extensions/CustomFieldsExtension@1.0.0/ns.package.json`

```
1 {
2   "gulp": {
3     "ssp-files": {
4       "SuiteScript/customFieldsMetadata.ssp": "customFieldsMetadata.ssp"
5     }
6   },
7   "overrides": {
8     "suitecommerce/CustomFields@X.Y.Z/SuiteScript/customFieldsMetadata.ssp" : "SuiteScript/customFieldsMetadata.ssp"
9   }
10 }
```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```
1 "modules": {
2   "extensions/CustomFieldsExtension": "1.0.0",
3   ...
4 }
```

Step 6: Prepare the Developer Tools for Your Patch – checkout.environment.shortcache.ssp

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the Modules directory: `/Modules/extensions/CheckoutApplicationExtension@1.0.0/ns.package.json`

```
1 {
2   "gulp": {
3     "ssp-files": {
4       "SuiteScript/checkout.environment.shortcache.ssp":
5       "checkout.environment.shortcache.ssp"
6     }
7   },
8   "overrides": {
9     "suitecommerce/CheckoutApplication@X.Y.Z/SuiteScript/checkout.environment.shortcache.ssp" : "SuiteScript/checkout.environment.shortcache.ssp"
10  }
11 }
```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```
1 | "modules": {
2 |   "extensions/CheckoutApplicationExtension": "1.0.0",
3 |   ...
4 | }
```

Step 7: Prepare the Developer Tools for Your Patch – myaccount.environment.shortcache.ssp

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the Modules directory: `/Modules/extensions/MyAccountApplicationExtension@1.0.0/ns.package.json`

```
1 | {
2 |   "gulp": {
3 |     "ssp-files": {
4 |       "SuiteScript/myaccount.environment.shortcache.ssp":
5 |       "myaccount.environment.shortcache.ssp"
6 |     }
7 |   },
8 |   "overrides": {
9 |     "suitecommerce/MyAccountApplication@X.Y.Z/SuiteScript/myaccount.environment.shortcache.ssp" : "SuiteScript/myaccount.environment.shortcache.ssp"
10 |   }
11 | }
```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```
1 | "modules": {
2 |   "extensions/MyAccountApplicationExtension": "1.0.0",
3 |   ...
4 | }
```

Step 8: Prepare the Developer Tools for Your Patch – shopping.environment.shortcache.ssp

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the Modules directory: `/Modules/extensions/ShoppingApplicationExtension@1.0.0/ns.package.json`

```
1 | {
```

```

2  "gulp": {
3      "ssp-files": {
4          "SuiteScript/shopping.environment.shortcache.ssp":
5          "shopping.environment.shortcache.ssp"
6      }
7  },
8  "overrides": {
9      "suitecommerce/ShoppingApplication@X.Y.Z/SuiteScript/shopping.environment.shortcache.ssp" : "SuiteScript/shopping.environment.shortcache.ssp"
10 }
11 }

```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```

1  "modules": {
2      "extensions/ShoppingApplicationExtension": "1.0.0",
3      ...
4  }

```

Step 9: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 10: Check Custom SSPs

In addition to the `customFieldsMetadata.ssp` file, your custom SSPs may also be affected. Follow these instructions to confirm they are correctly set up.

1. Review your custom SSPs to confirm that the Content-Type set in the response header matches the content returned.
2. If no Content-Type is set, you will need to define the correct one for your custom SSP.
3. Make sure the services you created to handle http requests follow best practices. See the help topic [Create and Use SSP Applications](#)

If your services are not following best practice, you should extend `ServiceController`. See [Create a Service to Handle HTTP Requests](#).

Resolve Content-Type Mismatch for SCA Elbrus to Aconcagua, SB Extensions Elbrus and Kilimanjaro



Note: For SiteBuilder Extensions, this patch applies to both the premium and non-premium versions of Elbrus and Kilimanjaro.

To implement this patch, create a custom module to override the `customFieldsMetadata.ssp` SuiteScript file, which is part of the CustomFields module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Create the Override File for customFieldsMetadata.ssp

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/CustomFieldsExtension@1.0.0/SuiteScript/customFieldsMetadata.ssp
2. In the new customFieldsMetadata.ssp file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | response.write('SC.ENVIRONMENT.customFieldsMetadata=' + JSON.stringify(metadata));
```

Replace it with the following code:

```
1 | response.setContentType('JAVASCRIPT');
2 | response.write('SC.ENVIRONMENT.customFieldsMetadata=' + JSON.stringify(metadata));
```

Step 2: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new ns.package.json file that you created in the Modules directory: /Modules/extensions/CustomFieldsExtension@1.0.0/ns.package.json

```
1 | {
2 |   "gulp": {
3 |     "ssp-files": {
4 |       "SuiteScript/customFieldsMetadata.ssp": "customFieldsMetadata.ssp"
5 |     }
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/CustomFields@X.Y.Z/SuiteScript/customFieldsMetadata.ssp" : "SuiteScript/customFieldsMetadata.ssp"
9 |   }
10 | }
```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the distro.json file in the root SCA development directory and add your custom module to the modules object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```
1 | "modules": {
2 |   "extensions/CustomFieldsExtension": "1.0.0",
3 |   ...
4 | }
```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 4: Check Custom SSPs

In addition to the customFieldsMetadata.ssp file, your custom SSPs may also be affected. Follow these instructions to confirm they are correctly set up.

1. Review your custom SSPs to confirm that the Content-Type set in the response header matches the content returned.
 2. If no Content-Type is set, you will need to define the correct one for your custom SSP.
 3. Make sure the services you created to handle http requests follow best practices. See the help topic [Create and Use SSP Applications](#)
- If your services are not following best practice, you should extend ServiceController. See [Create a Service to Handle HTTP Requests](#).

Configuration Changes for Custom Records

From March 11, 2025, the behavior for how a custom record is checked for its existence will be updated. After this update, instead of performing a create record (CreateRecord) to check for the existence of a custom record, a search (lookupfield) is performed. This change improves performance and ensures that your web store continues to function as expected.

In some implementations of SuiteCommerce Advanced (SCA) and Site Builder, you must apply a patch to ensure the correct custom record behavior is implemented for your website.

If you have one of the following implementations, you need to apply a patch before March 11, 2025 to avoid any loss of website functionality.

- SCA, SiteBuilder Extensions, or SiteBuilder Extensions Premium – Vinson, Mont Blanc, or Denali
- SCA Reference My Account – 1.04 to 1.06
- SCA Reference ShopFlow – 1.04 to 1.07

Be sure to follow the corresponding patch instructions for the version of SCA you have implemented.

- [SCA and Site Builder Vinson, Mont Blanc, and Denali](#)
- [SCA Reference My Account 1.04 – 1.06 and ShopFlow 1.04 – 1.07](#)

SCA and Site Builder Vinson, Mont Blanc, and Denali

To implement this patch, create a custom module to override the `Utils.js` JavaScript file, which is part of the `SspLibraries` module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Create the Override File for Utils.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/SspLibrariesExtension@1.0.0/SuiteScript/Utils.js`
2. In the new `Utils.js` file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | , recordTypeExists: function (record_type_name)
2 |     {
3 |         try
4 |         {
```

```

5         nlapiCreateRecord(record_type_name);
6     }
7     catch (error)
8     {
9         return false;
10    }
11    return true;
12 }
13
14 // @method recordTypeHasField returns true if and only if the given field_name exists on the given
record_type_name.
15 // @param {String} record_type_name @param {String} field_name @return {Boolean}
16 , recordTypeHasField: function (record_type_name, field_name)
17 {
18     try
19     {
20         var record = nlapiCreateRecord(record_type_name);
21         return _.contains(record.getAllFields(), field_name);
22     }
23     catch (error)
24     {
25         return false;
26     }
27 }

```

Replace it with the following code:

```

1 , recordTypeExists: function (record_type_name)
2 {
3     try
4     {
5         nlapiCreateSearch(record_type_name, null, [], []);
6     }
7     catch (error)
8     {
9         return false;
10    }
11    return true;
12 }
13
14 // @method recordTypeHasField returns true if and only if the given field_name exists on the given
record_type_name.
15 // @param {String} record_type_name @param {String} field_name @return {Boolean}
16 , recordTypeHasField: function (record_type_name, field_name)
17 {
18     try
19     {
20         nlapiLookupField(record_type_name, 1, field_name);
21         return true;
22     }
23     catch (error)
24     {
25         return false;
26     }
27 }

```

3. Save the file.

Step 2: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the directory: `/Modules/extensions/SspLibrariesExtension@1.0.0/ns.package.json`

```

1 {
2   "gulp": {

```

```

3 |     "ssp-libraries": [
4 |         "SuiteScript/*.js"
5 |     ],
6 | },
7 | "overrides": {
8 |     "suitecommerce/SspLibraries@X.Y.Z/SuiteScript/Utils.js" : "SuiteScript/Utils.js"
9 | }
10| }

```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the "modules" key.

```

1 |     "modules": {
2 |         "extensions/SspLibrariesExtension": "1.0.0",
3 |         ...}

```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

SCA Reference My Account 1.04 – 1.06 and ShopFlow 1.04 – 1.07

You can edit the file, `commons.js`, in the NetSuite File Cabinet. See the help topic [Editing Files in the File Cabinet](#).

1. Go to Documents > Files > File Cabinet.
2. For My Account, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - My Account X.Y.Z > Custom My Account > `ssp-libraries`.
For ShopFlow, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - ShopFlow X.Y.Z > Custom ShopFlow > `ssp-libraries`.



Important: Replace the string, X.Y.Z, with the version of My Account or ShopFlow that you are using.

3. Create a backup of the file, `commons.js`.



Note: If `commons.js` does not exist in the `ssp-libraries` directory, make a copy from Web Site Hosting Files > Live Hosting Files > SSP Applications > Netsuite Inc. - ShopFlow X.Y.Z > Reference My Account > `ssp-libraries` > `commons.js`.

4. In the `commons.js` file, find and replace the following line of code with the provided code sample. Find the following code:

```

1 | function recordTypeExists (record_type_name)
2 | {
3 |     'use strict';
4 |
5 |     try
6 |     {
7 |         nlapiCreateRecord(record_type_name);
8 |     }
9 |     catch (error)

```

```

10 | {
11 |     return false;
12 | }
13 | return true;
14 | }
15 |
16 | // returns true if and only if the given field_name exists on the given record_type_name.
17 | function recordTypeHasField (record_type_name, field_name)
18 | {
19 |     'use strict';
20 |
21 |     try
22 |     {
23 |         var record = nlapiCreateRecord(record_type_name);
24 |         return _.contains(record.getAllFields(), field_name);
25 |     }
26 |     catch (error)
27 |     {
28 |         return false;
29 |     }
30 | }

```

Replace it with the following code:

```

1 | function recordTypeExists (record_type_name)
2 | {
3 |     'use strict';
4 |
5 |     try
6 |     {
7 |         nlapiCreateSearch(record_type_name, null, [], []);
8 |     }
9 |     catch (error)
10 |    {
11 |        return false;
12 |    }
13 |    return true;
14 | }
15 |
16 | // returns true if and only if the given field_name exists on the given record_type_name.
17 | function recordTypeHasField (record_type_name, field_name)
18 | {
19 |     'use strict';
20 |
21 |     try
22 |     {
23 |         nlapiLookupField(record_type_name, 1, field_name);
24 |         return true;
25 |     }
26 |     catch (error)
27 |     {
28 |         return false;
29 |     }
30 | }
31 | }

```

5. Save and deploy the file.

Changes in Product Reviews and Product List Permissions Levels

In some implementations of SuiteCommerce Advanced (SCA), permissions need to be changed to provide a higher level of security for the custom records created for Reference Product Reviews and Reference Product List.

These patch instructions describe how to update Product Reviews and Product List permissions so that your web store continues working properly.

Reference Product Reviews

The following patch instructions describe the steps to update the permissions for Product Reviews. Be sure to follow the instructions for your implemented version of SCA.

Implementation	Patch Instructions
SuiteCommerce SCA 2024.2.10 and later	Reference Product Reviews Patch for SuiteCommerce, SCA 2024.2.10, and Later
SCA 2024.2	Reference Product Reviews Patch for SCA 2024.2
SCA 2019.2 – 2024.1	Reference Product Reviews Patch for SCA 2019.2 to 2024.1
SCA 2018.2 – 2019.1	Reference Product Reviews Patch for SCA 2018.2 to 2019.1
SCA Aconcagua	Reference Product Reviews Patch for SCA Aconcagua
SCA Kilimanjaro	Reference Product Reviews Patch for SCA Kilimanjaro
SCA Elbrus	Reference Product Reviews Patch for SCA Elbrus
SCA Vinson	Reference Product Reviews Patch for SCA Vinson
SCA Mont Blanc	Reference Product Reviews Patch for SCA Mont Blanc
SCA Denali	Reference Product Reviews Patch for SCA Denali
Reference ShopFlow	Reference Product Reviews Patch for Reference ShopFlow

Reference Product List

The following patch instructions describe the steps to update the permissions for Product List. Be sure to follow the instructions for your implemented version of SCA.

Implementation	Patch Instructions
SuiteCommerce SCA 2024.2.10 and later	Reference Product List Patch for SuiteCommerce, SCA 2024.2.10, and Later
SCA 2024.2	Reference Product List Patch for SCA 2024.2
SCA 2019.2 – 2024.1	Reference Product List Patch for SCA 2019.2 to 2024.1
SCA 2018.2 – 2019.1	Reference Product List Patch for SCA 2018.2 to 2019.1
SCA Aconcagua	Reference Product List Patch for SCA Aconcagua
SCA Kilimanjaro	Reference Product List Patch for SCA Kilimanjaro
SCA Elbrus	Reference Product List Patch for SCA Elbrus
SCA Vinson	Reference Product List Patch for SCA Vinson
SCA Mont Blanc	Reference Product List Patch for SCA Mont Blanc
SCA Denali	Reference Product List Patch for SCA Denali
Reference ShopFlow	Reference Product List Patch for Reference ShopFlow

Implementation	Patch Instructions
Reference My Account	Reference Product List Patch for Reference My Account

Reference Product Reviews Patch for SuiteCommerce, SCA 2024.2.10, and Later

Follow these steps to confirm the Custom Record ID for Product Reviews is correct and the Access Type is set to Use Permission List.

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Reviews.
3. On the Custom Record Type page for Product Reviews, confirm that **ID** has the value, customrecord_ns_pr_review.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record types:
 - Product Reviews Attr. Rating – the ID should be customrecord_ns_pr_attribute_rating and the access type should be Use Permission List.
 - Product Reviews Images – the ID should be customrecord_ns_pr_image and the access type should be Use Permission List.

Reference Product Reviews Patch for SCA 2024.2

To implement this patch, set the permissions for the ProductReviews.Service.ss file.

Step 1: Set Permissions for ProductReviews.Service.ss

To set permissions for ProductReviews.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductReviews.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product Review**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Reviews.

3. On the Custom Record Type page for Product Reviews, confirm that **ID** has the value, `customrecord_ns_pr_review`.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record types:
 - Product Reviews Attr. Rating – the ID should be `customrecord_ns_pr_attribute_rating` and the access type should be Use Permission List.
 - Product Reviews Images – the ID should be `customrecord_ns_pr_image` and the access type should be Use Permission List.

Reference Product Reviews Patch for SCA 2019.2 to 2024.1

To implement this patch, create a custom module to override the `ProductReviews.Model.js` JavaScript file, which is part of the `ProductReviews` module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductReviews.Service.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to `ProductReviews.Service.ss`, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product Review**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override File for ProductReviews.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Commons/extensions/ProductReviewsExtension/SuiteScript/ProductReviews.Model.js`
2. In the new `ProductReviews.Model.js` file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn2()) {
2 |     throw unauthorizedError;
3 | }
```

Replace it with the following code:

```
1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn3()) {
```



```

2 |     throw unauthorizedError;
3 | }

```

Find the following code:

```

1 | if (ModelsInit.session.isLoggedIn2()) {
2 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
3 | }

```

Replace it with the following code:

```

1 | if (ModelsInit.session.isLoggedIn3()) {
2 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
3 | }

```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the directory: `/Commons/extensions/ProductReviewsExtension/ns.package.json`

```

1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "Commons/ProductReviews/SuiteScript/ProductReviews.Model.js" : "SuiteScript/ProductReviews.Model.js"
9 |   }
10 | }

```

2. Open the `distro.json` file in the `Advanced` directory and then add your custom module to the `modules` array as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the "modules" key.

```

1 | "modules": [
2 |   "../Commons/extensions/ProductReviewsExtension",
3 |   ...]

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Reviews.
3. On the Custom Record Type page for Product Reviews, confirm that **ID** has the value, `customrecord_ns_pr_review`.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.

6. Repeat steps 1 to 5 for the following custom record types:
 - Product Reviews Attr. Rating – the ID should be `customrecord_ns_pr_attribute_rating` and the access type should be Use Permission List.
 - Product Reviews Images – the ID should be `customrecord_ns_pr_image` and the access type should be Use Permission List.

Reference Product Reviews Patch for SCA 2018.2 to 2019.1

To implement this patch, create a custom module to override the `ProductReviews.Model.js` JavaScript file, which is part of the ProductReviews module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductReviews.Service.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to `ProductReviews.Service.ss`, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product Review**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override File for ProductReviews.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/ProductReviewsExtension@1.0.0/SuiteScript/ProductReviews.Model.js`
2. In the new `ProductReviews.Model.js` file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn2()) {
2 |     throw unauthorizedError;
3 | }
```

Replace it with the following code:

```
1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn3()) {
2 |     throw unauthorizedError;
3 | }
```

Find the following code:

```
1 | if (ModelsInit.session.isLoggedIn2()) {
```

```

2 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
3 | }

```

Replace it with the following code:

```

1 | if (ModelsInit.session.isLoggedIn()) {
2 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
3 | }

```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the directory: `/Modules/extensions/ProductReviewsExtension@1.0.0/ns.package.json`

```

1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductReviews@X.Y.Z/SuiteScript/ProductReviews.Model.js" : "SuiteScript/ProductReviews.Model.js"
9 |   }
10 | }

```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```

1 |   "modules": {
2 |     "extensions/ProductReviewsExtension": "1.0.0",
3 |     ...}

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Reviews.
3. On the Custom Record Type page for Product Reviews, confirm that **ID** has the value, `customrecord_ns_pr_review`.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record types:

- Product Reviews Attr. Rating – the ID should be `customrecord_ns_pr_attribute_rating` and the access type should be Use Permission List.
- Product Reviews Images – the ID should be `customrecord_ns_pr_image` and the access type should be Use Permission List.

Reference Product Reviews Patch for SCA Aconcagua

To implement this patch, create a custom module to override the `ProductReviews.Model.js` JavaScript file, which is part of the ProductReviews module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductReviews.Service.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to `ProductReviews.Service.ss`, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product Review**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override File for ProductReviews.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/ProductReviewsExtension@1.0.0/SuiteScript/ProductReviews.Model.js`
2. In the new `ProductReviews.Model.js` file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Find the following code:

```
1 | if (ModelsInit.session.isLoggedIn2())
```

```

2 | {
3 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
4 | }

```

Replace it with the following code:

```

1 | if (ModelsInit.session.isLoggedIn3())
2 | {
3 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
4 | }

```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the directory: `/Modules/extensions/ProductReviewsExtension@1.0.0/ns.package.json`

```

1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductReviews@X.Y.Z/SuiteScript/ProductReviews.Model.js" : "SuiteScript/ProductReviews.Model.js"
9 |   }
10 | }

```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```

1 |   "modules": {
2 |     "extensions/ProductReviewsExtension": "1.0.0",
3 |     ...}

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Reviews.
3. On the Custom Record Type page for Product Reviews, confirm that **ID** has the value, `customrecord_ns_pr_review`.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.

6. Repeat steps 1 to 5 for the following custom record types:
 - Product Reviews Attr. Rating – the ID should be `customrecord_ns_pr_attribute_rating` and the access type should be Use Permission List.
 - Product Reviews Images – the ID should be `customrecord_ns_pr_image` and the access type should be Use Permission List.

Reference Product Reviews Patch for SCA Kilimanjaro

To implement this patch, create a custom module to override the `ProductReviews.Model.js` JavaScript file, which is part of the ProductReviews module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductReviews.Service.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to `ProductReviews.Service.ss`, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product Review**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override File for ProductReviews.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/ProductReviewsExtension@1.0.0/SuiteScript/ProductReviews.Model.js`
2. In the new `ProductReviews.Model.js` file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Find the following code:

```

1 | if (ModelsInit.session.isLoggedIn2())
2 | {
3 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
4 | }

```

Replace it with the following code:

```

1 | if (ModelsInit.session.isLoggedIn3())
2 | {
3 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
4 | }

```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the directory: `/Modules/extensions/ProductReviewsExtension@1.0.0/ns.package.json`

```

1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductReviews@X.Y.Z/SuiteScript/ProductReviews.Model.js" : "SuiteScript/ProductReviews.Model.js"
9 |   }
10 | }

```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```

1 |   "modules": {
2 |     "extensions/ProductReviewsExtension": "1.0.0",
3 |     ...}

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Reviews.
3. On the Custom Record Type page for Product Reviews, confirm that **ID** has the value, `customrecord_ns_pr_review`.
4. Confirm that **Access Type** has Use Permission List selected.

If the access type is not correct, select Use Permission List from **Access Type**.

5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record types:
 - Product Reviews Attr. Rating – the ID should be `customrecord_ns_pr_attribute_rating` and the access type should be Use Permission List.
 - Product Reviews Images – the ID should be `customrecord_ns_pr_image` and the access type should be Use Permission List.

Reference Product Reviews Patch for SCA Elbrus

To implement this patch, create a custom module to override the `ProductReviews.Model.js` JavaScript file, which is part of the ProductReviews module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductReviews.Service.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to `ProductReviews.Service.ss`, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product Review**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override File for ProductReviews.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/ProductReviewsExtension@1.0.0/SuiteScript/ProductReviews.Model.js`
2. In the new `ProductReviews.Model.js` file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```


Find the following code:

```
1 | if (ModelsInit.session.isLoggedIn2())
2 | {
3 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
4 | }
```

Replace it with the following code:

```
1 | if (ModelsInit.session.isLoggedIn3())
2 | {
3 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
4 | }
```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the directory: `/Modules/extensions/ProductReviewsExtension@1.0.0/ns.package.json`

```
1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductReviews@X.Y.Z/SuiteScript/ProductReviews.Model.js" : "SuiteScript/ProductReviews.Model.js"
9 |   }
10 | }
```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```
1 |   "modules": {
2 |     "extensions/ProductReviewsExtension": "1.0.0",
3 |     ...}
```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Reviews.
3. On the Custom Record Type page for Product Reviews, confirm that **ID** has the value, `customrecord_ns_pr_review`.
4. Confirm that **Access Type** has Use Permission List selected.

If the access type is not correct, select Use Permission List from **Access Type**.

5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record types:
 - Product Reviews Attr. Rating – the ID should be customrecord_ns_pr_attribute_rating and the access type should be Use Permission List.
 - Product Reviews Images – the ID should be customrecord_ns_pr_image and the access type should be Use Permission List.

Reference Product Reviews Patch for SCA Vinson



Important: Before applying this patch, apply the patch for [Configuration Changes for Custom Records](#).

To implement this patch, create a custom module to override the ProductReviews.Model.js JavaScript file, which is part of the ProductReviews module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductReviews.Service.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductReviews.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product Review**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override File for ProductReviews.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductReviewsExtension@1.0.0/SuiteScript/ProductReviews.Model.js
2. In the new ProductReviews.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```

1 | if (this.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Find the following code:

```

1 | if (ModelsInit.session.isLoggedIn2())
2 | {
3 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
4 | }

```

Replace it with the following code:

```

1 | if (ModelsInit.session.isLoggedIn3())
2 | {
3 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
4 | }

```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the directory: `/Modules/extensions/ProductReviewsExtension@1.0.0/ns.package.json`

```

1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductReviews@X.Y.Z/SuiteScript/ProductReviews.Model.js" : "SuiteScript/ProductReviews.Model.js"
9 |   }
10 | }

```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “`modules`” key.

```

1 |   "modules": {
2 |     "extensions/ProductReviewsExtension": "1.0.0",
3 |     ...}

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.

2. Click the link for the custom record type, Product Reviews.
3. On the Custom Record Type page for Product Reviews, confirm that **ID** has the value, customrecord_ns_pr_review.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record types:
 - Product Reviews Attr. Rating – the ID should be customrecord_ns_pr_attribute_rating and the access type should be Use Permission List.
 - Product Reviews Images – the ID should be customrecord_ns_pr_image and the access type should be Use Permission List.

Reference Product Reviews Patch for SCA Mont Blanc



Important: Before applying this patch, apply the patch for [Configuration Changes for Custom Records](#).

To implement this patch, create a custom module to override the ProductReviews.Model.js JavaScript file, which is part of the ProductReviews module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductReviews.Service.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductReviews.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product Review**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override File for ProductReviews.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductReviewsExtension@1.0.0/SuiteScript/ProductReviews.Model.js
2. In the new ProductReviews.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.loginRequired && !session.isLoggedIn2())
```

```

2 | {
3 |     throw unauthorizedError;
4 | }

```

Replace it with the following code:

```

1 | if (this.loginRequired && !session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Find the following code:

```

1 | if (session.isLoggedIn2())
2 | {
3 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
4 | }

```

Replace it with the following code:

```

1 | if (session.isLoggedIn3())
2 | {
3 |     review.setFieldValue('custrecord_ns_prr_writer', nlapiGetUser() + '');
4 | }

```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the directory: `/Modules/extensions/ProductReviewsExtension@1.0.0/ns.package.json`

```

1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductReviews@X.Y.Z/SuiteScript/ProductReviews.Model.js" : "SuiteScript/ProductReviews.Model.js"
9 |   }
10 | }

```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```

1 |   "modules": {
2 |     "extensions/ProductReviewsExtension": "1.0.0",
3 |     ...}

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Reviews.
3. On the Custom Record Type page for Product Reviews, confirm that **ID** has the value, customrecord_ns_pr_review.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record types:
 - Product Reviews Attr. Rating – the ID should be customrecord_ns_pr_attribute_rating and the access type should be Use Permission List.
 - Product Reviews Images – the ID should be customrecord_ns_pr_image and the access type should be Use Permission List.

Reference Product Reviews Patch for SCA Denali



Important: Before applying this patch, apply the patch for [Configuration Changes for Custom Records](#).

To implement this patch, create a custom module to override the ProductReviews.Model.js JavaScript file, which is part of the ProductReviews module.

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductReviews.Service.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductReviews.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product Review**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override File for ProductReviews.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductReviewsExtension@1.0.0/SuiteScript/ProductReviews.Model.js
2. In the new ProductReviews.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.loginRequired && !session.isLoggedin())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.loginRequired && !session.isLoggedin3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the directory: `/Modules/extensions/ProductReviewsExtension@1.0.0/ns.package.json`

```
1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductReviews@X.Y.Z/SuiteScript/ProductReviews.Model.js" : "SuiteScript/ProductReviews.Model.js"
9 |   }
10 | }
```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```
1 |   "modules": {
2 |     "extensions/ProductReviewsExtension": "1.0.0",
3 |     ...}
```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Reviews.
3. On the Custom Record Type page for Product Reviews, confirm that **ID** has the value, `customrecord_ns_pr_review`.
4. Confirm that **Access Type** has Use Permission List selected.

If the access type is not correct, select Use Permission List from **Access Type**.

5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record types:
 - Product Reviews Attr. Rating – the ID should be customrecord_ns_pr_attribute_rating and the access type should be Use Permission List.
 - Product Reviews Images – the ID should be customrecord_ns_pr_image and the access type should be Use Permission List.

Reference Product Reviews Patch for Reference ShopFlow

 **Important:** Before applying this patch, apply the patch for [Configuration Changes for Custom Records](#).

To implement this patch, set the permissions for the product-reviews.ss file and modify the ssp_libraries/Models.js file.

Step 1: Set Permissions for ProductReviews.Service.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - ShopFlow X.Y.Z > Custom ShopFlow > services.

 **Important:** Replace the string, X.Y.Z, with the version of ShopFlow you are using.

3. Next to product-reviews.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product Review**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Modify the Models.js File

You can edit the file, Models.js, in the NetSuite File Cabinet. See the help topic [Editing Files in the File Cabinet](#).

1. Go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - ShopFlow X.Y.Z > Custom ShopFlow > ssp_libraries.

 **Important:** Replace the string, X.Y.Z, with the version of ShopFlow you are using.

3. Create a backup of the file, Models.js.

 **Note:** If Models.js does not exist in the ssp_libraries directory, make a copy from Web Site Hosting Files > Live Hosting Files > SSP Applications > Netsuite Inc. - ShopFlow X.Y.Z > Reference ShopFlow > ssp_libraries > Models.js.

4. In the Models.js file, find and replace the following line of code with the provided code sample.
Find the following code:


```

1 | if (this.loginRequired && !session.isLoggedIn())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Replace it with the following code:

```

1 | if (this.loginRequired && !session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }

```

5. Save and deploy the file.

Step 3: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Reviews.
3. On the Custom Record Type page for Product Reviews, confirm that **ID** has the value, customrecord_ns_pr_review.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record types:
 - Product Reviews Attr. Rating – the ID should be customrecord_ns_pr_attribute_rating and the access type should be Use Permission List.
 - Product Reviews Images – the ID should be customrecord_ns_pr_image and the access type should be Use Permission List.

Reference Product List Patch for SuiteCommerce, SCA 2024.2.10, and Later

Follow these steps to confirm the Custom Record ID for Product Lists is correct and the Access Type is set to Use Permission List.

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.
3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, customrecord_ns_pl_productlist.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be customrecord_ns_pl_productlistitem and the access type should be Use Permission List.

Reference Product List Patch for SCA 2024.2

To implement this patch, set the permissions for the ProductList files.

Step 1: Set Permissions for ProductList.Service.ss

To set permissions for ProductList.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Set Permissions for ProductList.Item.Service.ss

To set permissions for ProductList.Item.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Item.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 3: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.
3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, customrecord_ns_pl_productlist.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be customrecord_ns_pl_productlistitem and the access type should be Use Permission List.

Step 4: Add Item List Permission to View

1. In NetSuite, go to Setup > Users/Roles > Manage Roles.
2. Next to the SC Product Lists role with the ID, customrole_sc_product_lists, click **Edit**.
3. Go to the **Permissions** tab and the **Lists** subtab.
4. For the permission, Items, confirm the level is set to View.
If the Items permission level is not View, update the level to View, then click **Save**.

Reference Product List Patch for SCA 2019.2 to 2024.1

To implement this patch, create a custom module to override the following JavaScript files, which are part of the ProductList module:

- ProductList.Item.Model.js
- ProductList.Item.Search.js
- ProductList.Model.js
- ProductList.ServiceController.js

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductList Files

To implement this patch, set the permissions for the ProductList files.

To set permissions for ProductList.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

To set permissions for ProductList.Item.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Item.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override Files

Create the Override File for ProductList.Item.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Commons/extensions/ProductListExtension/SuiteScript/ProductList.Item.Model.js
2. In the new ProductList.Item.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2()) {
2 |     throw unauthorizedError;
3 | }
```

Replace it with the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3()) {
2 |     throw unauthorizedError;
3 | }
```

Create the Override File for ProductList.Item.Search.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Commons/extensions/ProductListExtension/SuiteScript/ProductList.Item.Search.js
2. In the new ProductList.Item.Search.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2()) {
2 |     throw unauthorizedError;
3 | }
```

Replace it with the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3()) {
2 |     throw unauthorizedError;
3 | }
```

Create the Override File for ProductList.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Commons/extensions/ProductListExtension/SuiteScript/ProductList.Model.js
2. In the new ProductList.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (
2 |     !_result(this.configuration, 'loginRequired') &&
3 |     !ModelsInit.session.isLoggedIn2()
4 | ) {
5 |     throw unauthorizedError;
6 | }
```

Replace it with the following code:

```
1 | if (
2 |     !_result(this.configuration, 'loginRequired') &&
3 |     !ModelsInit.session.isLoggedIn3()
4 | ) {
5 |     throw unauthorizedError;
6 | }
```

Create the Override File for ProductList.ServiceController.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Commons/extensions/ProductListExtension/SuiteScript/ProductList.ServiceController.jss
2. In the new ProductList.ServiceController.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | let user = ModelsInit.session.isLoggedIn2() ? nlapiGetUser() : 0;
```

Replace it with the following code:

```
1 | let user = ModelsInit.session.isLoggedIn3() ? nlapiGetUser() : 0;
```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new ns.package.json file that you created in the directory: /Commons/extensions/ProductListExtension/ns.package.json

```
1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "Commons/ProductList/SuiteScript/ProductList.Item.Model.js" : "SuiteScript/ProductList.Item.Model.js",
9 |     "Commons/ProductList/SuiteScript/ProductList.Item.Search.js" : "SuiteScript/ProductList.Item.Search.js",
10 |    "Commons/ProductList/SuiteScript/ProductList.Model.js" : "SuiteScript/ProductList.Model.js",
11 |    "Commons/ProductList/SuiteScript/ProductList.ServiceController.js" : "SuiteScript/ProductList.ServiceController.js"
12 |   }
13 | }
```

2. Open the distro.json file in the Advanced directory and then add your custom module to the modules array as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the "modules" key.

```

1 | "modules": [
2 |   "../Commons/extensions/ProductListExtension",
3 |   ...]

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.
3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, customrecord_ns_pl_productlist.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be customrecord_ns_pl_productlistitem and the access type should be Use Permission List.

Step 6: Add Item List Permission to View

1. In NetSuite, go to Setup > Users/Roles > Manage Roles.
2. Next to the SC Product Lists role with the ID, customrole_sc_product_lists, click **Edit**.
3. Go to the **Permissions** tab and the **Lists** subtab.
4. For the permission, Items, confirm the level is set to View.
If the Items permission level is not View, update the level to View, then click **Save**.

Reference Product List Patch for SCA 2018.2 to 2019.1

To implement this patch, create a custom module to override the following JavaScript files, which are part of the ProductList module:

- ProductList.Item.Model.js
- ProductList.Item.Search.js
- ProductList.Model.js
- ProductList.ServiceController.js

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductList Files

To implement this patch, set the permissions for the ProductList files.

To set permissions for ProductList.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

To set permissions for ProductList.Item.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Item.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override Files

Create the Override File for ProductList.Item.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Model.js
2. In the new ProductList.Item.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
```

```
4 | }
```

Create the Override File for ProductList.Item.Search.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Search.js
2. In the new ProductList.Item.Search.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Create the Override File for ProductList.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Model.js
2. In the new ProductList.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (!_.result(this.configuration, 'loginRequired') && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (!_.result(this.configuration, 'loginRequired') && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Create the Override File for ProductList.ServiceController.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.ServiceController.js
2. In the new ProductList.ServiceController.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | var user = ModelsInit.session.isLoggedIn2() ? nlapiGetUser() : 0
2 | , role = ModelsInit.context.getRoleId();
```


Replace it with the following code:

```
1 | var user = ModelsInit.session.isLoggedIn3() ? nlapiGetUser() : 0
2 | , role = ModelsInit.context.getRoleId();
```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the directory: `/Modules/extensions/ProductListExtension@1.0.0/ns.package.json`

```
1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Model.js" : "SuiteScript/ProductList.Item.Model.js",
9 |     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Search.js" : "SuiteScript/
ProductList.Item.Search.js",
10 |    "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Model.js" : "SuiteScript/ProductList.Model.js",
11 |    "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.ServiceController.js" : "SuiteScript/ProductList.ServiceCon
troller.js"
12 |   }
13 | }
```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “`modules`” key.

```
1 |   "modules": {
2 |     "extensions/ProductListExtension": "1.0.0",
3 |     ...}
```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.
3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, `customrecord_ns_pl_productlist`.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.

6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be customrecord_ns_pl_productlistitem and the access type should be Use Permission List.

Step 6: Add Item List Permission to View

1. In NetSuite, go to Setup > Users/Roles > Manage Roles.
 2. Next to the SC Product Lists role with the ID, customrole_sc_product_lists, click **Edit**.
 3. Go to the **Permissions** tab and the **Lists** subtab.
 4. For the permission, Items, confirm the level is set to View.
- If the Items permission level is not View, update the level to View, then click **Save**.

Reference Product List Patch for SCA Aconcagua

To implement this patch, create a custom module to override the following JavaScript files, which are part of the ProductList module:

- ProductList.Item.Model.js
- ProductList.Item.Search.js
- ProductList.Model.js
- ProductList.ServiceController.js

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductList Files

To implement this patch, set the permissions for the ProductList files.

To set permissions for ProductList.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

To set permissions for ProductList.Item.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.

2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Item.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override Files

Create the Override File for ProductList.Item.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Model.js
2. In the new ProductList.Item.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2()) {
2 |     throw unauthorizedError;
3 | }
```

Replace it with the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3()) {
2 |     throw unauthorizedError;
3 | }
```

Create the Override File for ProductList.Item.Search.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Search.js
2. In the new ProductList.Item.Search.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Create the Override File for ProductList.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Model.js
2. In the new ProductList.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (!!_.result(this.configuration, 'loginRequired') && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (!!_.result(this.configuration, 'loginRequired') && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Create the Override File for ProductList.ServiceController.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.ServiceController.js
2. In the new ProductList.ServiceController.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | var user = ModelsInit.session.isLoggedIn2() ? nlapiGetUser() : 0
2 | , role = ModelsInit.context.getRoleId();
```

Replace it with the following code:

```
1 | var user = ModelsInit.session.isLoggedIn3() ? nlapiGetUser() : 0
2 | , role = ModelsInit.context.getRoleId();
```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new ns.package.json file that you created in the directory: /Modules/extensions/ProductListExtension@1.0.0/ns.package.json

```
1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Model.js" : "SuiteScript/ProductList.Item.Model.js",
9 |     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Search.js" : "SuiteScript/
10 | ProductList.Item.Search.js",
11 |     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Model.js" : "SuiteScript/ProductList.Model.js",
12 |     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.ServiceController.js" : "SuiteScript/ProductList.ServiceCon
13 | troller.js"
```

```

12 |   }
13 | }

```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```

1 |   "modules": {
2 |     "extensions/ProductListExtension": "1.0.0",
3 |     ...}

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.
3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, `customrecord_ns_pl_productlist`.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be `customrecord_ns_pl_productlistitem` and the access type should be Use Permission List.

Step 6: Add Item List Permission to View

1. In NetSuite, go to Setup > Users/Roles > Manage Roles.
2. Next to the SC Product Lists role with the ID, `customrole_sc_product_lists`, click **Edit**.
3. Go to the **Permissions** tab and the **Lists** subtab.
4. For the permission, Items, confirm the level is set to View.
If the Items permission level is not View, update the level to View, then click **Save**.

Reference Product List Patch for SCA Kilimanjaro

To implement this patch, create a custom module to override the following JavaScript files, which are part of the ProductList module:

- `ProductList.Item.Model.js`
- `ProductList.Item.Search.js`
- `ProductList.Model.js`

- ProductList.ServiceController.js

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductList Files

To implement this patch, set the permissions for the ProductList files.

To set permissions for ProductList.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

To set permissions for ProductList.Item.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Item.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override Files

Create the Override File for ProductList.Item.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Model.js
2. In the new ProductList.Item.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```

1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2()) {
2 |     throw unauthorizedError;
3 | }

```

Replace it with the following code:

```

1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3()) {
2 |     throw unauthorizedError;
3 | }

```

Create the Override File for ProductList.Item.Search.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Search.js
2. In the new ProductList.Item.Search.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```

1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Replace it with the following code:

```

1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Create the Override File for ProductList.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Model.js
2. In the new ProductList.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```

1 | if (!_.result(this.configuration, 'loginRequired') && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Replace it with the following code:

```

1 | if (!_.result(this.configuration, 'loginRequired') && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Create the Override File for ProductList.ServiceController.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.ServiceController.js

2. In the new `ProductList.ServiceController.js` file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | var user = ModelsInit.session.isLoggedIn2() ? nlapiGetUser() : 0
2 | , role = ModelsInit.context.getRoleId();
```

Replace it with the following code:

```
1 | var user = ModelsInit.session.isLoggedIn3() ? nlapiGetUser() : 0
2 | , role = ModelsInit.context.getRoleId();
```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the directory: `/Modules/extensions/ProductListExtension@1.0.0/ns.package.json`

```
1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Model.js" : "SuiteScript/ProductList.Item.Model.js",
9 |     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Search.js" : "SuiteScript/ProductList.Item.Search.js",
10 |    "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Model.js" : "SuiteScript/ProductList.Model.js",
11 |    "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.ServiceController.js" : "SuiteScript/ProductList.ServiceController.js"
12 |   }
13 | }
```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```
1 |   "modules": {
2 |     "extensions/ProductListExtension": "1.0.0",
3 |     ...}
```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.

3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, `customrecord_ns_pl_productlist`.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be `customrecord_ns_pl_productlistitem` and the access type should be Use Permission List.

Step 6: Add Item List Permission to View

1. In NetSuite, go to Setup > Users/Roles > Manage Roles.
2. Next to the SC Product Lists role with the ID, `customrole_sc_product_lists`, click **Edit**.
3. Go to the **Permissions** tab and the **Lists** subtab.
4. For the permission, Items, confirm the level is set to View.
If the Items permission level is not View, update the level to View, then click **Save**.

Reference Product List Patch for SCA Elbrus

To implement this patch, create a custom module to override the following JavaScript files, which are part of the ProductList module:

- `ProductList.Item.Model.js`
- `ProductList.Item.Search.js`
- `ProductList.Model.js`
- `ProductList.ServiceController.js`

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductList Files

To implement this patch, set the permissions for the ProductList files.

To set permissions for `ProductList.Service.ss`:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to `ProductList.Service.ss`, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.

5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

To set permissions for `ProductList.Item.Service.ss`:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to `ProductList.Item.Service.ss`, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override Files

Create the Override File for `ProductList.Item.Model.js`

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Model.js`
2. In the new `ProductList.Item.Model.js` file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Create the Override File for `ProductList.Item.Search.js`

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Search.js`
2. In the new `ProductList.Item.Search.js` file, find and replace the following line of code with the provided code samples.

Find the following code:

```

1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2()) {
2 |     throw unauthorizedError;
3 | }

```

Replace it with the following code:

```

1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3()) {
2 |     throw unauthorizedError;
3 | }

```

Create the Override File for ProductList.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Model.js
2. In the new ProductList.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```

1 | if (!_.result(this.configuration, 'loginRequired') && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Replace it with the following code:

```

1 | if (!_.result(this.configuration, 'loginRequired') && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Create the Override File for ProductList.ServiceController.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.ServiceController.js
2. In the new ProductList.ServiceController.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```

1 | var user = ModelsInit.session.isLoggedIn2() ? nlapiGetUser() : 0
2 | , role = ModelsInit.context.getRoleId();

```

Replace it with the following code:

```

1 | var user = ModelsInit.session.isLoggedIn3() ? nlapiGetUser() : 0
2 | , role = ModelsInit.context.getRoleId();

```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new ns.package.json file that you created in the directory: /Modules/extensions/ProductListExtension@1.0.0/ns.package.json

```

1  {
2    "gulp": {
3      "ssp-libraries": [
4        "SuiteScript/*.js"
5      ]
6    },
7    "overrides": {
8      "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Model.js" : "SuiteScript/ProductList.Item.Model.js",
9      "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Search.js" : "SuiteScript/
ProductList.Item.Search.js",
10     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Model.js" : "SuiteScript/ProductList.Model.js",
11     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.ServiceController.js" : "SuiteScript/ProductList.ServiceCon
troller.js"
12   }
13 }

```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the `"modules"` key.

```

1  "modules": {
2    "extensions/ProductListExtension": "1.0.0",
3    ...}

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.
3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, `customrecord_ns_pl_productlist`.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be `customrecord_ns_pl_productlistitem` and the access type should be Use Permission List.

Step 6: Add Item List Permission to View

1. In NetSuite, go to Setup > Users/Roles > Manage Roles.
2. Next to the SC Product Lists role with the ID, `customrole_sc_product_lists`, click **Edit**.
3. Go to the **Permissions** tab and the **Lists** subtab.
4. For the permission, Items, confirm the level is set to View.
If the Items permission level is not View, update the level to View, then click **Save**.

Reference Product List Patch for SCA Vinson



Important: Before applying this patch, apply the patch for [Configuration Changes for Custom Records](#).

To implement this patch, create a custom module to override the following JavaScript files, which are part of the ProductList module:

- ProductList.Item.Model.js
- ProductList.Item.Search.js
- ProductList.Model.js
- ProductList.ServiceController.js

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductList Files

To implement this patch, set the permissions for the ProductList files.

To set permissions for ProductList.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

To set permissions for ProductList.Item.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Item.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override Files

Create the Override File for ProductList.Item.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Model.js
2. In the new ProductList.Item.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Create the Override File for ProductList.Item.Search.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Search.js
2. In the new ProductList.Item.Search.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.configuration.loginRequired && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Create the Override File for ProductList.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Model.js
2. In the new ProductList.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (!!_.result(this.configuration, 'loginRequired') && !ModelsInit.session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
```

```
4 | }
```

Replace it with the following code:

```
1 | if (!_.result(this.configuration, 'loginRequired') && !ModelsInit.session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Create the Override File for ProductList.ServiceController.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.ServiceController.jss
2. In the new ProductList.ServiceController.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | var user = ModelsInit.session.isLoggedIn2() ? nlapiGetUser() : 0
2 | , role = ModelsInit.context.getRoleId();
```

Replace it with the following code:

```
1 | var user = ModelsInit.session.isLoggedIn3() ? nlapiGetUser() : 0
2 | , role = ModelsInit.context.getRoleId();
```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new ns.package.json file that you created in the directory: /Modules/extensions/ProductListExtension@1.0.0/ns.package.json

```
1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Model.js" : "SuiteScript/ProductList.Item.Model.js",
9 |     "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Search.js" : "SuiteScript/
ProductList.Item.Search.js",
10 |    "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Model.js" : "SuiteScript/ProductList.Model.js",
11 |    "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.ServiceController.js" : "SuiteScript/ProductList.ServiceCon
troller.js"
12 |   }
13 | }
```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the distro.json file in the root SCA development directory and then add your custom module to the modules object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the "modules" key.

```
1 | "modules": {
```

```

2 | "extensions/ProductListExtension": "1.0.0",
3 | ...}

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.
3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, customrecord_ns_pl_productlist.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be customrecord_ns_pl_productlistitem and the access type should be Use Permission List.

Step 6: Add Item List Permission to View

1. In NetSuite, go to Setup > Users/Roles > Manage Roles.
2. Next to the SC Product Lists role with the ID, customrole_sc_product_lists, click **Edit**.
3. Go to the **Permissions** tab and the **Lists** subtab.
4. For the permission, Items, confirm the level is set to View.
If the Items permission level is not View, update the level to View, then click **Save**.

Reference Product List Patch for SCA Mont Blanc



Important: Before applying this patch, apply the patch for [Configuration Changes for Custom Records](#).

To implement this patch, create a custom module to override the following files, which are part of the ProductList module:

- ProductList.Item.Model.js
- ProductList.Item.Search.js
- ProductList.Model.js
- ProductList.Item.Service.ss
- ProductList.Service.ss

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductList Files

To implement this patch, set the permissions for the ProductList files.

To set permissions for ProductList.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

To set permissions for ProductList.Item.Service.ss:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to ProductList.Item.Service.ss, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override Files

Create the Override File for ProductList.Item.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Model.js
2. In the new ProductList.Item.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (this.configuration.loginRequired && !session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.configuration.loginRequired && !session.isLoggedIn3())
```

```

2 | {
3 |     throw unauthorizedError;
4 | }

```

Create the Override File for ProductList.Item.Search.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Search.js
2. In the new ProductList.Item.Search.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```

1 | if (this.configuration.loginRequired && !session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Replace it with the following code:

```

1 | if (this.configuration.loginRequired && !session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Create the Override File for ProductList.Model.js

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Model.js
2. In the new ProductList.Model.js file, find and replace the following line of code with the provided code samples.

Find the following code:

```

1 | if (this.configuration.loginRequired && !session.isLoggedIn2())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Replace it with the following code:

```

1 | if (this.configuration.loginRequired && !session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }

```

Create the Override File for ProductList.Item.Service.ss

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Item.Service.ss
2. In the new ProductList.Item.Service.ss file, find and replace the following line of code with the provided code samples.

Find the following code:

```

1 | if (isRegistrationDisabled || !SC.Configuration.passwordProtectedSite || session.isLoggedIn2())

```

Replace it with the following code:

```
1 | if (isRegistrationDisabled || !SC.Configuration.passwordProtectedSite || session.isLoggedIn3())
```

Find the following code:

```
1 | if (isRegistrationDisabled || !SC.Configuration.passwordProtectedSite || session.isLoggedIn2())
```

Replace it with the following code:

```
1 | if (isRegistrationDisabled || !SC.Configuration.passwordProtectedSite || session.isLoggedIn3())
```

Create the Override File for ProductList.Service.ss

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/ProductListExtension@1.0.0/SuiteScript/ProductList.Service.ss
2. In the new ProductList.Service.ss file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | if (isRegistrationDisabled || !SC.Configuration.passwordProtectedSite || session.isLoggedIn2())
```

Replace it with the following code:

```
1 | if (isRegistrationDisabled || !SC.Configuration.passwordProtectedSite || session.isLoggedIn3())
```

Find the following code:

```
1 | var context = nlapiGetContext()
2 | , role = context.getRoleId()
3 | , method = request.getMethod()
4 | , data = JSON.parse(request.getBody() || '{}')
5 | , id = request.getParameter('internalid') || data.internalid
6 | , user = session.isLoggedIn2() ? nlapiGetUser() : 0;
```

Replace it with the following code:

```
1 | if (isRegistrationDisabled || !SC.Configuration.passwordProtectedSite || session.isLoggedIn3())
2 | var context = nlapiGetContext()
3 | , role = context.getRoleId()
4 | , method = request.getMethod()
5 | , data = JSON.parse(request.getBody() || '{}')
6 | , id = request.getParameter('internalid') || data.internalid
7 | , user = session.isLoggedIn3() ? nlapiGetUser() : 0;
```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new ns.package.json file that you created in the Modules directory: /Modules/extensions/ProductListExtension@1.0.0/ns.package.json

```
1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
```

```

4 |     "SuiteScript/*.js"
5 |   ],
6 |   "services": [
7 |     "SuiteScript/*.Service.ss"
8 |   ]
9 | },
10 | "overrides": {
11 |   "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Model.js" : "SuiteScript/ProductList.Item.Model.js",
12 |   "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Search.js" : "SuiteScript/
ProductList.Item.Search.js",
13 |   "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Model.js" : "SuiteScript/ProductList.Model.js",
14 |   "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Item.Service.ss" : "SuiteScript/ProductList.Item.Ser
vice.ss",
15 |   "suitecommerce/ProductList@X.Y.Z/SuiteScript/ProductList.Service.ss" : "SuiteScript/ProductList.Service.ss"
16 | }
17 | }

```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the "modules" key.

```

1 |   "modules": {
2 |     "extensions/ProductListExtension": "1.0.0",
3 |     ...}

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.
3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, `customrecord_ns_pl_productlist`.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be `customrecord_ns_pl_productlistitem` and the access type should be Use Permission List.

Step 6: Add Item List Permission to View

1. In NetSuite, go to Setup > Users/Roles > Manage Roles.
2. Next to the SC Product Lists role with the ID, `customrole_sc_product_lists`, click **Edit**.
3. Go to the **Permissions** tab and the **Lists** subtab.
4. For the permission, Items, confirm the level is set to View.
If the Items permission level is not View, update the level to View, then click **Save**.

Reference Product List Patch for SCA Denali



Important: Before applying this patch, apply the patch for [Configuration Changes for Custom Records](#).

To implement this patch, create a custom module to override the `Utils.js` JavaScript file, which is part of the `SspLibraries` module

If you are unfamiliar with implementing patches for SuiteCommerce Advanced (SCA), refer to:

- [Best Practices for Customizing SCA](#)
- [Patch Using Override Mode](#)

Step 1: Set Permissions for ProductList Files

To implement this patch, set the permissions for the `ProductList` files.

To set permissions for `ProductList.Service.ss`:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to `ProductList.Service.ss`, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

To set permissions for `ProductList.Item.Service.ss`:

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > services.



Important: Replace the string, X.Y.Z, with the version of SCA you are using.

3. Next to `ProductList.Item.Service.ss`, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Create the Override File for `Utils.js`

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/SspLibrariesExtension@1.0.0/SuiteScript/Utils.js`

2. In the new `Utils.js` file, find and replace the following line of code with the provided code samples.

Find the following code:

```
1 | return session.isLoggedin();
```

Replace it with the following code:

```
1 | return session.isLoggedin3();
```

Step 3: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you created in the Modules directory: `/Modules/extensions/SspLibrariesExtension@1.0.0/ns.package.json`

```
1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/SspLibraries@X.Y.Z/SuiteScript/Utils.js" : "SuiteScript/Utils.js"
9 |   }
10 | }
```



Important: Replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```
1 |   "modules": {
2 |     "extensions/SspLibrariesExtension": "1.0.0",
3 |     ...}
```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Step 5: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.
3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, `customrecord_ns_pl_productlist`.
4. Confirm that **Access Type** has Use Permission List selected.

If the access type is not correct, select Use Permission List from **Access Type**.

5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be customrecord_ns_pl_productlistitem and the access type should be Use Permission List.

Step 6: Add Item List Permission to View

1. In NetSuite, go to Setup > Users/Roles > Manage Roles.
 2. Next to the SC Product Lists role with the ID, customrole_sc_product_lists, click **Edit**.
 3. Go to the **Permissions** tab and the **Lists** subtab.
 4. For the permission, Items, confirm the level is set to View.
- If the Items permission level is not View, update the level to View, then click **Save**.

Reference Product List Patch for Reference ShopFlow

 **Important:** Before applying this patch, apply the patch for [Configuration Changes for Custom Records](#).

To implement this patch, set the permissions for the product-list.ss and product-list-item.ss files and modify the commons.js file.

Step 1: Set Permissions for product-list.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Custom ShopFlow > services.

 **Important:** Replace the string, X.Y.Z, with the version of ShopFlow you are using.

3. Next to product-list.js, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Set Permissions for product-list-item.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Custom ShopFlow > services.

 **Important:** Replace the string, X.Y.Z, with the version of ShopFlow you are using.

3. Next to product-list-item.js, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.

6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 3: Modify the commons.js File

You can edit the file, `commons.js`, in the NetSuite File Cabinet. See the help topic [Editing Files in the File Cabinet](#).

1. In the `Models.js` file, find and replace the following line of code with the provided code sample.
Find the following code:

```
1 | if (this.loginRequired && !session.isLoggedIn())
2 | {
3 |     throw unauthorizedError;
4 | }
```

Replace it with the following code:

```
1 | if (this.loginRequired && !session.isLoggedIn3())
2 | {
3 |     throw unauthorizedError;
4 | }
```

2. Save and deploy the file.
1. Go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - ShopFlow X.Y.Z > Custom ShopFlow > `ssp_libraries`.



Important: Replace the string, X.Y.Z, with the version of ShopFlow you are using.

3. Create a backup of the file, `commons.js`.



Note: If `commons.js` does not exist in the `ssp_libraries` directory, make a copy from Web Site Hosting Files > Live Hosting Files > SSP Applications > Netsuite Inc. - ShopFlow X.Y.Z > Reference ShopFlow > `ssp_libraries` > `commons.js`.

4. In the `commons.js` file, find and replace the following line of code with the provided code sample.
Find the following code:

```
1 | return session.isLoggedIn();
```

Replace it with the following code:

```
1 | return session.isLoggedIn3();
```

5. Save and deploy the file.

Step 4: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.
3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, `customrecord_ns_pl_productlist`.

4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be customrecord_ns_pl_productlistitem and the access type should be Use Permission List.

Step 5: Add Item List Permission to View

1. In NetSuite, go to Setup > Users/Roles > Manage Roles.
2. Next to the SC Product Lists role with the ID, customrole_sc_product_lists, click **Edit**.
3. Go to the **Permissions** tab and the **Lists** subtab.
4. For the permission, Items, confirm the level is set to View.
If the Items permission level is not View, update the level to View, then click **Save**.

Reference Product List Patch for Reference My Account



Important: Before applying this patch, apply the patch for [Configuration Changes for Custom Records](#).

To implement this patch, set the permissions for the product-list.ss and product-list-item.ss files and modify the commons.js file.

Step 1: Set Permissions for product-list.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - My Account X.Y.Z > Custom My Account > services.



Important: Replace the string, X.Y.Z, with the version of My Account you are using.

3. Next to product-list.js, click **Edit**.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Set Permissions for product-list-item.ss File

1. In NetSuite, go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - My Account X.Y.Z > Custom My Account > services.



Important: Replace the string, X.Y.Z, with the version of My Account you are using.

3. Next to product-list-item.js, click **Edit**.

4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **SC Product List**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 3: Modify the commons.js File

You can edit the file, `commons.js`, in the NetSuite File Cabinet. See the help topic [Editing Files in the File Cabinet](#).

1. Go to Documents > Files > File Cabinet.
2. In the File Cabinet, go to Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - My Account X.Y.Z > Custom My Account > `ssp_libraries`.

 **Important:** Replace the string, X.Y.Z, with the version of My Account you are using.

3. Create a backup of the file, `commons.js`.

 **Note:** If `commons.js` does not exist in the `ssp_libraries` directory, make a copy from Web Site Hosting Files > Live Hosting Files > SSP Applications > Netsuite Inc. - ShopFlow X.Y.Z > Reference My Account > `ssp_libraries` > `commons.js`.

4. In the `commons.js` file, find and replace the following line of code with the provided code sample. Find the following code:

```
1 | return session.isLoggedIn();
```

Replace it with the following code:

```
1 | return session.isLoggedIn3();
```

5. Save and deploy the file.

Step 4: Confirm Custom Record ID and Access Type

1. In NetSuite, go to Customization > List, Records & Fields > Record Types.
2. Click the link for the custom record type, Product Lists.
3. On the Custom Record Type page for Product Lists, confirm that **ID** has the value, `customrecord_ns_pl_productlist`.
4. Confirm that **Access Type** has Use Permission List selected.
If the access type is not correct, select Use Permission List from **Access Type**.
5. Click **Save** to save any changes you made.
6. Repeat steps 1 to 5 for the following custom record type:
 - Product Lists Items – the ID should be `customrecord_ns_pl_productlistitem` and the access type should be Use Permission List.

Step 5: Add Item List Permission to View

1. In NetSuite, go to Setup > Users/Roles > Manage Roles.

2. Next to the SC Product Lists role with the ID, customrole_sc_product_lists, click **Edit**.
3. Go to the **Permissions** tab and the **Lists** subtab.
4. For the permission, Items, confirm the level is set to View.

If the Items permission level is not View, update the level to View, then click **Save**.

Add Parameters Required for Shopper Access to Suitelet Internal URLs

Applies to: SuiteCommerce Advanced | 2018.2 | 2019.1 | 2019.2

Beginning October 30, 2024, access to the Suitelet's internal URL for the Shopper role requires two additional parameters.

Visitors to your website are assigned the Shopper role to maintain their session while browsing. With the Shopper role, users can access the Suitelet URL for Suitelets with the **Available Without Login** and **Select All Roles** boxes checked if you have used the internal **URL** instead of the **External URL** in your site design. This may be done to maintain the shopper's session, including for cart items.

If you use the following versions of SuiteCommerce Advanced (SCA), you will need to update the files, CMSadapter.Impl.Categories.v3.js and Logger.js.

- SCA 2018.2
- SCA 2019.1
- SCA 2019.2

The files must be updated to include two additional parameters, &compid=<VALUE> and &ns-at=<VALUE>, in the URL. This will ensure the Suitelet remains available and maintains the session for users with the Shopper role. The <VALUE> field should match the values of the parameters in the **External URL** field on the Suitelet's script deployment record.

These patch instructions describe the steps to add the two parameters.

Be sure to follow the instructions for the version of SCA you have implemented.

- Suite Commerce Advanced versions 2018.2 and 2019.1 – [Update CMSadapter and Logger Files \(SCA 2018.2 and 2019.1\)](#)
- Suite Commerce Advanced version 2019.2 – [Update CMSadapter and Loggers Files \(SCA 2019.2\)](#)

Update CMSadapter and Logger Files (SCA 2018.2 and 2019.1)

Update CMSadapter.Impl.Categories.v3.ts File

1. In NetSuite, go to Customization > Scripting > Script Deployments.
2. Search for customdeploy_ns_sca_environment in the list of scripts and click **View**.
3. From the **External URL** field, copy the parameter information for compid and ns-at from the URL.
4. In your SCA directory, open the following file: Modules\suitecommerce\CMSadapter\JavaScript\CMSadapter.Impl.Categories.v3.js
5. ■ Search for the following method: setUpEndpoints

- At the end of the parameter of the method, `jQuery.getJSON`, add the two parameters you copied in Step 3.

For example, find the following code:

```
1 | (...) '_environment')
```

And update it to the following:

```
1 | (...) '_environment&compid=VALUE&ns-at=VALUE')
```



Note: Replace the string **VALUE** with the values of the parameters in the **External URL** field on the Suitelet's script deployment record.

- Save the file.

Update Logger.js File

- In NetSuite, go to Customization > Scripting > Script Deployments.
- Search for `customdeploy_sca_loggerendpoint` (or `customscript_cs_loggerendpoint`) in the list of scripts and click **View**.
- From the **External URL** field, copy the parameter information for `compid` and `ns-at` from the URL.
- In your SCA directory, open the following file: `Modules\suitecommerce\Instrumentation\JavaScript\Logger.js`
- Search for the following method: `sendQueues`
- At the end of the value of the variable, `URL`, add the two parameters you copied in Step 3.

For example, find the following code:

```
1 | (...) '_loggerendpoint';
```

And update it to the following:

```
1 | (...) '_loggerendpoint&compid=VALUE&ns-at=VALUE
```



Note: Replace the string **VALUE** with the values of the parameters in the **External URL** field on the Suitelet's script deployment record.

- Save the file.

Update CMSadapter and Loggers Files (SCA 2019.2)

Update CMSadapter.Impl.Categories.v3.ts File

- In NetSuite, go to Customization > Scripting > Script Deployments.
- Search for `customdeploy_ns_sca_environment` in the list of scripts and click **View**.
- From the **External URL** field, copy the parameter information for `compid` and `ns-at` from the URL.
- In your SCA directory, open the following file: `Advanced\CMSadapter\JavaScript\CMSadapter.Impl.Categories.v3.ts`
- Search for the following method: `setUpEndPoints`

- At the end of the parameter of the method, `jQuery.getJSON`, add the two parameters you copied in Step 3.

For example, find the following code:

```
1 | (...) '_environment')
```

And update it to the following:

```
1 | (...) '_environment&compid=VALUE&ns-at=VALUE')
```

Note: Replace the string `VALUE` with the values of the parameters in the **External URL** field on the Suitelet's script deployment record.

- Save the file.

Update `Loggers.Appender.ElasticLogger.ts` File

- In NetSuite, go to Customization > Scripting > Script Deployments.
- Search for `customdeploy_ns_sca_environment` (or `customscript_cs_loggerendpoint`) in the list of scripts and click **View**.
- From the **External URL** field, copy the parameter information for `compid` and `ns-at` from the URL.
- In your SCA directory, open the following file: `Commons\Loggers\JavaScript\Loggers.Appender.ElasticLogger.ts`
- Search for the following method: `sendQueues`
- At the end of the value of the variable, `relativeURL`, add the two parameters you copied in Step 3. For example, find the following code:

```
1 | (...) '_loggerendpoint';
```

And update it to the following:

```
1 | (...) '_loggerendpoint&compid=VALUE&ns-at=VALUE
```

Note: Replace the string `VALUE` with the values of the parameters in the **External URL** field on the Suitelet's script deployment record.

- Save the file.

CMS Page Type Website Records Not Displaying

Applies to: SuiteCommerce Advanced

In some implementations of SuiteCommerce Advanced (SCA) versions 2019.1 to 2023.2, CMS Page Type website records are not displaying in Commerce websites. This occurs when CMS Page records are linked to CMS Page Types with a custom record, like a blog, linked.

These patch instructions describe the steps to fix this problem so that all CMS Page settings correctly display in your website.

After you apply the patch, all CMS Page Type custom record scripts will be lazy loaded. This means that the information that is specific to that custom record will be loaded when the linked CMS page is

requested. Because custom records require a permission level of Create or higher to work in Commerce websites, the elevated permission of the Store Manager role will be enabled and set to run without login.

Be sure to apply the patch for the version of SCA you have implemented.

- Suite Commerce Advanced versions 2022.1 to 2023.2 – [CMS Page Type Website Records Not Displaying \(SCA 2022.1 – 2023.2\)](#)
- Suite Commerce Advanced versions 2019.1 to 2021.2 – [CMS Page Type Website Records Not Displaying \(SCA 2019.1 – 2021.2\)](#)

CMS Page Type Website Records Not Displaying (SCA 2022.1 – 2023.2)

i Applies to: SuiteCommerce Advanced

Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Update the CMS.json File

1. Open the following file: Advanced/CMSadapter/Configuration/CMS.json
2. Find and delete the following code:

```

1  },
2  "cms.lazySettingsLoad": {
3    "group": "integrations",
4    "subtab": "cms",
5    "title": "CMS Pages Lazy Load",
6    "description": "Enter the Custom Script Record IDs for the settings of the CMS pages you do not want to load
7    into the shopping environment short cache.",
8    "type": "array",
9    "items": {
10     "type": "string",
11     "title": "Custom Record Script Id"
12   },
13   "default": []

```

3. Save the file.

Step 2: Update the CMSadapter.Model.v3.js File

1. Open the following file: Advanced/CMSadapter/SuiteScript/CMSadapter.Model.v3.js
2. Find the following code:

```

1  'Utils',
2  'Configuration'
3  ], function(SCModel, SiteSettingsModel, __, Utils, Configuration) {

```

3. Replace it with the following code:

```

1  'Utils'
2  ], function(SCModel, SiteSettingsModel, __, Utils) {

```

4. In the same file, find and delete the following code:

```

1  const lazySettingsLoad = __.map(

```

```

2 | Configuration.get('cms.lazySettingsLoad', []),
3 | scriptId => scriptId.toUpperCase()
4 | );
5 | _.each(settingsRecords, function(ids, recname) {
6 |   if (lazySettingsLoad.indexOf(recname.toUpperCase()) === -1) {
7 |     const cr = nlapiCreateRecord(recname);
8 |     const crFilters = [
9 |       new nlobjSearchFilter('internalid', null, 'anyof', _.keys(ids))
10 |     ];
11 |     const crColumnsRaw = cr.getAllFields().filter(function(fieldname) {
12 |       return fieldname.indexOf('custrecord') === 0;
13 |     });
14 |     const crColumns = crColumnsRaw.map(function(column) {
15 |       return new nlobjSearchColumn(column);
16 |     });
17 |     const settings = nlapiSearchRecord(recname, null, crFilters, crColumns);
18 |     settings.forEach(function(setting) {
19 |       const { id } = setting;
20 |       settingsRecords[recname][id].fields = {};
21 |       _.each(crColumnsRaw, function(columnName) {
22 |         settingsRecords[recname][id].fields[
23 |           columnName
24 |         ] = setting.getValue(columnName);
25 |       });
26 |     });
27 |   }
28 | });

```

5. Save the file.

Step 3: Update the PageTypeSettings.Handler.ts File

1. Open the following file: Backend/Common/PageType/PageTypeSettings.Handler.ts
2. Find the following code:

```

1 | recSearch.run().each(function(data) {
2 |   util.each(columns, function(column) {
3 |     result[column] = data.getValue(column);
4 |   });
5 |   return true;
6 | });
7 | return result;
8 | }
9 | }

```

3. In the line between the last two curly brackets, insert the following code:

```

1 | public isCustomRecordValid(recordType: string) {
2 |   const recSearch = search
3 |     .create({
4 |       type: 'cmspagetype',
5 |       filters: ['customrecordscriptid', 'is', recordType],
6 |       columns: ['customrecordscriptid']
7 |     })
8 |     .run();
9 |   return recSearch.getRange({
10 |     start: 0,
11 |     end: 1
12 |   }).length;
13 | }

```

The updated code should look like this:

```

1 | recSearch.run().each(function(data) {
2 |   util.each(columns, function(column) {
3 |     result[column] = data.getValue(column);
4 |   });

```

```

5 |     return true;
6 |   });
7 |   return result;
8 | }
9 | public isCustomRecordValid(recordType: string) {
10 |   const recSearch = search
11 |     .create({
12 |       type: 'cmspagetype',
13 |       filters: ['customrecordscriptid', 'is', recordType],
14 |       columns: ['customrecordscriptid']
15 |     })
16 |     .run();
17 |   return recSearch.getRange({
18 |     start: 0,
19 |     end: 1
20 |   }).length;
21 | }
22 | }

```

4. Save the file.

Step 4: Update the PageTypeSettings.ServiceController.ts File

1. Open the following file: Backend/Common/PageType/PageTypeSettings.ServiceController.ts
2. Find the following code:

```

1 | import { notFoundError } from '../Common/Controller/RequestErrors';

```

3. Replace it with the following code:

```

1 | import { notFoundError, forbiddenError } from '../Common/Controller/RequestErrors';

```

4. In the same file, find the following code:

```

1 | ): HttpResponse<PageTypeSettings> {
2 |   const record: PageTypeSettings = this.PageTypeSettingsHandler.get(id, params);

```

5. Between the two lines of code, insert the following code:

```

1 |   if (!this.PageTypeSettingsHandler.isCustomRecordValid(params.recname)) {
2 |     throw forbiddenError;
3 |   }

```

The updated code should look like this:

```

1 | ): HttpResponse<PageTypeSettings> {
2 |   if (!this.PageTypeSettingsHandler.isCustomRecordValid(params.recname)) {
3 |     throw forbiddenError;
4 |   }
5 |   const record: PageTypeSettings = this.PageTypeSettingsHandler.get(id, params);

```

6. Save the file.

Step 5: Update the PageType.Component.ts File

1. Open the following file: Commons/PageType/JavaScript/PageType.Component.ts
2. Find the following code:

```

1 | const lazySettingsLoad = _.map(
2 |   Configuration.get('cms.lazySettingsLoad', []),

```



```

3 | (scriptId: string) => scriptId.toUpperCase()
4 | );
5 | if (
6 |   page.get('customrecordscriptid') &&
7 |   lazySettingsLoad.indexOf(
8 |     page.get('customrecordscriptid').toUpperCase()
9 |   ) !== -1
10 | ) {

```

3. Replace it with the following code:

```

1 | if (page.get('customrecordscriptid')) {

```

4. Save the file.

Step 6: Set Permissions for PageTypeSettings.ss

1. In NetSuite, go to Documents > Files > File Cabinet.
2. Go to the following location: Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development2 > services



Important: You must replace the string, X.Y.Z, with the version of SCA you are using.

3. Click **Edit** next to the file, PageTypeSettings.ss.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **Store Manager**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

CMS Page Type Website Records Not Displaying (SCA 2019.1 – 2021.2)

 **Applies to:** SuiteCommerce Advanced

Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Set Permissions for checkout.environment.shortcache.ssp

1. In NetSuite, go to Documents > Files > File Cabinet.
2. Go to the following location: Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development



Important: You must replace the string, X.Y.Z, with the version of SCA you are using.

3. Click **Edit** next to the file, checkout.environment.shortcache.ssp.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **Store Manager**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 2: Set Permissions for myaccount.environment.shortcache.ssp

1. In NetSuite, go to Documents > Files > File Cabinet.
2. Go to the following location: Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development



Important: You must replace the string, X.Y.Z, with the version of SCA you are using.

3. Click **Edit** next to the file, myaccount.environment.shortcache.ssp.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **Store Manager**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Step 3: Set Permissions for shopping.environment.shortcache.ssp

1. In NetSuite, go to Documents > Files > File Cabinet.
2. Go to the following location: Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - X.Y.Z > Development > public



Important: You must replace the string, X.Y.Z, with the version of SCA you are using.

3. Click **Edit** next to the file, shopping.environment.shortcache.ssp.
4. Go to the **Permission** tab and check the **Enabled** box.
5. From the **Execute as Role** list, select **Store Manager**.
6. Check the **Run Script Without Login** box.
7. Click **Save**.

Google Tag Manager Events Not Displaying in Data Layer Upon Checkout

i Applies to: SuiteCommerce Advanced

In some implementations of SuiteCommerce Advanced (SCA) 2018.2 – 2022.2 using Google Analytics 4 with the Google Tag Manager Commerce Extension, the data layer code may incorrectly track the following Google Analytics events:

- Add To Wishlist
- Add to Cart
- Product Click
- View Event Data
- Product View
- Transaction

To correct this problem, review the data layer structure of SCA version 2023.1 or later, compare it with your current version, and make modifications as required.

Note: You need to modify the file, `GoogleTagManager`, but the file extension and path varies depending on your version of SCA.

This task requires developer experience.

The following code is a sample of how the data sent to the data layer should look:

```

1  -----
2  trackAddToWishlist
3  -----
4
5  addToWishlistEventData = {
6    event: 'addToWishlist',
7    data: {
8      action: 'Add To Wishlist',
9      currencyCode: String,
10     sku: String,
11     id: String,
12     name: String,
13     price: Number,
14     category: 'Shopping - User Interaction',
15     item_category: String
16   },
17   ecommerce: {
18     items: [
19       {
20         action: 'Add To Wishlist',
21         currencyCode: String,
22         sku: String,
23         id: String,
24         name: String,
25         price: Number,
26         category: 'Shopping - User Interaction',
27         item_category: String
28       }
29     ]
30   }
31 };
32
33
34 -----
35 trackAddToCart
36 -----
37
38 addToCartEventData = {
39   event: 'addToCart',
40   ecommerce: {
41     currencyCode: String,
42     add: {
43       products: [
44         {
45           sku: String,
46           id: String,
47           name: String,
48           price: Number,
49           variant: String,
50           category: String,
51           quantity: Number,
52           currency: String
53         }
54       ]
55     }
56   }
57 };
58
59
60 -----
61 trackProductClick
62 -----
63
64 productClickEventData = {
65   event: 'productClick',

```

```

66     ecommerce: {
67         click: {
68             actionField: { list: String },
69             products: [
70                 {
71                     name: String,
72                     price: Number,
73                     sku: String,
74                     id: String,
75                     category: String,
76                     position: Number,
77                     currency: String
78                 }
79             ]
80         }
81     }
82 };
83
84 -----
85 trackPageviewForCart
86 -----
87
88 cartViewEventData = {
89     event: 'cartView',
90     data: {
91         page: String,
92         currencyCode: String,
93         total: String,
94         items: [{
95             sku: String,
96             id: String,
97             title: String,
98             price: Number,
99             quantity: Number,
100            image: String,
101            currency: String
102        }]
103    }
104 };
105
106 -----
107 trackProductView
108 -----
109
110 productViewEventData = {
111     event: 'productView',
112     ecommerce: {
113         detail: {
114             actionField: {
115                 list: String
116             },
117             products: [
118                 {
119                     name: String,
120                     sku: String,
121                     id: String,
122                     category: String,
123                     variant: String,
124                     price: Number,
125                     currency: String,
126                     item_list_name: String
127                 }
128             ]
129         }
130     },
131     page: String
132 };
133
134 -----
135 trackTransaction

```

```

139 -----
140
141 transactionEventData = {
142     event: 'transaction',
143     ecommerce: {
144         purchase: {
145             actionField: {
146                 id: String,
147                 affiliation: String,
148                 revenue: Number,
149                 currency: String,
150                 subtotal: Number,
151                 tax: Number,
152                 shipping: Number,
153                 coupon: [
154                     String
155                 ]
156             },
157             products: [{
158                 name: String,
159                 sku: String,
160                 id: String,
161                 price: Number,
162                 category: String,
163                 variant: String,
164                 quantity: Number,
165                 currency: String
166             }]
167         }
168     }
169 };

```

Customer not Directed to the Payment Review Page after Checkout using the Checkout Flow

i Applies to: Kilimanjaro | 2018.2 | Site Builder

In the following implementations, customers are not directed to the payment review page after checking out even when Windcave is the only configured payment method:

- SiteBuilder Extensions Kilimanjaro
- SiteBuilder Extensions Premium Kilimanjaro
- SuiteCommerce Advanced (SCA) Kilimanjaro
- SCA 2018.2
- SCA 2020.1
- SCA 2020.1.12
- SCA 2021.2
- SCA 2021.2.3

These patch instructions describe the steps to fix this problem so that customers are directed to the correct payment review page.

To implement this patch, create a custom module to override the `OrderWizard.Module.PaymentMethod.Selector.js` JavaScript file which is part of the `OrderWizard.Module.PaymentMethod` module.

If you are not familiar with implementing patches for SuiteCommerce Advanced, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).
- For step by step instructions, refer to [Patch Using Override Mode](#).

Step 1: Create the Override File

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/OrderWizard.Module.PaymentMethod@1.0.0/JavaScript/OrderWizard.Module.PaymentMethod.Selector.js
2. In the OrderWizard.Module.PaymentMethod.Selector.js file, in the setModuleByType() method, find the following code:

```
1 | if (!this.selectedModule)
2 | {
3 |   this.selectedModule = _.first(this.modules);
4 | }
```

And replace it with the following code:

```
1 | if (!this.selectedModule)
2 | {
3 |   this.selectedModule = _.findWhere(this.modules, { isActive: true });
4 | }
```

3. In the render() method of the same file, find the following line of code:

```
1 | var selected_payment = this.model.get('paymentmethods').findWhere({primary: true})
```

And replace it with the following code:

```
1 | var selected_payment = this.wizard.options.profile.get('creditcards').length > 0 &&
2 | this.model.get('paymentmethods').findWhere({primary: true})
```

4. From the render() method, move the following code to the end of the initialize() method:

```
1 |   _.each(this.modules, function (module)
2 |   {
3 |     module.isActive = module.instance.isActive();
4 |   })
```

5. Finally, in the getContext() method, find the following code:

```
1 | module.isSelected = (self.selectedModule.type === module.type);
```

And replace it with the following code:

```
1 | module.isSelected = (self.selectedModule && self.selectedModule.type === module.type);
```

6. Save the file.

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, you should:

1. Paste the following sample code into the new ns.package.json file that you create in the Modules directory: Modules/extensions/OrderWizard@1.0.0/ns.package.json

```
1 | {
2 |   "gulp": {
3 |     "javascript": [
```

```

4 |         "JavaScript/*.js"
5 |     ],
6 | },
7 | "overrides": {
8 |     "suitecommerce/OrderWizard@X.Y.Z/JavaScript/PaymentMethod.Selector.js" : "JavaScript/PaymentMethod.Selector.js"
9 | }
10| }

```



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

2. Open the `distro.json` file in the root SuiteCommerce Advanced development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the `"modules"` key.

```

1 | "modules": {
2 |     "extensions/OrderWizard.Module.PaymentMethod": "1.0.0",
3 |     . . .

```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Customer not Directed to the Payment Review Page after Checkout after using the Make A Payment Flow

Applies to: Kilimanjaro | 2018.2 | Site Builder

In the following implementations, customers are not directed to the payment review page after checking out using the Make A Payment flow, even when Windcave is the only configured payment method:

- SiteBuilder Extensions Kilimanjaro
- SiteBuilder Extensions Premium Kilimanjaro
- SuiteCommerce Advanced (SCA) Kilimanjaro
- SCA 2018.2
- SCA 2020.1
- SCA 2020.1.12
- SCA 2021.2
- SCA 2021.2.3

These patch instructions describe the steps to fix this problem so that customers are directed to the correct payment review page.

To implement this patch, create a custom module to override the `OrderWizard.Module.PaymentMethod.Selector.js` JavaScript file which is part of the `OrderWizard.Module.PaymentMethod` module.

If you are not familiar with implementing patches for SuiteCommerce Advanced, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).

- For step by step instructions, refer to [Patch Using Override Mode](#).

Step 1: Create the Override File

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/suitecommerce/ExternalPayment@1.0.0/SuiteScript/ExternalPayment.Model.js
2. In the new ExternalPayment.Model.js file, in the _isDone method, find the following line of code:

```
1 | , status_hold_value = this._getConfiguration(record_type, 'statusHoldValue', 'HOLD');
```

And replace it with the following code:

```
1 | , status_hold_value = this._getConfiguration(record_type, 'statusHoldValue', 'HOLD')
2 | , status_success_value = this._getConfiguration(record_type, 'statusSuccessValue', 'SUCCESS');
```

3. In the same file, find the following line of code:

```
1 | return status === status_accept_value || status === status_hold_value;
```

And replace it with the following code:

```
1 | return status === status_accept_value || status === status_hold_value || status === status_success_value;
```

4. In the _getStatusFromRequest method, find the following line of code:

```
1 | return request.getParameter(this._getConfiguration(record_type, 'statusParameterName', 'status'));
```

And replace it with the following code:

```
1 | var default_status = 'status';
2 | if(record_type === 'customerpayment') {
3 |   default_status = 'nlstatus';
4 | }
5 | return request.getParameter(this._getConfiguration(record_type, 'statusParameterName', default_status))
```

5. In the _validateStatusFromRequest method, find the following line of code:

```
1 | , status_reject_value = this._getConfiguration(record_type, 'statusRejectValue', 'REJECT');
```

And replace it with the following code:

```
1 | , status_reject_value = this._getConfiguration(record_type, 'statusRejectValue', 'REJECT')
2 | , status_success_value = this._getConfiguration(record_type, 'statusAcceptValue', 'SUCCESS');
```

6. In the same file, find the following line of code:

```
1 | return status === status_accept_value || status === status_hold_value || status === status_reject_value;
```

And replace it with the following code:

```
1 | return status === status_accept_value || status === status_hold_value || status === status_reject_value || status ===
   | status_success_value;
```

7. In the _getDataFromRedirect method, find the following line of code:

```
1 | , configuration_parameters = this._getConfiguration(record_type, 'parameters', ['tranid', 'authcode', 'status'])
```

And replace it with the following code:


```
1 | , configuration_parameters = this._getConfiguration(record_type, 'parameters' , ['tranid', 'result', 'authcode', 'status', 'nlstatus'])
```

8. Save the file.

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you create in the Modules directory: `/Modules/extensions/SspLibraries@1.0.0/ns.package.json`

```
1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ExternalPayment@X.Y.Z/SuiteScript/ExternalPayment.Model.js" : "SuiteScript/ExternalPayment.Model.js"
9 |   }
10 | }
```



Important: You must replace the string `X.Y.Z` with the version of the module in your implementation of SuiteCommerce Advanced.

2. Open the `distro.json` file in the root SuiteCommerce Advanced development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the `"modules"` key.

```
1 | "modules": {
2 |   "extensions/ExternalPaymentExtension": "1.0.0",
3 |   ...
}
```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Guide to Enable Google Analytics 4

i Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus | Kilimanjaro | Aconcagua | 2018.2 | 2019.1 | 2019.2 | 2020.1 | 2020.2 | 2021.1

Google will end data collection for the Google Universal Analytics (GUA) service on July 1, 2023, and the feature will be replaced with Google Analytics 4 (GA4). GA4 support comes standard with SuiteCommerce, SuiteCommerce MyAccount, and SuiteCommerce Advanced (SCA) versions 2022.2 and later.

For earlier versions of SCA, you can implement GA4 support by following the instructions provided in the following guides. Be sure to follow the instructions for the version of SCA you have implemented.

SCA Implementation	Instructions
2022.1	Enable Google Analytics 4 (SCA 2019.2 – 2022.1)

SCA Implementation	Instructions
2021.2	
2021.1	
2020.2	
2020.1	
2019.2	
2019.1	Enable Google Analytics 4 (SCA Vinson – 2019.1)
2018.2	
Aconcagua	
Kilimanjaro	
Elbrus	
Vinson	
Mont Blanc	Enable Google Analytics 4 (Denali – Mont Blanc)
Denali	

Note: Do not implement GA4 if you use GA4 with Google Tag Manager (GTM) because this causes duplicate tracking results. See the help topic [Google Tag Manager](#) for information about GTM.

Enable Google Analytics 4 (SCA 2019.2 – 2022.1)

Applies to: 2019.2 | 2020.1 | 2020.2 | 2021.1

Prepare your SuiteCommerce Advanced (SCA) implementation of Google Analytics 4 (GA4) without Google Tag Manager (GTM) as described in the following steps. You need to create new folders, files, and modules, so be sure to complete all of the required steps as outlined in the following procedures:

- [Prepare GoogleAnalytics4 Folders Structure](#)
- [Create GoogleAnalytics4.json File](#)
- [Create GoogleAnalytics4.ts File](#)
- [Create ns.package.json File](#)
- [Add GoogleAnalytics4 Module](#)
- [Add entryPoints Module to Application Dependencies](#)
- [Test and Deploy Your Customizations](#)

Prepare GoogleAnalytics4 Folders Structure

1. Go to `.../SC_XX.X(_Live)/Advanced` where `XX.X` is your version of SCA. The `_Live` folder will be present depending on your version of SCA.
2. In the Advanced folder, create a new folder called, `GoogleAnalytics4`.
3. In the `GoogleAnalytics4` folder, create folders and files using the following structure:

```

1 | SC_XX.X(_Live)/
2 |   └─ Advanced/
3 |       └─ GoogleAnalytics4/
4 |           └─ Configuration/
5 |               └─ GoogleAnalytics4.json
6 |           └─ JavaScript/

```

```

7 |   └─ GoogleAnalytics4.ts
8 |   └─ ns.package.json

```

Create GoogleAnalytics4.json File

1. Go to the Configuration folder in the structure you created in [Prepare GoogleAnalytics4 Folders Structure](#).
2. Create and save the GoogleAnalytics4.json file with the following code:

```

1 | {
2 |   "type": "object",
3 |   "subtab": {
4 |     "id": "analyticsFour",
5 |     "title": "Google Analytics 4",
6 |     "docRef": "bridgehead_4667032443",
7 |     "description": "Google Analytics 4",
8 |     "group": "integrations"
9 |   },
10 |   "properties": {
11 |     "tracking.googleAnalyticsFour.propertyID": {
12 |       "group": "integrations",
13 |       "subtab": "analyticsFour",
14 |       "type": "string",
15 |       "title": "Google Analytics 4 ID",
16 |       "description": "Enter the Google Analytics 4 Tracking ID. SuiteCommerce Advanced inserts this property into
17 |       each page of the application. To locate the tracking ID, login to your Google Analytics 4 account and go to Admin > Data
18 |       Streams > select your data stream > MEASUREMENT ID.",
19 |       "default": ""
20 |     },
21 |     "tracking.googleAnalyticsFour.domainName": {
22 |       "group": "integrations",
23 |       "subtab": "analyticsFour",
24 |       "type": "string",
25 |       "title": "Google Analytics 4 domain",
26 |       "description": "Enter the domain used for Google Analytics. The value you enter depends on which application
27 |       you want to use Google Analytics",
28 |       "default": ""
29 |     },
30 |     "tracking.googleAnalyticsFour.domainNameSecure": {
31 |       "group": "integrations",
32 |       "subtab": "analyticsFour",
33 |       "type": "string",
34 |       "title": "Google Analytics 4 secure domain",
35 |       "description": "Enter the secure domain used for Google Analytics 4. Possible values are the same as the
36 |       Google Analytics Domain property.",
37 |       "default": ""
38 |     }
39 |   }
40 | }

```

Create GoogleAnalytics4.ts File

1. Go to the JavaScript folder in the structure you created in [Prepare GoogleAnalytics4 Folders Structure](#).
2. Create the TypeScript file, GoogleAnalytics4.ts, with the following code:

```

1 | ///
2 |
3 | /* Same imports as in the file Advanced\GoogleUniversalAnalytics\JavaScript\GoogleUniversalAnalytics.ts */
4 |
5 | export = (function(win: any, name) {
6 |   // @class GoogleAnalytics4 @extends ApplicationModule
7 |   // -----
8 |   // Loads google analytics 4 script and extends application with methods:
9 |   // * trackPageview
10 |  // * trackEvent

```

```

11 // * trackTransaction
12 // Also wraps layout's showModal
13
14 interface GA4Options {
15     send_page_view?: boolean;
16     client_id?: string;
17     session_id?: string;
18 }
19
20 const dataLayerName = name + 'DataLayer';
21
22 const GoogleAnalytics4: any = {
23     // @method doCallback Indicates if this module do a callback after some particular events
24     // @return {Boolean}
25     doCallback: function() {
26         return !!win[name];
27     },
28
29     // @method trackPageview
30     // @param {String} url
31     // @return {GoogleAnalytics4}
32     trackPageview: function(url) {
33         if (!_.isString(url)) {
34             url = url.split('?')[0];
35             win.gtag('event', 'page_view', { page_location: url, page_title: url });
36         }
37
38         return this;
39     },
40
41     trackLogin: function(event) {
42         win.gtag('event', 'login', { method: event.category, event_callback: event.callback });
43
44         return this;
45     },
46
47     trackRegister: function(event) {
48         win.gtag('event', 'sign_up', {
49             method: event.category,
50             event_callback: event.callback
51         });
52
53         return this;
54     },
55     trackCheckoutAsGuest: function(event) {
56         win.gtag('event', 'checkout_as_guest', {
57             method: event.category,
58             event_callback: event.callback
59         });
60
61         return this;
62     },
63
64     // @method trackEvent
65     // @param {TrackEvent} event
66     // @return {GoogleAnalytics}
67     trackEvent: function(event) {
68         if (event && event.category && event.action) {
69             win.gtag('event', 'generic_event', {
70                 category: event.category,
71                 action: event.action,
72                 label: event.label,
73                 value: parseFloat(event.value) || 0,
74                 page: event.page || '/' + Backbone.history.fragment,
75                 event_callback: event.callback
76             });
77         }
78
79         return this;
80     },
81
82     // @method trackTransaction
83     // Based on the created SalesOrder we trigger each of the analytics

```

```

84 // ecommerce methods passing the required information
85 // @param {Tracker.Transaction.Model} @extends Backbone.Model transaction
86 // @return {GoogleAnalytics|Void}
87 trackTransaction: function(transaction) {
88     const transaction_id = transaction.get('confirmationNumber');
89
90     const purchase = {
91         transaction_id: transaction_id,
92         value: transaction.get('subTotal'),
93         tax: transaction.get('taxTotal'),
94         shipping: transaction.get('shippingCost') + transaction.get('handlingCost'),
95         currency:
96             (SC.ENVIRONMENT.currentCurrency && SC.ENVIRONMENT.currentCurrency.code) || '',
97         affiliation: Configuration.get('siteSettings.displayName'),
98         items: []
99     };
100
101     transaction.get('products').each(function(product, index) {
102         purchase.items.push({
103             item_id: product.get('id'),
104             item_name: product.get('name'),
105             affiliation: Configuration.get('siteSettings.displayName'),
106             currency:
107                 (SC.ENVIRONMENT.currentCurrency && SC.ENVIRONMENT.currentCurrency.code) ||
108                 '',
109             index: index,
110             item_category: product.get('category') || '',
111             item_variant: product.get('options'),
112             price: product.get('rate'),
113             quantity: product.get('quantity')
114         });
115     });
116
117     win.gtag('event', 'purchase', purchase);
118
119     return this;
120 },
121
122 // @method addCrossDomainParameters
123 // @param {string} url
124 // @return {String} url
125 addCrossDomainParameters: function(url) {
126     // We only need to add the parameters if the URL we are trying to go
127     // is not a sub domain of the tracking domain
128     if (!_isString(url)) {
129         url = Utils.addParamsToUrl(url, {
130             client_id: win[name].client_id,
131             session_id: win[name].session_id
132         });
133     }
134
135     return url;
136 },
137
138 // @method loadScript
139 // @return {jQuery.Promise|Void}
140 loadScript: function(id: string) {
141     return jQuery.getScript(
142         `//www.googletagmanager.com/gtag/js?id=${id}&l=${dataLayerName}`
143     );
144 },
145
146 // @method mountToApp
147 // @param {ApplicationSkeleton} application
148 // @return {Void}
149 mountToApp: function(application) {
150     const tracking = Configuration.get('tracking.googleAnalyticsFour');
151
152     // if track page view needs to be tracked
153     if (tracking && tracking.propertyID && !SC.isPageGenerator()) {
154         win[dataLayerName] = win[dataLayerName] || [];
155         win.gtag =
156             win.gtag ||

```

```

157         function() {
158             win[dataLayerName].push(arguments);
159         };
160         const domains = [];
161         if (tracking.domainName) {
162             domains.push(tracking.domainName);
163         }
164         if (tracking.domainNameSecure) {
165             domains.push(tracking.domainNameSecure);
166         }
167
168         if (domains.length) {
169             win.gtag('set', 'linker', {
170                 domains: domains
171             });
172         }
173
174         const configOptions: GA4Options = {
175             send_page_view: false
176         };
177
178         const client_id = Utils.getParameterByName(win.location.href, 'client_id');
179         const session_id = Utils.getParameterByName(win.location.href, 'session_id');
180
181         if (client_id && session_id) {
182             configOptions.client_id = client_id;
183             configOptions.session_id = session_id;
184         }
185
186         win.gtag('js', new Date());
187         win.gtag('config', tracking.propertyID, configOptions);
188
189         // Please keep this statement here or the first pageView on site load
190         // will not be triggered (ApplicationSkeleton.Layout._showContent)
191         Tracker.getInstance().trackers.push(GoogleAnalytics4);
192
193         // the analytics script is only loaded if we are on a browser
194         application.getLayout().once('afterAppendView', function() {
195             GoogleAnalytics4.loadScript(tracking.propertyID)
196                 .done(function() {
197                     win[name] = {};
198                     win.gtag('get', tracking.propertyID, 'client_id', client_id => {
199                         win[name].client_id = client_id;
200                     });
201                     win.gtag('get', tracking.propertyID, 'session_id', session_id => {
202                         win[name].session_id = session_id;
203                     });
204                 })
205                 .fail(function(jqXHR) {
206                     jqXHR.preventDefault = true;
207                     console.warn('Google Analytics 4 was unable to load');
208                 });
209             });
210         }
211     };
212
213     return GoogleAnalytics4;
214 })(window, 'ga4');
215

```

3. Open the file, .../Advanced/GoogleUniversalAnalytics/JavaScript/GoogleUniversalAnalytics.ts and copy the imports.
4. In the GoogleAnalytics4.ts file, find the following comment:

```
1 | /* Same imports as in the file Advanced\GoogleUniversalAnalytics\JavaScript\GoogleUniversalAnalytics.ts */
```

And replace it with the imports you copied.

The following sample shows what the imports might look like:

```

1 import * as _ from 'underscore';
2 import * as Utils from '../../Commons/Utilities/JavaScript/Utils';
3 import * as jQuery from '../../Commons/Core/JavaScript/jquery';
4 import { Configuration } from '../../SCA/JavaScript/Configuration';
5
6 import Tracker = require('../../Commons/Tracker/JavaScript/Tracker');
7 import Backbone = require('../../Commons/Utilities/JavaScript/backbone.custom');

```

5. Save the GoogleAnalytics4.ts file.

Create ns.package.json File

1. Go to the GoogleAnalytics4 folder in the structure you created in [Prepare GoogleAnalytics4 Folders Structure](#).
2. Create and save the json file, ns.package.json, with the following code:

```

1 {
2   "gulp": {
3     "javascript": ["JavaScript/*.ts"],
4     "configuration": ["Configuration/*.json"]
5   }
6 }

```

Add GoogleAnalytics4 Module

1. Open the distro.json file in the ../Advanced directory and add GoogleAnalytics4 to the list of modules. The following sample shows the value to add to the list of existing values that follow the "modules" key:

```

1 {
2   ...
3   "modules": [
4     "GoogleAnalytics4",
5     ...
6   ],
7   ...
8 }

```

2. Save your changes.

Add entryPoints Module to Application Dependencies

1. Open the distro.json file in the ../Advanced directory and verify if the entryPoints modules for the Shopping, Checkout, and MyAccount applications have dependencies attributes.

The following sample shows what the entryPoints code will look like for the Shopping application:

```

1 {
2   ...
3   "tasksConfig": {
4     ...
5     "javascript": {
6       "entryPoints": [
7         {
8           "names": ["SC.Shopping.Starter.js"],
9           "exportFile": "shopping.js",
10          "dependencies": [
11            ...
12          ]
13        },
14        ...
15      ]
16    }
17  }
18 }

```

```

17 | ...
18 | }

```

2. Add "GoogleAnalytics4" to the list of dependencies for each application.

The following sample shows what your updated code will look like for the Shopping application:

```

1 | {
2 |   ...
3 |   "tasksConfig": {
4 |     ...
5 |     "javascript": {
6 |       "entryPoints": [
7 |         {
8 |           "names": ["SC.Shopping.Starter.js"],
9 |           "exportFile": "shopping.js",
10 |          "dependencies": [
11 |            "GoogleAnalytics4",
12 |            ...
13 |          ]
14 |        },
15 |        ...
16 |        ...
17 |        ...
18 |        ...
19 |      ]

```

3. Save the `distro.json` file.
4. If the `entryPoints` modules for the Shopping, Checkout, and MyAccount applications do not have dependencies attributes, you need to update the dependencies files for each application.
5. Go to `.../Advanced/CheckoutApplication/JavaScript/` and open the TypeScript file, `SC.Checkout.Starter.Dependencies.ts`.
6. Add the following import:

```

1 | import GoogleAnalytics4 = require('../../GoogleAnalytics4/JavaScript/GoogleAnalytics4');

```

7. Add the following dependency to the `entryPointModules` array:

```

1 | export const entryPointModules = [
2 |   GoogleAnalytics4,
3 |   ...
4 | ]

```

8. Save the `SC.Checkout.Starter.Dependencies.ts` file.
9. Go to `.../Advanced/ShoppingApplication/JavaScript` and open the TypeScript file `SC.Shopping.Starter.Dependencies.ts`.
10. Add the following import:

```

1 | import GoogleAnalytics4 = require('../../GoogleAnalytics4/JavaScript/GoogleAnalytics4');

```

11. Add the following dependency to the `entryPointModules` array:

```

1 | export const entryPointModules = [
2 |   GoogleAnalytics4,
3 |   ...
4 | ]

```

12. Save the `SC.Shopping.Starter.Dependencies.ts` file.
13. Go to `.../Advanced/MyAccountApplication/SCA/JavaScript` and open the TypeScript file, `SC.MyAccount.Starter.Dependencies.ts`.
14. Add the following import:


```
1 | import GoogleAnalytics4 = require('../../GoogleAnalytics4/JavaScript/GoogleAnalytics4');
```

15. Add the following dependency to the `entryPointModules` array:

```
1 | export const entryPointModules = [
2 |   GoogleAnalytics4,
3 |   ...
4 | ]
```

16. Save the `SC.MyAccount.Starter.Dependencies.ts` file.

Test and Deploy Your Customizations

Follow the instructions provided in [Test SCA Customizations on a Local Server](#) and [Deploy SCA Customizations to NetSuite](#) to test and deploy your customizations.

Enable Google Analytics 4 (SCA Vinson – 2019.1)

i Applies to: Vinson | Elbrus | Kilimanjaro | Aconcagua | 2018.2 | 2019.1

Prepare your SuiteCommerce Advanced (SCA) implementation of Google Analytics 4 (GA4) without Google Tag Manager (GTM) as described in the following steps. You need to create new folders, files, and modules, so be sure to complete all of the required steps as outlined in the following procedures:

- [Prepare GoogleAnalytics4@1.0.0 Folders Structure](#)
- [Create GoogleAnalytics4.json File](#)
- [Create GoogleAnalytics4.js File](#)
- [Create ns.package.json File](#)
- [Add GoogleAnalytics4 Module](#)
- [Add entryPoints Module to Application Dependencies](#)
- [Test and Deploy Your Customizations](#)

Prepare GoogleAnalytics4@1.0.0 Folders Structure

1. Go to `.../Modules/suitecommerce`.
2. In the `suitecommerce` folder, create a new folder called `GoogleAnalytics4@1.0.0`.
3. In the `GoogleAnalytics4@1.0.0` folder, create folders and files using the following structure:

```
1 | Modules/
2 |   └─ suitecommerce/
3 |     └─ GoogleAnalytics4@1.0.0/
4 |         └─ Configuration/
5 |             └─ GoogleAnalytics4.json
6 |         └─ JavaScript/
7 |             └─ GoogleAnalytics4.js
8 |         └─ ns.package.json
```

Create GoogleAnalytics4.json File

1. Go to the `Configuration` folder in the structure you created in [Prepare GoogleAnalytics4@1.0.0 Folders Structure](#).
2. Create and save the json file, `GoogleAnalytics4.json`, with the following code:

```
1 | {
2 |   "type": "object",
```

```

3      "subtab": {
4          "id": "analyticsFour",
5          "title": "Google Analytics 4",
6          "docRef": "bridgehead_4667032443",
7          "description": "Google Analytics 4",
8          "group": "integrations"
9      },
10     "properties": {
11         "tracking.googleAnalyticsFour.propertyID": {
12             "group": "integrations",
13             "subtab": "analyticsFour",
14             "type": "string",
15             "title": "Google Analytics 4 ID",
16             "description": "Enter the Google Analytics 4 Tracking ID. SuiteCommerce Advanced inserts this property into
each page of the application. To locate the tracking ID, login to your Google Analytics 4 account and go to Admin > Data
Streams > select your data stream > MEASUREMENT ID.",
17             "default": ""
18         },
19         "tracking.googleAnalyticsFour.domainName": {
20             "group": "integrations",
21             "subtab": "analyticsFour",
22             "type": "string",
23             "title": "Google Analytics 4 domain",
24             "description": "Enter the domain used for Google Analytics. The value you enter depends on which application
you want to use Google Analytics",
25             "default": ""
26         },
27         "tracking.googleAnalyticsFour.domainNameSecure": {
28             "group": "integrations",
29             "subtab": "analyticsFour",
30             "type": "string",
31             "title": "Google Analytics 4 secure domain",
32             "description": "Enter the secure domain used for Google Analytics 4. Possible values are the same as the
Google Analytics Domain property.",
33             "default": ""
34         }
35     }
36 }

```

Create GoogleAnalytics4.js File

1. Go to the JavaScript folder in the structure you created in [Prepare GoogleAnalytics4@1.0.0 Folders Structure](#).
2. Create the JavaScript file, GoogleAnalytics4.js, with the following code:

```

1  // @module GoogleAnalytics4
2  (function(win, name) {
3      // @class GoogleAnalytics4 @extends ApplicationModule
4      // -----
5      // Loads google analytics 4 script and extends application with methods:
6      // * trackPageview
7      // * trackEvent
8      // * trackTransaction
9      // Also wraps layout's showInModal
10     define('GoogleAnalytics4', [
11         'Tracker',
12         'underscore',
13         'jQuery',
14         'Backbone',
15         'Utils',
16         'SC.Configuration'
17     ], function(Tracker, _, jQuery, Backbone, Utils, Configuration) {
18         var dataLayerName = name + 'DataLayer';
19         var GoogleAnalytics4 = {
20             // @method doCallback Indicates if this module do a callback after some particular events
21             // @return {Boolean}
22             doCallback: function() {
23                 return !!win[name];
24             },
25         },

```

```

26 // @method trackPageview
27 // @param {String} url
28 // @return {GoogleAnalytics4}
29 trackPageview: function(url) {
30     if (_.isString(url)) {
31         url = url.split('?')[0];
32         win.gtag('event', 'page_view', { page_location: url, page_title: url });
33     }
34
35     return this;
36 },
37
38 trackLogin: function(event) {
39     win.gtag('event', 'login', {
40         method: event.category,
41         event_callback: event.callback
42     });
43
44     return this;
45 },
46
47 trackRegister: function(event) {
48     win.gtag('event', 'sign_up', {
49         method: event.category,
50         event_callback: event.callback
51     });
52
53     return this;
54 },
55 trackCheckoutAsGuest: function(event) {
56     win.gtag('event', 'checkout_as_guest', {
57         method: event.category,
58         event_callback: event.callback
59     });
60
61     return this;
62 },
63
64 // @method trackEvent
65 // @param {TrackEvent} event
66 // @return {GoogleAnalytics}
67 trackEvent: function(event) {
68     if (event && event.category && event.action) {
69         win.gtag('event', 'generic_event', {
70             category: event.category,
71             action: event.action,
72             label: event.label,
73             value: parseFloat(event.value) || 0,
74             page: event.page || "/" + Backbone.history.fragment,
75             event_callback: event.callback
76         });
77     }
78
79     return this;
80 },
81
82 // @method trackTransaction
83 // Based on the created SalesOrder we trigger each of the analytics
84 // ecommerce methods passing the required information
85 // @param {Tracker.Transaction.Model} @extends Backbone.Model transaction
86 // @return {GoogleAnalytics|Void}
87 trackTransaction: function(transaction) {
88     var transaction_id = transaction.get('confirmationNumber');
89
90     var purchase = {
91         transaction_id: transaction_id,
92         value: transaction.get('subTotal'),
93         tax: transaction.get('taxTotal'),
94         shipping: transaction.get('shippingCost') + transaction.get('handlingCost'),
95         currency:
96             (SC.ENVIRONMENT.currentCurrency && SC.ENVIRONMENT.currentCurrency.code) ||
97             '',
98         affiliation: Configuration.get('siteSettings.displayName'),

```

```

99         items: []
100     };
101
102     transaction.get('products').each(function(product, index) {
103         purchase.items.push({
104             item_id: product.get('id'),
105             item_name: product.get('name'),
106             affiliation: Configuration.get('siteSettings.displayName'),
107             currency:
108                 (SC.ENVIRONMENT.currentCurrency &&
109                  SC.ENVIRONMENT.currentCurrency.code) ||
110                 '',
111             index: index,
112             item_category: product.get('category') || '',
113             item_variant: product.get('options'),
114             price: product.get('rate'),
115             quantity: product.get('quantity')
116         });
117     });
118
119     win.gtag('event', 'purchase', purchase);
120
121     return this;
122 },
123
124 // @method addCrossDomainParameters
125 // @param {string} url
126 // @return {String} url
127 addCrossDomainParameters: function(url) {
128     // We only need to add the parameters if the URL we are trying to go
129     // is not a sub domain of the tracking domain
130     if (_.isString(url)) {
131         url = Utils.addParamsToUrl(url, {
132             client_id: win[name].client_id,
133             session_id: win[name].session_id
134         });
135     }
136
137     return url;
138 },
139
140 // @method loadScript
141 // @return {jQuery.Promise|Void}
142 loadScript: function(id) {
143     return jQuery.getScript(
144         '//' + www.googletagmanager.com/gtag/js?id=' + id + '&l=' + dataLayerName
145     );
146 },
147
148 // @method mountToApp
149 // @param {ApplicationSkeleton} application
150 // @return {Void}
151 mountToApp: function(application) {
152     var tracking = Configuration.get('tracking.googleAnalyticsFour');
153
154     // if track page view needs to be tracked
155     if (tracking && tracking.propertyID && !SC.isPageGenerator()) {
156         win[dataLayerName] = win[dataLayerName] || [];
157         win.gtag =
158             win.gtag ||
159             function() {
160                 win[dataLayerName].push(arguments);
161             };
162         var domains = [];
163         if (tracking.domainName) {
164             domains.push(tracking.domainName);
165         }
166         if (tracking.domainNameSecure) {
167             domains.push(tracking.domainNameSecure);
168         }
169
170         if (domains.length) {
171             win.gtag('set', 'linker', {

```

```

172         domains: domains
173     });
174 }
175
176     var configOptions = {
177         send_page_view: false
178     };
179
180     var client_id = Utils.getParameterByName(win.location.href, 'client_id');
181     var session_id = Utils.getParameterByName(win.location.href, 'session_id');
182
183     if (client_id && session_id) {
184         configOptions.client_id = client_id;
185         configOptions.session_id = session_id;
186     }
187
188     win.gtag('js', new Date());
189     win.gtag('config', tracking.propertyID, configOptions);
190
191     // Please keep this statement here or the first pageView on site load
192     // will not be triggered (ApplicationSkeleton.Layout._showContent)
193     Tracker.getInstance().trackers.push(GoogleAnalytics4);
194
195     // the analytics script is only loaded if we are on a browser
196     application.getLayout().once('afterAppendView', function() {
197         GoogleAnalytics4.loadScript(tracking.propertyID)
198             .done(function() {
199                 win[name] = {};
200                 win.gtag('get', tracking.propertyID, 'client_id', function(client_id) {
201                     win[name].client_id = client_id;
202                 });
203                 win.gtag('get', tracking.propertyID, 'session_id', function(session_id) {
204                     win[name].session_id = session_id;
205                 });
206             })
207             .fail(function(jqXHR) {
208                 jqXHR.preventDefault = true;
209                 console.warn('Google Analytics 4 was unable to load');
210             });
211     });
212 }
213 }
214 };
215
216     return GoogleAnalytics4;
217 });
218 })(window, 'ga4');

```

Create ns.package.json File

1. Go to the GoogleAnalytics4 folder in the structure you created in [Prepare GoogleAnalytics4@1.0.0 Folders Structure](#).
2. Create and save the json file, ns.package.json, with the following code:

```

1  {
2      "gulp": {
3          "javascript": ["JavaScript/*.js"],
4          "configuration": ["Configuration/*.json"]
5      }
6  }

```

Add GoogleAnalytics4 Module

1. Open the distro.json file in the root SCA development directory and add the GoogleAnalytics4 module to the list of modules. The following sample shows the value to add to the list of existing values that follow the "modules" key:

```

1 {
2   ...
3   "modules": {
4     "suitecommerce/GoogleAnalytics4": "1.0.0",
5     ...
6   },
7   ...
8 }

```

2. Save your changes.

Add entryPoints Module to Application Dependencies

1. Open the `distro.json` file in the root SCA development directory and add "GoogleAnalytics4" to the dependencies attribute in the entryPoints for the Shopping, Checkout and MyAccount applications. The following sample shows what your updated code will look like:

```

1 {
2   ...
3   "tasksConfig": {
4     ...
5     "javascript": {
6       "entryPoints": [
7         {
8           "names": ["SC.Shopping.Starter.js"],
9           "exportFile": "shopping.js",
10          "dependencies": [
11            "GoogleAnalytics4",
12            ...
13          ]
14        },
15        ...
16      ],
17      ...
18    },
19  }

```

2. Save the `distro.json` file.

Test and Deploy Your Customizations

Follow the instructions provided in [Test SCA Customizations on a Local Server](#) and [Deploy SCA Customizations to NetSuite](#) to test and deploy your customizations.

Enable Google Analytics 4 (Denali – Mont Blanc)

i Applies to: Denali | Mont Blanc

Prepare your SuiteCommerce Advanced (SCA) implementation of Google Analytics 4 (GA4) without Google Tag Manager (GTM) as described in the following steps. You need to create new folders, files, and modules, so be sure to complete all of the required steps as outlined in the following procedures:

- [Prepare GoogleAnalytics4@1.0.0 Folders Structure](#)
- [Create GoogleAnalytics4.js File](#)
- [Create ns.package.json File](#)
- [Add entryPoints Module to Application Dependencies](#)
- [Add googleAnalyticsFour to Tracking Property](#)
- [Test and Deploy Your Customizations](#)

Prepare GoogleAnalytics4@1.0.0 Folders Structure

1. Go to .../Modules/suitecommerce.
2. In the suitecommerce folder, create a new folder called GoogleAnalytics4@1.0.0.
3. In the GoogleAnalytics4@1.0.0 folder, create folders and files using the following structure:

```

1 Modules/
2   └─ suitecommerce/
3     └─ GoogleAnalytics4@1.0.0/
4         └─ JavaScript/
5             └─ GoogleAnalytics4.js
6             └─ ns.package.json

```

Create GoogleAnalytics4.js File

1. Go to the JavaScript folder in the structure you created in [Prepare GoogleAnalytics4@1.0.0 Folders Structure](#).
2. Create the JavaScript file GoogleAnalytics4.js with the following code:

```

1 // @module GoogleAnalytics4
2 /* eslint-disable */
3 (function(win, name) {
4     // @class GoogleAnalytics4 @extends ApplicationModule
5     // -----
6     // Loads google analytics 4 script and extends application with methods:
7     // * trackPageview
8     // * trackEvent
9     // * trackTransaction
10    // Also wraps layout's showInModal
11    define('GoogleAnalytics4', [
12        'Tracker',
13        'underscore',
14        'jQuery',
15        'Backbone',
16        'Utils'
17    ], function(Tracker, _, jQuery, Backbone, Utils) {
18        var dataLayerName = name + 'DataLayer';
19        var GoogleAnalytics4 = {
20            // @method doCallback Indicates if this module do a callback after some particular events
21            // @return {Boolean}
22            doCallback: function() {
23                return !!win[name];
24            },
25
26            // @method trackPageview
27            // @param {String} url
28            // @return {GoogleAnalytics4}
29            trackPageview: function(url) {
30                if (_.isString(url)) {
31                    url = url.split('?')[0];
32                    win.gtag('event', 'page_view', { page_location: url, page_title: url });
33                }
34
35                return this;
36            },
37
38            trackLogin: function(event) {
39                win.gtag('event', 'login', {
40                    method: event.category,
41                    event_callback: event.callback
42                });
43
44                return this;
45            },
46
47            trackRegister: function(event) {
48                win.gtag('event', 'sign_up', {

```

```

49         method: event.category,
50         event_callback: event.callback
51     });
52
53     return this;
54 },
55 trackCheckoutAsGuest: function(event) {
56     win.gtag('event', 'checkout_as_guest', {
57         method: event.category,
58         event_callback: event.callback
59     });
60
61     return this;
62 },
63
64 // @method trackEvent
65 // @param {TrackEvent} event
66 // @return {GoogleAnalytics}
67 trackEvent: function(event) {
68     if (event && event.category && event.action) {
69         win.gtag('event', 'generic_event', {
70             category: event.category,
71             action: event.action,
72             label: event.label,
73             value: parseFloat(event.value) || 0,
74             page: event.page || "/" + Backbone.history.fragment,
75             event_callback: event.callback
76         });
77     }
78
79     return this;
80 },
81
82 // @method trackTransaction
83 // Based on the created SalesOrder we trigger each of the analytics
84 // ecommerce methods passing the required information
85 // @param {LiveOrder.Model} order
86 // @return {GoogleAnalytics|Void}
87 trackTransaction: function(order) {
88     if (order && order.get('confirmation')) {
89         var orderSummary = order.get('summary');
90         var item = null;
91
92         var purchase = {
93             transaction_id: order.get('confirmation').confirmationnumber,
94             value: orderSummary.subtotal,
95             tax: orderSummary.taxtotal,
96             shipping: orderSummary.shippingcost + orderSummary.handlingcost,
97             currency:
98                 (SC.ENVIRONMENT.currentCurrency && SC.ENVIRONMENT.currentCurrency.code) ||
99                 '',
100             affiliation: GoogleAnalytics4.application.getConfig('siteSettings.displayName'),
101             items: []
102         };
103
104         order.get('lines').each(function (line, index) {
105             item = line.get('item');
106
107             purchase.items.push({
108                 item_id: item.get('_sku'),
109                 item_name: item.get('_name'),
110                 affiliation: GoogleAnalytics4.application.getConfig('siteSettings.displayName'),
111                 currency:
112                     (SC.ENVIRONMENT.currentCurrency &&
113                     SC.ENVIRONMENT.currentCurrency.code) ||
114                     '',
115                 index: index,
116                 item_category: item.get('_category') || '',
117                 item_variant: '',
118                 price: line.get('rate'),
119                 quantity: line.get('quantity')
120             });
121         });

```



```

122         win.gtag('event', 'purchase', purchase);
123     }
124
125     },
126
127     return this;
128 },
129
130 // @method addCrossDomainParameters
131 // @param {string} url
132 // @return {String} url
133 addCrossDomainParameters: function(url) {
134     // We only need to add the parameters if the URL we are trying to go
135     // is not a sub domain of the tracking domain
136     if (!_.isString(url)) {
137         url = Utils.addParamsToUrl(url, {
138             client_id: win[name].client_id,
139             session_id: win[name].session_id
140         });
141     }
142
143     return url;
144 },
145
146 // @method loadScript
147 // @return {jQuery.Promise|Void}
148 loadScript: function(id) {
149     return jQuery.getScript(
150         '//www.googletagmanager.com/gtag/js?id=' + id + '&l=' + dataLayerName
151     );
152 },
153
154 // @method mountToApp
155 // @param {ApplicationSkeleton} application
156 // @return {Void}
157 mountToApp: function(application) {
158     GoogleAnalytics4.application = application;
159     var tracking = GoogleAnalytics4.application.getConfig('tracking.googleAnalyticsFour');
160
161     // if track page view needs to be tracked
162     if (tracking && tracking.propertyID && !SC.isPageGenerator()) {
163         win[dataLayerName] = win[dataLayerName] || [];
164         win.gtag =
165             win.gtag ||
166             function() {
167                 win[dataLayerName].push(arguments);
168             };
169         var domains = [];
170         if (tracking.domainName) {
171             domains.push(tracking.domainName);
172         }
173         if (tracking.domainNameSecure) {
174             domains.push(tracking.domainNameSecure);
175         }
176
177         if (domains.length) {
178             win.gtag('set', 'linker', {
179                 domains: domains
180             });
181         }
182
183         var configOptions = {
184             send_page_view: false
185         };
186
187         var client_id = Utils.getParameterByName(win.location.href, 'client_id');
188         var session_id = Utils.getParameterByName(win.location.href, 'session_id');
189
190         if (client_id && session_id) {
191             configOptions.client_id = client_id;
192             configOptions.session_id = session_id;
193         }
194     }

```

```

195     win.gtag('js', new Date());
196     win.gtag('config', tracking.propertyID, configOptions);
197
198     // Please keep this statement here or the first pageView on site load
199     // will not be triggered (ApplicationSkeleton.Layout._showContent)
200     Tracker.getInstance().trackers.push(GoogleAnalytics4);
201
202     // the analytics script is only loaded if we are on a browser
203     application.getLayout().once('afterAppendView', function() {
204         GoogleAnalytics4.loadScript(tracking.propertyID)
205             .done(function() {
206                 win[name] = {};
207                 win.gtag('get', tracking.propertyID, 'client_id', function(client_id) {
208                     win[name].client_id = client_id;
209                 });
210                 win.gtag('get', tracking.propertyID, 'session_id', function(session_id) {
211                     win[name].session_id = session_id;
212                 });
213             })
214             .fail(function(jqXHR) {
215                 jqXHR.preventDefault = true;
216                 console.warn('Google Analytics 4 was unable to load');
217             });
218     });
219 }
220 }
221 };
222
223     return GoogleAnalytics4;
224 });
225 })(window, 'ga4');

```

Create ns.package.json File

1. Go to the GoogleAnalytics4 folder in the structure you created in [Prepare GoogleAnalytics4@1.0.0 Folders Structure](#).
2. Create and save the json file, ns.package.json, with the following code:

```

1  {
2      "gulp": {
3          "javascript": ["JavaScript/*.js"]
4      }
5  }

```

3. Save your changes.

Add entryPoints Module to Application Dependencies

1. Open the distro.json file in the root SCA development directory and add GoogleAnalytics4 to the dependencies attribute in the entryPoints for the Shopping, Checkout, and MyAccount applications.

The following sample shows what your updated code will look like:

```

1  {
2      ...
3      "tasksConfig": {
4          ...
5          "javascript": {
6              "entryPoints": [
7                  {
8                      "names": ["SC.Shopping.Starter"],
9                      "exportFile": "shopping.js",
10                     "dependencies": [
11                         "GoogleAnalytics4",
12                     ...

```

```

13     ]
14   },
15   {
16     "names": ["SC.MyAccount.Starter"],
17     "exportFile": "myaccount.js",
18     "dependencies": [
19       "GoogleAnalytics4",
20       ...
21     ]
22   },
23   {
24     "names": ["SC.Checkout.Starter"],
25     "exportFile": "checkout.js",
26     "dependencies": [
27       "GoogleAnalytics4",
28       ...
29     ]
30   }
31 ]
32 ...
33 ...
34 ...
35 }

```

2. Save the `distro.json` file.

Add googleAnalyticsFour to Tracking Property

1. Go to `.../Modules/suitecommerce/ShoppingApplication@X.X.X/JavaScript/` and open the JavaScript file, `SC.Shopping.Configuration.js`.
2. In the Configuration object, inside the tracking property, add the `googleAnalyticsFour` property.

The following sample shows what your updated code will look like:

```

1  var Configuration = {
2    ...
3    ,           tracking: {
4      ...
5      ,           googleAnalyticsFour: {
6        // Enter the Google Analytics 4 Tracking ID. SuiteCommerce Advanced
        // inserts this property into each page of the application.
7        // To locate the tracking ID, login to your Google Analytics 4 account
        // and go to Admin > Data Streams > select your data stream > MEASUREMENT ID.
8        // Format: G-XXXXXXXXXX
9        propertyID: '',
10       // Enter the domain used for Google Analytics. The value you enter
        // depends on which application you want to use Google Analytics
11       domainName: '',
12       // Enter the secure domain used for Google Analytics 4. Possible values
        // are the same as the Google Analytics Domain property.
13       domainNameSecure: ''
14     }
15   }
16 };

```

3. Save the `SC.Shopping.Configuration.js` file.
 4. Go to `.../Modules/suitecommerce/CheckoutApplication@X.X.X/JavaScript/` and open the JavaScript file, `SC.Checkout.Configuration.js`.
 5. In the Configuration object, inside the tracking property, add the `googleAnalyticsFour` property.
- The following sample shows what your updated code will look like:

```

1  var Configuration = {
2    ...
3    ,           tracking: {
4      ...
5      ,           googleAnalyticsFour: {

```

```

6 // Enter the Google Analytics 4 Tracking ID. SuiteCommerce Advanced
  inserts this property into each page of the application.
7 // To locate the tracking ID, login to your Google Analytics 4 account
  and go to Admin > Data Streams > select your data stream > MEASUREMENT ID.
8 // Format: G-XXXXXXXXXX
9 propertyID: '',
10 // Enter the domain used for Google Analytics. The value you enter
  depends on which application you want to use Google Analytics
11 domainName: '',
12 // Enter the secure domain used for Google Analytics 4. Possible values
  are the same as the Google Analytics Domain property.
13 domainNameSecure: ''
14 }
15 }
16 };

```

6. Save the SC.Checkout.Configuration.js file.
7. Go to ../Modules/suitecommerce/MyAccountApplication.SCA@X.X.X /JavaScript/ and open the JavaScript file, SC.MyAccount.Configuration.js.
8. In the Configuration object, inside the tracking property, add the googleAnalyticsFour property. The following sample shows what your updated code will look like:

```

1 var Configuration = {
2   ...
3   , tracking: {
4     ...
5     , googleAnalyticsFour: {
6       // Enter the Google Analytics 4 Tracking ID. SuiteCommerce Advanced
7       inserts this property into each page of the application.
8       // To locate the tracking ID, login to your Google Analytics 4 account
9       and go to Admin > Data Streams > select your data stream > MEASUREMENT ID.
10      // Format: G-XXXXXXXXXX
11      propertyID: '',
12      // Enter the domain used for Google Analytics. The value you enter
13      depends on which application you want to use Google Analytics
14      domainName: '',
15      // Enter the secure domain used for Google Analytics 4. Possible values
16      are the same as the Google Analytics Domain property.
17      domainNameSecure: ''
18    }
19  }
20 };

```

9. Save the SC.MyAccount.Configuration.js file.

Test and Deploy Your Customizations

Follow the instructions provided in [Test SCA Customizations on a Local Server](#) and [Deploy SCA Customizations to NetSuite](#) to test and deploy your customizations.

Customer Redirected to My Account Overview Instead of Successful Payment Page

i Applies to: SuiteCommerce Advanced | 2020.1 | 2018.2 | Kilimanjaro | Site Builder

In the following implementations, customers are redirected to the My Account Overview page after they have paid an invoice using Windcave's external checkout:

- SiteBuilder Extensions Kilimanjaro
- SiteBuilder Extensions Premium Kilimanjaro
- SuiteCommerce Advanced (SCA) Kilimanjaro

- SCA 2018.2
- SCA 2020.1
- SCA 2020.1.12
- SCA 2021.2
- SCA 2021.2.3

These patch instructions describe how to correct this problem so that customers are redirected to the Successful Payment page after they pay the invoice.

To implement this patch, create a custom module to override the `ExternalPayment.Model.js` JavaScript file, which is part of the ExternalPayment module.

If you are unfamiliar with implementing patches for SCA, refer to the following:

- [Best Practices for Customizing SCA.](#)
- [Patch Using Override Mode.](#)

Step 1: Create the Override File

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/ExternalPayment@1.0.0/JavaScript/ ExternalPayment.Model.js`.
2. In the new `ExternalPayment.Model.js` file, in the method `_isDone`, find and replace the following line of code with the provided code sample.

Find the following code:

```
1 | , status_hold_value = this._getConfiguration(record_type, 'statusHoldValue', 'HOLD');
```

And replace it with the following code:

```
1 | , status_hold_value = this._getConfiguration(record_type, 'statusHoldValue', 'HOLD')
2 | , status_success_value = this._getConfiguration(record_type, 'statusSuccessValue', 'SUCCESS');
```

3. Then find and replace the following line of code with the provided code sample.

Find the following code:

```
1 | return status === status_accept_value || status === status_hold_value;
```

And replace it with the following code:

```
1 | return status === status_accept_value || status === status_hold_value || status === status_success_value;
```

4. In the method `_getStatusFromRequest`, find and replace the following line of code with the provided code sample.

Find the following code:

```
1 | return request.getParameter(this._getConfiguration(record_type, 'statusParameterName', 'status'));
```

And replace it with the following code:

```
1 | var default_status = 'status'; if(record_type === 'customerpayment'){ default_status = 'nstatus'; } return request.getPa
parameter(this._getConfiguration(record_type, 'statusParameterName', default_status));
```

5. In the method `_validateStatusFromRequest`, find and replace the following line of code with the provided code sample.

Find the following code:

```
1 | , status_reject_value = this._getConfiguration(record_type, 'statusRejectValue' , 'REJECT');
```

And replace it with the following code:

```
1 | , status_reject_value = this._getConfiguration(record_type, 'statusRejectValue' , 'REJECT')
2 | , status_success_value = this._getConfiguration(record_type, 'statusAcceptValue' , 'SUCCESS');
```

6. Then find and replace the following line of code with the provided code sample.

Find the following code:

```
1 | return status === status_accept_value || status === status_hold_value || status === status_reject_value;
```

And replace it with the following code:

```
1 | return status === status_accept_value || status === status_hold_value || status === status_reject_value || status ===
   | status_success_value;
```

7. In the method `_getDataFromRedirect`, find and replace the following line of code with the provided code sample.

Find the following code:

```
1 | , configuration_parameters = this._getConfiguration(record_type, 'parameters' , ['tranid', 'authcode', 'status'])
```

And replace it with the following code:

```
1 | , configuration_parameters = this._getConfiguration(record_type, 'parameters' , ['tranid', 'authcode', 'status', 'nlsta
   | tus'])
```

8. Save the file.

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you create in the Modules directory: `/Modules/extensions/ExternalPayment@1.0.0/ns.package.json`

```
1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ExternalPayment@X.Y.Z/SuiteScript/ExternalPayment.Model.js" : "SuiteScript/ExternalPayment.Model.js"
9 |   }
10 | }
```



Important: : You must replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the "modules" key.

```

1 | "modules": {
2 |   "extensions/ExternalPaymentExtension": "1.0.0",
3 |   . . .

```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Hide Add to Cart Button from Facet View

i Applies to: Vinson | Elbrus | Kilimanjaro | Aconcagua

In some implementations of SuiteCommerce Advanced (SCA) Vinson, Elbrus, Kilimanjaro, and Aconcagua, the **Add to Cart** button appears in the Product List Page for guest shoppers and shoppers who are not logged in to the Commerce web store.

These patch instructions describe how to correct this problem so that the **Add to Cart** button is hidden from guest shoppers and only displayed to logged-in shoppers.

To implement this patch, create a custom module to override the `header.tpl` and `footer.tpl` template files, which are part of the Header and Footer modules.

If you are unfamiliar with implementing patches for SCA, refer to the following:

- [Best Practices for Customizing SCA.](#)
- [Patch Using Override Mode.](#)

Step 1: Create the Override File

1. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file:

```
/Modules/extensions/Header@1.0.0/Templates/header.tpl
```

2. At the start of the new `header.tpl` file, enter the following code:

```

1 | <script>
2 | setInterval(function(){
3 |   if ( document.getElementsByClassName("product-views-price-login-to-see-prices")[0]
4 | ) { var divsToHide = document.getElementsByClassName("cart-add-to-cart-button-button"); for(var i = 0; i < divsTo
   | Hide.length; i++){ divsToHide[i].style.display = "none"; }
5 | }
6 | }, 100);
7 | </script>

```

3. Save the `header.tpl` file.
4. Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file:

```
/Modules/extensions/Footer@1.0.0/Templates/footer.tpl
```

5. At the end of the new `footer.tpl` file, enter the following code:

```
1 | <script>
```

```

2 | setInterval(function(){
3 |   if ( !document.getElementsByClassName("product-views-price-login-to-see-prices")[0]
4 | ) { var divsToShow = document.getElementsByClassName("cart-add-to-cart-button-button"); for(var i = 0; i <
   |   divsToShow.length; i++){ divsToShow[i].style.display = "initial";
5 | }
6 | }
7 | }, 100);
8 | </script>

```

6. Save the footer.tpl file.

Step 2: Prepare the Developer Tools for Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Enter the following sample code into the new ns.package.json file that you created in the following Modules directories:

/Modules/extensions/Header@1.0.0/ns.package.json

/Modules/extensions/Footer@1.0.0/ns.package.json

```

1 | { "gulp": { "templates": [ "Templates/*" ] }, "overrides": { "suitecommerce/Header@X.Y.Z/Templates/header.tpl"
2 |   : "Templates/header.tpl" "suitecommerce/Footer@X.Y.Z/Templates/footer.tpl" : "Templates/footer.tpl" }

```



Important: You must replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Save the ns.package.json file.
3. Open the distro.json file in the root SCA development directory and then add your custom modules to the modules object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the "modules" key.

```

1 | "modules": {
2 |   "extensions/HeaderExtension": "1.0.0",
3 |   "extensions/FooterExtension": "1.0.0",
4 |   ...

```

4. Save the distro.json file.

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Removing Item Redirects to Login Page Under Secure Domain

Applies to: Site Builder

In some implementations of Site Builder Reference Checkout 2.05, shoppers are redirected to the login page after they remove an item from their shopping cart. This problem occurs in the following situations:

- Shoppers are not logged in to a customer account.
- The website is using a secure (HTTPS) shopping domain.
- The patch for [Secure Shopping for Site Builder Implementations](#) has been applied.

The following patch instructions describe how to correct this problem so that the item is removed from the shopping cart and the shopper is not redirected to the login page.

To update the live-order-line.ss file:

1. Open the live-order-line.ss file in the following location: Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - Checkout 2.05.0 > Custom Checkout > services.
2. Find the following line of code:

```
1 // If we are not in the checkout OR we are logged in
2 // When on store, login in is not required
3 // When on checkout, login is required
4 if (Utils.isCheckoutDomain() || session.isLoggedIn2())
5 {
```

3. And replace it with the following code:

```
1 // If we are not in the checkout OR we are logged in
2 // When on store, login in is not required
3 // When on checkout, login is required
4 if (Utils.isShoppingDomain() || Utils.isCheckoutDomain() || session.isLoggedIn2())
5 {
```

4. Save the updated file.
5. Check the web store to confirm that items are removed from the shopping cart for both guests and logged-in shoppers.

Token-Based Authentication Updates for Developer Tools

i Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus | Kilimanjaro | Aconcagua | 2018.2 | 2019.1 | 2019.2 | 2020.1 | 2020.2 | Site Builder

All versions of the developer tools, including the extension, theme, and SuiteCommerce Advanced (SCA) developer tools, must be updated to continue to support an ongoing secure implementation of token-based authentication. The updates described in these patch instructions are needed to remove the NLAAuth authorization header from token-based authentication flows. Support for the the NLAAuth authorization header ends with the release of SuiteCommerce/SuiteCommerce Advanced 2022.2.1.



Important: If you have not applied the patch described in [Enable Token-Based Authentication](#), you must do so before continuing with the updates described in these patch instructions.

Follow the steps in these procedures to update the developer tools for your SCA implementation:

- [Modify the net.js File for Gulp Deployments](#)
- [Create a New Integration Record](#)
- [Create a New .env File](#)
- [Update Theme and Extension Developer Tools](#)

- [Generate a New Token for Authentication](#)

Modify the net.js File for Gulp Deployments

The modifications to the net.js file are different depending on the version of SCA you have implemented. Be sure to follow the patch instructions appropriate for your version of SCA.

- [net.js Modifications for SCA Kilimanjaro and Versions Running Node.js 11 and Earlier](#)
- [net.js Modifications for SCA 2020.2.3 and Earlier Versions](#)
- [net.js Modification for SCA 2020.2.4 and Later Versions](#)

net.js Modifications for SCA Kilimanjaro and Versions Running Node.js 11 and Earlier

1. In the SuiteCommerce Advanced source directory, open this file:
`.../gulp/ns-deploy/net.js`
2. Find this OAuth1 call:

```
1 | var oauth1 = new OAuth1({ molecule: args.m, vm: args.vm, key: args.key, secret: args.secret });
```

And replace it with this call:

```
1 | var oauth1 = new OAuth1({
2 |   molecule: args.m,
3 |   vm: args.vm,
4 |   key: args.key || process.env.CONSUMER_KEY,
5 |   secret: args.secret || process.env.CONSUMER_SECRET,
6 |   account: args.account
7 | });
```

3. Find the getUploader method and replace the current credentials object:

```
1 | var credentials = {
2 |   account: deploy.info.account,
3 |   authID: deploy.info.authID,
4 |   user_agent: deploy.info.user_agent || undefined,
5 |   molecule: args.m || undefined,
6 |   nsVersion: args.nsVersion || undefined,
7 |   applicationId: args.applicationId || undefined,
8 |   vm: args.vm || undefined,
9 |   key: args.key,
10 |   secret: args.secret
11 | };
```

With this one:

```
1 | var credentials = {
2 |   account: deploy.info.account,
3 |   authID: deploy.info.authID,
4 |   user_agent: deploy.info.user_agent || undefined,
5 |   molecule: args.m || undefined,
6 |   nsVersion: args.nsVersion || undefined,
7 |   applicationId: args.applicationId || undefined,
8 |   vm: args.vm || undefined,
9 |   key: args.key || process.env.CONSUMER_KEY,
10 |   secret: args.secret || process.env.CONSUMER_SECRET
11 | }
```

4. Save the modifications to the net.js file.

5. Pass the --key --secret values on the gulp deploy command. For example:
 gulp deploy --to --key secretkeyvalue1 --secret secretkeyvalue2

 **Note:** Replace secretkeyvalue1 and secretkeyvalue2 with your secret key values.

net.js Modifications for SCA 2020.2.3 and Earlier Versions

1. In the SuiteCommerce Advanced source directory, open this file:
 ../gulp/ns-deploy/net.js
2. Find the following line of code:

```
1 | var package_manager = require('../package-manager');
```

And replace it with the following lines of code:

```
1 |   var package_manager = require('../package-manager');
2 |   require('dotenv').config();
```

 **Note:** Step 2 only applies to SCA implementations using Node.js versions 12 and later.

3. Find this OAuth1 call:

```
1 | var oauth1 = new OAuth1({ molecule: args.m, vm: args.vm, key: args.key, secret: args.secret });
```

And replace it with this call:

```
1 | var oauth1 = new OAuth1({
2 |   molecule: args.m,
3 |   vm: args.vm,
4 |   key: args.key || process.env.CONSUMER_KEY,
5 |   secret: args.secret || process.env.CONSUMER_SECRET,
6 |   account: args.account
7 | });
```

4. Find the getUploader method and replace the current credentials object:

```
1 |   var credentials = {
2 |     account: deploy.info.account,
3 |     authID: deploy.info.authID,
4 |     user_agent: deploy.info.user_agent || undefined,
5 |     molecule: args.m || undefined,
6 |     nsVersion: args.nsVersion || undefined,
7 |     applicationId: args.applicationId || undefined,
8 |     vm: args.vm || undefined,
9 |     key: args.key,
10 |    secret: args.secret
11 |  };
```

With this one:

```
1 |   var credentials = {
2 |     account: deploy.info.account,
3 |     authID: deploy.info.authID,
4 |     user_agent: deploy.info.user_agent || undefined,
5 |     molecule: args.m || undefined,
6 |     nsVersion: args.nsVersion || undefined,
7 |     applicationId: args.applicationId || undefined,
8 |     vm: args.vm || undefined,
9 |     key: args.key || process.env.CONSUMER_KEY,
```

```

10 |     secret: args.secret || process.env.CONSUMER_SECRET
11 | }

```

5. Save the modifications to the `net.js` file.
6. Open a command line or terminal window and run the following command:

```
1 | npm install dotenv@16.0.1 --save
```

net.js Modification for SCA 2020.2.4 and Later Versions

1. In the SuiteCommerce Advanced source directory, open this file:
`.../gulp/ns-deploy/net.js`
2. Find the following line of code:

```
1 | const package_manager = require('../package-manager');
```

And replace it with the following lines of code:

```

1 |     const package_manager = require('../package-manager');
2 |     require('dotenv').config();

```

 **Note:** Step 2 only applies to SCA implementations using Node.js versions 12 and later.

3. Find this OAuth1 call:

```
1 | const oauth1 = new OAuth1({ molecule: args.m, vm: args.vm, key: args.key, secret: args.secret });
```

And replace it with this call:

```

1 | const oauth1 = new OAuth1({
2 |     molecule: args.m,
3 |     vm: args.vm,
4 |     key: args.key || process.env.CONSUMER_KEY,
5 |     secret: args.secret || process.env.CONSUMER_SECRET,
6 |     account: args.account
7 | });

```

4. Find the `getUploader` method and replace the current credentials object:

```

1 |     const credentials = {
2 |         account: deploy.info.account,
3 |         authID: deploy.info.authID,
4 |         user_agent: deploy.info.user_agent || undefined,
5 |         molecule: args.m || undefined,
6 |         nsVersion: args.nsVersion || undefined,
7 |         applicationId: args.applicationId || undefined,
8 |         vm: args.vm || undefined,
9 |         key: args.key,
10 |         secret: args.secret
11 |     };

```

With this one:

```

1 |     const credentials = {
2 |         account: deploy.info.account,
3 |         authID: deploy.info.authID,
4 |         user_agent: deploy.info.user_agent || undefined,
5 |         molecule: args.m || undefined,
6 |         nsVersion: args.nsVersion || undefined,

```

```

7 |     applicationId: args.applicationId || undefined,
8 |     vm: args.vm || undefined,
9 |     key: args.key || process.env.CONSUMER_KEY,
10 |    secret: args.secret || process.env.CONSUMER_SECRET
11 |  }

```

5. Save the modifications to the `net.js` file.
6. Open a command line or terminal window and run the following command:

```
1 | npm install dotenv@16.0.1 --save
```

Create a New Integration Record

1. In NetSuite, go to Setup > Integration > Manage Integrations > New
2. Enter a **Name** for this integration record.
3. On the **Authentication** tab, provide the following:
 - Check the Token-Based Authentication box to enable it.
 - Check the TBA: Authorization Flow box to enable it.
 - Enter a URL in the Callback URL field.

The callback URL is where the token-based authentication process responds with the token values. For example: `http://localhost:7777/tba`
4. Disable the Authorization Code Grant box and all other OAuth 2.0 options.
5. Click **Save**.

The confirmation page displays the Client Credentials (Consumer Key and Consumer Secret). You will need this information in the following steps.



Warning: The system displays the client ID and client secret only the first time you save the integration record. Be sure to record these values to use in the following steps.

Create a New .env File

1. At the root level of the SCA source directory, create a new `.env` file. The root level of the SCA source directory is where the `package.json` file resides.

For example, if working with the 2019.2 release of SCA, the new `.env` file resides in the following directory: `SC_19.2_Live/.env`

The contents of the new file should look like this:

```

# .env file
CONSUMER_KEY="446765bc51e70086cf582d32f0486fc0a764354ffec6c983b94e63af331346c4"
CONSUMER_SECRET="97fda39654d2f0cf3f0f9011d97616f96a3136e2c7659b867e57a1e1bbcc566"

```

2. Replace the values for `CONSUMER_KEY` and `CONSUMER_SECRET` shown in the preceding example with the values you copied from the integration record.
3. Save and close the `.env` file.

Update Theme and Extension Developer Tools

SCA versions Aconcagua and later support the theme and extension developer tools. If you are implementing one of these versions of SCA, you must replace your existing theme and extension

developer tools with the updated theme and extension developer tools available with the 2022.1.1 minor release or later of the SuiteCommerce Extension Manager.

Note: Before setting up the theme and extension developer tools, make sure you have the correct version of Node.js installed for your implementation of SuiteCommerce Advanced. For required versions, see [Install Node.js](#).

You can download and set up the theme and extension developer tools as described in the following topics:

- [Set Up Theme Developer Tools](#)
- [Set Up Extension Developer Tools](#)

Generate a New Token for Authentication

After completing all updates outlined in the preceding sections, the first time you fetch or deploy using the SCA, extension, or theme developer tools, you will need to generate a new token. For more information, see [Generate Tokens for Authentication](#).

Domain Bridging Update

Applies to: SuiteCommerce Advanced | Elbrus | Kilimanjaro | Vinson

In some implementations of the Kilimanjaro, Elbrus, and Vinson releases of SuiteCommerce Advanced (SCA), an error is returned in the Checkout page when the Encrypted Payload Domain Bridging feature is active.

These patch instructions describe how to correct this problem.

To implement this patch, create a custom module to override the `Configuration.js` JavaScript file, which is part of the `SspLibraries` module.

If you are unfamiliar with implementing patches for SCA, refer to the following:

- [Best Practices for Customizing SCA](#).
- [Patch Using Override Mode](#).

Step 1: Create the Override File

Create the Override File for SCA Kilimanjaro and Elbrus

Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/SspLibraries@1.0.0/JavaScript/Configuration.js`. In the new `Configuration.js` file, find and replace the following line of code with the provided code sample.

Find the following code:

```
1 | var domain = ModelsInit.session.getSiteSettings(['touchpoints']).touchpoints.home.match(/^http(s?)\:\/\/\/([^\?#]+)(?:[\/?#]|$)/i)
  | [2];
```

And replace it with the following code:

```
1 | var domain = ModelsInit.session.getEffectiveShoppingDomain();
```

Create the Override File for SCA Vinson

Following the instructions and in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/SspLibraries@1.0.0/JavaScript/Configuration.js. In the new Configuration.js file, find and replace the following line of code with the provided code sample.

Find the following code:

```
1 | var domain = session.getSiteSettings(['touchpoints']).touchpoints.home.match(/^http(s?)\:\/\/\/([^\?#]+)(?:[\/?#]($)/i)[2];
```

And replace it with the following code:

```
1 | var domain = ModelsInit.session.getEffectiveShoppingDomain()
```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new ns.package.json file that you create in the Modules directory: /Modules/extensions/SspLibraries@1.0.0/ns.package.json

```
1 | { "gulp": { "javascript": [ "JavaScript/*.js" ] }, "overrides": { "suitecommerce/SspLibraries@X.Y.Z/SuiteScript/Configura
2 |   tion.js" : "SuiteScript/Configuration.js" }
  | }
```



Important: You must replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the distro.json file in the root SCA development directory and then add your custom module to the modules object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```
1 | "modules": { "extensions/SspLibraries": "1.0.0", . . .
```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Address Not Set for Orders for Unfulfillable Items

Applies to: SuiteCommerce Advanced | Kilimanjaro

In some implementations of the Kilimanjaro release of SuiteCommerce Advanced (SCA), the default address is not set for orders for unfulfillable (service or downloaded) items. Because the default address

fields cannot be empty in the final order record, items are cleared from the shopping cart after tax recalculation if SuiteTax is disabled and the address is outside the US.

These patch instructions describe how to prevent the default address field from being not set for orders for service or items that are downloaded.

To implement this patch, create a custom module to override the `LiveOrder.Model.js` JavaScript file, which is part of the `LiveOrder` module.

If you are unfamiliar with implementing patches for SCA, refer to the following:

- [Best Practices for Customizing SCA.](#)
- [Patch Using Override Mode.](#)

Step 1: Create the Override File

Following the instructions in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/LiveOrder@1.0.0/JavaScript/LiveOrder.Model.js`. In the new `LiveOrder.Model.js` file, find and replace the following code with the provided code sample.

Find the following code:

```

1  setShippingAddress: function setShippingAddress (data, current_order)
2  {
3      if (data.shipaddress !== current_order.shipaddress)
4      {
5          if (data.shipaddress)
6          {
7              if (this.isSecure && !~data.shipaddress.indexOf('null'))
8              {
9                  // Heads Up!: This "new String" is to fix a nasty bug
10                 ModelsInit.order.setShippingAddress(new String(data.shipaddress).toString());
11             }
12             else
13             {
14                 var address = _.find(data.addresses, function (address)
15                 {
16                     return address.internalid === data.shipaddress;
17                 });
18
19                 address && ModelsInit.order.estimateShippingCost(address);
20             }
21         }
22         else if (this.isSecure)
23         {
24             ModelsInit.order.removeShippingAddress();
25         }
26         else
27         {
28             ModelsInit.order.estimateShippingCost({
29                 zip: null
30                 , country: null
31             });
32             ModelsInit.order.removeShippingMethod();
33         }
34     }
35 }
```

And replace it with the following code:

```

1  setShippingAddress: function setShippingAddress (data, current_order)
2  {
3      if (data.shipaddress !== current_order.shipaddress)
```



```

4      {
5          if (data.shipaddress)
6          {
7              if (this.isSecure && !~data.shipaddress.indexOf('null'))
8              {
9                  // Heads Up!: This "new String" is to fix a nasty bug
10                 ModelsInit.order.setShippingAddress(new String(data.shipaddress).toString());
11             }
12             else
13             {
14                 var address = _.find(data.addresses, function (address)
15                 {
16                     return address.internalid === data.shipaddress;
17                 });
18
19                 address && ModelsInit.order.estimateShippingCost(address);
20             }
21         }
22         else if (this.isSecure)
23         {
24             if(ModelsInit.context.getSetting('FEATURE', 'tax_overhauling') === 'T')
25             {
26                 ModelsInit.order.removeShippingAddress();
27             }
28         }
29         else
30         {
31             ModelsInit.order.estimateShippingCost({
32                 zip: null
33                 ,   country: null
34             });
35             ModelsInit.order.removeShippingMethod();
36         }
37     }
38 }

```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you create in the Modules directory: `/Modules/extensions/LiveOrderExtension@1.0.0/ns.package.json`

```

1  { "gulp": { "javascript": [ "JavaScript/*.js" ] }, "overrides": { "suitecommerce/LiveOrder@X.Y.Z/JavaScript/LiveOrder.Model.js" : "JavaScript/LiveOrder.Model.js" }
2  }

```



Important: You must replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```

1  | "modules": { "extensions/LiveOrderExtension": "1.0.0", . . .

```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Gift Certificate Error After Return Request Submitted

i Applies to: SuiteCommerce Advanced | Elbrus

In some implementations of the Elbrus release of SuiteCommerce Advanced (SCA), a return authorization request is not submitted if the invoice status is set to Approved and there is a gift certificate included in the order. This error message appears on Commerce websites affected by this problem: "Gift Certificate From, Recipient Name, and Recipient Email are required."

These patch instructions describe how to correct this problem.

To implement this patch, create a custom module to override the `ReturnAuthorization.Model.js` JavaScript file, which is part of the `ReturnAuthorization` module.

If you are unfamiliar with implementing patches for SCA, refer to the following:

- [Best Practices for Customizing SCA.](#)
- [Patch Using Override Mode.](#)

Step 1: Create the Override File

Following the instructions and recommendations in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/ReturnAuthorization@1.0.0/JavaScript/ReturnAuthorization.Model.js`. In the new `ReturnAuthorization.Model.js` file, find and replace the following line of code with the provided code sample.

Find the following code:

```
1 | setLines: function (return_authorization, lines, transaction_lines)
```

And replace it with the following code:

```
1 | setLines: function (return_authorization, lines, transaction_lines)
2 | {
3 |   var line_count = return_authorization.getLineItemCount('item')
4 |   , add_line = true
5 |   , i = 1;
6 |   while (i <= line_count)
7 |   {
8 |     var line_item_value =
9 |     return_authorization.getLineItemValue('item', 'id', i);
10 |    add_line = this.findLine(line_item_value, lines);
11 |    if (add_line)
12 |    {
13 |      var transaction_line = _.findWhere(transaction_lines,
14 |      { line: line_item_value });
15 |      if (transaction_line)
16 |      {
17 |        return_authorization.setLineItemValue('item',
18 |        'rate', i, transaction_line.rate);
19 |      }
20 |      return_authorization.setLineItemValue('item',
21 |      'quantity', i, add_line.quantity);
22 |      return_authorization.setLineItemValue('item',
23 |      'description', i, add_line.reason);
24 |      i++;
25 |    }
26 |    else
27 |    {
28 |      var item_type =
```

```

29 return_authorization.getLineItemValue('item', 'itemtype', i);
30 if (item_type == 'GiftCert') {
31   return_authorization.removeLineItem('item', i);
32   line_count--;
33 } else {
34   return_authorization.setLineItemValue('item',
35     'quantity', i, 0);
36   i++;
37 }
38 }
39 }
40 }

```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you create in the Modules directory: `/Modules/extensions/ReturnAuthorizationExtension@1.0.0/ns.package.json`

```

1 {
2   "gulp": {
3     "javascript": [
4       "JavaScript/*.js"
5     ]
6   },
7   "overrides": {
8     "suitecommerce/ReturnAuthorization@X.Y.Z/JavaScript/ReturnAuthorization.Model.js" : "JavaScript/ReturnAuthorization.Model.js"
9   }
10 }

```



Important: You must replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```

1 "modules": {
2   "extensions/ReturnAuthorizationExtension": "1.0.0",
3   ...

```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Cannot Disable Default Credit Card

i Applies to: SuiteCommerce Advanced | 2018.2 | Aconcagua | Kilimanjaro

In some pre-2019.1 implementations of SuiteCommerce Advanced (SCA), customers cannot disable the **Make this my default credit card** option. This issue occurs when a customer has only one credit card set up as their payment method. In this case, your Commerce website should display a message to inform

the customer that a default payment method is required and so their single credit card must remain the default.

Because no message is displayed however, customers may be left confused since the **Make this my default credit card** box remains checked even after they have attempted to clear it. Therefore, the following workarounds are available:

- For SCA Aconcagua implementations with Base Theme 2018.2, you can implement SuiteCommerce Base Theme 2019.1 to correct this issue.
- For SCA Kilimanjaro implementations and earlier, you can override the `creditcard_edit_form.tpl` file as described in the following patch procedures.

To implement this patch, create a custom module to override the `creditcard_edit_form.tpl` template file, which is part of the CreditCard module.

If you are not familiar with implementing patches for SCA, refer to the following:

- [Best Practices for Customizing SCA.](#)
- [Patch Using Override Mode.](#)

Step 1: Create the Override File

Following the instructions and recommendations in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/CreditCard@1.0.0/Templates/creditcard_edit_form.tpl`. In the new `creditcard_edit_form.tpl` file, find and replace the following lines with the provided code sample.

Find the following code:

```
1  {{#if showDefaults}}
2  <div class="creditcard-edit-form">
3    <label class="creditcard-edit-form-checkbox">
4      <input type="checkbox" id="ccdefault" value="T" data-unchecked-value="F" name="ccdefault" {{#if ccdefault}} checked {{/
if}} >
5      {{translate 'Make this my default credit card'}}
6    </label>
7  </div>
8  {{/if}}
```

And replace it with the following code:

```
1  {{#if showDefaults}}
2    <div class="creditcard-edit-form">
3      <label class="creditcard-edit-form-checkbox">
4        <input type="checkbox" id="ccdefault" value="T" data-unchecked-value="F" name="ccdefault" {{#if ccdefault}} checked dis
abled {{/if}} >
5        {{translate 'Make this my default credit card'}}
6
7        {{#if ccdefault}} <i class="creditcard-default-icon" data-toggle="tooltip" title="{{translate 'You need to have at
least one card selected by default'}}"></i> {{/if}}
8      </label>
9    </div>
10  {{/if}}
```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, perform the following actions:

1. Paste the following sample code into the new `ns.package.json` file that you create in the Modules directory: `/Modules/extensions/CreditCardExtension@1.0.0/ns.package.json`

```

1 {
2   "gulp": {
3     "templates": [
4       "Templates/*"
5     ]
6   },
7   "overrides": {
8     "suitecommerce/CreditCard@X.Y.Z/Templates/creditcard_edit_form.tpl" : "Templates/creditcard_edit_form.tpl"
9   }
10 }

```



Important: You must replace the string, X.Y.Z, with the version of the module in your implementation of SCA.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```

1 "modules": {
2   "extensions/CreditCardExtension": "1.0.0",
3   . . .

```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Image Tracker Resized After Placing Order

Applies to: SuiteCommerce Advanced | Elbrus

In some implementations of the Elbrus release of SuiteCommerce Advanced (SCA), the Google AdWords image tracker image is resized after a shopper places an order.

These patch instructions describe how to correct this problem.

To implement this patch, create a custom module to override the `GoogleAdWords.js` file, which is part of the `GoogleAdWords` module.

If you are not familiar with implementing patches for SCA, refer to the following:

- [Best Practices for Customizing SCA](#) — to get you started
- [Patch Using Override Mode](#) — for detailed step-by-step instructions

Step 1: Create the Override File

Following the instructions and recommendations in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/GoogleAdWordsExtension@1.0.0/Javascript/GoogleAdWords.js`. In the new `GoogleAdWords.js` file, find and replace the following lines with the provided code sample.

Find the following code:

```
1 | src: '//www.googleadservices.com/pagead/conversion/' + config.id + '/?value=' + value + '&label=' + config.label + '&guid=ON&script=0'
```

And replace it with the following code:

```
1 | src: '//www.googleadservices.com/pagead/conversion/' + config.id + '/?value=' + value + '&label=' + config.label + '&guid=ON&script=0' , style: 'height: 0px; width: 0px'
```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure:

1. Paste the following sample code into the new `ns.package.json` file that you create in the Modules directory: `/Modules/extensions/GoogleAdWordsExtension@1.0.0/ns.package.json`

```
1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/GoogleAdWords@X.Y.Z/JavaScript/GoogleAdWords.js" : "JavaScript/GoogleAdWords.js"
9 |   }
10 | }
```



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

2. Open the `distro.json` file in the root SCA development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```
1 | "modules": {
2 |   "extensions/GoogleAdWordsExtension": "1.0.0",
3 |   . . .
```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Single Secure Domain Requires Login for Pricing

Applies to: SuiteCommerce Advanced | Elbrus

In some implementations of the Elbrus release of SuiteCommerce Advanced (SCA), logging in to a Commerce website using a single secure domain does not work as expected when the Required Login for

Pricing feature is enabled. For example, when the shopper goes to the web store and clicks **Log In to See Price**, they are redirected to a Page Not Found page instead of the My Account Login page.

These patch instructions describe how to correct this problem.

To implement this patch, create a custom module to override the `product_views_price.tpl` template file, which is part of the ProductViews module.

If you are not familiar with implementing patches for SuiteCommerce Advanced, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).
- For step by step instructions, refer to [Patch Using Override Mode](#).

Step 1: Create the Override File

Following the instructions and recommendations in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/ProductViewsExtension@1.0.0/Templates/product_views_price.tpl`. In the new `product_views_price.tpl` file, find and replace the following lines with the provided code sample.

Find the following text:

```
1 | {{translate 'Please <a href="{{$0}}">log in</a> to see price or purchase this item' urlLogin}}
```

And replace with the following text:

```
1 | {{translate 'Please <a href="{{$0}}" data-navigation="ignore-click">log in</a> to see price or purchase this item' urlLogin}}
```

Then find the following text:

```
1 | {{translate '<a href="{{$0}}">Log in</a> to see price' urlLogin}}
```

And replace with the following text:

```
1 | {{translate '<a href="{{$0}}" data-navigation="ignore-click">Log in</a> to see price' urlLogin}}
```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, you should:

1. Paste the following sample code into the new `ns.package.json` file that you create in the Modules directory: `/Modules/extensions/ProductViewsExtension@1.0.0/ns.package.json`

```
1 | {
2 |   "gulp": {
3 |     "templates": [
4 |       "Templates/*"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/ProductViews@X.Y.Z/Templates/product_views_price.tpl" : "Templates/product_views_price.tpl"
9 |   }
}
```

```
10 | }
```



Important: You must replace the string, X.Y.Z, with the version of the module in your implementation of SuiteCommerce Advanced.

2. Open the `distro.json` file in the root SuiteCommerce Advanced development directory and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the "modules" key.

```
1 | "modules": {
2 |   "extensions/ProductViewsExtension": "1.0.0",
3 |   . . .
```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Page Not Found Error for SMT Landing Pages

Applies to: Mont Blanc

In the Mont Blanc release of SuiteCommerce Advanced (SCA), landing pages created using the Site Management Tools (SMT) may generate 404 Page Not Found Errors. This issue has been fixed in the 2018.2 and later releases of SuiteCommerce Advanced. If you experience this issue and cannot migrate to the latest release, you can fix this issue as described in the following procedure.

To implement this patch, create a custom module to override the `Application.js` file, which is part of the **SspLibraries** module.

If you are not familiar with implementing patches for SCA, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).
- For step by step instructions, refer to [Patch Using Override Mode](#).

Step 1: Create the Override File

Following the instructions and recommendations in the [Patch Using Override Mode](#) procedure, create a new directory and file: `../Modules/extensions/SspLibrariesExtension@1.0.0/SuiteScript/Application.js`. Then replace code in the new file as described in the following steps.

1. Find the following line of code in the `getEnvironment` method:

```
1 | SC.Configuration.useCMS = !isSecure && SC.Configuration.useCMS && context.getSetting('FEATURE', 'ADVANCEDSITEMANAGEMENT')
   | === 'T';
```

2. And replace it with the following code:

```
1 | SC.Configuration.useCMS = SC.Configuration.useCMS && context.getSetting('FEATURE', 'ADVANCEDSITEMANAGEMENT') === 'T';
```


Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, you should:

1. Paste the following sample code into the new `ns.package.json` file that you create in the Modules directory: `.../Modules/extensions/SspLibrariesExtension@1.0.0/ns.package.json`

```

1  {
2    "gulp": {
3      "ssp-libraries": [
4        "SuiteScript/*.js"
5      ]
6    },
7    "overrides": {
8      "suitecommerce/SspLibraries@2.2.0/SuiteScript/Application.js" : "SuiteScript/Application.js"
9    }
10 }
```

2. Open the `distro.json` file and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The `distro.json` file resides in the top-level directory.

The following sample shows the value to add to the list of existing values that follow the "modules" key.

```

1  "modules": {
2    "extensions/SspLibrariesExtension": "1.0.0",
3    ...
```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

CVE-2020-14728: Vulnerability in SuiteCommerce Advanced Services

i Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus | Kilimanjaro | Aconcagua | 2018.2 | 2019.1 | 2019.2

Cross-site Scripting (XSS) is a technique where malicious scripts may be injected into your website. Current versions of SuiteCommerce Advanced (SCA) may be vulnerable to CVE-2020-14728. These instructions describe how to protect your site from this vulnerability. To protect your site, implement two patches as described in the following table:

Patch Instruction	Description	Required For
XSS Vulnerability Patch 1	Overrides the <code>Application.js</code> or <code>ServiceController.js</code> file, depending on your release, to escape error messages.	Denali, Montblanc, Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, 2019.1, and 2019.2
XSS Vulnerability Patch 2	Overrides the <code>Backbone.FormView.js</code> file to transform HTML error messages.	Denali, Montblanc, Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, 2019.1, and 2019.2

XSS Vulnerability Patch 1

i Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus | Kilimanjaro | Aconcagua | 2018.2 | 2019.1 | 2019.2

To implement this patch, create a custom module to override the appropriate file for your release within the SspLibraries module. The following table lists the appropriate files for each release.

The location of the file that you need to override depends on the SCA release that you need to patch:

Release	File Location
2019.2	.../SC_19.2_Live/Commons/SspLibraries/SuiteScript/ServiceController.js
2019.1	.../Modules/suitecommerce/SspLibraries@sc-2019.1.0/SuiteScript/ServiceController.js
2018.2	.../Modules/suitecommerce/SspLibraries@sc-2018.2.0/SuiteScript/ServiceController.js
Aconcagua	.../Modules/suitecommerce/SspLibraries@aconcaguaR2/SuiteScript/ServiceController.js
Kilimanjaro	.../Modules/suitecommerce/SspLibraries@4.2.0/SuiteScript/ServiceController.js
Elbrus	.../Modules/suitecommerce/SspLibraries@4.0.0/SuiteScript/ServiceController.js
Vinson	.../Modules/suitecommerce/SspLibraries@3.0.0/SuiteScript/ServiceController.js
Montblanc	.../Modules/suitecommerce/SspLibraries@2.2.0/SuiteScript/Application.js
Denali	.../Modules/suitecommerce/SspLibraries@2.1.1/SuiteScript/Application.js

If you are not familiar with implementing patches for SCA, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).
- For step by step instructions, refer to [Patch Using Override Mode](#).

Step 1: Create the Override Files

Following the instructions and recommendations in the [Patch Using Override Mode](#) procedure, copy and paste the following code samples in the new directory and file you create. Where you create the new directory and file depends on the SCA release that you need to patch:

Release	Location of New Directory and File
2019.2	.../SC_19.2_Live/Commons/extensions/SspLibrariesExtension@1.00/SuiteScript/ServiceController.js  Note: If you are patching 2019.2, you also need to complete the steps in Additional Edits for the 2019.2 Release.
Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, and 2019.1	.../Modules/extensions/SspLibrariesExtension@1.00/SuiteScript/ServiceController.js
Denali and Montblanc	.../Modules/extensions/SspLibrariesExtension@1.00/SuiteScript/Application.js

- In the ServiceController.js file in the 2019.2 release, find the following lines of code in the processError method:

```
1 | const content = {
2 |   errorCode: parseInt(status,10).toString(),
```

```

3 |     errorCode: code,
4 |     errorMessage: message
5 | };

```

And replace them with the following code:

```

1 | const content = {
2 |     errorStatusCode: parseInt(status, 10).toString(),
3 |     errorCode: code,
4 |     errorMessage: _.escape(message)
5 | };

```

- In the Application.js file in the Denali and Montblanc releases and the ServiceController.js file in the Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, and 2019.1 releases, find the following lines of code in the processError method:

```

1 | var content = {
2 |   errorStatusCode: parseInt(status,10).toString()
3 |   , errorCode: code
4 |   , errorMessage: message
5 | }

```

And replace them with the following code:

```

1 | var content = {
2 |   errorStatusCode: parseInt(status, 10).toString(),
3 |   errorCode: code,
4 |   errorMessage: _.escape(message)
5 | };

```

Additional Edits for the 2019.2 Release

For the 2019.2 release, you also need to directly edit the ServiceController.ts file in the following directory:
 .../SC_19.2_Live/Backend/Libraries/SspLibraries

1. Find the following lines of code in the ServiceController.ts file:

```

1 | const content = {
2 |     errorStatusCode: status.toString(),
3 |     errorCode: code,
4 |     errorMessage: message,
5 |     errorDetails: null,
6 |     errorMessageParams: null
7 | };

```

2. And replace them with the following code:

```

1 | const content = {
2 |     errorStatusCode: status.toString(),
3 |     errorCode: code,
4 |     errorMessage: _.escape(message),
5 |     errorDetails: null,
6 |     errorMessageParams: null
7 | };

```

3. Save the ServiceController.ts file and then continue with [Step 2: Prepare the Developer Tools For Your Patch](#).

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, you should:

1. Paste the code appropriate for the SCA release you are patching into a new `ns.package.json` file that you create. Where you create the `ns.package.json` file depends upon the release you are working with.

Release	Location of New <code>ns.package.json</code> File
2019.2	<code>.../Commons/extensions/SspLibrariesExtension@1.00/</code> <code>ns.package.json</code>
Denali, Montblanc, Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, and 2019.1	<code>.../Modules/extensions/SspLibrariesExtension@1.00/</code> <code>ns.package.json</code>

- Use the following code if you are patching the 2019.2 release:

```

1  {
2    "gulp": {
3      "ssp-libraries": [
4        "SuiteScript/*.js"
5      ]
6    },
7    "overrides": {
8      "Commons/SspLibraries/SuiteScript/ServiceController.js" : "SuiteScript/ServiceController.js"
9    }
10 }

```

- Use the following code if you are patching the Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, or 2019.1 release:

```

1  {
2    "gulp": {
3      "ssp-libraries": [
4        "SuiteScript/*.js"
5      ]
6    },
7    "overrides": {
8      "suitecommerce/SspLibraries@X.Y.Z/SuiteScript/ServiceController.js" : "SuiteScript/ServiceController.js"
9    }
10 }

```



Important: In the preceding code sample, you must replace the string `X.Y.Z` with the version of the module in your implementation of SuiteCommerce Advanced.

- Use the following code if you are patching the Denali or Montblanc release:

```

1  {
2    "gulp": {
3      "ssp-libraries": [
4        "SuiteScript/*.js"
5      ]
6    },
7    "overrides": {
8      "suitecommerce/SspLibraries@2.2.0/SuiteScript/Application.js" : "SuiteScript/Application.js"
9    }
10 }

```

2. Open the `distro.json` file and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The location of the `distro.json` file depends on the version of SCA you are patching.

Release	Location of the <code>distro.json</code> File
2019.2	For the 2019.2 release, the <code>distro.json</code> file resides in the Advanced directory. For example, if you accepted the default name for the top-level directory for 2019.2,

Release	Location of the distro.json File
	the complete path is: SuiteCommerce Advanced 2019.2/SC_19.2_Live/Advanced/distro.json.
Denali, Montblanc, Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, and 2019.1	For these releases, the distro.json file resides in the top-level directory. For example, for the 2018.2 release, if you accepted the default directory name for your release, the top-level directory would be SuiteCommerce Advanced 2018.2.

The following sample shows the value to add to the list of existing values that follow the "modules" key. Refer to the appropriate sample for the version of SCA you are working with.

- Use the following sample if you are patching the 2019.2 release:

```
1 | "modules": {
2 |   "../Commons/extensions/SspLibrariesExtension@1.00",
3 |   . . .
```

- Use the following sample if you are patching the Denali, Montblanc, Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, or 2019.1 release:

```
1 | "modules": {
2 |   "extensions/SspLibrariesExtension": "1.0.0",
3 |   . . .
```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.



Important: If you are patching the 2019.2 release, you need to complete the steps in the [Deploy to NetSuite Fails with Errors](#) patch instructions to insure that your patch deploys without errors.

XSS Vulnerability Patch 2

Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus | Kilimanjaro | Aconcagua | 2018.2 | 2019.1 | 2019.2

To implement this patch, create a custom module to override the appropriate file for your release within the Backbone.FormView module. The following table lists the appropriate files for each release.

The location of the file that you need to override depends on the SCA release that you need to patch:

Release	File Location
2019.2	.../SC_19.2_Live/Commons/Backbone.FormView/JavaScript/Backbone.FormView.ts
2019.1	.../Modules/suitecommerce/Backbone.FormView@sc-2019.1.0/JavaScript/Backbone.FormView.js
2018.2	.../Modules/suitecommerce/Backbone.FormView@sc-2018.2.0/JavaScript/Backbone.FormView.js
Aconcagua	.../Modules/suitecommerce/Backbone.FormView@aconcaguaR2/JavaScript/Backbone.FormView.js
Kilimanjaro	.../Modules/suitecommerce/Backbone.FormView@1.2.2/JavaScript/Backbone.FormView.js
Elbrus	.../Modules/suitecommerce/Backbone.FormView@1.2.0/JavaScript/Backbone.FormView.js
Vinson	.../Modules/suitecommerce/Backbone.FormView@1.1.1/JavaScript/Backbone.FormView.js

Release	File Location
Montblanc	.../Modules/suitecommerce/Backbone.FormView@1.1.0/JavaScript/Backbone.FormView.js
Denali	.../Modules/suitecommerce/Backbone.FormView@1.0.1/JavaScript/Backbone.FormView.js

If you are not familiar with implementing patches for SCA, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).
- For step by step instructions, refer to [Patch Using Override Mode](#).

Step 1: Create the Override Files

Following the instructions and recommendations in the [Patch Using Override Mode](#) procedure, copy and paste the following code samples in the new directory and file you create. Where you create the new directory and file depends on the SCA release that you need to patch:

Release	Location of New Directory and File
2019.2	.../SC_19.2_Live/Commons/extensions/Backbone.FormViewExtension@1.00/JavaScript/Backbone.FormView.ts
Denali, Montblanc, Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, and 2019.1	.../Modules/extensions/Backbone.FormViewExtension@1.00/JavaScript/Backbone.FormView.js

1. In the Backbone.FormView.js or Backbone.FormView.ts file, find the saveForm function. Add a new method called transformResponseText before the saveForm function as shown in the following example:

```

1 |   transformResponseText: function(response) {},
2 |
3 |   // @method saveForm will serialize the input of some form and save() the given model using it
4 |   // @param {HTMLEvent} e @param {Backbone.Model} model @param {Object} props properties to pass to model.save()
5 |   // @return {jQuery.Deferred}
6 |   saveForm: function(e, model, props) {
7 |       e.preventDefault();

```



Important: If the Backbone.FormView file for your version of SCA includes a comma before the saveForm function, add the new transformResponseText method like so:
,transformResponseText: function(response) {}

2. In the saveForm function, find the error definition as shown in the following example:

```

1 |           error: function(model, response) {
2 |               buttonSubmitDone(self.$savingForm);
3 |
4 |               if (response.responseText) {
5 |                   model.trigger(
6 |                       'error',
7 |                       jQuery.parseJSON(response.responseText || 'null')
8 |                   );
9 |               }
10 |           }

```

And replace it with the following code:

```

1 |           error: function(model, response) {
2 |               buttonSubmitDone(self.$savingForm);
3 |
4 |               if (response.responseText) {

```

```

5 |         self.transformResponseText(response);
6 |         model.trigger(
7 |             'error',
8 |             jQuery.parseJSON(response.responseText || 'null')
9 |         );
10 |     }
11 | }

```

3. In the Backbone.FormView.js or Backbone.FormView.ts file, find this line of code:

```
1 | view.saveForm = this.saveForm;
```

And replace it with the following code:

```

1 | view.saveForm = this.saveForm;
2 | view.transformResponseText = view.transformResponseText || this.transformResponseText;

```

4. Double-check to make sure that the reference path and the paths to the imported dependencies are accurate in the override file you create. If these paths are not accurate, the deploy to NetSuite may fail with errors that necessary files and modules cannot be found.

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, you should:

1. Paste the code appropriate for the SCA release you are patching into a new ns.package.json file that you create. Where you create the ns.package.json file depends upon the release you are working with.

Release	Location of New ns.package.json File
2019.2	.../Commons/extensions/Backbone.FormViewExtension@1.00/ ns.package.json
Denali, Montblanc, Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, and 2019.1	.../Modules/extensions/Backbone.FormViewExtension@1.00/ ns.package.json

- Use the following code if you are patching the 2019.2 release:

```

1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.ts"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "Commons/Backbone.FormView/JavaScript/Backbone.FormView.ts" : "JavaScript/Backbone.FormView.ts"
9 |   }
10 | }

```

- Use the following code if you are patching the Denali, Montblanc, Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, or 2019.1 release:

```

1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/Backbone.FormView@X.Y.Z/JavaScript/Backbone.FormView.js" : "JavaScript/Backbone.FormView.js"
9 |   }

```

```
10 | }
```



Important: In the preceding code sample, you must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

- Open the `distro.json` file and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The location of the `distro.json` file depends on the version of SCA you are patching.

Release	Location of the <code>distro.json</code> File
2019.2	For the 2019.2 release, the <code>distro.json</code> file resides in the Advanced directory. For example, if you accepted the default name for the 2019.2 top-level directory, the complete path is: SuiteCommerce Advanced 2019.2/SC_19.2_Live/Advanced/ <code>distro.json</code> .
Denali, Montblanc, Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, and 2019.1	For these releases, the <code>distro.json</code> file resides in the top-level directory. For example, for the 2018.2 release, if you accepted the default directory name for your release, the top-level directory would be SuiteCommerce Advanced 2018.2.

The following sample shows the value to add to the list of existing values that follow the "modules" key. Refer to the appropriate sample for the version of SCA you are working with.

- Use the following sample if you are patching the 2019.2 release:

```
1 | "modules": {
2 |   "../Commons/extensions/Backbone.FormViewExtension@1.00",
3 |   . . .
```

- Use the following sample if you are patching the Denali, Montblanc, Vinson, Elbrus, Kilimanjaro, Aconcagua, 2018.2, or 2019.1 release:

```
1 | "modules": {
2 |   "extensions/Backbone.FormViewExtension": "1.0.0",
3 |   . . .
```

Step 3: Add the `transformResponseText` Method to the `LoginRegister.Login.View` file

After completing the steps to override the `Backbone.FormView.js` or `Backbone.FormView.ts` file as described in the preceding steps, add the `transformResponseText` method to the `LoginRegister.Login.View` file. Where the `LoginRegister.Login.View` file resides depends on the SCA release that you need to patch:

Release	Location of the <code>LoginRegister.Login.View</code> File
2019.2	<code>.../SC_19.2/Live/Advanced/LoginRegister/JavaScript/LoginRegister.Login.View.ts</code>
2019.1	<code>.../Modules/suitecommerce/LoginRegister@sc-2019.1.0/JavaScript/LoginRegister.Login.View.js</code>
2018.2	<code>.../Modules/suitecommerce/LoginRegister@sc-2018.2.0/JavaScript/LoginRegister.Login.View.js</code>
Aconcagua	<code>.../Modules/suitecommerce/LoginRegister@aconcaguaR2/JavaScript/LoginRegister.Login.View.js</code>
Kilimanjaro	<code>.../Modules/suitecommerce/LoginRegister@2.4.0/JavaScript/LoginRegister.Login.View.js</code>
Elbrus	<code>.../Modules/suitecommerce/LoginRegister@2.3.0/JavaScript/LoginRegister.Login.View.js</code>
Vinson	<code>.../Modules/suitecommerce/LoginRegister@2.2.0/JavaScript/LoginRegister.Login.View.js</code>

Release	Location of the LoginRegister.Login.View File
Montblanc	.../Modules/suitecommerce/LoginRegister@2.1.0/JavaScript/LoginRegister.Login.View.js
Denali	.../Modules/suitecommerce/LoginRegister@2.0.1/JavaScript/LoginRegister.Login.View.js

1. Open the LoginRegister.Login.View file.
2. Find the getContext function, and add the transformResponseText method right above it as shown in the following example:

```

1 | ,transformResponseText: function(response) {
2 |     response.responseText = _unescape(response.responseText);
3 | }
4 |
5 | //@method getContext @return {LoginRegister.Login.View.Context}
6 | ,getContext: function ()

```

Step 4: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.



Important: If you are patching the 2019.2 release, you need to complete the steps in the [Deploy to NetSuite Fails with Errors](#) patch instructions to insure that your patch deploys without errors.

CVE-2020-14729: Vulnerability in SuiteCommerce Advanced Sites

i Applies to: SuiteCommerce Advanced

You can set X-Frame-Options (XFO) and Content-Security-Policy (CSP) HTTP headers to ensure SuiteCommerce Advanced (SCA) sites prior to 2020.1.4 cannot be framed by any other site or origin. These patch instructions describe how to add these HTTP security headers to help prevent any potential security risks.



Note: XFO is being deprecated by most modern browsers in favor of CSP. But some browsers, such as versions of Internet Explorer that predate Edge, still do not support CSP. These patch instructions include both types of security headers to help minimize security risks on all browsers.

To learn more about these security headers and the options available with them, refer to the following resources:

- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>

To implement this patch, choose from the following options and then refer to the instructions for that option.

Your site does not need to be framed by any other site/origin	Framing Not Required
Your site needs to be framed by another single site/origin	Framing by a Single Site/Origin

Your site needs to be framed by multiple sites/origins	Framing by Multiple Sites/Origins  Note: X-Frame-Options (XFO) do not support multiple sites/origins, so these instructions include only the Content-Security-Policy (CSP) header.
Your site needs to be framed by any site/origin	Framing by any Site

SSP Files

These patch instructions require that you add headers to the SSP files in your local SCA directory. Where the SSP files are stored depends on the release of SCA that you have installed. To locate all required files, you can search your SCA directory using a placeholder. For example: *.ssp

Current SCA installations will include the following .ssp files, while older versions may not include all of these files:

- shopping.ssp
- checkout.ssp
- my_account.ssp
- challengeShopperForm.ssp
- deviceAuthenticationForm.ssp
- threedsecure.ssp
- external_payment.ssp
- download.ssp
- print-statement.ssp
- goToCart.ssp
- logOut.ssp
- redirections.ssp
- searchApi.ssp
- index-local.ssp (Each application includes an index-local.ssp file.)
- shopping-local.ssp, checkout-local.ssp, ... (When you compile local SCA source by running `gulp deploy`, SCA generates these files in the `DeployDistribution` directory.)

Framing Not Required

If your site does not need to be framed by any other site, add the following lines to all SSP files in your local SCA directory. For example, shopping.ssp, checkout.ssp, and so on. For more information about the files to update, see [SSP Files](#).

1. At the very top of each .ssp file, copy and paste the following lines to set the headers. Make sure to add these lines inside a `<%` block as shown in the following code.



Important: These lines allow your site to be framed only by itself, which is required to ensure that Site Management Tools (SMT) continue to function properly.

```

1 <%
2 response.addHeader('X-Frame-Options', 'SAMEORIGIN');
3 response.addHeader('Content-Security-Policy', 'frame-ancestors \'self\'');
```

```
4 | %>
```

2. After adding the headers, deploy the updated source files. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.

Framing by a Single Site/Origin

If your site does need to be framed by another single site, add the following lines to all SSP files in your local SCA directory. For example, shopping.ssp, checkout.ssp, and so on. For more information about the files to update, see [SSP Files](#).

1. At the very top of each .ssp file, copy and paste the following lines to set the headers. Make sure to:
 - Add these lines inside a <% block as shown in the following code.
 - Change tempdomain.com in the URL on each of these lines to the domain for the single site that is allowed to frame your site. Note that the URL for the allowed site must include http:// or https://.

```
1 | <%
2 | response.addHeader('X-Frame-Options', 'ALLOW-FROM http://tempdomain.com');
3 | response.addHeader('Content-Security-Policy', 'frame-ancestors http://tempdomain.com');
4 | %>
```

2. After adding the headers, deploy the updated source files. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.

Framing by Multiple Sites/Origins

If your site needs to be framed by multiple sites, add the following line to all SSP files in your local SCA directory. For example, shopping.ssp, checkout.ssp, and so on. For more information about the files to update, see [SSP Files](#).

Note: X-Frame-Options (XFO) do not support multiple sites/origins, so these instructions include only the Content-Security-Policy (CSP) header.

1. At the very top of each .ssp file, copy and paste the following line to set the header. Make sure to:
 - Add this line inside a <% block as shown in the following code.
 - Include your own site as shown in the following code to ensure that Site Management Tools (SMT) continue to function properly.
 - Change tempdomain.com in the URLs to the domains for the sites that are allowed to frame your site.

```
1 | <%
2 | response.addHeader('Content-Security-Policy', 'frame-ancestors \'self\' http://tempdomain1.com http://tempdomain2.com');
3 | %>
```

2. After adding the header, deploy the updated source files. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.

Framing by any Site

If you don't want to restrict framing and want any site to be able to frame your site, you do not need to make any changes.



Important: Allowing your site to be framed by any other site by omitting security headers makes your site vulnerable to clickjacking attacks.

Enable Token-Based Authentication

Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus | Kilimanjaro | Aconcagua | 2018.2 | 2019.1 | 2019.2 | 2020.1 | 2020.2 | Site Builder

With the release of NetSuite 2021.1, all versions of the developer tools, including the extension, theme, and SuiteCommerce Advanced command-line developer tools, require token-based authentication. Developer tools can no longer accept NetSuite user credentials for authentication. This update to token-based authentication is required for compliance with the required Two-Factor Authentication (2FA) policy for NetSuite administrators and other highly privileged roles.

SuiteCommerce Advanced (SCA) 2020.2.4 and later releases support token-based authentication. If you cannot migrate to the latest release of SCA, you can prepare your current installation of SCA for token-based authentication as described in the following patch instructions. Be sure to follow the patch instructions appropriate for the version of SuiteCommerce Advanced you have implemented.

Implementation	Patch Instructions
SCA Aconcagua, 2018.2, 2019.1, 2019.2, 2020.1, and 2020.2	Enable Token-Based Authentication (Aconcagua and Later Versions)
SCA Montblanc, Vinson, Elbrus, and Kilimanjaro Site Builder Extensions and Extensions Premium Montblanc, Vinson, Elbrus, and Kilimanjaro	Enable Token-Based Authentication (Montblanc, Vinson, Elbrus, and Kilimanjaro)
SCA Denali Site Builder Extensions and Extensions Premium Denali	Enable Token-Based Authentication (Denali)



Note: For all SCA versions:

- You must enable the Token Based Authentication feature in your NetSuite account to update to token-based authentication. For more information, see the help topic [Enable the Token-based Authentication Feature](#).
- After you complete the steps in these patch instructions, you must continue with additional updates for token-based authentication. For detailed instructions, see the help topic [Token-Based Authentication Updates for Developer Tools](#).

Enable Token-Based Authentication (Aconcagua and Later Versions)

Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus | Kilimanjaro | Aconcagua | 2018.2 | 2019.1 | 2019.2 | 2020.1 | 2020.2 | Site Builder

Prepare your SuiteCommerce Advanced implementation to use token-based authentication with the developer tools, including the extension, theme, and SuiteCommerce Advanced tools, as described in the following steps. You need to update several files, add new files, and then also download new theme and

extension developer tools, so be sure to complete all of the required steps as outlined in the following procedures:

- [Modify the index.js File for Gulp Deployments](#)
- [Modify the tool.js File in the NPM Repository](#)
- [Modify the getDataCenterUrls.tpl File](#)
- [Modify the template.js File in the NPM Repository](#)
- [Modify the communication.js File in the NPM Repository](#)
- [Modify the passport.tpl File in the NPM Repository](#)
- [Modify package.json Dependencies](#)
- [Create the oauth1 Directory](#)
- [Replace Gulp Files](#)
- [Run NPM Command for oauth1](#)

Modify the index.js File for Gulp Deployments

1. In the SuiteCommerce Advanced source directory, open this file: `.../gulp/ns-deploy/index.js`
2. Find the following line of code:

```
1 | , doUntilGetRoles
```

And replace it with the following lines of code:

```
1 | , ui.selectToken
2 | , net.authorize
```

3. Find the following lines of code:

```
1 | async.apply(ui.roles, deploy),
2 | net.targetFolder,
```

And replace them with the following code:

```
1 | async.apply(net.targetFolder, deploy),
```

4. Find the following lines of code:

```
1 | async.apply(ui.roles, deploy),
2 | net.getWebsitesAndDomains,
```

And replace them with the following code:

```
1 | async.apply(net.getWebsitesAndDomains, deploy),
```

5. Find the following lines of code:

```
1 | , ui.email
2 | , ui.password
3 | , net.roles
4 | , ui.roles
```

And replace them with the following code:

```
1 | , ui.selectToken
2 | , net.authorize
```

Modify the tool.js File in the NPM Repository

1. In the SuiteCommerce Advanced source directory, open this file: `.../ns_npm_repository/suitetalk4node/src/tool.js`

Note: In versions 2018.2 and 2019.1, the `ns_npm_repository` folder resides at the top level of the SuiteCommerce Advanced source directory. In versions 2019.2 through 2020.2, the `ns_npm_repository` folder resides in the Commons directory.

2. Find the following line of code:

```
1 | nsVersion: '2015_1'
```

And replace it with the following code:

```
1 | nsVersion: '2024_2'
```

Modify the getDataCenterUrls.tpl File

1. In the SuiteCommerce Advanced source directory, open this file: `.../ns_npm_repository/suitetalk4node/src/templates/getDataCenterUrls.tpl`

Note: In versions 2018.2 and 2019.1, the `ns_npm_repository` folder resides at the top level of the SuiteCommerce Advanced source directory. In versions 2019.2 through 2020.2, the `ns_npm_repository` folder resides in the Commons directory.

2. Find the following line of code:

```
1 | <nsmessages:account>{{account}}</nsmessages:account>
```

And replace it with the following code:

```
1 | <nsmessages:account>{{credentials.account}}</nsmessages:account>
```

Modify the template.js File in the NPM Repository

1. In the SuiteCommerce Advanced source directory, open this file: `.../ns_npm_repository/suitetalk4node/src/template.js`

Note: In versions 2018.2 and 2019.1, the `ns_npm_repository` folder resides at the top level of the SuiteCommerce Advanced source directory. In versions 2019.2 through 2020.2, the `ns_npm_repository` folder resides in the Commons directory.

2. Find the following lines of code:

```
1 | var Handlebars = require('handlebars')
2 | , fs = require('fs')
```

And replace them with the following code:

```
1 | var Handlebars = require('handlebars')
2 | , OAuth1 = require('oauth1').OAuth1
3 | , fs = require('fs')
```

3. Find the `_template` method:

```

1 | , _template: function(file_name, params)
2 | {
3 |     this._initTemplateContext(params);
4 |     var fileName = path.join(__dirname, '/templates/', file_name);
5 |     var template = Handlebars.compile(fs.readFileSync(fileName, {encoding: 'utf8'})).toString();
6 |     return template(params);
7 | }

```

And replace it with the following code:

```

1 | , _template: function(file_name, params)
2 | {
3 |     var self = this;
4 |     var oauth1 = new OAuth1(this.credentials);
5 |     return oauth1.soapAuthorize(this.credentials.authID).then(function(credentials) {
6 |         self._initTemplateContext(params);
7 |         params.credentials = params.credentials || {};
8 |         params.credentials.vm = self.credentials.vm;
9 |         params.credentials.molecule = self.credentials.molecule;
10 |        params.credentials.authID = self.credentials.authID;
11 |        params.credentials.token = credentials.token || self.credentials.token;
12 |        params.credentials.signature = credentials.signature || self.credentials.signature;
13 |        params.credentials.nonce = credentials.nonce || self.credentials.nonce;
14 |        params.credentials.timestamp = credentials.timestamp || self.credentials.timestamp;
15 |        params.credentials.account = credentials.account || self.credentials.account;
16 |        params.credentials.consumerKey = credentials.consumerKey || self.credentials.consumerKey;
17 |        var fileName = path.join(__dirname, '/templates/', file_name);
18 |        var template = Handlebars.compile(fs.readFileSync(fileName, {encoding: 'utf8'})).toString();
19 |        return template(params);
20 |    });
21 | }

```

Modify the communication.js File in the NPM Repository

1. In the SuiteCommerce Advanced source directory, open this file: `.../ns_npm_repository/suitetalk4node/src/communications.js`

Note: In versions 2018.2 and 2019.1, the `ns_npm_repository` folder resides at the top level of the SuiteCommerce Advanced source directory. In versions 2019.2 through 2020.2, the `ns_npm_repository` folder resides in the Commons directory.

2. Find the following lines of code:

```

1 | , _request: function(action, payload, cb)
2 | {

```

And replace them with the following code:

```

1 | , _request: function(action, payload, cb)
2 | {
3 |     cb = cb || function(){};

```

3. Find the `_request` method:

```

1 | , _request: function(action, payload, cb)
2 | {
3 |     var self = this
4 |     , deferred = Q.defer();
5 |     cb = cb || function(){};
6 |     var datacenterDomain = self.dataCenterDomains && self.dataCenterDomains.webservices || this.getDefaultWebSer
viceUrl();
7 |     // console.log('datacenterDomain', datacenterDomain, datacenterDomain + '/services/NetSuitePort_' + self.nsVer
sion)

```

```

8
9      this.getQueue().add(self, function(taskDone)
10     {
11         var req = request({
12             method: 'POST'
13             , uri: datacenterDomain + '/services/NetSuitePort_' + self.nsVersion
14             , headers: {
15                 'User-Agent': 'Node-SOAP/0.0.1'
16                 , 'Accept': 'text/html,application/xhtml+xml,application/xml;text/xml;q=0.9,*/*;q=0.8'
17                 , 'Accept-Encoding': 'none'
18                 , 'Accept-Charset': 'utf-8'
19                 , 'Connection': 'close'
20                 , 'Content-Type': 'text/xml; charset=utf-8'
21                 , 'SOAPAction': '"" + action + ""'
22                 , 'Expect': '100-continue'
23             }
24         }, function(err, response)
25         {
26             if (err)
27             {
28                 return cb(err);
29             }
30
31             self.log('Response text for action: ' + action + '\n' + response.body);
32             xml2js.parseString(response.body, self._xml2jsOptions, function(err, result)
33             {
34                 if (err)
35                 {
36                     deferred.reject(err);
37                     return cb(err);
38                 }
39                 var soap_body = result.Envelope.Body[0];
40                 if (soap_body && soap_body.Fault)
41                 {
42                     deferred.reject(new Error(soap_body.Fault[0].faultstring[0]));
43                     return cb(new Error(soap_body.Fault[0].faultstring[0]));
44                 }
45                 deferred.resolve(soap_body);
46                 taskDone();
47
48                 cb(null, soap_body, result);
49             });
50         });
51
52         self.log('Request text for action: ' + action + '\n' + payload);
53         req.end(payload);
54     });
55
56     return deferred.promise;
57 }

```

And replace it with the following code:

```

1  , __request: function(action, payload, cb) {
2      const self = this;
3
4      cb = cb || function() {};
5      const datacenterDomain =
6          (self.dataCenterDomains && self.dataCenterDomains.webservices) ||
7          this.getDefaultWebServiceUrl();
8
9      const args = require('yargs').argv;
10     if (args.proxy) {
11         request = request.defaults({ proxy: args.proxy });
12     }
13     return payload.then(function(payloadResult) {
14         return new Promise(function(resolve, reject) {
15             self.getQueue().add(self, function(taskDone) {
16                 const req = request(
17                     {
18                         method: 'POST',
19                         uri: datacenterDomain + "/services/NetSuitePort_" + self.nsVersion,

```



```

20         headers: {
21             'User-Agent': self.credentials.user_agent || 'Node-SOAP/0.0.1',
22             Accept:
23                 'text/html,application/xhtml+xml,application/xml;text/xml;q=0.9,*/*;q=0.8',
24             'Accept-Encoding': 'none',
25             'Accept-Charset': 'utf-8',
26             Connection: 'close',
27             'Content-Type': 'text/xml; charset=utf-8',
28             SOAPAction: '' + action + ''
29         }
30     },
31     function(err, response) {
32         if (err) {
33             return cb(err);
34         }
35
36         xml2js.parseString(response.body, self._xml2jsOptions, function(
37             err,
38             result
39         ) {
40             if (err) {
41                 reject(err);
42                 return cb(err);
43             }
44             const soap_body = result.Envelope.Body[0];
45             if (soap_body && soap_body.Fault) {
46                 reject(new Error(soap_body.Fault[0].faultstring[0]));
47                 return cb(new Error(soap_body.Fault[0].faultstring[0]));
48             }
49             resolve(soap_body);
50             taskDone();
51
52             cb(null, soap_body, result);
53         });
54     }
55 );
56
57 req.end(payloadResult);
58 });
59 });
60 });
61 }

```

Modify the passport.tpl File in the NPM Repository

1. In the SuiteCommerce Advanced source directory, open this file: `.../ns_npm_repository/suitetalk4node/src/templates/_passport.tpl`



Note: In versions 2018.2 and 2019.1, the `ns_npm_repository` folder resides at the top level of the SuiteCommerce Advanced source directory. In versions 2019.2 through 2020.2, the `ns_npm_repository` folder resides in the Commons directory.

2. Find the following lines of code:

```

1 <nsmessages:passport>
2   {{#if credentials.email}}<nscore:email>{{credentials.email}}</nscore:email>{{/if}}
3   {{#if credentials.password}}<nscore:password>{{credentials.password}}</nscore:password>{{/if}}
4   {{#if credentials.account}}<nscore:account>{{credentials.account}}</nscore:account>{{/if}}
5   {{#if credentials.roleId}}<nscore:role internalId="{{credentials.roleId}}"/>{{/if}}
6 </nsmessages:passport>

```

And replace them with the following code:

```

1 <ns:tokenPassport actor="http://schemas.xmlsoap.org/soap/actor/next" mustUnderstand="0" xmlns:ns="urn:mes
  sages_2020_1.platform.webservices.netsuite.com">
2   {{#if credentials.account}}<ns:account>{{credentials.account}}</ns:account>{{/if}}
3   {{#if credentials.consumerKey}}<ns:consumerKey>{{credentials.consumerKey}}</ns:consumerKey>{{/if}}

```

```

4  {{#if credentials.token}}<ns:token>{{credentials.token}}</ns:token>{{/if}}
5  {{#if credentials.nonce}}<ns:nonce>{{credentials.nonce}}</ns:nonce>{{/if}}
6  {{#if credentials.timestamp}}<ns:timestamp>{{credentials.timestamp}}</ns:timestamp>{{/if}}
7  {{#if credentials.signature}}<ns:signature algorithm="HMAC_SHA256">{{credentials.signature}}</ns:signature>{{/if}}
8  </ns:tokenPassport>

```

Modify package.json Dependencies

1. In the top level of the SuiteCommerce Advanced source directory, open the package.json file.
2. Add the following line to the dependencies:

```
1 | "oauth1": "file:../ns_npm_repository/oauth1",
```

As shown in the following example excerpt:

```

1 | "dependencies": {
2 |   "oauth1": "file:../ns_npm_repository/oauth1",
3 |   "ansi-colors": "2.0.1",
4 |   "archiver": "2.1.1",
5 |   "async": "2.6.1",

```

Create the oauth1 Directory

1. In the SuiteCommerce Advanced source directory, create a new directory named **oauth1** in the ns_npm_repository folder: `.../ns_npm_repository/oauth1`



Note: In versions 2018.2 and 2019.1, the ns_npm_repository folder resides at the top level of the SuiteCommerce Advanced source directory. In versions 2019.2 through 2020.2, the ns_npm_repository folder resides in the Commons directory.

2. Copy the source files from the oauth1 folder in the [TBAPatch-aconcagua-18.2-19.2-20.1-20.2.zip](#) file to the new oauth1 directory you created in the preceding step.

Replace Gulp Files

Replace the following files in the SuiteCommerce Advanced source directory with the modified versions of the same files available in the .zip file for this patch: [TBAPatch-aconcagua-18.2-19.2-20.1-20.2.zip](#)

- .../gulp/ns-deploy/net.js
- .../gulp/ns-deploy/ui.js

Run NPM Command for oauth1

After updating all files as required, adding new files from the .zip file for the patch, and downloading and setting up the theme and extension developer tools, you must run the following command to ensure that token-based authentication functions properly.

Run the following command at the top level of the SuiteCommerce Advanced source directory:

```
npm i oauth1 suitetalk
```

When this command completes, continue with additional updates for token-based authentication. For detailed instructions, see the help topic [Token-Based Authentication Updates for Developer Tools](#).

Enable Token-Based Authentication (Montblanc, Vinson, Elbrus, and Kilimanjaro)

① Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus | Kilimanjaro | Aconcagua | 2018.2 | 2019.1 | 2019.2 | 2020.1 | 2020.2 | Site Builder

Prepare your SuiteCommerce Advanced (SCA) or Site Builder Extensions implementation to use token-based authentication with the developer tools, including the extension, theme, and SuiteCommerce Advanced tools, as described in the following steps. You need to update several files, add new files, and then also download new theme and extension developer tools, so be sure to complete all of the required steps as outlined in the following procedures:

- [Modify the index.js File for Gulp Deployments](#)
- [Modify the tool.js File in the NPM Repository](#)
- [Modify the getDataCenterUrls.tpl File](#)
- [Modify the template.js File in the NPM Repository](#)
- [Modify the communication.js File in the NPM Repository](#)
- [Modify the passport.tpl File in the NPM Repository](#)
- [Modify the fs.js File \(Vinson and Montblanc\)](#)
- [Modify package.json Dependencies](#)
- [Create the oauth1 Directory](#)
- [Replace Gulp Files](#)
- [Run NPM Command for oauth1](#)
- [Additional Steps for SCA Vinson](#)

Modify the index.js File for Gulp Deployments

1. In the SuiteCommerce Advanced source directory, open this file: `.../gulp/ns-deploy/index.js`
2. Find the following line of code:

```
1 | , doUntilGetRoles
```

And replace it with the following lines of code:

```
1 | , ui.selectToken
2 | , net.authorize
```

3. Find the following lines of code:

```
1 | async.apply(ui.roles, deploy),
2 | net.targetFolder,
```

And replace them with the following code:

```
1 | async.apply(net.targetFolder, deploy),
```

4. This step is required only if the version of your SCA implementation is Kilimanjaro. If so, find the following lines of code:

```
1 | async.apply(ui.roles, deploy),
2 | net.getWebsitesAndDomains,
```

And replace them with the following code:

```
1 | async.apply(net.getWebsitesAndDomains, deploy),
```

- Find the following lines of code:

```
1 | , ui.email
2 | , ui.password
3 | , net.roles
4 | , ui.roles
```

And replace them with the following code:

```
1 | , ui.selectToken
2 | , net.authorize
```

Modify the tool.js File in the NPM Repository

- In the SuiteCommerce Advanced source directory, open this file: `.../ns_npm_repository/suitetalk4node/src/tool.js`
- Find the following line of code:

```
1 | nsVersion: '2015_1'
```

And replace it with the following code:

```
1 | nsVersion: '2024_2'
```

Modify the getDataCenterUrls.tpl File

- In the SuiteCommerce Advanced source directory, open this file: `.../ns_npm_repository/suitetalk4node/src/templates/getDataCenterUrls.tpl`
- Find the following line of code:

```
1 | <nsmessages:account>{{account}}</nsmessages:account>
```

And replace it with the following code:

```
1 | <nsmessages:account>{{credentials.account}}</nsmessages:account>
```

Modify the template.js File in the NPM Repository

- In the SuiteCommerce Advanced source directory, open this file: `.../ns_npm_repository/suitetalk4node/src/template.js`
- Find the following lines of code:

```
1 | var Handlebars = require('handlebars')
2 | , fs = require('fs')
```

And replace them with the following code:

```
1 | var Handlebars = require('handlebars')
2 | , OAuth1 = require('oauth1').OAuth1
3 | , fs = require('fs')
```

- Find the `_template` method:

```

1 | , _template: function(file_name, params)
2 | {
3 |     this._initTemplateContext(params);
4 |     var fileName = path.join(__dirname, '/templates/', file_name);
5 |     var template = Handlebars.compile(fs.readFileSync(fileName, {encoding: 'utf8'})).toString();
6 |     return template(params);
7 | }

```

And replace it with the following code:

```

1 | , _template: function(file_name, params)
2 | {
3 |     var self = this;
4 |     var oauth1 = new OAuth1(this.credentials);
5 |     return oauth1.soapAuthorize(this.credentials.authID).then(function(credentials) {
6 |         self._initTemplateContext(params);
7 |         params.credentials = params.credentials || {};
8 |         params.credentials.vm = self.credentials.vm;
9 |         params.credentials.molecule = self.credentials.molecule;
10 |        params.credentials.authID = self.credentials.authID;
11 |        params.credentials.token = credentials.token || self.credentials.token;
12 |        params.credentials.signature = credentials.signature || self.credentials.signature;
13 |        params.credentials.nonce = credentials.nonce || self.credentials.nonce;
14 |        params.credentials.timestamp = credentials.timestamp || self.credentials.timestamp;
15 |        params.credentials.account = credentials.account || self.credentials.account;
16 |        params.credentials.consumerKey = credentials.consumerKey || self.credentials.consumerKey;
17 |        var fileName = path.join(__dirname, '/templates/', file_name);
18 |        var template = Handlebars.compile(fs.readFileSync(fileName, {encoding: 'utf8'})).toString();
19 |        return template(params);
20 |    });
21 | }

```

Modify the communication.js File in the NPM Repository

1. In the SuiteCommerce Advanced source directory, open this file: `.../ns_npm_repository/suitetalk4node/src/communications.js`
2. Find the following lines of code:

```

1 | , _request: function(action, payload, cb)
2 | {

```

And replace them with the following code:

```

1 | , _request: function(action, payload, cb)
2 | {
3 |     cb = cb || function(){};

```

3. Find the `_request` method:

```

1 | , _request: function(action, payload, cb)
2 | {
3 |     var self = this
4 |     , deferred = Q.defer();
5 |     cb = cb || function(){};
6 |     var datacenterDomain = self.dataCenterDomains && self.dataCenterDomains.webservices || this.getDefaultWebSer
viceUrl();
7 |     // console.log('datacenterDomain', datacenterDomain, datacenterDomain + '/services/NetSuitePort_' + self.nsVer
sion)
8 |
9 |     this.getQueue().add(self, function(taskDone)
10 |    {
11 |        var req = request({
12 |            method: 'POST'
13 |            , uri: datacenterDomain + '/services/NetSuitePort_' + self.nsVersion

```

```

14     , headers: {
15         'User-Agent': 'Node-SOAP/0.0.1'
16     , 'Accept': 'text/html,application/xhtml+xml,application/xml,text/xml;q=0.9,*/*;q=0.8'
17     , 'Accept-Encoding': 'none'
18     , 'Accept-Charset': 'utf-8'
19     , 'Connection': 'close'
20     , 'Content-Type': 'text/xml; charset=utf-8'
21     , 'SOAPAction': '"" + action + ""'
22     , 'Expect': '100-continue'
23     }
24 }, function(err, response)
25 {
26     if (err)
27     {
28         return cb(err);
29     }
30
31     self.log('Response text for action: ' + action + '\n' + response.body);
32     xml2js.parseString(response.body, self._xml2jsOptions, function(err, result)
33     {
34         if (err)
35         {
36             deferred.reject(err);
37             return cb(err);
38         }
39         var soap_body = result.Envelope.Body[0];
40         if (soap_body && soap_body.Fault)
41         {
42             deferred.reject(new Error(soap_body.Fault[0].faultstring[0]));
43             return cb(new Error(soap_body.Fault[0].faultstring[0]));
44         }
45         deferred.resolve(soap_body);
46         taskDone();
47
48         cb(null, soap_body, result);
49     });
50 });
51
52 self.log('Request text for action: ' + action + '\n' + payload);
53 req.end(payload);
54 });
55
56 return deferred.promise;
57 }

```

And replace it with the following code:

```

1  , __request: function(action, payload, cb) {
2      const self = this;
3
4      cb = cb || function() {};
5      const datacenterDomain =
6          (self.dataCenterDomains && self.dataCenterDomains.webservices) ||
7          this.getDefaultWebServiceUrl();
8
9      const args = require('yargs').argv;
10     if (args.proxy) {
11         request = request.defaults({ proxy: args.proxy });
12     }
13     return payload.then(function(payloadResult) {
14         return new Promise(function(resolve, reject) {
15             self.getQueue().add(self, function(taskDone) {
16                 const req = request(
17                     {
18                         method: 'POST',
19                         uri: datacenterDomain + "/services/NetSuitePort_" + self.nsVersion,
20                         headers: {
21                             'User-Agent': self.credentials.user_agent || 'Node-SOAP/0.0.1',
22                             Accept:
23                                 'text/html,application/xhtml+xml,application/xml,text/xml;q=0.9,*/*;q=0.8',
24                             'Accept-Encoding': 'none',
25                             'Accept-Charset': 'utf-8',

```

```

26         Connection: 'close',
27         'Content-Type': 'text/xml; charset=utf-8',
28         SOAPAction: '"" + action + ""'
29     }
30 },
31 function(err, response) {
32     if (err) {
33         return cb(err);
34     }
35
36     xml2js.parseString(response.body, self._xml2jsOptions, function(
37         err,
38         result
39     ) {
40         if (err) {
41             reject(err);
42             return cb(err);
43         }
44         const soap_body = result.Envelope.Body[0];
45         if (soap_body && soap_body.Fault) {
46             reject(new Error(soap_body.Fault[0].faultstring[0]));
47             return cb(new Error(soap_body.Fault[0].faultstring[0]));
48         }
49         resolve(soap_body);
50         taskDone();
51
52         cb(null, soap_body, result);
53     });
54 }
55 );
56
57 req.end(payloadResult);
58 });
59 });
60 });
61 }

```

Modify the passport.tpl File in the NPM Repository

1. In the SuiteCommerce Advanced source directory, open this file: `.../ns_npm_repository/suitetalk4node/src/templates/_passport.tpl`
2. Find the following lines of code:

```

1 <ns:messages:passport>
2     {{#if credentials.email}}<nscore:email>{{credentials.email}}</nscore:email>{{/if}}
3     {{#if credentials.password}}<nscore:password>{{credentials.password}}</nscore:password>{{/if}}
4     {{#if credentials.account}}<nscore:account>{{credentials.account}}</nscore:account>{{/if}}
5     {{#if credentials.roleId}}<nscore:role internalId="{{credentials.roleId}}"/>{{/if}}
6 </ns:messages:passport>

```

And replace them with the following code:

```

1 <ns:tokenPassport actor="http://schemas.xmlsoap.org/soap/actor/next" mustUnderstand="0" xmlns:ns="urn:mes
sages_2020_1.platform.webservices.netsuite.com">
2     {{#if credentials.account}}<ns:account>{{credentials.account}}</ns:account>{{/if}}
3     {{#if credentials.consumerKey}}<ns:consumerKey>{{credentials.consumerKey}}</ns:consumerKey>{{/if}}
4     {{#if credentials.token}}<ns:token>{{credentials.token}}</ns:token>{{/if}}
5     {{#if credentials.nonce}}<ns:nonce>{{credentials.nonce}}</ns:nonce>{{/if}}
6     {{#if credentials.timestamp}}<ns:timestamp>{{credentials.timestamp}}</ns:timestamp>{{/if}}
7     {{#if credentials.signature}}<ns:signature algorithm="HMAC_SHA256">{{credentials.signature}}</ns:signature>{{/if}}
8 </ns:tokenPassport>

```

Modify the fs.js File (Vinson and Montblanc)

Follow these steps only if your SCA implementation is Vinson or Montblanc.

1. In the SuiteCommerce Advanced source directory, open this file: `.../gulp/ns-deploy/fs.js`

- Find the following lines of code:

```
1 | , processBackup: function(deploy, took, cb)
2 | {
3 |     guttil.log('Finished', guttil.colors.cyan('Deploy website' + (took ? ', took ' + took : '' ) ));
```

And replace them with the following code:

```
1 | , processBackup: function(deploy, cb)
2 | {
3 |     guttil.log('Finished', guttil.colors.cyan('Deploy website') );
```

- Find the following line of code:

```
1 | cb(null, deploy, context, took);
```

And replace it with the following code:

```
1 | cb(null, deploy, context);
```

Modify package.json Dependencies

- In the top level of the SuiteCommerce Advanced source directory, open the package.json file.
- Add the following line to the dependencies:

```
1 | "oauth1": "file:./ns_npm_repository/oauth1",
```

As shown in the following example excerpt:

```
1 | "dependencies": {
2 |     "oauth1": "file:./ns_npm_repository/oauth1",
3 |     "ansi-colors": "2.0.1",
4 |     "archiver": "2.1.1",
5 |     "async": "2.6.1",
```

Create the oauth1 Directory

- In the SuiteCommerce Advanced source directory, create a new directory named **oauth1** in the ns_npm_repository folder: `../ns_npm_repository/oauth1`
- Copy the source files from the oauth1 folder in the [TBAPatch-elbrus-vinson-kilimanjaro.zip](#) file to the new oauth1 directory you created in the preceding step.

Replace Gulp Files

Replace the following files in the SuiteCommerce Advanced source directory with the modified versions of the same files available in the .zip file for this patch: [TBAPatch-elbrus-vinson-kilimanjaro.zip](#)

- ../gulp/ns-deploy/net.js
- ../gulp/ns-deploy/ui.js

Run NPM Command for oauth1

After updating all files as required, adding new files from the .zip file for the patch, and downloading and setting up the theme and extension developer tools, you must run the following command to ensure that token-based authentication functions properly.

Run the following command at the top level of the SuiteCommerce Advanced source directory:


```
npm i oauth1 suitetalk
```

When this command completes, continue with additional updates for token-based authentication. For detailed instructions, see the help topic [Token-Based Authentication Updates for Developer Tools](#).

Additional Steps for SCA Vinson

In some implementations of the Vinson release of SCA, completing the preceding steps can return the error message, "getFileNameSearchError undefined," after you run the `gulp deploy` command.

The following procedure describes the additional steps to enable token-based authentication on Vinson implementations of SCA if you encounter errors.

To enable token-based authentication on SCA Vinson:

1. Prepare your Vinson installation of SCA for token-based authentication as described in the preceding procedures.
2. In the top level of the SCA source directory, open the `package.json` file.
3. Add the following line to the dependencies:

```
1 | "xmlbuilder": "8.0.0",
```

4. Save the file and close it.
5. Run the following command at the top level of the SCA source directory:
`npm cache clear -force`
6. After the command successfully completes, run the following command in the same directory:
`npm install`
7. After steps 5 and 6 are successfully completed, go to the `node_modules\gulp-ttf2eot` directory and open the `package.json` file.
8. Find the following line of code:

```
1 | "ttf2eot": "latest"
```

9. And replace it with the following code:

```
1 | "ttf2eot": "2.0.0"
```

10. Save the file and close it.
11. From the `node_modules\gulp-ttf2eot` directory, run the following command:
`npm install`
12. Go back to the top level of the SCA source directory, then go to the `node_modules\gulp-ttf2woff` directory and open the `package.json` file.
13. Find the following line of code:

```
1 | "ttf2woff": "latest"
```

14. And replace it with the following code:

```
1 | "ttf2woff": "2.0.2"
```

15. Save the file and close it.
16. From the `node_modules\gulp-ttf2woff`, run the following command:
`npm install`

17. Go back to the top level of the SCA source directory and run the following command:

```
gulp deploy
```

Enable Token-Based Authentication (Denali)

❗ Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus | Kilimanjaro | Aconcagua | 2018.2 | 2019.1 | 2019.2 | 2020.1 | 2020.2 | Site Builder

Prepare your SuiteCommerce Advanced or Site Builder Extensions implementation to use token-based authentication with the developer tools, including the extension, theme, and SuiteCommerce Advanced tools, as described in the following steps. You need to update several files, add new files, and then also download new developer tools, so be sure to complete all of the required steps as outlined in the following procedures:

- [Modify the index.js File for Gulp Deployments](#)
- [Modify the tool.js File](#)
- [Modify the getDataCenterUrls.tpl File](#)
- [Modify the template.js File](#)
- [Modify the communication.js File](#)
- [Modify the passport.tpl File](#)
- [Modify package.json Dependencies](#)
- [Modify the fs.js File](#)
- [Modify the filecabinet-util.js File](#)
- [Create the ns_npm_repository/oauth1 Directory](#)
- [Replace Gulp Files](#)
- [Run NPM Command for oauth1](#)

Modify the index.js File for Gulp Deployments

1. In the SuiteCommerce Advanced source directory, open this file: `.../gulp/ns-deploy/index.js`
2. Find the following line of code:

```
1 | , doUntilGetRoles
```

And replace it with the following lines of code:

```
1 | , ui.selectToken
2 | , net.authorize
```

3. Find the following lines of code:

```
1 | async.apply(ui.roles, deploy),
2 | net.targetFolder,
```

And replace them with the following code:

```
1 | async.apply(net.targetFolder, deploy),
```

4. Find the following lines of code:

```
1 | , ui.email
2 | , ui.password
```

```
3 | , net.roles
4 | , ui.roles
```

And replace them with the following code:

```
1 | , ui.selectToken
2 | , net.authorize
```

Modify the tool.js File

1. In the SuiteCommerce Advanced source directory, open this file: `.../gulp/ns-deploy/suitetalk4node/src/tool.js`
2. Find the following line of code:

```
1 | nsVersion: '2015_1'
```

And replace it with the following code:

```
1 | nsVersion: '2024_2'
```

Modify the getDataCenterUrls.tpl File

1. In the SuiteCommerce Advanced source directory, open this file: `.../gulp/ns-deploy/suitetalk4node/src/templates/getDataCenterUrls.tpl`
2. File the following line:

```
1 | <nsmessages:account>{{account}}</nsmessages:account>
```

And replace it with the following code:

```
1 | <nsmessages:account>{{credentials.account}}</nsmessages:account>
```

Modify the template.js File

1. In the SuiteCommerce Advanced source directory, open this file: `.../gulp/ns-deploy/suitetalk4node/src/template.js`
2. Find the following lines of code:

```
1 | var Handlebars = require('handlebars')
2 | , fs = require('fs')
```

And replace them with the following code:

```
1 | var Handlebars = require('handlebars')
2 | , OAuth1 = require('oauth1').OAuth1
3 | , fs = require('fs')
```

3. Find the `_template` method:

```
1 | , _template: function(file_name, params)
2 | {
3 |     this._initTemplateContext(params);
4 |     var fileName = path.join(__dirname, '/templates/', file_name);
5 |     var template = Handlebars.compile(fs.readFileSync(fileName, {encoding: 'utf8'})).toString();
6 |     return template(params);
7 | }
```

And replace it with the following code:

```

1 | , _template: function(file_name, params)
2 | {
3 |     var self = this;
4 |     var oauth1 = new OAuth1(this.credentials);
5 |     return oauth1.soapAuthorize(this.credentials.authID).then(function(credentials) {
6 |         self._initTemplateContext(params);
7 |         params.credentials = params.credentials || {};
8 |         params.credentials.vm = self.credentials.vm;
9 |         params.credentials.molecule = self.credentials.molecule;
10 |         params.credentials.authID = self.credentials.authID;
11 |         params.credentials.token = credentials.token || self.credentials.token;
12 |         params.credentials.signature = credentials.signature || self.credentials.signature;
13 |         params.credentials.nonce = credentials.nonce || self.credentials.nonce;
14 |         params.credentials.timestamp = credentials.timestamp || self.credentials.timestamp;
15 |         params.credentials.account = credentials.account || self.credentials.account;
16 |         params.credentials.consumerKey = credentials.consumerKey || self.credentials.consumerKey;
17 |         var fileName = path.join(__dirname, '/templates/', file_name);
18 |         var template = Handlebars.compile(fs.readFileSync(fileName, {encoding: 'utf8'})).toString();
19 |         return template(params);
20 |     });
21 | }

```

Modify the communication.js File

1. In the SuiteCommerce Advanced source directory, open this file: .../gulp/ns-deploy/suitetalk4node/src/communications.js
2. Find the following lines of code:

```

1 | , _request: function(action, payload, cb)
2 | {

```

And replace them with the following code:

```

1 | , _request: function(action, payload, cb)
2 | {
3 |     cb = cb || function({});

```

3. Find the _request method:

```

1 | , _request: function(action, payload, cb)
2 | {
3 |     var self = this
4 |     , deferred = Q.defer();
5 |     cb = cb || function({});
6 |     var datacenterDomain = self.dataCenterDomains && self.dataCenterDomains.webservices || 'https://webservices.net
suite.com';
7 |     var req = request({
8 |         method: 'POST'
9 |         , uri: datacenterDomain + '/services/NetSuitePort_' + self.nsVersion
10 |         , headers: {
11 |             'User-Agent': 'Node-SOAP/0.0.1'
12 |             , 'Accept': 'text/html,application/xhtml+xml,application/xml,text/xml;q=0.9,*/*;q=0.8'
13 |             , 'Accept-Encoding': 'none'
14 |             , 'Accept-Charset': 'utf-8'
15 |             , 'Connection': 'close'
16 |             , 'Content-Type': 'text/xml; charset=utf-8'
17 |             , 'SOAPAction': '' + action + ''
18 |             , 'Expect': '100-continue'
19 |         }
20 |     }, function(err, response)
21 |     {
22 |         if (err)
23 |         {
24 |             return cb(err);

```

```

25     }
26
27     self.log('Response text for action: ' + action + '\n' + response.body);
28     xml2js.parseString(response.body, self._xml2jsOptions, function(err, result)
29     {
30         // console.log('json response: ', JSON.stringify(result.Envelope.Body, null,2))
31         if (err)
32         {
33             deferred.reject(err);
34             return cb(err);
35         }
36         var soap_body = result.Envelope.Body[0];
37         if (soap_body && soap_body.Fault)
38         {
39             deferred.reject(new Error(soap_body.Fault[0].faultstring[0]));
40             return cb(new Error(soap_body.Fault[0].faultstring[0]));
41         }
42         deferred.resolve(soap_body);
43         cb(null, soap_body, result);
44     });
45 });
46
47 self.log('Request text for action: ' + action + '\n' + payload);
48 // console.log(payload);
49 // payload = prettyData.xmlmin(payload); //minify xml
50 req.end(payload);
51 return deferred.promise;
52 }

```

And replace it with the following code:

```

1  , __request: function(action, payload, cb) {
2      const self = this;
3
4      cb = cb || function() {};
5      const datacenterDomain =
6          (self.dataCenterDomains && self.dataCenterDomains.webservices) ||
7          this.getDefaultWebServiceUrl();
8
9      const args = require('yargs').argv;
10     if (args.proxy) {
11         request = request.defaults({ proxy: args.proxy });
12     }
13     return payload.then(function(payloadResult) {
14         return new Promise(function(resolve, reject) {
15             const req = request(
16                 {
17                     method: 'POST',
18                     uri: datacenterDomain + "/services/NetSuitePort_" + self.nsVersion,
19                     headers: {
20                         'User-Agent': self.credentials.user_agent || 'Node-SOAP/0.0.1',
21                         Accept:
22                             'text/html,application/xhtml+xml,application/xml;text/xml;q=0.9,*/*;q=0.8',
23                         'Accept-Encoding': 'none',
24                         'Accept-Charset': 'utf-8',
25                         Connection: 'close',
26                         'Content-Type': 'text/xml; charset=utf-8',
27                         SOAPAction: '' + action + ''
28                     }
29                 },
30                 function(err, response) {
31                     if (err) {
32                         return cb(err);
33                     }
34
35                     xml2js.parseString(response.body, self._xml2jsOptions, function(
36                         err,
37                         result
38                     ) {
39                         if (err) {
40                             reject(err);
41                             return cb(err);

```

```

42     }
43     const soap_body = result.Envelope.Body[0];
44     if (soap_body && soap_body.Fault) {
45         reject(new Error(soap_body.Fault[0].faultstring[0]));
46         return cb(new Error(soap_body.Fault[0].faultstring[0]));
47     }
48     resolve(soap_body);
49
50     cb(null, soap_body, result);
51 });
52 }
53 );
54
55 req.end(payloadResult);
56 });
57 });
58 }

```

4. Add the `getDefaultWebServiceUrl` method directly above the request method in the `communications.js` file as shown in the following example:

```

1      , getDefaultWebServiceUrl: function()
2      {
3          if (this.credentials.molecule)
4          {
5              return 'https://webservices.' + this.credentials.molecule + '.netsuite.com'
6          }
7          if(this.credentials.vm)
8          {
9              return this.credentials.vm;
10         }
11         else
12         {
13             return 'https://webservices.netsuite.com'
14         }
15     }
16
17     //@@method __request - has the same signature as _request but it won't perform the data center domain verification.
18     , __request: function(action, payload, cb) {

```

Modify the passport.tpl File

1. In the SuiteCommerce Advanced source directory, open this file: `...gulp/ns-deploy/suitetalk4node/src/templates/_passport.tpl`
2. Find the following lines of code:

```

1 <ns:messages:passport>
2   {{#if credentials.email}}<nscore:email>{{credentials.email}}</nscore:email>{{/if}}
3   {{#if credentials.password}}<nscore:password>{{credentials.password}}</nscore:password>{{/if}}
4   {{#if credentials.account}}<nscore:account>{{credentials.account}}</nscore:account>{{/if}}
5   {{#if credentials.roleId}}<nscore:role internalId={{credentials.roleId}}>{{/if}}
6 </ns:messages:passport>

```

And replace them with the following code:

```

1 <ns:tokenPassport actor="http://schemas.xmlsoap.org/soap/actor/next" mustUnderstand="0" xmlns:ns="urn:mes
sages_2020_1.platform.webservices.netsuite.com">
2   {{#if credentials.account}}<ns:account>{{credentials.account}}</ns:account>{{/if}}
3   {{#if credentials.consumerKey}}<ns:consumerKey>{{credentials.consumerKey}}</ns:consumerKey>{{/if}}
4   {{#if credentials.token}}<ns:token>{{credentials.token}}</ns:token>{{/if}}
5   {{#if credentials.nonce}}<ns:nonce>{{credentials.nonce}}</ns:nonce>{{/if}}
6   {{#if credentials.timestamp}}<ns:timestamp>{{credentials.timestamp}}</ns:timestamp>{{/if}}
7   {{#if credentials.signature}}<ns:signature algorithm="HMAC_SHA256">{{credentials.signature}}</ns:signature>{{/if}}
8 </ns:tokenPassport>

```

Modify package.json Dependencies

1. In the top level of the SuiteCommerce Advanced source directory, open the package.json file.
2. Add the following line to the dependencies:

```
1 | "oauth1": "file:../ns_npm_repository/oauth1",
```

As shown in the following example excerpt:

```
1 | "dependencies": {
2 |   "oauth1": "file:../ns_npm_repository/oauth1",
3 |   "ansi-colors": "2.0.1",
4 |   "archiver": "2.1.1",
5 |   "async": "2.6.1",
```

Modify the fs.js File

1. In the SuiteCommerce Advanced source directory, open this file: .../gulp/ns-deploy/fs.js
2. Find the following lines of code:

```
1 | , processBackup: function(deploy, took, cb)
2 | {
3 |   guttil.log('Finished', guttil.colors.cyan('Deploy website' + (took ? ', took ' + took : '')) );
```

And replace them with the following code:

```
1 | , processBackup: function(deploy, cb)
2 | {
3 |   guttil.log('Finished', guttil.colors.cyan('Deploy website') );
```

3. Find the following line of code:

```
1 | cb(null, deploy, context, took);
```

And replace it with the following code:

```
1 | cb(null, deploy, context);
```

Modify the filecabinet-util.js File

1. In the SuiteCommerce Advanced source directory, open this file: .../gulp/ns-deploy/ns-uploader/src/filecabinet-util.js
2. Add the getFolderNamed method directly above the addOrUplodateFile method as shown in the following example:

```
1 | , getFolderNamed: function(parentFolderId, fileName, doGet)
2 | {
3 |   return this.getFileNamed(parentFolderId, fileName, doGet, true);
4 | }
5 |
6 | , addOrUplodateFile: function(parentFolderId, fileName, fileContents, fileType)
```

Create the ns_npm_repository/oauth1 Directory

1. In the top level of the SuiteCommerce Advanced source directory, create a new directory named **ns_npm_repository**.

2. Copy the **oauth1** folder from the [TBAPatch-denali.zip](#) file to the new `ns_npm_repository` directory you created in the preceding step.

Replace Gulp Files

Replace the following files in the SuiteCommerce Advanced source directory with the modified versions of the same files available in the .zip file for this patch: [TBAPatch-denali.zip](#)

- `.../gulp/ns-deploy/net.js`
- `.../gulp/ns-deploy/ui.js`

Run NPM Command for oauth1

After updating all files as required, adding new files from the .zip file for the patch, and downloading and setting up the theme and extension developer tools, you must run the following command to ensure that token-based authentication functions properly.

Run the following command at the top level of the SuiteCommerce Advanced source directory:

```
npm i oauth1
```

When this command completes, continue with additional updates for token-based authentication. For detailed instructions, see the help topic [Token-Based Authentication Updates for Developer Tools](#).

Site Pages Not Indexed

i Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus | Kilimanjaro | Aconcagua | 2018.2 | 2019.1

In some implementations of SuiteCommerce Advanced (SCA) if prerender of site pages fails, those pages may not be indexed. This issue occurs because the `noindex,nofollow` meta tag is not removed from the header in the `shopping.ssp` file as expected.

This issue has been fixed in the 2019.2 and later releases of SuiteCommerce Advanced. If you experience this issue and cannot migrate to the latest release, you can fix this issue as described in the following procedure.

To implement this patch, create a custom module to override the `shopping.ssp` file, which is part of the **ShoppingApplication** module. The location of this file depends on the SCA release that you need to patch:

2019.1	.../Modules/suitecommerce/ShoppingApplication@sc-2019.1.0/SuiteScript/shopping.ssp
2018.2	.../Modules/suitecommerce/ShoppingApplication@sc-2018.2.0/SuiteScript/shopping.ssp
Aconcagua	.../Modules/suitecommerce/ShoppingApplication@aconcaguaR2/SuiteScript/shopping.ssp
Kilimanjaro	.../Modules/suitecommerce/ShoppingApplication@4.1.0/SuiteScript/shopping.ssp
Elbrus	.../Modules/suitecommerce/ShoppingApplication@4.0.0/SuiteScript/shopping.ssp
Vinson	.../Modules/suitecommerce/ShoppingApplication@3.0.0/SuiteScript/shopping.ssp
Montblanc	.../Modules/suitecommerce/ShoppingApplication@2.1.0/SuiteScript/shopping.ssp
Denali	.../Modules/suitecommerce/ShoppingApplication@2.0.8/SuiteScript/shopping.ssp

If you are not familiar with implementing patches for SCA, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).
- For step by step instructions, refer to [Patch Using Override Mode](#).

Step 1: Create the Override File

Following the instructions and recommendations in the [Patch Using Override Mode](#) procedure, copy and paste the following code samples in the new directory and file you create: `.../Modules/extensions/ShoppingApplicationExtension@1.00/SuiteScript/shopping.ssp`

1. Find the following lines of code in the `<head>` tag:

```
1 <meta charset="utf-8">
2 <meta http-equiv="X-UA-Compatible" content="IE=edge">
3 <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no" />
4 <meta name="robots" content="nofollow,noindex">
5 <link rel="shortcut icon" href="%= session.getAbsoluteUrl("shopping", "img/favicon.ico") %=" />
6 <title>Shopping</title>
```

2. And replace them with the following code:

```
1 <meta charset="utf-8">
2 <meta http-equiv="X-UA-Compatible" content="IE=edge">
3 <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no" />
4 <link rel="shortcut icon" href="%= session.getAbsoluteUrl("shopping", "img/favicon.ico") %=" />
5 <title></title>
```

3. Find and remove the following if block:

```
1 if (SC.isPageGenerator())
2 {
3     var metas = document.head.getElementsByTagName('meta');
4
5     for (var i = 0; i < metas.length; i++)
6     {
7         if (metas[i].getAttribute('content') === 'nofollow,noindex')
8         {
9             document.head.removeChild(metas[i]);
10        }
11    }
12 }
```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, you should:

1. Paste the following sample code into the new `ns.package.json` file that you create in the Modules directory: `.../Modules/extensions/ ShoppingApplicationExtension@1.00/ns.package.json`

```
1 {
2     "gulp": {
3         "ssp-files": [
4             "SuiteScript/*.ssp"
5         ]
6     },
7     "overrides": {
8         "suitecommerce/ShoppingApplication@X.Y.Z/SuiteScript/shopping.ssp" : "SuiteScript/shopping.ssp"
9     }
10 }
```



Important: In the preceding code sample, you must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

2. Open the `distro.json` file and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. This file is located in the top-level directory. For example, for the 2018.2 release, if you accepted the default directory name for your release, the top-level directory would be SuiteCommerce Advanced 2018.2.

The following sample shows the value to add to the list of existing values that follow the "modules" key.

```
1 | "modules": {
2 |   "extensions/ShoppingApplicationExtension": "1.0.0",
3 |   . . .
```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

1. Test your patch by deploying to NetSuite (see the help topic [Deploy SCA Customizations to NetSuite](#)). Because this patch modifies a SuiteScript file, changes are not visible in your local environment. SuiteScript files run on the server and must be deployed to NetSuite to take effect.
2. Confirm your results.

Deploy to NetSuite Fails with Errors

Applies to: SuiteCommerce Advanced | 2019.2 | 2020.1

Developers working with the 2019.2 and 2020.1 releases of SuiteCommerce Advanced may not be able to successfully deploy to NetSuite and may receive errors when trying to deploy.

This issue has been fixed in the 2020.2 release of SuiteCommerce Advanced. If you experience this issue and cannot migrate to the latest release, you can fix this issue as described in the following steps.

1. Open the following file in the `gulp` directory for the release your are working with:

Release	File Location
2019.2	.../SC_19.2_Live/gulp/tasks/javascript.js
2020.1	.../SC_20.1/gulp/tasks/javascript.js

2. Find both instances where the `handleOverrides` method is called and add 'javascript' as shown in the following example:

```
1 | .pipe(package_manager.handleOverrides('javascript'))
```



Important: Be sure to change both instances where the `handleOverrides` method is called.

3. Open the following file in the `gulp` directory for the release your are working with:

2019.2	.../SC_19.2_Live/gulp/package-manager/index.js
--------	--

2020.1	.../SC_20.1/gulp/package-manager/index.js
--------	---

- Find the `handleOverrides` method definition and add a resource parameter as shown in the following example:

```
1 | handleOverrides: function(resource) {
```

- In the `handleOverrides` method, find the following code:

```
1 |         if (override_info.isOverriden) {
2 |             // if original file is overridden replace its stat and content.
3 |             // stat needs to change for other
4 |             // gulp-plugins to work properly (i.e. changed)
5 |
6 |             override_info.file = file;
7 |             override_info.file.stat = fs.statSync(
8 |                 override_info.overridePath.replace(/\.js$/, '.ts')
9 |             );
10 |             return;
11 |         }
```

- And replace it with the following code:

```
1 |         if (override_info.isOverriden) {
2 |             // if original file is overridden replace its stat and content.
3 |             // stat needs to change for other
4 |             // gulp-plugins to work properly (i.e. changed)
5 |             override_info.file = file;
6 |             const { overridePath } = override_info;
7 |             if (resource === 'javascript') {
8 |                 override_info.file.stat = fs.statSync(overridePath.replace(/\.js$/, '.ts'));
9 |             } else {
10 |                 override_info.file.stat = fs.statSync(overridePath);
11 |             }
12 |             return;
13 |         }
```

- After editing and saving the files described in the preceding steps, run the `gulp deploy` command as described in [Deploy SCA Customizations to NetSuite](#) to make sure deployments are working as expected.

Values Not Stored in Custom Transaction Body Fields

In some Elbrus implementations of SuiteCommerce Advanced (SCA), text entered into custom transaction body fields during the checkout flow do not display in the Custom subtab of the resulting sales order as expected. These patch instructions describe how to correct this problem.

To implement this patch, create a custom module to override the `CustomFields.js` file, which is part of the CustomFields module. If you are not familiar with implementing patches for SCA, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).
- For step by step instructions, refer to [Patch Using Override Mode](#).

Step 1: Create the Override File

Following the instructions and recommendations in the [Patch Using Override Mode](#) procedure, create a new directory and file: `/Modules/extensions/CustomFieldsExtension@1.0.0/JavaScript/CustomFields.js`.

In the new `CustomFields.js` file, find and replace the following lines with the provided code sample.

Find these lines:

```

1 WizardStep.prototype.afterModuleInstanceCreated.install({
2     name: 'custom-fields-wizard-step-module-instance-created'
3     ,
4     execute: function(moduleInstance)
5     {
6
7         //Add the model of the wizard to the context of all the modules to support transaction body fields
8         var originalGetContextFunction = moduleInstance.getContext;
9         moduleInstance.getContext = function()
10        {
11            var context = originalGetContextFunction.apply(moduleInstance, arguments);
12            if(!context.model)
13            {
14                context.model = moduleInstance.model;
15            }
16            return context;
17        };
18        moduleInstance.events = moduleInstance.events || {};
19        _.each(moduleInstance.model.get('options'), function(optionValue, optionKey)
20        {
21            moduleInstance.events['change [name="' + optionKey + '"'] = updateOption;
22        });
23    });

```

And replace them with the following code:

```

1 WizardStep.prototype.afterModuleInstanceCreated.install({
2     name: 'custom-fields-wizard-step-module-instance-created'
3     ,
4     execute: function(moduleInstance)
5     {
6
7         //Add the model of the wizard to the context of all the modules to support transaction body fields
8         var originalGetContextFunction = moduleInstance.getContext;
9         moduleInstance.getContext = function()
10        {
11            var context = originalGetContextFunction.apply(moduleInstance, arguments);
12            if(!context.model)
13            {
14                context.model = moduleInstance.model;
15            }
16            moduleInstance.events = moduleInstance.events || {};
17            _.each(context.model.get('options'), function(optionValue, optionKey)
18            {
19                moduleInstance.events['change [name="' + optionKey + '"'] = updateOption;
20            });
21            return context;
22        };
23    });

```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, you should:

1. Paste the following sample code into the new ns.package.json file that you create in the Modules directory: /Modules/extensions/CustomFieldsExtension@1.0.0/ns.package.json

```

1 {
2     "gulp": {
3         "javascript": [
4             "JavaScript/*.js"
5         ]
6     },
7     "overrides": {
8         "suitecommerce/CustomFields@X.Y.Z/JavaScript/CustomFields.js" : "JavaScript/CustomFields.js"
9     }

```

```
10 | }
```



Important: In the preceding code sample, you must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

2. Open the `distro.json` file and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the “modules” key.

```
1 | "modules": {
2 |   "extensions/CustomFieldsExtension": "1.0.0",
3 |   . . .
```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Patch Overrides Not Visible in DeployDistribution Folder

Applies to: SuiteCommerce Advanced | 2018.2

Developers working with the 2018.2 release of SuiteCommerce Advanced may not be able to confirm updates to local files after successfully deploying a patch that includes an override. If you deploy a patch that overrides a file in the `SspLibraries@sc-2018.2.0` module, after a successful patch deployment you should see updates from the patch in the `ssp_libraries.js` file in the `DeployDistribution` folder. However, due to this issue the updates are not visible despite successful deployment.

This issue has been fixed in the 2019.1 and subsequent releases of SuiteCommerce Advanced. If you experience this issue and cannot migrate to the latest release, you can fix this issue as described in the following steps.

1. Open the following file: `.../gulp/tasks/ssp-libraries.js`
2. Find the `generateEntryPointContent` method as shown in the following example:

```
1 | function generateEntryPointContent(config)
```

3. Find the following code within the `generateEntryPointContent` method:

```
1 | if (isSCIS && moduleName === 'Configuration')
2 | {
3 |   if (relativePath.includes('PosApplication'))
4 |   {
5 |     config.amdConfig.paths[inv_path[moduleName] || moduleName] = relativePath;
6 |   }
7 | }
8 | else
9 | {
10 |   config.amdConfig.paths[inv_path[moduleName] || moduleName] = relativePath;
11 | }
```

4. After the ending bracket of the `else` condition, add this line:

```
1 | config.amdConfig.rawText[relativePath] = file.contents.toString();
```

After you add the preceding line, the code should look like the following example:

```

1 | if (isSCIS && moduleName === 'Configuration')
2 |     {
3 |         if (relativePath.includes('PosApplication'))
4 |         {
5 |             config.amdConfig.paths[inv_path[moduleName] || moduleName] = relativePath;
6 |         }
7 |     }
8 |     else
9 |     {
10 |         config.amdConfig.paths[inv_path[moduleName] || moduleName] = relativePath;
11 |     }
12 |     config.amdConfig.rawText[relativePath] = file.contents.toString();

```

5. After editing and saving the file as described in the preceding steps, run the `gulp deploy` command as described in [Deploy SCA Customizations to NetSuite](#) to verify that the updates from your original patch are now visible in the `ssp_libraries.js` file in the `DeployDistribution` folder.

Shopping User Environment Experiences Degraded Performance

i Applies to: SuiteCommerce Advanced | Aconcagua | 2018.2 | 2019.1 | 2019.2

In some implementations of SuiteCommerce Advanced (SCA), `shopping.user.environment.ssp` experiences degraded performance. For example, time to first byte (TTFB) may increase from less than 500 milliseconds to 2 seconds. These patch instructions describe how to correct this problem.

To implement this patch, create a custom module to override the `shopping.ssp` file, which is part of the **ShoppingApplication** module. The location of this file depends on the SCA release that you need to patch:

Aconcagua	SuiteCommerce Advanced Aconcagua/Modules/suitecommerce/ShoppingApplication@aconcaguaR2/SuiteScript/shopping.ssp
2018.2	SuiteCommerce Advanced 2018.2.0/Modules/suitecommerce/ShoppingApplication@sc-2018.2.0/SuiteScript/shopping.ssp
2019.1	SuiteCommerce Advanced 2019.1/Modules/suitecommerce/ShoppingApplication@sc-2019.1.0/SuiteScript/shopping.ssp
2019.2	SuiteCommerce Advanced 2019.2/SC_19.2_Live/Advanced/ShoppingApplication/SuiteScript/shopping.ssp

If you are not familiar with implementing patches for SCA, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).
- For step by step instructions, refer to [Patch Using Override Mode](#).

Step 1: Create the Override File

Following the instructions and recommendations in the [Patch Using Override Mode](#) procedure, copy and paste the following code sample in the new directory and file you create. Where you create the new directory and file depends on the SCA release that you need to patch:

Aconcagua, 2018.2, and 2019.1	<code>.../Modules/extensions/ShoppingApplicationExtension@1.00/SuiteScript/shopping.ssp</code>
-------------------------------	--

2019.2	.../SC_19.2_Live/Advanced/extensions/ShoppingApplicationExtension@1.00/SuiteScript/shopping.ssp
--------	---

- For the Aconcagua release, find this line:

```
1 new_script_tag.src = '<%= session.getAbsoluteUrl2("shopping.user.environment.ssp?lang=" + Language + "&cur=" + Currency + "&X-SC-Touchpoint=shopping") %>&t=' + datetime + "";
```

And replace it with the following code:

```
1 new_script_tag.src = '<%= session.getAbsoluteUrl2("shopping.user.environment.ssp?lang=" + Language + "&cur=" + Currency + "&X-SC-Touchpoint=shopping") %> + "";
```

- For the 2018.2, 2019.1, and 2019.2 releases, find this line:

```
1 loadJSON('<%= session.getAbsoluteUrl2("services/ShoppingUserEnvironment.Service.ss?lang=" + Language + "&cur=" + Currency + "&X-SC-Touchpoint=shopping") %>' + googletagmanager_cookie + '&t=' + datetime + "", 'shoppingUserEnvironment')
```

And replace it with the following code:

```
1 loadJSON('<%= session.getAbsoluteUrl2("services/ShoppingUserEnvironment.Service.ss?lang=" + Language + "&cur=" + Currency + "&X-SC-Touchpoint=shopping") %>' + googletagmanager_cookie + "", 'shoppingUserEnvironment')
```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, you should:

- Paste the code appropriate for the SCA release you are patching into a new ns.package.json file that you create. Where you create the ns.package.json file depends upon the release you are working with.

Aconcagua, 2018.2, and 2019.1	.../Modules/extensions/ShoppingApplicationExtension@1.00/ns.package.json
2019.2	.../SC_19.2_Live/Advanced/extensions/ShoppingApplicationExtension@1.00/ns.package.json

- Use the following code if you are patching the Aconcagua, 2018.2, or 2019.1 release.

```
1 {
2   "gulp": {
3     "ssp-files": [
4       "SuiteScript/*.ssp"
5     ]
6   },
7   "overrides": {
8     "suitecommerce/ShoppingApplication@X.Y.Z/SuiteScript/shopping.ssp" : "SuiteScript/shopping.ssp"
9   }
10 }
```



Important: In the preceding code sample, you must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

- Use the following code if you are patching the 2019.2 release.

```
1 {
2   "gulp": {
```

```

3      "ssp-files": [
4          "SuiteScript/*.ssp"
5      ],
6  },
7  "overrides": {
8      "suitecommerce/ShoppingApplication/SuiteScript/shopping.ssp" : "SuiteScript/shopping.ssp"
9  }
10 }

```

2. Open the `distro.json` file and then add your custom module to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The location of the `distro.json` file depends on the version of SCA you are patching.

Aconcagua, 2018.2, and 2019.1	For these releases, the <code>distro.json</code> file resides in the top-level directory. If you accepted the default directory name for your release, these would be SuiteCommerce Advanced Aconcagua, SuiteCommerce Advanced 2018.2, and SuiteCommerce Advanced 2019.1.
2019.2	For this release, the <code>distro.json</code> file resides in the <code>SC_19.2_Live/Advanced</code> directory. If you accepted the default name for the top-level directory, the complete path is: SuiteCommerce Advanced 2019.2/SC_19.2_Live/Advanced/ <code>distro.json</code> .

The following sample shows the value to add to the list of existing values that follow the “`modules`” key. Refer to the appropriate sample for the version of SCA you are working with.

- Use the following sample if you are patching the Aconcagua, 2018.2, or 2019.1 release.

```

1  "modules": {
2      "extensions/ShoppingApplicationExtension": "1.0.0",
3      . . .

```

- Use the following sample if you are patching the 2019.2 release.

```

1  "modules": {
2      "extensions/ShoppingApplicationExtension",
3      . . .

```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

beforeShowContent Event Stops Working After Initial Trigger

i Applies to: SuiteCommerce Advanced | 2019.1 | 2019.2

In some implementations of SuiteCommerce Advanced (SCA), the `beforeShowContentEvent` stops working after the initial trigger in these components: `ProductListPageComponent` and `ProductDetailsComponent`. These patch instructions describe how to correct this problem.

To implement this patch, create a custom module to override the `NavigationHelper.Plugins.Modals` file, which is part of the **NavigationHelper** module. The location of this file depends on the SCA release that you need to patch:

2019.1	SuiteCommerce Advanced 2019.1/Modules/suitecommerce/NavigationHelper@sc-2019.1.0/JavaScript/NavigationHelper.Plugins.Modals.js
--------	--

2019.2	SuiteCommerce Advanced 2019.2/SC_19.2_Live/Commons/NavigationHelper/JavaScript/NavigationHelper.Plugins.Modals.ts
--------	---

If you are not familiar with implementing patches for SCA, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).
- For step by step instructions, refer to [Patch Using Override Mode](#).

Step 1: Create the Override File

Following the instructions and in the [Patch Using Override Mode](#) procedure, copy and paste the following code sample in the new directory and file you create. Where you create the new directory and file depends on the SCA release that you need to patch:

2019.1	.../Modules/extensions/NavigationHelperExtension@1.0.0/JavaScript/NavigationHelper.Plugins.Modals.js
2019.2	.../SC_19.2_Live/Commons/extensions/NavigationHelperExtension@1.0.0/JavaScript/NavigationHelper.Plugins.Modals.ts

Find this line:

```
1 | layout.originalShowContent = layout._showContent;
```

And replace it with the following code:

```
1 | layout.originalShowContent = layout.showContent
```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, you should:

1. Paste the code appropriate for the SCA release you are patching into a new ns.package.json file that you create. Where you create the ns.package.json file depends upon the release you are working with.

2019.1	.../Modules/extensions/NavigationHelperExtension@1.0.0/ns.package.json
2019.2	.../SC_19.2_Live/Commons/extensions/NavigationHelperExtension@1.0.0/ns.package.json

- Use the following code if you are patching the 2019.1 release.

```
1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/NavigationHelper@SC-2019.1.0/JavaScript/NavigationHelper.Plugins.Modals.js" : "JavaScript/NavigationHelper.Plugins.Modals.js"
9 |   }
10 | }
```

- Use the following code if you are patching the 2019.2 release.

```
1 | {
```

```

2  "gulp": {
3    "javascript": [
4      "JavaScript/*.js"
5    ]
6  },
7  "overrides": {
8    "suitecommerce/NavigationHelper/JavaScript/NavigationHelper.Plugins.Modals.ts" : "JavaScript/Navigation
9    Helper.Plugins.Modals.ts"
10 }

```

2. Open the `distro.json` file and then add your extension to the `modules` object as described in the [Patch Using Override Mode](#) procedure. The location of the `distro.json` file depends on the version of SCA you are patching.

2019.1	For this release, the <code>distro.json</code> file resides in the top-level directory. If you accepted the default directory name for the top-level directory, the complete path is: SuiteCommerce Advanced 2019.1/ <code>distro.json</code> .
2019.2	For this release, the <code>distro.json</code> file resides in the <code>SC_19.2_Live/Advanced</code> directory. If you accepted the default name for the top-level directory, the complete path is: SuiteCommerce Advanced 2019.2/ <code>SC_19.2_Live/Advanced/distro.json</code> .

The following sample shows the value to add to the list of existing values that follow the “`modules`” key. Refer to the appropriate sample for the version of SCA you are working with.

- Use the following sample if you are patching the 2019.1 release.

```

1  "modules": {
2    "extensions/NavigationHelperExtension": "1.0.0",
3    . . .

```

- Use the following sample if you are patching the 2019.2 release.

```

1  "modules": {
2    "../Commons/extensions/NavigationHelperExtension",
3    . . .

```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Filter Site Option Does Not Work As Expected

Applies to: SuiteCommerce Advanced | Vinson

In some implementations of the Vinson release of SuiteCommerce Advanced (SCA), site filtering does not work as expected. For example, in NetSuite if you go to Commerce > Websites > Configuration, select the Advanced Tab, and then on the Filter Site Subtab you set the Filter Site option to current, you should only see orders for the current web store when viewing the Purchase History in the My Account home page. But, orders for other web stores from the same account are also listed in the Purchase History. These patch instructions describe how to correct this problem.

To implement this patch, create a custom module to override the SuiteScript `Transaction.Model.js` file, which is part of the **Transaction** module.

If you are not familiar with implementing patches for SuiteCommerce Advanced, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).
- For step by step instructions, refer to [Patch Using Override Mode](#).

Step 1: Create the Override File

Following the instructions and recommendations in the [Patch Using Override Mode](#) procedure, create a new directory and file: /Modules/extensions/TransactionExtension@1.0.0/SuiteScript/Transaction.Model.js. In the new Transaction.Model.js file, find and replace the following lines with the provided code sample.

Find these lines:

```

1  if (this.isMultiSite)
2      {
3      var site_id = ModelsInit.session.getSiteSettings(['siteid']).siteid
4      , filter_site = SC.Configuration.filter_site
5      , search_filter_array = null;
6
7      if (_.isString(filter_site) && filter_site === 'current')
8      {
9          search_filter_array = [site_id, '@NONE@'];
10     }
11     else if (_.isString(filter_site) && filter_site === 'all')
12     {
13         search_filter_array = [];
14     }
15     else if (_.isArray(filter_site))
16     {
17         search_filter_array = filter_site;
18         search_filter_array.push('@NONE@');
19     }
20
21
22     if (search_filter_array && search_filter_array.length)
23     {
24         this.filters.site_operator = 'and';
25         this.filters.site = ['website', 'anyof', _.uniq(search_filter_array)];
26     }
27 }
```

And replace them with the following code:

```

1  if (this.isMultiSite)
2      {
3      var site_id = ModelsInit.session.getSiteSettings(['siteid']).siteid
4      , filter_site = SC.Configuration.filterSite.option
5      , filter_site_id = SC.Configuration.filterSite.ids
6      , search_filter_array = null;
7
8
9
10     if (_.isString(filter_site) && filter_site === 'current')
11     {
12
13         search_filter_array = [site_id, '@NONE@'];
14     }
15     else if (_.isString(filter_site) && filter_site === 'all')
16     {
17
18         search_filter_array = [];
19     }
20
21     else if (_.isArray(filter_site_id) && filter_site === 'siteIds')
22     {
23
24         search_filter_array = filter_site_id;
25         search_filter_array.push('@NONE@');
```

```

26         }
27
28
29         if (search_filter_array && search_filter_array.length)
30         {
31             this.filters.site_operator = 'and';
32             this.filters.site = ['website', 'anyof', _.uniq(search_filter_array)];
33         }
34     }

```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Override Mode](#) procedure, you should:

1. Paste the following sample code into the new ns.package.json file that you create in the Modules directory: /Modules/extensions/TransactionExtension@1.0.0/ns.package.json

```

1  {
2      "gulp": {
3          "ssp-libraries": [
4              "SuiteScript/*.js"
5          ]
6      },
7      "overrides": {
8          "suitecommerce/Transaction@1.2.0/SuiteScript/Transaction.Model.js" : "SuiteScript/Transaction.Model.js"
9      }
10 }

```

2. Open the distro.json file in the root SuiteCommerce Advanced development directory and then add your custom module to the modules object as described in the [Patch Using Override Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the "modules" key.

```

1  "modules": {
2      "extensions/TransactionExtension": "1.0.0",
3      . . .

```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Override Mode](#) procedure to test and deploy your patch.

Quantity Facet Displays Dollar Sign

Applies to: SuiteCommerce Advanced | Elbrus | Kilimanjaro | Vinson

In some implementations of the Elbrus, Kilimanjaro, and Vinson releases of SuiteCommerce Advanced, adjusting the filter for a quantity facet on your site displays a numerical value that includes a dollar sign.

Using Extend mode, the following patch corrects this issue in Facets.FacetsDisplay.View.js, which is part of the **Facets** module. To implement this patch, create a custom module to extend the prototype object for the `getContext()` method within Facets.FacetsDisplay.View.js.

If you are not familiar with implementing patches for SuiteCommerce Advanced, refer to the following:

- Before you get started, familiarize yourself with [Best Practices for Customizing SCA](#).

- For step by step instructions, refer to [Patch Using Extend Mode](#).

Step 1: Develop An Extension

Use the following code sample to extend the prototype of the `getContext()` method within `Facets.FacetsDisplay.View.js`.

Following the instructions and recommendations in the [Patch Using Extend Mode](#) procedure, copy and paste the following code sample in the new directory and file you create: `Modules/extensions/FacetsDisplayViewExtension@1.0.0/JavaScript/Facets.FacetsDisplay.View.Extend.js`

```

1  define(
2    'Facets.FacetsDisplay.View.Extend'
3  , [
4    'Facets.FacetsDisplay.View'
5    , 'Backbone.CollectionView'
6
7    , 'Backbone'
8    , 'underscore'
9
10   ]
11  , function(
12    FacetsFacetsDisplayViewExtend
13    , BackboneCollectionView
14
15    , Backbone
16    , _
17  )
18  {
19    'use strict';
20    _.extend(FacetsFacetsDisplayView.prototype,
21      {
22        getContext: function()
23        {
24          var facets = this.options.facets
25          , translator = this.options.translator;
26
27          _.each(facets, function(facet) {
28            facet.value = _.isArray(facet.value) ? facet.value : [facet.value];
29          });
30
31          var facet_values = [];
32
33          _.each(facets, function(facet)
34          {
35            var parser = facet.config.parser;
36
37            _.each(facet.value, function(value)
38            {
39              var from = _.isObject(value) ? value.from : '';
40              var to = _.isObject(value) ? value.to : '';
41
42              var value_data = {
43                facetValueIsObject: _.isObject(value)
44                , from: from && parser ? parser(from) : from
45                , to: to && parser ? parser(to) : to
46                , valueLabel: translator.getLabelForValue(facet.id, value)
47                , facetValueUrl: translator.cloneForFacetId(facet.id, value).getUrl()
48                , facetValue: facet.value
49              };
50
51              facet_values.push(value_data);
52            });
53          });
54
55          // @class Facets.FacetsDisplay.View.Context
56          return {
57
58            // @property {Boolean} hasFacets

```

```

59         hasFacets: facets.length > 0
60
61         // @property {String} clearAllFacetsLink
62         , clearAllFacetsLink: translator.cloneWithoutFacets().getUrl()
63
64         // @property {Array} values
65         , values: facet_values
66     };
67     // @class Facets.FacetsDisplay.View
68 }
69 });
70 });

```

Step 2: Prepare the Developer Tools For Your Patch

When preparing the Developer Tools for your patch as described in the [Patch Using Extend Mode](#) procedure, you should:

1. Paste the following sample code into the new `ns.package.json` file that you create in the Modules directory: `Modules/extensions/FacetsDisplayViewExtension@1.0.0/ns.package.json`.

```

1  {
2      "gulp": {
3          "javascript": [
4              "JavaScript/*.js"
5          ]
6      }
7  }

```

2. Open the `distro.json` file in the root SuiteCommerce Advanced development directory and then add your extension to the `modules` object as described in the [Patch Using Extend Mode](#) procedure. The following sample shows the value to add to the list of existing values that follow the "modules" key.

```

1  "modules": {
2      "extensions/FacetsDisplayViewExtension": "1.0.0",
3      . . .

```

3. Add `Facets.FacetsDisplay.View.Extend` as a dependency to the entry point within the `SC.Shopping.Starter` array.

Your `distro.js` file should look similar to:

```

1  "tasksConfig": {
2      //...
3      "javascript": [
4          //...
5          {
6              "entryPoint": "SC.Shopping.Starter",
7              "exportFile": "shopping.js",
8              "dependencies": [
9                  //...
10                 "ProductDetailToQuote",
11                 "StoreLocator",
12                 "Facets.FacetsDisplay.View.Extend"
13             ],
14             //...

```

Step 3: Test and Deploy Your Patch

Follow the instructions provided in the [Patch Using Extend Mode](#) procedure to test and deploy your patch.

Custom Overrides Not Applying When Deployed

Applies to: SuiteCommerce Advanced | 2018.2

Developers working with the 2018.2 release of SuiteCommerce Advanced may notice that overrides to core source files implemented via the SCA developer tools are not working properly. For more information about working with SCA developer tools, see [Set Up SCA Developer Tools](#).

For example, if you try to override a template as described in [Override a Template File](#) and then run `gulp ssp-libraries` to compile and view the modifications, the overrides are not implemented as expected.

This issue has been fixed in the 2019.1 release of SuiteCommerce Advanced. If you experience this issue and cannot migrate to the latest release, you can fix this issue as described in the following procedure.

Edit the `ssp-libraries.js` File

1. Locate the `gulp/tasks/ssp-libraries.js` file in the top-level directory of your SCA developer tools installation.
2. Add the following line of code to the `ssp-libraries.js` file:

```
1 | config.amdConfig.rawText[relativePath] = file.contents.toString();
```

The following example shows where to place the new line:

```
1 | if (isSCIS && moduleName === 'Configuration')
2 | {
3 |     if (relativePath.includes('PosApplication'))
4 |     {
5 |         config.amdConfig.paths[inv_path[moduleName] || moduleName] = relativePath;
6 |     }
7 | }
8 | else
9 | {
10 |     config.amdConfig.paths[inv_path[moduleName] || moduleName] = relativePath;
11 | }
12 | config.amdConfig.rawText[relativePath] = file.contents.toString();
13 | };
```

3. Run the following command and then view your site to verify the changes in your override are visible:

```
1 | gulp ssp-libraries
```

Theme Developer Tool Overrides May Cause Error On Activation

Applies to: SuiteCommerce | 2018.2

Overrides implemented using the 2018.2 theme developer tools may cause an error on activation.

For example, if you copy a template (.tpl) or Sass (.scss) file from the Modules directory of the published extension to a theme Overrides directory, customize the new file, and then deploy and activate that theme, you may receive this error: `Templates Job Error: Could not find overriding file`. For more information about overriding an active theme using the theme developer tools, refer to [Override Active Extension Files](#).

This issue occurs because the theme developer tools are not adding overrides to your theme's manifest.json file as part of the deployment process as expected. This issue has been fixed in the 2019.1.2 release of SuiteCommerce Advanced. If you experience this issue and cannot migrate to the latest release, you can fix this issue as described in the following procedure.

Step 1: Edit the overrides.js File

1. Locate the gulp/extension-mechanism/overrides.js file in the top-level directory of your theme developer tools installation.
2. Add the following lines of code to the overrides.js file:

```
1 file_path = file_path.replace(extension_manifest.vendor + '/', '');
2 file_path = file_path.replace(extension_manifest.name + '/', '');
```

The following example shows where to place the new lines:

```
1 else if(ext === '.tpl')
2 {
3     var aux = dst_path.split('/')
4     ,   extensions_name = aux.length > 1 && aux[1]
5     ,   file_path = aux.join('/')
6     ,   extension_manifest = manifest_manager.getManifestByName(extensions_name);
7
8     file_path = file_path.replace(extension_manifest.vendor + '/', '');
9     file_path = file_path.replace(extension_manifest.name + '/', '');
10
11     _.each(extension_manifest.templates.application, function(app, app_name)
12     {
13         var files = theme_manifest.templates.application[app_name].files;
14
15         if(_.contains(app.files, file_path) && !_.contains(files, src_path))
16         {
17             files.push(src_path);
18         }
19     });
20
21 }
```

Step 2: Deploy and Activate Your Theme

1. Deploy the theme to your NetSuite account. Refer to [Deploy a Theme to NetSuite](#) for details.
2. Activate the theme as described in [Activating Themes and Extensions](#).

After following the preceding steps, the theme development tools will automatically add overrides to your theme's manifest.json file so that activations are successful.

Categories Navigate Adapter Event Removed in SMT

Applies to: SuiteCommerce Advanced | 2018.2

In the 18.2 release of SuiteCommerce Advanced, the categories:navigate adapter event in SMT was removed. Consequently, SMT admins cannot preview current and future dated categories, as well as the preview of the category structure and the items in the categories. The patch instructions described below correct this problem.

To implement this patch, create a custom module to override CMSadapter.Impl.Categories.v3.js, which is part of the **CMSadapter** module. You can download the code samples described in this procedure here: [CategoriesNavigateAdapterEventRemovedinSMT](#).

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file in a way that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the Override File

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name similar to the module being customized. For example: `Modules/extensions/CMSadapterExtension@1.0.0`.
3. In your new `CMSadapterExtension@1.0.0` directory, create a folder titled `JavaScript`. For example: `Modules/extensions/CMSadapterExtension@1.0.0/JavaScript`
4. Copy the following source file and paste into the correct location:

Copy This File:	Place a Copy Here:
<code>Modules/suitecommerce/CMSadapter@X.Y.Z/JavaScript/</code>	<code>Modules/extensions/CMSadapterExtension@1.0.0/JavaScript</code>

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

5. Open your new copy of `CMSadapter.Impl.Categories.v3.js` and locate the `listenForCMS()` method.
6. Inside the `listenForCMS()` method, add the following event:

```

1 | self.CMS.on('categories:navigate', function (promise, data)
2 | {
3 |     Backbone.localCache = {};
4 |
5 |     Backbone.history.navigate(url, { trigger: false });
6 |     Backbone.history.loadUrl(url);
7 |
8 |     self.reloadCategories({ 'dontRefresh': true }).then(function()
9 |     {
10 |         promise.resolve();
11 |     });
12 | });

```

7. Locate the following header of the `reloadCategories()` method:

```

1 | CMSadapterImplCategories3.prototype.reloadCategories = function () {

```

Change the header of the `reloadCategories()` to the following code:

```

1 | CMSadapterImplCategories3.prototype.reloadCategories = function (options) {

```

8. Locate the following line in the `reloadCategories()` method:

```

1 | self.refreshPLP();

```

9. Replace this line with the following code:

```

1 | if (!(options && options.dontRefresh))
2 | {
3 |     self.refreshPLP();

```

```
4 | }
```

10. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your Modules/extensions/CMSadapterExtension@1.0.0 module directory.
2. Create a file in this directory titled ns.package.json.
Modules/extensions/CMSadapterExtension@1.0.0/ns.package.json
3. Paste the following code in your new ns.package.json file:



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

```
1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/CMSadapter@X.Y.Z/JavaScript/CMSadapter.Impl.Categories.v3.js" : "JavaScript/CMSadapter.Impl.Categories.v3.js"
9 |   }
10 | }
```

4. Open the distro.json file. This file is located in your root directory.
5. Add your custom module to the modules object.

Your code should look similar to:

```
1 | {
2 |   "name": "SuiteCommerce Advanced 2018.2.0 (sc-2018.2.0)",
3 |   "version": "2.0",
4 |   "isSCA": true,
5 |   "buildToolsVersion": "sc-2018.2.0",
6 |   "folders": {
7 |     "modules": "Modules",
8 |     "suitecommerceModules": "Modules/suitecommerce",
9 |     "extensionsModules": "Modules/extensions",
10 |     "thirdPartyModules": "Modules/third_parties",
11 |     "distribution": "LocalDistribution",
12 |     "deploy": "DeployDistribution"
13 |   },
14 |   "modules": {
15 |     "extensions/CMSadapterExtension": "1.0.0",
16 |     "suitecommerce/Account": "sc-2018.2.0",
17 |     ...
18 |   }
19 | }
```

This step ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

Upon successful deployment, you can go to both current and future dated categories in SMT.

Categories Do Not Display Properly

i Applies to: SuiteCommerce Advanced | Kilimanjaro

In some implementations of the Kilimanjaro release of SuiteCommerce Advanced, the parent of a product category would incorrectly overwrite the data of a child category when it displays the child category. This could result in a Page Not Found (404) or the display of the wrong category.

The fix in this patch sets the data for a category correctly, no matter if the category data came first or the overwrite data from the parent came first.

To implement this patch, create a custom module to override Categories.Model.js, which is part of the **Categories** module. You can download the code samples described in this procedure here:

[CategoriesDoNotDisplayProperlyinSCA.zip](#)

i Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations will continue to work when migrating to a new version of SuiteCommerce Advanced. However, this patch requires you to modify a file in a way that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the Override File

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name similar to the module being customized. For example: Modules/extensions/CategoriesExtension@1.0.0.
3. In your new CategoriesExtension@1.0.0 directory, create a folder titled SuiteScript. For example: Modules/extensions/CategoriesExtension@1.0.0/SuiteScript
4. Copy the following source file and paste into the correct location:

Copy This File:	Place a Copy Here:
Modules/suitecommerce/Categories@X.Y.Z/SuiteScript/Categories.Model.js	Modules/extensions/Categories@X.Y.Z/SuiteScript

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

5. Open your new copy of Categories.Model.js and locate the following lines:

```

1 | if (cat)
2 | {
3 |   if (cat.isprimaryurl === 'T')
4 |   {
5 |     cat.idpath = category.idpath;
6 |     cat.canonical = cat.fullurl;
7 |   }
8 |   else
9 |   {
10 |    cat.canonical = category.fullurl;
11 |   }
12 | }
13 | else

```

```

14 | {
15 |     categories[line.getValue('internalid')] = category;
16 | }

```

6. Replace these lines with the following code:

```

1 | if (cat)
2 | {
3 |     if (category.isprimaryurl === 'T')
4 |     {
5 |         cat.canonical = category.fullurl;
6 |     }
7 |
8 |     if (categoryIds && categoryIds.fullurl === category.fullurl)
9 |     {
10 |         cat.idpath = category.idpath;
11 |     }
12 | }
13 | else
14 | {
15 |     category.canonical = category.fullurl;
16 |     categories[line.getValue('internalid')] = category;
17 | }

```

7. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your Modules/extensions/CategoriesExtension@1.0.0 module directory.
2. Create a file in this directory titled ns.package.json.
Modules/extensions/CategoriesExtension@1.0.0/ns.package.json
3. Paste the following code in your new ns.package.json file:

 **Important:** You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

```

1 | {
2 |     "gulp": {
3 |         "javascript": [
4 |             "JavaScript/*.js"
5 |         ]
6 |     },
7 |     "overrides": {
8 |         "suitecommerce/Categories@X.Y.Z/SuiteScript/Categories.Model.js" : "SuiteScript/Categories.Model.js"
9 |     }
10 | }

```

4. Open the distro.json file. This file is located in your root directory.
5. Add your custom module to the modules object.

Your code should look similar to:

```

1 | {
2 |     "name": "SuiteCommerce Advanced Kilimanjaro",
3 |     "version": "2.0",
4 |     "isSCA": true,
5 |     "buildToolsVersion": "1.3.1",
6 |     "folders": {
7 |         "modules": "Modules",
8 |         "suitecommerceModules": "Modules/suitecommerce",
9 |         "extensionsModules": "Modules/extensions",
10 |         "thirdPartyModules": "Modules/third_parties",
11 |         "distribution": "LocalDistribution",
12 |         "deploy": "DeployDistribution"

```

```

13     },
14     "modules": {
15         "extensions/CategoriesExtension": "1.0.0",
16         "extensions/MyExampleCartExtension1": "1.0.0",
17         ...

```

This step ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Before you deploy, run the following command to compile and deploy the ssp-libraries:

```
1 | gulp --source ssp-libraries
```

2. Deploy your source code customizations to your NetSuite account and test the functionality. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.

Note: Since this patch modifies a SuiteScript file, changes are not visible in your local environment. SuiteScript files run on the server and must be deployed to NetSuite to take effect.

3. Confirm your results.

Upon successful deployment, both parent categories and their associated child directories display correctly.

Incorrect Value for Shipping Estimate Occurs in Shopping Cart

Applies to: SuiteCommerce Advanced | Elbrus

In some implementations of the Elbrus release of SuiteCommerce Advanced, a value of \$0.00 appears for the shipping estimate when calculating the actual estimated value for shipping. As a result, this indicates an incorrect value of shipping while SuiteCommerce is calculating the actual shipping value. This error occurs if you use SiteBuilder Extensions to create the site.

The following patch corrects a problem in Cart.Detailed.View.js, which is part of the **Cart** module. To implement this patch, create a custom module to extend the prototype object for the estimateTaxShip() method. You can download the code samples described in this procedure here: [IncorrectValueforEstimateOccursinShoppingCart.zip](#).

Note: Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Extend Cart.Detailed.View.js

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name similar to the module being customized. For example, create Modules/extensions/Cart@1.0.0.
3. In your new Cart@1.0.0 directory, create a subdirectory called JavaScript. For example: Modules/extensions/Cart@1.0.0/JavaScript

4. In your new JavaScript subdirectory, create a JavaScript file to extend Cart.Detailed.View.js. Name this file according to best practices. For example:
Cart.Detailed.View.Extension.js
5. Open this file and extend the estimateTaxShip() method as shown in the following code snippet.

```

1  define('Cart.Detailed.View.Extension'
2  , [ 'Utils'
3    , 'underscore'
4    , 'Backbone'
5    ]
6  , function (
7    , Utils
8    , _
9    , Backbone
10
11    , jQuery
12  )
13  {
14    'use strict';
15
16    _.extend(estimateTaxShip.prototype, function estimateTaxShip(e),
17    {
18
19      // Build this code in a text editor to make sure that the indents are correct.
20      {
21        var options = this.$(e.target).serializeObject()
22        ,   address_internalid = options.zip + '-' + options.country + '-null'
23        ,   self = this;
24
25        e.preventDefault();
26
27        var address = {
28          internalid: address_internalid
29        ,   zip: options.zip
30        ,   country: options.country
31        };
32
33        // Render method should not fire right now, the accordion must close when the promise resolves.
34        this.model.set('shipaddress', address_internalid, {silent: true});
35        this.$('.cart-summary-button-estimate').text(_('Estimating...').translate());
36        this.$('.cart-summary-button-estimate').prop('disabled', true);
37        var promise = this.saveForm(e);
38
39        if (promise)
40        {
41          promise.then(function()
42          {
43            var address_internalid = self.model.get('shipaddress');
44            self.model.set('shipaddress', address_internalid);
45            self.$('.cart-summary-button-estimate').text(_('Estimate').translate());
46            self.$('.cart-summary-button-estimate').prop('disabled', false);
47            self.swapEstimationStatus();
48          });
49        }
50      }
51    });

```

6. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new Cart@1.0.0 module directory.
2. Create a file in this directory and name it ns.package.json.
Modules/extensions/Cart@1.0.0/ns.package.json
3. Paste the following code into your new ns.package.json file:

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5      ]
6    }
7  }

```

4. Open the distro.json file. This is located in your root directory.
5. Add your custom module to the modules object.

Your code should look similar to the following example:

```

1  {
2    "name": "SuiteCommerce Advanced Elbrus",
3    "version": "2.0",
4    "buildToolsVersion": "1.3.0",
5    "folders": {
6      "modules": "Modules",
7      "suitecommerceModules": "Modules/suitecommerce",
8      "extensionsModules": "Modules/extensions",
9      "thirdPartyModules": "Modules/third_parties",
10     "distribution": "LocalDistribution",
11     "deploy": "DeployDistribution"
12   },
13   "modules": {
14     "extensions/Cart.Extension": "1.0.0",
15     "extensions/MyExampleCartExtension1": "1.0.0",
16     ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Add Cart.Extension as a dependency to SCA entry point within the SC.Shopping.Starter endpoint of the JavaScript array.

Your Distro.js file should look similar to the following:

```

1  "tasksConfig": {
2    //...
3    "javascript": [
4      //...
5      {
6        "entryPoint": "SC.Checkout.Starter",
7        "exportFile": "checkout.js",
8        "dependencies": [
9          //...
10         "Cart.Extension",
11         //...
12       ],
13       //...

```

7. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, the estimated shipping does not display a value until the calculation is complete.

Standard Promotion with Inline Discount and Rate as Percentage Not Updating the Amount in Checkout

i Applies to: SuiteCommerce Advanced | Vinson | Mont Blanc | Denali

In some implementations of the Vinson, Mont Blanc, and Denali releases of SuiteCommerce Advanced, the amount in checkout is not updating when applying a standard promotion with an inline discount and the rate as percentage.

The following patch corrects a problem in `ItemViews.Cell.Navigable.View.js`, which is part of the **ItemViews** module. To implement this patch, create a custom module to extend the prototype object for the `getContext()` method. You can download the code samples described in this procedure here: [ItemViews.Cell.Navigable.View.Extension@1.0.0-Vinson.zip](#).

i Note: Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Extend `ItemViews.Cell.Navigable.View.js`

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name similar to the module being customized. For example: `Modules/extensions/ItemViews.Cell.Navigable.View.Extension@1.0.0`.
3. Open your new `ItemViews.Cell.Navigable.View.Extension@1.0.0` module and create a subfolder titled `JavaScript`.
For example: `Modules/extensions/ItemViews.Cell.Navigable.View.Extension@1.0.0/JavaScript`
4. In your new `JavaScript` subdirectory, create a `JavaScript` file to extend `ItemViews.Cell.Navigable.View.js`.
Name this file according to best practices.
For example: `ItemViews.Cell.Navigable.View.Extension.js`
5. Open this file and extend the `getContext()` function as shown in the following code snippet.

```

1  define('ItemViews.Cell.Navigable.View.Extension'
2  , [
3  , 'ItemViews.SelectedOption.View'
4  , 'underscore'
5  ]
6  , function (
7  , ItemViewsSelectedOptionView
8  , _
9  )
10 {
11   'use strict';
12   _._extend(ItemViews.Cell.Navigable.View.prototype,
13   {
14     getContext: function ()
15     {
16       var item = this.model.get('item')
17       , line = this.model;
18
19       //@class ItemViews.Navigable.View.Context
20       return {
21         //@property {String} itemId
22         itemId: item.get('internalid')
23         //@property {String} itemName
24         , itemName: item.get('_name')

```



```

25     // @property {String} cellClassName
26     , cellClassName: this.options.cellClassName
27     // @property {Boolean} isNavigable
28     , isNavigable: !!this.options.navigable && !!item.get('_isPurchasable')
29     // @property {String} rateFormatted
30     , rateFormatted: line.get('rate_formatted')
31     // @property {String} itemSKU
32     , itemSKU: item.get('_sku')
33     // @property {Boolean} showOptions
34     , showOptions: !(line.get('options') && line.get('options').length)
35     // @property {String} itemImageUrl
36     , itemImageUrl: item.get('_thumbnail').url
37     // @property {String} itemImageAltText
38     , itemImageAltText: item.get('_thumbnail').altimagetext
39     // @property {String} itemURLAttributes
40     , itemURLAttributes: item.get('_linkAttributes')
41     // @property {Number} quantity
42     , quantity: line.get('quantity')
43     // @property {Boolean} showDetail2Title
44     , showDetail2Title: !!this.options.detail2Title
45     // @property {String} detail2Title
46     , detail2Title: this.options.detail2Title
47     // @property {String} detail2
48     , detail2: line.get(this.options.detail2)
49     // @property {Boolean} showReason
50     , showBlockDetail2: !!line.get(this.options.detail2)
51     // @property {Boolean} showDetail3Title
52     , showDetail3Title: !!this.options.detail3Title
53     // @property {String} detail3Title
54     , detail3Title: this.options.detail3Title
55     // @property {String} detail3
56     , detail3: line.get('amount') > line.get('total') ? line.get('total_formatted') : line.get(this.options.de
tail3)
57     // @property {Boolean} showComparePrice
58     , showComparePrice: line.get('amount') > line.get('total')
59     // @property {String} comparePriceFormatted
60     , comparePriceFormatted: line.get('amount_formatted')
61   };
62   }
63 }
64 });

```

6. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new ItemViews.Cell.Navigable.View.Extension@1.0.0 module directory.
2. Create a file in this directory titled ns.package.json.
Modules/extensions/ItemViews.Cell.Navigable.View.Extension@1.0.0/ns.package.json
3. Paste the following code into your new ns.package.json file:

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5      ]
6    }
7  }

```

4. Open the distro.json file. This is located in your root directory.
5. Add your custom modules to the modules object.

Your code should look similar to:

```

1  {
2    "name": "SuiteCommerce Advanced Vinson Release",
3    "version": "2.0",

```

```

4  "buildToolsVersion": "1.2.1",
5  "folders": {
6    "modules": "Modules",
7    "suitecommerceModules": "Modules/suitecommerce",
8    "extensionsModules": "Modules/extensions",
9    "thirdPartyModules": "Modules/third_parties",
10   "distribution": "LocalDistribution",
11   "deploy": "DeployDistribution"
12 },
13   "modules": {
14     "extensions/ItemViews.Cell.Navigable.View.Extension": "1.0.0",
15     "suitecommerce/Account": "2.2.0",
16     ...

```

This ensures that the Gulp tasks include your modules when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Add `ItemViews.Cell.Navigable.View.Extension` as a dependency to SCA entry point within the `SC.Shopping.Starter` entrypoint of the JavaScript array.

Your `Distro.js` file should look similar to the following:

```

1  "tasksConfig": {
2    //...
3    "javascript": [
4      //...
5      {
6        "entryPoint": "SC.Checkout.Starter",
7        "exportFile": "checkout.js",
8        "dependencies": [
9          //...
10         "ProductDetailToQuote",
11         "ItemViews.Cell.Navigable.View.Extension"
12       ],
13       //...

```

7. Save the `distro.json` file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, the amount displays on checkout pages when applying standard promotion with inline discount and the rate set as a percentage.

Page With URL Fragments Redirects Too Many Times

Applies to: SuiteCommerce Advanced | Elbrus | Kilimanjaro

In some implementations of the Elbrus and Kilimanjaro releases of SuiteCommerce Advanced, some pages return `ERR_TOO_MANY_REDIRECTS` instead of displaying the Product Details page or a 404 error. This error occurs when the URL contains one or more URL fragments. The patch instructions described below correct this problem.

To implement this patch, create a custom module to override `ProductDetails.Router.js`, which is part of the **ProductDetails** module. You can download the code samples described in this procedure here: [PageWithURLFragmentsRedirectsTooManyTimes.zip](#).

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file in a way that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the Override File

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name similar to the module being customized. For example, create `Modules/extensions/ProductDetailsExtension@1.0.0`.
3. In your new `ProductDetailsExtension@1.0.0` directory, create a subdirectory called `JavaScript`. For example: `Modules/extensions/ProductDetailsExtension@1.0.0/JavaScript`
4. Copy the following source file and paste into the correct location:

Copy This File:	Place a Copy Here:
<code>Modules/suitecommerce/ProductDetails@X.Y.Z/JavaScript/ProductDetails.Router.js</code>	<code>Modules/extensions/ProductDetailsExtension@1.0.0/JavaScript</code>

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

5. Open your new copy of `ProductDetails.Router.js` and locate the following lines:

```

1  if (api_query.id && item.get('urlcomponent') && SC.ENVIRONMENT.jsEnvironment === 'server')
2  {
3      nsglobal.statusCode = 301;
4      nsglobal.location = product.generateURL();
5  }
6
7  if (data.corrections && data.corrections.length > 0)
8  {
9      if (item.get('urlcomponent') && SC.ENVIRONMENT.jsEnvironment === 'server')
10     {
11         nsglobal.statusCode = 301;
12         nsglobal.location = data.corrections[0].url + product.getQuery();
13     }
14     else
15     {
16         return Backbone.history.navigate('#' + data.corrections[0].url + product.getQuery(), {trigger: true});
17     }
18 }
```

6. Replace these lines with the following code:

```

1  if (api_query.id && item.get('urlcomponent') && SC.ENVIRONMENT.jsEnvironment === 'server')
2  {
3      var productUrl = product.generateURL();
4      productUrl = productUrl[0] === '/' ? productUrl : '/' + productUrl;
5
6      nsglobal.statusCode = 301;
7      nsglobal.location = productUrl;
8  }
9
10 if (data.corrections && data.corrections.length > 0)
11 {
12     if (item.get('urlcomponent') && SC.ENVIRONMENT.jsEnvironment === 'server')
13     {
14         var url = data.corrections[0].url;
```

```

15     url = url[0] === '/' ? url : '/' + url;
16
17     nsglobal.statusCode = 301;
18     nsglobal.location = url + product.getQuery();
19   }
20   else
21   {
22     return Backbone.history.navigate('#' + data.corrections[0].url + product.getQuery(), {trigger: true});
23   }
24 }

```

7. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your Modules/extensions/ProductDetailsExtension@1.0.0 module directory.
2. Create a file in this directory and name it ns.package.json.
Modules/extensions/ProductDetailsExtension@1.0.0/ns.package.json
3. Paste the following code in your new ns.package.json file:

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5      ]
6    },
7    "overrides": {
8      "Modules/suitecommerce/ProductDetails@X.Y.Z/JavaScript/ProductDetails.Router.js" : "JavaScript/ProductDetail
9      s.Router.js"
10   }
11 }

```



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

4. Open the distro.json file. This file is located in your root directory.
5. Add your custom module to the modules object.

Your code should look similar to the following example:

```

1  {
2    "name": "SuiteCommerce Advanced Kilimanjaro",
3    "version": "2.0",
4    "isSCA": true,
5    "buildToolsVersion": "1.3.1",
6    "folders": {
7      "modules": "Modules",
8      "suitecommerceModules": "Modules/suitecommerce",
9      "extensionsModules": "Modules/extensions",
10     "thirdPartyModules": "Modules/third_parties",
11     "distribution": "LocalDistribution",
12     "deploy": "DeployDistribution"
13   },
14   "modules": {
15     "extensions/ProductDetailsExtension": "1.0.0",
16     "extensions/MyExampleCartExtension1": "1.0.0",
17     ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results. Upon successful deployment, instead of showing ERR_TOO_MANY_REDIRECTS, the browser shows a 404 error or the Product Details page.

See Complete List of Stores Link on Store Locator Page Does Not Show Store List

Applies to: SuiteCommerce Advanced | Kilimanjaro | Vinson | Elbrus

In some implementations of the Kilimanjaro, Vinson and Elbrus releases of SuiteCommerce Advanced, the See Complete List of Stores link on the Store Locator page reloads the page but does not show the complete list of stores. The patch instructions described below correct this problem.

To implement this patch, create a custom module to override the `store_locator_main.tpl`, `store_locator_list.tpl`, and `store_locator_list_all_store.tpl` files, which are part of the **StoreLocator@X.Y.Z** module. In this example, X.Y.Z represents the version of the StoreLocator module in your implementation of SuiteCommerce Advanced. You can download the code samples described in this procedure here: [StoreLocator.Extension@1.0.0 ElbrusCodeSamples.zip](#)

Step 1: Create the Override Files

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist. For example, create `Modules/extensions`
2. Within this directory, create a custom module with a name similar to the module being customized. For example, create `Modules/extensions/StoreLocator.Extension@1.0.0`.
3. In your new `StoreLocator.Extension@1.0.0` directory, create a subdirectory called `Templates`. For example: `Modules/extensions/StoreLocator.Extension@1.0.0/Templates`
4. Copy the following source file and paste into the correct location:

Copy This File:	Place a Copy Here:
<code>Modules/suitecommerce/StoreLocator@X.Y.Z/Templates/store_locator_main.tpl</code>	<code>Modules/extensions/StoreLocator.Extension@1.0.0/Templates</code>
<code>Modules/suitecommerce/StoreLocator@X.Y.X/Templates/store_locator_list.tpl</code>	<code>Modules/extensions/StoreLocator.Extension@1.0.0/Templates</code>
<code>Modules/suitecommerce/StoreLocator@X.Y.Z/Templates/store_locator_list_all_store.tpl</code>	<code>Modules/extensions/StoreLocator.Extension@1.0.0/Templates</code>

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

5. Revise your new `store_locator_main.tpl` file following these steps:
 - a. Locate the following line:

```
1 | <a data-touchpoint="{{touchpoint}}" data-hashtag="stores/all" href="stores/ all" href="s
   | tores/all">{{translate 'See complete list of stores'}}</a>
```

- b. Replace this line with the following code:

```
1 | <a href="stores/all">{{translate 'See complete list of stores'}}</a>
```

- c. Save the file.

6. Revise your new store_locator_list.tpl file following these steps:

- a. Locate the following line:

```
1 | <a data-hashtag="stores/details/{{storeId}}" data-touchpoint="{{touchpoint}}" data-toggle="show-in-
  | pusher">
```

- b. Replace these line with the following code:

```
1 | <a href="stores/details/{{storeId}}" data-toggle="show-in-pusher">
```

- c. Save the file.

7. Revise your new store_locator_list_all_store.tpl file following these steps:

- a. Locate the following lines:

```
1 | <a class="store-locator-list-all-store-name" data-touchpoint="{{touchpoint}}" data-toggle="show-in-
  | pusher" data-hashtag="stores/details/{{storeId}}" href="stores/details/{{storeId}}">{{name}}</a>
```

- b. Replace this line with the following code:

```
1 | <a class="store-locator-list-all-store-name" data-toggle="show-in-pusher" href="stores/details/
  | {{storeId}}">{{name}}</a>
```

- c. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your Modules/extensions/StoreLocator.Extension@1.0.0 module directory.
2. Create a file in this directory and name it ns.package.json.
Modules/extensions/StoreLocator.Extension@1.0.0/ns.package.json
3. Paste the following code in your new ns.package.json file:

```
1 | {
2 |   "gulp": {
3 |     "templates": [
4 |       "Templates/*"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/StoreLocator@X.Y.Z/Templates/store_locator_main.tpl": "Templates/store_locator_main.tpl",
9 |     "suitecommerce/StoreLocator@X.Y.Z/Templates/store_locator_list.tpl": "Templates/store_locator_list.tpl",
10 |    "suitecommerce/StoreLocator@X.Y.Z/Templates/store_locator_list_all_store.tpl": "Templates/store_locator_list_al
11 | l_store.tpl"
12 |   }
13 | }
```



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

4. Open the distro.json file. This file is located in your root directory.
5. Add your custom module to the modules object.

Your code should look similar to the following example:

```

1  {
2    "name": "SuiteCommerce Advanced Elbrus",
3    "version": "2.0",
4    "isSCA": true,
5    "buildToolsVersion": "1.3.0",
6    "folders": {
7      "modules": "Modules",
8      "suitecommerceModules": "Modules/suitecommerce",
9      "extensionsModules": "Modules/extensions",
10     "thirdPartyModules": "Modules/third_parties",
11     "distribution": "LocalDistribution",
12     "deploy": "DeployDistribution"
13   },
14   "modules": {
15     "extensions/StoreLocator.Extension": "1.0.0",
16     "suitecommerce/Account": "2.3.0",
17     ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, Store Locator reloads the page and shows the complete list of stores.

Quantity Pricing Displayed in Web Store Even When 'Require Login for Pricing' is Enabled

Applies to: SuiteCommerce Advanced | Kilimanjaro | Vinson | Elbrus

In some implementations of the Kilimanjaro, Vinson, and Elbrus releases of SuiteCommerce Advanced, quantity pricing displays in the web store even when Require Login for Pricing is enabled.

The following patch corrects a problem in QuantityPricing.View.js, which is part of the **QuantityPricing** module. To implement this patch, create a custom module to extend the prototype object for the initialize function. You can download the code samples described in this procedure here: [QuantityPricingAconcaguaCodeSamples..](#)

Note: Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Extend QuantityPricing.View.js

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name similar to the module being customized.

For example, create Modules/extensions/QuantityPricing.Extension@1.0.0.

3. In your new QuantityPricing.Extension@1.0.0 directory, create a subdirectory called JavaScript.
For example: Modules/extensions/QuantityPricing.Extension@1.0.0/JavaScript
4. In your new JavaScript subdirectory, create a JavaScript file to extend QuantityPricing.View.js.
Name this file according to best practices.
QuantityPricing.View.Extension.js
5. Open this file and extend the QuantityPricing.View method as shown in the following code snippet.

```

1  define(
2    'QuantityPricing.View.Extension'
3  , [
4    'QuantityPricing.View'
5    , 'QuantityPricing.Utils'
6    , 'SC.Configuration'
7    , 'Profile.Model'
8    , 'Backbone'
9    , 'underscore'
10  ]
11  , function (
12    QuantityPricingView
13    , QuantityPricingUtils
14    , Configuration
15    , ProfileModel
16    , Backbone
17    , -
18  )
19  {
20    'use strict';
21
22    _.extend(QuantityPricingView.prototype,
23    {
24      initialize: function ()
25      {
26
27        this.profileModel = ProfileModel.getInstance();
28
29        this._isEnabled = !this.profileModel.hidePrices();
30
31        this.price_schedule = QuantityPricingUtils.rearrangeQuantitySchedule(this.model.get('item'), _.isFunction(this.model.getSelectedMatrixChilds) ? this.model.getSelectedMatrixChilds() : []);
32
33        this.model.on('change', function ()
34        {
35          var new_price_schedule = QuantityPricingUtils.rearrangeQuantitySchedule(this.model.get('item'), _.isFunction(this.model.getSelectedMatrixChilds) ? this.model.getSelectedMatrixChilds() : []);
36
37          if (!_.isEqual(this.price_schedule, new_price_schedule))
38          {
39            this.price_schedule = new_price_schedule;
40            this.render();
41          }
42        }, this);
43
44        this.item_key = this.model.get('item').id + ' ' + (new Date()).getMilliseconds();
45      }
46    });
47  });
48

```

6. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new QuantityPricing.Extension@1.0.0 module directory.

2. Create a file in this directory and name it ns.package.json.

Modules/extensions/QuantityPricing.Extension@1.0.0/ns.package.json

3. Paste the following code into your new ns.package.json file:

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5      ]
6    }
7  }
```

4. Open the distro.json file. This is located in your root directory.
5. Add your custom module to the modules object.

Your code should look similar to the following example:

```

1  {
2    "name": "SuiteCommerce Advanced Aconcagua",
3    "version": "2.0",
4    "isSCA": true,
5    "buildToolsVersion": "sco-2018.1.0",
6    "folders": {
7      "modules": "Modules",
8      "suitecommerceModules": "Modules/suitecommerce",
9      "extensionsModules": "Modules/extensions",
10     "thirdPartyModules": "Modules/third_parties",
11     "distribution": "LocalDistribution",
12     "deploy": "DeployDistribution"
13   },
14   "modules": {
15     "extensions/QuantityPricing.Extension": "1.0.0",
16     "suitecommerce/Account": "sco-2018.1.0",
17     ...
18   }
```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Add QuantityPricing.Extension as a dependency to SCA entry point within the SC.Shopping.Starter and SC.MyAccount.Starter entrypoints of the JavaScript array.

Your Distro.js file should look similar to the following:

```

1  "tasksConfig": {
2    //...
3    "javascript": [
4      //...
5      {
6        "entryPoint": "SC.Shopping.Starter",
7        "exportFile": "shopping.js",
8        "dependencies": [
9          //...
10         "ProductDetailToQuote",
11         "StoreLocator",
12         "SC.CCT.Html",
13         "QuantityPricing.View.Extension"
14       ],
15       //...
16     },
17     {
18       "entryPoint": "SC.MyAccount.Starter",
19       "exportFile": "myaccount.js",
20       "dependencies": [
21         //...
22         "GoogleMap.Configuration",
23         "StoreLocatorAccessPoints",
24         "SC.CCT.Html",
25       ]
26     }
27   ]
28 }
```

```

24 |         "QuantityPricing.View.Extension"
25 |     ],
26 |     //...

```

7. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

Upon successful deployment, quantity pricing will not display in the web store when Require Login for Pricing is enabled.

Edited Shipping Address on the Review Your Order Page is Not Showing Changes

i Applies to: SuiteCommerce Advanced | Aconcagua | Kilimanjaro

In some implementations of the Aconcagua and Kilimanjaro releases of SuiteCommerce Advanced, the Review Your Order page is not showing changes when user edits the shipping address.

To implement this patch, create a custom module to override the OrderWizard.Module.ShowShipments.js file, which is part of the **OrderWizard.Module.Shipmethod@X.Y.Z** module. In this example, X.Y.Z represents the version of the OrderWizard.Module.Shipment module in your implementation of SuiteCommerce Advanced. You can download the code samples described in this procedure here: [OrderWizard.Module.Shipmethod.AconcaguaSampleCode.zip](#)

In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file in a way that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the Override File

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist. For example, create Modules/extensions
2. Within this directory, create a custom module with a name similar to the module being customized. For example, create Modules/extensions/OrderWizard.Module.Shipmethod.Extension@1.0.0.
3. In your new StoreLocator.Extension@1.0.0 directory, create a subdirectory called JavaScript. For example: Modules/extensions/OrderWizard.Module.Shipmethod.Extension@1.0.0/JavaScript
4. Copy the following source file and paste into the correct location:

Copy This File:	Place a Copy Here:
Modules/suitecommerce/OrderWizard.Module.Shipment@X.Y.Z/OrderWizard.Module.ShowShipments.js	Modules/extensions/OrderWizard.Module.Shipmethod.Extension@1.0.0/JavaScript

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

5. Revise your new OrderWizard.Module.ShowShipments.js file following these steps:
 - a. Replace the existing initialize function with this code:

```

1 | ,   initialize: function ()
2 |     {
3 |         WizardModule.prototype.initialize.apply(this, arguments);
4 |
5 |         this.application = this.wizard.application;
6 |         this.options.application = this.wizard.application;
7 |
8 |         this.addressSource = this.options.useModelAddresses ?
9 |             this.model.get('addresses') : this.wizard.options.profile.get('addresses');
10 |
11 |         BackboneCompositeView.add(this);
12 |
13 |         this.wizard.model.on('ismultishiptoUpdated', this.render, this);
14 |         this.wizard.model.on('promocodeUpdated', this.render, this);
15 |         this.address = this.addressSource.get(this.model.get('shipaddress'));
16 |         this.address && this.address.on('change', this.render, this);
17 |     }

```

- b. Add this new destroy function after the initialize function:

```

1 | ,   destroy: function destroy()
2 |     {
3 |         this.address && this.address.off('change', this.render, this);
4 |         return this._destroy();
5 |     }

```

- c. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your Modules/extensions/StoreLocator.Extension@1.0.0 module directory.
2. Create a file in this directory and name it ns.package.json.
Modules/extensions/OrderWizard.Module.Shipmethod.Extension@1.0.0/ns.package.json
3. Paste the following code in your new ns.package.json file:

```

1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/OrderWizard.Module.Shipmethod@X.Y.Z/JavaScript/OrderWizard.Module.ShowShipments.js": "JavaScript/
OrderWizard.Module.ShowShipments.js"
9 |   }
10 | }

```



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

4. Open the distro.json file. This file is located in your root directory.
5. Add your custom module to the modules object.

Your code should look similar to the following example:

```

1 | {
2 |   "name": "SuiteCommerce Advanced Aconcagua",
3 |   "version": "2.0",

```

```

4  "isSCA": true,
5  "buildToolsVersion": "sc0-2018.1.0",
6  "folders": {
7    "modules": "Modules",
8    "suitecommerceModules": "Modules/suitecommerce",
9    "extensionsModules": "Modules/extensions",
10   "thirdPartyModules": "Modules/third_parties",
11   "distribution": "LocalDistribution",
12   "deploy": "DeployDistribution"
13 },
14   "modules": {
15     "extensions/OrderWizard.Module.Shipmethod.Extension": "1.0.0",
16     "suitecommerce/Account": "sc0-2018.1.0",
17     ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, edited shipping address information displays on the Review Your Order page.

Custom Page Title in SMT Does Not Display Correctly

i Applies to: SuiteCommerce Advanced | Aconcagua

In some implementations of SuiteCommerce Advanced, creating a title for a custom facets page using Site Management Tools does not display the correct title. In these cases, editing the page name on a custom page results in the page title inheriting the facet path instead of the custom title.

The following patch corrects a problem in CMSadapter.Page.Router.js, which is part of the **CMSAdapter** module. To implement this patch, create a custom module to extend the prototype object for the getPageForFragment method. You can download the code samples described in this procedure here: [CustomPageTitlesNotDisplayingCorrectly.zip](#).

i Note: Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Extend CMSadapter.Page.Router.js

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name similar to the module being customized. For example: Modules/extensions/CMSadapterExtension@1.0.0.
3. Open your new CMSadapterExtension@1.0.0 module and create a subfolder titled JavaScript. For example: Modules/extensions/CMSadapterExtension@1.0.0/JavaScript
4. In your new JavaScript subdirectory, create a JavaScript file to extend CMSadapter.Page.Router.Extend.js.

Name this file according to best practices.

For example: CMSAdapter.Page.Router.Extend.js

5. Open this file and extend the `getPageForFragment()` method as shown in the following code snippet.

```

1  define(
2    'CMSAdapter.Page.Router.Extend'
3  , [
4    'CMSAdapter.Page.Router'
5    , 'underscore'
6    , 'Backbone'
7  ]
8  , function (
9    CMSAdapterPageRouter
10
11    ,
12    Backbone
13  )
14  {
15    'use strict';
16
17    _._extend(CMSAdapterPageRouter.prototype,
18      {
19        getPageForFragment: function getPageForFragment(fragment, allPages)
20        {
21          fragment = fragment || Backbone.history.fragment || '';
22          fragment = fragment.split('?')[0]; //remove options
23
24          var collection = allPages ? this.allPages : this.landingPages;
25          return collection.find(function(page)
26          {
27            return ((page.get('url') === fragment) || (page.get('url') === '/' + fragment));
28          });
29        }
30      });
31  });

```

6. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new CMSAdapterExtension@1.0.0 module directory.
2. Create a file in this directory titled `ns.package.json`.
Modules/extensions/CMSAdapterExtension@1.0.0/ns.package.json
3. Paste the following code into your new `ns.package.json` file:

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5      ]
6    }
7  }

```

4. Open the `distro.json` file. This is located in your root directory.
5. Add your custom module to the `modules` object.

Your code should look similar to:

```

1  {
2    "name": "SuiteCommerce Advanced Kilimanjaro",
3    "version": "2.0",
4    "isSCA": true,

```

```

5  "buildToolsVersion": "1.3.1",
6  "folders": {
7    "modules": "Modules",
8    "suitecommerceModules": "Modules/suitecommerce",
9    "extensionsModules": "Modules/extensions",
10   "thirdPartyModules": "Modules/third_parties",
11   "distribution": "LocalDistribution",
12   "deploy": "DeployDistribution"
13 },
14   "modules": {
15     "extensions/CMSadapterExtension": "1.0.0",
16     "extensions/MyExampleCartExtension1": "1.0.0",
17     ...

```

This step ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Add `CMSadapter.Page.Router.Extend` as a dependency to `SC.Shopping.Starter`, `SC.Checkout.Starter`, and `SC.MyAccount.Starter` entrypoints of the JavaScript array.

Your `distro.js` file should look similar to:

```

1  "tasksConfig": {
2    //...
3    "javascript": [
4      //...
5      {
6        "entryPoint": "SC.Shopping.Starter",
7        "exportFile": "shopping.js",
8        "dependencies": [
9          //...
10         "ProductDetailToQuote",
11         "StoreLocator",
12         "CMSadapter.Page.Router.Extend"
13       ],
14       //...
15
16       "entryPoint": "SC.MyAccount.Starter",
17       "exportFile": "myaccount.js",
18       "dependencies": [
19         //...
20         "StoreLocatorAccessPoints",
21         "StoreLocator",
22         "SC.CCT.Html",
23         "CMSadapter.Page.Router.Extend"
24       ],
25       //...
26
27       "entryPoint": "SC.Checkout.Starter",
28       "exportFile": "checkout.js",
29       "dependencies": [
30         //...
31         "StoreLocatorAccessPoints",
32         "StoreLocator",
33         "SC.CCT.Html",
34         "CMSadapter.Page.Router.Extend"
35       ],
36       //...

```

7. Save the `distro.json` file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

Upon successful deployment, custom page titles should display as expected.

Currencies Change to the Default in the Shopping Application

Applies to: SuiteCommerce Advanced | Kilimanjaro

In some implementations of SuiteCommerce Advanced using multi-subsidiaries, users cannot change the currency on the site. The webstore continues to use the default currency.

The following patch corrects the `GlobalViews.CurrencySelector.View.js`, which is part of the **GlobalViews** module. To implement this patch, create a custom module to extend the prototype object for the `getContext()` method. You can download the code samples described in this procedure here: [CurrenciesChangeToDefault-Kilimanjaro.zip](#).

Step 1: Extend `GlobalViews.CurrencySelector.View.js`

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name that is unique but similar to the module being customized.

For example: `Modules/extensions/GlobalViewsExtension@1.0.0`.

3. Open your new `GlobalViewsExtension@1.0.0` module and create a subfolder titled `JavaScript`.

For example: `Modules/extensions/GlobalViewsExtension@1.0.0/JavaScript`

4. In your new `JavaScript` subdirectory, create a `JavaScript` file to extend `GlobalViews.CurrencySelector.View.js`.

Name this file according to best practices.

For example: `GlobalViews.CurrencySelector.View.Extend.js`

5. Open this file and extend the `getContext()` method as shown in the following code snippet.

```

1  define(
2    'GlobalViews.CurrencySelector.View.Extend'
3    , [
4      'GlobalViews.CurrencySelector.View'
5
6      , 'Backbone'
7      , 'underscore'
8    ]
9    , function(
10      GlobalViewsCurrencySelectorView
11
12      , Backbone
13      , _
14    )
15    {
16      'use strict';
17
18      _.extend(GlobalViewsCurrencySelectorView.prototype,
19        {
20          getContext: function()
21          {
22            var available_currencies = _.map(SC.ENVIRONMENT.availableCurrencies, function(currency)
23            {
24              // @class GlobalViews.CurrencySelector.View.Context.Currency

```

```

25         return {
26             // @property {String} code
27             code: currency.code
28             // @property {String} internalId
29             , internalId: currency.internalId
30             // @property {String} isDefault
31             , isDefault: currency.isdefault
32             // @property {String} symbol
33             , symbol: currency.symbol
34             // @property {Boolean} symbolPlacement
35             , symbolPlacement: currency.symbolplacement
36             // @property {String} displayName
37             , displayName: currency.title || currency.name
38             // @property {Boolean} isSelected
39             , isSelected: SC.getSessionInfo('currency').code === currency.code
40         };
41     });
42
43     // @class GlobalViews.CurrencySelector.View.Context
44     return {
45         // @property {Boolean} showCurrencySelector
46         showCurrencySelector: !(SC.ENVIRONMENT.availableCurrencies && SC.ENVIRONMENT.availableCurrencies.length
47 > 1)
48         // @property {Array<GlobalViews.CurrencySelector.View.Context.Currency>} availableCurrencies
49         , availableCurrencies: available_currencies || []
50         // @property {String} currentCurrencyCode
51         , currentCurrencyCode: SC.getSessionInfo('currency').code
52         // @property {String} currentCurrencySymbol
53         , currentCurrencySymbol: SC.getSessionInfo('currency').symbol
54     };
55 });
56 });

```

6. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new Transaction.Line.Views.Extension@1.0.0 module directory.
2. Create a file in this directory titled ns.package.json.
Modules/extensions/Transaction.Line.Views.Extension@1.0.0/ns.package.json
3. Paste the following code into your new ns.package.json file:

```

1  {
2      "gulp": {
3          "javascript": [
4              "JavaScript/*.js"
5          ]
6      }
7  }

```

4. Open the distro.json file. This is located in your root directory.
5. Add your custom module to the modules object.

Your code should look similar to:

```

1  {
2      "name": "SuiteCommerce Advanced Kilimanjaro",
3      "version": "2.0",
4      "isSCA": true,
5      "buildToolsVersion": "1.3.1",
6      "folders": {
7          "modules": "Modules",
8          "suitecommerceModules": "Modules/suitecommerce",
9          "extensionsModules": "Modules/extensions",
10         "thirdPartyModules": "Modules/third_parties",

```



```

11     "distribution": "LocalDistribution",
12     "deploy": "DeployDistribution"
13 },
14 "modules": {
15     "extensions/GlobalViewsExtension": "1.0.0",
16     "extensions/MyExampleCartExtension1": "1.0.0",
17     ...

```

This step ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

Upon successful deployment, Checkout Summary pages should display correct original and discounted sub-total amounts. Strike-through amounts also display correctly.

Order Confirmation Page Not Displayed When Using Single Page Checkout and External Payment

Applies to: SuiteCommerce Advanced | Vinson | Elbrus

In some implementations of the Vinson and Elbrus releases of SuiteCommerce Advanced, the Order Confirmation Page is not displayed when using one page checkout with an external payment system.

To implement this patch, create a custom module to override SC.Checkout.Configuration.Steps.OPC.js, which is part of the **CheckoutApplication@2.2.0** module. You can download the code samples described in this procedure here: [CheckoutApplication.Extension@1.0.0 VinsonCodeSamples.zip](#)

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file in a way that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the Override File

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within the extension directory, create a directory for a new custom module with a name similar to the module being customized.

For example, create Modules/extensions/CheckoutApplication.Extension@1.0.0.

3. In your new CheckoutApplicationExtension@1.0.0 module directory, create a directory called JavaScript.

For example: Modules/extensions/CheckoutApplication.Extension@1.0.0/JavaScript

4. Copy the following source file and paste into the correct location:

Copy This File:	Place a Copy Here:
Modules/suitecommerce/CheckoutApplication@X.Y.Z/ JavaScript/ SC.Checkout.Configuration.Steps.OPC.js	Modules/extensions/CheckoutApplication. Extension@1.0.0/JavaScript

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

5. Open your new copy of SC.Checkout.Configuration.Steps.OPC.js and locate the following line:

```
1 | , [OrderWizardModulePaymentMethodSelector, {record_type:'salesorder', preventDefault: true}]
```

6. Replace this line with the following code:

```
1 | , [OrderWizardModulePaymentMethodSelector, {record_type:'salesorder', prevent_default: true}]
```

7. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your Modules/extensions/CheckoutApplication.Extension@1.0.0 directory.
2. Create a file in this directory and name it ns.package.json.
Modules/extensions/CheckoutApplication.Extension@1.0.0/ns.package.json
3. Paste the following code in your new ns.package.json file:

```
1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/CheckoutApplication@2.2.0/JavaScript/SC.Checkout.Configuration.Steps.OPC.js" : "JavaScript/SC.Check
out.Configuration.Steps.OPC.js"
9 |   }
10 | }
```



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

4. Open the distro.json file. This file is located in your root directory.
5. Add your custom module to the modules object.

Your code should look similar to the following example:

```
1 | {
2 |   "name": "SuiteCommerce Advanced Vinson Release",
3 |   "version": "2.0",
4 |   "isSCA": true,
5 |   "buildToolsVersion": "1.2.1",
6 |   "folders": {
7 |     "modules": "Modules",
8 |     "suitecommerceModules": "Modules/suitecommerce",
9 |     "extensionsModules": "Modules/extensions",
10 |    "thirdPartyModules": "Modules/third_parties",
11 |    "distribution": "LocalDistribution",
12 |    "deploy": "DeployDistribution"
13 |   },
14 |   "modules": {
```

```

15 | "extensions/CheckoutApplication.Extension": "1.0.0",
16 | "suitecommerce/Account": "2.2.0",
17 | ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, the Order Confirmation Page is displayed after completing an order when using one page checkout with an external payment system.

Incorrect Discounted Amounts on Checkout Summary

Applies to: SuiteCommerce Advanced | Aconcagua | Kilimanjaro | Elbrus

In some implementations of SuiteCommerce Advanced, Checkout Summary pages display incorrect discounted sub-total amounts and inconsistent strike-through and original amounts. These issues occur on Subtotal and Items to Ship sections of the checkout flow. The patch instructions described below correct this problem.

The following patch corrects a problem in `Transaction.Line.Views.Cell.Navigable.View.js`, which is part of the **Transaction.Line.Views** module. To implement this patch, create a custom module to extend the prototype object for the `getContext()` method. You can download the code samples described in this procedure here: [IncorrectDiscountedAmountsOnCheckout.zip](#).

Step 1: Extend Transaction.Line.Views.Cell.Navigable.View.js

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name that is unique but similar to the module being customized.
For example: `Modules/extensions/Transaction.Line.Views.Extension@1.0.0`.
3. Open your new `Transaction.Line.Views.Extension@1.0.0` module and create a subfolder titled `JavaScript`.
For example: `Modules/extensions/Transaction.Line.Views.Extension@1.0.0/JavaScript`
4. In your new `JavaScript` subdirectory, create a JavaScript file to extend `Transaction.Line.Views.Cell.Navigable.View.js`.
Name this file according to best practices.
For example: `Transaction.Line.Views.Cell.Navigable.View.Extend.js`
5. Open this file and extend the `getContext()` method as shown in the following code sample.

```

1 | define('Transaction.Line.Views.Cell.Navigable.View.Extend'

```

```

2  , [ 'Transaction.Line.Views.Cell.Navigable.View'
3    , 'underscore'
4    , 'Backbone'
5  ]
6  , function (
7      TransactionLineViewsCellNavigableView
8      , _
9      , Backbone
10 )
11 {
12     'use strict';
13
14     _._extend(TransactionLineViewsCellNavigableView.prototype,
15         {
16             getContext: function ()
17             {
18                 var item = this.model.get('item')
19                 , line = this.model;
20
21                 // @class Transaction.Line.Views.Navigable.View.Context
22                 return {
23                     // @property {Transaction.Line.Model} model
24                     model: this.model
25                     // @property {String} itemId
26                     , itemId: item.get('internalid')
27                     // @property {String} itemName
28                     , itemName: item.get('_name')
29                     // @property {String} cellClassName
30                     , cellClassName: this.options.cellClassName
31                     // @property {Boolean} isNavigable
32                     , isNavigable: !!this.options.navigable && !!item.get('_isPurchasable')
33                     // @property {String} rateFormatted
34                     , rateFormatted: line.get('rate_formatted')
35                     // @property {Boolean} showOptions
36                     , showOptions: !(line.get('options') && line.get('options').length)
37                     // @property {String} itemURLAttributes
38                     , itemURLAttributes: line.getFullLink({quantity:null,location:null,fulfillmentChoice:null})
39                     // @property {Number} quantity
40                     , quantity: line.get('quantity')
41                     // @property {Boolean} showDetail2Title
42                     , showDetail2Title: !!this.options.detail2Title
43                     // @property {String} detail2Title
44                     , detail2Title: this.options.detail2Title
45                     // @property {String} detail2
46                     , detail2: line.get(this.options.detail2)
47                     // @property {Boolean} showBlockDetail2
48                     , showBlockDetail2: !!line.get(this.options.detail2)
49                     // @property {Boolean} showDetail3Title
50                     , showDetail3Title: !!this.options.detail3Title
51                     // @property {String} detail3Title
52                     , detail3Title: this.options.detail3Title
53                     // @property {String} detail3
54                     , detail3: line.get('amount') > line.get('total') ? line.get('total_formatted') : line.get(this.option
55 s.detail3)
56                     // @property {Boolean} showComparePrice
57                     , showComparePrice: line.get('amount') > line.get('total')
58                     // @property {String} comparePriceFormatted
59                     , comparePriceFormatted: line.get('amount_formatted')
60                     // @property {ImageContainer} thumbnail
61                     , thumbnail: this.model.getThumbnail()
62                     // @property {Boolean} isFreeGift
63                     , isFreeGift: line.get('free_gift') === true
64                 };
65             }
66         });

```

6. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new Transaction.Line.Views.Extension@1.0.0 module directory.

2. Create a file in this directory titled `ns.package.json`.

`Modules/extensions/Transaction.Line.Views.Extension@1.0.0/ns.package.json`

3. Paste the following code into your new `ns.package.json` file:

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5      ]
6    }
7  }
```

4. Open the `distro.json` file. This is located in your root directory.
5. Add your custom module to the `modules` object.

Your code should look similar to:

```

1  {
2    "name": "SuiteCommerce Advanced Aconcagua",
3    "version": "2.0",
4    "isSCA": true,
5    "buildToolsVersion": "aconcaguaR2",
6    "folders": {
7      "modules": "Modules",
8      "suitecommerceModules": "Modules/suitecommerce",
9      "extensionsModules": "Modules/extensions",
10     "thirdPartyModules": "Modules/third_parties",
11     "distribution": "LocalDistribution",
12     "deploy": "DeployDistribution"
13   },
14   "modules": {
15     "extensions/Transaction.Line.Views.Extension": "1.0.0",
16     "suitecommerce/Account": "aconcaguaR2",
17     ...
18   }
```

This step ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the `modules` object. The order of precedence in this list does not matter.

6. Add `Transaction.Line.Views.Cell.Navigable.View.Extend` as a dependency to SCA entry point within the `SC.Checkout.Starter` entrypoint of the JavaScript array.

Your `distro.js` file should look similar to:

```

1  "tasksConfig": {
2    //...
3    "javascript": [
4      //...
5      {
6        "entryPoint": "SC.Checkout.Starter",
7        "exportFile": "checkout.js",
8        "dependencies": [
9          //...
10         "StoreLocator",
11         "SC.CCT.Html",
12         "Transaction.Line.Views.Cell.Navigable.View.Extend"
13       ]
14     },
15     //...
16   }
```

7. Save the `distro.json` file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA](#))

[Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

Upon successful deployment, Checkout Summary pages should display correct original and discounted sub-total amounts. Strike-through amounts also display correctly.

DeployDistribution Folder Does Not Include Local Files

i Applies to: SuiteCommerce Advanced | Aconcagua

In some implementations of the Aconcagua R2 release of SuiteCommerce Advanced, local files are not deploying to the DeployDistribution folder after running the gulp deploy command using the developer tools. As a consequence, accessing Local SSP pages result in a Page Not Found error. This problem occurs because the R2 release of Aconcagua did not include three important files. This patch adds these files to your implementation.

To implement this patch, you must add the following files to your source and deploy:

- CheckoutApplication@aconcaguaR2\Internal\ index-local.ssp
- MyAccountApplication@aconcaguaR2\Internal\ index-local.ssp
- ShoppingApplication@aconcaguaR2\Internal\ index-local.ssp

Because these files do not exist, you cannot use traditional best practices, such as extending or overriding. This instruction simply requires adding three files, renaming them to match the applicable module, and deploying.

Step 1: Download the Patch Files

1. Download the .zip file containing the three local.ssp files here: [LocalFilesNotDeploying-AconcaguaR2.zip](#).

This zip file includes the following files:

- checkout-local.ssp
- my_account-local.ssp
- shopping-local.ssp

2. Extract the contents of this file to any location locally.
3. For the Checkout application:
 - a. Copy checkout-local.ssp to the CheckoutApplication@aconcaguaR2/Internal folder.
If the Internal folder does not exist, create it.
 - b. Rename checkout-local.ssp to **index-local.ssp**.
4. For the MyAccount application:
 - a. Copy myaccount-local.ssp to the MyAccountApplication@aconcaguaR2/Internal folder.
If the Internal folder does not exist, create it.
 - b. Rename myaccount-local.ssp to **index-local.ssp**.
5. For the Shopping application:
 - a. Copy shopping-local.ssp to the ShoppingApplication@aconcaguaR2/Internal folder.
If the Internal folder does not exist, create it.

- b. Rename shopping- local.ssp to **index-local.ssp**.

Your source directory structure should look like this:

```

1  ...
2  CheckoutApplication@aconcaguaR2/
3      Internal/
4          index-local.ssp
5  ...
6  MyAccountApplication@aconcaguaR2/
7      Internal/
8          index-local.ssp
9  ...
10 ShoppingApplication@aconcaguaR2/
11     Internal/
12         index-local.ssp
13     ....

```

Step 2: Deploy Your Customization

1. Deploy your source code customizations to your NetSuite account and test the functionality. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.

Note: Since this patch adds SuiteScript files, changes are not visible in your local environment. SuiteScript files run on the server and must be deployed to NetSuite to take effect.

2. Confirm your results.

Upon successful deployment, local files should exist in your DeployDistribution folder.

HTML List Styles Do Not Display

Applies to: SuiteCommerce | SuiteCommerce Advanced | Aconcagua | Kilimanjaro | Elbrus | Vinson

In some implementations of SuiteCommerce and SuiteCommerce Advanced, editing item descriptions on an Item record (**Web Store** tab, **Detailed Description** field) using `<li>` and `<ul>` styles does not display as expected in the website. This patch corrects this issue. However, the correct patching instructions depend on your implementation, as described below:

Implementation	Patching Instructions
SCA Kilimanjaro	HTML List Styles Do Not Display (Kilimanjaro)
SCA Vinson, Elbrus	HTML List Styles Do Not Display (Elbrus and Vinson)
Any theme created using the 2018.1 release of the SuiteCommerce Base Theme bundle	HTML List Style Does Not Display (Theme)

HTML List Styles Do Not Display (Kilimanjaro)

Applies to: SuiteCommerce Advanced | Kilimanjaro

Apply the changes in this patch using the override method, as described in this section. The following table lists the files to be overridden and their locations in the source code:

Module Location	File
BaseSassStyles	_base.scss
ProductDetails	_product-details-information.scss

You can download the code samples described in this procedure here: [HtmlListStyleNotDisplaying-Kilimanjaro.zip](#)

Step 1: Create the Override File

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name that is unique, but similar to each module being customized. Give these new modules a version of **1.0.0**.

Your implementation should look like this:

- Modules/extensions/BaseSassStylesExtension@1.0.0
- Modules/extensions/ProductDetailsExtension@1.0.0

3. Open your new BaseSassStylesExtension@1.0.0 module and create nested subfolders titled **Sass** and **base**.

For example: Modules/extensions/BaseSassStylesExtension@1.0.0/Sass/base

4. Open your new ProductDetailsExtension@1.0.0 module and create a subfolder titled Sass.

For example: Modules/extensions/ProductDetailsExtension@1.0.0/Sass

5. Copy the following source files and paste into the correct location, where X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced:

Copy This File:	Place a Copy Here:
Modules/suitecommerce/BaseSassStyles@X.Y.Z/Sass/base/_base.scss	Modules/extensions/BaseSassStyles@1.0.0/Sass/base
Modules/suitecommerce/ProductDetails@X.Y.Z/Sass/_product-details-information.scss	Modules/extensions/ProductDetails@1.0.0/Sass

6. Open your new copy of _base.scss and add the following lines to the end of file:

```

1 | .cms-content ul, .cms-content ol {
2 |     margin-left: $sc-margin-lv4;
3 | }
4 |
5 | .cms-content ul li {
6 |     list-style: circle;
7 | }
8 |
9 | .cms-content ol li {
10 |     list-style: decimal;
11 | }
```

7. Save the file.
8. Open your new copy of _product-details-information.scss and add the following lines to the end of file:

```

1 | .product-details-information-tab-content-container ul, .product-details-information-tab-content-container ol {
2 |     margin-left: $sc-margin-lv4;
3 | }
4 |
5 | .product-details-information-tab-content-container ul li {
```



```

6   list-style: outside;
7   }
8
9   .product-details-information-tab-content-container ol li {
10    list-style: decimal;
11  }

```

9. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your Modules/extensions/BaseSassStyles@1.0.0 module.
2. Create a file in this directory titled **ns.package.json**.
For example: Modules/extensions/BaseSassStyles@1.0.0/ns.package.json
3. Paste the following code in your new ns.package.json file:

 **Important:** You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

```

1  {
2    "gulp": {
3      "sass": [
4        "Sass/**/*.scss"
5      ]
6    },
7    "overrides": {
8      "suitecommerce/BaseSassStyles@X.Y.Z/Sass/base/_base.scss" : "Sass/base/_base.scss"
9    }
10 }

```

4. Save the file.
5. Open your Modules/extensions/ProductDetails@1.0.0 module.
6. Create a file in this directory titled **ns.package.json**.
For example: Modules/extensions/ProductDetails@1.0.0/ns.package.json
7. Paste the following code in your new ns.package.json file:

 **Important:** You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

```

1  {
2    "gulp": {
3      "sass": [
4        "Sass/**/*.scss"
5      ]
6    },
7    "overrides": {
8      "suitecommerce/ProductDetails@X.Y.Z/Sass/_product-details-information.scss" : "Sass/_product-details-informa
9      tion.scss"
10   }

```

8. Open the distro.json file. This file is located in your root directory.
9. Add both custom modules to the modules object.

Your code should look similar to:

```

1  {
2    "name": "SuiteCommerce Advanced Kilimanjaro",
3    "version": "2.0",

```

```

4      "isSCA": true,
5      "buildToolsVersion": "1.3.1",
6      "folders": {
7        "modules": "Modules",
8        "suitecommerceModules": "Modules/suitecommerce",
9        "extensionsModules": "Modules/extensions",
10       "thirdPartyModules": "Modules/third_parties",
11       "distribution": "LocalDistribution",
12       "deploy": "DeployDistribution"
13     },
14     "modules": {
15       "extensions/BaseSassStylesExtension": "1.0.0",
16       "extensions/ProductDetailsExtension": "1.0.0",
17       "extensions/MyExampleCartExtension1": "1.0.0",
18       ...

```

This step ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

10. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, the HTML list (and) styles should display on your webstore.

HTML List Styles Do Not Display (Elbrus and Vinson)

Applies to: SuiteCommerce Advanced | Elbrus | Vinson

Apply the changes in this patch using the override method, as described in this section. The following table lists the files to be overridden and their locations in the source code:

Module Location	File
BaseSassStyles	_base.scss
ProductDetails	_product-details-information.scss

You can download the code samples described in this procedure here: [HtmlListStylesNotDisplaying-Elbrus-Vinson.zip](#)

Step 1: Create the Override File

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name that is unique, but similar to each module being customized. Give these new modules a version of **1.0.0**.

Your implementation should look like this:

- Modules/extensions/BaseSassStylesExtension@1.0.0
- Modules/extensions/ProductDetailsExtension@1.0.0

3. Create a subfolder for each new module titled **Sass**.

Your file structure should look like:

- Modules/extensions/BaseSassStylesExtension@1.0.0/Sass
- Modules/extensions/ProductDetailsExtension@1.0.0/Sass

4. Copy the applicable source files and paste into the correct location, where X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced:

Copy This File:	Place a Copy Here:
Modules/suitecommerce/BaseSassStyles@X.Y.Z/Sass/_base.scss	Modules/extensions/BaseSassStyles@1.0.0/Sass
Modules/suitecommerce/ProductDetails@X.Y.Z/Sass/_product-details-information.scss	Modules/extensions/ProductDetails@1.0.0/Sass

5. Open your new copy of _base.scss and add the following lines to the end of file:

```

1 | .cms-content ul, .cms-content ol {
2 |     margin-left: $sc-margin-lv4;
3 | }
4 |
5 | .cms-content ul li {
6 |     list-style: circle;
7 | }
8 |
9 | .cms-content ol li {
10 |     list-style: decimal;
11 | }
```

6. Save the file.
7. Open your new copy of _product-details-information.scss and add the following lines to the end of file:

```

1 | .product-details-information-tab-content-container ul, .product-details-information-tab-content-container ol {
2 |     margin-left: $sc-margin-lv4;
3 | }
4 |
5 | .product-details-information-tab-content-container ul li {
6 |     list-style: outside;
7 | }
8 |
9 | .product-details-information-tab-content-container ol li {
10 |     list-style: decimal;
11 | }
```

8. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Create an ns.package.json file for each custom module.
 - a. Open your Modules/extensions/BaseSassStyles@1.0.0 module.
 - b. Create a file in this directory titled **ns.package.json**.
For example: Modules/extensions/BaseSassStyles@1.0.0/ns.package.json
 - c. Paste the following code in your new ns.package.json file:



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

```

1 | {
2 |     "gulp": {
```

```

3      "sass": [
4          "Sass/**/*.scss"
5      ],
6  },
7  "overrides": {
8      "suitecommerce/BaseSassStyles@X.Y.Z/Sass/_base.scss" : "Sass/_base.scss"
9  }
10 }

```

- d. Save the file.
- e. Open your Modules/extensions/ProductDetails@1.0.0 module.
- f. Create a file in this directory titled **ns.package.json**.
For example: Modules/extensions/ProductDetails@1.0.0/ns.package.json
- g. Paste the following code in your new ns.package.json file:

 **Important:** You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

```

1  {
2      "gulp": {
3          "sass": [
4              "Sass/**/*.scss"
5          ]
6      },
7      "overrides": {
8          "suitecommerce/ProductDetails@X.Y.Z/Sass/_product-details-information.scss" : "Sass/_product-details-information.scss"
9      }
10 }

```

- h. Save the file.
2. Edit your distro.json file.

This step ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

- a. Open the distro.json file. This file is located in your root directory.
- b. Add both custom modules to the modules object.

Your code should look similar to:

```

1  {
2      "name": "SuiteCommerce Advanced Elbrus",
3      "version": "2.0",
4      "buildToolsVersion": "1.3.0",
5      "folders": {
6          "modules": "Modules",
7          "suitecommerceModules": "Modules/suitecommerce",
8          "extensionsModules": "Modules/extensions",
9          "thirdPartyModules": "Modules/third_parties",
10         "distribution": "LocalDistribution",
11         "deploy": "DeployDistribution"
12     },
13     "modules": {
14         "extensions/BaseSassStylesExtension": "1.0.0",
15         "extensions/ProductDetailsExtension": "1.0.0",
16         "extensions/MyExampleCartExtension1": "1.0.0",
17         ...

```

- c. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.
Upon successful deployment, the HTML list (and) styles should display on your webstore.

HTML List Style Does Not Display (Theme)

i Applies to: SuiteCommerce | SuiteCommerce Advanced

The instructions in the section apply to you if you are implementing themes and you are implementing a theme created using the SuiteCommerce Base Theme (2018.1 release) as a baseline.

This topic explains how to correct the theme manually using the theme developer tools. Because you cannot patch a theme and you cannot make changes to a published theme, you have three options, depending on your implementation.

- If the theme exhibiting this behavior is a published theme and you want to make the changes yourself, you must create a new theme using the theme developer tools, applying the new code as described in this section.
- If the theme is owned by you, you can make changes to your theme using the theme developer tools as described in this section.
- If this is a published theme and you do not want to make this change manually, you must wait until the theme publisher provides an update or use a different theme.

This fix requires adding some code to the theme's existing Sass files. This step is the same if you own the theme or not. If you do not own the theme, the developer tools prompt you to create a new one when you deploy your changes.

You can download the code samples described in this procedure here: [HtmlListStyleNotDisplaying-Themes.zip](#)

i Note: Any themes created using the SuiteCommerce Base Theme (2018.2 release and later) already include this fix.

Step 1: Fetch the Active Theme

To make the change, the first step is to fetch the theme files from NetSuite and place them in your local developer environment. This step assumes that you have already set up your local developer environment and are using the applicable theme developer tools. See [Set Up the Developer Environment](#) for more details.

To fetch the theme:

1. Ensure that the theme exhibiting this behavior is activated on a domain associated with your site. See the help topic [Manage Themes and Extensions](#) for details.
2. Open a command line or terminal.
3. Access the top-level theme development directory you created when you downloaded the developer tools.
4. Enter the following command:

```
gulp theme:fetch
```

5. Follow the prompts to access your account, website, and the domain that currently has the theme activated.

Step 2: Customize the Theme

The next step is to customize your theme by editing two .scss files.

To customize the theme:

1. In your theme development source code, navigate to the following location:
`<THEME_DEV_TOOLS_DIRECTORY>/Workspace/<THEME_NAME>/Modules/BaseSassStyles@X.Y.Z/Sass/base`
 In this example, X.Y.Z represents the version of the BaseSassStyles module in your theme.
2. Open `_base.scss` and add the following lines to the end of file:

```
1 | .cms-content ul, .cms-content ol {
2 |     margin-left: $sc-margin-lv4;
3 | }
4 |
5 | .cms-content ul li {
6 |     list-style: circle;
7 | }
8 |
9 | .cms-content ol li {
10 |     list-style: decimal;
11 | }
```

3. Save the file.
4. Navigate to the following location:
`./Modules/ProductDetails@X.Y.Z/Sass`
 In this example, X.Y.Z represents the version of the ProductDetails module in your theme.
5. Open `_product-details-information.scss` and add the following lines to the end of file:

```
1 | .product-details-information-tab-content-container ul,
2 | .product-details-information-tab-content-container ol {
3 |     margin-left: $sc-margin-lv4;
4 | }
5 |
6 | .product-details-information-tab-content-container ul li {
7 |     list-style: outside;
8 | }
9 |
10 | .product-details-information-tab-content-container ol li {
11 |     list-style: decimal;
12 | }
```

6. Save the file.

Step 3: Deploy Your Changes

This step involves deploying your changes to a NetSuite account and activating on a domain. The steps are the same, regardless of the state of your theme, but the developer tools perform different actions, as described below:

- If you own the theme, the developer tools prompt you to include a new revision number. You can later update any bundles that include the theme for publication.

- If you do not own the theme, the developer tools prompt you to create a new one. This process results in a new theme that is based on the original, but includes your customizations. These changes do not affect the existing theme.



Important: After you deploy the theme, you must activate the update to apply your changes to a domain.

To deploy changes:

1. In your local developer environment, open a command line or terminal and access the top-level theme directory.
2. Run the following command:
`gulp theme:deploy`
3. Follow the prompts to confirm your account, website, and other information about your theme, as required.

If you are customizing a published theme, the developer tools prompt you to create a new theme.
4. After deployment is completed, activate the theme on a domain. See the help topic [Manage Themes and Extensions](#) for details.
5. Confirm your results.

Upon successful deployment, the HTML list (and) styles should display on your webstore.

For more information:

- To use theme developer tools, see [Develop Themes](#).
- To create bundles for a theme, see [Bundle Themes and Extensions as SuiteApps](#).

Item Search Displays Incorrect Results

Applies to: SuiteCommerce Advanced | Elbrus

In some implementations of the Elbrus release of SuiteCommerce Advanced, the application returns incorrect results when entering more than one term in the search criteria.

The following patch corrects a problem in `ItemsSearcher.View.js`, which is part of the **ItemsSearcher** module. To implement this patch, create a custom module to change the `defaultOptions.highlightFirst` property. You can download the code samples described in this procedure here: [ElbrusItemSearchResultsIncorrect.zip](#).



Note: Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Extend `ItemsSearcher.View.js`

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Within this directory, create a custom module with a name similar to the module being customized.
For example: `Modules/extensions/ItemsSearcherExtension@1.0.0`.
3. In your new `ItemsSearcherExtension@1.0.0` module, create a subdirectory called `JavaScript`.

For example: Modules/extensions/ItemsSearcherExtension/JavaScript

4. In your new JavaScript subdirectory, create a JavaScript file.
Name this file according to best practices. For example:
`ItemsSearcher.View.Extend.js`
5. Open this file and extend `ItemsSearcher.View.js` to customize the `highlightFirst` property from `true` to `false`, as shown in the following code snippet.

```

1  define(
2    'ItemsSearcher.View.Extend'
3    , [
4      'ItemsSearcher.View'
5    ]
6    , function (
7      ItemsSearcherView
8    )
9    {
10
11    'use strict';
12
13    ItemsSearcherView.prototype.defaultOptions.highlightFirst = false
14
15  });

```

6. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new `ItemsSearcherExtension@1.0.0` module directory.
2. Create a file in this directory and name it `ns.package.json`.
`Modules/extensions/ItemsSearcherExtension@1.0.0/ns.package.json`
3. Paste the following code into your new `ns.package.json` file:

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5      ]
6    }
7  }

```

4. Open the `distro.json` file. This is located in your root directory.
5. Add your custom module to the `modules` object.
Your code should look similar to the following example:

```

1  {
2    "name": "SuiteCommerce Advanced Elbrus",
3    "version": "2.0",
4    "buildToolsVersion": "1.3.0",
5    "folders": {
6      "modules": "Modules",
7      "suitecommerceModules": "Modules/suitecommerce",
8      "extensionsModules": "Modules/extensions",
9      "thirdPartyModules": "Modules/third_parties",
10     "distribution": "LocalDistribution",
11     "deploy": "DeployDistribution"
12   },
13   "modules": {
14     "extensions/ItemsSearcherExtension": "1.0.0",
15     "extensions/MyExampleCartExtension1": "1.0.0",
16     //...

```


This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Add `ItemsSearcher.View.Extend` as a dependency to SCA entry point within the `SC.Shopping.Starter` entrypoint of the JavaScript array.

Your `distro.js` file should look similar to the following:

```

1  "tasksConfig": {
2    //...
3  "javascript": [
4    //...
5    {
6      "entryPoint": "SC.Shopping.Starter",
7      "exportFile": "shopping.js",
8      "dependencies": [
9        //...
10       "StoreLocator",
11       "SC.CCT.Html",
12       "ItemsSearcher.View.Extend"
13     ],
14     //...

```

7. Save the `distro.json` file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, item search strings should display correct results.

Category Product Lists Return Page Not Found

i Applies to: SuiteCommerce Advanced | 2018.2

In some implementations of the 2018.2 release of SuiteCommerce Advanced, category product lists return Page Not Found when viewing a table or list.

The following patch corrects a problem in `Facets.Translator.js`, which is part of the **Facets** module. To implement this patch, create a custom module to extend the prototype object for the `FacetsTranslator` function. You can download the code samples described in this procedure here: [CategoryPageNotFound.zip](#).

i Note: Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Extend the `Facets.Translator.js` File

1. Create an **extensions** directory to store your custom module. Depending on your implementation, this directory might already exist.
2. Create a custom module with a name similar to the module being customized.
For example, `Modules/extensions/FacetsExtension@1.0.0`.
3. In your new `FacetsExtension@1.0.0` directory, create a subdirectory called `JavaScript`.

For example: Modules/extensions/FacetsExtension@1.0.0/JavaScript

4. In your new JavaScript subdirectory, create a JavaScript file to extend Facets.Translator.js.

Name this file according to best practices.

For example:

Facets.Translator.Extension.js

5. Open this file and extend the FacetsTranslator() method as shown in the following code snippet:

```

1  define('Facets.Translator.Extension'
2  , [ 'Facets.Translator'
3  , 'underscore'
4  , 'jQuery'
5  , 'SC.Configuration'
6  , 'Utils'
7  , 'UrlHelper'
8  ]
9  , function (
10     FacetsTranslator
11     , _
12     , jQuery
13     , Configuration
14 )
15 {
16     'use strict';
17
18     _.extend(FacetsTranslator.prototype,
19     {
20         FacetsTranslator (facets, options, configuration, category)
21         {
22             // Enforces new
23             if (!(this instanceof FacetsTranslator))
24             {
25                 return new FacetsTranslator(facets, options, configuration, category);
26             }
27
28             // Facets go Here
29             this.facets = [];
30
31             // Other options like page, view, etc. goes here
32             this.options = {};
33
34             // This is an object that must contain a fallbackUrl and a lists of facet configurations
35             this.configuration = configuration || default_config;
36
37             // Get the facets that are in the sitesettings but not in the config.
38             // These facets will get a default config (max, behavior, etc.) - Facets.Translator
39             // Include facet aliases to be considered as a possible route
40             var facets_data = Configuration.get('siteSettings.facetfield')
41             , facets_to_include = [];
42
43             _.each(facets_data, function(facet)
44             {
45                 if (facet.facetfieldid !== 'commercecategory')
46                 {
47                     facets_to_include.push(facet.facetfieldid);
48
49                     // If the facet has an urlcomponent defined, then add it to the possible values list.
50                     facet.urlcomponent && facets_to_include.push(facet.urlcomponent);
51
52                     // Finally, include URL Component Aliases...
53                     _.each(facet.urlcomponentaliases, function(facet_alias)
54                     {
55                         facets_to_include.push(facet_alias.urlcomponent);
56                     });
57                 }
58             });
59
60             facets_to_include = _.union(facets_to_include, _.pluck(Configuration.get('facets'), 'id'));
61             facets_to_include = _.uniq(facets_to_include);

```

```

62     this.facetsToInclude = facets_to_include;
63
64
65     this.isCategoryPage = !!category;
66
67     if (_.isBoolean(category) && category)
68     {
69         var index = facets.length
70         ,   facetsToInclude = this.facetsToInclude.slice(0)
71         ,   facets_delimiter = this.configuration.facetDelimiters.betweenDifferentFacets
72         ,   facet_value_delimiter = this.configuration.facetDelimiters.betweenFacetNameAndValue;
73
74         facetsToInclude.push(this.configuration.facetDelimiters.betweenFacetsAndOptions);
75
76         _.each(facetsToInclude, function(facetname)
77         {
78             facetname = facets_delimiter + facetname + facet_value_delimiter;
79             var i = facets.lastIndexOf(facetname);
80
81             if (i !== -1 && i < index)
82             {
83                 index = i;
84             }
85         });
86
87         var categoryUrl = facets.substring(0, index);
88
89         facets = facets.substring(index);
90
91         if (categoryUrl[0] !== '/')
92         {
93             categoryUrl = '/' + categoryUrl;
94         }
95
96         if (categoryUrl[categoryUrl.length - 1] === '/')
97         {
98             categoryUrl = categoryUrl.substring(0, categoryUrl.length - 1);
99         }
100
101         if (facets && facets[0] === facets_delimiter)
102         {
103             facets = facets.substring(1, facets.length);
104         }
105
106         this.categoryUrl = categoryUrl;
107     }
108     else if (_.isString(category))
109     {
110         this.categoryUrl = category;
111     }
112
113     // We cast on top of the passed in parameters.
114     if (facets && options)
115     {
116         this.facets = facets;
117         this.options = options;
118     }
119     else if (_.isString(facets))
120     {
121         // It's a url
122         this.parseUrl(facets);
123     }
124     else if (facets)
125     {
126         // It's an API option object
127         this.parseOptions(facets);
128     }
129 }
130 });
131 });

```

6. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new FacetsExtension@1.0.0 module directory.
2. Create a file in this directory and name it ns.package.json.
Modules/extensions/FacetsExtension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code:

```

1 {
2   "gulp": {
3     "javascript": [
4       "JavaScript/*.js"
5     ]
6   }
7 }

```

4. Open the distro.json file. This is located in your root directory.
5. Add your custom modules to the modules object.
Your code should look similar to the following example:

```

1 {
2   "name": "SuiteCommerce Advanced 2018.2.0 (sc-2018.2.0)",
3   "version": "2.0",
4   "isSCA": true,
5   "buildToolsVersion": "sc-2018.2.0",
6   "folders": {
7     "modules": "Modules",
8     "suitecommerceModules": "Modules/suitecommerce",
9     "extensionsModules": "Modules/extensions",
10    "thirdPartyModules": "Modules/third_parties",
11    "distribution": "LocalDistribution",
12    "deploy": "DeployDistribution"
13  },
14  "modules": {
15    "extensions/FacetsExtension": "1.0.0",
16    "suitecommerce/Account": "sc-2018.2.0",
17    //...

```

This step ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Add Facets.Translator.Extension as a dependency to SCA entry point within the SC.Shopping.Starter entrypoint of the JavaScript array.

Your distro.js file should look similar to:

```

1 "tasksConfig": {
2   //...
3   "javascript": [
4     //...
5     {
6       "entryPoint": "SC.Shopping.Starter",
7       "exportFile": "shopping.js",
8       "dependencies": [
9         //...
10        "Instrumentation.Cart",
11        "SiteSearch",
12        "Facets.Translator.Extension"
13      ],
14      //...

```

7. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.
Upon successful deployment, Commerce Categories filter based on the selected facet values.

Cannot Test an Extension on a Local Server

Applies to: SuiteCommerce Advanced | 2018.2

In some 2018.2 implementations of SuiteCommerce Advanced, developers might experience the following console error when testing an extension on a local server: `TypeError: SC.addExtensionModule is not a function`. The patch instructions described below correct this error.

To implement this patch, you must create two custom modules to override different instances of `index-local.ssp`. These files are located in the **Internals** directory of the following modules (where X.Y.Z represents the version of the module in your implementation):

- CheckoutApplication@X.Y.Z
- MyAccountApplication@X.Y.Z

You can download the code samples described in this procedure here: [RunExtensionsOnALocalServer.zip](#).

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file in a way that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the Override Files

1. Create an **extensions** directory to store each of your new custom modules. Depending on your implementation, this directory might already exist.

Create two custom modules and give them names similar to the modules being customized. For example, create the following files:

Modules/extensions/CheckoutApplicationExtension@1.0.0

Modules/extensions/MyAccountApplicationExtension@1.0.0

2. In each module, create a subdirectory called **Internal**.

For example:

■ Modules/extensions/CheckoutApplicationExtension@1.0.0/Internal

■ Modules/extensions/MyAccountApplicationExtension@1.0.0/Internal

3. Copy the following source files and paste into the correct location:

Copy This File:	Place a Copy Here:
Modules/suitecommerce/CheckoutApplication@X.Y.Z/Internal/index-local.ssp	Modules/extensions/CheckoutApplicationExtension@1.0.0/Internal
Modules/suitecommerce/MyAccountApplication@X.Y.Z/Internal/index-local.ssp	Modules/extensions/MyAccountApplicationExtension@1.0.0/Internal

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

4. For each new copy of index-local.ssp, add the following lines after the loadscript() function:

```
1 SC.extensionModules = [];
2 SC.addExtensionModule = function addExtensionModule(appModuleName) {
3     SC.extensionModules.push(appModuleName);
4 };
```

Each file should look similar to the following example:

```
1 //...
2 function loadScript (url)
3 {
4     'use strict';
5     var reference_tag = document.getElementsByTagName("script")[0];
6     var new_script_tag = document.createElement("script");
7     new_script_tag.src = url;
8     new_script_tag.type = "text/javascript";
9     new_script_tag.async = false;
10    reference_tag.parentNode.insertBefore(new_script_tag, reference_tag);
11 }
12
13 SC.extensionModules = [];
14 SC.addExtensionModule = function addExtensionModule(appModuleName) {
15     SC.extensionModules.push(appModuleName);
16 };
17 //...
```

5. Save each file.

Step 2: Prepare the Developer Tools For Your Customization

1. Create a file called ns.package.json in each of your custom modules.

For example:

- Modules/extensions/CheckoutApplicationExtension@1.0.0/ns.package.json
- Modules/extensions/MyAccountApplicationExtension@1.0.0/ns.package.json

2. Paste the following code in each ns.package.json file, as applicable:

CheckoutApplication

```
1 {
2     "gulp": {
3         "ssp-files": [
4             "Internal/*.ssp"
5         ]
6     },
7     "overrides": {
8         "suitecommerce/CheckoutApplication@X.Y.Z/Internal/index-local.ssp" : "Internal/index-local.ssp"
9     }
10 }
```

MyAccountApplication

```
1 {
2     "gulp": {
3         "ssp-files": [
4             "Internal/*.ssp"
```

```

5     }
6   },
7   "overrides": {
8     "suitecommerce/MyAccountApplication@X.Y.Z/Internal/index-local.ssp" : "Internal/index-local.ssp"
9   }
10 }

```



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

3. Open the `distro.json` file. This file is located in your root directory.
4. Add both custom modules to the `modules` object.

Your code should look similar to the following example:

```

1 {
2   "name": "SuiteCommerce Advanced 2018.2.0",
3   "version": "2.0",
4   "isSCA": true,
5   "buildToolsVersion": "1.3.1",
6   "folders": {
7     "modules": "Modules",
8     "suitecommerceModules": "Modules/suitecommerce",
9     "extensionsModules": "Modules/extensions",
10    "thirdPartyModules": "Modules/third_parties",
11    "distribution": "LocalDistribution",
12    "deploy": "DeployDistribution"
13  },
14  "modules": {
15    "extensions/CheckoutApplicationExtension": "1.0.0",
16    "extensions/MyAccountApplicationExtension": "1.0.0",
17    "suitecommerce/Account": "sc-2018.2.0",
18    ...

```

This step ensures that the Gulp tasks include your module when you deploy your code to NetSuite. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the `modules` object. The order of precedence in this list does not matter.

5. Save the `distro.json` file.

Step 3: Test and Deploy Your Customization

1. Deploy your changes to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.



Note: Because this patch modifies an SSP library file, changes are not visible in your local environment until you first deploy the customizations to NetSuite.

2. Confirm your results.

Upon successful deployment, you should be able to test your extensions on a local server.

Secure Shopping for Site Builder Implementations

Applies to: Site Builder | Vinson | pre-Denali

A secure shopping domain creates an encrypted connection between your web server and your customer's web browser. You can use an SSL certificate to secure the shopping portion of your web

store under an HTTPS domain on your Site Builder sites. HTTPS is currently the industry standard for ecommerce services and consumers prefer seeing the secure icon in their browser address bar.

Important: If setting up Secure Shopping on a Site Builder site, you must set up the shopping portion of your site to use an HTTPS domain. You cannot set up a single secure domain for Shopping and Checkout.

Some Site Builder implementations must be patched to take advantage of this functionality. Refer to the following table to determine if you need this patch:

Implementation	Instructions
Site Builder	No patch required. See the help topic Set Up a Domain for instructions on setting up Secure Shopping for your site.
- Site Builder Extensions Premium Kilimanjaro - Site Builder Extensions Kilimanjaro - Site Builder Extensions Premium Elbrus - Site Builder Extensions Elbrus	No patch required. See the help topic Set Up a Domain for instructions on setting up Secure Shopping for your site.
- Site Builder Extensions Premium Vinson - Site Builder Extensions Vinson	Patch required. See Secure Shopping for Site Builder Extensions (Vinson) for instructions.
- Reference Checkout 2.05 - Reference Checkout 2.04 - Reference My Account Premium 1.06 - Reference My Account 1.06 - Reference My Account Premium 1.05 - Reference My Account 1.05	Patch required. See Secure Shopping for Site Builder (Pre-Denali) for instructions.

Secure Shopping for Site Builder Extensions (Vinson)

Applies to: Site Builder

This topic explains how to patch the following implementations of Site Builder to take advantage of Secure Shopping domains:

- SiteBuilder Extensions Vinson
- SiteBuilder Extensions Premium Vinson

Note: Site Builder sites implementing the Kilimanjaro or Elbrus releases of Site Builder Extensions and Extensions Premium already include the ability to create a secure Shopping domain by default and do not require a patch. Denali and Mont Blanc Site Builder implementations cannot set up secure Shopping domains.

Note: To patch pre-Denali reference implementations for Site Builder, see [Secure Shopping for Site Builder \(Pre-Denali\)](#).

Follow these steps to set up a secure Shopping domain for your Site Builder Extensions implementation:

1. [Configure a Secure Shopping Domain](#)
2. [Apply Patch to Source Files](#)
3. [Deploy Files](#)

To leverage a secure shopping domain in your implementation of Site Builder Extensions, you need to introduce these changes as custom modules and deploy using the Core SCA developer tools. This patch is highly technical and involves customizing multiple files.

Configure a Secure Shopping Domain

Applies to: Site Builder

Before can use an SSL certificate to secure the shopping portion of your web store under an HTTPS domain, you must set up a secure shopping domain in NetSuite. See the help topic [Set Up a Domain](#) for more information.

When setting up a secure domain, be aware that sites implementing Site Builder can only secure the shopping portion of the web store under a separate HTTPS domain. Site Builder sites cannot implement a single secure domain for both Shopping and Checkout applications.

When you have configured your domain, continue with the next step: [Apply Patch to Source Files](#).

Apply Patch to Source Files

Applies to: Site Builder

Apply the changes in this patch using the Override method, as described in this section. The following table lists the files to be overridden and their locations in the Vinson source code, where X.Y.Z represents the version of the module in your implementation:

Module	Subdirectory	File
CheckoutApplication@X.Y.Z	/Internal	/index-local.ssp
	/JavaScript	/SC.Checkout.Configuration.js
	/SuiteScript	/checkout.environment.ssp
		/SuiteScript/checkout.ssp
CheckoutSkipLogin@X.Y.Z	/JavaScript	/CheckoutSkipLogin.js
GoogleUniversalAnalytics@X.Y.Z	/JavaScript	/GoogleUniversalAnalytics.js
LiveOrder@X.Y.Z	/SuiteScript	/LiveOrder.Model.js
MyAccountApplication@X.Y.Z	/Internal	/index-local.ssp
	/SuiteScript	/my_account.ssp
	/SuiteScript	/myaccount.environment.ssp
NavigationHelper@X.Y.Z	/JavaScript	/NavigationHelper.Plugins.DataTouchPoint.js
ProductList@X.Y.Z	/SuiteScript	/ProductList.Model.js
Profile@X.Y.Z	/SuiteScript	/ProductList.Model.js

Module	Subdirectory	File
		/Profile.Model.js
SiteSettings@X.Y.Z	/SuiteScript	/SiteSettings.Model.js
SspLibraries@X.Y.Z	/SuiteScript	/Application.js
		/ServiceController.Validations.js
		/Utils.js
Utilities@X.Y.Z	/JavaScript	/Utils.js
jQueryExtras@X.Y.Z	/JavaScript	/jQuery.ajaxSetup.js
	/JavaScript	/jQuery.ajaxSetup.noLoader.js

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify files in a way that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Create Override Files

The first step is to create a space to contain your customized files. You set up your new files to override the existing code when you deploy to NetSuite.

To create override files:

1. Open the Modules directory and create a subdirectory titled extensions to maintain your customizations.
2. In your extensions directory, create a new directory for each module according to the table above. Give these new modules a version of **1.0.0**.
For example:
Modules/extensions/CheckoutApplication@1.0.0/
Modules/extensions/CheckoutSkipLogin@1.0.0/
Modules/extensions/GoogleUniversalAnalytics@1.0.0/
...
3. In each new module directory, create the required subdirectories according to the table above. For example, the directory structure for your custom CheckoutApplication@1.0.0 module should look similar to the following:

```

1 Modules/extensions/CheckoutApplication@1.0.0
2   /Internal
3   /JavaScript
4   /SuiteScript

```

4. Locate the files in your current implementation to be overridden according to the table.
5. Make a copy of each file and paste the copy into the applicable subdirectory.
For example, your directory structure for your custom CheckoutApplication@1.0.0 module should look similar to the following:

```

1 Modules/extensions/CheckoutApplication@1.0.0/
2   Internal/
3     index-local.ssp
4   JavaScript/
5     SC.Checkout.Configuration.js
6   SuiteScript/
7     checkout.environment.ssp

```

6. Repeat this procedure for each file you need to override.

Implement the Patch Changes

The next step is to implement the patch in each new override file.

To implement the patch:

1. Download the .patch file associated with this patch guide here: [HTTPS_SiteBuilder-Vinson.zip](https://sitebuilder.vinson.com/patches/HTTPS_SiteBuilder-Vinson.zip).
This file includes all changes described in this procedure.
2. Open each new file within your extensions directory and add or remove lines (as applicable) according to the .patch file.
See the help topic [How to Apply .patch Files](#) for instructions on how to understand .patch files and apply changes. The following example describes one scenario.

Example

The following code snippet comes from the **Vinson Site Builder Extensions Premium** .patch file:

```

1 ...
2 diff --git a/Modules/suitecommerce/CheckoutApplication@2.2.0/JavaScript/SC.Checkout.Configuration.js b/Modules/suitecommerce/CheckoutApplication@2.2.0/JavaScript/SC.Checkout.Configuration.js
3 index 48cd87d..8dbc7eb 100644
4 --- a/Modules/suitecommerce/CheckoutApplication@2.2.0/JavaScript/SC.Checkout.Configuration.js
5 +++ b/Modules/suitecommerce/CheckoutApplication@2.2.0/JavaScript/SC.Checkout.Configuration.js
6 @@ -62,15 +62,15 @@ define(
7     ,*/
8
9
10 -     currentTouchpoint: _.isSecureDomain() ? 'checkout' : 'viewcart'
11 +     currentTouchpoint: Utils.isInShopping() ? 'viewcart' : 'checkout'
12
13     , modulesConfig: {
14       'ItemDetails': {startRouter: true}
15     }
16   );

```

In this example, these lines specify that the file you need to customize is **SC.Checkout.Configuration.js**:

```

1 diff --git a/Modules/suitecommerce/CheckoutApplication@2.2.0/JavaScript/SC.Checkout.Configuration.js b/Modules/suitecommerce/CheckoutApplication@2.2.0/JavaScript/SC.Checkout.Configuration.js

```

The next lines in the example provide a reference point to locate the code in the file that needs updated. Two sets of @@ symbols specify that the code to be customized is located on line 62 of the file and that 15 lines of code are displayed. The code after this reference is included as a visual reference to help locate the code affected by this change:

```

1 @@ -62,15 +62,15 @@ define(
2     ,*/

```



Important: The line reference (@@) specifies the line in the original source code. If you have made any preexisting modifications to the file prior to this patch, the code needing customized code may be located on a different line.

Any lines prepended with a - symbol indicate code to delete/remove as part of this patch. In this example, you remove the following line from SC.Checkout.Configuration.js:

```
1 | -    currentTouchpoint: _.isSecureDomain() ? 'checkout' : 'viewcart'
```

Lines prepended with a + symbol indicate code to add. In this example, you add the following line:

```
1 | +    currentTouchpoint: Utils.isInShopping() ? 'viewcart' : 'checkout'
```

Lines not prepended with a - or + symbol do not require change. See [How to Apply .patch Files](#) for more information on how to apply these changes manually.

Deploy Files

i Applies to: Site Builder

When all customizations are complete, you are ready to deploy your custom files to your Site Builder SSP Application in NetSuite. The following topics explain how to deploy custom files to the Custom implementation:

- [Prepare the Developer Tools For Your Customizations](#)
- [Deploy Your Customizations to NetSuite](#)



Important: The customizations introduced by this patch are extensive. Always test changes in a sandbox account or other test site before deploying to a production account.

Prepare the Developer Tools For Your Customizations

Before you deploy your changes, you must prepare your Vinson developer tools to include your customizations. To do this:

1. Create an ns.package.json file for each new module in your extensions subdirectory.
2. Edit the distro.json file.

To create an ns.package.json file for each new module:

1. Create a new file for each custom module in your extensions directory and title it: ns.package.json. You should end up with one unique ns.package.json file for each custom module.
2. Open each ns.package.json file and paste the following code:
The following example includes <placeholders> that you must change within each ns.package.json file.

```
1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
```

```

5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/<MODULE>@<X.Y.Z>/<SUBDIRECTORY>/<EXISTING_FILE>" : "<NEW_SUBDIRECTORY>/<NEW_FILE>.js"
9 |   }
10 | }

```

3. Edit the placeholders in the overrides object as described below:
 - <MODULE> represents the name of each module being changed.
 - <X.Y.Z> represents the module version in your implementation.
 - <SUBDIRECTORY> represents the subdirectory containing the original file you are overriding.
 - <EXISTING_FILE> represents the original file you are overriding.
 - <NEW_SUBDIRECTORY> represents the subdirectory according to the table in [Apply Patch to Source Files](#).
 - <NEW_FILE> represents the new file according to the table in [Apply Patch to Source Files](#).

4. Add the following as applicable:
 - Add an Internal, JavaScript, or SuiteScript array (as applicable) to the gulp object for each file type in the module.

The table

- Add a path to the overrides object for each file in the module.

For example, the ns.package.json file for your custom CheckoutApplication@1.0.0 module should look similar to the following:

```

1 | {
2 |   "gulp": {
3 |     "internal": [
4 |       "Internal/*.ssp"
5 |     ]
6 |   ,   "javascript": [
7 |     "JavaScript/*.js"
8 |   ]
9 |   ,   "ssp-libraries": [
10 |     "SuiteScript/*.ssp"
11 |     "SuiteScript/*.js"
12 |   ]
13 | }
14 |
15 | ,   "overrides": {
16 |     "suitecommerce/CheckoutApplication@2.2.0>/Internal/index-local.ssp" : "Internal/index-local.ssp"
17 |     ,   "suitecommerce/CheckoutApplication@2.2.0>/JavaScript/SC.Checkout.Configuration.js" : "JavaScript/SC.Checkout.Configuration.js"
18 |     ,   "suitecommerce/CheckoutApplication@2.2.0>/SuiteScript/checkout.environment.ssp" : "SuiteScript/checkout.environment.ssp"
19 |   }
20 | }
21 | }

```

5. Repeat this procedure for each module, following the same basic structure.

To edit the distro.json file:

1. Open the distro.json file. This is located in your root directory.
2. Add each custom module to the modules object.

Your code should look similar to the following example:

```

1 | {
2 |   "name": "SuiteCommerce Site Builder 2.0",
3 |   "version": "2.0",

```

```

4  "buildToolsVersion": "1.2.1",
5  "folders": {
6    "modules": "Modules",
7    "suitecommerceModules": "Modules/suitecommerce",
8    "thirdPartyModules": "Modules/third_parties",
9    "distribution": "LocalDistribution",
10   "deploy": "DeployDistribution"
11  },
12  "modules": {
13    "suitecommerce/CheckoutApplication": "1.0.0",
14    "suitecommerce/CheckoutSkipLogin": "1.0.0",
15    "suitecommerce/GoogleUniversalAnalytics": "1.0.0",
16    "suitecommerce/LiveOrder": "1.0.0",
17    "suitecommerce/MyAccountApplication": "1.0.0",
18    "suitecommerce/NavigationHelper": "1.0.0",
19    "suitecommerce/ProductList": "1.0.0",
20    "suitecommerce/Profile": "1.0.0",
21    "suitecommerce/SiteSettings": "1.0.0",
22    "suitecommerce/SsLibraries": "1.0.0",
23    "suitecommerce/Utilities": "1.0.0",
24    "suitecommerce/jqueryExtras": "1.0.0",
25    "suitecommerce/Account": "2.2.0",
26    //...

```

This ensures that the Gulp tasks include your modules when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

3. Save the distro.json file.

Deploy Your Customizations to NetSuite

The last step is to deploy your customizations.

To deploy your customizations:

1. Deploy your source code customizations to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)).
2. Confirm your results.

Upon successful deployment, you should be able to take advantage of secure shopping domains on your Site Builder site.

Secure Shopping for Site Builder (Pre-Denali)

Applies to: Site Builder

This topic explains how to patch the following implementations of Site Builder to take advantage of Secure Shopping domains:

- Site Builder Reference Checkout 2.05
- Site Builder Reference Checkout 2.04
- Site Builder Reference My Account 1.06
- Site Builder Reference My Account Premium 1.06
- Site Builder Reference My Account 1.05
- Site Builder Reference My Account Premium 1.05

Follow these steps to allow for secure Shopping domains on your Site Builder site:

1. [Configure a Secure Shopping Domain](#)
2. [Apply the Patch to Source Files](#)
3. [Deploy Files and Configure the Implementation](#)

Configure a Secure Shopping Domain

i Applies to: Site Builder

Before you can use an SSL certificate to secure the shopping portion of your web store under an HTTPS domain, you must set up a secure shopping domain in NetSuite. See the help topic [Set Up a Domain](#) for more information.

When setting up a secure domain for a site implementing Site Builder, be aware that sites implementing Site Builder can only implement SSL for the Shopping area of their site using a separate, secure shopping domain. Site Builder sites cannot implement a single secure domain for both Shopping and Checkout applications.

When you have configured your domain, continue with the next step: [Apply the Patch to Source Files](#).

Apply the Patch to Source Files

i Applies to: Site Builder

To leverage a secure shopping domain in your pre-Denali implementation of Site Builder, you need to introduce these changes as custom files and deploy to your Custom implementation.

This patch is highly technical and involves customizing multiple files. Before you begin making changes, back up any custom files that already exist. Each .zip file listed in the procedure below includes a .patch file for a specific implementation. Launch the .patch file and introduce changes manually.

For each file that needs updating, go to the SSP Application in NetSuite and copy the Reference files to the same location in the Custom folder:

Documents > Files > File Cabinet > Web Site Hosting Files > Live Hosting Files > SSP Applications

To implement the patch:

1. Download the correct patch .zip file, depending on you implementation.

These files include all changes described in this procedure:

Implementation	Download
Site Builder Reference Checkout 2.0.4 and 2.0.5	HTTPS_SiteBuilder-Reference-Checkout-2-0-4_and_2-0-5.zip
Site Builder Reference My Account and My Account Premium 1.0.5	HTTPS_SiteBuilder-Reference-My-Account.1-0-5.zip
Site Builder Reference My Account and My Account Premium 1.0.6	HTTPS_SiteBuilder-Reference-My-Account-1-0-6.zip

2. Extract the .zip file and open the .patch file you find there.
The .patch file describes each file and subsequent code to be customized.
3. If your implementation includes previous customizations, ensure that each file being patched is located in the custom folder and downloaded to your local environment.

4. If your implementation does not include any previous customizations, perform the following steps:
 - a. Copy the corresponding source file located in the Reference implementation in NetSuite.
If a preexisting file exists in the Custom implementation, then a customization has already been implemented on that file in the past. Back up this file and customize it instead.
 - b. Create a local copy for development.
 5. Access the root of your local development environment for this implementation.
 6. Manually modify your files as instructed in the patch files to add or remove lines, as required.
- See [How to Apply .patch Files](#) for instructions on how to understand .patch files and apply changes. The following example describes one scenario.

When you have successfully uploaded your customizations, continue with the next step: [Deploy Files and Configure the Implementation](#).

Patch Update Example

The following code snippet comes from the Site Builder Reference Checkout 2.0.4 .patch file:

```
1 | diff --git a/index-local.ssp b/index-local.ssp
2 | index 3f974de..ac20076 100644
3 | --- a/index-local.ssp
4 | +++ b/index-local.ssp
5 | @@ -30,11 +30,11 @@
6 |     login = true;
7 | }
8 | else if (
9 | -     (SiteSettings.registration.registrationoptional != 'T' && !session.isLoggedIn())
10 | -     || (!SC.Configuration.checkout_skip_login && !session.isLoggedIn())
11 | -     || (parameters.is && (parameters.is === 'login' || parameters.is === 'register') && !session.isLoggedIn())
12 | +     (SiteSettings.registration.registrationoptional != 'T' && !session.isLoggedIn2())
13 | +     || (!SC.Configuration.checkout_skip_login && !session.isLoggedIn2())
14 | +     || (parameters.is && (parameters.is === 'login' || parameters.is === 'register') && !session.isLoggedIn2())
15 | +     || (parameters.is && (parameters.is === 'login' || parameters.is === 'register') && session.getCustomer().isGuest())
16 | -     || (SC.Configuration.checkout_skip_login && !session.isLoggedIn() && session.isRecognized())
17 | +     || (SC.Configuration.checkout_skip_login && !session.isLoggedIn2() && session.isRecognized())
18 | )
19 | {
20 |     delete parameters.sitepath;
```

In this example, these lines specify that the file you need to customize is **index-local.ssp**:

```
1 | diff --git a/index-local.ssp b/index-local.ssp
```

The next lines in the example provide a reference point to locate the code in the file that needs updated. Two sets of @@ symbols specify that the code to be customized is located on line 30 of the file and that 11 lines of code are displayed. The code after this line reference provides a reference to help locate the area in the file affected by this change:

```
1 | @@ -30,11 +30,11 @@
2 |     login = true;
3 | }
4 | else if (
```



Important: The line reference (@@) specifies the line in the original source code. If you have made any preexisting modifications to the file prior to this patch, the code needing customized code may be located on a different line.

Any lines prepended with a - symbol indicate code to delete/remove as part of this patch. In this example, you remove the following lines from index-local.ssp:


```

1 -      (SiteSettings.registration.registrationoptional !== 'T' && !session.isLoggedIn())
2 -      || (!ISC.Configuration.checkout_skip_login && !session.isLoggedIn())
3 -      || (parameters.is && (parameters.is === 'login' || parameters.is === 'register') && !session.isLoggedIn())

```

Lines prepended with a + symbol indicate code to add:

```

1 +      (SiteSettings.registration.registrationoptional !== 'T' && !session.isLoggedIn2())
2 +      || (!ISC.Configuration.checkout_skip_login && !session.isLoggedIn2())
3 +      || (parameters.is && (parameters.is === 'login' || parameters.is === 'register') && !session.isLoggedIn2())

```

Lines not prepended with a - or + symbol do not require change. See [How to Apply .patch Files](#) for more information on how to apply these changes manually.

Deploy Files and Configure the Implementation

 **Applies to:** Site Builder

When all customizations are complete, you are ready to deploy your custom files to your Site Builder SSP Application in NetSuite. The following topics explain how to deploy custom files to the Custom implementation:

1. [Deploy Your Custom Files](#)
2. [Refresh Your Backend Environment](#)
3. [Configure the Models.js Script Path](#)



Important: The customizations introduced by this patch are extensive. Always test changes in a sandbox account or other test site before deploying to a production account.

Deploy Your Custom Files

The first task is to upload your changes to the correct location in the Custom implementation of your Site Builder SSP Application.

Site Builder SSP Applications each have a locked **Reference** implementation that contains all source files and an unlocked **Custom** implementation, where you place your customizations.

The Custom implementation directory (in the File Cabinet) is organized to match the structure of the Reference implementation. Upload your customized files to the Custom implementation in the location mirroring the original source location.

Upload your customizations as a .zip file using the Advanced Add feature of the File Cabinet. Assuming that the location of each customized file in the Custom implementation mirrors the location in Reference, your customizations overwrite the source when your site compiles for runtime.

For example, if you upload a custom version of index-local.ssp to a My Account Premium 1.05.0 implementation, the correct location for deployment is the root:

Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - My Account Premium 1.05 > Custom My Account Premium

To deploy customized files:

1. In your local developer environment, open your root implementation folder.

2. Select the entire contents of the folder and include all files within a local .zip file.



Important: Include the root folder contents only. Do not include the root folder itself.

3. In NetSuite, navigate to the SSP Application folder for your implementation.

For example:

Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - My Account Premium 1.05 > Custom My Account Premium

4. Add your zip file to the File Cabinet using the Advanced Add feature. See the help topic [Uploading Files to the File Cabinet](#) for details.

Refresh Your Backend Environment

The next task is to update two configuration files in the NetSuite backend. These two files are:

- Application.js
- Templates.js

These files must be automatically regenerated. You can force this action by editing two configuration files in your File Cabinet.

To reset your environment:

1. Complete the following steps for both combiner.config and templates.config.
2. In your NetSuite File Cabinet, navigate to Web Site Hosting Files > Live Hosting Files > SSP Applications > [APPLICATION] > [CUSTOM IMPLEMENTATION FOLDER] > js.
3. Click **Edit** next to combiner.config.
4. Locate the **Media Item** section and click **Edit** to preview the file.
5. Place your cursor at the very end of the code in this file.
6. Add a single space to the end of the code and click **Save**.
This change forces a background process to generate a new, concatenated Application.js file.
7. Confirm that a new Application.js file is generated.
The date in the **Last Modified** column should reflect an updated date and time when the process is complete.
8. Repeat this procedure for the templates.config file, located here:
Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. - My Account Premium 1.05 > Custom My Account Premium > templates.
This action updates the Templates.js file.

Configure the Models.js Script Path

The last task to complete is updating the Models.js library script path and touch points. This file is generated when the Models.js script runs. Before this happens, you must update the script path manually to point to the Custom folder.

To configure the Models.js script path:

1. Go to Commerce > Hosting > SSP Applications.

2. Click **Edit** to modify the SSP Application associated with your implementation.
3. On the **Library** subtab under **Scripts**, locate the following library script:
Web Site Hosting Files/Live Hosting Files/SSP Applications/[APPLICATION]/ssp_libraries/Models.js
4. Manually change this path to point to the Custom Files location:
Web Site Hosting Files/**Custom Files**/SSP Applications/[APPLICATION]/ssp_libraries/Models.js
In the above examples, [APPLICATION] equals the SSP Application of your current implementation.
5. Click **Save**.
6. Update touch points. See the help topic [Select Supported Touch Points](#)

Upon successful deployment, you should be able to take advantage of secure shopping domains on your Site Builder site.

Cannot Scroll Through Long Menu Lists Using iOS

Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus | Kilimanjaro

In some cases, iOS users might encounter an issue where they cannot scroll through a long list of menu items if the list reaches beyond the device screen. If you experience this issue on your SuiteCommerce Advanced site, perform the updates described in the section corresponding to your implementation:

- [Elbrus Release and Earlier – Menu List Scrolling Patch \(iOS\)](#)
- [Kilimanjaro – Menu List Scrolling Patch \(iOS\)](#)

In addition to making these changes, you must create an ns.package.json file and update your distro.json file for any custom modules you include.

Elbrus Release and Earlier – Menu List Scrolling Patch (iOS)

Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus

This section explains how to override the _mixins.scss to correct the iOS menu item scrolling issue for any Elbrus implementations of SuiteCommerce Advanced and earlier. Due to the structure of dependencies within these implementations, you cannot divide the entry point within the distro.json file. Therefore, you must override this file. You can download the code samples described in this procedure here: [MixinsSassOverride-ElbrusAndEarlierSample.zip](#).



Important: Be aware of any existing extensions or customizations to the existing file and familiarize yourself with the [Best Practices for Customizing SCA](#) before proceeding with any customizations.

Step 1: Override the _mixins.scss File

1. Create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your customizations.

Give this directory a unique name that is similar to the module being customized. For example:

Modules/extensions/MixinsSassOverride@1.0.0/

3. In your new module, create a subdirectory named Sass.
4. Copy the `_mixins.scss` file located in Modules > suitecommerce > BaseSassStyles@x.y.z and paste into your new Sass directory (where x.y.z represents the version of the module in your implementation of SuiteCommerce Advanced).

For example, paste your copy of this into the following location:

Modules/extensions/MixinsSassOverride@1.0.0/Sass/_mixins.scss

5. Open this file and remove the following line:

```
1 | -webkit-overflow-scrolling: touch;
```

When you are finished, your file should match the following code snippet:

```
1  /*
2  © 2017 NetSuite Inc.
3  User may not copy, modify, distribute, or re-bundle or otherwise make available this code;
4  provided, however, if you are an authorized user with a NetSuite account or log-in, you
5  may use this code subject to the terms that govern your access and use.
6  */
7
8  @mixin border-radius($radius) {
9    -webkit-border-radius: $radius;
10   -moz-border-radius: $radius;
11   -ms-border-radius: $radius;
12   border-radius: $radius;
13 }
14
15 %scroll-y{
16   overflow-y: auto;
17 }
18
19 // avoid using BS mixin cause wont generate "-webkit-transform"
20 @mixin transition-transform($duration...) {
21
22   -webkit-transition: -webkit-transform $duration;
23   -moz-transition: -moz-transform $duration;
24   -o-transition: -o-transform $duration;
25   transition: transform $duration;
26 }
27
28 @mixin appearance($appearance) {
29   -webkit-appearance: $appearance;
30   -moz-appearance: $appearance;
31   appearance: $appearance;
32 }
33
34 //used for select arrow down
35 @mixin angle-down-background($color){
36   $color-local: remove-hash-from-color($color);
37   background-image: url("data:image/svg+xml;utf8,<svg width='2000px' height='2000px' fill='%23#{ $color-local }'
38   xmlns='http://www.w3.org/2000/svg'><path d='M1075 352q0 -13 -10 -23l-50 -50q-10 -10 -23 -10t-23 10l-393 393l-393q-10
39   -10 -23 -10t-23 10l-50 50q-10 10 -10 23t10 23l466 466q10 10 23 10t23 -10l466 -466q10 -10 10 -23z' /></svg>");
40 }
41
42 @function remove-hash-from-color($color) {
43   @return str-slice(ie-hex-str($color), 4);
44 }
45
46 //All placeholders equal
47 @mixin placeholder {
48   ::-webkit-input-placeholder { @content; }
49   ::-moz-placeholder { @content; }
50   ::-moz-placeholder { @content; }
```

```
50 | :-ms-input-placeholder      {@content}
51 | }
```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Override

1. Open the MixinsSassOverride@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/MixinsSassOverride@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```
1 | {
2 |   "gulp": {
3 |     "sass": [
4 |       "Sass/**/*.scss"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/BaseSassStyles@2.0.0/Sass/_mixins.scss" : "Sass/_mixins.scss"
9 |   }
10 | }
```

Note: Replace the string x.y.z in the above example with the version of the module in your version of SuiteCommerce Advanced.

4. Save the ns.package.json file.
5. Open the distro.json file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.
6. Add your module to the modules object to ensure that the Gulp tasks include it when you deploy.
Your code should look similar to the following example:

```
1 | {
2 |   "name": "SuiteCommerce Advanced Elbrus",
3 |   "version": "2.0",
4 |   "buildToolsVersion": "1.3.0",
5 |   "folders": {
6 |     "modules": "Modules",
7 |     "suitecommerceModules": "Modules/suitecommerce",
8 |     "extensionsModules": "Modules/extensions",
9 |     "thirdPartyModules": "Modules/third_parties",
10 |     "distribution": "LocalDistribution",
11 |     "deploy": "DeployDistribution"
12 |   },
13 |   "modules": {
14 |     "extensions/MixinsSassOverride": "1.0.0",
15 |     "extensions/MyExampleCartExtension1": "1.0.0",
16 |     //...
```

7. Save the file.

Step 3: Deploy Your Override

1. Deploy your source code customizations to your NetSuite account and test the functionality using a phone running iOS. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.
2. Confirm your results.
Upon successful deployment, iOS mobile users should be able to scroll through long menu lists.

Kilimanjaro – Menu List Scrolling Patch (iOS)

Applies to: SuiteCommerce Advanced | Kilimanjaro

In your Kilimanjaro implementation of SuiteCommerce Advanced, you can extend the `_mixins.scss` file to overwrite the `%scroll-y` selector as described in this section. You can download the code samples described in this procedure here: [MixinsSassExtension-KilimanjaroSample.zip](#)

Step 1: Extend the `_mixins.scss` File

1. If you have not done so already, create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your customizations.
Give this directory a unique name that is similar to the module being customized. For example:
`Modules/extensions/MixinsSassExtension@1.0.0/`
3. In your new module, create a subdirectory named `Sass`.
4. In your `Sass` subdirectory, create a new `.scss` file.
Give this file a unique name that is similar to the file being modified. For example:
`Modules/extensions/mixinsSassExtension@1.0.0/Sass/_mixinsExtension.scss`
5. Open this file and overwrite the `%scroll-y` selector.
Your file should match the following code snippet:

```
1 | %scroll-y{
2 |     overflow-y: auto;
3 | }
```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Customizations

1. Open the `MixinsSassExtension@1.0.0` module.
2. Create a file in this module and name it **ns.package.json**.
`Modules/extensions/MixinSassExtension@1.0.0/ns.package.json`
3. Build the `ns.package.json` file using the following code

```
1 | {
2 |   "gulp": {
3 |     "sass": [
4 |       "Sass/**/*.scss"
5 |     ]
6 |   }
7 | }
```

4. Save the `ns.package.json` file.
5. Open the `distro.json` file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the `modules` object to ensure that the Gulp tasks include it when you deploy.

Your code should look similar to the following example:

```

1  {
2    "name": "SuiteCommerce Advanced Kilimanjaro",
3    "version": "2.0",
4    "isSCA": true,
5    "buildToolsVersion": "1.3.1",
6    "folders": {
7      "modules": "Modules",
8      "suitecommerceModules": "Modules/suitecommerce",
9      "extensionsModules": "Modules/extensions",
10     "thirdPartyModules": "Modules/third_parties",
11     "distribution": "LocalDistribution",
12     "deploy": "DeployDistribution"
13   },
14   "modules": {
15     "extensions/MixinsSassExtension": "1.0.0",
16     "extensions/MyExampleCartExtension1": "1.0.0",
17     //...

```

7. Split the `BaseSassStyle` entry in the `sass` object of each application (Shopping, MyAccount, and Checkout) and add your custom module (`MixinsSassExtension`) between them. This ensures that the Sass variables are loaded in the correct order.

Note: You do not need to wrap your custom module in an `include` array. The compiler adds all files within your new module by default.

```

1  //...
2  "sass": {
3    //...
4    "applications": [
5      {
6        "name": "Shopping",
7        "exportFile": "shopping.css",
8        "dependencies": [
9          //...
10         {
11           "module": "BaseSassStyles",
12           "include": [
13             "main-variables",
14             "main-base"
15           ]
16         },
17         "MixinsSassExtension",
18         {
19           "module": "BaseSassStyles",
20           "include": [
21             "main-atoms",
22             "main-molecules",
23             "molecules/datepicker"
24           ]
25         }
26       ],
27       //...
28     ]
29   },
30   {
31     "name": "MyAccount",
32     "exportFile": "myaccount.css",
33     "dependencies": [
34       //...
35       {
36         "module": "BaseSassStyles",
37         "include": [
38           "main-variables",

```

```

40         "main-base"
41     ],
42 },
43     "MixinsSassExtension",
44     {
45         "module": "BaseSassStyles",
46         "include": [
47             "main-atoms",
48             "main-molecules",
49             "molecules/datepicker"
50         ]
51     },
52     //...
53 ],
54 {
55     "name": "Checkout",
56     "exportFile": "checkout.css",
57     "dependencies": [
58         //...
59         {
60             "module": "BaseSassStyles",
61             "include": [
62                 "main-variables",
63                 "main-base"
64             ]
65         },
66     ],
67     "MixinsSassExtension",
68     {
69         "module": "BaseSassStyles",
70         "include": [
71             "main-atoms",
72             "main-molecules",
73             "molecules/datepicker"
74         ]
75     },
76     //...
77 ],
78 },
79 //...
80 ],
81 },
82 //...

```

8. Save the distro.json file.

Step 3: Deploy Your Extension

1. Deploy your source code customizations to your NetSuite account and test the functionality using a phone running iOS. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.
2. Confirm your results.

Upon successful deployment, iOS mobile users should be able to scroll through long menu lists.

Pages Not Indexed Using Google's Mobile-First Indexing

Applies to: SuiteCommerce Advanced | 2018.2 | Aconcagua | Kilimanjaro | Elbrus | Vinson | Mont Blanc | Denali

In some cases, search engine crawlers discard content from the page generator and do not load all the required resources to render pages correctly. This results in SuiteCommerce pages not rendering content. Applying this patch corrects this issue by maintaining the SEO pre-render result.

This patch uses the override method to replace `shopping.ssp`, located in the **ShoppingApplication** module. You can download the code samples described in this procedure here: [MobileFirstIndexingPatch.zip](#).

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify an SSP file, which requires that you use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the shopping.ssp Override File

1. Open the Modules/extensions directory and create a subdirectory to maintain your customizations and store your custom module.

Give this subdirectory a name similar to the module being customized. For example: Modules/extensions/ShoppingApplication.Extension@1.0.0.

Note: If you are implementing the Elbrus release of SCA or later, the **extensions** directory already exists in your source code. If implementing Vinson release or earlier, you must add it manually.

2. In your new ShoppingApplication.Extension@1.0.0 directory, create a subdirectory called SuiteScript.

For example: Modules/extensions/ShoppingApplication.Extension@1.0.0/SuiteScript

3. Find the following file in your existing source:

Modules/suitecommerce/ShoppingApplication@X.Y.Z/shopping.ssp

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

4. Copy this entire file and paste a copy in your new module's SuiteScript directory.

For example: Modules/extensions/ShoppingApplication.Extension@1.0.0/SuiteScript/shopping.ssp

5. Open your new copy of `shopping.ssp` and locate the following line:

```
1 | document.getElementById("main").innerHTML = '';
```

6. Replace this line with the following code:

```
1 | if (!navigator.userAgent.match(/googlebot/i))
2 | {
3 |     document.getElementById('main').innerHTML='';
4 | }
```

7. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your ShoppingApplication.Extension@1.0.0 module directory.
2. Create a file in this directory and name it `ns.package.json`.
For example: Modules/extensions/ShoppingApplication.Extension@1.0.0/ns.package.json
3. Build the `ns.package.json` file using the following code:

```
1 | {
```

```

2  "gulp": {
3    "ssp-files": {
4      "SuiteScript/shopping.ssp": "shopping.ssp"
5    }
6  },
7  "overrides": {
8    "suitecommerce/ShoppingApplication@X.Y.Z/SuiteScript/shopping.ssp" : "SuiteScript/shopping.ssp"
9  }
10 }

```

In your code, replace X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

4. Open the distro.json file. This is located in your root directory.
5. Add your custom module to the modules object.

Your code should look similar to the following example:

```

1  {
2    "name": "SuiteCommerce Advanced 2018.2.0 (sc-2018.2.0)",
3    "version": "2.0",
4    "isSCA": true,
5    "buildToolsVersion": "sc-2018.2.0",
6    "folders": {
7      "modules": "Modules",
8      "suitecommerceModules": "Modules/suitecommerce",
9      "extensionsModules": "Modules/extensions",
10     "thirdPartyModules": "Modules/third_parties",
11     "distribution": "LocalDistribution",
12     "deploy": "DeployDistribution"
13   },
14   "modules": {
15     "extensions/ShoppingApplication.Extension": "1.0.0",
16     "suitecommerce/Account": "sc-2018.2.0",
17     ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations by deploying them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)).

Note: Since this patch modifies a SuiteScript file, changes are not visible in your local environment. SuiteScript files run on the server and must be deployed to NetSuite to take effect.

2. Confirm your results.

Disabling Display of SuiteCommerce Gift Wrap & Message Extension Transaction Line Fields

Applies to: SuiteCommerce Advanced

After installing the SuiteCommerce Gift Wrap & Message extension, the application adds the following transaction line item fields to all items in your account:

- Gift Wrap Item
- Gift Wrap Item ID
- Gift Wrap Message From
- Gift Wrap Message To
- Gift Wrap Message Text

After installing the extension, these added fields may display on your site, even if you do not activate the extension. Follow these instructions, depending on your implementation, to prevent these fields from displaying on your site:

- [SuiteCommerce and SuiteCommerce Advanced Aconcagua and Later](#)
- [SuiteCommerce Advanced Denali through Kilimanjaro](#)
- [SuiteCommerce Advanced Pre-Denali](#)
- [Site Builder Websites](#)

SuiteCommerce and SuiteCommerce Advanced Aconcagua and Later

Follow the steps below to disable the display of SuiteCommerce Gift Wrap & Message extension transaction line item fields on product display pages.

Step 1: Activate the SuiteCommerce Gift Wrap & Message Extension.

1. If you have not done so already, install the SuiteCommerce Gift Wrap & Message extension into your NetSuite account. See the help topic [Installing Theme and Extension SuiteApps](#).
2. Activate the SuiteCommerce Gift Wrap & Message extension. See the help topic [Manage Themes and Extensions](#).

Step 2: Update the Configuration Record for the Web Site and Domain

1. In NetSuite, go to Commerce > Websites> Configuration.
2. Select the web site and domain.
3. Click **Configure**.
4. Navigate to the **My Extensions** tab.
5. On the **Gift Wrap & Message** subtab, clear the **Enable Gift Wrap** box.
6. Click **Save**.

Step 3: Clear the Domain Cache and Confirm Results

1. Go to Commerce > Content Management > Cache Invalidation Requests.
2. Click **New Invalidation Request**.
3. Double-click on the domain for which you want to clear the cache.
4. Enable the **Clear cache for the whole domain(s)** option.
5. Click **Submit**.
6. Click **OK**.
7. Confirm your results.

After a successful configuration, your site will not display the gift wrap transaction line item fields.

SuiteCommerce Advanced Denali through Kilimanjaro

This patch disables the display of the SuiteCommerce Gift Wrap & Message extension transaction line item fields on product display pages.

You can download the code samples described in this procedure here:

[GiftWrap.ItemOptionsExtension@1.0.0—KilimanjaroSample.zip](#)

Step 1: Create the Directory Structure to Store Your Custom Module

1. Open the Modules/extensions directory and create a subdirectory to maintain your customizations.

Note: If you are implementing the Vinson release of SuiteCommerce Advanced or earlier, you might need to create the extensions directory manually.

2. In the extensions directory, create a subdirectory called GiftWrap.ItemOptionsExtension@1.0.0.

When creating a new custom module, the module name must be unique. You must not have duplicate module names, even if those modules reside in different folders.

3. In your new module, create a subdirectory called Sass.

4. In your new Sass subdirectory, create a text file.

Give this file a unique name that is similar to the module directory. For example:

`_giftWrap-item-options-extension.scss`

Step 2: Create a Sass File

1. In the Sass directory, create a new file called `_giftWrap-item-options-extension.scss`.
2. Add the following text to the `_giftWrap-item-options-extension.scss` file:

```
1 [id*='custcol_ns_sc_ext_gw_item'],
2 [name*='custcol_ns_sc_ext_gw_item'],
3 [id*='custcol_ns_sc_ext_gw_id'],
4 [name*='custcol_ns_sc_ext_gw_id'],
5 [id*='custcol_ns_sc_ext_gw_message_from'],
6 [name*='custcol_ns_sc_ext_gw_message_from'],
7 [id*='custcol_ns_sc_ext_gw_message_to'],
8 [name*='custcol_ns_sc_ext_gw_message_to'],
9 [id*='custcol_ns_sc_ext_gw_message_text'],
10 [name*='custcol_ns_sc_ext_gw_message_text'] {
11     display: none;
12 }
```

Step 3: Create the ns.package.json File

1. Open the GiftWrap.ItemOptionsExtension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/GiftWrap.ItemOptionsExtension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code:

```
1 {
2   "gulp": {
3     "sass": [
4       "Sass/**/*.scss"
5     ]
6   }
7 }
```

4. Save the ns.package.json file.

Step 4: Update the distro.json File and Deploy Your Customizations to Your NetSuite Account

1. Open the distro.json file.

This file is located in the top-level directory of your SuiteCommerce source code.

2. Add your custom module to the modules object to ensure that the Gulp tasks include it when you deploy.

In this example, the `extensions/GiftWrap.ItemOptionsExtension@1.0.0` module is added at the beginning of the list of the other extensions modules. However, you can add the module anywhere in the modules object. The order of precedence in this list does not matter.

```

1  {
2    "name": "SuiteCommerce Advanced Kilimanjaro",
3    "version": "2.0",
4    "isSCA": true,
5    "buildToolsVersion": "1.3.1",
6    "folders": {
7      "modules": "Modules",
8      "suitecommerceModules": "Modules/suitecommerce",
9      "extensionsModules": "Modules/extensions",
10     "thirdPartyModules": "Modules/third_parties",
11     "distribution": "LocalDistribution",
12     "deploy": "DeployDistribution"
13   },
14   "modules": {
15     "extensions/GiftWrap.ItemOptionsExtension": "1.0.0",
16     "extensions/QuoteToSalesOrderExtension": "1.0.0",
17     "extensions/TransactionExtension": "1.0.0",
18     "extensions/OrderWizardPaymentMethodExtension": "1.0.0",
19     ...

```

3. Include the module definition (`GiftWrap.ItemOptionsExtension`) in the dependencies/applications array of the sass object within applications/dependencies/ for each application (Shopping, MyAccount, and Checkout). You must include this file after the module you are overriding. Your distro.json file should look similar to the following:

```

1  //...
2  "tasksConfig": {
3    //...
4    "sass": {
5      //...
6      "applications": [
7        {
8          "name": "Shopping",
9          "exportFile": "shopping.css",
10         "dependencies": [
11           //...
12           "Location.SCA",
13           "GiftWrap.ItemOptionsExtension"
14         ]
15       }
16     //...
17     {
18       "name": "MyAccount",
19       "exportFile": "myaccount.css",
20       "dependencies": [
21         //...
22         "Location.SCA",
23         "GiftWrap.ItemOptionsExtension"
24       ]
25     }
26     //...
27     {
28       "name": "Checkout",
29       "exportFile": "checkout.css",
30       "dependencies": [

```

```

31 //...
32                                     "Location.SCA",
33                                     "GiftWrap.ItemOptionsExtension"
34                                 ]
35                             }
36                         ]
37                     }
38 //...

```

4. Save the distro.json file.
5. Deploy your source code customizations to your NetSuite account. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.

Step 5: Clear Your Domain Cache and Confirm Results

1. Go to Commerce > Content Management > Cache Invalidation Requests.
2. Click **New Invalidation Request**.
3. Double-click on the domain for which you want to clear the cache.
4. Enable the **Clear cache for the whole domain(s)** option.
5. Click **Submit**.
6. Click **OK**.
7. Confirm your results.

After a successful deployment, your site will not display the gift wrap transaction line item fields.

SuiteCommerce Advanced Pre-Denali

Follow the steps below to disable the display of SuiteCommerce Gift Wrap & Message extension transaction line item fields on product display pages.

Step 1: Add code to your application stylesheet

1. Open any of the SuiteCommerce Advanced instance stylesheets located in your local development files. For example:
Custom ShopFlow 1.07.0/skins/standard/styles.css
2. Paste the following code at the bottom of the file:

```

1  [id*='custcol_ns_sc_ext_gw_item'],
2  [name*='custcol_ns_sc_ext_gw_item'],
3  [id*='custcol_ns_sc_ext_gw_id'],
4  [name*='custcol_ns_sc_ext_gw_id'],
5  [id*='custcol_ns_sc_ext_gw_message_from'],
6  [name*='custcol_ns_sc_ext_gw_message_from'],
7  [id*='custcol_ns_sc_ext_gw_message_to'],
8  [name*='custcol_ns_sc_ext_gw_message_to'],
9  [id*='custcol_ns_sc_ext_gw_message_text'],
10 [name*='custcol_ns_sc_ext_gw_message_text'] {
11     display: none;
12 }

```

Step 2: Upload the stylesheet to the File Cabinet and Compile the Change

1. Upload the file to the File Cabinet in the appropriate location. Example:
CustomShopFlow 1.07.0/skins/standard/styles.css
2. Trigger the style's combiner.config file to compile the change by editing and saving the styles.css file without making any changes.

Step 3: Clear Your Domain Cache and Confirm Results

1. Go to Commerce > Content Management > Cache Invalidation Requests.
2. Click **New Invalidation Request**.
3. Double-click on the domain for which you want to clear the cache.
4. Enable the **Clear cache for the whole domain(s)** option.
5. Click **Submit**.
6. Click **OK**.
7. Confirm your results.

After a successful deployment, your site will not display the gift wrap transaction line item fields.

Site Builder Websites

For Site Builder web sites, you only need to add some code to your Web Site Setup record and save the changes. After a successful configuration, your site no longer displays the gift wrap transaction line item fields.

Adding Code to Your Web Site Setup Record

1. In NetSuite, go to Commerce > Websites > Website List.
2. Click **Edit** for the website you want to patch.
3. Go to the **Analytics** subtab and paste the following code into the **Additional to <head>** field:

```

1 <style type="text/css">
2   [id*='custcol_ns_sc_ext_gw_item'],
3   [name*='custcol_ns_sc_ext_gw_item'],
4   [id*='custcol_ns_sc_ext_gw_id'],
5   [name*='custcol_ns_sc_ext_gw_id'],
6   [id*='custcol_ns_sc_ext_gw_message_from'],
7   [name*='custcol_ns_sc_ext_gw_message_from'],
8   [id*='custcol_ns_sc_ext_gw_message_to'],
9   [name*='custcol_ns_sc_ext_gw_message_to'],
10  [id*='custcol_ns_sc_ext_gw_message_text'],
11  [name*='custcol_ns_sc_ext_gw_message_text'] {
12    display: none;
13  }
14 </style>

```

4. Click **Save**.

Users Not Redirected to External Payment System

Applies to: SuiteCommerce Advanced | Vinson

In Vinson releases of SCA, if a user chooses an external payment method during checkout, SuiteCommerce Advanced does not correctly open the third-party payment system.

When a user clicks the **Continue to External Payment** button during checkout, instead of sending the user to the external payment system, SCA redirects the user to the beginning of the checkout process.

This error occurs because SuiteCommerce Advanced deprecated the existing payment processing status result, HOLD, in 17.1. Instead, SCA uses a new status result, PENDING. As a result, any SCA sites implementing the Vinson release requires this patch to use third-party payment systems.

To implement this patch, you must extend the prototype object for the `ExternalPaymentModel` function that includes the `_isDone()`, `_validateRecordStatus()`, and `_validateStatusFromRequest()` methods. This function is located in `ExternalPayment.Model.js` for the **ExternalPayment** module. You can download the code samples described in this procedure here: [UsersNotRedirectedExternalPaymentSystem.zip](#).

Step 1: Extend the ExternalPayment.Model.js File

1. If you have not done so already, create a directory to store your custom module.
Give this directory a name similar to the module being customized. For example, create `Modules/extensions/ExternalPaymentExtension@1.0.0`
2. In your new `ExternalPaymentExtension@1.0.0` directory, create a subdirectory called `JavaScript`.
For example: `Modules/extensions/ExternalPaymentExtension@1.0.0/SuiteScript`
3. In your new `JavaScript` subdirectory, create a JavaScript file to extend `ExternalPayment.Model.js`.
Name this file according to best practices. For example:
`ExternalPayment.Model.Extension.js`
4. Open this file and extend the `ExternalPaymentModel()` function as shown in the following code snippet.

```

1  define('ExternalPayment.Model.Extension'
2  , [
3    'ExternalPayment.Model'
4    , 'underscore'
5  ]
6  , function (
7    ExternalPaymentModel
8    , _
9  )
10 {
11   'use strict';
12
13   _extend(ExternalPaymentModel.prototype,
14   {
15     _isDone: function (request, record_type)
16     {
17       var status = this._getStatusFromRequest(request, record_type)
18       , status_accept_value = this._getConfiguration(record_type, 'statusAcceptValue', 'ACCEPT')
19       , status_hold_value = this._getConfiguration(record_type, 'statusHoldValue', 'HOLD')
20
21       //Added by payment
22       , status_pending_value = this._getConfiguration(record_type, 'statusPendingValue', 'PENDING')
23       return status === status_accept_value || status === status_hold_value || status === status_pending_value;
24     }
25
26     , _validateRecordStatus: function (record)
27     {
28       //Modified by Payment
29       return record.getFieldValue('paymenteventholdreason') === 'FORWARD_REQUESTED';
30     }
31
32     , _validateStatusFromRequest: function (request, record_type)
33     {
34       var status = this._getStatusFromRequest(request, record_type)
35       , status_accept_value = this._getConfiguration(record_type, 'statusAcceptValue', 'ACCEPT')
36       , status_hold_value = this._getConfiguration(record_type, 'statusHoldValue', 'HOLD')
37
38       //Added by Payment
39       , status_pending_value = this._getConfiguration(record_type, 'statusPendingValue', 'PENDING')
40       , status_reject_value = this._getConfiguration(record_type, 'statusRejectValue', 'REJECT');
41
42       //Modified by Payment
43       return status === status_accept_value || status === status_pending_value || status === status_hold_value || status
44       === status_reject_value;
45     }
46   }
47 }
```



```

45 | });
46 | });

```

5. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new ExternalPaymentExtension@1.0.0 module directory.
2. Create a file in this directory and name it ns.package.json.
Modules/extensions/ExternalPaymentExtension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code:

```

1 | {
2 |   "gulp": {
3 |     "ssp-libraries": [
4 |       "SuiteScript/*.js"
5 |     ]
6 |   }
7 | }

```

4. Open the distro.json file. This is located in your root directory.
5. Add your custom modules to the modules object.

Your code should look similar to the following example:

```

1 | {
2 |   "name": "SuiteCommerce Advanced Vinson Release",
3 |   "version": "2.0",
4 |   "buildToolsVersion": "1.2.1",
5 |   "folders": {
6 |     "modules": "Modules",
7 |     "suitecommerceModules": "Modules/suitecommerce",
8 |     "thirdPartyModules": "Modules/third_parties",
9 |     "distribution": "LocalDistribution",
10 |    "deploy": "DeployDistribution"
11 |  },
12 |   "modules": {
13 |     "extensions/ExternalPaymentExtension": "1.0.0",
14 |     "suitecommerce/Account": "2.2.0",
15 |     ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Add ExternalPaymentExtension as a dependency to SCA entry point within the SC.Checkout.Starter entripoint of the JavaScript array.

Your distro.json file should look similar to the following:

```

1 | "tasksConfig": {
2 |   // ...
3 |   "javascript": [
4 |     // ...
5 |     {
6 |       "entryPoint": "SC.Checkout.Starter",
7 |       "exportFile": "checkout.js",
8 |       "dependencies": [
9 |         "ExternalPaymentExtension",
10 |        "Backbone.View.Plugins",
11 |        "jQuery.html",
12 |        // ...
13 |      ],
14 |    }

```

7. Add your custom module to the ssp-libraries dependencies.

Your code should look similar to the following example:

```

1  "ssp-libraries": {
2      "entryPoint": "SCA",
3      "dependencies": [
4          "ExternalPaymentExtension",
5          "Application",
6          "Account.ForgotPassword.ServiceController",
7          // ...
8      ],

```

8. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Deploy your source code customizations to your NetSuite account and test the functionality. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.

Note: Since this patch modifies a SuiteScript file, changes are not visible in your local environment. SuiteScript files run on the server and must be deployed to NetSuite to take effect.

2. Confirm your results.

Upon successful deployment, the checkout process redirects to an external payment system when you click **Continue to External Payment**.

Invoices Do Not Include the Request for a Return Button

Applies to: SuiteCommerce Advanced | Vinson

In Reference My Account v1.06, invoices do not include the **Request for a Return** button. As a result, a customer cannot request to return items on an invoice in NetSuite. However, this button does appear on invoices for Reference My Account Premium v1.06.

To add this button, this patch overrides invoice_details.tpl, Invoice.Router.js, and Invoice.Details.View.js in the **Invoice** module. You can download the code samples described in this procedure here: [RequestForAReturnButtonOnInvoices.zip](#)

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify files that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1. Override the JavaScript and Template Files

1. If you have not done so already, create a directory to store your custom module.

Give this directory a name similar to the module being customized. For example, create Modules/extensions/Invoice.Extension@1.0.0

2. In your new Invoice.Extension@1.0.0 directory, create subdirectories called JavaScript and Templates.

For example: Modules/extensions/Invoice.Extension@1.0.0/JavaScript and Modules/extensions/Invoice.Extension@1.0.0/Templates.

3. Copy the following files:

```
Modules/suitecommerce/Invoice@X.Y.Z/JavaScript/Invoice.Router.js
Modules/suitecommerce/Invoice@X.Y.Z/JavaScript/Invoice.Details.View.js
Modules/suitecommerce/Invoice@X.Y.Z/Templates/invoice_details.tpl
```

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

4. Paste a copy in your new module's JavaScript and Templates directories.

For example: Modules/extensions/Invoice.Extension@1.0.0/JavaScript/Invoice.Router.js or Modules/extensions/Invoice.Extension@1.0.0/Templates/invoice_details.tpl

5. Open your new copy of Invoice.Router.js and locate the showInvoice() method.

Replace the existing showInvoice() method with the following code:

```
1 showInvoice: function (invoice_id, referer)
2 {
3   var invoice = new InvoiceModel({internalid: invoice_id})
4   , view = new DetailsView({
5     application: this.application
6     , model: invoice
7     , referer: referer
8   });
9   invoice
10    .fetch({
11      killerId: AjaxRequestsKiller.getKillerId()
12    })
13    .done(function() {
14      view.isReturnable()
15        .done(function() {
16          view.showContent();
17        })
18    });
19 }
```

6. Save the file.

7. Open your new copy of Invoice.Details.View.js and make the following changes.

- a. Find the following line:

```
1 | , 'underscore'
```

Replace this line with the following code:

```
1 | , 'underscore'
2 | , 'ReturnAuthorization.GetReturnableLines'
3 | , 'OrderHistory.Model'
4 | , 'AjaxRequestsKiller'
5 | , 'jQuery'
```

- b. Find the following line:

```
1 | , _
```

Replace this line with the following code:

```
1 | , _
2 | , ReturnLinesCalculator
3 | , OrderHistoryModel
4 | , AjaxRequestsKiller
5 | , JQuery
```

- c. Find the following lines:

```
1 | , makeAPayment: function ()
2 | {
3 |     LivePaymentModel.getInstance().selectInvoice(this.model.id);
4 | }
```

Replace these lines with the following code:

```
1 | , makeAPayment: function ()
2 | {
3 |     LivePaymentModel.getInstance().selectInvoice(this.model.id);
4 | }
5 |
6 | , makeAPayment: function ()
7 | {
8 |     LivePaymentModel.getInstance().selectInvoice(this.model.id);
9 | }
10 | //method isReturnable calculates a value based on the fulfillments and returns made,
11 | // if the invoice accepts returns
12 | , isReturnable: function ()
13 | {
14 |     var deferred = JQuery.Deferred();
15 |
16 |     var self = this;
17 |     var model = new OrderHistoryModel();
18 |     model
19 |     .fetch({
20 |         data: {
21 |             internalid: this.model.get('createdfrom').internalid
22 |             , recordtype: this.model.get('createdfrom').recordtype
23 |         }
24 |         , killerId: AjaxRequestsKiller.getKillerId()
25 |     })
26 |     .done(function() {
27 |         var ret_calculator = new ReturnLinesCalculator(model, {notConsiderFulfillments: true});
28 |         self.validLines = ret_calculator.calculateLines().validLines.length;
29 |         deferred.resolve();
30 |     });
31 |
32 |     return deferred.promise();
33 | }
```

- d. Find the following line:

```
1 | , showOpenedAccordion: _.isTabletDevice() || _.isDesktopDevice()
```

Replace this line with the following code:

```
1 | , showOpenedAccordion: _.isTabletDevice() || _.isDesktopDevice()
2 |     // @property {Boolean} showRequestReturnButton
3 | , showRequestReturnButton: !!this.validLines
```

- e. Find the following line inside the initialize() function:

```
1 | this.navSource = options.referer === 'paidinvoices' ? '/paid-invoices' : '/invoices';
```

Replace this line with the following code:

```
1 | this.navSource = options.referer === 'paidinvoices' ? '/paid-invoices' : '/invoices';
2 | this.model = options.model;
3 | this.validLines = 0;
```

8. Save the file.
9. Open your new copy of invoice_details.tpl and make the following change.

Find the following lines:

```
1 <a target="_blank" class="invoice-details-button-download-as-pdf" href="{{downloadPdfUrl}}">
2   {{translate 'Download as PDF'}}
3 </a>
```

Replace these lines with the following code:

```
1 <a target="_blank" class="invoice-details-button-download-as-pdf" href="{{downloadPdfUrl}}">
2   {{translate 'Download as PDF'}}
3 </a>
4
5 {{#if showRequestReturnButton}}
6
7 <a href="/returns/new/invoice/{{model.internalid}}" class="invoice-details-button-request" data-permissions="transaction
8   s.tranRtnAuth.2">
9   {{translate 'Request a Return'}}
10 </a>
11 {{/if}}
```

10. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your Modules/extensions/ module directory.
2. Create a file in this directory and name it ns.package.json.

Modules/extensions/Invoice.Extension@1.0.0/ns.package.json

3. Build the ns.package.json file using the following code:

```
1 {
2   "gulp": {
3     "javascript": [
4       "JavaScript/*.js"
5     ]
6   },
7   "overrides": {
8     "suitecommerce/Invoice@X.Y.Z/Javascript/Invoice.Router.js" : "JavaScript/Invoice.Router.js",
9     "suitecommerce/Invoice@X.Y.Z/Javascript/Invoice.Details.View.js" : "JavaScript/Invoice.Details.View.js",
10    "suitecommerce/Invoice@X.Y.Z/Templates/invoice_details.tpl" : "Templates/invoice_details.tpl"
11  }
12 }
```



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

4. Open the distro.json file. This is located in your root directory.
5. Add your custom modules to the modules object.

Your code should look similar to the following example:

```
1 {
2   "name": "SuiteCommerce Advanced Vinson Release",
3   "version": "2.0",
4   "buildToolsVersion": "1.2.1",
5   "folders": {
6     "modules": "Modules",
7     "suitecommerceModules": "Modules/suitecommerce",
8     "thirdPartyModules": "Modules/third_parties",
9     "distribution": "LocalDistribution",
10    "deploy": "DeployDistribution"
11  }
```

```

11     },
12     "modules": {
13         "extensions/Invoice.Extension": "1.0.0",
14         "suitecommerce/Account": "2.2.0", ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

Upon successful deployment, Invoice records under My Account contain a **Request a Return** button .

Unable to Select Item Options Within the SuiteCommerce Configuration Record in NetSuite

Applies to: SuiteCommerce Advanced | Elbrus

In some implementations of Elbrus, site administrators cannot choose item options when configuring a domain using the SuiteCommerce Configuration record in NetSuite (**Shopping Catalog** tab, **Item Options** subtab). This can occur using Google Chrome with Mac OS.

This patch extends the ProductDetails.Base.View.js file in the **ProductDetails** module to overwrite the optionBindEventByType object. This code change ensures that site administrators can select item options in the SuiteCommerce Configuration record without error and then specify a default template to render the selected item option.

Step 1: Extend the ProductDetails.Base.View.js File

1. Open the Modules/extensions directory and create a subdirectory to maintain your customizations. Give this directory a name similar to the module being customized.
For example: Modules/extensions/ProductDetails.Extension@1.0.0
2. In your new ProductDetails.Extension@1.0.0 module, create a subdirectory called JavaScript.
3. In your new JavaScript subdirectory, create a JavaScript file to extend the JavaScript in ProductDetails.Base.View.js.
Name this file according to best practices.
For example: Modules/extensions/ProductDetails.Extension@1.0.0/JavaScript/ProductDetails.Base.View.Extension.js
4. Open this file and extend the prototype of ProductDetailsBaseView with the optionBindEventByType object as shown in the following code snippet.

```

1  define(
2    'ProductDetails.Base.View.Extension'
3    , ['ProductDetails.Base.View'
4      , 'jQuery'
5      , 'underscore'
6    ]
7    , function (
8      ProductDetailsBaseView
9      , jQuery
10     , -
11   )
12   {
13     'use strict';
14
15     _._extend(ProductDetailsBaseView.prototype, {optionBindEventByType: {
16       'select': 'change'
17     , 'text': 'blur'
18     , 'date': 'change'
19     }
20   });

```

5. Save the file.

Step 2: Prepare the Developer Tools for Your Customization

1. Open the ProductDetails.Extension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.

For example: Modules/extensions/ProductDetails.Extension@1.0.0/ns.package.json

3. Build the ns.package.json file using the following code

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5      ]
6    }
7  }

```

4. Save the file.
5. Open the distro.json file.

This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the modules object to ensure that the Gulp tasks include it when you deploy.

Your code should look similar to the following example:

```

1  {
2    "name": "SuiteCommerce Advanced Elbrus",
3    "version": "2.0",
4    "buildToolsVersion": "1.3.0",
5    "folders": {
6      "modules": "Modules",
7      "suitecommerceModules": "Modules/suitecommerce",
8      "extensionsModules": "Modules/extensions",
9      "thirdPartyModules": "Modules/third_parties",
10     "distribution": "LocalDistribution",
11     "deploy": "DeployDistribution"
12   },
13   "modules": {
14     "extensions/ProductDetails.Extension": "1.0.0",
15     "extensions/MyExampleCartExtension1": "1.0.0",

```

```
16 | //...
```

7. Include the module definition (`ProductDetails.Base.View.Extension`) as a dependency within the JavaScript `SC.Shopping.Starter` entry point.

Your `distro.json` file should look similar to the following:

```
1 | "tasksConfig": {
2 |   //...
3 |   "javascript": [
4 |     {
5 |       "entryPoint": "SC.Shopping.Starter",
6 |       "exportFile": "shopping.js",
7 |       "dependencies": [
8 |         //...
9 |         "StoreLocator",
10 |        "ProductDetails.Base.View.Extension"
11 |      ]
12 |    }
13 |   ]
14 |   //...
```

8. Save the `distro.json` file.

Step 3: Test and Deploy Your Customization

1. Test your source code customization on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, you should be able to configure item options using the SuiteCommerce Configuration record (**Shopping Catalog** tab, **Item Options** subtab).

Incorrect Redirect URL for External Payments

i Applies to: SuiteCommerce Advanced | Elbrus | Vinson

On releases of Elbrus and Vinson, the checkout process that uses an external payment system does not correctly cancel the process and return to the SuiteCommerce site. Instead, when you click **Cancel** while using an external payment system, a 404 error occurs. This error occurs due to an incorrect URL variable value in the **OrderWizard.Module.PaymentMethod** module.

To implement this patch, you must extend the prototype object for the `render()` method in `OrderWizard.Module.PaymentMethod.External.js` for the **OrderWizard.Module.PaymentMethod** module. You can download the code samples described in this procedure here: [IncorrectRedirectURLforExternalPaymentsElbrus.zip](#) or [IncorrectRedirectURLforExternalPaymentsVinson.zip](#).

i Note: Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Extend the `OrderWizard.Module.PaymentMethod.External.js` File

1. If you have not done so already, create a directory to store your custom module.

Give this directory a name similar to the module being customized. For example, create `Modules/extensions/OrderWizard.Module.PaymentMethod.Extension@1.0.0`.

2. In your new `OrderWizard.Module.PaymentMethod.Extension@1.0.0` directory, create a subdirectory called `JavaScript`.

For example: `Modules/extensions/OrderWizard.Module.PaymentMethod.Extension@1.0.0/JavaScript`

3. In your new `JavaScript` subdirectory, create a `JavaScript` file to extend `OrderWizard.Module.PaymentMethod.External.js`.

Name this file according to best practices. For example:

`OrderWizard.Module.PaymentMethod.External.Extension.js`

4. Open this file and extend the `render()` method as shown in the following code snippet.

```

1  define(
2    'OrderWizard.Module.PaymentMethod.External.Extension'
3  , [
4    'OrderWizard.Module.PaymentMethod'
5    , 'Session'
6    , 'OrderPaymentMethod.Model'
7
8    , 'order_wizard_paymentmethod_external_module.tpl'
9
10   , 'Backbone'
11   , 'underscore'
12   , 'SC.Configuration'
13   , 'Utils'
14 ]
15 , function (
16   OrderWizardModulePaymentMethod
17   , Session
18   , OrderPaymentMethodModel
19
20   , order_wizard_paymentmethod_external_module_tpl
21
22   , Backbone
23   , _
24   , Configuration
25   , Utils
26 )
27 {
28   'use strict';
29
30   _.extend(OrderWizard.Module.PaymentMethod.prototype, {
31     render: function ()
32     {
33       var n = Configuration.get('siteSettings.id')
34       , status_accept_value = Configuration.get('siteSettings.externalCheckout.' + this.options.record_type.toUpper
35 Case() + '.statusAcceptValue' , 'ACCEPT')
36       , status_reject_value = Configuration.get('siteSettings.externalCheckout.' + this.options.record_type.toUpper
37 Case() + '.statusRejectValue' , 'REJECT')
38       , status_parameter_name = Configuration.get('siteSettings.externalCheckout.' + this.options.record_type.toUpper
39 Case() + '.statusParameterName' , 'status')
40       , origin = window.location.origin ? window.location.origin : (window.location.protocol + '//' + window.loca
41 tion.hostname + (window.location.port ? ':' + window.location.port : ''))
42       , url = origin + _.getAbsolutePath('external_payment.ssp')
43       , current_touchpoint = Configuration.get('currentTouchpoint')
44       , thankyouurl_parameters = {n: n, externalPaymentDone: 'T', touchpoint: current_touchpoint, recordType: this.op
45 tions.record_type || 'salesorder' }
46       , errorurl_parameters = {n: n, externalPaymentDone: 'T', touchpoint: current_touchpoint, recordType: this.op
47 tions.record_type || 'salesorder' }
48       , returnurl_parameters = {n: n, externalPaymentDone: 'T', touchpoint: current_touchpoint, recordType: this.op
49 tions.record_type || 'salesorder'};
50
51       thankyouurl_parameters[status_parameter_name] = status_accept_value;
52       errorurl_parameters[status_parameter_name] = status_reject_value;
53     }
54   });
55 }

```

```

48     if (this.options.prevent_default)
49     {
50         var prevent_default_parameter_name = Configuration.get('siteSettings.externalCheckout.' + this.option
51 s.record_type.toUpperCase() + '.preventDefaultParameterName', 'preventDefault')
52         , prevent_default_value = Configuration.get('siteSettings.externalCheckout.' + this.options.record_type.toUp
53 perCase() + '.preventDefaultValue', 'T');
54
55         thankyouurl_parameters[prevent_default_parameter_name] = prevent_default_value;
56         errorurl_parameters[prevent_default_parameter_name] = prevent_default_value;
57         returnurl_parameters[prevent_default_parameter_name] = prevent_default_value;
58     }
59
60     this.paymentMethod = new OrderPaymentMethodModel({
61         type: 'external_checkout_' + this.options.paymentmethod.key
62         , isexternal: 'T'
63         , internalid: this.options.paymentmethod.internalid
64         , merchantid: this.options.paymentmethod.merchantid
65         , key: this.options.paymentmethod.key
66         , thankyouurl: Utils.addParamsToUrl(url, thankyouurl_parameters) // Commerce API
67         , errorurl: Utils.addParamsToUrl(url, errorurl_parameters) // Commerce API
68         , returnurl: Utils.addParamsToUrl(url, returnurl_parameters) // SuiteScript
69     });
70
71     this._render();
72
73 }
74
75 })
76 });

```

5. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new OrderWizard.Module.PaymentMethod.Extension@1.0.0 module directory.
2. Create a file in this directory and name it ns.package.json.
Modules/extensions/OrderWizard.Module.PaymentMethod.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code:

```

1  {
2      "gulp": {
3          "javascript": [
4              "JavaScript/*.js"
5          ]
6      }
7  }

```

4. Open the distro.json file. This is located in your root directory.
5. Add your custom modules to the modules object.

Your code should look similar to the following example:

```

1  {
2      "name": "SuiteCommerce Advanced Vinson Release",
3      "version": "2.0",
4      "buildToolsVersion": "1.2.1",
5      "folders": {
6          "modules": "Modules",
7          "suitecommerceModules": "Modules/suitecommerce",
8          "thirdPartyModules": "Modules/third_parties",
9          "distribution": "LocalDistribution",
10         "deploy": "DeployDistribution"
11     },

```

```

12     "modules": {
13       "extensions/OrderWizard.Module.PaymentMethod.Extension": "1.0.0",
14       "suitecommerce/Account": "2.2.0",
15       ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the `modules` object. The order of precedence in this list does not matter.

6. Add `OrderWizard.Module.PaymentMethod.Extension` as a dependency to `SC.Checkout.Starter` entry point within the JavaScript array.

Your `distro.js` file should look similar to the following:

```

1  "tasksConfig": {
2    //...
3    "javascript": [
4      //...
5      {
6        "entryPoint": "SC.Checkout.Starter",
7        "exportFile": "checkout.js",
8        "dependencies": [
9          "Backbone.View.Plugins",
10         "OrderWizard.Module.PaymentMethod.Extension",
11         // ...
12       ],
13       //...

```

7. Save the `distro.json` file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, the **Cancel** button for an external payment system redirects back to the SuiteCommerce site from where it was launched.

npm Error on Implementations

Applies to: SuiteCommerce Advanced | Denali

In some implementations of Denali R2, the `npm install` command returns an error. As a result, the installation cannot continue. This error occurs due to a version mismatch with the request HTTP client. You must change the version of the request client to correct the error.

Step 1: Modify package.json

1. If you have not already done so, create a backup of `package.json`, located in the root of your project directory.
2. Open `package.json` and make the following changes.

Locate the line:

```
1 | "request": "2.39.0",
```

Replace it with the following line:

```
1 | "request": "2.81.0",
```

3. Save the file.

Step 2: Test Your Changes

1. Confirm your results by running the `npm install` command again.

The `npm install` command successfully installs the required Node.js packages.

Content Appears Incorrectly in a Merchandising Zone

Applies to: SuiteCommerce Advanced | Elbrus | Kilimanjaro

In some implementations of Elbrus and Kilimanjaro, items may appear in a merchandising zone that do not meet the criteria for the zone. This error applies to merchandising zones that are added in Site Management Tools (SMT) and appear on either the product detail or cart pages.

The process to implement the fix in this patch is the same whether you update Elbrus or Kilimanjaro. To implement this patch, you create a new JavaScript file and override `CMSadapter.js` in the **CMSadapter** module. The new JavaScript file adds a new merchandising implementation in the module.

Note: This fix only applies if you are using CMS Adapter Version 3 on Kilimanjaro or CMS Adapter Version 2 on Elbrus.

For an example of the changes needed for this patch, see one of the following links for the source files, depending on your implementation:

- [MerchandisingZoneContentAppearsIncorrectlyElbrus.zip](#)
- [MerchandisingZoneContentAppearsIncorrectlyKilimanjaro.zip](#)

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file in a way that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the `CMSadapterImpl.Merchandising.js` File and the `CMSadapter.js` Override File

1. Override the `CMSadapter.js` file and create the `CMSadapterImpl.Merchandising.js` file, which are both located in the **CMSadapter** module.

Open the Modules/extensions directory and create a subdirectory to maintain your customizations. For example, create `Modules/extensions/CMSadapter.Extension@1.0.0`

2. In your new CMSAdapter.Extension@1.0.0 directory, create a subdirectory called JavaScript.
For example, create Modules/extensions/CMSAdapter.Extension@1.0.0/JavaScript
3. Copy the following file:
Modules/suitecommerce/CMSAdapter@X.Y.Z/JavaScript/CMSAdapter.js
In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.
4. Paste a copy in your new JavaScript directory in the new module.
For example: Modules/extensions/CMSAdapter.Extension@1.0.0/JavaScript/CMSAdapter.js
5. Open your new copy of the CMSAdapter.js file and locate the following lines:

```
1 | , 'CMSAdapterImpl.Enhanced'
```

Replace this line with the following code:

```
1 | , 'CMSAdapterImpl.Enhanced'
2 | , 'CMSAdapterImpl.Merchandising'
```

6. In the new CMSAdapter.js, add this line at the end of the initAdapter() method:

```
1 | this.adapterMerchandising = new CMSAdapterImplMerchandising(application, CMS);
```

Your code should look similar to the following:

```
1 | , initAdapter: function(application, landingRouter)
2 | {
3 |   if (typeof CMS === 'undefined')
4 |   {
5 |     console.log('Error: CMS global variable not found - CMS adapter not initialized');
6 |     return;
7 |   }
8 |
9 |   this.adapterCore = new CMSAdapterImplCore(application, CMS);
10 |   this.adapterLanding = new CMSAdapterImplLanding(application, CMS, landingRouter);
11 |   this.adapterCategories = new CMSAdapterImplCategories(application, CMS);
12 |
13 |   this.adapterMerchandising = new CMSAdapterImplMerchandising(application, CMS);
14 | }
```

7. Save the file.
8. In the new JavaScript folder, build out the new CMSAdapterImpl.Merchandising.js file with the following code:

```
1 | /*
2 | @module CMSAdapter
3 | @class CMSAdapterImpl.Merchandising
4 | */
5 | define('CMSAdapterImpl.Merchandising'
6 | , [
7 |   'LiveOrder.Model'
8 |   , 'underscore'
9 | ]
10 | , function (
11 |   LiveOrderModel
12 | , _
13 | )
14 | {
15 |   'use strict';
16 |   function AdapterMerchandising(application, CMS)
17 |   {
18 |     this.CMS = CMS;
```

```

19   this.application = application;
20   this.listenForCMS();
21 }
22
23   _._extend(AdapterMerchandising.prototype, {
24     listenForCMS: function ()
25     {
26       var self = this;
27       self.CMS.on('cart:items:get', function (promise)
28       {
29         var linesId = []
30         , cartInstance = LiveOrderModel.getInstance();
31         cartInstance.cartLoad.then(function()
32         {
33           var lines = cartInstance.get('lines');
34           lines.each(function(line)
35           {
36             var item = line.get('item')
37             , internalId = item && item.get('internalid');
38             if (internalId)
39             {
40               linesId.push(internalId);
41             }
42           });
43           promise.resolve(linesId);
44         });
45       });
46       self.CMS.on('page:items:get', function (promise)
47       {
48         var linesId = []
49         , PDP_FULL_VIEW = 'ProductDetails.Full.View'
50         , PDP_QUICK_VIEW = 'ProductDetails.QuickView.View'
51         , view = self.application.getLayout().getCurrentView()
52         , view_identifier = view && view.attributes &&
53           view.attributes.id
54         , view_prototype_id = view && view.prototype &&
55           view.prototype.attributes && view.prototype.attributes.id;
56         if (view && (view_identifier === PDP_FULL_VIEW ||
57           view_identifier === PDP_QUICK_VIEW ||
58           view_prototype_id === PDP_FULL_VIEW || view_prototype_id
59             === PDP_QUICK_VIEW))
60         {
61           var internalId = view.model && view.model.get('item') &&
62             view.model.get('item').get('internalid');
63           if (internalId)
64           {
65             linesId.push(internalId);
66           }
67         }
68         promise.resolve(linesId);
69       });
70     }
71   });
72   return AdapterMerchandising;
73 });

```

9. Save CMSAdapterImpl.Merchandising.js.

The following procedure shows how to create the ns.package.json file for the custom module and shows how to modify the distro.json file to make sure that the Gulp tasks include your modules when you deploy.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new Modules/extensions/CMSadapter.Extension@1.0.0 module.
2. Create a file in this directory and name it ns.package.json.
Modules/extensions/CMSadapterImpl.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code:

```

1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   },
7 |   "suitecommerce/CMSadapter@X.Y.Z/JavaScript/CMSadapter.js" : "JavaScript/CMSadapter.js"
8 | }

```



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

4. Open the distro.json file. This is located in your root directory.
5. Add your custom modules to the modules object.

Your code should look similar to the following example:

```

1 | {
2 |   "name": "SuiteCommerce Advanced Elbrus",
3 |   "version": "2.0",
4 |   "buildToolsVersion": "1.3.0",
5 |   "folders": {
6 |     "modules": "Modules",
7 |     "suitecommerceModules": "Modules/suitecommerce",
8 |     "extensionsModules": "Modules/extensions",
9 |     "thirdPartyModules": "Modules/third_parties",
10 |    "distribution": "LocalDistribution",
11 |    "deploy": "DeployDistribution"
12 |   },
13 |   "modules": {
14 |     ...
15 |     "extensions/CMSadapter.Extension": "1.0.0",
16 |     ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, the merchandising zones for products contain only items that meet the criteria for the zone.

Reference My Account Generates Error on Load

Applies to: SuiteCommerce Advanced | pre-Denali

This section applies to the **Reference My Account v1.04** bundle.

In some pre-Denali releases of SuiteCommerce Advanced, viewing the home page of My Account results in a blank page and SuiteCommerce returns the following error:

Uncaught TypeError: Cannot read property 'is_logged_in' of undefined.

This error can occur because of the number of invoices returned for My Account is too large for the browser to load. This patch limits the number of returned invoices at 1000. To implement this patch, you must update the `Application.getModel` function in `Models.js`.

To modify Models.js:

1. If you have not done so already, copy the `Models.js` file from the reference folder to your custom folder in the NetSuite File Cabinet.

`Models.js` is located in the File Cabinet at My Account 1.04 > Reference My Account > `ssp_libraries`.

2. Update the `setPaymentMethod` function in the `Models.js` file from your custom folder:

Locate the following lines in `Models.js`:

```
1 | var invoices_info = Application.getModel('Receipts').list({
2 |   type: 'invoice'
3 |   , internalid: _.pluck(result.invoices, 'internalid')
4 | });
```

Replace these lines with the following code:

```
1 | var invoices_id = _.pluck(result.invoices, 'internalid');
2 | invoices_id.sort(function(a,b){ return parseInt(b) - parseInt(a) });
3 |
4 | var invoices_info = Application.getModel('Receipts').list({
5 |   type: 'invoice'
6 |   , internalid: invoices_id.slice(0, 1000)
7 | });
```

3. Save `Models.js`.
4. Configure your SSP Application to use the custom `Models.js` file.
 1. Open the My Account SSP application record at Setup > SuiteCommerce Advanced > SSP Applications.
 2. On the **Scripts** subtab, update the **Libraries** list to include the `Models.js` file from your custom folder.
This file must maintain the correct position at the bottom of the list of library files.
 3. Click Save.
5. Verify that the customization was successful.

Log in to the server and go to My Account > Overview. The home page for My Account loads successfully.

Error Loading Shopping Page Due to Uncaught TypeError

i Applies to: SuiteCommerce Advanced | Elbrus

In some implementations of Elbrus, the shopping page returns a blank page and the following error is returned in the log for the page:

Uncaught TypeError: Cannot read property 'indexOf' of undefined

This error can occur because the URL includes the percent sign (%) and SuiteCommerce does not correctly handle the symbol. This patch overrides Utils.js in the **Utilities** module to add the `getDecodedURLParameter` function and correctly handle the symbol. You can download the code samples described in this procedure here: [ShoppingPageErrorUncaughtTypeError.zip](#).

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the Utils.js Override File

1. To override the Utils.js file, located in the **Utilities** module, open the Modules/extensions directory and create a subdirectory to maintain your customizations.

Give this directory a name similar to the module being customized. For example, create Modules/extensions/UtilitiesExtension@1.0.0

2. In your new UtilitiesExtension@1.0.0 directory, create a subdirectory called JavaScript.

For example: Modules/extensions/UtilitiesExtension@1.0.0/JavaScript

3. Copy the following file:

Modules/suitecommerce/Utilities@X.Y.Z/JavaScript/Utils.js

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

4. Paste a copy in the JavaScript directory for the new module.

For example: Modules/extensions/Utilities@X.Y.Z/JavaScript/Utils.js

5. Open your new copy of Utils.js and make the following change.

Add the following `getDecodedURLParameter` function before the existing `parseUrlOptions` function:

```

1 function getDecodedURLParameter (url_parameter)
2 {
3   url_parameter = url_parameter || '';
4   var position;
5
6   for(var temporal = ''; (position = url_parameter.indexOf('%')) >= 0; url_parameter =
7     url_parameter.substring(position + 3))
8   {
9     temporal += url_parameter.substring(0, position);
10    var x = url_parameter.substring(position, position + 3);
11
12    try
13    {
14      temporal += decodeURIComponent(x);
15    }
16    catch (e)
17    {
18      temporal += x;
19    }
20  }
21  return temporal + url_parameter;
22 }
```

Your final result should look similar to the following:

```

1 | ...
```

```

2   return baseUrl;
3   }
4
5   function getDecodedURLParameter (url_parameter)
6   {
7     // new function
8   }
9
10  function parseUrlOptions (options_string)
11  {
12    ...

```

6. Add the call to the getDecodedURLParameter function in SC.Utills:

```

1   var Utills = SC.Utills = {
2     ...
3     , deepCopy: deepCopy
4     , getDecodedURLParameter: getDecodedURLParameter
5   };

```

7. Locate the following line in the parseUrlOptions function:

```

1   options[current_token[0]] = decodeURIComponent(current_token[1]);

```

Replace this line with the following line:

```

1   options[current_token[0]] = Utills.getDecodedURLParameter(current_token[1]);

```

8. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. To make sure that the Gulp tasks include your modules when you deploy, set up the ns.package.json file for your custom module and modify distro.json.

Open the Modules/extensions/UtilitiesExtension@1.0.0 module directory.

2. Create a file in this directory and name it ns.package.json:

Modules/extensions/UtilitiesExtension@1.0.0/ns.package.json

3. Build the ns.package.json file using the following code:

```

1   {
2     "gulp": {
3       "javascript": [
4         "JavaScript/*.js"
5       ]
6     },
7     "overrides": {
8       "suitecommerce/Utilities@X.Y.Z/JavaScript/Utils.js" : "JavaScript/Utils.js"
9     }
10  }

```

4. Open the distro.json file. This is located in your root directory.
5. Add your custom modules to the modules object.

Your code should look similar to the following example:

```

1   {
2     "name": "SuiteCommerce Advanced Elbrus",
3     "version": "2.0",
4     "buildToolsVersion": "1.3.0",
5     "folders": {

```

```

6   "modules": "Modules",
7   "suitecommerceModules": "Modules/suitecommerce",
8   "extensionsModules": "Modules/extensions",
9   "thirdPartyModules": "Modules/third_parties",
10  "distribution": "LocalDistribution",
11  "deploy": "DeployDistribution"
12 },
13 "modules": {
14   "extensions/UtilitiesExtension": "1.0.0",
15   "extensions/MyExampleCartExtension1": "1.0.0",
16   ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, URLs that contain the percent sign symbol load properly.

Users Redirected to Checkout Application Instead of Shopping Application

Applies to: SuiteCommerce Advanced | Elbrus | Kilimanjaro

In some implementations of Elbrus and Kilimanjaro, users who access the Shopping application are redirected to the login page for the Checkout application. The users stay in the Checkout application after they log in. Users should be able to access the Shopping application, log in, and stay in the Shopping application. This error only occurs for websites that have the **Password-protect Entire Site** field in the Configuration record for the web site enabled.

To implement this patch, you must override `Header.Menu.MyAccount()` in the `Header.Profile.View.js` file of the **Header** module. For an example of the changes needed for this patch, see [UsersRedirectedtoCheckoutApp.zip](#).

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file in a way that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Note: The instructions to install the patch are the same for either the Elbrus or Kilimanjaro releases.

Step 1: Create the Header.Profile.View.js Override File

1. If you have not done so already, create a directory to store your custom module.
2. Open this directory and create a subdirectory to maintain your customizations.
Give this directory a name similar to the module being customized. For example:
`Modules/extensions/Header.Extension@1.0.0`
3. In your new `Header.Extension@1.0.0` directory, create a subdirectory called `JavaScript`:
`Modules/extensions/Header.Extension@1.0.0/JavaScript`
4. Copy the following file:
`Modules/suitecommerce/Header@X.Y.Z/JavaScript/Header.Profile.View.js`
In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.
5. Paste a copy in your new module's `JavaScript` directory.
For example: `Modules/extensions/Header.Extension@1.0.0/JavaScript/Header.Profile.View.js`
6. Open your new copy of `Header.Profile.View.js` and locate the following lines in the new file:

```

1 | 'Header.Menu.MyAccount': function ()
2 | {
3 |     return new HeaderMenuMyAccountView(this.options);
4 | }

```

7. Replace the lines with the following code:

```

1 | 'Header.Menu.MyAccount': function ()
2 | {
3 |     var password_protected_site =
4 |         SC.ENVIRONMENT.siteSettings.siteloginrequired === 'T';
5 |     var profile = ProfileModel.getInstance();
6 |     var isLoggedIn = profile.get('isLoggedIn') === 'T';
7 |
8 |     if (!password_protected_site || isLoggedIn) {
9 |         return new HeaderMenuMyAccountView(this.options);
10 |     } else {
11 |         return null;
12 |     }
13 | }

```

8. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open the `Modules/extensions/Header.Extension@1.0.0` module directory.
2. Create a file in this directory and name it `ns.package.json`:
`Modules/extensions/Header.Extension@1.0.0/ns.package.json`
3. Create a file in this directory and name it `ns.package.json`.
4. Build the `ns.package.json` file using the following code:

```

1 | {
2 |     "gulp": {
3 |         "javascript": [
4 |             "JavaScript/*.js"
5 |         ]
6 |     },
7 |     "overrides": {
8 |         "suitecommerce/Header@X.Y.Z/JavaScript/Header.Profile.View.js" : "JavaScript/Header.Profile.View.js"
9 |     }

```

```
10 | }
```



Important: You must replace the string X.Y.Z with the version of the module in your implementation of SuiteCommerce Advanced.

5. In `distro.json`, add your custom modules to the `modules` object.

Your code should look similar to the following example:

```
1 | {
2 |   "name": "SuiteCommerce Advanced Vinson Release",
3 |   "version": "2.0",
4 |   "buildToolsVersion": "1.2.1",
5 |   "folders": {
6 |     "modules": "Modules",
7 |     "suitecommerceModules": "Modules/suitecommerce",
8 |     "thirdPartyModules": "Modules/third_parties",
9 |     "distribution": "LocalDistribution",
10 |    "deploy": "DeployDistribution"
11 |  },
12 |  "modules": {
13 |    "extensions/Header.Extension": "1.0.0",
14 |    "suitecommerce/Account": "2.2.0",
15 |    ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the `modules` object. The order of precedence in this list does not matter.

6. Save the `distro.json` file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, users can access the Shopping application and stay there after they log in.

Add to Cart Button Does Not Work If Quantity Field Selected

Applies to: SuiteCommerce Advanced | Elbrus

In implementations of Elbrus, clicking the **Add to Cart** button one time does not add the chosen items to the cart. Instead, users must click this button twice. This error only occurs if the focus is on the **Quantity** field.

To implement this patch, you must override the events object of the `Cart.AddToCart.Button.View.js` file, located in the **Cart** module. Because of the type of change that you need to make to the View file, you must override the view instead of extending it. You can download the code samples described in this procedure here: [AddtoCartButtonDoesNotWork.zip](#).

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file in a way that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the Cart.AddToCart.Button.View.js Override File

1. To override the Cart.AddToCart.Button.View.js file, located in the **Cart** module, create a directory to store your custom module.
2. Open this directory and create a subdirectory to maintain your customizations.
Give this directory a name similar to the module being customized. For example:
`Modules/extensions/Cart.Extension@1.0.0`
3. In your new `Cart.Extension@1.0.0` directory, create a subdirectory called `JavaScript`.
`Modules/extensions/Cart.Extension@1.0.0/JavaScript`
4. Copy the following file into the new `JavaScript` directory:
`Modules/suitecommerce/Cart@X.Y.Z/JavaScript/Cart.AddToCart.Button.View.js`
In this case, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.
5. Open `Cart.AddToCart.Button.View.js` and make the following change:
Locate the following line in the new file:

```
1 | 'click [data-type="add-to-cart"]': 'addToCart'
```

Replace the line with the following lines:

```
1 | 'mouseup [data-type="add-to-cart"]': 'addToCart' , 'click [data-type="add-to-cart"]': 'addToCart'
```

6. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. To make sure that the Gulp tasks include your modules when you deploy, open the `Modules/extensions/Cart.Extension@1.0.0` module directory.
2. Create a file in this directory and name it `ns.package.json`:
`Modules/extensions/Cart.Extension@1.0.0/ns.package.json`
3. Build the `ns.package.json` file using the following code:

```
1 | { "gulp": { "javascript": [ "JavaScript/*.js" ] }, "overrides": { "suitecommerce/Cart@X.Y.Z/JavaScript/Cart.AddToCart.Button.View.js" : "JavaScript/Cart.AddToCart.Button.View.js" }
2 | }
```

4. In `distro.json`, add your custom modules to the `modules` object.
Your code should look similar to the following example:

```
1 | { "name": "SuiteCommerce Advanced Elbrus", "version": "2.0", "buildToolsVersion": "1.3.0", "folders": { "modules": "Modules", "suitecommerceModules": "Modules/suitecommerce", "extensionsModules": "Modules/extensions", "thirdPartyModules": "Modules/third_parties", "distribution": "LocalDistribution", "deploy": "DeployDistribution" }, "modules": { "extensions/Cart.Extension": "1.0.0", "extensions/MyExampleCartExtension1": "1.0.0", ...
```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the `modules` object. The order of precedence in this list does not matter.

5. Save the `distro.json` file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

Upon successful deployment, the **Add to Cart** button works with one click.

URLs with Redundant Facets Generated

i Applies to: SuiteCommerce Advanced | Elbrus

This topic applies to the Elbrus release of SuiteCommerce Advanced (SCA).

When searching for an item using facets, the application can create a URL containing redundant facets. This error occurs because new facets are appended to the existing URL with a forward slash (/). As a result, the appended URL is relative rather than absolute. For example, the following filter can appear for a web site:

```
1 | http://<baseUrl>/Girls/Mens/Girls/Mens
```

This patch extends the `getUrl()` method in `Facets.Translator.js` for the **Facets** module. The code changes ensure that each URL is prepended with a forward slash (if it does not already have one), making it an absolute URL.

To implement this patch, you must extend the prototype object for the `getUrl()` method. You can download the code samples described in this procedure here: [RandomFacetsGenerated.zip](#).

i Note: Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Extend the `Facets.Translator.js` File

1. To extend the `Facets.Translator.js` file, located in the **Facets** module, open the `Modules/extensions` directory and create a subdirectory to maintain your customizations.

Give this directory a name similar to the module being customized. For example, create `Modules/extensions/FacetsExtension@1.0.0`

2. In your new `FacetsExtension@1.0.0` directory, create a subdirectory called `JavaScript`.

For example: `Modules/extensions/FacetsExtension@1.0.0/JavaScript`

3. In your new `JavaScript` subdirectory, create a JavaScript file to extend `Facets.Translator.js`.

Name this file according to best practices. For example:

Facets.Translator.Extension.js

4. Open this file and extend the getUrl() method as shown in the following code snippet.

```

1  define('Facets.Translator.Extension'
2  , [ 'Facets.Translator'
3    , 'underscore'
4    , 'jQuery'
5    , 'SC.Configuration'
6    , 'UrlHelper'
7  ]
8  , function (
9    FacetsTranslator
10   , _
11   , jQuery
12   , Configuration
13 )
14 {
15   'use strict';
16
17   _.extend(FacetsTranslator.prototype,
18     {
19       // @method getUrl
20       // Gets the url for current state of the object
21       getUrl: function getUrl ()
22       {
23         var url = this.categoryUrl || ''
24         , self = this;
25
26         // Prepares seo limits
27         var facets_seo_limits = {};
28
29         if (SC.ENVIRONMENT.jsEnvironment === 'server')
30         {
31           facets_seo_limits = {
32             numberOfFacetsGroups: this.configuration.facetsSeoLimits && this.configuration.facetsSeoLimits.numberOf
33             FacetsGroups || false
34             , numberOfFacetsValues: this.configuration.facetsSeoLimits && this.configuration.facetsSeoLimits.numberOf
35             FacetsValues || false
36             , options: this.configuration.facetsSeoLimits && this.configuration.facetsSeoLimits.options || false
37           };
38
39           // If there are too many facets selected
40           if (facets_seo_limits.numberOfFacetsGroups && this.facets.length > facets_seo_limits.numberOfFacetsGroups)
41           {
42             return '#';
43           }
44
45           // Encodes the other Facets
46           var sorted_facets = _.sortBy(this.facets, 'url')
47           , facets_as_options = [];
48
49           for (var i = 0; i < sorted_facets.length; i++)
50           {
51             var facet = sorted_facets[i];
52
53             // Category should be already added
54             if ((facet.id === 'commercecategoryname') || (facet.id === 'category'))
55             {
56               break;
57             }
58
59             var name = facet.url || facet.id
60             , value = '';
61
62             switch (facet.config.behavior)
63             {
64               case 'range':
65                 facet.value = (typeof facet.value === 'object') ? facet.value : {from: 0, to: facet.value};
66                 value = facet.value.from + self.configuration.facetDelimiters.betweenRangeFacetsValues + facet.val
67 ue.to;

```



```

66         break;
67         case 'multi':
68             value = facet.value.sort().join(self.configuration.facetDelimiters.betweenDifferentFacetsValues);
69
70             if (facets_seo_limits.numberOfFacetsValues && facet.value.length > facets_seo_limits.numberOf
FacetsValues)
71             {
72                 return '#';
73             }
74
75             break;
76         default:
77             value = facet.value;
78     }
79
80     if (self.facetIsParameter(name))
81     {
82         facets_as_options.push({ facetName: name, facetValue: value });
83     }
84     else
85     {
86         // Do not add a facet separator at the beginning of an url
87         if (url !== '')
88         {
89             url += self.configuration.facetDelimiters.betweenDifferentFacets;
90         }
91
92         url += name + self.configuration.facetDelimiters.betweenFacetNameAndValue + value;
93     }
94 }
95
96 url = (url !== '') ? url : '/' + this.configuration.fallbackUrl;
97
98 // Encodes the Options
99 var tmp_options = {}
100 ,   separator = this.configuration.facetDelimiters.betweenOptionNameAndValue;
101
102 if (this.options.order && this.options.order !== this.configuration.defaultOrder)
103 {
104     tmp_options.order = 'order' + separator + this.options.order;
105 }
106
107 if (this.options.page && parseInt(this.options.page, 10) !== 1)
108 {
109     tmp_options.page = 'page' + separator + encodeURIComponent(this.options.page);
110 }
111
112 if (this.options.show && parseInt(this.options.show, 10) !== this.configuration.defaultShow)
113 {
114     tmp_options.show = 'show' + separator + encodeURIComponent(this.options.show);
115 }
116
117 if (this.options.display && this.options.display !== this.configuration.defaultDisplay)
118 {
119     tmp_options.display = 'display' + separator + encodeURIComponent(this.options.display);
120 }
121
122 if (this.options.keywords && this.options.keywords !== this.configuration.defaultKeywords)
123 {
124     tmp_options.keywords = 'keywords' + separator + encodeURIComponent(this.options.keywords);
125 }
126
127 for (i = 0; i < facets_as_options.length; i++)
128 {
129     var facet_option_obj = facets_as_options[i];
130
131     tmp_options[facet_option_obj.facetName] = facet_option_obj.facetName + separator + encodeURIComponent
t(facet_option_obj.facetValue);
132 }
133
134 var tmp_options_keys = _.keys(tmp_options)
135 ,   tmp_options_vals = _.values(tmp_options);
136

```

```

137 // If there are options that should not be indexed also return #
138 if (facets_seo_limits.options && _.difference(tmp_options_keys, facets_seo_limits.options).length)
139 {
140     return '#';
141 }
142
143 url += (tmp_options_vals.length) ? this.configuration.facetDelimiters.betweenFacetsAndOptions + tmp_option
s_vals.join(this.configuration.facetDelimiters.betweenDifferentOptions) : '';
144
145 if (url && url[0] !== this.configuration.facetDelimiters.betweenDifferentFacets)
146 {
147     url = this.configuration.facetDelimiters.betweenDifferentFacets + url;
148 }
149 return _(url).fixUrl();
150 }
151 });
152 });

```

5. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. To make sure that the Gulp tasks include your modules when you deploy, set up the ns.package.json file for your custom module and modify distro.json.
Open your new FacetsExtension@1.0.0 module.
2. Create a file in this directory and name it ns.package.json:
Modules/extensions/FacetsExtension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code:

```

1 {
2   "gulp": {
3     "javascript": [
4       "JavaScript/*.js"
5     ]
6   }
7 }

```

4. In distro.json, add your custom modules to the modules object.
Your code should look similar to the following example:

```

1 {
2   "name": "SuiteCommerce Advanced Elbrus",
3   "version": "2.0",
4   "buildToolsVersion": "1.3.0",
5   "folders": {
6     "modules": "Modules",
7     "suitecommerceModules": "Modules/suitecommerce",
8     "extensionsModules": "Modules/extensions",
9     "thirdPartyModules": "Modules/third_parties",
10    "distribution": "LocalDistribution",
11    "deploy": "DeployDistribution"
12  },
13  "modules": {
14    "extensions/FacetsExtension": "1.0.0",
15    "extensions/MyExampleCartExtension1": "1.0.0",
16    //...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

5. Add Facets.Translator.Extension as a dependency to SCA entry point within the SC.Shopping.Starter entrypoint of the JavaScript array.
Your Distro.js file should look similar to the following:

```

1  "tasksConfig": {
2    //...
3    "javascript": [
4      //...
5      {
6        "entryPoint": "SC.Shopping.Starter",
7        "exportFile": "shopping.js",
8        "dependencies": [
9          //...
10         "Newsletter",
11         "ProductDetailToQuote",
12         "StoreLocator",
13         "Facets.Translator.Extension"
14       ],
15       //...

```

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.
Upon successful deployment, filters to the products do not contain redundancies.

Content Flickers or Disappears When Browsing the Product Listing Page

i Applies to: SuiteCommerce Advanced | Kilimanjaro

In some implementations of Kilimanjaro, content added to the site through SMT may render intermittently or disappear and appear briefly on the product listing page. This error does not occur in SMT but does occur in the published content on the site.

This patch overrides CMSadapter.Component.js in the **CMSadapter** module with the correct code. You can download the code samples described in this procedure here: [Kilimanjaro-ContentFlickerPatch.zip](#).

i Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file in a way that you cannot extend, and therefore requires you to use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the CMSadapter.Component.js Override File

1. Open the Modules/extensions directory and create a subdirectory to maintain your customizations. Give this directory a name similar to the module being customized. For example, create Modules/extensions/CMSadapter.Extension@1.0.0
2. In your new CMSadapter.Extension@1.0.0 directory, create a subdirectory called JavaScript. For example: Modules/extensions/CMSadapter.Extension@1.0.0/JavaScript
3. Copy the following file:
Modules/suitecommerce/CMSadapter@X.Y.Z/CMSadapter.Component.js
In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

4. Paste a copy in your new module's JavaScript directory.
For example: Modules/extensions/CMSadapter.Extension@1.0.0/JavaScript/CMSadapter.Component.js
5. Open your new copy of CMSadapter.Component.js and locate the following lines:

```

1 | var parent_view_instance = cct_container.parent;
2 |
3 | parent_view_instance.addChildViewInstances(generator, true);
4 |
5 | self._addViewCctsToRerender(parent_view_instance, generator);

```

6. Replace these lines with the following:

```

1 | var placeholders = self.getPlaceholderViews('[data-cms-area="' + container_name + '"]');
2 |
3 | if (placeholders && placeholders.views)
4 | {
5 |     _.each(placeholders.views, function(parent_view_instance)
6 |     {
7 |         cct_container.parent = parent_view_instance;
8 |         cct_container.container.parentView = parent_view_instance;
9 |
10 |         parent_view_instance.addChildViewInstances(generator, true);
11 |
12 |         self._addViewCctsToRerender(parent_view_instance, generator);
13 |     });
14 | }

```

7. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your Modules/extensions/CMSadapter.Extension@1.0.0 module directory.
2. Create a file in this directory and name it ns.package.json.
Modules/extensions/CMSadapter.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code:

```

1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   },
7 |   "overrides": {
8 |     "suitecommerce/CMSadapter@X.Y.Z/JavaScript/CMSadapter.Component.js" : "JavaScript/CMSadapter.Component.js"
9 |   }
10 | }

```

In this example, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

4. Open the distro.json file. This is located in your root directory.
5. Add your custom modules to the modules object.
Your code should look similar to the following example:

```

1 | {
2 |   "name": "SuiteCommerce Advanced Kilimanjaro",
3 |   "version": "2.0",
4 |   "isSCA": true,
5 |   "buildToolsVersion": "1.3.1",
6 |   "folders": {
7 |     "modules": "Modules",

```

```

8      "suitecommerceModules": "Modules/suitecommerce",
9      "extensionsModules": "Modules/extensions",
10     "thirdPartyModules": "Modules/third_parties",
11     "distribution": "LocalDistribution",
12     "deploy": "DeployDistribution"
13   },
14   "modules": {
15     "extensions/CMSadapter.Extension": "1.0.0",
16     "extensions/MyExampleCartExtension1": "1.0.0",
17     ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, the content on the product listing page does not flicker or disappear.

Enabling Google AdWords Causes Error on Login

Applies to: SuiteCommerce Advanced | Vinson

In some implementations of Vinson, Google Ads (previously called Google Adwords) can cause the following error:

"Uncaught TypeError: e.doCallback is not a function"

This error can occur when the following conditions are met:

- Google Ads is enabled for a domain.
- A customer adds items to their cart and logs out without completing the transaction.
- The customer then attempts to log in again.

This error occurs in the **Tracker** module. To fix the error, you must extend the prototype object of the Tracker.prototype.trackEvent() function. For an example of the changes needed for this patch, see [EnablingGoogleAdWordsCausesErroronLogin.zip](#).

Note: Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#).

Step 1: Extend the Tracker.js File

1. If you have not done so already, create a directory to store your custom module.
Give this directory a name similar to the module being customized. For example, create Modules/extensions/TrackerExtension@1.0.0
2. In your new TrackerExtension@1.0.0 directory, create a subdirectory called JavaScript.
For example: Modules/extensions/TrackerExtension@1.0.0/JavaScript
3. In your new JavaScript subdirectory, create a JavaScript file to extend Tracker.js.
Name this file according to best practices. For example:

TrackerExtension.js

4. Open this file and extend the `Tracker.prototype.trackEvent()` method as shown in the following code snippet.

```

1  // @module Tracker
2  define('TrackerExtension'
3  , [
4      'Tracker',
5      'underscore'
6  ]
7  , function
8  (
9      Tracker,
10     -
11 )
12 {
13     'use strict';
14
15     _.extend(Tracker.prototype,
16     Tracker.prototype.trackEvent = function Tracker.prototype.trackEvent (event)
17     {
18         this.track('trackEvent', event);
19
20         if (event.callback)
21         {
22             var doCallback = _.find(this.trackers, function (tracker)
23             {
24                 return tracker.doCallback && tracker.doCallback();
25             });
26
27             !doCallback && event.callback();
28         }
29
30         return this;
31     };
32 }
33 }
34 );

```

5. Save the file.

Step 2: Prepare the Developer Tools For Your Customization

1. Open your new TrackerExtension@1.0.0 module directory.
2. Create a file in this directory and name it `ns.package.json`.
Modules/extensions/TrackerExtension@1.0.0/ns.package.json
3. Build the `ns.package.json` file using the following code:

```

1  {
2      "gulp": {
3          "javascript": [
4              "JavaScript/*.js"
5          ]
6      }
7  }

```

4. Open the `distro.json` file. This is located in your root directory.
5. Add your custom modules to the `modules` object.
Your code should look similar to the following example:

```

1  {
2      "name": "SuiteCommerce Advanced Vinson Release",
3      "version": "2.0",

```

```

4   "buildToolsVersion": "1.2.1",
5   "folders": {
6     "modules": "Modules",
7     "suitecommerceModules": "Modules/suitecommerce",
8     "thirdPartyModules": "Modules/third_parties",
9     "distribution": "LocalDistribution",
10    "deploy": "DeployDistribution"
11  },
12  "modules": {
13    "extensions/TrackerExtension": "1.0.0",
14    "suitecommerce/Account": "2.2.0",
15    ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

6. Add TrackerExtension as a dependency to SCA entry point within the SC.Shopping.Starter entripoint of the JavaScript array.

Your distro.js file should look similar to the following:

```

1  "tasksConfig": {
2    // ...
3    "javascript": [
4      // ...
5      {
6        "entryPoint": "SC.Shopping.Starter",
7        "exportFile": "shopping.js",
8        "dependencies": [
9          // ...
10         "TrackerExtension",
11         "Backbone.View.Plugins",
12         "jQuery.html",
13         // ...

```

7. Save the distro.json file.

Step 3: Test and Deploy Your Customization

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, enabling Google Ads does not cause errors when a customer logs in or out with items in the cart.

URL for Commerce Categories Contains Incorrect Delimiters

Applies to: SuiteCommerce Advanced | Elbrus

In some implementations of SCA, a URL that uses the Item Search API to access a commerce category contains incorrect delimiters. As a result, accessing pages for a specific category in a SuiteCommerce store returns a 404 error code, even if items for the category exist. For example, a URL might contain %3F instead of the question mark (?) character or %2F instead of forward slash (/).

This patch contains updated code for the FacetsTranslator object constructor. To implement this patch, you must override the FacetsTranslator.js JavaScript file in the **Facets** module. You can download the code samples described in this procedure here: [IncorrectDelimitersforCommerceCategories.zip](#)

Note: In general, best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file that you cannot extend, and you must use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

The following procedure shows how to override the Facets.Translator.js file, located in the **Facets** module.

Step 1: Create the Facets.Translation.js Override File

1. To override the Facets.Translator.js file, located in the **Facets** module, create a directory to store your custom module.
2. Open this directory and create a custom module to maintain your customizations.
Give this module a name similar to the module being customized.
For example:
Modules/extensions/FacetsExtension@1.0.0
3. In your new module, create a subdirectory called JavaScript.
For example:
Modules/extensions/FacetsExtension@1.0.0/JavaScript
4. Copy the following file into the new directory:
Modules/suitecommerce/Facets@X.Y.Z/JavaScript/Facets.Translator.js
In this case, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.
5. Open your new Facets.Translator.js and make the following change:
Locate the JavaScript code for the FacetsTranslator constructor in your new file:

```

1  // @constructor @param {Array} facets @param {Object} options @param {Object} configuration
2  function FacetsTranslator (facets, options, configuration, category)
3  {
4      // Enforces new
5      if (!(this instanceof FacetsTranslator))
6      {
7          return new FacetsTranslator(facets, options, configuration, category);
8      }
9
10     ...
11
12     else if (facets)
13     {
14         // It's an API option object
15         this.parseOptions(facets);
16     }
17 }
```

Replace these lines with the following code:

```

1      // @constructor @param {Array} facets @param {Object} options @param {Object} configuration
2      function FacetsTranslator (facets, options, configuration, category)
3      {
4          // Enforces new
5          if (!(this instanceof FacetsTranslator))
6          {
7              return new FacetsTranslator(facets, options, configuration, category);
8          }
9
10         // Facets go Here
11         this.facets = [];
```



```

12 // Other options like page, view, etc. goes here
13 this.options = {};
14
15
16 // This is an object that must contain a fallbackUrl and a lists of facet configurations
17 this.configuration = configuration || default_config;
18
19 // Get the facets that are in the sitesettings but not in the config.
20 // These facets will get a default config (max, behavior, etc.) - Facets.Translator
21 // Include facet aliases to be considered as a possible route
22 var facets_data = Configuration.get('siteSettings.facetfield')
23 , facets_to_include = [];
24
25 _.each(facets_data, function(facet)
26 {
27     if (facet.facetfieldid !== 'commercecategory')
28     {
29         facets_to_include.push(facet.facetfieldid);
30         // If the facet has an urlcomponent defined, then add it to the possible values list.
31         facet.urlcomponent && facets_to_include.push(facet.urlcomponent);
32
33         // Finally, include URL Component Aliases...
34         _.each(facet.urlcomponentaliases, function(facet_alias)
35         {
36             facets_to_include.push(facet_alias.urlcomponent);
37         });
38     }
39 });
40
41 facets_to_include = _.union(facets_to_include, _.pluck(Configuration.get('facets'), 'id'));
42 facets_to_include = _.uniq(facets_to_include);
43
44 this.facetsToInclude = facets_to_include;
45
46 this.isCategoryPage = !!category;
47
48 if (_.isBoolean(category) && category)
49 {
50     var index = facets.length
51     , facetsToInclude = this.facetsToInclude.slice(0);
52
53     facetsToInclude.push(this.configuration.facetDelimiters.betweenFacetsAndOptions);
54
55     _.each(facetsToInclude, function(facetname)
56     {
57         var i = facets.indexOf(facetname);
58
59         if (i !== -1 && i < index)
60         {
61             index = i;
62         }
63     });
64
65     var categoryUrl = facets.substring(0, index);
66
67     facets = facets.substring(index);
68
69     if (categoryUrl[0] !== '/')
70     {
71         categoryUrl = '/' + categoryUrl;
72     }
73
74     if (categoryUrl[categoryUrl.length - 1] === '/')
75     {
76         categoryUrl = categoryUrl.substring(0, categoryUrl.length - 1);
77     }
78
79     this.categoryUrl = categoryUrl;
80 }
81
82 else if (_.isString(category))
83 {
84     this.categoryUrl = category;

```

```

85     }
86
87     // We cast on top of the passed in parameters.
88     if (facets && options)
89     {
90         this.facets = facets;
91         this.options = options;
92     }
93
94     else if (_.isString(facets))
95     {
96         // It's a url
97         this.parseUrl(facets);
98     }
99
100    else if (facets)
101    {
102        // It's an optional API object
103        this.parseOptions(facets);
104    }
105 }

```

6. Save the file.

Step 2. Prepare the Developer Tools For Your Override

1. To make sure that the Gulp tasks include your modules when you deploy, create the `ns.package.json` file for the custom module and modify the `distro.json` file.
Open the `Modules/extensions/FacetsExtension@1.0.0` module directory.
2. Create a file in this directory and name it `ns.package.json`.
`Modules/extensions/FacetsExtension@1.0.0/ns.package.json`
3. Build the `ns.package.json` file using the following code (where X.Y.Z indicates the current version of your module in SuiteCommerce):

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5      ]
6    },
7    "overrides": {
8      "suitecommerce/Facets@X.Y.Z/JavaScript/Facets.Translations.js" :
9      "JavaScript/Facets.Translations.js"
10   }
11 }
12 }

```

4. Save the `ns.package.json` file.
5. Open the `distro.json` file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.
6. In the `distro.json` file, add your custom modules to the `modules` object.
Your code should look similar to the following example:

```

1  {
2    "name": "SuiteCommerce Advanced Elbrus",
3    "version": "2.0",
4    "buildToolsVersion": "1.3.0",
5    "folders": {
6      "modules": "Modules",
7      "suitecommerceModules": "Modules/suitecommerce",
8      "extensionsModules": "Modules/extensions",
9      "thirdPartyModules": "Modules/third_parties",
10     "distribution": "LocalDistribution",

```

```

11     "deploy": "DeployDistribution"
12   },
13   "modules": {
14     "extensions/FacetsExtension": "1.0.0",
15     ...

```

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

7. Save the distro.json file.

Step 3: Test and Deploy Your Extension

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, the URL used to access products by category includes the proper delimiters and shows the products instead of a 404 error.

Order Summary for Item-Based Promotions

i Applies to: SuiteCommerce Advanced | Elbrus

If an order includes an item with an item-based promotion, the Order Summary does not properly display the price for the items. The Shopping Cart correctly displays the price but the price in the Order Summary is incorrect.

This patch updates the detail3 variable in the JavaScript files used to display the item price after discount in the Order Summary. You can download the code samples described in this procedure here: [FixTotalInOrderSummary.zip](#) in the File Cabinet.

i Note: In general, best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file that you cannot extend, and you must use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

To apply this patch, you override the modules used to display the total in the Order Summary.

Step 1: Create and Update the Required Override Files

1. If you have not done so already, create a directory to store your custom module, for example, Modules\extensions.
2. Open this directory and create the following subdirectories to maintain your customizations. Give these directories a name similar to the modules being customized. For example, create the following directories in the Modules/extensions directory:
 - CreditMemoExtension@1.0.0\JavaScript
 - InvoiceExtension@1.0.0\JavaScript
 - OrderWizard.Module.CartItemsExtension@1.0.0\JavaScript
 - OrderWizard.Module.ShipmethodExtension@1.0.0\JavaScript

- QuoteExtension@1.0.0 \JavaScript
 - ReturnAuthorizationExtension@1.0.0 \JavaScript
3. Copy the following source files into their respective target directories, where the target directory looks like the following:

Modules/extensions/[module]Extension@1.0.0/JavaScript

4. Copy the files in the following list:
 - Modules/suitecommerce/CreditMemo@X.Y.Z/JavaScript/CreditMemo.Details.View.js
 - Modules/suitecommerce/Invoiceo@X.Y.Z/JavaScript/Invoice.Details.View.js
 - Modules/suitecommerce/OrderHistory@X.Y.Z/JavaScript/OrderHistory.ReturnAuthorization.View.js
 - Modules/suitecommerce/OrderWizard.Module.CartItems@X.Y.Z/JavaScript/OrderWizard.Module.CartItems.PickupInStore.Package.View.js
 - Modules/suitecommerce/OrderWizard.Module.CartItems@X.Y.Z/JavaScript/OrderWizard.Module.CartItems.Ship.js
 - Modules/ extensions/OrderWizard.Module.Shipmethod.Extension@X.Y.Z/JavaScript/OrderWizard.Module.MultiShipTo.Shipmethod.Package.View.js
 - Modules/suitecommerce/Quote@X.Y.Z/JavaScript/Quote.Details.View.js
 - Modules/suitecommerce/ReturnAuthorization@X.Y.Z/JavaScript/ReturnAuthorization.Detail.View.js
 - Modules/suitecommerce/ReturnAuthorization@X.Y.Z/JavaScript/ReturnAuthorization.Form.View.js



Note: Remember, X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

5. For each of the target files in the table listed above, make the following code change.
Locate the following line in the new file:

```
1 | ,      detail3: 'amount_formatted'
```

Replace this line with the following code:

```
1 | ,      detail3: 'total_formatted'
```

Step 2. Prepare the Developer Tools for Your Overrides

1. Create an ns.package.json file in each of the following directories:
 - CreditMemoExtension@1.0.0
 - InvoiceExtension@1.0.0
 - OrderWizard.Module.CartItemsExtension@1.0.0
 - OrderWizard.Module.ShipmethodExtension@1.0.0
 - QuoteExtension@1.0.0
 - ReturnAuthorizationExtension@1.0.0
2. In each ns.package.json file, enter the following code, where <module name> is the name of the module you are overriding and <file name> is the name of the JavaScript file that contains the overridden JavaScript code:

```

1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*"
5 |     ],
6 |   }
7 |   "overrides": {
8 |     "suitecommerce/<module name>@X.Y.Z/JavaScript/<file name>" :
9 |       "extensions/JavaScript/<file name>",
10 |   }
11 | }

```

3. Update the distro.json file in the root directory with the following code for each of the modules you are overriding:

Your code should look similar to the following example:

```

1 | ...
2 | },
3 | "extensions/CreditMemoExtension" : "X.Y.Z",
4 | "extensions/InvoiceExtension" : "X.Y.Z",
5 | "extensions/OrderHistoryExtension" : "X.Y.Z",
6 | "extensions/OrderWizard.Module.CartItems.Extension" : "X.Y.Z",
7 | "extensions/OrderWizard.Module.CartItems.Extension" : "X.Y.Z",
8 | "extensions/QuoteExtension" : "X.Y.Z",
9 | "extensions/ReturnAuthorizationExtension" : "X.Y.Z",
10 | "extensions/MyExampleCartExtension1" : "1.0.0",
11 | ...

```

X.Y.Z represents the version of the module in your implementation of SuiteCommerce Advanced.

Step 3. Test and Deploy Your Override

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)).
2. Confirm your results. The prices in the Shopping Cart now correctly match the prices in the Order Summary.

CSS Error Hides First div Element on Product Details Page

i Applies to: Mont Blanc | Vinson

In some implementations of SCA, the CSS hides the first div element, which typically includes the product description on the product details page. As a result, the content in the product description is excluded from the output of the SEO Page Generator, which can affect the SEO ranking for the product.

This patch adds an additional check to validate that the Page Generator generates the content and then unhides the div. To implement this patch, you must override a view file in the **ItemDetails** module. You can download the code samples described in this procedure here: [CSSHidesFirstDiv.zip](#).

i Note: In general, best practice is to extend JavaScript using the JavaScript prototype object. This ensures the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify a file that you cannot extend, and you must use a custom module to override the existing module file. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1. Override the Item.Details.View.js File

1. To override the Item.Details.View.js file, located in the **ItemDetails** module, create a directory in which to store your custom modules.

For example, Modules/extensions.

2. Open this directory and create a subdirectory to maintain your customizations.

Give this directory a name similar to the module being customized.

For example, create Modules/extensions/ItemDetailsExtension@1.0.0

3. In your new ItemDetailsExtension@1.0.0 directory, create a subdirectory called JavaScript.

4. Copy the following file into this directory:

Modules/suitecommerce/ItemDetails@X.Y.Z/JavaScript/Item.Details.View.js

5. Open your new Item.Details.View.js file and make the following change:

Locate the following JavaScript code:

```
1 | if(!overflow)
2 | {
3 |     this.$('.item-details-more').hide();
4 | }
```

Replace it with the following JavaScript code:

```
1 | if(!overflow && !SC.isPageGenerator())
2 | {
3 |     this.$('.item-details-more').hide();
4 | }
```

6. Save Item.Details.View.js .

Step 2. Prepare the Developer Tools for Your Override

1. Set up the ns.package.json file for your custom module and modify distro.json to include the custom module.

Open the Modules/extensions/ItemDetailsExtension@1.0.0 module directory.

2. Create a file in this directory and name it ns.package.json.

Modules/extensions/ItemDetailsExtension@1.0.0/ns.package.json

3. Build the ns.package.json file using the following code (where X.Y.Z indicates the current version of your module in SuiteCommerce) :

```
1 | {
2 |     "gulp": {
3 |         "javascript": [
4 |             "JavaScript/*"
5 |         ]
6 |     },
7 |     "overrides": {
8 |         "suitecommerce/ItemDetails@X.Y.Z/JavaScript/Item.Details.View.js" : "JavaScript/Item.Details.View.js",
9 |     }
10 | }
```

4. Save the ns.package.json file.

5. Open the distro.json file.

This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. In distro.json , add your custom module to the modules object.

Your code should look similar to the following example:

```

1 | {
2 |   "name": "SuiteCommerce Advanced Vinson Release",
3 |   "version": "2.0",
4 |   "buildToolsVersion": "1.2.1",
5 |   "folders": {
6 |     "modules": "Modules",
7 |     "suitecommerceModules": "Modules/suitecommerce",
8 |     "thirdPartyModules": "Modules/third_parties",
9 |     "distribution": "LocalDistribution",
10 |    "deploy": "DeployDistribution"
11 |  },
12 |   "modules": {
13 |     "extensions/ItemDetailsExtension": "X.Y.Z",
14 |     ...

```

7. This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the modules object. The order of precedence in this list does not matter.

Step 3. Test and Deploy Your Override

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, all the required div elements appear in the Product Details Page.

Invoice Terms Not Included In Order Details

Applies to: SuiteCommerce Advanced | pre-Denali

In Reference My Account v1.05, if a shopper checks out using Invoice (Terms), the order in My Account does not include the invoice terms under **Payment Method** in the order details.

This patch updates the setPaymentMethod function in Models.js . To implement this patch, make a copy of Models.js and update it to use the terms object to set the payment terms in Reference My Account.

To customize the Models.js file:

1. If you have not done so already, copy the Models.js file from the reference folder to your custom folder in the NetSuite File Cabinet.

Models.js is located in the File Cabinet at My Account 1.05 > Reference My Account > ssp_libraries.

2. Update the setPaymentMethod function in the Models.js file from your custom folder:

Locate the following code in Models.js:

```

1 | if (record.getFieldValue('terms'))
2 | {
3 |   paymentmethod.type = 'invoice';
4 |
5 |   paymentmethod.purchasenum = record.getFieldValue('otherrefnum');
6 |
7 |   paymentmethod.paymentterms = {
8 |     internalid: record.getFieldValue('terms')
9 |     , name: record.getFieldText('terms')

```

```

10 | };
11 | }

```

Replace these lines with the following code:

```

1 | if (terms)
2 | {
3 |     paymentmethod.type = 'invoice';
4 |
5 |     paymentmethod.purchasenum = record.getFieldValue('otherrefnum');
6 |
7 |     paymentmethod.paymentterms = {
8 |         internalid: record.getFieldValue('terms') || terms
9 |         , name: record.getFieldText('terms') || nlapilookupField(record.recordType
10 |         , record.getFieldValue('id'), 'terms', true)
11 |     };
12 | }

```

3. Configure your SSP Application to use the custom Models.js file.
 1. Open the My Account SSP application record at Setup > SuiteCommerce Advanced > SSP Applications.
 2. On the **Scripts** subtab, update the **Libraries** list to include the Models.js file from your custom folder.
This file must maintain the correct position at the bottom of the list of library files.
 3. Click Save.
4. Verify that the customization was successful.
If a shopper checks out with Invoice (Terms) for an order, the payment method appears correctly under **Payment Method** for the order details in My Account.

Users Required to Re-enter Credit Card Payment Method Details on Payment Page

i Applies to: SuiteCommerce Advanced | Elbrus | Kilimanjaro

This section applies to the **Elbrus and Kilimanjaro** releases of SuiteCommerce Advanced.

In some cases, users making a purchase must re-enter credit card payment method details on the payment page to successfully place an order.

i Note: Before proceeding, familiarize yourself with the [Best Practices for Customizing SCA](#).

Step 1: Extend the OrderWizard.Module.PaymentMethod.Creditcard.js File

1. This step explains how to extend the OrderWizard.Module.PaymentMethod.Creditcard.js file, which is located in the **OrderWizard.Module.PaymentMethod@x.y.z** module. You can download the code samples described in this procedure here: [OrderWizardPaymentMethodExtension-Elbrus and Kilimanjaro.zip](#)
If you have not done so already, create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your customizations.
Give this directory a unique name that is similar to the module being customized. For example:

Modules/extensions/OrderWizardPaymentMethodExtension@1.0.0

3. In your new module, create a subdirectory called JavaScript.
4. In your new JavaScript subdirectory, create a JavaScript file.

Give this file a unique name that is similar to the file being modified. For example:

OrderWizard.Module.PaymentMethod.Creditcard.Extension.js

5. Open this file and overwrite the following method as described below:

■ `_submit()`

Your file should match the following code snippet:

```

1  define(
2    'OrderWizard.Module.PaymentMethod.Creditcard.Extension'
3    , [
4      'Utils'
5      , 'OrderWizard.Module.PaymentMethod'
6      , 'underscore'
7      , 'jQuery'
8      , 'Profile.Model'
9    ]
10   , function (
11     Utils
12     , OrderWizardModulePaymentMethod
13     , _
14     , jQuery
15     , ProfileModel )
16   {
17     'use strict';
18     _._extend(OrderWizardModulePaymentMethodCreditcard.prototype,
19
20     // @method submit
21     submit: function ()
22     {
23       // This order is being paid with some other method (Gift Cert probably)
24       if (this.wizard.hidePayment())
25       {
26         return jQuery.Deferred().resolve();
27       }
28
29       var self = this;
30
31       if (this.requireccsecuritycode)
32       {
33         this.isSecurityNumberInvalid = false;
34         // we need to store this temporarily (frontend) in case a module in the same step
35         // fails validation, making the credit card section re-rendered.
36         // We don't want the user to have to type the security number multiple times
37         this.ccsecuritycode = this.$('input[name="ccsecuritycode"]').val();
38       }
39
40       // if we are adding a new credit card
41       if (this.creditcardView)
42       {
43         var fake_event = jQuery.Event('click', {
44           target: this.creditcardView.$('form').get(0)
45         });
46         , result = this.creditcardView.saveForm(fake_event);
47
48         if (!result || result.frontEndValidationError)
49         {
50           // There were errors so we return a rejected promise
51           return jQuery.Deferred().reject({
52             errorCode: 'ERR_CHK_INCOMPLETE_CREDITCARD'
53             , errorMessage: _('The Credit Card is incomplete').translate()
54           });
55         }
56
57         var save_result = jQuery.Deferred();

```

```

58         result.done(function (model)
59         {
60             self.creditcardView = null;
61
62             ProfileModel.getInstance().get('creditcards').add(model, {
63                 silent: true
64             });
65
66             self.setCreditCard({
67                 model: model
68             });
69
70             save_result.resolve();
71         }).fail(function (error)
72         {
73             save_result.reject(error.responseJSON);
74         });
75
76         return save_result;
77     }
78     // if there are already credit cards
79     else
80     {
81         this.setSecurityNumber();
82
83         OrderWizardModulePaymentMethod.prototype.submit.apply(this, arguments);
84
85         this.setCreditCard({
86             model: this.paymentMethod.get('creditcard')
87         });
88
89         return this.isValid().fail(function (error)
90         {
91             if (error === self.securityNumberErrorMessage)
92             {
93                 self.isSecurityNumberInvalid = true;
94             }
95
96         }).done(function ()
97         {
98             self.isSecurityNumberInvalid = false;
99
100         }).always(function ()
101         {
102             // Call self.render() instead of self._render() because the last one didn't assign the events to the DOM
103             self.render();
104         });
105     }
106 }
107 }
108 });
109 });

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Customizations

1. Open the OrderWizardPaymentMethodExtension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/OrderWizardPaymentMethodExtension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code:

```

1 {
2   "gulp": {
3     "javascript": [
4       "JavaScript/*.js"
5     ]
6   }
7 }

```

4. Save the ns.package.json file.

5. Open the distro.json file.

This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the modules object to ensure that the Gulp tasks include it when you deploy.

In this example, the extensions/OrderWizardPaymentMethodExtension module is added at the beginning of the list of modules. However, you can add the module anywhere in the modules object. The order of precedence in this list does not matter.

```

1  {
2    "name": "SuiteCommerce Advanced Elbrus Release",
3    "version": "2.0",
4    "buildToolsVersion": "1.2.1",
5    "folders": {
6      "modules": "Modules",
7      "suitecommerceModules": "Modules/suitecommerce",
8      "extensionsModules": "Modules/extensions",
9      "thirdPartyModules": "Modules/third_parties",
10     "distribution": "LocalDistribution",
11     "deploy": "DeployDistribution"
12   },
13   "modules": {
14     "extensions/OrderWizardPaymentMethodExtension": "1.0.0",
15     "extensions/Configuration.Extension": "1.0.0",
16     "extensions/SiteSettings.Extension": "1.0.0",
17     "extensions/MyExampleCartExtension1": "1.0.0",
18     "extensions/MyExamplePDPEExtension1": "1.0.0",
19     "extensions/MyExamplePDPEExtension2": "1.0.0",
20     "suitecommerce/Account": "2.3.0",
21     ...

```

7. Include the module definition (OrderWizard.Module.PaymentMethod.Creditcard.Extension) in the dependencies array of the javascript object within the SC.Checkout.Starter entryPoint. You must include this file after the module you are extending. Your distro.json file should look similar to the following:

```

1  //...
2  "tasksConfig": {
3    //...
4    "javascript": [
5      {
6        //...
7        "entryPoint": "SC.Checkout.Starter",
8        "exportFile": "checkout.js",
9        "dependencies": [
10         //...
11         "StoreLocator",
12         "OrderWizard.Module.PaymentMethod.Creditcard.Extension"
13       ],
14     //...

```

8. Save the distro.json file.

Step 3: Deploy Your Extension

1. Deploy your source code customizations to your NetSuite account and test the functionality. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.



Note: Since this patch modifies a SuiteScript file, changes are not visible in your local environment. SuiteScript files run on the server and must be deployed to NetSuite to take effect.

2. Confirm your results.

After a successful deployment, your site properly completes the purchase process using the credit card payment method.

Selected Invoice Not Displayed When Making an Invoice Payment

Applies to: SuiteCommerce Advanced | Mont Blanc

In some cases, users making an invoice payment experience an issue where the **Make a Payment** page does not display the selected invoice. Instead, users receive the following message:

You don't have any Open Invoices at the moment, see Invoices Paid in Full

Perform the following steps to resolve this problem:

- Extend the `LivePayment.Model.js` file
- Prepare the Developer Tools for Your Customizations
- Deploy your extension

You can download the code samples described in this procedure here: [MakePaymentPageNotDisplaying-MontBlanc.zip](#).

Step 1: Extend the `LivePayment.Model.js` File

1. The `LivePayment.Model.js` file is located within the `Modules/suitecommerce/LivePayment@x.x.x/` SuiteScript folder.
If you have not done so already, create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory.
Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your customizations.
Give this directory a unique name that is similar to the module being customized. For example:
`Modules/extensions/LivePayment.Extension@1.0.0/`
3. In your new module, create a subdirectory named **SuiteScript**.
4. In your SuiteScript subdirectory, create a new JavaScript file.
Give this file a unique name that is similar to the file being modified. For example:
`Modules/extensions/LivePayment.Extension@1.0.0/SuiteScript/LivePayment.Model.Extension.js`
5. Open this file and set it up to overwrite the `setInvoices()` method of the `LivePayment.Model.js` file.
Your file should match the following code snippet:

```

1  define(
2    'LivePayment.Model.Extension'
3    , ['LivePayment.Model', 'Utils', 'underscore']
4    , function (LivePaymentModel, Utils, _)
5    {
6      'use strict';
7
8      _._extend(LivePaymentModel.prototype,
9      {
10        setInvoices: function(customer_payment, result)
11        {
12          result.invoices = [];
13
14          var invoice_ids_to_search = [];
15

```

```

16     for (var i = 1; i <= customer_payment.getLineItemCount('apply'); i++)
17     {
18         if (customer_payment.getLineItemValue('apply', 'type', i) === 'Invoice')
19         {
20             var invoice = {
21                 internalid: customer_payment.getLineItemValue('apply', 'internalid', i)
22                 , total: Utils.toCurrency(customer_payment.getLineItemValue('apply', 'total', i))
23                 , total_formatted: Utils.formatCurrency(customer_payment.getLineItemValue('apply', 'total', i))
24                 , apply: customer_payment.getLineItemValue('apply', 'apply', i) === 'T'
25                 , applydate: customer_payment.getLineItemValue('apply', 'applydate', i)
26                 , currency: customer_payment.getLineItemValue('apply', 'currency', i)
27                 , discamt: Utils.toCurrency(customer_payment.getLineItemValue('apply', 'discamt', i))
28                 , discamt_formatted: Utils.formatCurrency(customer_payment.getLineItemValue('apply', 'discamt', i))
29                 , disc: Utils.toCurrency(customer_payment.getLineItemValue('apply', 'disc', i))
30                 , disc_formatted: Utils.formatCurrency(customer_payment.getLineItemValue('apply', 'disc', i))
31                 , discdate: customer_payment.getLineItemValue('apply', 'discdate', i)
32                 , amount: Utils.toCurrency(customer_payment.getLineItemValue('apply', 'amount', i))
33                 , amount_formatted: Utils.formatCurrency(customer_payment.getLineItemValue('apply', 'amount', i))
34                 , due: Utils.toCurrency(customer_payment.getLineItemValue('apply', 'due', i))
35                 , due_formatted: Utils.formatCurrency(customer_payment.getLineItemValue('apply', 'due', i))
36                 , tranid: customer_payment.getLineItemValue('apply', 'refnum', i)
37             };
38             result.invoices.push(invoice);
39             invoice_ids_to_search.push(invoice.internalid);
40         }
41     }
42
43     return result;
44 }
45
46 });
47 });

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Customizations

1. Open the LivePayment.Extension @1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.

Modules/extensions/LivePayment.Extension@1.0.0/ns.package.json

3. Build the ns.package.json file using the following code:

```

1  {
2    "gulp": {
3      "ssp-libraries": [
4        "SuiteScript/*.js"
5      ]
6    }
7  }

```

4. Save the ns.package.json file.
5. Open the distro.json file.

This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the modules object to ensure that the Gulp tasks include your extension when you deploy.

Your code should look similar to the following example:

```

1  {
2    "name": "SuiteCommerce Advanced Mont Blanc",
3    "version": "2.0",
4    "buildToolsVersion": "1.1.0",
5    "folders": {
6      "modules": "Modules",
7      "suitecommerceModules": "Modules/suitecommerce",

```

```

8      "thirdPartyModules": "Modules/third_parties",
9      "distribution": "LocalDistribution",
10     "deploy": "DeployDistribution"
11   },
12   "modules": {
13     "extensions/LivePayment.Extension": "1.0.0",
14     "suitecommerce/Account": "2.1.0",
15     //...

```

7. Add `LivePayment.Model.Extension` as a dependency to SCA entry point within the `ssp-libraries` object:

Your code should look similar to the following example:

```

1  //...
2      "ssp-libraries": {
3        "entryPoint": "SCA",
4        "dependencies": [
5          "Application",
6          //..
7          "LivePayment.Model.Extension"
8          //..
9        ],
10     //...

```

8. Save the `distro.json` file.

Step 3: Deploy Your Extension

1. Deploy your customizations to your NetSuite account. See the help topic [Deploy SCA Customizations to NetSuite](#).

Note: Since this patch modifies a SuiteScript file, changes are not visible in your local environment. SuiteScript files run on the server and must be deployed to NetSuite.

2. Confirm your results.

Upon successful deployment, invoices selected for payment should be visible on the Make a Payment page.

Log In to See Prices Message Appears When Users are Logged In

Applies to: SuiteCommerce Advanced | Elbrus

If you use the Log In to See Prices feature on your Elbrus implementation of SuiteCommerce Advanced, you might experience an issue where the application prompts users to log in to see prices even though they have successfully logged into your site.

To correct this issue, apply the patch described here. Due to the nature of this change, the best practice is to override the `SC.Configuration.js` file. You can download the code samples described in this procedure here: [LogInToSeePricesPatch--Elbrus.zip](#).

Important: These downloadable code samples assume that you have not made any previous customizations to the `SC.Configuration.js` file. The Override method refers to making changes by replacing the functionality of an entire file with your own custom version. This can potentially introduce errors when you deploy your code. Be aware of any existing customizations to this file and familiarize yourself with the [Best Practices for Customizing SCA](#) before overriding any files.

Step 1: Override the `SC.Configuration.js` File

1. Create a directory to store your custom module.

Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.

2. Open your extensions directory and create a custom module to maintain your customizations.

Give this directory a unique name that is similar to the module being customized. For example:

Modules/extensions/SCA.Extension@1.0.0/

3. In your new module, create a subdirectory named JavaScript.

4. Copy the source SC.Configuration.js file located in Modules > suitecommerce > SCA@x.y.z/JavaScript/ and paste into your new JavaScript directory (where x.y.z represents the version of the module in your implementation of SuiteCommerce Advanced).

For example, paste your copy of this into the following location:

Modules/extensions/SCA.Extension@1.0.0/JavaScript/SC.Configuration.js

5. Open locate the following lines:

```
1 | if(entry.placeholder)
2 | {
3 |   entry.text = '';
4 | }
5 | entry['class'] = 'header-menu-level' + entry.level + '-anchor';
```

6. Replace these lines with the following code:

```
1 | if(entry.dataTouchpoint) entry['data-touchpoint']=entry.dataTouchpoint;
2 | if(entry.dataHashtag) entry['data-hashtag']=entry.dataHashtag;
3 | if(entry.placeholder)
4 | {
5 |   entry.text = '';
6 | }
7 | entry['class'] = 'header-menu-level' + entry.level + '-anchor';
```

When you are finished, your custom file should include the following code:

```
1 | //...
2 |
3 | // navigation hierarchy bindings.
4 | _each(baseConfiguration.navigationData, function (entry)
5 | {
6 |   if (!entry)
7 |   {
8 |     return;
9 |   }
10 |  else
11 |  {
12 |    if(entry.dataTouchpoint) entry['data-touchpoint']=entry.dataTouchpoint;
13 |    if(entry.dataHashtag) entry['data-hashtag']=entry.dataHashtag;
14 |    if(entry.placeholder)
15 |    {
16 |      entry.text = '';
17 |    }
18 |    entry['class'] = 'header-menu-level' + entry.level + '-anchor';
19 |  }
20 |  if (entry.parentId)
21 |  {
22 |    var parent = _find(baseConfiguration.navigationData, function (e)
23 |    {
24 |      return e.id===entry.parentId;
25 |    });
26 |    parent = parent || {};
27 |    parent.categories = parent.categories || [];
28 |    parent.categories.push(entry);
29 |  }
30 |  if (entry.classnames)
```

```

31     {
32         entry['class'] += ' ' + entry.classnames;
33     }
34 });
35
36 //...

```

7. Save the file.

Step 2: Prepare the Developer Tools for Your Override

1. Open the SCA.Extension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/SCA.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code, where x.y.z equals the version of the module you are overriding.

```

1  {
2      "gulp": {
3          "javascript": [
4              "JavaScript/*.js"
5          ]
6      },
7      "overrides": {
8          "suitecommerce/SCA@x.y.z/JavaScript/SC.Configuration.js" : "JavaScript/SC.Configuration.js"
9      }
10 }

```

Note: Replace the string x.y.z in the above example with the version of the module in your version of SuiteCommerce Advanced.

4. Save the ns.package.json file.
5. Open the distro.json file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.
6. Add your module to the modules object to ensure that the Gulp tasks include it when you deploy.
Your code should look similar to the following example:

```

1  {
2      "name": "SuiteCommerce Advanced Elbrus",
3      "version": "2.0",
4      "buildToolsVersion": "1.3.0",
5      "folders": {
6          "modules": "Modules",
7          "suitecommerceModules": "Modules/suitecommerce",
8          "extensionsModules": "Modules/extensions",
9          "thirdPartyModules": "Modules/third_parties",
10         "distribution": "LocalDistribution",
11         "deploy": "DeployDistribution"
12     },
13     "modules": {
14         "extensions/SCA.Extension": "1.0.0",
15         "extensions/MyExampleCartExtension1": "1.0.0",
16         //...

```

7. Save the file.

Step 3:

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA](#))

[Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

Upon successful deployment, the application no longer requests that logged-in users need to log in to see prices.

Item Record HTML Meta Data Not Appearing on Page Meta Data

Applies to: SuiteCommerce Advanced | Elbrus

Elbrus implementations of SuiteCommerce Advanced experience an issue where the value of an Item record's **Meta Tag HTML** field does not appear in the page's meta data. To correct this issue, create a custom module to extend `ProductDetails.Base.Views.js` as described in this section.

In addition to making these changes, you must create an `ns.package.json` file and update your `distro.json` file for any custom modules you include. You can download the code samples described in this procedure here: [HTMLMetaTagsNotAppearingPatch--Elbrus.zip](#).

Step 1: Extend the `ProductDetails.Base.Views.js` File

1. If you have not done so already, create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your customizations.
Give this directory a unique name that is similar to the module being customized. For example:
`Modules/extensions/ProductDetails.Extension@1.0.0/`
3. In your new module, create a subdirectory named `JavaScript`.
4. In your `JavaScript` subdirectory, create a new JavaScript file.
Give this file a unique name that is similar to the file being modified. For example:
`Modules/extensions/ProductDetails.Extension@1.0.0/ProductDetails.Base.View.Extension.js`
5. Open this file and set it up to overwrite the `getMetaTags()` method of the `ProductDetails.Base.View.js` file.

Your file should match the following code snippet:

```

1 | define(
2 |   'ProductDetails.Base.View.Extension'
3 | , [
4 |   'ProductDetails.Base.View'
5 |
6 |   , 'jQuery'
7 |   , 'underscore'
8 | ]
9 | , function (
10 |   ProductDetailsBaseView
11 |
12 |   , jQuery
13 |   , -
14 | )
15 | {
16 |   'use strict';
17 |
18 |   return {
19 |     mountToApp: function ()
20 |     {

```

```

21     __.extend(ProductDetailsBaseView.prototype,
22     {
23         getMetaTags: function getMetaTags ()
24         {
25             return jQuery('<head/>').html(
26                 jQuery.trim(
27                     this.model.get('item').get('_metaTags')
28                 )
29             ).children('meta');
30         }
31     });
32 }
33 }
34 });

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Customizations

1. Open the ProductDetails.Extension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/ProductDetails.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```

1  {
2      "gulp": {
3          "javascript": [
4              "JavaScript/*.js"
5          ]
6      }
7  }

```

4. Save the ns.package.json file.
5. Open the distro.json file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.
6. Add your custom module to the modules object to ensure that the Gulp tasks include it when you deploy.

Your code should look similar to the following example:

```

1  {
2      "name": "SuiteCommerce Advanced Elbrus",
3      "version": "2.0",
4      "buildToolsVersion": "1.3.0",
5      "folders": {
6          "modules": "Modules",
7          "suitecommerceModules": "Modules/suitecommerce",
8          "extensionsModules": "Modules/extensions",
9          "thirdPartyModules": "Modules/third_parties",
10         "distribution": "LocalDistribution",
11         "deploy": "DeployDistribution"
12     },
13     "modules": {
14         "extensions/ProductDetails.Extension": "1.0.0",
15         "extensions/MyExampleCartExtension1": "1.0.0",
16         //...

```

7. Include the module definition (ProductDetails.Base.View.Extension) as a dependency within the JavaScript SC.Shopping.Starter entry point.

Your distro.json file should look similar to the following:

```

1  "tasksConfig": {
2      //...

```

```

3      "javascript": [
4        {
5          "entryPoint": "SC.Shopping.Starter",
6          "exportFile": "shopping.js",
7          "dependencies": [
8            //...
9            "StoreLocator",
10           "ProductDetails.Base.View.Extension"
11         ]
12       }
13     ]
14     //...

```

8. Save the distro.json file.

Step 3: Test and Deploy Your Extension

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, your Item record Meta Tag HTML value should appear in the meta data within the DOM of the associated product details page.

Delivery Options Not Appearing After Editing the Cart and Re-entering a Shipping Address

i Applies to: SuiteCommerce Advanced | Elbrus

In some cases, delivery method options do not display after editing the cart. If a user performs the following series of events, the delivery method options do not display properly:

i Note: This issue applies to Site Builder Extensions implementations only.

1. The user adds an item to the cart and proceeds to checkout.
2. The user enters a shipping address, including a zip code.
3. The user clicks out of the zip code field and clicks **Edit Cart**.
4. Without making any changes, the user again proceeds to Checkout.
5. The user enters the same shipping information, including the same zip code.

If you experience this issue on your Site Builder Extensions implementation, create a custom module to extend OrderWizard.Module.Shipmethod.js as described in this section.

In addition to making these changes, you must create an ns.package.json file and update your distro.json file for any custom modules you include. You can download the code samples described in this procedure here: [DeliverMethodsNotAppearingPatch---SiteBuilderExtensions.zip](#).

Step 1: Extend the OrderWizard.Module.Shipmethod.js File

1. If you have not done so already, create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your customizations.
Give this directory a unique name that is similar to the module being customized. For example:
Modules/extensions/OrderWizard.Module.Shipmethod.Extension@1.0.0/

3. In your new module, create a subdirectory named JavaScript.
4. In your JavaScript subdirectory, create a new JavaScript file.
Give this file a unique name that is similar to the file being modified. For example:
Modules/extensions/OrderWizard.Module.Shipmethod.Extension@1.0.0/
OrderWizard.Module.Shipmethod.Extension.js
5. Open this file and set it up to overwrite the shipAddressChange() method of the OrderWizard.Module.Shipmethod.js file.
Your file should match the following code snippet:

```

1  define(
2    'OrderWizard.Module.Shipmethod.Extension'
3    , [
4
5      'OrderWizard.Module.Shipmethod'
6
7      , 'jQuery'
8      , 'underscore'
9    ]
10
11    , function (
12      OrderWizardModuleShipmethod
13
14      , jQuery
15      , _
16    )
17  {
18
19    'use strict';
20
21    _.extend(OrderWizardModuleShipmethod.prototype,
22    {
23      shipAddressChange: function (model, value)
24      {
25        {
26          this.currentAddress = value;
27
28          var order_address = this.model.get('addresses')
29          , previous_address = this.previousAddress && (order_address.get(this.previousAddress) || this.address
30es.get(this.previousAddress))
31          , current_address = this.currentAddress && order_address.get(this.currentAddress) || this.address
32es.get(this.currentAddress)
33          , changed_zip = previous_address && current_address && previous_address.get('zip') !== current_ad
34dress.get('zip')
35          , changed_state = previous_address && current_address && previous_address.get('state') !== current_ad
36dress.get('state')
37          , changed_country = previous_address && current_address && previous_address.get('country') !== current_ad
38dress.get('country');
39
40          // if previous address is equal to current address we compare the previous values on the model.
41          if (this.previousAddress && this.currentAddress && this.previousAddress === this.currentAddress)
42          {
43            changed_zip = current_address.previous('zip') !== current_address.get('zip');
44            changed_country = current_address.previous('country') !== current_address.get('country');
45            changed_state = current_address.previous('state') !== current_address.get('state');
46          }
47
48          // reload ship methods only if there is no previous address or when change the country or zipcode
49          if ((!previous_address && current_address) || changed_zip || changed_country || changed_state)
50          {
51            // if its selected a valid address, reload Methods
52            if (this.model.get('isEstimating') || this.addresses.get(this.model.get('shipaddress')))
53            {
54              this.reloadMethods();
55            }
56          }
57          else
58          {
59            this.render();
60          }
61        }
62      }
63    });
64  }
65  );

```

```

55     }
56
57     if (value)
58     {
59         this.previousAddress = value;
60     }
61
62     // if we select a new address, bind the sync method for possible address edits
63     if (this.currentAddress)
64     {
65         var selected_address = this.addresses.get(this.currentAddress);
66         if(selected_address)
67         {
68             selected_address.on('sync', jQuery.proxy(this, 'reloadMethods'), this);
69         }
70
71         // if there was a different previous address, remove the sync handler
72         if(this.previousAddress && this.previousAddress !== this.currentAddress)
73         {
74             var previous_selected_address = this.addresses.get(this.previousAddress);
75             if(previous_selected_address)
76             {
77                 previous_selected_address.off('sync');
78             }
79         }
80     }
81 }
82 });
83 });

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Customizations

1. Open the OrderWizard.Module.Shipmethod.Extension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/OrderWizard.Module.Shipmethod.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```

1  {
2      "gulp": {
3          "javascript": [
4              "JavaScript/*.js"
5          ]
6      }
7  }

```

4. Save the ns.package.json file.
5. Open the distro.json file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.
6. Add your custom module to the modules object to ensure that the Gulp tasks include it when you deploy.

Your code should look similar to the following example:

```

1  {
2      "name": "SuiteCommerce Site Builder - Elbrus",
3      "version": "2.0",
4      "buildToolsVersion": "1.3.0",
5      "folders": {
6          "modules": "Modules",
7          "suitecommerceModules": "Modules/suitecommerce",
8          "thirdPartyModules": "Modules/third_parties",
9          "distribution": "LocalDistribution",
10         "deploy": "DeployDistribution"
11     },

```

```

12     "modules": {
13         "extensions/OrderWizard.Module.Shipmethod.Extension": "1.0.0",
14         "suitecommerce/Account": "2.3.0",
15         //...

```

7. Include the module definition (OrderWizard.Module.Shipmethod.Extension) as a dependency within the JavaScript SC.Checkout.Starter entry point.

Your distro.json file should look similar to the following:

```

1  "tasksConfig": {
2    //...
3    "javascript": [
4      {
5        "entryPoint": "SC.Checkout.Starter",
6        "exportFile": "checkout.js",
7        "dependencies": [
8          //...
9          "OrderWizard.Module.Shipmethod.Extension"
10       ]
11     }
12   ]
13   //...

```

8. Save the distro.json file.

Step 3: Test and Deploy Your Extension

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

Upon successful deployment, all delivery options will appear after editing the cart and re-entering a shipping address.

Order Confirmation and Thank You Page is Blank

Applies to: SuiteCommerce Advanced | Mont Blanc | Vinson

In some cases, an Order Confirmation and Thank You page might not appear after a user places an order in your web store. If you experience this issue on your SuiteCommerce Advanced site, perform the following steps to correct the problem.

The suggested action involves extending the LiveOrderModel.js file (within the SuiteScript folder) to overwrite the confirmationCreateResult() method.

In addition to making these changes, you must create an ns.package.json file and update your distro.json file for any custom modules you include. You can download the code samples described in this procedure here: [LiveOrderModelThankYouPagePatch.zip](#).

Step 1: Extend the LiveOrderModel.js File

1. If you have not done so already, create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your customizations.
Give this directory a unique name that is similar to the module being customized. For example:
Modules/extensions/LiveOrder.Extension@1.0.0/

3. In your new module, create a subdirectory named SuiteScript.
4. In your SuiteScript subdirectory, create a new JavaScript file.

Give this file a unique name that is similar to the file being modified. For example:

Modules/extensions/LiveOrder.Extension@1.0.0/SuiteScript/LiveOrder.Model.Extension.js

5. Open this file and set it up to overwrite the `confirmationCreateResult()` method of the `LiveOrder.Model.js` file.

Your file should match the following code snippet:

```

1  define(
2    'LiveOrder.Model.Extension'
3    , [
4
5      'LiveOrder.Model'
6      , 'Utils'
7
8      , 'underscore'
9    ]
10
11    , function (
12
13      LiveOrderModel
14      , Utils
15
16      , -
17    )
18  {
19
20    'use strict';
21
22    _._extend(LiveOrderModel.prototype,
23    {
24      confirmationCreateResult: function (placed_order)
25      {
26        var result = {
27          internalid: placed_order.getId()
28          , tranid: placed_order.getFieldValue('tranid')
29          , summary: {
30            subtotal: Utils.toCurrency(placed_order.getFieldValue('subtotal'))
31            , subtotal_formatted: Utils.formatCurrency(placed_order.getFieldValue('subtotal'))
32
33            , taxtotal: Utils.toCurrency(placed_order.getFieldValue('taxtotal'))
34            , taxtotal_formatted: Utils.formatCurrency(placed_order.getFieldValue('taxtotal'))
35
36            , shippingcost: Utils.toCurrency(placed_order.getFieldValue('shippingcost'))
37            , shippingcost_formatted: Utils.formatCurrency(placed_order.getFieldValue('shippingcost'))
38
39            , handlingcost: Utils.toCurrency(placed_order.getFieldValue('alhandlingcost'))
40            , handlingcost_formatted: Utils.formatCurrency(placed_order.getFieldValue('alhandlingcost'))
41
42
43            , discounttotal: Utils.toCurrency(placed_order.getFieldValue('discounttotal'))
44            , discounttotal_formatted: Utils.formatCurrency(placed_order.getFieldValue('discounttotal'))
45
46
47            , giftcertapplied: Utils.toCurrency(placed_order.getFieldValue('giftcertapplied'))
48            , giftcertapplied_formatted: Utils.formatCurrency(placed_order.getFieldValue('giftcertapplied'))
49
50            , total: Utils.toCurrency(placed_order.getFieldValue('total'))
51            , total_formatted: Utils.formatCurrency(placed_order.getFieldValue('total'))
52          }
53        };
54
55        result.promocode = (placed_order.getFieldValue('promocode')) ? {
56          internalid: placed_order.getFieldValue('promocode')
57          , name: placed_order.getFieldText('promocode')
58          , code: placed_order.getFieldText('couponcode')
59          , invalid: true
60        } : null;

```

```

61     result.paymentmethods = [];
62     for (var i = 1; i <= placed_order.getLineItemCount('giftcertredemption'); i++)
63     {
64         result.paymentmethods.push({
65             type: 'giftcertificate'
66             , giftcertificate: {
67                 code: placed_order.getLineItemValue('giftcertredemption', 'authcode_display', i)
68                 , amountapplied: placed_order.getLineItemValue('giftcertredemption', 'authcodeapplied', i)
69                 , amountapplied_formatted: Utils.formatCurrency(placed_order.getLineItemValue('giftcertredemp
70 tion', 'authcodeapplied', i))
71                 , amountremaining: placed_order.getLineItemValue('giftcertredemption', 'authcodeamtremaining', i)
72                 , amountremaining_formatted: Utils.formatCurrency(placed_order.getLineItemValue('giftcertredemp
73 tion', 'authcodeamtremaining', i))
74                 , originalamount: placed_order.getLineItemValue('giftcertredemption', 'giftcertavailable', i)
75                 , originalamount_formatted: Utils.formatCurrency(placed_order.getLineItemValue('giftcertredemp
76 tion', 'giftcertavailable', i))
77             }
78         });
79     }
80     result.lines = [];
81     for (var i = 1; i <= placed_order.getLineItemCount('item'); i++)
82     {
83         result.lines.push({
84             item: {
85                 internalid: placed_order.getLineItemValue('item', 'item', i)
86                 // 'item_display' returns the 'sku' or if is a matrix returns 'sku_parent : sku_child'
87                 // getLineItemText of 'item_display' returns the same as getLineItemText of 'item'
88                 , itemDisplay: placed_order.getLineItemText('item', 'item', i)
89             }
90             , quantity: parseInt(placed_order.getLineItemValue('item', 'quantity', i), 10)
91             , rate: parseInt(placed_order.getLineItemValue('item', 'rate', i), 10)
92             , options: placed_order.getLineItemValue('item', 'options', i)
93         });
94     }
95     return result;
96 }
97 });
98 });
99 });

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Customizations

1. Open the LiveOrder.Extension @1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/LiveOrder.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```

1  {
2      "gulp": {
3          "ssp-libraries": [
4              "SuiteScript/*.js"
5          ]
6      }
7  }

```

4. Save the ns.package.json file.
5. Open the distro.json file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.
6. Add your custom module to the modules object to ensure that the Gulp tasks include your extension when you deploy.
Your code should look similar to the following example:


```

1  {
2    "name": "SuiteCommerce Advanced Mont Blanc",
3    "version": "2.0",
4    "buildToolsVersion": "1.1.0",
5    "folders": {
6      "modules": "Modules",
7      "suitecommerceModules": "Modules/suitecommerce",
8      "thirdPartyModules": "Modules/third_parties",
9      "distribution": "LocalDistribution",
10     "deploy": "DeployDistribution"
11   },
12   "modules": {
13     "extensions/LiveOrder.Extension": "1.0.0",
14     "suitecommerce/Account": "2.1.0",
15     //...

```

7. Add `LiveOrder.Model.Extension` as a dependency to SCA entry point within the `ssp-libraries` object:

Your code should look similar to the following example:

```

1  //...
2    "ssp-libraries": {
3      "entryPoint": "SCA",
4      "dependencies": [
5        "Application",
6        //...
7        "LiveOrder.Model.Extension"
8      ],
9    //...

```

8. Save the `distro.json` file.

Step 3: Deploy Your Extension

1. Deploy your customizations to your NetSuite account. See the help topic [Deploy SCA Customizations to NetSuite](#).

Note: Since this patch modifies an SSP library file, changes are not visible in your local environment. SuiteScript files run on the server and must be deployed to NetSuite.

2. Confirm your results.

Upon successful deployment, completed orders should result in displaying an Order Confirmation and Thank You page.

Matrix Item Options Not Displaying With Google Tag Manager Enabled

Applies to: SuiteCommerce Advanced | Elbrus

In some cases, not all selected matrix item options appear in the Product Details Page. This can occur on sites using Google Tag Manager. To correct this error, apply the patch described here to extend `GoogleTagManager.js` and overwrite the `trackProductView()` method.

In addition to making these changes, you must create an `ns.package.json` file and update your `distro.json` file for any custom modules you include. You can download the code samples described in this procedure here: [MatrixItemDisplayGTM.zip](#).

Step 1: Extend the `GoogleTagManager.js` File

1. If you have not done so already, create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your customizations.
Give this directory a unique name that is similar to the module being customized. For example:
Modules/extensions/GoogleTagManager.Extension@1.0.0/
3. In your new module, create a subdirectory named JavaScript.
4. In your JavaScript subdirectory, create a new JavaScript file.
Give this file a unique name that is similar to the file being modified. For example:
Modules/extensions/GoogleTagManager.Extension@1.0.0/JavaScript/
GoogleTagManager.Extension.js
5. Open this file and set it up to overwrite the trackProductView() method of the GoogleTagManager.js file.

Your file should match the following code snippet:

```

1  define(
2    'GoogleTagManager.Extension'
3    , [
4      'GoogleTagManager'
5      , 'Tracker'
6      , 'underscore'
7    ]
8
9    , function (
10     GoogleTagManager
11     , Tracker
12     , _
13   )
14
15   {
16
17     'use strict';
18
19     _.extend(GoogleTagManager.prototype,
20       {
21         trackProductView: function (product)
22         {
23           var item = product.getItem();
24
25           if (this.item && this.item.get('itemId') === item.get('_id'))
26           {
27             item.set('category', this.item.get('category'), { silent: true });
28             item.set('list', this.item.get('list'), { silent: true });
29           }
30
31           var eventName = 'productView'
32           , selected_options = product.get('options').filter(function (option)
33             {
34               return option.get('value') && option.get('value').label;
35             })
36           , price = product.getPrice()
37           , eventData = {
38             'event': eventName
39           , 'data': {
40             'sku': product.getSku()
41             , 'name': item.get('_name')
42             , 'variant': _.map(selected_options, function (option)
43               {
44                 return option.get('value').label;
45               }).join(', ')
46             , 'price': ((price.price) ? price.price : 0).toFixed(2)
47             , 'category': item.get('category') || '' // as we do not support categories this is just the url
48             , 'list': item.get('list') || ''

```

```

49         , 'page': this.getCategory()
50       }
51     };
52
53     this.item = null;
54
55     //Triggers a Backbone.Event so others can subscribe to this event and add/replace data before is send it to
Google Tag Manager
56     Tracker.trigger(eventName, eventData, item);
57     this.pushData(eventData);
58
59     return this;
60   }
61   });
62 });

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Customizations:

1. Open the GoogleTagManager.Extension @1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/GoogleTagManager.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```

1  {
2    "gulp": {
3      "javascript": [
4        "JavaScript/*.js"
5      ]
6    }
7  }

```

4. Save the ns.package.json file.
5. Open the distro.json file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.
6. Add your custom module to the modules object to ensure that the Gulp tasks include your extension when you deploy.
Your code should look similar to the following example:

```

1  {
2    "name": "SuiteCommerce Advanced Elbrus",
3    "version": "2.0",
4    "buildToolsVersion": "1.3.0",
5    "folders": {
6      "modules": "Modules",
7      "suitecommerceModules": "Modules/suitecommerce",
8      "extensionsModules": "Modules/extensions",
9      "thirdPartyModules": "Modules/third_parties",
10     "distribution": "LocalDistribution",
11     "deploy": "DeployDistribution"
12   },
13   "modules": {
14     "extensions/GoogleTagManager.Extension": "1.0.0",
15     "extensions/MyExampleCartExtension1": "1.0.0",
16     //...

```

7. Add GoogleTagManager.Extension as a dependency to the following entry points of the JavaScript object:
 - SC.Shopping.Starter
 - SC.MyAccount.Starter
 - SC.Checkout.Starter

Your code should look similar to the following example:

```

1  //...
2  "javascript": [
3    {
4      "entryPoint": "SC.Shopping.Starter",
5      "exportFile": "shopping.js",
6      "dependencies": [
7        "Backbone.View.Plugins",
8        //...
9        "GoogleTagManager.Extension"
10     ],
11     //...
12   },
13   {
14     "entryPoint": "SC.MyAccount.Starter",
15     "exportFile": "myaccount.js",
16     "dependencies": [
17       "Backbone.View.Plugins",
18       //...
19       "GoogleTagManager.Extension"
20     ],
21     //...
22   },
23   {
24     "entryPoint": "SC.Checkout.Starter",
25     "exportFile": "checkout.js",
26     "dependencies": [
27       "Backbone.View.Plugins",
28       //...
29       "GoogleTagManager.Extension"
30     ],
31     //...
32   },
33   //...

```

8. Save the distro.json file.

Step 3: Test and Deploy Your Extension:

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, all selected matrix item options should appear in the Product Details Page.

Delivery Methods Not Appearing in One Page Checkout

i Applies to: SuiteCommerce Advanced | Mont Blanc | Vinson | Elbrus

In some cases, not all delivery methods appear when using the One Page Checkout flow. This issue sometimes occurs after a user enters a zip code when checking out as a guest.

To correct this error, extend the `mountToApp()` method located in the `CheckoutSkipLogin.js` file and hide the following line within comment tags:

```
data.cart && LiveOrderModel.getInstance().set(data.cart);
```

In addition to making these changes, you must create an `ns.package.json` file and update your `distro.json` file for any custom modules you include. You can download the code samples described in this procedure here: [OnePageCheckoutDeliveryMethodsPatch.zip](#).

Step 1: Extend the CheckoutSkipLogin.js File

1. If you have not done so already, create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your customizations.
Give this directory a unique name that is similar to the module being customized. For example:
Modules/extensions/CheckoutSkipLogin.Extension@1.0.0/
3. In your new module, create a subdirectory named JavaScript.
4. In your JavaScript subdirectory, create a new JavaScript file.
Give this file a unique name that is similar to the file being modified. For example:
Modules/extensions/CheckoutSkipLogin.Extension@1.0.0/JavaScript/CheckoutSkipLogin.Extension.js
5. Open this file and set it up to extend the mountToApp method of the CheckoutSkipLogin.js file.
Your file should match the following code snippet:

```

1  define(
2    'CheckoutSkipLogin.Extension'
3  , [
4
5    'CheckoutSkipLogin'
6    , 'Account.RegisterAsGuest.Model'
7
8    , 'jQuery'
9    , 'underscore'
10 ]
11
12 , function (
13   CheckoutSkipLogin
14   , AccountRegisterAsGuestModel
15
16   , jQuery
17   , _
18 )
19 {
20 {
21
22   'use strict';
23
24   _.extend(CheckoutSkipLogin.prototype,
25   {
26
27     mountToApp: function(application)
28     {
29       // do nothing if the mode is disabled
30       if (!application.getConfig('checkout.skipLogin'))
31       {
32         return;
33       }
34
35       //this function is called only if skip login mode is enabled
36       var registerUserAsGuest = function ()
37       {
38         var promise = jQuery.Deferred()
39         , profile_model = ProfileModel.getInstance();
40         if (profile_model.get('isGuest') === 'F' && profile_model.get('isLoggedIn') === 'F')
41         {
42           var checkoutStep = application.getLayout().currentView.currentStep;
43
44           checkoutStep && checkoutStep.disableNavButtons();
45
46           new AccountRegisterAsGuestModel().save().done(function(data)
47           {
48             var skipLoginDontUpdateProfile = profile_model.get('skipLoginDontUpdateProfile');

```

```

49         if(skipLoginDontUpdateProfile && data.user)
50         {
51             _.each(skipLoginDontUpdateProfile, function(attr)
52             {
53                 delete data.user[attr];
54             });
55         }
56     }
57
58     data.user && profile_model.set(data.user);
59     application.getLayout().currentView.wizard.options.profile = profile_model;
60     //data.cart && LiveOrderModel.getInstance().set(data.cart);
61     data.touchpoints && (application.Configuration.siteSettings.touchpoints = data.touchpoints);
62     promise.resolve();
63     checkoutStep && checkoutStep.enableNavButtons();
64     jQuery('[data-action="skip-login-message"]').remove();
65 });
66 }
67 else
68 {
69     promise.resolve();
70 }
71 return promise;
72 };
73
74 // add 'this.application' to models that doesn't have it.
75 AddressModel.prototype.application = application;
76 CreditCardModel.prototype.application = application;
77 ProfileModel.prototype.application = application;
78
79 // wrap save() method to LiveOrderModel, AddressModel and CreditCardModel
80 var wrapper = function(superFn)
81 {
82     var self = this
83     ,   super_arguments = Array.prototype.slice.apply(arguments, [1, arguments.length])
84     ,   promise = jQuery.Deferred();
85
86     if (!promiseGuest)
87     {
88         promiseGuest = registerUserAsGuest();
89     }
90
91     promiseGuest.done(function ()
92     {
93         var result = superFn.apply(self, super_arguments);
94
95         if (result)
96         {
97             result.done(function ()
98             {
99                 promise.resolve.apply(result, arguments);
100             }).fail(function()
101             {
102                 promise.reject.apply(result, arguments);
103             });
104         }
105         else
106         {
107             // Notify future promises that a front end validation took place and no promise is returned
108             promise.frontEndValidationError = true;
109             promise.reject.apply(result, super_arguments);
110         }
111     });
112
113     _(promise).extend({error: function(){return this;},success: function(){return this;}});
114
115     return promise;
116 };
117
118 // don't wrap on non-secure domains (Site Builder cart is in Checkout :/ )
119 if (window.location.protocol !== 'http:')
120 {
121     LiveOrderModel.prototype.save = _.wrap(LiveOrderModel.prototype.save, wrapper);

```

```

122     }
123
124     AddressModel.prototype.save = _wrap(AddressModel.prototype.save, wrapper);
125
126     CreditCardModel.prototype.save = _wrap(CreditCardModel.prototype.save, wrapper);
127
128     ProfileModel.prototype.save = _wrap(ProfileModel.prototype.save, wrapper);
129     }
130
131     });
132 });

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Customizations

1. Open the CheckoutSkipLogin.Extension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/CheckoutSkipLogin.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```

1 {
2   "gulp": {
3     "javascript": [
4       "JavaScript/*.js"
5     ]
6   }
7 }

```

4. Save the ns.package.json file.
5. Open the distro.json file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.
6. Add your custom module to the modules object to ensure that the Gulp tasks include your extension when you deploy.

Your code should look similar to the following example:

```

1 {
2   "name": "SuiteCommerce Advanced Mont Blanc",
3   "version": "2.0",
4   "buildToolsVersion": "1.1.0",
5   "folders": {
6     "modules": "Modules",
7     "suitecommerceModules": "Modules/suitecommerce",
8     "thirdPartyModules": "Modules/third_parties",
9     "distribution": "LocalDistribution",
10    "deploy": "DeployDistribution"
11  },
12  "modules": {
13    "extensions/CheckoutSkipLogin.Extension": "1.0.0",
14    "suitecommerce/Account": "2.1.0",
15    //...

```

7. Add CheckoutSkipLogin.Extension as a dependency to the SC.Checkout.Starter entry point of the JavaScript object.

Your code should look similar to the following example:

```

1 //...
2 "javascript": [
3   //...
4   {
5     "entryPoint": "SC.Checkout.Starter",

```

```

6      "exportFile": "checkout.js",
7      "dependencies": [
8          "Backbone.View.Plugins",
9          //...
10         "CheckoutSkipLogin.Extension"
11     ],
12 //...

```

8. Save the distro.json file.

Step 3: Test and Deploy Your Extension

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.
Upon successful deployment, entering a zip code as a guest displays all delivery method options.

Mastercard 2-Series BIN Regex Patch

i Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson

The Elbrus release of SuiteCommerce Advanced and later support the latest Mastercard 2-Series BIN range by default. To provide this capability for implementations prior to the Elbrus release, you need to perform the updates described in the section corresponding to your implementation:

- [Denali – Mastercard Regex Patch](#)
- [Mont Blanc – Mastercard Regex Patch](#)
- [Vinson – Mastercard Regex Patch](#)

Denali – Mastercard Regex Patch

i Applies to: SuiteCommerce Advanced | Denali

In your Denali implementation, the credit card regex values reside in the Utils.js file, which is located in the Utilities module. To update this module, redefine the `paymentMethodIdCreditCard()` method as described in this section. You can download the code samples described in this procedure here: [MastercardBinRegex---DenaliCodeSamples.zip](#).

Step 1: Extend the Utils.js File

1. If you have not done so already, create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your customizations.
Give this directory a unique name that is similar to the module being customized. For example: `Modules/extensions/Utilities.Extension@1.0.0/`
3. In your new Utilities.Extension@1.0.0 module, create a subdirectory called JavaScript.
4. In your JavaScript subdirectory, create a new JavaScript file.
Give this file a unique name that is similar to the file being modified. For example:

Modules/extensions/Utilities.Extension@1.0.0/JavaScript/Utils.Extension.js

5. Open this file and set it up to redefine the paymentmethodIdCreditCard method of the Utils.js file.



Important: This must be wrapped inside the mountToApp() method.

Your file should match the following code snippet:

```

1  define(
2    'Utils.Extension'
3    , [
4      'Utils'
5
6      , 'underscore'
7    ]
8    , function (
9      Utils
10
11      , -
12    )
13  {
14
15    'use strict';
16
17    return {
18      mountToApp: function ()
19      {
20        Utils.prototype.paymentmethodIdCreditCard=
21
22        function paymentmethodIdCreditCard (cc_number)
23        {
24          // regex for credit card issuer validation
25          var cards_reg_ex = {
26            'VISA': /^4[0-9]{12}(?:[0-9]{3})?$/,
27            'Master Card': /^(5[1-5][0-9]{14}|2(2[1-9]|[3-9][0-9])|3-6[0-9][0-9]|7([0-1][0-9]|20))[0-9]{12})$/ //
previous value: /^5[1-5][0-9]{14}$/
28            , 'American Express': /^3[47][0-9]{13}$/
29            , 'Discover': /^6(?:011|5[0-9]{2})[0-9]{12}$/
30            , 'Maestro': /^(?:5[0678]\d\d|6304|6390|67\d\d)\d{8,15}$/
31          }
32
33          // get the credit card name
34          , paymentmethod_name;
35
36          // validate that the number and issuer
37          _.each(cards_reg_ex, function (reg_ex, name)
38          {
39            if (reg_ex.test(cc_number))
40            {
41              paymentmethod_name = name;
42            }
43          });
44
45          var paymentmethod = paymentmethod_name && _.findWhere(SC.ENVIRONMENT.siteSettings.paymentmethods, {name: paymentmethod_name.toString()});
46
47          return paymentmethod && paymentmethod.internalid;
48        }
49      }
50    };
51  });

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Customizations

1. Open the Utilities.Extension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.

Modules/extensions/Utilities.Extension@1.0.0/ns.package.json

3. Build the ns.package.json file using the following code

```

1 {
2   "gulp": {
3     "javascript": [
4       "JavaScript/*.js"
5     ]
6   }
7 }
```

4. Save the ns.package.json file.

5. Open the distro.json file.

This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the modules object to ensure that the Gulp tasks include your extension when you deploy.

Your code should look similar to the following example:

```

1 {
2   "name": "SuiteCommerce Advanced Mont Blanc",
3   "version": "2.0",
4   "buildToolsVersion": "1.1.0",
5   "folders": {
6     "modules": "Modules",
7     "suitecommerceModules": "Modules/suitecommerce",
8     "thirdPartyModules": "Modules/third_parties",
9     "distribution": "LocalDistribution",
10    "deploy": "DeployDistribution"
11  },
12  "modules": {
13    "extensions/Utilities.Extension": "1.0.0",
14    "suitecommerce/Account": "2.1.0",
15    //...
```

7. Add Utils.Extension as a dependency to the following entry points of the JavaScript object:

- SC.Shopping.Starter
- SC.MyAccount.Starter
- SC.Checkout.Starter

Your code should look similar to the following example:

```

1 //...
2 "javascript": [
3   {
4     "entryPoint": "SC.Shopping.Starter",
5     "exportFile": "shopping.js",
6     "dependencies": [
7       "Backbone.View.Plugins",
8       //...
9       "Utils.Extension"
10    ],
11    //...
12  },
13  {
14    "entryPoint": "SC.MyAccount.Starter",
15    "exportFile": "myaccount.js",
16    "dependencies": [
17      "Backbone.View.Plugins",
18      //...
19      "Utils.Extension"
20    ],
21    //...
22  },
23  //...
```

```

23 | {
24 |     "entryPoint": "SC.Checkout.Starter",
25 |     "exportFile": "checkout.js",
26 |     "dependencies": [
27 |         "Backbone.View.Plugins",
28 |         //...
29 |         "Utils.Extension"
30 |     ],
31 |     //...
32 | },
33 | //...

```

8. Save the distro.json file.

Step 3: Test and Deploy Your Extension

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, the new Mastercard regex should be included in your compiled shopping.js, myaccount.js, and checkout.js files.

Mont Blanc – Mastercard Regex Patch

Applies to: SuiteCommerce Advanced | Mont Blanc

In your Mont Blanc implementation, the credit card regex values reside in the SC.Configuration.js file, which is located in the SCA module. To update this module, extend the paymentmethods array as described in this section. You can download the code samples described in this procedure here: [MastercardBinRegex---MontBlancCodeSample.zip](#).

Step 1: Extend the SC.Configuration.js File

1. If you have not done so already, create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your configuration modifications.
Give this directory a unique name that is similar to the module being customized. For example:
Modules/extensions/SCA.Extension@1.0.0/
3. In your new SCA.Extension@1.0.0 module, create a subdirectory called JavaScript.
4. In your JavaScript subdirectory, create a new JavaScript file.
Give this file a unique name that is similar to the file being modified. For example:
Modules/extensions/SCA.Extension@1.0.0/JavaScript/SC.Configuration.Extension.js
5. Open this file and set it up to extend the paymentmethods array of the SC.Configuration.js file.



Important: This must be wrapped inside the mountToApp() method.

Your file should match the following code snippet:

```

1  define(
2    'SC.Configuration.Extension'
3    , [
4
5      'SC.Configuration'
6
7      , 'underscore'
8    ]
9
10   , function (
11
12     Configuration
13
14     , -
15   )
16 {
17
18   'use strict';
19
20   return {
21     mountToApp: function ()
22     {
23       _.extend(Configuration,
24         {
25           paymentmethods: [
26             {
27               key: '5,5,1555641112' // 'VISA'
28               , regex: /^4[0-9]{12}(?:[0-9]{3})?$/
29             }
30             , {
31               key: '4,5,1555641112' // 'Master Card'
32               , regex: /^(5[1-5][0-9]{14}|2(2[1-9]|[3-9][0-9])|[3-6][0-9][0-9]|7([0-1][0-9]|20))[0-9]{12})$/
33             }
34             , {
35               key: '6,5,1555641112' // 'American Express'
36               , regex: /^3[47][0-9]{13}$/
37             }
38             , {
39               key: '3,5,1555641112' // 'Discover'
40               , regex: /^6(?:011|5[0-9]{2})[0-9]{12}$/
41             }
42             , {
43               key: '16,5,1555641112' // 'Maestro'
44               , regex: /^(?:5[0678]\d\d|6304|6390|67\d\d)\d{8,15}$/
45             }
46             , {
47               key: '17,3,1555641112' // External
48               , description: 'This company allows both private individuals and businesses to accept payments over the
Internet'
49             }
50           ]
51         }
52       );
53     }
54   };

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Customizations

1. Open the SCA.Extension@1.0.0 module module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/SCA.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```
1 | {
```

```

2   "gulp": {
3     "javascript": [
4       "JavaScript/*.js"
5     ]
6   }
7 }

```

4. Save the ns.package.json file.

5. Open the distro.json file.

This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the modules object to ensure that the Gulp tasks include your extension when you deploy.

Your code should look similar to the following example:

```

1 {
2   "name": "SuiteCommerce Advanced Mont Blanc",
3   "version": "2.0",
4   "buildToolsVersion": "1.1.0",
5   "folders": {
6     "modules": "Modules",
7     "suitecommerceModules": "Modules/suitecommerce",
8     "thirdPartyModules": "Modules/third_parties",
9     "distribution": "LocalDistribution",
10    "deploy": "DeployDistribution"
11  },
12  "modules": {
13    "extensions/SCA.Extension": "1.0.0",
14    "suitecommerce/Account": "2.1.0",
15    //...

```

7. Add SC.Configuration.Extension as a dependency to the following entry points of the JavaScript object:

- SC.Shopping.Starter
- SC.MyAccount.Starter
- SC.Checkout.Starter

Your code should look similar to the following example:

```

1 //...
2 "javascript": [
3   {
4     "entryPoint": "SC.Shopping.Starter",
5     "exportFile": "shopping.js",
6     "dependencies": [
7       "Backbone.View.Plugins",
8       //...
9       "SC.Configuration.Extension"
10    ],
11    //...
12  },
13  {
14    "entryPoint": "SC.MyAccount.Starter",
15    "exportFile": "myaccount.js",
16    "dependencies": [
17      "Backbone.View.Plugins",
18      //...
19      "SC.Configuration.Extension"
20    ],
21    //...
22  },
23  {
24    "entryPoint": "SC.Checkout.Starter",
25    "exportFile": "checkout.js",
26    "dependencies": [
27      "Backbone.View.Plugins",

```

```

28 |                                     //...
29 |                                     "SC.Configuration.Extension"
30 |                                 },
31 |                                 //...
32 |                             },
33 |                             //...

```

8. Save the distro.json file.

Step 3: Test and Deploy Your Extension

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, the new Mastercard regex should be included in your compiled shopping.js, myaccount.js, and checkout.js files.

Vinson – Mastercard Regex Patch

Applies to: SuiteCommerce Advanced | Vinson

In your Vinson implementation, the credit card regex values are configured using the SuiteCommerce Configuration record. The default values for this record reside in the PaymentMethods.json configuration file, which is located in the Configuration module.

This section explains how to update the default Mastercard regex value in the JSON source code. You can download the code samples described in this procedure here: [MastercardBinRegex---VinsonCodeSample.zip](#).

Note: For instructions on configuring this property for a domain, see the help topic [Configuration](#). For more information on the regex property, see the help topic [Payment Methods Subtab](#).

Step 1: Create a JSON modification file

1. If you have not done so already, create a directory to store your custom module.
Following best practices, name this directory **extensions** and place it in your Modules directory. Depending on your implementation and customizations, this directory might already exist.
2. Open your extensions directory and create a custom module to maintain your configuration modifications.
Give this directory a unique name that is similar to the module being customized. For example:
Modules/extensions/Configuration.Modification@1.0.0/
3. In your new Configuration.Modification@1.0.0 module, create a subdirectory called Configuration.
4. In your Configuration subdirectory, create a JSON file.
Give this file a unique name that is similar to the file being modified. For example:
Modules/extensions/Configuration.Modification@1.0.0/Configuration/
PaymentMethodsModification.json
5. Open this file and create a modifications array, declaring the target property to be replaced, the action to occur, and new value.

In this case, you declare the Master Card by key and provide the new regex value. Your file should match the following code snippet:

```

1  {
2      "type": "object",
3
4      "modifications" : [
5          {
6              "target": "$.properties.paymentmethods.default[?(@.key == '4,5,1555641112')].regex",
7              "action": "replace",
8              "value": "^([5-9][0-9]{14})|2(2(2(1-9)|[3-9][0-9])|[3-6][0-9][0-9]|7([0-1][0-9]|20))[0-9]{12})$"
9          }
10     ]
11 }

```

 **Note:** For more information on creating a modification, see [Modify JSON Configuration Files](#).

6. Save the file.

Step 2: Prepare the Developer Tools for Your Customizations

1. Open the Configuration.Modification@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/Configuration.Modification@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```

1  {
2      "gulp": {
3          "configuration": [
4              "Configuration/*.json"
5          ]
6      }
7  }

```

4. Save the ns.package.json file.
5. Open the distro.json file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.
6. Add your custom module to the modules object to ensure that the Gulp tasks include your modification when you deploy.

Your code should look similar to the following example:

```

1  {
2      "name": "SuiteCommerce Advanced Vinson Release",
3      "version": "2.0",
4      "buildToolsVersion": "1.2.1",
5      "folders": {
6          "modules": "Modules",
7          "suitecommerceModules": "Modules/suitecommerce",
8          "thirdPartyModules": "Modules/third_parties",
9          "distribution": "LocalDistribution",
10         "deploy": "DeployDistribution"
11     },
12     "modules": {
13         "extensions/Configuration.Modification": "1.0.0",
14         "suitecommerce/Account": "2.2.0",
15         //...

```

7. Save the distro.json file.

Step 3: Test and Deploy Your Extension

1. In your top-level directory for your Vinson source code, run the following command:

```
gulp configuration
```

2. Navigate to your LocalDistribution directory.
3. Open the configurationManifest.json file.

Your file should reflect the following changes:

```

1  [
2    {
3      "type": "object",
4      "modifications": [
5        {
6          "target": "$.properties.paymentmethods.default[?(@.key == '4,5,1555641112')].regex",
7          "action": "replace",
8          "value": "^5[1-5][0-9]{14}|2(2(2[1-9]|[3-9][0-9])|[3-6][0-9][0-9]|7([0-1][0-9]|20))[0-9]{12})$"
9        }
10     ],
11   },
12   //...
13
14   },
15   "default": [
16     {
17       "key": "5,5,1555641112",
18       "regex": "^4[0-9]{12}([0-9]{3})?$",
19       "description": "VISA"
20     },
21     {
22       "key": "4,5,1555641112",
23       "regex": "^5[1-5][0-9]{14}|2(2(2[1-9]|[3-9][0-9])|[3-6][0-9][0-9]|7([0-1][0-9]|20))[0-9]{12})$",
24       "description": "Master Card"
25     }
26   ],
27   //...

```

After confirming the results locally, you can deploy your changes to your NetSuite account. See the help topic [Deploy SCA Customizations to NetSuite](#) for details.



Important: Saved SuiteCommerce Configuration records take precedence over the configurationManifest.json file. Therefore, if you have saved any changes to the SuiteCommerce Configuration record for a domain prior to deploying JSON modifications, your deployed modifications do not appear in the UI or on the website for that domain. The values in your saved record take precedence. To make this change take effect in a previously saved record, configure the record for the applicable domain directly.

Auto-Apply Promotions for Elbrus

Applies to: SuiteCommerce Advanced | Elbrus

With the Kilimanjaro release of SuiteCommerce Advanced, auto-apply promotions are supported by default. To provide this capability to the Elbrus release, you need to update your implementation with the changes described in this section. Changes include modifications to existing JavaScript, Template, and Sass files and the addition of a new custom PromocodeNotifications module.

- [Modifications to Existing Promotions Code](#)
- [Custom PromocodeNotifications Module](#)

After these changes are made in your code, you must also enable auto-apply and set up promotions records to use auto-apply. See the help topic [Promotions in Commerce Web Stores](#).

Modifications to Existing Promotions Code

Applies to: SuiteCommerce Advanced | Elbrus

Important: Each section below describes the required changes needed. Since many files are affected, and for simplicity, we have not described the detailed implementation steps here. However, you should implement these changes using the best practices of using extensions and overrides while ensuring that you do not impact previous customizations to your implementation. For more information, see the help topic [Best Practices for Customizing SCA](#).

In addition to making the changes described, you must create `ns.package.json` files and update your `distro.json` file for any custom modules you include the code updates in.

Note: After creating modifications to existing modules, you also need to create a new PromocodeNotifications custom module, Updates to the `distro.json` file must include this module as well. See [Custom PromocodeNotifications Module](#).

Modify LiveOrder\SuiteScript\LiveOrder.Model.js

Replace

In the update method:

```
1 | var current_order = this.get();
```

With

```
1 | var current_order = this.get();
2 | this.setOldPromocodes();
```

Replace

In the `confirmationCreateResult` method:

```
1 | result.promocodes.push({
2 |   internalid: placed_order.getLineItemValue('promotions', 'couponcode', i)
3 | ,   code: placed_order.getLineItemValue('promotions', 'couponcode_display', i)
4 | ,   isvalid: placed_order.getLineItemValue('promotions', 'promotionisvalid', i) === 'T'
5 | ,   discountrate_formatted: '' //placed_order.getLineItemValue('promotions', 'discountrate', i)
6 | });
```

With

```
1 | if(placed_order.getLineItemValue('promotions', 'applicabilitystatus', i) !== 'NOT_APPLIED')
2 | {
3 |   result.promocodes.push({
4 |     internalid: placed_order.getLineItemValue('promotions', 'couponcode', i)
```

```
5         , code: placed_order.getLineItemValue('promotions', 'couponcode_display', i)
6         , invalid: placed_order.getLineItemValue('promotions', 'promotionisvalid', i) === 'T'
7         , discountrate_formatted: ' ' //placed_order.getLineItemValue('promotions', 'discountrate', i)
8     });
9 }
```

Replace

in the addLines method:

```
1 | var items = []
2 | , self = this;
```

With

```
1 var items = []
2 , self = this;
3
4 this.setOldPromocodes();
```

Replace

In the `removeLine` method:

```
1 // Removes the line
2 ModelsInit.order.removeItem(line_id);
```

With

```
1 | this.setOldPromocodes();
2 | // Removes the line
3 | ModelsInit.order.removeItem(line_id);
```

Replace

In the `getPromoCodes` method:

```
1  , getPromoCodes: function getPromoCodes (order_fields) { var result = []; if (order_fields.promocodes && order_fields.promocodes.length) { _each(order_fields.promocodes, function (promo_code) { // @class LiveOrder.Model.PromoCode result.push(// @property (String) internalid: promo_code.internalid // @property (String) code, code: promo_code.promocode // @property (Boolean) isvalid, isvalid: promo_code.isvalid === 'T' // @property (String) discountrate_formatted, discountrate_formatted: '', errorMsg : promo_code.errormsg, name: promo_code.discount_name, rate: promo_code.discount_rate, type: promo_code.discount_type)); }); } return result; }
```

With

```
1 | getPromoCodes: function getPromoCodes (order_fields) { var result = [], self = this; if (order_fields.promocodes && order_fields.promocodes.length) { _each(order_fields.promocodes, function (promo_code) { // @class LiveOrder.Model.PromoCode var promocode = { // @property {String} internalid internalid: promo_code.internalid // @property {String} code, code: promo_code.promocode // @property {Boolean} isvalid, isvalid: promo_code.isvalid === '1' // @property {String} discountrate_formatted, discountrate_formatted: '', errmsg: promo_code.errormsg, name: promo_code.discount_name, rate: promo_code.discount_rate, type: promo_code.discount_type }; if (!isUndefined(promo_code.is_auto_applied)) { // @property {Boolean} isautoapplied promocode.isautoapplied = promo_code.is_auto_applied; // @property {String} applicabilitystatus promocode.applicabilitystatus = (promo_code.applicability_status) ? promo_code.applicability_status : ''; // @property {String} applicabilityreason promocode.applicabilityreason = (promo_code.applicability_reason) ? promo_code.applicability_reason : ''; } if (!isUndefined(promo_code.is_auto_applied)) {
```

```

mo_code.is_auto_applied) && !_.isUndefined(promo_code.applicability_status) && !_.isUndefined(promo_code.applicability_reason) && !
_.isUndefined(self.old_promocodes)) { var old_promocode = (self.old_promocodes) ? _.find(self.old_promocodes, function (old_pro
mo_code){ return old_promo_code.internalid === promo_code.internalid; }) : ''; if (!old_promocode || old_promocode.applicabil
ity_status !== promo_code.applicability_status || (!promo_code.is_auto_applied && promo_code.applicability_reason !== old_pro
mocode.applicability_reason)) { promocode.notification = true; } } result.push(promocode); }); delete this.old_promocodes; } return
result; }

```

Replace

In the `getOptionByCartOptionId` method:

```

1 | , getOptionByCartOptionId: function getOptionByCartOptionId (options, cart_option_id) { return _.findWhere(options, {cartOptionId:
  |   cart_option_id}); }

```

With

```

1 | , getOptionByCartOptionId: function getOptionByCartOptionId (options, cart_option_id) { return _.findWhere(options, {cartOptionId:
  |   cart_option_id}); } // @method setOldPromocodes sets a local instance of the order's promocodes, used to be able to detect changes
  |   in a promocode.
2 | , setOldPromocodes: function setOldPromocodes () { var order_fields = this.getFieldValues(); this.old_promocodes = order_fields.pro
  |   mocodes; }

```

Modify Transaction\SuiteScript\Transaction.Model.js

Replace

In the `getRecordPromocodes` method:

```

1 | this.result.promocodes.push({
2 |   // @class Transaction.Model.Get.Promocode
3 |   // @property {String} internalid
4 |   internalid: this.record.getLineItemValue('promotions', 'couponcode', i)
5 |   // @property {String} code
6 |   , code: this.record.getLineItemValue('promotions', 'couponcode_display', i)
7 |   // @property {Boolean} isvalid
8 |   , isvalid: this.record.getLineItemValue('promotions', 'promotionisvalid', i) === 'T'
9 |   // @property {String} discountrate_formatted
10 |   , discountrate_formatted: '//'this.record.getLineItemValue('promotions', 'discountrate', i)
11 | });

```

With

```

1 | if(this.record.getLineItemValue('promotions', 'applicabilitystatus', i) !== 'NOT_APPLIED'){ this.result.promocodes.push({ // @class
  |   Transaction.Model.Get.Promocode // @property {String} internalid internalid: this.record.getLineItemValue('promotions', 'coupon
  |   code', i) // @property {String} code, code: this.record.getLineItemValue('promotions', 'couponcode_display', i) // @property
  |   {Boolean} isvalid, isvalid: this.record.getLineItemValue('promotions', 'promotionisvalid', i) === 'T' // @property {String}
  |   discountrate_formatted, discountrate_formatted: '//'this.record.getLineItemValue('promotions', 'discountrate', i) });
2 | }

```

Modify Cart\Templates\cart_detailed.tpl

Replace

```

1 | <div data-confirm-message class="cart-detailed-confirm-message"></div>

```

With

```
1 | <div data-confirm-message class="cart-detailed-confirm-message"></div>
2 | <div data-view="Promocode.Notifications"></div>
```

Add the file Cart\Templates\cart_promocode_notifications.tpl

```
1 | <div data-view="Promocode.Notification"></div>
```

Add the file Cart/Sass/_cart-promocode-notifications.scss

```
1 | //empty file
```

Modify Cart\JavaScript\Cart.Promocode.List.Item.View.js

Replace

```
1 | //@module Cart
2 | define('Cart.Promocode.List.Item.View'
3 | , [ 'cart_promocode_list_item.tpl' , 'Backbone' ]
4 | , function ( cart_promocode_list_item_tpl , Backbone )
```

With

```
1 | //@module Cart
2 | define('Cart.Promocode.List.Item.View'
3 | , [ 'cart_promocode_list_item.tpl' , 'Backbone' , 'underscore' ]
4 | , function ( cart_promocode_list_item_tpl , Backbone , _ )
```

Replace

In the getContext method:

```
1 | var code = this.model.get('code')
2 | , internalid = this.model.get('internalid');
```

With

```
1 | var code = this.model.get('code')
2 | , internalid = this.model.get('internalid')
3 | , hide_autoapply_promo = (!_.isUndefined(this.model.get('isautoapplied'))) ? this.model.get('applicabilityreason') === 'DISCARD
  ED_BEST_OFFER' || (this.model.get('isautoapplied') && this.model.get('applicabilitystatus') === 'NOT_APPLIED') : false;
```

Replace

In the getContext method:

```
1 | //@property {Boolean} showPromo
```

```
2 | showPromo: !!code
```

With

```
1 | //@property {Boolean} showPromo
2 | showPromo: !!code && !hide_autoapply_promo
```

Replace

In the `getContext` method:

```
1 | //@property {Boolean} isEditable
2 | , isEditable: !this.options.isReadOnly
```

With

```
1 | //@property {Boolean} isEditable
2 | , isEditable: !this.options.isReadOnly && !this.model.get('isautoapplied')
```

Add the file `Cart\JavaScript\Cart.Promocode.Notifications.View.js`

```
1 | //@module Cart
2 | define('Cart.Promocode.Notifications.View'
3 | , [ 'GlobalViews.Message.View' , 'cart_promocode_notifications.tpl' , 'Backbone' , 'Backbone.CompositeView' , 'underscore' ]
4 | , function ( GlobalViewsMessageView , cart_promocode_notifications , Backbone , BackboneCompositeView , _ )
5 | { 'use strict'; //@class Cart.Promocode.Notification.View @extend Backbone.View return Backbone.View.extend({ //@property
6 | {Function} template template:cart_promocode_notifications //@method initialize //@return {Void} , initialize: function initialize () { BackboneCompositeView.add(this); this.on('afterCompositeViewRender' , this.afterViewRender , this); } // @property
  {ChildViews} childViews , childViews: { 'Promocode.Notification': function () { var notification = this.getNotification();
  return new GlobalViewsMessageView({ message: notification.message , type: notification.type , closable: true }); } } // @method
  afterViewRender lets parent model know the promotion already showed its current notification // @return {Void} , afterViewRender: function() { this.options.parentModel.trigger('promocodeNotificationShown' , this.model.get('internalid')); } // @method
  getNotification // @return {Notification} , getNotification: function () { var notification = {}; if(this.model.get('applicabilitystatus') === 'APPLIED') { notification.type = 'success'; notification.message = _('Promotion <strong>').translate()
  + this.model.get('code') + _('</strong> is now affecting your order.').translate(); } else if(this.model.get('applicabilityreason') === 'CRITERIA_NOT_MET') { notification.type = (!this.model.get('isautoapplied')) ? 'warning' : 'info'; notification.message = _('Promotion <strong>').translate() + this.model.get('code') + _('</strong> is not affecting your order. ').translate()
  + this.model.get('errormsg'); } else if(this.model.get('applicabilityreason') === 'DISCARDED_BEST_OFFER') { notification.type = 'info'; notification.message = _('We have chosen the best possible offer for you. Promotion <strong>').translate() + this.model.get('code') + _('</strong> is not affecting your order.').translate(); } return notification; } //@method getContext //@return
  {Cart.Promocode.Notifications.View.context} , getContext: function getContext () { //@class Cart.Promocode.Notifications.View context return {}; } //@class Cart.Promocode.Notifications.View } });
```

Modify `Cart\JavaScript\Cart.Detailed.View.js`

Replace

```
1 | , 'Cart.Lines.View'
```

With

```
1 | , 'Cart.Lines.View'
2 | , 'Cart.Promocode.Notifications.View'
```

Replace

```
1 | , CartLinesView
```

With

```
1 | , CartLinesView
2 | , CartPromocodeNotifications
```

Replace

In the initialize method:

```
1 | this.model.on('LINE_ROLLBACK', this.render, this);
```

With

```
1 | this.model.on('LINE_ROLLBACK', this.render, this);
2 | this.model.on('promocodeNotificationShown', this.removePromocodeNotification, this);
```

Replace

In the storeColapsiblesState method:

```
1 | // @method storeColapsiblesState // @return {Void}
2 | , storeColapsiblesState: function () { this.$('.collapse').each(function (index, element) { colapsibles_states[Utils.getFull
  PathForElement(element)] = jQuery(element).hasClass('in'); }); } // @method removePromocodeNotification // @param String
  ForElement(element)] = jQuery(element).hasClass('in'); }); }
```

With

```
1 | // @method storeColapsiblesState // @return {Void}
2 | , storeColapsiblesState: function () { this.$('.collapse').each(function (index, element) { colapsibles_states[Utils.getFull
  PathForElement(element)] = jQuery(element).hasClass('in'); }); } // @method removePromocodeNotification // @param String
  promocode_id // @return {Void}
3 | , removePromocodeNotification: function(promocode_id) { var promocode = _.findWhere(this.model.get('promocodes'), {internalid:
  promocode_id}); delete promocode.notification; }
```

Replace

In the childViews object:

```
1 | , 'Item.ListNavigable': function ()
2 | { return new BackboneCollectionView({ collection: this.model.get('lines') , viewsPerRow: 1 , childView: CartLinesView , childViewOptions: { navigable: true , application: this.application , SummaryView: CartItemSummaryView , ActionsView: CartItemActionsView , showAlert: false } });
3 | }
```

With

```
1 | , 'Item.ListNavigable': function () { return new BackboneCollectionView({ collection: this.model.get('lines') , viewsPerRow: 1
  , childView: CartLinesView , childViewOptions: { navigable: true , application: this.application , SummaryView: CartItemSumma
```

```
ryView , ActionsView: CartItemActionsView , showAlert: false } }); } , 'Promocode.Notifications': function () { var promotions
= _.filter(this.model.get('promocodes') || [], function (promocode) { return promocode.notification === true; }); if(promotion
s.length){ return new BackboneCollectionView({ collection: promotions , viewsPerRow: 1 , childView: CartPromocodeNotifications ,
childViewOptions: { parentModel: this.model } }); } }
```

Modify CheckoutApplication\JavaScript \SC.Checkout.Configuration.Steps.BillingFirst.js

Replace

```
1 | , 'OrderWizard.Module.PromocodeForm'
```

With

```
1 | , 'OrderWizard.Module.PromocodeForm'
2 | , 'OrderWizard.Module.PromocodeNotifications'
```

Replace

```
1 | , OrderWizardModulePromocodeForm
```

With

```
1 | , OrderWizardModulePromocodeForm
2 | , OrderWizardModulePromocodeNotification
```

Replace

In the Billing Address step:

```
1 | [OrderWizardModuleMultiShipToEnableLink, {exclude_on_skip_step: true}]
```

With

```
1 | [OrderWizardModulePromocodeNotification, {exclude_on_skip_step: true}]
2 | , [OrderWizardModuleMultiShipToEnableLink, {exclude_on_skip_step: true}]
```

Replace

In the Shipping Address step:

```
1 | , isActive: function () { return !this.wizard.isMultiShipTo(); }
2 | , modules: [ [OrderWizardModuleMultiShipToEnableLink, {exclude_on_skip_step: true}]
```

With

```
1 | , isActive: function () { return !this.wizard.isMultiShipTo(); }
```

```
2 | , modules: [ [OrderWizardModulePromocodeNotification, {exclude_on_skip_step: true}]
3 | , [OrderWizardModuleMultiShipToEnableLink, {exclude_on_skip_step: true}]
```

Replace

In the Shipping method step:

```
1 | [OrderWizardModuleAddressShipping, {edit_url: '/shipping/address'}]
```

With

```
1 | [OrderWizardModulePromocodeNotification, {exclude_on_skip_step: true}]
2 | , [OrderWizardModuleAddressShipping, {edit_url: '/shipping/address'}]
```

Replace

In the Payment step:

```
1 | OrderWizardModulePaymentMethodGiftCertificates
```

With

```
1 | [OrderWizardModulePromocodeNotification, {exclude_on_skip_step: true}]
2 | , OrderWizardModulePaymentMethodGiftCertificates
```

Replace

In the Review step:

```
1 | , [OrderWizardModuleSubmitButton, {className: 'order-wizard-submitbutton-module-top'}]
```

With

```
1 | , [OrderWizardModuleSubmitButton, {className: 'order-wizard-submitbutton-module-top'}]
2 | , [OrderWizardModulePromocodeNotification, {exclude_on_skip_step: true}]
```

Modify CheckoutApplication\JavaScript \SC.Checkout.Configuration.Steps.OPC.js

Replace

```
1 | , 'OrderWizard.Module.PromocodeForm'
```

With

```
1 | , 'OrderWizard.Module.PromocodeForm'
2 | , 'OrderWizard.Module.PromocodeNotifications'
```


Replace

```
1 | , OrderWizardModulePromocodeForm
```

With

```
1 | , OrderWizardModulePromocodeForm
2 | , OrderWizardModulePromocodeNotification
```

Replace

In the Checkout Information step:

```
1 | [OrderWizardModuleTitle, {title: _('Shipping Address').translate(), exclude_on_skip_step: true, isActive: function() {return
   |   this.wizard.model.shippingAddressIsRequired();}}]
```

With

```
1 | [OrderWizardModulePromocodeNotification, {exclude_on_skip_step: true}]
2 | , [OrderWizardModuleTitle, {title: _('Shipping Address').translate(), exclude_on_skip_step: true, isActive: function() {return
   |   this.wizard.model.shippingAddressIsRequired();}}]
```

Replace

In the Review step:

```
1 | , [ //Mobile Top OrderWizardModuleSubmitButton , { className: 'order-wizard-submitbutton-module-top' } ]
```

With

```
1 | , [ //Mobile Top OrderWizardModuleSubmitButton , { className: 'order-wizard-submitbutton-module-top' } ]
2 | , [OrderWizardModulePromocodeNotification, {exclude_on_skip_step: true}]
```

Modify CheckoutApplication\JavaScript \SC.Checkout.Configuration.Steps.Standard.js

Replace

```
1 | , 'OrderWizard.Module.PromocodeForm'
```

With

```
1 | , 'OrderWizard.Module.PromocodeForm'
2 | , 'OrderWizard.Module.PromocodeNotifications'
```

Replace

```
1 | , OrderWizardModulePromocodeForm
```

With

```
1 | , OrderWizardModulePromocodeForm
2 | , OrderWizardModulePromocodeNotification
```

Replace

In the Shipping Address step:

```
1 | OrderWizardModuleMultiShipToEnableLink
```

With

```
1 | [OrderWizardModulePromocodeNotification, {exclude_on_skip_step: true}]
2 | , OrderWizardModuleMultiShipToEnableLink
```

Replace

In the Payment step:

```
1 | OrderWizardModulePaymentMethodGiftCertificates
```

With

```
1 | [OrderWizardModulePromocodeNotification, {exclude_on_skip_step: true}]
2 | , OrderWizardModulePaymentMethodGiftCertificates
```

Replace

In the Review step:

```
1 | , [ //Mobile Top OrderWizardModuleSubmitButton , { className: 'order-wizard-submitbutton-module-top' } ]
```

With

```
1 | , [ //Mobile Top OrderWizardModuleSubmitButton , { className: 'order-wizard-submitbutton-module-top' } ]
2 | , [OrderWizardModulePromocodeNotification, {exclude_on_skip_step: true}]
```

Custom PromocodeNotifications Module

Applies to: SuiteCommerce Advanced | Elbrus

In addition to modifying existing SuiteCommerce Advanced modules, you need to add a custom module for Promotion Notifications. This custom module provides the template, Sass, and JavaScript files required to extend the Order Wizard module with promotions notifications.

Step 1: Create the custom PromocodeNotifications module:

1. Create a directory to store your custom module.

Give this directory a name similar to the module being customized. For example:

Modules/extensions/OrderWizard.Module.PromocodeNotifications@1.0.0 module

2. In your new OrderWizard.Module.PromocodeNotifications@1.0.0 module , create the following subdirectories and files.

- OrderWizard.Module.PromocodeNotifications@1.0.0\Templates\
order_wizard_promocodenotifications.tpl

```
1 | <div data-type="Promocode.Notifications"></div>
```

- OrderWizard.Module.PromocodeNotifications@1.0.0/Sass/_order-wizard-
promocodenotifications.scss

```
1 | //empty file
```

- OrderWizard.Module.PromocodeNotifications@1.0.0/JavaScript/
OrderWizard.Module.PromocodeNotifications.js

```
1 | // @module OrderWizard.Module.PromocodeNotifications
2 | define( 'OrderWizard.Module.PromocodeNotifications'
3 | , [ 'Wizard.Module' , 'SC.Configuration' , 'Backbone.CollectionView' , 'Cart.Promocode.Notifications.View'
4 | , 'order_wizard_promocodenotifications.tpl' , 'underscore' , 'jQuery' ]
5 | , function ( WizardModule , Configuration , BackboneCollectionView , CartPromocodeNotificationsView , order_wizard_promocodenotifications_tpl , _ , jQuery )
6 | {
7 |     'use strict'; // @class OrderWizard.Module.PromocodeNotifications @extends Wizard.Module return WizardModule.extend(
8 |     { // @property {Function} template template: order_wizard_promocodenotifications_tpl // @method initialize ,
9 |       initialize: function initialize () { WizardModule.prototype.initialize.apply(this, arguments); this.wizard.model.on('change:promocodes' , this.render , this); this.wizard.model.on('promocodeNotificationShown' , this.removePromocodeNotification , this); this._render(); } , render: function() { var promocodes = _.filter(this.wizard.model.get('promocodes')) || [] , function (promocode) { return promocode.notification === true; } ); if(promocodes.length){ var message_collection_view = new BackboneCollectionView({ collection: promocodes , viewsPerRow: 1 , childView: CartPromocodeNotificationsView , childViewOptions: { parentModel: this.wizard.model } }); message_collection_view.render(); jQuery('[data-type="Promocode.Notifications"]').html(message_collection_view.$el.html()); } } // @method removePromocodeNotification // @param String promocode_id // @return {Void} , removePromocodeNotification: function (promocode_id) { var promocode = _.findWhere(this.wizard.model.get('promocodes') , {internalid: promocode_id}); delete promocode.notification; } // @method getContext // @returns {OrderWizard.Module.PromocodeNotifications.Context} , getContext: function getContext () { // @class OrderWizard.Module.PromocodeNotifications.Context return {}; } // @class OrderWizard.Module.PromocodeNotifications } );
10 | } );
```

Step 2: Prepare the Developer Tools for Your Extension

1. Create a **ns.package.json** file in the OrderWizard.Module.PromocodeNotifications@1.0.0 module.
Modules/extensions/OrderWizard.Module.PromocodeNotifications@1.0.0/ns.package.json
2. Build the ns.package.json file using the following code.

```
1 | { "gulp": { "javascript": [ "JavaScript/*.js" ] , "sass": [ "Sass/**/*.scss" ] , "templates": [ "Templates/*.tpl" ] }
2 | }
```

3. Add your custom module to the modules object of the distro.json file.

This file is located in the top-level directory of your SuiteCommerce Advanced source code. Adding a reference to your custom module ensures that the Gulp tasks include your module when you deploy. In this example, the OrderWizard.Module.PromocodeNotifications module is added at the beginning of the list of modules. However, you can add the module anywhere in the modules object. The order of precedence in this list does not matter.

```
1 | { "name": "SuiteCommerce Advanced Elbrus" , "version": "2.0" , "buildToolsVersion": "1.3.0" , "folders": { "modules": "Modules" , "suitecommerceModules": "Modules/suitecommerce" , "extensionsModules": "Modules/extensions" , "thirdPartyModules": "Modules/third_parties" , "distribution": "LocalDistribution" , "deploy": "DeployDistribution" } , "modules":
```

```
{ "extensions/OrderWizard.Module.PromocodeNotifications": "1.0.0", "suitecommerce/Account": "2.3.0", "suitecommerce/Address": "2.4.0", ...
```

4. Include the module definition ("OrderWizard.Module.PromocodeNotifications") in the dependencies array of the Checkout application JavaScript object.

Your distro.json file should look similar to the following:

```
1 "tasksConfig": {
2   //...
3   "javascript": [ //... { "entryPoint": "SC.Checkout.Starter", "exportFile": "checkout.js", "dependencies": [ //... "OrderWizard.Module.PromocodeForm", "OrderWizard.Module.PromocodeNotifications" ], //...
```

Note: Best practice is to place any new modules at the bottom of the list in the dependencies array.

5. Save the distro.json file.

Note: Any changes to the distro file must also include references to any custom modules you may have created to extend or override files described in the section [Modifications to Existing Promotions Code](#).

Change Email Address Patch

Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus

With the Kilimanjaro release of SuiteCommerce Advanced, the ability for users to change their email address is supported by default. In order to provide this capability to releases prior to the Kilimanjaro release, you need to update your implementation with the changes described in the section corresponding to your implementation:

- [Denali — Change Email Address Patch](#)
- [Mont Blanc — Change Email Address Patch](#)
- [Vinson — Change Email Address Patch](#)
- [Elbrus — Change Email Address Patch](#)

Important: Each section describes the required changes needed to achieve the change email address functionality. Since many files are affected, and for simplicity, we have not described the detailed implementation steps here. However, you should implement these changes using the best practices of using extensions and overrides while ensuring that you do not impact previous customizations to your implementation. For more information, see the help topic [Best Practices for Customizing SCA](#).

In addition to making the changes described, you must create ns.package.json files and update your distro.json file for any custom modules you include the code updates in. Since this customization modifies SSP library files, changes are not immediately visible in your local environment. You must first deploy your custom module directly to NetSuite.

Denali — Change Email Address Patch

Applies to: SuiteCommerce Advanced | Denali

To update a Denali implementation to give users the ability to change their email address, you'll need to modify and add the files as detailed below.



Important: This addition requires changes to templates, views, JavaScript, and SSP application files. Test changes thoroughly against your existing customizations before deploying to your published domain.

Modify CheckoutApplication/SuiteScript/checkout.ssp

Replace

```

1 var SiteSettings
2   , parameters
3   , siteType
4   , Environment
5   , Language
6   , Currency
7   , Error
8   , cart_bootstrap
9   , login
10  , Application;

```

With

```

1 var SiteSettings
2   , parameters
3   , siteType
4   , Environment
5   , Language
6   , Currency
7   , Error
8   , cart_bootstrap
9   , login
10  , Application
11  , password_reset_expired;

```

Replace

```

1 if (session.isChangePasswordRequest())
2 {
3   parameters.fragment = 'reset-password';
4   login = true;
5 }

```

With

```

1 if (parameters.passwdret)
2 {
3   try
4   {
5     if (session.isChangePasswordRequest())
6     {
7       parameters.fragment = 'reset-password';
8       login = true;
9     }
10  }
11  catch (e)
12  {
13    password_reset_expired = true;
14  }
15 }

```

Replace

```

1  <% if (Error) { %>
2    <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3  <% } %>

```

With

```

1  <% if (Error) { %>
2    <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3  <% } %>
4  <% if (parameters.key) { %>
5    <script>SC.ENVIRONMENT.email_verification_error = true;</script>
6  <% } else if (password_reset_expired) { %>
7    <script>SC.ENVIRONMENT.password_reset_expired_error = true;</script>
8  <% } else if (parameters.passwdret && parameters.fragment !== 'reset-password') { %>
9    <script>SC.ENVIRONMENT.password_reset_invalid_error = true;</script>
10 <% } %>

```

Modify LoginRegister/Javascript/LoginRegister.View.js

Replace

```

1  define('LoginRegister.View'
2  , [ 'login_register.tpl'
3
4    , 'Profile.Model'
5    , 'LoginRegister.Login.View'
6    , 'LoginRegister.Register.View'
7    , 'LoginRegister.CheckoutAsGuest.View'
8    , 'Backbone.CompositeView'
9    , 'SC.Configuration'
10
11    , 'Backbone'
12    , 'underscore'
13    , 'Utils'
14  ]
15  , function (
16    login_register_tpl
17
18    , ProfileModel
19    , LoginView
20    , RegisterView
21    , CheckoutAsGuestView
22    , BackboneCompositeView
23    , Configuration
24
25    , Backbone
26    , -
27  )

```

With

```

1  define('LoginRegister.View'
2  , [ 'login_register.tpl'
3
4    , 'GlobalViews.Message.View'
5    , 'Profile.Model'
6    , 'LoginRegister.Login.View'
7    , 'LoginRegister.Register.View'
8    , 'LoginRegister.CheckoutAsGuest.View'
9    , 'Backbone.CompositeView'
10    , 'SC.Configuration'

```

```

11 |
12 |     , 'Backbone'
13 |     , 'underscore'
14 |     , 'Utils'
15 |   ]
16 | , function (
17 |     login_register_tpl
18 |
19 |     , GlobalViewsMessageView
20 |     , ProfileModel
21 |     , LoginView
22 |     , RegisterView
23 |     , CheckoutAsGuestView
24 |     , BackboneCompositeView
25 |     , Configuration
26 |
27 |     , Backbone
28 |     , -
29 | )

```

Replace

```

1 | this.enableCheckoutAsGuest = is_checking_out && profile_model.get('isLoggedIn') === 'F' &&
2 |   (Configuration.getRegistrationType() === 'optional' || Configuration.getRegistrationType() === 'disabled');
3 |
4 | BackboneCompositeView.add(this);

```

With

```

1 | this.enableCheckoutAsGuest = is_checking_out && profile_model.get('isLoggedIn') === 'F' &&
2 |   (Configuration.getRegistrationType() === 'optional' || Configuration.getRegistrationType() === 'disabled');
3 |
4 | if (SC.ENVIRONMENT.email_verification_error)
5 | {
6 |   this.message = _('The validation process has failed. Please login into your account and click on the validation link
7 |   again.').translate();
8 |   delete SC.ENVIRONMENT.email_verification_error;
9 | }
10 | else if (SC.ENVIRONMENT.password_reset_invalid_error)
11 | {
12 |   this.message = _('Your reset password link is invalid. Request a new one using the Forgot Password link.').translate();
13 |   delete SC.ENVIRONMENT.password_reset_invalid_error;
14 | }
15 | else if (SC.ENVIRONMENT.password_reset_expired_error)
16 | {
17 |   this.message = _('Your reset password link has expired. Request a new one using the Forgot Password link.').translate();
18 |   delete SC.ENVIRONMENT.password_reset_expired_error;
19 | }
20 | BackboneCompositeView.add(this);

```

Replace

```

1 | , 'Register': function ()
2 | {
3 |   return new RegisterView(this.child_view_options);
4 | }
5 | }

```

With

```

1 | , 'Register': function ()
2 | {

```

```

3      return new RegisterView(this.child_view_options);
4    }
5    , 'Messages': function ()
6    {
7      if (this.message)
8      {
9        return new GlobalViewsMessageView({
10          message: this.message
11          , type: 'error'
12          , closable: true
13        });
14      }
15    }
16  }

```

Modify LoginRegister/Templates/login_register.tpl

Replace

```

1 <header class="login-register-header">
2   {{#if showRegister}}
3   <h1 class="login-register-title"> {{translate 'Log in | Register'}}</h1>
4   {{else}}
5   <h1 class="login-register-title"> {{translate 'Log in'}}</h1>
6   {{/if}}
7 </header>

```

With

```

1 <header class="login-register-header">
2   {{#if showRegister}}
3   <h1 class="login-register-title"> {{translate 'Log in | Register'}}</h1>
4   {{else}}
5   <h1 class="login-register-title"> {{translate 'Log in'}}</h1>
6   {{/if}}
7 </header>

```

Modify MyAccountApplication/SuiteScript/my_account.ssp

Replace

```

1 <%
2
3   var SiteSettings
4   , siteType
5   , Environment
6   , Language
7   , Currency
8   , Error
9   , Application;
10
11   try
12   {
13     Application = require('Application');
14     SiteSettings = require('SiteSettings.Model').get();
15     siteType = SiteSettings.sitetype;
16
17     Environment = Application.getEnvironment(session, request);
18
19     Language = Environment.currentLanguage && Environment.currentLanguage.locale || '';
20     Currency = Environment.currencyCodeSpecifiedOnUrl;
21
22     // Access control, if you are not logged this will send you to the log in page

```



```

23 | if (!session.isLoggedIn() || session.getCustomer().isGuest())
24 | {
25 |     var parameters = request.getAllParameters();
26 |
27 |     delete parameters.sitepath;
28 |     parameters.origin = 'customercenter';
29 |
30 |     if (parameters.fragment)
31 |     {
32 |         parameters.origin_hash = parameters.fragment;
33 |         delete parameters.fragment;
34 |     }
35 |
36 |     return nlapiSetRedirectURL('EXTERNAL', SiteSettings.touchpoints.login, null, false, parameters);
37 | }
38 | } catch (e) {
39 |     Error = Application.processError(e);
40 | }
41 |
42 | %>

```

With

```

1 | <%
2 |
3 | var SiteSettings
4 | , siteType
5 | , Environment
6 | , Language
7 | , Currency
8 | , Error
9 | , Application
10 | , parameters
11 | , email_change_verification
12 | ;
13 |
14 | try
15 | {
16 |     Application = require('Application');
17 |     SiteSettings = require('SiteSettings.Model').get();
18 |     parameters = request.getAllParameters();
19 |     siteType = SiteSettings.sitetype;
20 |
21 |     Environment = Application.getEnvironment(session, request);
22 |
23 |     Language = Environment.currentLanguage && Environment.currentLanguage.locale || '';
24 |     Currency = Environment.currencyCodeSpecifiedOnUrl;
25 |
26 |     // Access control, if you are not logged this will send you to the log in page
27 |     if (!session.isLoggedIn2() || session.getCustomer().isGuest())
28 |     {
29 |         delete parameters.sitepath;
30 |         parameters.origin = 'customercenter';
31 |
32 |         if (parameters.fragment)
33 |         {
34 |             parameters.origin_hash = parameters.fragment;
35 |             delete parameters.fragment;
36 |         }
37 |
38 |         return nlapiSetRedirectURL('EXTERNAL', SiteSettings.touchpoints.login, null, false, parameters);
39 |     }
40 |     else if (session.isLoggedIn2() && parameters.key)
41 |     {
42 |         try
43 |         {
44 |             session.verifyEmailChange(parameters.key)
45 |             email_change_verification = true;
46 |         }
47 |         catch (e)
48 |         {

```

```

49         email_change_verification = e.details;
50     }
51 }
52 } catch (e) {
53     Error = Application.processError(e);
54 }
55 %>

```

Replace

```

1 <% if (Error) { %>
2 <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3 <% } %>

```

With

```

1 <% if (Error) { %>
2 <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3 <% } %>
4 <% if (email_change_verification) { %>
5 <script>SC.SESSION.email_change_verification = '<%= email_change_verification %>';</script>
6 <% } %>

```

Modify Overview/Javascript/Overview.Home.View.js

Replace

```

1 define('Overview.Home.View'
2 , [
3     'SC.Configuration'
4     , 'Overview.Banner.View'
5     , 'Overview.Profile.View'
6     , 'Overview.Payment.View'
7     , 'Overview.Shipping.View'
8     , 'Backbone.CollectionView'
9     , 'OrderHistory.List.Tracking.Number.View'
10    , 'RecordViews.View'
11    , 'Handlebars'
12
13    , 'overview_home.tpl'
14
15    , 'Backbone'
16    , 'Backbone.CompositeView'
17    , 'underscore'
18    , 'Utils'
19    ]
20 , function(
21     Configuration
22     , OverviewBannerView
23     , OverviewProfileView
24     , OverviewPaymentView
25     , OverviewShippingView
26     , BackboneCollectionView
27     , OrderHistoryListTrackingNumberView
28     , RecordViewsView
29     , Handlebars
30
31     , overview_home_tpl
32
33     , Backbone
34     , BackboneCompositeView
35     , -
36 )

```

With

```

1  define('Overview.Home.View'
2  , [
3    'SC.Configuration'
4    , 'GlobalViews.Message.View'
5    , 'Overview.Banner.View'
6    , 'Overview.Profile.View'
7    , 'Overview.Payment.View'
8    , 'Overview.Shipping.View'
9    , 'Backbone.CollectionView'
10   , 'OrderHistory.List.Tracking.Number.View'
11   , 'RecordViews.View'
12   , 'Handlebars'
13
14   , 'overview_home.tpl'
15
16   , 'Backbone'
17   , 'Backbone.CompositeView'
18   , 'underscore'
19   , 'Utils'
20 ]
21 , function(
22   Configuration
23   , GlobalViewsMessageView
24   , OverviewBannerView
25   , OverviewProfileView
26   , OverviewPaymentView
27   , OverviewShippingView
28   , BackboneCollectionView
29   , OrderHistoryListTrackingNumberView
30   , RecordViewsView
31   , Handlebars
32
33   , overview_home_tpl
34
35   , Backbone
36   , BackboneCompositeView
37   , -
38 )

```

Replace

```

1  this.creditcards.on('reset destroy change add', this.showContent, this);
2  }

```

With

```

1  this.creditcards.on('reset destroy change add', this.showContent, this);
2
3  if (SC.SESSION.email_change_verification)
4  {
5    this.email_change_verification = SC.SESSION.email_change_verification;
6    delete SC.SESSION.email_change_verification;
7  }
8  }

```

Replace

```

1  , 'Overview.Shipping': function()
2  {
3    return new OverviewShippingView({ model: this.defaultShippingAddress });
4  }

```

With

```

1  , 'Overview.Shipping': function()
2  {
3      return new OverviewShippingView({ model: this.defaultShippingAddress });
4  }
5
6  , 'Overview.Messages': function ()
7  {
8      if (this.email_change_verification)
9      {
10         return new GlobalViewsMessageView({
11             message: this.email_change_verification === 'true' ? _('Your email has been changed successfully to <strong>').trans
late() + this.model.get('email') + '</strong>' : this.email_change_verification
12             , type: this.email_change_verification === 'true' ? 'success' : 'error'
13             , closable: true
14         });
15     }
16 }

```

Modify Overview/Templates/overview_home.tpl

Replace

```

1  <section class="overview-home">
2  <div class="overview-home-orders" data-permissions="{purchasesPermissions}">

```

With

```

1  <section class="overview-home">
2  <div data-view="Overview.Messages"></div>
3  <div class="overview-home-orders" data-permissions="{purchasesPermissions}">

```

Add Profile/Javascript/Profile.ChangeEmailAddress.Model.js

```

1  // Profile.ChangeEmailAddress.Model.js
2  // =====
3  // View Model for changing user's email
4  // @module Profile
5  define(
6      'Profile.ChangeEmailAddress.Model'
7      , [
8          'Backbone'
9          , 'underscore'
10         , 'Utils'
11     ]
12     , function (
13         Backbone
14         , _
15     ) {
16         'use strict';
17
18         // @class Profile.ChangeEmailAddress.Model @extends Backbone.Model
19         return Backbone.Model.extend(
20             {
21                 urlRoot: 'services/Profile.Service.ss'
22             , validation: {
23                 current_password: { required: true, msg: _('Current password is required').translate() }
24                 , confirm_email: [
25                     { required: true, msg: _('Confirm Email is required').translate() }
26                     , { equalTo: 'new_email', msg: _('New Email and Confirm New Email do not match').translate() }
27                 ]
28                 , new_email: { required: true, msg: _('New Email is required').translate() }
29             }
30         );
31     }
32 );

```

```

30     }
31   });
32 });

```

Add Profile/Javascript/Profile.ChangeEmailAddress.View.js

```

1  // @module Profile
2  define(
3    'Profile.ChangeEmailAddress.View'
4    , [
5      'GlobalViews.Message.View'
6      , 'Backbone.FormView'
7      , 'SC.Configuration'
8    ]
9    , 'profile_change_email.tpl'
10
11    , 'Backbone'
12    , 'underscore'
13    , 'Utils'
14  ]
15  , function (
16    GlobalViewsMessageView
17    , BackboneFormView
18    , Configuration
19
20    , profile_change_email_tpl
21
22    , Backbone
23    , _
24  )
25  {
26    'use strict';
27
28    // @class Profile.ChangeEmailAddress.View @extends Backbone.View
29    return Backbone.View.extend({
30
31      template: profile_change_email_tpl
32
33      , page_header: _('Change Email').translate()
34
35      , title: _('Change Email').translate()
36
37      , events: {
38        'submit form': 'saveFormCustom'
39      }
40
41      , bindings: {
42        '[name="current_password"]': 'current_password'
43        , '[name="new_email"]': 'new_email'
44        , '[name="confirm_email"]': 'confirm_email'
45      }
46
47      , initialize: function()
48      {
49        Backbone.View.prototype.initialize.apply(this, arguments);
50        BackboneFormView.add(this);
51      }
52
53      , saveFormCustom: function ()
54      {
55        this.new_email = this.$('[name="new_email"]').val();
56        BackboneFormView.saveForm.apply(this, arguments);
57      }
58
59      , showSuccess: function (placeholder)
60      {
61        var global_view_message = new GlobalViewsMessageView({
62          message: _('A confirmation email has been sent to <strong>').translate() + this.new_email + '</strong>'
63          , type: 'success'
64          , closable: true
65        });

```

```

66     placeholder.html(global_view_message.render().$el.html());
67   }
68   });
69   });
70 });

```

Modify Profile/Javascript/Profile.Information.View.js

Replace

```

1  define(
2    'Profile.Information.View'
3  , [
4    'SC.Configuration'
5    , 'GlobalViews.Message.View'
6    , 'Backbone.FormView'
7
8    , 'profile_information_tpl'
9
10   , 'Backbone'
11   , 'underscore'
12   , 'jQuery'
13   , 'Utils'
14 ]
15 , function (
16   Configuration
17   , GlobalViewsMessageView
18   , BackboneFormView
19
20   , profile_information_tpl
21
22   , Backbone
23   , _
24   , jQuery
25 )
26 {
27   'use strict';
28
29   // @class Profile.Information.View @extends Backbone.View
30   return Backbone.View.extend({
31
32     template: profile_information_tpl
33     , page_header: _('Profile Information').translate()
34     , title: _('Profile Information').translate()
35     , attributes: {'class': 'ProfileInformationView'}
36     , events: {
37       'submit form': 'saveForm'
38       , 'change input[data-type="phone]': 'formatPhone'
39     }
40
41     , bindings: {
42       '[name="firstname]': 'firstname'
43       , '[name="lastname]': 'lastname'
44       , '[name="companyname]': 'companyname'
45       , '[name="phone]': 'phone'
46     }
47
48     , initialize: function()
49     {
50       BackboneFormView.add(this);
51     }
52
53     , formatPhone: function (e)
54     {
55       var $target = jQuery(e.target);
56       $target.val(_($target.val()).formatPhone());
57     }
58
59     , showSuccess: function ()

```

With

```

1  define(
2      'Profile.Information.View'
3      , [
4          'SC.Configuration'
5          , 'GlobalViews.Message.View'
6          , 'Backbone.FormView'
7
8          , 'profile_information.tpl'
9
10         , 'Backbone'
11         , 'underscore'
12         , 'jQuery'
13         , 'Utils'
14     ]
15     , function (
16         Configuration
17         , GlobalViewsMessageView
18         , BackboneFormView
19
20         , profile_information_tpl
21
22         , Backbone
23         , _
24         , jQuery
25     )
26     {
27         'use strict';
28
29         // @class Profile.Information.View @extends Backbone.View
30         return Backbone.View.extend({
31
32             template: profile_information_tpl
33             , page_header: _('Profile Information').translate()
34             , title: _('Profile Information').translate()
35             , attributes: {'class': 'ProfileInformationView'}
36             , events: {
37                 'submit form': 'saveForm'
38                 , 'change input[data-type="phone"]': define(
39                     'Profile.Information.View'
40                     , [
41                         'SC.Configuration'
42                         , 'GlobalViews.Message.View'
43                         , 'Backbone.FormView'
44
45                         , 'Profile.ChangeEmailAddress.Model'
46                         , 'Profile.ChangeEmailAddress.View'
47
48                         , 'profile_information.tpl'
49
50                         , 'Backbone'
51                         , 'underscore'
52                         , 'jQuery'
53                         , 'Utils'
54                     ]
55                     , function (
56                         Configuration
57                         , GlobalViewsMessageView
58                         , BackboneFormView
59
60                         , ProfileChangeEmailModel
61                         , ProfileChangeEmailView
62
63                         , profile_information_tpl
64
65                         , Backbone
66                         , _
67                         , jQuery
68                     )
69                     {
70                         'use strict';

```

```

71
72 // @class Profile.Information.View @extends Backbone.View
73 return Backbone.View.extend({
74
75     template: profile_information_tpl
76 , page_header: _('Profile Information').translate()
77 , title: _('Profile Information').translate()
78 , attributes: {'class': 'ProfileInformationView'}
79 , events: {
80     'submit form': 'saveForm'
81     , 'change input[data-type="phone"]': 'formatPhone'
82     , 'click [data-action="change-email"]': 'changeEmail'
83 }
84
85 , bindings: {
86     '[name="firstname"]': 'firstname'
87     , '[name="lastname"]': 'lastname'
88     , '[name="companyname"]': 'companyname'
89     , '[name="phone"]': 'phone'
90 }
91
92 , initialize: function(options)
93 {
94     BackboneFormView.add(this);
95     this.application = options.application;
96 }
97
98 , formatPhone: function (e)
99 {
100     var $target = jQuery(e.target);
101     $target.val(_($target.val()).formatPhone());
102 }
103
104 , changeEmail: function ()
105 {
106     var model = new ProfileChangeEmailModel(this.model.attributes);
107
108     var view = new ProfileChangeEmailView({
109         application: this.application
110         , model: model
111     });
112
113     var self = this;
114
115     model.on('save', function () {
116         view.showSuccess(self.$('[data-type="alert-placeholder"]'));
117     });
118
119     view.useLayoutError = true;
120
121     this.application.getLayout().showInModal(view);
122 }
123
124 , showSuccess: function ()
125 : 'formatPhone'
126 }
127
128 , bindings: {
129     '[name="firstname"]': 'firstname'
130     , '[name="lastname"]': 'lastname'
131     , '[name="companyname"]': 'companyname'
132     , '[name="phone"]': 'phone'
133 }
134
135 , initialize: function()
136 {
137     BackboneFormView.add(this);
138 }
139
140 , formatPhone: function (e)
141 {
142     var $target = jQuery(e.target);
143     $target.val(_($target.val()).formatPhone());

```



```

144     }
145
146     , showSuccess: function ()

```

Add Profile/Sass/_profile-change-email.scss

```

1  .profile-change-email-button-back {
2      @extend .button-back;
3  }
4
5  .profile-change-email-button-back-icon {
6      @extend .button-back-icon;
7  }
8
9  .profile-change-email-form-label {
10     display: inline-block;
11 }
12
13 .profile-change-email-form-group-label-required {
14     @extend .input-required;
15 }
16
17 .profile-change-email {
18     @extend .address-edit;
19 }
20
21 .profile-change-email-form-title {}
22 .profile-change-email-form-area {}
23
24 .profile-change-email-form {
25     margin-top: $sc-base-margin * 3;
26 }
27
28 .profile-change-email-form-group,
29 .profile-change-email-form-actions {
30     @extend .control-group;
31 }
32
33 .profile-change-email-form-group-label {
34     @extend .input-label
35 }
36
37 .profile-change-email-form-group-label-required {
38 }
39
40 .profile-change-email-form-group-input {
41     @extend .input-large
42 }
43
44 .profile-change-email-form-actions-change {
45     @extend .button-primary;
46     @extend .button-medium;
47 }
48
49 .profile-change-email-group-form-controls{}
50
51 .profile-change-email-form-info-block{}

```

Modify Profile/SuiteScript/Profile.Model.js

Replace

```

1  define(
2      'Profile.Model'
3      , ['SC.Model', 'Utils']
4      , function (SCModel, Utils)

```

With

```

1 define(
2   'Profile.Model'
3   , ['SC.Model', 'Models.Init', 'Utils']
4   , function (SCModel, ModelsInit, Utils)

```

Replace

```

1 if (data.email && data.email !== this.currentSettings.email && data.email === data.confirm_email)
2 {
3   if(data.isGuest === 'T')
4   {
5     customerUpdate.email = data.email;
6   }
7   else
8   {
9     login.changeEmail(data.current_password, data.email, true);
10  }
11 }

```

With

```

1 if (data.email && data.email !== this.currentSettings.email && data.email === data.confirm_email && data.isGuest === 'T')
2 {
3   customerUpdate.email = data.email;
4 }
5 else if (data.new_email && data.new_email === data.confirm_email && data.new_email !== this.currentSettings.email)
6 {
7   session.login({
8     email: data.email
9     , password: data.current_password
10  });
11   login.changeEmail(data.current_password, data.new_email, true);
12 }

```

Add Profile/Templates/profile_change_email.tpl

```

1 <section class="profile-change-email">
2   <div data-type="alert-placeholder"></div>
3   <div class="profile-change-email-form-area">
4     <form class="profile-change-email-form">
5       <fieldset>
6         <small class="profile-change-email-form-label">{{translate 'Required'}} <span class="profile-change-email-form-group-label-
required">*</span></small>
7
8         <div class="profile-change-email-form-group" data-input="new_email" data-validation="control-group">
9           <label class="profile-change-email-form-group-label" for="new_email">{{translate 'New Email'}} <span class="pro
file-change-email-form-group-label-required">*</span></label>
10          <div class="profile-change-email-group-form-controls" data-validation="control">
11            <input type="email" class="profile-change-email-form-group-input" id="new_email" name="new_email" value="" placeholder=
er="{{translate 'your@example.com'}}">
12          </div>
13        </div>
14
15        <div class="profile-change-email-form-group" data-input="confirm_email" data-validation="control-group">
16          <label class="profile-change-email-form-group-label" for="confirm_email">{{translate 'Confirm New Email'}} <span
class="profile-change-email-form-group-label-required">*</span></label>
17          <div class="profile-change-email-group-form-controls" data-validation="control">
18            <input type="email" class="profile-change-email-form-group-input" id="confirm_email" name="confirm_email" val
ue="" placeholder="{{translate 'your@example.com'}}">
19          </div>
20        </div>

```

```

21
22     <div class="profile-change-email-form-group" data-input="current_email" data-validation="control-group">
23         <label class="profile-change-email-form-group-label" for="current_password">{{translate 'Password'}} <span class="pro
file-change-email-form-group-label-required">*</span></label>
24         <div class="profile-change-email-form-group-controls" data-validation="control">
25             <input type="password" class="profile-change-email-form-group-input" id="current_password" name="current_password" val
ue="">
26         </div>
27     </div>
28 </fieldset>
29 <p class="profile-change-email-form-info-block"><small> {{translate 'You will still be able to login with your current email
address and password until your new email address is verified.'}} </small></p>
30 <div class="profile-change-email-form-actions">
31     <button type="submit" class="profile-change-email-form-actions-change">{{translate 'Send Verification Email'}}</button>
32 </div>
33 </form>
34 </div>
35 </section>

```

Modify Profile/Templates/profile_information.tpl

Replace

```

1 | <p class="profile-information-input-email" id="email" name="email" >{{email}}</p>

```

With

```

1 | <p class="profile-information-input-email" id="email" name="email" >{{email}} | <a class="profile-information-change-email-
address" data-action="change-email">{{translate 'Change Address'}}</a></p>

```

Mont Blanc — Change Email Address Patch

 **Applies to:** SuiteCommerce Advanced | Mont Blanc

To update a Mont Blanc implementation to give users the ability to change their email address, you'll need to modify and add the files as detailed below.



Important: This addition requires changes to templates, views, JavaScript, and SSP application files. Test changes thoroughly against your existing customizations before deploying to your published domain.

Modify CheckoutApplication/SuiteScript/checkout.ssp

Replace

```

1 | var SiteSettings
2 | , parameters
3 | , siteType
4 | , Environment
5 | , Language
6 | , Currency
7 | , Error
8 | , cart_bootstrap
9 | , confirmation_order_id

```

```

10 | , login
11 | , Application;

```

With

```

1 | var SiteSettings
2 | , parameters
3 | , siteType
4 | , Environment
5 | , Language
6 | , Currency
7 | , Error
8 | , cart_bootstrap
9 | , confirmation_order_id
10 | , login
11 | , Application
12 | , password_reset_expired
13 | ;

```

Replace

```

1 | if (session.isChangePasswordRequest())
2 | {
3 |     parameters.fragment = 'reset-password';
4 |     login = true;
5 | }

```

With

```

1 | if (parameters.passwdret)
2 | {
3 |     try
4 |     {
5 |         if (session.isChangePasswordRequest())
6 |         {
7 |             parameters.fragment = 'reset-password';
8 |             login = true;
9 |         }
10 |     }
11 |     catch (e)
12 |     {
13 |         password_reset_expired = true;
14 |     }
15 |
16 | }

```

Replace

```

1 | <% if (Error) { %>
2 |     <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3 | <% } %>

```

With

```

1 | <% if (Error) { %>
2 |     <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3 | <% } %>
4 |

```

```

5  <% if (parameters.key) { %>
6    <script>SC.ENVIRONMENT.email_verification_error = true;</script>
7  <% } else if (password_reset_expired) { %>
8    <script>SC.ENVIRONMENT.password_reset_expired_error = true;</script>
9  <% } else if (parameters.password_reset_expired && parameters.fragment !== 'reset-password') { %>
10   <script>SC.ENVIRONMENT.password_reset_invalid_error = true;</script>
11 <% } %>

```

Modify LoginRegister/Javascript/LoginRegister.View.js

Replace

```

1  define('LoginRegister.View'
2  , [ 'login_register.tpl'
3
4    , 'Profile.Model'
5    , 'LoginRegister.Login.View'
6    , 'LoginRegister.Register.View'
7    , 'LoginRegister.CheckoutAsGuest.View'
8    , 'Backbone.CompositeView'
9    , 'SC.Configuration'
10   , 'Header.Simplified.View'
11   , 'Footer.Simplified.View'
12
13   , 'Backbone'
14   , 'underscore'
15   , 'Utils'
16 ]
17 , function (
18   login_register_tpl
19
20   , ProfileModel
21   , LoginView
22   , RegisterView
23   , CheckoutAsGuestView
24   , BackboneCompositeView
25   , Configuration
26   , HeaderSimplifiedView
27   , FooterSimplifiedView
28
29   , Backbone
30   , -
31 )

```

With

```

1  define('LoginRegister.View'
2  , [ 'login_register.tpl'
3
4    , 'GlobalViews.Message.View'
5    , 'Profile.Model'
6    , 'LoginRegister.Login.View'
7    , 'LoginRegister.Register.View'
8    , 'LoginRegister.CheckoutAsGuest.View'
9    , 'Backbone.CompositeView'
10   , 'SC.Configuration'
11   , 'Header.Simplified.View'
12   , 'Footer.Simplified.View'
13
14   , 'Backbone'
15   , 'underscore'
16   , 'Utils'
17 ]
18 , function (
19   login_register_tpl
20

```

```

21 | , GlobalViewsMessageView
22 | , ProfileModel
23 | , LoginView
24 | , RegisterView
25 | , CheckoutAsGuestView
26 | , BackboneCompositeView
27 | , Configuration
28 | , HeaderSimplifiedView
29 | , FooterSimplifiedView
30 |
31 | , Backbone
32 | , -
33 | )

```

Replace

```

1 | this.enableCheckoutAsGuest = is_checking_out && profile_model.get('isLoggedIn') === 'F' &&
2 |   (Configuration.getRegistrationType() === 'optional' || Configuration.getRegistrationType() === 'disabled');
3 |
4 | BackboneCompositeView.add(this);

```

With

```

1 | this.enableCheckoutAsGuest = is_checking_out && profile_model.get('isLoggedIn') === 'F' &&
2 |   (Configuration.getRegistrationType() === 'optional' || Configuration.getRegistrationType() === 'disabled');
3 |
4 | if (SC.ENVIRONMENT.email_verification_error)
5 | {
6 |   this.message = _('The validation process has failed. Please login into your account and click on the validation link
7 | again.').translate();
8 |   delete SC.ENVIRONMENT.email_verification_error;
9 | }
10 | else if (SC.ENVIRONMENT.password_reset_invalid_error)
11 | {
12 |   this.message = _('Your reset password link is invalid. Request a new one using the Forgot Password link.').translate();
13 |   delete SC.ENVIRONMENT.password_reset_invalid_error;
14 | }
15 | else if (SC.ENVIRONMENT.password_reset_expired_error)
16 | {
17 |   this.message = _('Your reset password link has expired. Request a new one using the Forgot Password link.').translate();
18 |   delete SC.ENVIRONMENT.password_reset_expired_error;
19 | }
20 | BackboneCompositeView.add(this);

```

Replace

```

1 | , 'Register': function ()
2 | {
3 |   return new RegisterView(this.child_view_options);
4 | }
5 | }

```

With

```

1 | , 'Register': function ()
2 | {
3 |   return new RegisterView(this.child_view_options);
4 | }
5 | , 'Messages': function ()
6 | {

```

```

7     if (this.message)
8     {
9         return new GlobalViewsMessageView({
10             message: this.message
11             , type: 'error'
12             , closable: true
13         });
14     }
15 }
16 }

```

Modify LoginRegister/Templates/login_register.tpl

Replace

```

1 <header class="login-register-header">
2     {{#if showRegister}}
3     <h1 class="login-register-title"> {{translate 'Log in | Register'}}</h1>
4     {{else}}
5     <h1 class="login-register-title"> {{translate 'Log in'}}</h1>
6     {{/if}}
7 </header>

```

With

```

1 <header class="login-register-header">
2     {{#if showRegister}}
3     <h1 class="login-register-title"> {{translate 'Log in | Register'}}</h1>
4     {{else}}
5     <h1 class="login-register-title"> {{translate 'Log in'}}</h1>
6     {{/if}}
7 </header>
8 <div data-view="Messages"></div>

```

Modify MyAccountApplication/SuiteScript/my_account.ssp

Replace

```

1 <img alt="NetSuite logo" data-bbox="150 650 180 665"/>
2
3 var SiteSettings
4 , siteType
5 , Environment
6 , Language
7 , Currency
8 , Error
9 , Application;
10
11 try
12 {
13     Application = require('Application');
14     SiteSettings = require('SiteSettings.Model').get();
15     siteType = SiteSettings.sitetype;
16
17     Environment = Application.getEnvironment(session, request);
18
19     Language = Environment.currentLanguage && Environment.currentLanguage.locale || '';
20     Currency = Environment.currencyCodeSpecifiedOnUrl;
21
22     // Access control, if you are not logged this will send you to the log in page
23     if (!session.isLoggedIn2() || session.getCustomer().isGuest())

```

```

24     {
25         var parameters = request.getAllParameters();
26
27         delete parameters.sitepath;
28         parameters.origin = 'customercenter';
29
30         if (parameters.fragment)
31         {
32             parameters.origin_hash = parameters.fragment;
33             delete parameters.fragment;
34         }
35
36         return nlapiSetRedirectURL('EXTERNAL', SiteSettings.touchpoints.login, null, false, parameters);
37     }
38 } catch (e) {
39     Error = Application.processError(e);
40 }
41
42 %>

```

With

```

1  <%
2
3  var SiteSettings
4  , siteType
5  , Environment
6  , Language
7  , Currency
8  , Error
9  , Application
10 , parameters
11 , email_change_verification
12 ;
13
14 try
15 {
16     Application = require('Application');
17     SiteSettings = require('SiteSettings.Model').get();
18     parameters = request.getAllParameters();
19     siteType = SiteSettings.siteType;
20
21     Environment = Application.getEnvironment(session, request);
22
23     Language = Environment.currentLanguage && Environment.currentLanguage.locale || '';
24     Currency = Environment.currencyCodeSpecifiedOnUrl;
25
26     // Access control, if you are not logged this will send you to the log in page
27     if (!session.isLoggedIn2() || session.getCustomer().isGuest())
28     {
29         delete parameters.sitepath;
30         parameters.origin = 'customercenter';
31
32         if (parameters.fragment)
33         {
34             parameters.origin_hash = parameters.fragment;
35             delete parameters.fragment;
36         }
37
38         return nlapiSetRedirectURL('EXTERNAL', SiteSettings.touchpoints.login, null, false, parameters);
39     }
40     else if (session.isLoggedIn2() && parameters.key)
41     {
42         try
43         {
44             session.verifyEmailChange(parameters.key)
45             email_change_verification = true;
46         }
47         catch (e)
48         {
49             email_change_verification = e.details;

```



```

50     }
51   }
52   } catch (e) {
53     Error = Application.processError(e);
54   }
55   %>

```

Replace

```

1  <% if (Error) { %>
2  <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3  <% } %>

```

With

```

1  <% if (Error) { %>
2  <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3  <% } %>
4  <% if (email_change_verification) { %>
5  <script>SC.SESSION.email_change_verification = '<%= email_change_verification %>';</script>
6  <% } %>

```

Modify Overview/Javascript/Overview.Home.View.js

Replace

```

1  define('Overview.Home.View'
2  , [
3    'SC.Configuration'
4    , 'Overview.Banner.View'
5    , 'Overview.Profile.View'
6    , 'Overview.Payment.View'
7    , 'Overview.Shipping.View'
8    , 'Backbone.CollectionView'
9    , 'OrderHistory.List.Tracking.Number.View'
10   , 'RecordViews.View'
11   , 'Handlebars'
12
13   , 'overview_home.tpl'
14
15   , 'Backbone'
16   , 'Backbone.CompositeView'
17   , 'underscore'
18   , 'Utils'
19   ]
20  , function(
21    Configuration
22    , OverviewBannerView
23    , OverviewProfileView
24    , OverviewPaymentView
25    , OverviewShippingView
26    , BackboneCollectionView
27    , OrderHistoryListTrackingNumberView
28    , RecordViewsView
29    , Handlebars
30
31    , overview_home_tpl
32
33    , Backbone
34    , BackboneCompositeView
35    , -
36  )

```

With

```

1  define('Overview.Home.View'
2  , [
3    'SC.Configuration'
4    , 'GlobalViews.Message.View'
5    , 'Overview.Banner.View'
6    , 'Overview.Profile.View'
7    , 'Overview.Payment.View'
8    , 'Overview.Shipping.View'
9    , 'Backbone.CollectionView'
10   , 'OrderHistory.List.Tracking.Number.View'
11   , 'RecordViews.View'
12   , 'Handlebars'
13
14   , 'overview_home.tpl'
15
16   , 'Backbone'
17   , 'Backbone.CompositeView'
18   , 'underscore'
19   , 'Utils'
20 ]
21 , function(
22   Configuration
23   , GlobalViewsMessageView
24   , OverviewBannerView
25   , OverviewProfileView
26   , OverviewPaymentView
27   , OverviewShippingView
28   , BackboneCollectionView
29   , OrderHistoryListTrackingNumberView
30   , RecordViewsView
31   , Handlebars
32
33   , overview_home_tpl
34
35   , Backbone
36   , BackboneCompositeView
37   , -
38 )

```

Replace

```

1  this.creditcards.on('reset destroy change add', this.showContent, this);
2  }

```

With

```

1  this.creditcards.on('reset destroy change add', this.showContent, this);
2
3  if (SC.SESSION.email_change_verification)
4  {
5    this.email_change_verification = SC.SESSION.email_change_verification;
6    delete SC.SESSION.email_change_verification;
7  }
8  }

```

Replace

```

1  , 'Overview.Shipping': function()
2  {
3    return new OverviewShippingView({ model: this.defaultShippingAddress });
4  }

```

With

```

1  , 'Overview.Shipping': function()
2  {
3      return new OverviewShippingView({ model: this.defaultShippingAddress });
4  }
5
6  , 'Overview.Messages': function ()
7  {
8      if (this.email_change_verification)
9      {
10         return new GlobalViewsMessageView({
11             message: this.email_change_verification === 'true' ? _('Your email has been changed successfully to <strong>').trans
late() + this.model.get('email') + '</strong>' : this.email_change_verification
12             , type: this.email_change_verification === 'true' ? 'success' : 'error'
13             , closable: true
14         });
15     }
16 }

```

Modify Overview/Templates/overview_home.tpl

Replace

```

1  <section class="overview-home">
2  <div class="overview-home-orders" data-permissions="{purchasesPermissions}">

```

With

```

1  <section class="overview-home">
2  <div data-view="Overview.Messages"></div>
3  <div class="overview-home-orders" data-permissions="{purchasesPermissions}">

```

Add Profile/Javascript/Profile.ChangeEmailAddress.Model.js

```

1  // Profile.ChangeEmailAddress.Model.js
2  // =====
3  // View Model for changing user's email
4  // @module Profile
5  define(
6      'Profile.ChangeEmailAddress.Model'
7      , [
8          'Backbone'
9          , 'underscore'
10         , 'Utils'
11     ]
12     , function (
13         Backbone
14         , _
15     ) {
16         'use strict';
17
18         // @class Profile.ChangeEmailAddress.Model @extends Backbone.Model
19         return Backbone.Model.extend(
20             {
21                 urlRoot: 'services/Profile.Service.ss'
22             , validation: {
23                 current_password: { required: true, msg: _('Current password is required').translate() }
24                 , confirm_email: [
25                     { required: true, msg: _('Confirm Email is required').translate() }
26                     , { equalTo: 'new_email', msg: _('New Email and Confirm New Email do not match').translate() }
27                 ]
28                 , new_email: { required: true, msg: _('New Email is required').translate() }
29             }
30         );
31     }
32 );

```

```

30     }
31   });
32 });

```

Add Profile/Javascript/Profile.ChangeEmailAddress.View.js

```

1  // @module Profile
2  define(
3    'Profile.ChangeEmailAddress.View'
4    , [
5      'GlobalViews.Message.View'
6      , 'Backbone.FormView'
7      , 'SC.Configuration'
8    ]
9    , 'profile_change_email.tpl'
10
11    , 'Backbone'
12    , 'underscore'
13    , 'Utils'
14  ]
15  , function (
16    GlobalViewsMessageView
17    , BackboneFormView
18    , Configuration
19
20    , profile_change_email_tpl
21
22    , Backbone
23    , _
24  )
25  {
26    'use strict';
27
28    // @class Profile.ChangeEmailAddress.View @extends Backbone.View
29    return Backbone.View.extend({
30
31      template: profile_change_email_tpl
32
33      , page_header: _('Change Email').translate()
34
35      , title: _('Change Email').translate()
36
37      , events: {
38        'submit form': 'saveFormCustom'
39      }
40
41      , bindings: {
42        '[name="current_password"]': 'current_password'
43        , '[name="new_email"]': 'new_email'
44        , '[name="confirm_email"]': 'confirm_email'
45      }
46
47      , initialize: function()
48      {
49        Backbone.View.prototype.initialize.apply(this, arguments);
50        BackboneFormView.add(this);
51      }
52
53      , saveFormCustom: function ()
54      {
55        this.new_email = this.$('[name="new_email"]').val();
56        BackboneFormView.saveForm.apply(this, arguments);
57      }
58
59      , showSuccess: function (placeholder)
60      {
61        var global_view_message = new GlobalViewsMessageView({
62          message: _('A confirmation email has been sent to <strong>').translate() + this.new_email + '</strong>'
63          , type: 'success'
64          , closable: true
65        });

```

```

66     placeholder.html(global_view_message.render().$el.html());
67   }
68   });
69   });
70 });

```

Modify Profile/Javascript/Profile.Information.View.js

Replace

```

1  define(
2    'Profile.Information.View'
3  , [
4    'SC.Configuration'
5    , 'GlobalViews.Message.View'
6    , 'Backbone.FormView'
7
8    , 'profile_information_tpl'
9
10   , 'Backbone'
11   , 'underscore'
12   , 'jQuery'
13   , 'Utils'
14 ]
15 , function (
16   Configuration
17   , GlobalViewsMessageView
18   , BackboneFormView
19
20   , profile_information_tpl
21
22   , Backbone
23   , _
24   , jQuery
25 )
26 {
27   'use strict';
28
29   // @class Profile.Information.View @extends Backbone.View
30   return Backbone.View.extend({
31
32     template: profile_information_tpl
33     , page_header: _('Profile Information').translate()
34     , title: _('Profile Information').translate()
35     , attributes: {'class': 'ProfileInformationView'}
36     , events: {
37       'submit form': 'saveForm'
38       , 'change input[data-type="phone]': 'formatPhone'
39     }
40
41     , bindings: {
42       '[name="firstname"]': 'firstname'
43       , '[name="lastname"]': 'lastname'
44       , '[name="companyname"]': 'companyname'
45       , '[name="phone"]': 'phone'
46     }
47
48     , initialize: function()
49     {
50       BackboneFormView.add(this);
51     }
52
53     , formatPhone: function (e)
54     {
55       var $target = jQuery(e.target);
56       $target.val(_($target.val()).formatPhone());
57     }
58
59     , showSuccess: function ()

```

With

```

1  define(
2      'Profile.Information.View'
3      , [
4          'SC.Configuration'
5          , 'GlobalViews.Message.View'
6          , 'Backbone.FormView'
7
8          , 'Profile.ChangeEmailAddress.Model'
9          , 'Profile.ChangeEmailAddress.View'
10
11         , 'profile_information_tpl'
12
13         , 'Backbone'
14         , 'underscore'
15         , 'jQuery'
16         , 'Utils'
17     ]
18     , function (
19         Configuration
20         , GlobalViewsMessageView
21         , BackboneFormView
22
23         , ProfileChangeEmailModel
24         , ProfileChangeEmailView
25
26         , profile_information_tpl
27
28         , Backbone
29         , _
30         , jQuery
31     )
32     {
33         'use strict';
34
35         // @class Profile.Information.View @extends Backbone.View
36         return Backbone.View.extend({
37
38             template: profile_information_tpl
39             , page_header: _('Profile Information').translate()
40             , title: _('Profile Information').translate()
41             , attributes: {'class': 'ProfileInformationView'}
42             , events: {
43                 'submit form': 'saveForm'
44                 , 'change input[data-type="phone"]': 'formatPhone'
45                 , 'click [data-action="change-email"]': 'changeEmail'
46             }
47
48             , bindings: {
49                 '[name="firstname"]': 'firstname'
50                 , '[name="lastname"]': 'lastname'
51                 , '[name="companyname"]': 'companyname'
52                 , '[name="phone"]': 'phone'
53             }
54
55             , initialize: function(options)
56             {
57                 BackboneFormView.add(this);
58                 this.application = options.application;
59             }
60
61             , formatPhone: function (e)
62             {
63                 var $target = jQuery(e.target);
64                 $target.val(_($target.val()).formatPhone());
65             }
66
67             , changeEmail: function ()
68             {
69                 var model = new ProfileChangeEmailModel(this.model.attributes);
70

```

```

71     var view = new ProfileChangeEmailView({
72         application: this.application
73         , model: model
74     });
75
76     var self = this;
77
78     model.on('save', function () {
79         view.showSuccess(self.$('[data-type="alert-placeholder"]'));
80     });
81
82     view.useLayoutError = true;
83
84     this.application.getLayout().showInModal(view);
85 }
86
87 , showSuccess: function ()

```

Add Profile/Sass/_profile-change-email.scss

```

1  .profile-change-email-button-back {
2      @extend .button-back;
3  }
4
5  .profile-change-email-button-back-icon {
6      @extend .button-back-icon;
7  }
8
9  .profile-change-email-form-label {
10     display: inline-block;
11 }
12
13 .profile-change-email-form-group-label-required {
14     @extend .input-required;
15 }
16
17 .profile-change-email {
18     @extend .address-edit;
19 }
20
21 .profile-change-email-form-title {}
22 .profile-change-email-form-area {}
23
24 .profile-change-email-form {
25     margin-top: $sc-base-margin * 3;
26 }
27
28 .profile-change-email-form-group,
29 .profile-change-email-form-actions {
30     @extend .control-group;
31 }
32
33 .profile-change-email-form-group-label {
34     @extend .input-label
35 }
36
37 .profile-change-email-form-group-label-required {
38 }
39
40 .profile-change-email-form-group-input {
41     @extend .input-large
42 }
43
44 .profile-change-email-form-actions-change {
45     @extend .button-primary;
46     @extend .button-medium;
47 }
48
49 .profile-change-email-group-form-controls{}
50
51 .profile-change-email-form-info-block{}

```

Modify Profile/SuiteScript/Profile.Model.js

Replace

```

1  define(
2    'Profile.Model'
3    , ['SC.Model', 'Utils']
4    , function (SCModel, Utils)

```

With

```

1  define(
2    'Profile.Model'
3    , ['SC.Model', 'Models.Init', 'Utils']
4    , function (SCModel, ModelsInit, Utils)

```

Replace

```

1  if (data.email && data.email !== this.currentSettings.email && data.email === data.confirm_email)
2  {
3    if(data.isGuest === 'T')
4    {
5      customerUpdate.email = data.email;
6    }
7    else
8    {
9      login.changeEmail(data.current_password, data.email, true);
10   }
11 }

```

With

```

1  if (data.email && data.email !== this.currentSettings.email && data.email === data.confirm_email && data.isGuest === 'T')
2  {
3    customerUpdate.email = data.email;
4  }
5  else if (data.new_email && data.new_email === data.confirm_email && data.new_email !== this.currentSettings.email)
6  {
7    ModelsInit.session.login({
8      email: data.email
9      , password: data.current_password
10    });
11    login.changeEmail(data.current_password, data.new_email, true);
12  }

```

Add Profile/Templates/profile_change_email.tpl

```

1  <section class="profile-change-email">
2    <div data-type="alert-placeholder"></div>
3    <div class="profile-change-email-form-area">
4      <form class="profile-change-email-form">
5        <fieldset>
6          <small class="profile-change-email-form-label">{{translate 'Required'}}<span class="profile-change-email-form-group-label-
required">*</span></small>
7
8          <div class="profile-change-email-form-group" data-input="new_email" data-validation="control-group">
9            <label class="profile-change-email-form-group-label" for="new_email">{{translate 'New Email'}}<span class="pro
file-change-email-form-group-label-required">*</span></label>
10           <div class="profile-change-email-group-form-controls" data-validation="control">
11             <input type="email" class="profile-change-email-form-group-input" id="new_email" name="new_email" value="" placeholder=
"{{translate 'your@example.com'}}">

```



```

12     </div>
13 </div>
14
15     <div class="profile-change-email-form-group" data-input="confirm_email" data-validation="control-group">
16         <label class="profile-change-email-form-group-label" for="confirm_email">{{translate 'Confirm New Email'}} <span
class="profile-change-email-form-group-label-required">*</span></label>
17         <div class="profile-change-email-group-form-controls" data-validation="control">
18             <input type="email" class="profile-change-email-form-group-input" id="confirm_email" name="confirm_email" val
ue="" placeholder="{{translate 'your@example.com'}}">
19         </div>
20     </div>
21
22     <div class="profile-change-email-form-group" data-input="current_email" data-validation="control-group">
23         <label class="profile-change-email-form-group-label" for="current_password">{{translate 'Password'}} <span class="pro
file-change-email-form-group-label-required">*</span></label>
24         <div class="profile-change-email-group-form-controls" data-validation="control">
25             <input type="password" class="profile-change-email-form-group-input" id="current_password" name="current_password" val
ue="">
26         </div>
27     </div>
28 </fieldset>
29 <p class="profile-change-email-form-info-block"><small> {{translate 'You will still be able to login with your current email
address and password until your new email address is verified.'}} </small></p>
30 <div class="profile-change-email-form-actions">
31     <button type="submit" class="profile-change-email-form-actions-change">{{translate 'Send Verification Email'}}</button>
32 </div>
33 </form>
34 </div>
35 </section>

```

Modify Profile/Templates/profile_information.tpl

Replace

```

1 | <p class="profile-information-input-email" id="email">{{email}}</p>

```

With

```

1 | <p class="profile-information-input-email" id="email">{{email}} | <a class="profile-information-change-email-address" data-
action="change-email">{{translate 'Change Address'}}</a></p>

```

Vinson — Change Email Address Patch

i Applies to: SuiteCommerce Advanced | Vinson

To update a Vinson implementation to give users the ability to change their email address, you'll need to modify and add the files as detailed below.



Important: This addition requires changes to templates, views, JavaScript, and SSP application files. Test changes thoroughly against your existing customizations before deploying to your published domain.

Modify CheckoutApplication/SuiteScript/checkout.ssp

Replace

```

1 | var SiteSettings
2 | , parameters
3 | , siteType

```

```

4 | , Environment
5 | , Language
6 | , Currency
7 | , Error
8 | , login
9 | , Application
10 | , environmentParameters

```

With

```

1 | var SiteSettings
2 | , parameters
3 | , siteType
4 | , Environment
5 | , Language
6 | , Currency
7 | , Error
8 | , login
9 | , Application
10 | , environmentParameters
11 | , password_reset_expired
12 | ;

```

Replace

```

1 | if (session.isChangePasswordRequest())
2 | {
3 |     parameters.fragment = 'reset-password';
4 |     login = true;
5 | }

```

With

```

1 | if (parameters.passwdret)
2 | {
3 |     try
4 |     {
5 |         if (session.isChangePasswordRequest())
6 |         {
7 |             parameters.fragment = 'reset-password';
8 |             login = true;
9 |         }
10 |    }
11 |    catch (e)
12 |    {
13 |        password_reset_expired = true;
14 |    }
15 |
16 | }

```

Replace

```

1 | <% if (Error) { %>
2 |     <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3 | <% } %>

```

With

```

1 | <% if (Error) { %>
2 |     <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3 | <% } %>

```

```

4
5 <% if (parameters.key) { %>
6   <script>SC.ENVIRONMENT.email_verification_error = true;</script>
7 <% } else if (password_reset_expired) { %>
8   <script>SC.ENVIRONMENT.password_reset_expired_error = true;</script>
9 <% } else if (parameters.password_reset_invalid_error) { %>
10   <script>SC.ENVIRONMENT.password_reset_invalid_error = true;</script>
11 <% } %>

```

Modify LoginRegister/Javascript/LoginRegister.View.js

Replace

```

1  define('LoginRegister.View'
2  , [ 'login_register.tpl'
3
4    , 'Profile.Model'
5    , 'LoginRegister.Login.View'
6    , 'LoginRegister.Register.View'
7    , 'LoginRegister.CheckoutAsGuest.View'
8    , 'Backbone.CompositeView'
9    , 'SC.Configuration'
10   , 'Header.Simplified.View'
11   , 'Footer.Simplified.View'
12
13   , 'Backbone'
14   , 'underscore'
15   , 'Utils'
16 ]
17 , function (
18   login_register_tpl
19
20   , ProfileModel
21   , LoginView
22   , RegisterView
23   , CheckoutAsGuestView
24   , BackboneCompositeView
25   , Configuration
26   , HeaderSimplifiedView
27   , FooterSimplifiedView
28
29   , Backbone
30   , _
31 )

```

With

```

1  define('LoginRegister.View'
2  , [ 'login_register.tpl'
3
4    , 'GlobalViews.Message.View'
5    , 'Profile.Model'
6    , 'LoginRegister.Login.View'
7    , 'LoginRegister.Register.View'
8    , 'LoginRegister.CheckoutAsGuest.View'
9    , 'Backbone.CompositeView'
10   , 'SC.Configuration'
11   , 'Header.Simplified.View'
12   , 'Footer.Simplified.View'
13
14   , 'Backbone'
15   , 'underscore'
16   , 'Utils'
17 ]
18 , function (
19   login_register_tpl
20

```

```

21 | , GlobalViewsMessageView
22 | , ProfileModel
23 | , LoginView
24 | , RegisterView
25 | , CheckoutAsGuestView
26 | , BackboneCompositeView
27 | , Configuration
28 | , HeaderSimplifiedView
29 | , FooterSimplifiedView
30 |
31 | , Backbone
32 | , -
33 | )

```

Replace

```

1 | this.enableCheckoutAsGuest = is_checking_out && profile_model.get('isLoggedIn') === 'F' &&
2 |   (Configuration.getRegistrationType() === 'optional' || Configuration.getRegistrationType() === 'disabled');
3 |
4 | BackboneCompositeView.add(this);

```

With

```

1 | this.enableCheckoutAsGuest = is_checking_out && profile_model.get('isLoggedIn') === 'F' &&
2 |   (Configuration.getRegistrationType() === 'optional' || Configuration.getRegistrationType() === 'disabled');
3 |
4 | if (SC.ENVIRONMENT.email_verification_error)
5 | {
6 |   this.message = _('The validation process has failed. Please login into your account and click on the validation link
7 | again.').translate();
8 |   delete SC.ENVIRONMENT.email_verification_error;
9 | }
10 | else if (SC.ENVIRONMENT.password_reset_invalid_error)
11 | {
12 |   this.message = _('Your reset password link is invalid. Request a new one using the Forgot Password link.').translate();
13 |   delete SC.ENVIRONMENT.password_reset_invalid_error;
14 | }
15 | else if (SC.ENVIRONMENT.password_reset_expired_error)
16 | {
17 |   this.message = _('Your reset password link has expired. Request a new one using the Forgot Password link.').translate();
18 |   delete SC.ENVIRONMENT.password_reset_expired_error;
19 | }
20 | BackboneCompositeView.add(this);

```

Replace

```

1 | , 'Register': function ()
2 | {
3 |   return new RegisterView(this.child_view_options);
4 | }
5 | }

```

With

```

1 | , 'Register': function ()
2 | {
3 |   return new RegisterView(this.child_view_options);
4 | }
5 | , 'Messages': function ()
6 | {
7 |   if (this.message)
8 |   {
9 |     return new GlobalViewsMessageView({

```

```

10     message: this.message
11     , type: 'error'
12     , closable: true
13   });
14   }
15 }
16 }

```

Modify LoginRegister/Templates/login_register.tpl

Replace

```

1 <header class="login-register-header">
2   {{#if showRegister}}
3   <h1 class="login-register-title"> {{translate 'Log in | Register'}}</h1>
4   {{else}}
5   <h1 class="login-register-title"> {{translate 'Log in'}}</h1>
6   {{/if}}
7 </header>

```

With

```

1 <header class="login-register-header">
2   {{#if showRegister}}
3   <h1 class="login-register-title"> {{translate 'Log in | Register'}}</h1>
4   {{else}}
5   <h1 class="login-register-title"> {{translate 'Log in'}}</h1>
6   {{/if}}
7 </header>
8 <div data-view="Messages"></div>

```

Modify MyAccountApplication/SuiteScript/my_account.ssp

Replace

```

1 <%
2   var SiteSettings
3   , siteType
4   , Environment
5   , Language
6   , Currency
7   , Error
8   , Application
9   , environmentParameters
10  , parameters
11  , external_payment;
12
13  try
14  {
15    SiteSettings = require('SiteSettings.Model').get();
16    parameters = request.getAllParameters();
17
18    // Access control, if you are not logged this will send you to the log in page
19    if (!session.isLoggedIn2() || session.getCustomer().isGuest())
20    {
21      delete parameters.sitepath;
22      parameters.origin = 'customercenter';
23
24      if (parameters.fragment)
25      {
26        parameters.origin_hash = parameters.fragment;
27        delete parameters.fragment;
28      }

```

```

29     return nlapiSetRedirectURL('EXTERNAL', SiteSettings.touchpoints.login, null, false, parameters);
30 }
31
32
33 Application = require('Application');
34 Environment = Application.getEnvironment(request);
35 environmentParameters = [];
36 siteType = SiteSettings.sitetype;
37
38 Language = Environment.currentLanguage && Environment.currentLanguage.locale || '';
39 Currency = Environment.currencyCodeSpecifiedOnUrl;
40
41 environmentParameters.push('lang=' + Language);
42 environmentParameters.push('cur=' + Currency);
43
44 _.each(require('ExternalPayment.Model').getParametersFromRequest(request), function(value, key) {
45     environmentParameters.push(key.concat('=', value));
46 });
47 }
48 catch (e)
49 {
50     Error = Application.processError(e);
51 }
52
53 %>

```

With

```

1 <%
2 var SiteSettings
3 , siteType
4 , Environment
5 , Language
6 , Currency
7 , Error
8 , Application
9 , environmentParameters
10 , parameters
11 , external_payment
12 , email_change_verification
13 ;
14
15 try
16 {
17     Application = require('Application');
18     Environment = Application.getEnvironment(request);
19     environmentParameters = [];
20     SiteSettings = require('SiteSettings.Model').get();
21     parameters = request.getAllParameters();
22
23     // Access control, if you are not logged this will send you to the log in page
24     if (!session.isLoggedIn2() || session.getCustomer().isGuest())
25     {
26         delete parameters.sitepath;
27         parameters.origin = 'customercenter';
28
29         if (parameters.fragment)
30         {
31             parameters.origin_hash = parameters.fragment;
32             delete parameters.fragment;
33         }
34
35         return nlapiSetRedirectURL('EXTERNAL', SiteSettings.touchpoints.login, null, false, parameters);
36     }
37     else if (session.isLoggedIn2() && parameters.key)
38     {
39         try
40         {
41             session.verifyEmailChange(parameters.key)
42             email_change_verification = true;
43         }

```

```

44     catch (e)
45     {
46         email_change_verification = e.details;
47     }
48 }
49
50 siteType = SiteSettings.sitetype;
51
52 Language = Environment.currentLanguage && Environment.currentLanguage.locale || '';
53 Currency = Environment.currencyCodeSpecifiedOnUrl;
54
55 environmentParameters.push('lang=' + Language);
56 environmentParameters.push('cur=' + Currency);
57
58 _.each(require('ExternalPayment.Model').getParametersFromRequest(request), function(value, key) {
59     environmentParameters.push(key.concat('=', value));
60 });
61 }
62 catch (e)
63 {
64     Error = Application.processError(e);
65 }
66
67 %>

```

Replace

```

1 <% if (Error) { %>
2 <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3 <% } %>

```

With

```

1 <% if (Error) { %>
2 <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3 <% } %>
4 <% if (email_change_verification) { %>
5 <script>SC.SESSION.email_change_verification = '<%= email_change_verification %>';</script>
6 <% } %>

```

Modify Overview/Javascript/Overview.Home.View.js

Replace

```

1 define('Overview.Home.View'
2 , [
3     'SC.Configuration'
4     , 'Overview.Banner.View'
5     , 'Overview.Profile.View'
6     , 'Overview.Payment.View'
7     , 'Overview.Shipping.View'
8     , 'Backbone.CollectionView'
9     , 'OrderHistory.List.Tracking.Number.View'
10    , 'RecordViews.View'
11    , 'Handlebars'
12
13    , 'overview_home.tpl'
14
15    , 'Backbone'
16    , 'Backbone.CompositeView'
17    , 'underscore'
18    , 'Utils'
19    ]
20 , function(
21     Configuration

```

```

22 | , OverviewBannerView
23 | , OverviewProfileView
24 | , OverviewPaymentView
25 | , OverviewShippingView
26 | , BackboneCollectionView
27 | , OrderHistoryListTrackingNumberView
28 | , RecordViewsView
29 | , Handlebars
30 |
31 | , overview_home_tpl
32 |
33 | , Backbone
34 | , BackboneCompositeView
35 | , -
36 | )

```

With

```

1 | define('Overview.Home.View'
2 | , [
3 |   'SC.Configuration'
4 |   , 'GlobalViews.Message.View'
5 |   , 'Overview.Banner.View'
6 |   , 'Overview.Profile.View'
7 |   , 'Overview.Payment.View'
8 |   , 'Overview.Shipping.View'
9 |   , 'Backbone.CollectionView'
10 |   , 'OrderHistory.List.Tracking.Number.View'
11 |   , 'RecordViews.View'
12 |   , 'Handlebars'
13 |
14 |   , 'overview_home.tpl'
15 |
16 |   , 'Backbone'
17 |   , 'Backbone.CompositeView'
18 |   , 'underscore'
19 |   , 'Utils'
20 | ]
21 | , function(
22 |   Configuration
23 |   , GlobalViewsMessageView
24 |   , OverviewBannerView
25 |   , OverviewProfileView
26 |   , OverviewPaymentView
27 |   , OverviewShippingView
28 |   , BackboneCollectionView
29 |   , OrderHistoryListTrackingNumberView
30 |   , RecordViewsView
31 |   , Handlebars
32 |
33 |   , overview_home_tpl
34 |
35 |   , Backbone
36 |   , BackboneCompositeView
37 |   , -
38 | )

```

Replace

```

1 | this.creditcards.on('reset destroy change add', this.showContent, this);
2 | }

```

With

```

1 | this.creditcards.on('reset destroy change add', this.showContent, this);
2 |

```



```

3 | if (SC.SESSION.email_change_verification)
4 | {
5 |     this.email_change_verification = SC.SESSION.email_change_verification;
6 |     delete SC.SESSION.email_change_verification;
7 | }
8 | }

```

Replace

```

1 | , 'Overview.Shipping': function()
2 | {
3 |     return new OverviewShippingView({ model: this.defaultShippingAddress });
4 | }

```

With

```

1 | , 'Overview.Shipping': function()
2 | {
3 |     return new OverviewShippingView({ model: this.defaultShippingAddress });
4 | }
5 |
6 | , 'Overview.Messages': function ()
7 | {
8 |     if (this.email_change_verification)
9 |     {
10 |         return new GlobalViewsMessageView({
11 |             message: this.email_change_verification === 'true' ? _('Your email has been changed successfully to <strong>').trans
late() + this.model.get('email') + '</strong>' : this.email_change_verification
12 |             , type: this.email_change_verification === 'true' ? 'success' : 'error'
13 |             , closable: true
14 |         });
15 |     }
16 | }

```

Modify Overview/Templates/overview_home.tpl

Replace

```

1 | <section class="overview-home">
2 |     <div class="overview-home-orders" data-permissions="{purchasesPermissions}">

```

With

```

1 | <section class="overview-home">
2 |     <div data-view="Overview.Messages"></div>
3 |     <div class="overview-home-orders" data-permissions="{purchasesPermissions}">

```

Add Profile/Javascript/Profile.ChangeEmailAddress.Model.js

```

1 | // Profile.ChangeEmailAddress.Model.js
2 | // =====
3 | // View Model for changing user's email
4 | // @module Profile
5 | define(
6 |     'Profile.ChangeEmailAddress.Model'
7 |     , [
8 |         'Backbone'
9 |         , 'underscore'
10 |         , 'Utils'
11 |     ]

```

```

12 , function (
13     Backbone
14     , -
15 ) -
16 {
17     'use strict';
18
19     // @class Profile.ChangeEmailAddress.Model @extends Backbone.Model
20     return Backbone.Model.extend(
21     {
22         urlRoot: 'services/Profile.Service.ss'
23     , validation: {
24         current_password: { required: true, msg: _('Current password is required').translate() }
25     , confirm_email: [
26         { required: true, msg: _('Confirm Email is required').translate() }
27         , { equalTo: 'new_email', msg: _('New Email and Confirm New Email do not match').translate() }
28         ]
29     , new_email: { required: true, msg: _('New Email is required').translate() }
30     }
31     );
32 });

```

Add Profile/Javascript/Profile.ChangeEmailAddress.View.js

```

1 // @module Profile
2 define(
3     'Profile.ChangeEmailAddress.View'
4     , [
5         'GlobalViews.Message.View'
6         , 'Backbone.FormView'
7         , 'SC.Configuration'
8
9         , 'profile_change_email.tpl'
10
11         , 'Backbone'
12         , 'underscore'
13         , 'Utils'
14     ]
15     , function (
16         GlobalViewsMessageView
17         , BackboneFormView
18         , Configuration
19
20         , profile_change_email_tpl
21
22         , Backbone
23         , -
24     )
25     {
26         'use strict';
27
28         // @class Profile.ChangeEmailAddress.View @extends Backbone.View
29         return Backbone.View.extend({
30
31             template: profile_change_email_tpl
32
33             , page_header: _('Change Email').translate()
34
35             , title: _('Change Email').translate()
36
37             , events: {
38                 'submit form': 'saveFormCustom'
39             }
40
41             , bindings: {
42                 '[name="current_password"]': 'current_password'
43                 , '[name="new_email"]': 'new_email'
44                 , '[name="confirm_email"]': 'confirm_email'
45             }
46
47             , initialize: function()

```

```

48     {
49         Backbone.View.prototype.initialize.apply(this, arguments);
50         BackboneFormView.add(this);
51     }
52
53     , saveFormCustom: function ()
54     {
55         this.new_email = this.$('[name="new_email"]').val();
56         BackboneFormView.saveForm.apply(this, arguments);
57     }
58
59     , showSuccess: function (placeholder)
60     {
61         var global_view_message = new GlobalViewsMessageView({
62             message: _('A confirmation email has been sent to <strong>').translate() + this.new_email + '</strong>'
63             , type: 'success'
64             , closable: true
65         });
66
67         placeholder.html(global_view_message.render().$el.html());
68     }
69     });
70 });

```

Modify Profile/Javascript/Profile.Information.View.js

Replace

```

1  define(
2      'Profile.Information.View'
3      , [
4          'SC.Configuration'
5          , 'GlobalViews.Message.View'
6          , 'Backbone.FormView'
7
8          , 'profile_information_tpl'
9
10         , 'Backbone'
11         , 'underscore'
12         , 'jQuery'
13         , 'Utils'
14     ]
15     , function (
16         Configuration
17         , GlobalViewsMessageView
18         , BackboneFormView
19
20         , profile_information_tpl
21
22         , Backbone
23         , _
24         , jQuery
25     )
26     {
27         'use strict';
28
29         // @class Profile.Information.View @extends Backbone.View
30         return Backbone.View.extend({
31
32             template: profile_information_tpl
33             , page_header: _('Profile Information').translate()
34             , title: _('Profile Information').translate()
35             , attributes: {'class': 'ProfileInformationView'}
36             , events: {
37                 'submit form': 'saveForm'
38                 , 'change input[data-type="phone]': 'formatPhone'
39             }
40
41             , bindings: {
42                 '[name="firstname]': 'firstname'

```

```

43     , '[name="lastname"]': 'lastname'
44     , '[name="companyname"]': 'companyname'
45     , '[name="phone"]': 'phone'
46   }
47
48   , initialize: function()
49   {
50     BackboneFormView.add(this);
51   }
52
53   , formatPhone: function (e)
54   {
55     var $target = jQuery(e.target);
56     $target.val(_($target.val()).formatPhone());
57   }
58
59   , showSuccess: function ()

```

With

```

1  define(
2    'Profile.Information.View'
3  , [
4    'SC.Configuration'
5    , 'GlobalViews.Message.View'
6    , 'Backbone.FormView'
7
8    , 'Profile.ChangeEmailAddress.Model'
9    , 'Profile.ChangeEmailAddress.View'
10
11    , 'profile_information_tpl'
12
13    , 'Backbone'
14    , 'underscore'
15    , 'jQuery'
16    , 'Utils'
17  ]
18  , function (
19    Configuration
20    , GlobalViewsMessageView
21    , BackboneFormView
22
23    , ProfileChangeEmailModel
24    , ProfileChangeEmailView
25
26    , profile_information_tpl
27
28    , Backbone
29    , _
30    , jQuery
31  )
32  {
33    'use strict';
34
35    // @class Profile.Information.View @extends Backbone.View
36    return Backbone.View.extend({
37
38      template: profile_information_tpl
39    , page_header: _('Profile Information').translate()
40    , title: _('Profile Information').translate()
41    , attributes: {'class': 'ProfileInformationView'}
42    , events: {
43      'submit form': 'saveForm'
44      , 'change input[data-type="phone"]': 'formatPhone'
45      , 'click [data-action="change-email"]': 'changeEmail'
46    }
47
48    , bindings: {
49      '[name="firstname"]': 'firstname'
50      , '[name="lastname"]': 'lastname'
51      , '[name="companyname"]': 'companyname'

```

```

52     , '[name="phone"]': 'phone'
53   }
54
55   , initialize: function(options)
56   {
57     BackboneFormView.add(this);
58     this.application = options.application;
59   }
60
61   , formatPhone: function (e)
62   {
63     var $target = jQuery(e.target);
64     $target.val(_($target.val()).formatPhone());
65   }
66
67   , changeEmail: function ()
68   {
69     var model = new ProfileChangeEmailModel(this.model.attributes);
70
71     var view = new ProfileChangeEmailView({
72       application: this.application
73       , model: model
74     });
75
76     var self = this;
77
78     model.on('save', function () {
79       view.showSuccess(self.$('[data-type="alert-placeholder"]'));
80     });
81
82     view.useLayoutError = true;
83
84     this.application.getLayout().showInModal(view);
85   }
86
87   , showSuccess: function ()

```

Add Profile/Sass/_profile-change-email.scss

```

1  .profile-change-email-button-back {
2    @extend .button-back;
3  }
4
5  .profile-change-email-button-back-icon {
6    @extend .button-back-icon;
7  }
8
9  .profile-change-email-form-label {
10    display: inline-block;
11  }
12
13  .profile-change-email-form-group-label-required {
14    @extend .input-required;
15  }
16
17  .profile-change-email {
18    @extend .address-edit;
19  }
20
21  .profile-change-email-form-title {}
22  .profile-change-email-form-area {}
23
24  .profile-change-email-form {
25    margin-top: $sc-base-margin * 3;
26  }
27
28  .profile-change-email-form-group,
29  .profile-change-email-form-actions {
30    @extend .control-group;
31  }
32

```

```

33 | .profile-change-email-form-group-label {
34 |     @extend .input-label
35 | }
36 |
37 | .profile-change-email-form-group-label-required {
38 | }
39 |
40 | .profile-change-email-form-group-input {
41 |     @extend .input-large
42 | }
43 |
44 | .profile-change-email-form-actions-change {
45 |     @extend .button-primary;
46 |     @extend .button-medium;
47 | }
48 |
49 | .profile-change-email-group-form-controls{}
50 |
51 | .profile-change-email-form-info-block{}

```

Modify Profile/SuiteScript/Profile.Model.js

Replace

```

1 | if (data.email && data.email !== this.currentSettings.email && data.email === data.confirm_email)
2 | {
3 |     if(data.isGuest === 'T')
4 |     {
5 |         customerUpdate.email = data.email;
6 |     }
7 |     else
8 |     {
9 |         login.changeEmail(data.current_password, data.email, true);
10 |     }
11 | }

```

With

```

1 | if (data.email && data.email !== this.currentSettings.email && data.email === data.confirm_email && data.isGuest === 'T')
2 | {
3 |     customerUpdate.email = data.email;
4 | }
5 | else if (data.new_email && data.new_email === data.confirm_email && data.new_email !== this.currentSettings.email)
6 | {
7 |     ModelsInit.session.login({
8 |         email: data.email
9 |         , password: data.current_password
10 |     });
11 |     login.changeEmail(data.current_password, data.new_email, true);
12 | }

```

Add Profile/Templates/profile_change_email.tpl

```

1 | <section class="profile-change-email">
2 |     <div data-type="alert-placeholder"></div>
3 |     <div class="profile-change-email-form-area">
4 |         <form class="profile-change-email-form">
5 |             <fieldset>
6 |                 <small class="profile-change-email-form-label">{{translate 'Required'}} <span class="profile-change-email-form-group-label-
required">*</span></small>
7 |
8 |                 <div class="profile-change-email-form-group" data-input="new_email" data-validation="control-group">
9 |                     <label class="profile-change-email-form-group-label" for="new_email">{{translate 'New Email'}} <span class="pro
file-change-email-form-group-label-required">*</span></label>
10 |                     <div class="profile-change-email-group-form-controls" data-validation="control">

```

```

11         <input type="email" class="profile-change-email-form-group-input" id="new_email" name="new_email" value="" placeholder
12         er="{{translate 'your@example.com'}}">
13     </div>
14 </div>
15     <div class="profile-change-email-form-group" data-input="confirm_email" data-validation="control-group">
16         <label class="profile-change-email-form-group-label" for="confirm_email">{{translate 'Confirm New Email'}} <span
17         class="profile-change-email-form-group-label-required">*</span></label>
18         <div class="profile-change-email-group-form-controls" data-validation="control">
19             <input type="email" class="profile-change-email-form-group-input" id="confirm_email" name="confirm_email" val
20             ue="" placeholder="{{translate 'your@example.com'}}">
21         </div>
22     </div>
23 </div>
24     <div class="profile-change-email-form-group" data-input="current_email" data-validation="control-group">
25         <label class="profile-change-email-form-group-label" for="current_password">{{translate 'Password'}} <span class="pro
26         file-change-email-form-group-label-required">*</span></label>
27         <div class="profile-change-email-group-form-controls" data-validation="control">
28             <input type="password" class="profile-change-email-form-group-input" id="current_password" name="current_password" val
29             ue="">
30         </div>
31     </div>
32 </div>
33 </div>
34 </div>
35 </div>
36 </div>
37 </div>
38 </div>
39 </div>
40 </div>
41 </div>
42 </div>
43 </div>
44 </div>
45 </div>
46 </div>
47 </div>
48 </div>
49 </div>
50 </div>
51 </div>
52 </div>
53 </div>
54 </div>
55 </div>
56 </div>
57 </div>
58 </div>
59 </div>
60 </div>
61 </div>
62 </div>
63 </div>
64 </div>
65 </div>
66 </div>
67 </div>
68 </div>
69 </div>
70 </div>
71 </div>
72 </div>
73 </div>
74 </div>
75 </div>
76 </div>
77 </div>
78 </div>
79 </div>
80 </div>
81 </div>
82 </div>
83 </div>
84 </div>
85 </div>
86 </div>
87 </div>
88 </div>
89 </div>
90 </div>
91 </div>
92 </div>
93 </div>
94 </div>
95 </div>
96 </div>
97 </div>
98 </div>
99 </div>
100 </div>

```

Modify Profile/Templates/profile_information.tpl

Replace

```

1 <p class="profile-information-input-email" id="email">{{email}}</p>

```

With

```

1 <p class="profile-information-input-email" id="email">{{email}} | <a class="profile-information-change-email-address" data-
  action="change-email">{{translate 'Change Address'}}</a></p>

```

Elbrus — Change Email Address Patch

Applies to: SuiteCommerce Advanced | Elbrus

To update an Elbrus implementation to give users the ability to change their email address, you'll need to modify and add the files as detailed below.



Important: This addition requires changes to templates, views, JavaScript, and SSP application files. Test changes thoroughly against your existing customizations before deploying to your published domain.

Modify CheckoutApplication/SuiteScript/checkout.ssp

Replace

```

1 var SiteSettings

```

```

2      , parameters
3      , siteType
4      , Environment
5      , Language
6      , Currency
7      , Error
8      , login
9      , order
10     , session
11     , Application
12     , environmentParameters
13     ;

```

With

```

1      var SiteSettings
2      , parameters
3      , siteType
4      , Environment
5      , Language
6      , Currency
7      , Error
8      , login
9      , order
10     , session
11     , Application
12     , environmentParameters
13     , password_reset_expired
14     ;

```

Replace

```

1      if (session.isChangePasswordRequest())
2      {
3          parameters.fragment = 'reset-password';
4          login = true;
5      }

```

With

```

1      if (parameters.passwdret)
2      {
3          try
4          {
5              if (session.isChangePasswordRequest())
6              {
7                  parameters.fragment = 'reset-password';
8                  login = true;
9              }
10             }
11             catch (e)
12             {
13                 password_reset_expired = true;
14             }
15         }
16     }

```

Replace

```

1      <% if (Error) { %>
2          <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>

```



```
3 | <% } %>
```

With

```
1 | <% if (Error) { %>
2 |   <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3 | <% } %>
4 |
5 | <% if (parameters.key) { %>
6 |   <script>SC.ENVIRONMENT.email_verification_error = true;</script>
7 | <% } else if (password_reset_expired) { %>
8 |   <script>SC.ENVIRONMENT.password_reset_expired_error = true;</script>
9 | <% } else if (parameters.password_reset && parameters.fragment !== 'reset-password') { %>
10 |   <script>SC.ENVIRONMENT.password_reset_invalid_error = true;</script>
11 | <% } %>
```

Modify LoginRegister/Javascript/LoginRegister.View.js

Replace

```
1 | define('LoginRegister.View'
2 | , [ 'login_register.tpl'
3 |
4 |   , 'Profile.Model'
5 |   , 'LoginRegister.Login.View'
6 |   , 'LoginRegister.Register.View'
7 |   , 'LoginRegister.CheckoutAsGuest.View'
8 |   , 'Backbone.CompositeView'
9 |   , 'SC.Configuration'
10 |   , 'Header.Simplified.View'
11 |   , 'Footer.Simplified.View'
12 |
13 |   , 'Backbone'
14 |   , 'underscore'
15 |   , 'Utils'
16 | ]
17 | , function (
18 |   login_register_tpl
19 |
20 |   , ProfileModel
21 |   , LoginView
22 |   , RegisterView
23 |   , CheckoutAsGuestView
24 |   , BackboneCompositeView
25 |   , Configuration
26 |   , HeaderSimplifiedView
27 |   , FooterSimplifiedView
28 |
29 |   , Backbone
30 |   , _
31 |   , Utils
32 | )
```

With

```
1 | define('LoginRegister.View'
2 | , [ 'login_register.tpl'
3 |
4 |   , 'GlobalViews.Message.View'
5 |   , 'Profile.Model'
6 |   , 'LoginRegister.Login.View'
7 |   , 'LoginRegister.Register.View'
8 |   , 'LoginRegister.CheckoutAsGuest.View'
9 |   , 'Backbone.CompositeView'
```

```

10 | , 'SC.Configuration'
11 | , 'Header.Simplified.View'
12 | , 'Footer.Simplified.View'
13 |
14 | , 'Backbone'
15 | , 'underscore'
16 | , 'Utils'
17 | ]
18 | , function (
19 |     login_register_tpl
20 |
21 |     , GlobalViewsMessageView
22 |     , ProfileModel
23 |     , LoginView
24 |     , RegisterView
25 |     , CheckoutAsGuestView
26 |     , BackboneCompositeView
27 |     , Configuration
28 |     , HeaderSimplifiedView
29 |     , FooterSimplifiedView
30 |
31 |     , Backbone
32 |     , _
33 |     , Utils
34 | )

```

Replace

```

1 | this.enableCheckoutAsGuest = is_checking_out && profile_model.get('isLoggedIn') === 'F' &&
2 |   (Configuration.getRegistrationType() === 'optional' || Configuration.getRegistrationType() === 'disabled');
3 |
4 | BackboneCompositeView.add(this);

```

With

```

1 | this.enableCheckoutAsGuest = is_checking_out && profile_model.get('isLoggedIn') === 'F' &&
2 |   (Configuration.getRegistrationType() === 'optional' || Configuration.getRegistrationType() === 'disabled');
3 |
4 | if (SC.ENVIRONMENT.email_verification_error)
5 | {
6 |     this.message = _('The validation process has failed. Please login into your account and click on the validation link
7 | again.').translate();
8 |     delete SC.ENVIRONMENT.email_verification_error;
9 | }
10 | else if (SC.ENVIRONMENT.password_reset_invalid_error)
11 | {
12 |     this.message = _('Your reset password link is invalid. Request a new one using the Forgot Password link.').translate();
13 |     delete SC.ENVIRONMENT.password_reset_invalid_error;
14 | }
15 | else if (SC.ENVIRONMENT.password_reset_expired_error)
16 | {
17 |     this.message = _('Your reset password link has expired. Request a new one using the Forgot Password link.').translate();
18 |     delete SC.ENVIRONMENT.password_reset_expired_error;
19 | }
20 | BackboneCompositeView.add(this);

```

Replace

```

1 | , 'Register': function ()
2 | {
3 |     return new RegisterView(this.child_view_options);
4 | }
5 | }

```

With

```

1  {
2    return new RegisterView(this.child_view_options);
3  }
4  , 'Messages': function ()
5  {
6    if (this.message)
7    {
8      return new GlobalViewsMessageView({
9        message: this.message
10       , type: 'error'
11       , closable: true
12     });
13   }
14 }
15 }

```

Modify LoginRegister/Templates/login_register.tpl

Replace

```

1 <header class="login-register-header">
2   {{#if showRegister}}
3   <h1 class="login-register-title"> {{translate 'Log in | Register'}}</h1>
4   {{else}}
5   <h1 class="login-register-title"> {{translate 'Log in'}}</h1>
6   {{/if}}
7 </header>

```

With

```

1 <header class="login-register-header">
2   {{#if showRegister}}
3   <h1 class="login-register-title"> {{translate 'Log in | Register'}}</h1>
4   {{else}}
5   <h1 class="login-register-title"> {{translate 'Log in'}}</h1>
6   {{/if}}
7 </header>
8
9 <div data-view="Messages"></div>

```

Modify MyAccountApplication/SuiteScript/my_account.ssp

Replace

```

1 <%
2   var SiteSettings
3   , siteType
4   , Environment
5   , Language
6   , Currency
7   , Error
8   , Application
9   , environmentParameters
10  , session
11  , parameters
12  , external_payment
13  ;
14
15  try
16  {

```

```

17 SiteSettings = require('SiteSettings.Model').get();
18 parameters = request.getAllParameters();
19 session = require('SC.Models.Init').session;
20
21 // Access control, if you are not logged this will send you to the log in page
22 if (!session.isLoggedIn2() || session.getCustomer().isGuest())
23 {
24     delete parameters.sitepath;
25     parameters.origin = 'customercenter';
26
27     if (parameters.fragment)
28     {
29         parameters.origin_hash = parameters.fragment;
30         delete parameters.fragment;
31     }
32
33     return nlapiSetRedirectURL('EXTERNAL', SiteSettings.touchpoints.login, null, false, parameters);
34 }
35
36 Application = require('Application');
37 Environment = Application.getEnvironment(request);
38 environmentParameters = [];
39 siteType = SiteSettings.siteType;
40
41 Language = Environment.currentLanguage && Environment.currentLanguage.locale || '';
42 Currency = Environment.currencyCodeSpecifiedOnUrl;
43
44 environmentParameters.push('lang=' + Language);
45 environmentParameters.push('cur=' + Currency);
46 environmentParameters.push('X-SC-Touchpoint=myaccount');
47
48 _each(require('ExternalPayment.Model').getParametersFromRequest(request), function(value, key) {
49     environmentParameters.push(key.concat('=', value));
50 });
51 }
52 catch (e)
53 {
54     Error = Application.processError(e);
55 }
56
57 %>

```

With

```

1 <%
2 var SiteSettings
3 , siteType
4 , Environment
5 , Language
6 , Currency
7 , Error
8 , Application
9 , environmentParameters
10 , session
11 , parameters
12 , external_payment
13 , email_change_verification
14 ;
15
16 try
17 {
18     Application = require('Application');
19     Environment = Application.getEnvironment(request);
20     environmentParameters = [];
21     SiteSettings = require('SiteSettings.Model').get();
22     parameters = request.getAllParameters();
23     session = require('SC.Models.Init').session;
24
25     // Access control, if you are not logged this will send you to the log in page
26     if (!session.isLoggedIn2() || session.getCustomer().isGuest())
27     {

```

```

28     delete parameters.sitepath;
29     parameters.origin = 'customercenter';
30
31     if (parameters.fragment)
32     {
33         parameters.origin_hash = parameters.fragment;
34         delete parameters.fragment;
35     }
36
37     return nlapiSetRedirectURL('EXTERNAL', SiteSettings.touchpoints.login, null, false, parameters);
38 }
39 else if (session.isLoggedIn2() && parameters.key)
40 {
41     try
42     {
43         session.verifyEmailChange(parameters.key)
44         email_change_verification = true;
45     }
46     catch (e)
47     {
48         email_change_verification = e.details;
49     }
50 }
51
52 siteType = SiteSettings.sitetype;
53
54 Language = Environment.currentLanguage && Environment.currentLanguage.locale || '';
55 Currency = Environment.currencyCodeSpecifiedOnUrl;
56
57 environmentParameters.push('lang=' + Language);
58 environmentParameters.push('cur=' + Currency);
59 environmentParameters.push('X-SC-Touchpoint=myaccount');
60
61 _each(require('ExternalPayment.Model').getParametersFromRequest(request), function(value, key) {
62     environmentParameters.push(key.concat('=', value));
63 });
64 }
65 catch (e)
66 {
67     Error = Application.processError(e);
68 }
69
70 %>

```

Replace

```

1  <% if (Error) { %>
2  <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3  <% } %>

```

With

```

1  <% if (Error) { %>
2  <script>SC.ENVIRONMENT.contextError = <%= JSON.stringify(Error) %>;</script>
3  <% } %>
4  <% if (email_change_verification) { %>
5  <script>SC.SESSION.email_change_verification = '<%= email_change_verification %>';</script>
6  <% } %>

```

Modify Overview/Javascript/Overview.Home.View.js

Replace

```

1  define('Overview.Home.View'

```

```

2  , [
3      'SC.Configuration'
4      , 'Overview.Banner.View'
5      , 'Overview.Profile.View'
6      , 'Overview.Payment.View'
7      , 'Overview.Shipping.View'
8      , 'Backbone.CollectionView'
9      , 'OrderHistory.List.Tracking.Number.View'
10     , 'RecordViews.View'
11     , 'Handlebars'
12
13     , 'overview_home.tpl'
14
15     , 'Backbone'
16     , 'Backbone.CompositeView'
17     , 'underscore'
18     , 'Utils'
19 ]
20 , function(
21     Configuration
22     , OverviewBannerView
23     , OverviewProfileView
24     , OverviewPaymentView
25     , OverviewShippingView
26     , BackboneCollectionView
27     , OrderHistoryListTrackingNumberView
28     , RecordViewsView
29     , Handlebars
30
31     , overview_home_tpl
32
33     , Backbone
34     , BackboneCompositeView
35     , -
36 )

```

With

```

1  define('Overview.Home.View'
2  , [
3      'SC.Configuration'
4      , 'GlobalViews.Message.View'
5      , 'Overview.Banner.View'
6      , 'Overview.Profile.View'
7      , 'Overview.Payment.View'
8      , 'Overview.Shipping.View'
9      , 'Backbone.CollectionView'
10     , 'OrderHistory.List.Tracking.Number.View'
11     , 'RecordViews.View'
12     , 'Handlebars'
13
14     , 'overview_home.tpl'
15
16     , 'Backbone'
17     , 'Backbone.CompositeView'
18     , 'underscore'
19     , 'Utils'
20 ]
21 , function(
22     Configuration
23     , GlobalViewsMessageView
24     , OverviewBannerView
25     , OverviewProfileView
26     , OverviewPaymentView
27     , OverviewShippingView
28     , BackboneCollectionView
29     , OrderHistoryListTrackingNumberView
30     , RecordViewsView
31     , Handlebars
32
33     , overview_home_tpl

```

```

34 |
35 |     , Backbone
36 |     , BackboneCompositeView
37 |     , -
38 | )

```

Replace

```

1 | this.creditcards.on('reset destroy change add', this.showContent, this);
2 | }

```

With

```

1 | this.creditcards.on('reset destroy change add', this.showContent, this);
2 |
3 | if (SC.SESSION.email_change_verification)
4 | {
5 |     this.email_change_verification = SC.SESSION.email_change_verification;
6 |     delete SC.SESSION.email_change_verification;
7 | }
8 | }

```

Replace

```

1 | , 'Overview.Shipping': function()
2 | {
3 |     return new OverviewShippingView({ model: this.defaultShippingAddress });
4 | }

```

With

```

1 | , 'Overview.Shipping': function()
2 | {
3 |     return new OverviewShippingView({ model: this.defaultShippingAddress });
4 | }
5 |
6 | , 'Overview.Messages': function ()
7 | {
8 |     if (this.email_change_verification)
9 |     {
10 |         return new GlobalViewsMessageView({
11 |             message: this.email_change_verification === 'true' ? _('Your email has been changed successfully to <strong>').trans
late() + this.model.get('email') + '</strong>' : this.email_change_verification
12 |             , type: this.email_change_verification === 'true' ? 'success' : 'error'
13 |             , closable: true
14 |         });
15 |     }
16 | }

```

Modify Overview/Templates/overview_home.tpl

Replace

```

1 | <section class="overview-home">
2 |     <div class="overview-home-orders" data-permissions="{purchasesPermissions}">

```

With

```

1 <section class="overview-home">
2   <div data-view="Overview.Messages"></div>
3   <div class="overview-home-orders" data-permissions="{purchasesPermissions}">

```

Add Profile/Javascript/Profile.ChangeEmailAddress.Model.js

```

1 // Profile.ChangeEmailAddress.Model.js
2 // =====
3 // View Model for changing user's email
4 // @module Profile
5 define(
6   'Profile.ChangeEmailAddress.Model'
7   , [
8     'Backbone'
9     , 'underscore'
10    , 'Utils'
11   ]
12   , function (
13     Backbone
14     , _
15   ) {
16     {
17       'use strict';
18
19       // @class Profile.ChangeEmailAddress.Model @extends Backbone.Model
20       return Backbone.Model.extend(
21         {
22           urlRoot: 'services/Profile.Service.ss'
23         , validation: {
24             current_password: { required: true, msg: _('Current password is required').translate() }
25             , confirm_email: [
26               { required: true, msg: _('Confirm Email is required').translate() }
27               , { equalTo: 'new_email', msg: _('New Email and Confirm New Email do not match').translate() }
28             ]
29             , new_email: { required: true, msg: _('New Email is required').translate() }
30           }
31         });
32     });

```

Add Profile/Javascript/Profile.ChangeEmailAddress.View.js

```

1 // @module Profile
2 define(
3   'Profile.ChangeEmailAddress.View'
4   , [
5     'GlobalViews.Message.View'
6     , 'Backbone.FormView'
7     , 'SC.Configuration'
8
9     , 'profile_change_email.tpl'
10
11     , 'Backbone'
12     , 'underscore'
13     , 'Utils'
14   ]
15   , function (
16     GlobalViewsMessageView
17     , BackboneFormView
18     , Configuration
19
20     , profile_change_email_tpl
21
22     , Backbone
23     , _
24   ) {

```



```

25 {
26   'use strict';
27
28   // @class Profile.ChangeEmailAddress.View @extends Backbone.View
29   return Backbone.View.extend({
30
31     template: profile_change_email_tpl
32
33     , page_header: _('Change Email').translate()
34
35     , title: _('Change Email').translate()
36
37     , events: {
38       'submit form': 'saveFormCustom'
39     }
40
41     , bindings: {
42       '[name="current_password"]': 'current_password'
43       , '[name="new_email"]': 'new_email'
44       , '[name="confirm_email"]': 'confirm_email'
45     }
46
47     , initialize: function()
48     {
49       Backbone.View.prototype.initialize.apply(this, arguments);
50       BackboneFormView.add(this);
51     }
52
53     , saveFormCustom: function ()
54     {
55       this.new_email = this.$('[name="new_email"]').val();
56       BackboneFormView.saveForm.apply(this, arguments);
57     }
58
59     , showSuccess: function (placeholder)
60     {
61       var global_view_message = new GlobalViewsMessageView({
62         message: _('A confirmation email has been sent to <strong>').translate() + this.new_email + '</strong>'
63         , type: 'success'
64         , closable: true
65       });
66
67       placeholder.html(global_view_message.render().$el.html());
68     }
69   });
70 });

```

Modify Profile/Javascript/Profile.Information.View.js

Replace

```

1  define(
2    'Profile.Information.View'
3  , [
4    'SC.Configuration'
5    , 'GlobalViews.Message.View'
6    , 'Backbone.FormView'
7
8    , 'profile_information.tpl'
9
10   , 'Backbone'
11   , 'underscore'
12   , 'jQuery'
13   , 'Utils'
14 ]
15 , function (
16   Configuration
17   , GlobalViewsMessageView
18   , BackboneFormView

```

```

19
20 , profile_information_tpl
21
22 , Backbone
23 , _
24 , jQuery
25 )
26 {
27   'use strict';
28
29   // @class Profile.Information.View @extends Backbone.View
30   return Backbone.View.extend({
31
32     template: profile_information_tpl
33     , page_header: _('Profile Information').translate()
34     , title: _('Profile Information').translate()
35     , attributes: {'class': 'ProfileInformationView'}
36     , events: {
37       'submit form': 'saveForm'
38       , 'change input[data-type="phone"]': 'formatPhone'
39     }
40
41     , bindings: {
42       '[name="firstname"]': 'firstname'
43       , '[name="lastname"]': 'lastname'
44       , '[name="companyname"]': 'companyname'
45       , '[name="phone"]': 'phone'
46     }
47
48     , initialize: function()
49     {
50       BackboneFormView.add(this);
51     }
52
53     , formatPhone: function (e)
54     {
55       var $target = jQuery(e.target);
56       $target.val(_($target.val()).formatPhone());
57     }
58
59     , showSuccess: function ()

```

With

```

1 define(
2   'Profile.Information.View'
3   , [
4     'SC.Configuration'
5     , 'GlobalViews.Message.View'
6     , 'Backbone.FormView'
7
8     , 'Profile.ChangeEmailAddress.Model'
9     , 'Profile.ChangeEmailAddress.View'
10
11     , 'profile_information_tpl'
12
13     , 'Backbone'
14     , 'underscore'
15     , 'jQuery'
16     , 'Utils'
17   ]
18   , function (
19     Configuration
20     , GlobalViewsMessageView
21     , BackboneFormView
22
23     , ProfileChangeEmailModel
24     , ProfileChangeEmailView
25
26     , profile_information_tpl
27

```

```

28 | , Backbone
29 | , _
30 | , jQuery
31 | )
32 | {
33 |   'use strict';
34 |
35 |   // @class Profile.Information.View @extends Backbone.View
36 |   return Backbone.View.extend({
37 |
38 |     template: profile_information_tpl
39 |     , page_header: _('Profile Information').translate()
40 |     , title: _('Profile Information').translate()
41 |     , attributes: {'class': 'ProfileInformationView'}
42 |     , events: {
43 |       'submit form': 'saveForm'
44 |       , 'change input[data-type="phone]': 'formatPhone'
45 |       , 'click [data-action="change-email]': 'changeEmail'
46 |     }
47 |
48 |     , bindings: {
49 |       '[name="firstname"]': 'firstname'
50 |       , '[name="lastname"]': 'lastname'
51 |       , '[name="companyname"]': 'companyname'
52 |       , '[name="phone"]': 'phone'
53 |     }
54 |
55 |     , initialize: function(options)
56 |     {
57 |       BackboneFormView.add(this);
58 |       this.application = options.application;
59 |     }
60 |
61 |     , formatPhone: function (e)
62 |     {
63 |       var $target = jQuery(e.target);
64 |       $target.val(_($target.val()).formatPhone());
65 |     }
66 |
67 |     , changeEmail: function ()
68 |     {
69 |       var model = new ProfileChangeEmailModel(this.model.attributes);
70 |
71 |       var view = new ProfileChangeEmailView({
72 |         application: this.application
73 |         , model: model
74 |       });
75 |
76 |       var self = this;
77 |
78 |       model.on('save', function () {
79 |         view.showSuccess(self.$('[data-type="alert-placeholder"]'));
80 |       });
81 |
82 |       view.useLayoutError = true;
83 |
84 |       this.application.getLayout().showInModal(view);
85 |     }
86 |
87 |     , showSuccess: function ()

```

Add Profile/Sass/_profile-change-email.scss

```

1 | .profile-change-email-button-back {
2 |   @extend .button-back;
3 | }
4 |
5 | .profile-change-email-button-back-icon {
6 |   @extend .button-back-icon;
7 | }
8 |

```

```

9  .profile-change-email-form-label {
10     display: inline-block;
11 }
12
13 .profile-change-email-form-group-label-required {
14     @extend .input-required;
15 }
16
17 .profile-change-email {
18     @extend .address-edit;
19 }
20
21 .profile-change-email-form-title {}
22 .profile-change-email-form-area {}
23
24 .profile-change-email-form {
25     margin-top: $sc-base-margin * 3;
26 }
27
28 .profile-change-email-form-group,
29 .profile-change-email-form-actions {
30     @extend .control-group;
31 }
32
33 .profile-change-email-form-group-label {
34     @extend .input-label
35 }
36
37 .profile-change-email-form-group-label-required {
38 }
39
40 .profile-change-email-form-group-input {
41     @extend .input-large
42 }
43
44 .profile-change-email-form-actions-change {
45     @extend .button-primary;
46     @extend .button-medium;
47 }
48
49 .profile-change-email-group-form-controls{}
50
51 .profile-change-email-form-info-block{}

```

Modify Profile/SuiteScript/Profile.Model.js

Replace

```

1  if (data.email && data.email !== this.currentSettings.email && data.email === data.confirm_email)
2  {
3      if(data.isGuest === 'T')
4      {
5          customerUpdate.email = data.email;
6      }
7      else
8      {
9          login.changeEmail(data.current_password, data.email, true);
10     }
11 }

```

With

```

1  if (data.email && data.email !== this.currentSettings.email && data.email === data.confirm_email && data.isGuest === 'T')
2  {
3      customerUpdate.email = data.email;
4  }
5  else if (data.new_email && data.new_email === data.confirm_email && data.new_email !== this.currentSettings.email)

```

```

6 {
7   ModelsInit.session.login({
8     email: data.email
9     , password: data.current_password
10  });
11  login.changeEmail(data.current_password, data.new_email, true);
12 }

```

Add Profile/Templates/profile_change_email.tpl

```

1 <section class="profile-change-email">
2   <div data-type="alert-placeholder"></div>
3   <div class="profile-change-email-form-area">
4     <form class="profile-change-email-form">
5       <fieldset>
6         <small class="profile-change-email-form-label">{{translate 'Required'}} <span class="profile-change-email-form-group-label-
required">*</span></small>
7
8         <div class="profile-change-email-form-group" data-input="new_email" data-validation="control-group">
9           <label class="profile-change-email-form-group-label" for="new_email">{{translate 'New Email'}} <span class="pro
file-change-email-form-group-label-required">*</span></label>
10          <div class="profile-change-email-group-form-controls" data-validation="control">
11            <input type="email" class="profile-change-email-form-group-input" id="new_email" name="new_email" value="" placehold
er="{{translate 'your@example.com'}}">
12          </div>
13        </div>
14
15        <div class="profile-change-email-form-group" data-input="confirm_email" data-validation="control-group">
16          <label class="profile-change-email-form-group-label" for="confirm_email">{{translate 'Confirm New Email'}} <span
class="profile-change-email-form-group-label-required">*</span></label>
17          <div class="profile-change-email-group-form-controls" data-validation="control">
18            <input type="email" class="profile-change-email-form-group-input" id="confirm_email" name="confirm_email" val
ue="" placeholder="{{translate 'your@example.com'}}">
19          </div>
20        </div>
21
22        <div class="profile-change-email-form-group" data-input="current_email" data-validation="control-group">
23          <label class="profile-change-email-form-group-label" for="current_password">{{translate 'Password'}} <span class="pro
file-change-email-form-group-label-required">*</span></label>
24          <div class="profile-change-email-group-form-controls" data-validation="control">
25            <input type="password" class="profile-change-email-form-group-input" id="current_password" name="current_password" val
ue="">
26          </div>
27        </div>
28      </fieldset>
29      <p class="profile-change-email-form-info-block"><small> {{translate 'You will still be able to login with your current email
address and password until your new email address is verified.'}} </small></p>
30      <div class="profile-change-email-form-actions">
31        <button type="submit" class="profile-change-email-form-actions-change">{{translate 'Send Verification Email'}}</button>
32      </div>
33    </form>
34  </div>
35 </section>

```

Modify Profile/Templates/profile_information.tpl

Replace

```

1 <p class="profile-information-input-email" id="email">{{email}}</p>

```

With

```

1 <p class="profile-information-input-email" id="email">{{email}} | <a class="profile-information-change-email-address" data-
action="change-email">{{translate 'Change Address'}}</a></p>

```

Duplicate Product Lists in Internet Explorer 11

Applies to: SuiteCommerce Advanced | pre-Denali

In pre-Denali releases of SuiteCommerce Advanced, product lists in My Account are displayed in duplicate when using Internet Explorer 11. The duplication is a result of the caching behavior in IE 11 where product lists are cached by default. To correct this issue, modify the ProductList.js file as described here to force a new request to NetSuite and set the cache to false for product lists.

To modify the caching behavior defined in the ProductList.js file:

1. Copy the ProductList.js file at Reference Shopflow > js > src > app > modules > ProductList to the same location in the custom folder.
2. Modify the functions getProductListsPromise and getProductList with {cache: false} as shown in the following code snippet.

```

1 application.getProductListsPromise = function ()
2 {
3   if (!application.productListsInstancePromise)
4   {
5     application.productListsInstancePromise = jQuery.Deferred();
6     application.productListsInstance = new ProductListCollection();
7     application.productListsInstance.application = application;
8
9     // MODIFIED CODE FOR IE 11 CACHING
10    application.productListsInstance.fetch({cache: false}).done(function(jsonCollection)
11    {
12      application.productListsInstance.set(jsonCollection);
13      application.productListsInstancePromise.resolve(application.productListsInstance);
14    });
15  }
16
17  return application.productListsInstancePromise;
18 };
19
20 // obtain a single ProductList with all its item's data
21 application.getProductList = function (id)
22 {
23   var productList = new ProductListModel();
24   productList.set('internalid', id);
25
26   // MODIFIED CODE FOR IE 11 CACHING
27   return productList.fetch({cache: false});
28 };

```

3. Save your changes.

Save for Later Item not Moved to Cart

Applies to: SuiteCommerce Advanced | Mont Blanc

In Mont Blanc releases of SuiteCommerce Advanced, when users set an item as Save for Later and then return to move that item to the cart an error is returned. To correct this error, apply the patch described here to extend the addItemToCartHandler() method in the ProductList.DetailsLater.View.js file.

Note: Before proceeding, familiarize yourself with the [Best Practices for Customizing SCA](#).

You can download the code samples described in this procedure here: [Product.List.Extension@1.0.0.zip](#)

Step 1: Extend the ProductList.DetailsLater.View.js File

1. To extend the ProductList.DetailsLater.View.js file, which is located in the **ProductList** module, create a directory to store your custom module.
2. Open this directory and create a subdirectory to maintain your customizations.
Give this directory a name similar to the module being customized. For example:
Modules/extensions/Product.List.Extension@1.0.0
3. In your new Product.List.Extension@1.0.0 module, create a subdirectory called JavaScript.
Modules/extensions/Product.List.Extension@1.0.0/JavaScript
4. In your new JavaScript subdirectory, create a JavaScript file to extend ProductList.DetailsLater.View.js .
Name this file according to best practices. For example:
ProductList.DetailsLater.View.Extension.js
5. Open this file and extend the addItemToCartHandler() method as shown in the following code snippet.

```

1  define('ProductList.DetailsLater.View.Extension'
2  , [
3    'ProductList.DetailsLater.View'
4    , 'underscore'
5  ]
6  , function (
7    ProductListDetailView
8    , -
9  )
10 {
11   'use strict';
12   _.extend(ProductListDetailView.prototype,
13   {
14     addItemToCartHandler : function (e)
15     {
16       e.stopPropagation();
17       e.preventDefault();
18       if (this.application.getConfig('addToCartBehavior') === 'showCartConfirmationModal')
19       {
20         this.cart.optimistic = null;
21       }
22       var self = this
23       , selected_product_list_item_id = self.$(e.target).closest('article').data('id')
24       , selected_product_list_item = self.model.get('items').findWhere({
25         internalid: selected_product_list_item_id.toString()
26       })
27       , selected_item = selected_product_list_item.get('item')
28       , selected_item_internalid = selected_item.internalid
29       , item_detail = selected_product_list_item.getItemForCart(selected_item_internalid, selected_product_list_item.get('quantity'), selected_item.itemoptions_detail, selected_product_list_item.getOptionsArray())
30       , add_to_cart_promise = this.addItemToCart(item_detail)
31       , whole_promise = jQuery.when(add_to_cart_promise, this.deleteListItem(selected_product_list_item)).then(jQuery.proxy(this, 'executeAddToCartCallback'));
32
33       if (whole_promise)
34       {
35         this.disableElementsOnPromise(whole_promise, 'article[data-item-id="' + selected_item_internalid + '" ] a, article[data-item-id="' + selected_item_internalid + '" ] button');
36       }
37     }
38   }
39   );
40   });
41   });
42 }
43 );

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Extension

1. Open the Product.List.Extension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/Product.List.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```

1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   }
7 | }

```

4. Save the ns.package.json file.
5. Open the distro.json file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the modules object.
This ensures that the Gulp tasks include your extension when you deploy. In this example, the extensions/ProductList.DetailsLater.View.Extension module is added at the beginning of the list of modules. However, you can add the module anywhere in the modules object. The order of precedence in this list does not matter.

```

1 | {
2 |   "name": "SuiteCommerce Advanced Elbrus",
3 |   "version": "2.0",
4 |   "buildToolsVersion": "1.3.0",
5 |   "folders": {
6 |     "modules": "Modules",
7 |     "suitecommerceModules": "Modules/suitecommerce",
8 |     "extensionsModules": "Modules/extensions",
9 |     "thirdPartyModules": "Modules/third_parties",
10 |    "distribution": "LocalDistribution",
11 |    "deploy": "DeployDistribution"
12 |  },
13 |   "modules": {
14 |     "extensions/ProductList.DetailsLater.View.Extension": "1.0.0",
15 |     "suitecommerce/Account": "2.3.0",
16 |     "suitecommerce/Address": "2.4.0",
17 |     ...

```

7. Include the module definition ("ProductList.DetailsLater.View.Extension") in the dependencies array of the Checkout application JavaScript object.

Your distro.json file should look similar to the following:

```

1 | "tasksConfig": {
2 |   //...
3 |   "javascript": [
4 |     //...
5 |     {
6 |       "entryPoint": "SC.Checkout.Starter",
7 |       "exportFile": "checkout.js",
8 |       "dependencies": [
9 |         //...
10 |        "CMSAdaptor",
11 |        "Sensors",
12 |        "ProductList.DetailsLater.View.Extension"
13 |      ],
14 |     //...

```


Note: Best practice is to place any new modules at the bottom of the list in the dependencies array.

8. Save the distro.json file.

Step 3: Test and Deploy Your Extension

1. Deploy your customizations to your NetSuite account and test. See the help topic [Deploy SCA Customizations to NetSuite](#).
2. Verify your changes.
An error should not be returned when items set as Save for Later are later moved to the cart.

Running Gulp Commands Results in a Syntax Error

Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson

This section applies to the **Vinson** release of SuiteCommerce Advanced and earlier.

Running any gulp commands from a command line or terminal can result in the following error:

SyntaxError: Invalid flags supplied to RegExp constructor 'u' at new RegExp (native)

To correct this error, edit the package.json file and reinstall Node.js dependencies for your top-level source files directory.

Note: This procedure requires editing the package.json code directly. You cannot override or extend this file.

You can download the code samples described in this procedure here: [package.json.zip](#).

Step 1: Edit the package.json File

1. To add a new dependency to the package.json file located in your top-level source code directory, navigate to the top-level directory containing your SuiteCommerce Advanced source files.
2. Open the package.json file.
3. Add the following dependency to the dependencies object:

```
1 | "xmlbuilder": "8.2.2"
```

Your edited code should look similar to the following:

```
1 | {
2 |     "name": "suitecommerce-builder",
3 |     "version": "0.0.1",
4 |     "description": "Sets of tasks to build a Reference Implementation",
5 |     "main": "gulpfile.js",
6 |     "private": true,
7 |     "scripts": {
8 |         "test": "echo \"Error: no test specified\" && exit 1"
9 |     },
10 |     "author": "",
11 |     "license": "ISC",
12 |     "dependencies": {
13 |         //...
14 |         "suitetalk": "file:../ns_npm_repository/suitetalk4node",
15 |         "preconditions": "file:../ns_npm_repository/preconditions",
16 |         "xmlbuilder": "8.2.2"
```

```

17 |         },
18 |         //...

```

4. Save the file.

Step 2: Reinstall Dependencies and Test Your Changes

1. To reinstall the Node.js dependencies required by the developer tools In your top-level source files directory, first delete the **node_modules/** subdirectory.

Deleting this subdirectory removes the dependencies and other files that were created when you initially set up your developer environment. See [Install Additional Files in the Source Directory](#) for more information on this process.

2. Open a command line or terminal.
3. Navigate to the top-level directory containing your SuiteCommerce Advanced source files.
4. Run the following command to reinstall the dependencies and include your edits:

```
npm install
```

5. Open a command line or terminal and navigate to your top-level source files directory.
6. Run Gulp.js using the following command:

```
gulp
```

This and any other gulp commands should run with no errors.

Missing Promo Code on Return Request

i Applies to: SuiteCommerce Advanced | Mont Blanc

This section applies to the **Mont Blanc** release of SuiteCommerce Advanced.

In Mont Blanc releases of SuiteCommerce Advanced, promo codes applied to the original sales order are not included in the calculations for a return request. To correct this error, apply the patch described here to extend the `setLines()` method in the `ReturnAuthorization.Model.js` file.

i Note: Before proceeding, familiarize yourself with the [Best Practices for Customizing SCA](#).

You can download the code samples described in this procedure here:

[Return.Authorization.Extension@1.0.0.zip](#)

Step 1: Extend the ReturnAuthorization.Model.js File

1. To extend the `ReturnAuthorization.Model.js` file, which is located in the **ReturnAuthorization** module, create a directory to store your custom module.
2. Open this directory and create a subdirectory to maintain your customizations.
Give this directory a name similar to the module being customized. For example:
`Modules/extensions/Return.Authorization.Extension@1.0.0`
3. In your new `Return.Authorization.Extension@1.0.0` module, create a subdirectory called `SuiteScript`.
`Modules/extensions/Return.Authorization.Extension@1.0.0/SuiteScript`
4. In your new `SuiteScript` subdirectory, create a JavaScript file to extend `ReturnAuthorization.Model.js`.

Name this file according to best practices. For example:

```
ReturnAuthorization.Model.Extension.js
```

5. Open this file and extend the `setLines()` method as shown in the following code snippet.

```

1  define('ReturnAuthorization.Model.Extension'
2  , [
3    'ReturnAuthorization.Model'
4    , 'underscore'
5  ]
6  , function (
7    ReturnAuthorizationModel
8    , -
9  )
10 {
11   'use strict';
12   _.extend(ReturnAuthorizationModel.prototype,
13   {
14
15     setLines: function (return_authorization, lines, transaction_lines)
16     {
17       var line_count = return_authorization.getLineItemCount('item')
18       ,   add_line = true
19       ,   i = 1;
20
21       while (i <= line_count)
22       {
23         var line_item_value = return_authorization.getLineItemValue('item', 'id', i);
24
25         add_line = this.findLine(line_item_value, lines);
26
27         if (add_line)
28         {
29           var transaction_line = _.findWhere(transaction_lines, { line: line_item_value });
30
31           if (transaction_line)
32           {
33             return_authorization.setLineItemValue('item', 'rate', i, transaction_line.rate);
34           }
35
36           return_authorization.setLineItemValue('item', 'quantity', i, add_line.quantity);
37           return_authorization.setLineItemValue('item', 'description', i, add_line.reason);
38         }
39         else
40         {
41           return_authorization.setLineItemValue('item', 'quantity', i, 0);
42         }
43
44         i++;
45       }
46     }
47   });
48 }
49 );

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Extension

1. Open the `Return.Authorization.Extension@1.0.0` module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/Return.Authorization.Extension@1.0.0/ns.package.json
3. Build the `ns.package.json` file using the following code

```

1  {
2    "gulp": {
3      "ssp-libraries": [
4        "SuiteScript/*.js"
5      ]
6    }
7  }

```

```

6 |   }
7 | }

```

4. Save the ns.package.json file.

5. Open the distro.json file.

This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the modules object.

This ensures that the Gulp tasks include your extension when you deploy. In this example, the extensions/Return.Authorization.Extension module is added at the beginning of the list of modules. However, you can add the module anywhere in the modules object. The order of precedence in this list does not matter.

```

1 | {
2 |   "name": "SuiteCommerce Advanced Elbrus",
3 |   "version": "2.0",
4 |   "buildToolsVersion": "1.3.0",
5 |   "folders": {
6 |     "modules": "Modules",
7 |     "suitecommerceModules": "Modules/suitecommerce",
8 |     "extensionsModules": "Modules/extensions",
9 |     "thirdPartyModules": "Modules/third_parties",
10 |    "distribution": "LocalDistribution",
11 |    "deploy": "DeployDistribution"
12 |  },
13 |   "modules": {
14 |     "extensions/Return.Authorization.Extension": "1.0.0",
15 |     "suitecommerce/Account": "2.3.0",
16 |     "suitecommerce/Address": "2.4.0",
17 |     ...

```

7. Include the module definition ("ReturnAuthorization.Model.Extension") in the dependencies array of the SCA application of the ssp-libraries object.

Your distro.json file should look similar to the following:

```

1 | "tasksConfig": {
2 |   //...
3 |   "ssp-libraries": {
4 |     "entryPoint": "SCA",
5 |     "dependencies": [
6 |       "Application",
7 |       "Account.Model",
8 |       "Address.Model",
9 |       //..
10 |      "ReturnAuthorization.Model.Extension"
11 |     ],

```



Note: Best practice is to place any new modules at the bottom of the list in the dependencies array.

8. Save the distro.json file.

Step 3: Test and Deploy Your Extension

1. Deploy your customizations to your NetSuite account and test. See the help topic [Deploy SCA Customizations to NetSuite](#).



Note: Since this patch modifies an SSP library file, changes are not visible in your local environment until you first deploy the customizations to NetSuite.

2. Verify your changes.

A returned order should include the original promo code that was used when the customer placed the order. The value on the sales order should match the value on the Return Authorization.

Enhanced Page Content Disappears when Resizing the Browser

Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus

This section applies to the **Denali** release of SuiteCommerce Advanced and later.

Users might experience cases where enhanced pages do not maintain content when dynamically resizing the browser window. A typical scenario might include a user shrinking the browser window to such a point where the enhanced content disappears from view. Enlarging the browser window should cause any enhanced content to reinsert and appear. However, users might experience some cases where the enhanced content does not reappear in the browser.

To prevent this from happening, extend the `ApplicationSkeleton.Layout.js` and `Content.js` files as described in the steps below.

For more information on Enhanced Pages, see the help topic [Pages](#).

Note: Before proceeding, familiarize yourself with the [Best Practices for Customizing SCA](#).

You can download the code samples described in this procedure here:

[ApplicationSkeletonExtension@1.0.0.zip](#)

Step 1: Extend the `ApplicationSkeleton.Layout.js` File

1. To extend the `ApplicationSkeleton.Layout.js` file, which is located in the **ApplicationSkeleton** module, create a directory to store your custom module. For example:
Modules/extensions
2. Open this directory and create a subdirectory to maintain your customizations.
Give this directory a name similar to the module being customized. For example:
Modules/extensions/ApplicationSkeletonExtension@1.0.0
3. In your new `ApplicationSkeletonExtension@1.0.0` module, create a subdirectory called `JavaScript`.
Modules/extensions/ApplicationSkeletonExtension@1.0.0/JavaScript
4. In your new `JavaScript` subdirectory, create a JavaScript file to extend `ApplicationSkeleton.Layout.js`.
Name this file according to best practices. For example:
`ApplicationSkeleton.Layout.Extension.js`
5. Open this file and extend the `initialize()` method of the original module to include the following line:

```
1 | self.trigger('resize', self.currentView);
```

Assuming that you have followed best practices outlined in this procedure, your extension should look like this:

```
1 | define(
2 |   'ApplicationSkeleton.Layout.Extension'
3 | , [
```

```

4      'ApplicationSkeleton.Layout'
5      , 'Header.View'
6      , 'Footer.View'
7      , 'Backbone.CompositeView'
8      , 'underscore'
9      , 'jQuery'
10     ]
11     , function (
12         ApplicationSkeletonLayout
13         , HeaderView
14         , FooterView
15         , BackboneCompositeView
16         , _
17         , jQuery
18     )
19     {
20         'use strict';
21
22         _.extend(ApplicationSkeletonLayout.prototype,
23         {
24             initialize: function (Application)
25             {
26                 BackboneCompositeView.add(this);
27
28                 this.headerView = this.originalHeaderView = HeaderView;
29                 this.footerView = this.originalFooterView = FooterView;
30
31                 this.application = Application;
32                 this.windowWidth = jQuery(window).width();
33
34                 // @property {jQuery.Deferred} afterAppendViewPromise a promise that is resolve only if one view was shown
35                 in this layout
36                 this.afterAppendViewPromise = jQuery.Deferred();
37
38                 var self = this;
39
40                 this.once('afterAppendView', function ()
41                 {
42                     self.afterAppendViewPromise.resolve();
43                 });
44
45                 jQuery(window).on('resize', _.throttle(function ()
46                 {
47                     if (_.getDeviceType(self.windowWidth) === _.getDeviceType(jQuery(window).width()))
48                     {
49                         return;
50                     }
51
52                     _.resetViewportWidth();
53
54                     self.updateHeader();
55                     self.updateFooter();
56
57                     self.trigger('resize', self.currentView);
58
59                     self.updateLayoutSB && self.updateLayoutSB();
60
61                     self.windowWidth = jQuery(window).width();
62
63                     }, 1000));
64                 }
65             });
66     });

```

6. Save the file.

Step 2: Extend the Content.js File

1. In this step, you will extend the Content.js file, which is located in the **Content** module. The code required for your Content.js extension is the same for all versions of SCA. However, the original

Content.js file differs slightly for each version of SCA. This procedure provides code examples for each version of SCA.

Download the appropriate code sample according to the implementation of SCA that you are customizing:

SCA Release	Download
Elbrus	ContentExtension@1.0.0---ElbrusSample.zip
Vinson	ContentExtension@1.0.0---VinsonSample.zip
Mont Blanc	ContentExtension@1.0.0---MontBlancSample.zip
Denali	ContentExtension@1.0.0---DenaliSample.zip

- Open your custom extensions directory and create a subdirectory to maintain your customizations. Give this directory a name similar to the module being customized. For example:
Modules/extensions/ContentExtension@1.0.0
- In your new ContentExtension@1.0.0 module, create a subdirectory called JavaScript.
Modules/extensions/ContentExtension@1.0.0/JavaScript
- In your new JavaScript subdirectory, create a JavaScript file to extend Content.js. Name this file according to best practices. For example:
Content.Extension.js
- Open this file and extend the `mountToApp()` method.
 - In your new `mountToApp()` method, search for the following line:

```
1 | Layout.showInModal = _wrap(Layout.showInModal, show_content_wrapper);
```

- Below this line, add the following:

```
1 | Layout.on('resize', function(view)
2 | {
3 |     show_content_wrapper(function() { return jQuery.Deferred().resolve(); }, view);
4 | });
```

- Save the file.

Step 3: Prepare the Developer Tools for Your Customizations

- Open the ApplicationSkeletonExtension@1.0.0 module.
- Create a file in this module and name it **ns.package.json**.
Modules/extensions/ApplicationSkeletonExtension@1.0.0/ns.package.json
- Build the ns.package.json file using the following code

```
1 | {
2 |     "gulp": {
3 |         "javascript": [
4 |             "JavaScript/*.js"
5 |         ]
6 |     }
7 | }
```

- Save the ns.package.json file.
- Repeat steps 2–4 for the ContentExtension@1.0.0 module. The two ns.package.js files should be identical.

6. Open the `distro.json` file.

This file is located in the top-level directory of your SuiteCommerce Advanced source code.

7. Add your custom module to the `modules` object to ensure that the Gulp tasks include your extension when you deploy.

In this example, the modules are added at the beginning of the list of modules. However, you can add the module anywhere in the `modules` object. The order of precedence in this list does not matter.



Note: The following example depicts the `Distro.json` file for Elbrus release. However, your customization of the `modules` object should look similar to what is depicted below.

```

1 {
2   "name": "SuiteCommerce Advanced Elbrus",
3   "version": "2.0",
4   "buildToolsVersion": "1.3.0",
5   "folders": {
6     "modules": "Modules",
7     "suitecommerceModules": "Modules/suitecommerce",
8     "extensionsModules": "Modules/extensions",
9     "thirdPartyModules": "Modules/third_parties",
10    "distribution": "LocalDistribution",
11    "deploy": "DeployDistribution"
12  },
13  "modules": {
14    "extensions/ApplicationSkeletonExtension": "1.0.0",
15    "extensions/ContentExtension": "1.0.0",
16    "suitecommerce/Account": "2.2.0",
17    "suitecommerce/Address": "2.3.0",
18    ...

```

8. Include the file definition ("Content.Extension") in the dependencies array of the Shopping, Checkout, and MyAccount applications of the JavaScript object.

Your `distro.json` file should look similar to the following:

```

1  "tasksConfig": {
2    //...
3    "javascript": [
4      //...
5      {
6        "entryPoint": "SC.Shopping.Starter",
7        "exportFile": "shopping.js",
8        "dependencies": [
9          //...
10         "Newsletter",
11         "ProductDetailToQuote",
12         "Content.Extension"
13       ],
14       //...
15       {
16         "entryPoint": "SC.MyAccount.Starter",
17         "exportFile": "myaccount.js",
18         "dependencies": [
19           //...
20           "Location.Model",
21           "StoreLocator.Model",
22           "Content.Extension"
23         ],
24       //...
25       {
26         "entryPoint": "SC.Checkout.Starter",
27         "exportFile": "checkout.js",
28         "dependencies": [
29           //...
30           "StoreLocatorAccessPoints",
31           "StoreLocator",

```



```

32         "Content.Extension"
33     ],
34     //...

```

9. Save the distro.json file.

Step 4: Test and Deploy Your Extension

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

After a successful deployment, dynamically resizing a browser window on a page with enhanced content results in no errors. The enhanced content appears on the page after enlarging the window to its original size.

Invoices Page Displays Incorrect Date Sort (pre-Denali)

Applies to: SuiteCommerce Advanced | pre-Denali

This section applies to the **Reference My Account v1.05** bundle.

In Reference My Account v1.05, when shoppers sort invoices **By Due Date** or **By Invoice Date** on the Invoices page of the web store, the invoices display out of order. To correct the sort order, customize the `list()` method in the `Models.js` file as described in this section.

To customize the `Models.js` file:

1. If you have not done so already, copy the `Models.js` file from the reference folder to your custom folder in the NetSuite File Cabinet.
`Models.js` is located in the File Cabinet at `My Account 1.05 > Reference My Account > ssp_libraries`.
2. Update the `list()` method in the `Models.js` file from your custom folder to use the `nlapiStringToDate()` method.

Find the following code:

```

1      return _.map(results || [], function (record)
2      {
3
4          var due_date = record.getValue('duedate')
5          ,   close_date = record.getValue('closedate')
6          ,   tran_date = record.getValue('trandate')
7          ,   due_in_milliseconds = new Date(due_date).getTime() - now
8          ,   total = toCurrency(record.getValue(amount_field))
9          ,   total_formatted = formatCurrency(record.getValue(amount_field));
10
11      return {
12          internalid: record.getId()
13          ,   tranid: record.getValue('tranid')
14          ,   order_number: record.getValue('tranid') // Legacy attribute
15          ,   date: tran_date // Legacy attribute
16          ,   summary: { // Legacy attribute
17              total: total
18              ,   total_formatted: total_formatted
19          }
20          ,   total: total
21          ,   total_formatted: total_formatted
22          ,   recordtype: record.getValue('type')
23          ,   mainline: record.getValue('mainline')

```

```

24 , amountremaining: toCurrency(record.getValue(amount_remaining))
25 , amountremaining_formatted: formatCurrency(record.getValue(amount_remaining))
26 , closedate: close_date
27 , closedateInMilliseconds: new Date(close_date).getTime()
28 , tran_date: tran_date
29 , tranDateInMilliseconds: new Date(tran_date).getTime()
30 , duedate: due_date
31 , dueinmilliseconds: due_in_milliseconds
32 , isOverdue: due_in_milliseconds <= 0 && ((-1 * due_in_milliseconds) / 1000 / 60 / 60 / 24) >= 1
33 , status: {
34     internalid: record.getValue('status')
35     , name: record.getText('status')
36 }
37 , currency: {
38     internalid: record.getValue('currency')
39     , name: record.getText('currency')
40 }
41 };
42 });

```

Replace these lines with the following code:

```

1 return _.map(results || [], function (record)
2 {
3     var due_date = record.getValue('duedate')
4     , close_date = record.getValue('closedate')
5     , tran_date = record.getValue('trandate')
6     , due_in_milliseconds = (!!due_date ? nlapiStringToDate(due_date).getTime() : (new Date()).getTime()) - now
7     , total = toCurrency(record.getValue(amount_field))
8     , total_formatted = formatCurrency(record.getValue(amount_field));
9     return {
10         internalid: record.getId()
11         , tranid: record.getValue('tranid')
12         , order_number: record.getValue('tranid') // Legacy attribute
13         , date: tran_date // Legacy attribute
14         , summary: { // Legacy attribute
15             total: total
16             , total_formatted: total_formatted
17         }
18         , total: total
19         , total_formatted: total_formatted
20         , recordtype: record.getValue('type')
21         , mainline: record.getValue('mainline')
22         , amountremaining: toCurrency(record.getValue(amount_remaining))
23         , amountremaining_formatted: formatCurrency(record.getValue(amount_remaining))
24         , closedate: close_date
25         , closedateInMilliseconds: close_date ? nlapiStringToDate(close_date).getTime() : 0
26         , tran_date: tran_date
27         , tranDateInMilliseconds: tran_date ? nlapiStringToDate(tran_date).getTime() : 0
28         , duedate: due_date
29         , dueinmilliseconds: due_in_milliseconds
30         , isOverdue: due_in_milliseconds <= 0 && ((-1 * due_in_milliseconds) / 1000 / 60 / 60 / 24) >= 1
31         , status: {
32             internalid: record.getValue('status')
33             , name: record.getText('status')
34         }
35         , currency: {
36             internalid: record.getValue('currency')
37             , name: record.getText('currency')
38         }
39     };
40 });

```

3. Configure your SSP Application to use the custom Models.js file.
 1. Open the My Account SSP application record at Setup > SuiteCommerce Advanced > SSP Applications.
 2. On the **Scripts** subtab, update the Libraries list to include the Models.js file from your custom folder.

This file must maintain the correct position at the bottom of the list of library files.

3. Click **Save**.
4. Verify that the customization was successful.

The **By Due Date** and **By Invoice Date** sort order should be correct on the Invoices page in your web store. The invoices appear in ascending or descending order based on what the customer selects on the Invoice page in the web store.

PayPal Payments Cause Error at Checkout

i Applies to: SuiteCommerce Advanced | Mont Blanc

This section applies to the **Mont Blanc** release of SuiteCommerce Advanced.

In **Mont Blanc** releases of SuiteCommerce Advanced, customers may receive the following error message when placing orders using PayPal as the payment option:

Your order is below the minimum order amount of 0.

Although the error message is displayed, a sales order is created in NetSuite. To prevent this error message from displaying, extend the `past()` method as described in this procedure. Note that the error message is returned intermittently and is related to how PayPal returns sales amounts. You should apply this patch to avoid any cases of the error message being returned incorrectly.

i Note: Before proceeding, familiarize yourself with the [Best Practices for Customizing SCA](#).

Step 1: Extend the `OrderWizard.Module.PaymentMethod.PayPal.js` File

1. This step explains how to extend the `OrderWizard.Module.PaymentMethod.PayPal.js` file, which is located in the **OrderWizard.Module.PaymentMethod** module. You can download the code samples described in this procedure here: [PaymentMethod.PayPal.Extension@1.0.0.zip](#)

If you have not done so already, create a directory to store your custom module.

2. Open this directory and create a subdirectory to maintain your customizations.

Give this directory a name similar to the module being customized. For example:

`Modules/extensions/PaymentMethod.PayPal.Extension@1.0.0`

3. In your new `PaymentMethod.PayPal.Extension@1.0.0` module, create a subdirectory called `JavaScript`.

`Modules/extensions/PaymentMethod.PayPal.Extension@1.0.0/JavaScript`

4. In your new `JavaScript` subdirectory, create a JavaScript file to extend `OrderWizard.Module.PaymentMethod.PayPal.js`.

Name this file according to best practices. For example:

`OrderWizard.Module.PaymentMethod.PayPal.Extension.js`

5. Open this file and extend the `past()` method as shown in the following code snippet.

```
1 | define('OrderWizard.Module.PaymentMethod.PayPal.Extension'
2 | , [
3 |   'OrderWizard.Module.PaymentMethod.PayPal'
4 |   , 'underscore'
5 | ]
6 | , function (
7 |   OrderWizardModulePaymentMethodPayPal
8 | , -
```

```

9  )
10 {
11 'use strict';
12 _._extend(OrderWizardModulePaymentMethodPayPal.prototype,
13 {
14     past: function()
15     {
16         if (this.isActive() && !this.wizard.isPaypalComplete() && !this.wizard.hidePayment() && this.wizard.mod
17         el.get('confirmation').isNew())
18         {
19             var checkout_url = Session.get('touchpoints.checkout')
20             , joint = ~checkout_url.indexOf('?') ? '&' : '?'
21             , previous_step_url = this.wizard.getPreviousStepUrl();
22
23             checkout_url += joint + 'paypal=T&next_step=' + previous_step_url;
24
25             Backbone.history.navigate(previous_step_url, {trigger: false, replace: true});
26
27             document.location.href = checkout_url;
28
29             throw new Error('This is not an error. This is just to abort javascript');
30         }
31     }
32 });
33 }
34 );

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Extension

1. Open the PaymentMethod.PayPal.Extension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/PaymentMethod.PayPal.Extension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```

1  {
2      "gulp": {
3          "javascript": [
4              "JavaScript/*.js"
5          ]
6      }
7  }

```

4. Save the ns.package.json file.
5. Open the distro.json file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the modules object.

This ensures that the Gulp tasks include your extension when you deploy. In this example, the extensions/PaymentMethod.PayPal.Extension module is added at the beginning of the list of modules. However, you can add the module anywhere in the modules object. The order of precedence in this list does not matter.

```

1  {
2      "name": "SuiteCommerce Advanced Elbrus",
3      "version": "2.0",
4      "buildToolsVersion": "1.3.0",
5      "folders": {
6          "modules": "Modules",
7          "suitecommerceModules": "Modules/suitecommerce",
8          "extensionsModules": "Modules/extensions",
9          "thirdPartyModules": "Modules/third_parties",

```

```

10     "distribution": "LocalDistribution",
11     "deploy": "DeployDistribution"
12 },
13     "modules": {
14         "extensions/PaymentMethod.PayPal.Extension": "1.0.0",
15         "suitecommerce/Account": "2.3.0",
16         "suitecommerce/Address": "2.4.0",
17         ...

```

7. Include the module definition ("OrderWizard.Module.PaymentMethod.PayPal") in the dependencies array of the Shopping application of the JavaScript object.

Your `distro.json` file should look similar to the following:

```

1  "tasksConfig": {
2  //...
3  "javascript": [
4      //...
5      {
6          "entryPoint": "SC.Shopping.Starter",
7          "exportFile": "shopping.js",
8          "dependencies": [
9              //...
10             "Newsletter",
11             "ProductDetailToQuote",
12             //...
13             "OrderWizard.Module.PaymentMethod.PayPal.Extension"
14         ],

```



Note: Best practice is to place any new modules at the bottom of the list in the dependencies array.

8. Save the `distro.json` file.

Step 3: Test and Deploy Your Extension

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)).

If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

An order placed in your web store using PayPal as the payment method should not display an error after the customer clicks **Place Order**. The customer should see an order confirmation and summary.

Canonical Tags Populated With Relative Paths

Applies to: SuiteCommerce Advanced | Vinson

This section applies to the **Vinson** release of SuiteCommerce Advanced only.

In Vinson releases of SuiteCommerce Advanced, canonical URLs for commerce categories are generated as **relative** paths. Search engines do not index relative canonical URLs. Therefore, to ensure optimal SEO rankings, you must apply the patch described here to change your commerce category canonical URLs to use absolute paths. Best practices require customizing the existing source code using the `.extend` method, as detailed below.

Note: Before proceeding, familiarize yourself with the [Best Practices for Customizing SCA](#).

Step 1: Extend the Facets.Browse.View.js File

1. To extend the Facets.Browse.View.js file, which is located in the **Facets** module, create a directory to store your custom module.
2. Open this directory and create a subdirectory to maintain your customizations.
Give this directory a name similar to the module being customized. For example:
Modules/extensions/FacetsExtension@1.0.0
3. In your new FacetsExtension@1.0.0 module, create a subdirectory called JavaScript.
Modules/extensions/FacetsExtension@1.0.0/JavaScript
4. In your new JavaScript subdirectory, create a JavaScript file to extend Facets.Browse.View.js .
Name this file according to best practices. For example:
Facets.Browse.View.Extension.js
5. Open this file and extend the getPath method to modify the url as follows.

```

1  define('Facets.Browse.View.Extension'
2  , [
3    'Facets.Browse.View'
4    , 'underscore'
5  ]
6  , function (
7    FacetsBrowseView
8    , _
9  ) {
10   {
11     'use strict';
12     _.extend(FacetsBrowseView.prototype,
13       {
14         getPath: function ()
15         {
16           var base_url = window.location.protocol + '//' + window.location.hostname;
17           if (this.model.get('category'))
18           {
19             var category_canonical = this.model.get('category').get('canonical') || this.model.get('category').get('fullurl');
20             return (category_canonical.indexOf('/') === 0 ? base_url : '') + category_canonical;
21           }
22           else
23           {
24             var canonical = base_url + '/' + Backbone.history.fragment
25             , index_of_query = canonical.indexOf('?');
26             // !~ means: indexOf == -1
27             return !~index_of_query ? canonical : canonical.substring(0, index_of_query);
28           }
29         }
30       });
31   }
32   );

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Extension

1. Open the FacetsExtension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/FacetsExtension@1.0.0/ns.package.json

3. Build the ns.package.json file using the following code

```

1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   }
7 | }

```

4. Save the ns.package.json file.

5. Open the distro.json file.

This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the modules object.

This ensures that the Gulp tasks include your extension when you deploy. In this example, the extensions/FacetsExtension module is added at the beginning of the list of modules. However, you can add the module anywhere in the modules object. The order of precedence in this list does not matter.

```

1 | {
2 |   "name": "SuiteCommerce Advanced Elbrus",
3 |   "version": "2.0",
4 |   "buildToolsVersion": "1.3.0",
5 |   "folders": {
6 |     "modules": "Modules",
7 |     "suitecommerceModules": "Modules/suitecommerce",
8 |     "extensionsModules": "Modules/extensions",
9 |     "thirdPartyModules": "Modules/third_parties",
10 |    "distribution": "LocalDistribution",
11 |    "deploy": "DeployDistribution"
12 |  },
13 |   "modules": {
14 |     "extensions/FacetsExtension": "1.0.0",
15 |     "suitecommerce/Account": "2.3.0",
16 |     "suitecommerce/Address": "2.4.0",
17 |     ...

```

7. Include the module definition ("Facets.Browse.View.Extension") in the dependencies array of the Shopping application of the JavaScript object.

Your distro.json file should look similar to the following:

```

1 | "tasksConfig": {
2 |   //...
3 |   "javascript": [
4 |     //...
5 |     {
6 |       "entryPoint": "SC.Shopping.Starter",
7 |       "exportFile": "shopping.js",
8 |       "dependencies": [
9 |         //...
10 |         "Newsletter",
11 |         "ProductDetailToQuote",
12 |         //...
13 |         "Facets",
14 |         "Facets.Browse.View.Extension"
15 |       ],

```



Note: Best practice is to place any new modules at the bottom of the list in the dependencies array. In this case, FacetsBrowseExtension must be placed after Facets as there are dependencies on that module.

8. Save the distro.json file.

Step 3: Test and Deploy Your Extension

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)).

If you are currently running SCA on a local server, your changes should appear on your local site immediately.

2. Confirm your results.

All commerce category canonical URLs have absolute paths.

Shopping Cart Not Scrolling (Mobile)

Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson

This section applies to the **Vinson** release of SuiteCommerce Advanced and earlier.

In some cases, mobile users encounter an issue where their Cart ceases to scroll after removing an item from the **Saved for Later** product list. This issue can occur in the following scenario:

- The user adds more than one item to their Cart.
- The user adds items to their **Saved for Later** product list.
- The user later removes an item from the **Saved for Later** product list and returns to their Cart.
- The user can no longer scroll their Cart.

If you experience this issue on your SCA site, perform the following steps to correct. Best practices require customizing the existing source code using the `.extend` method, as detailed below.

Note: Before proceeding, familiarize yourself with the [Best Practices for Customizing SCA](#).

Step 1: Extend the `ProductList.DetailsLater.View.js` File

1. To extend the `ProductList.DetailsLater.View.js` file, which is located in the **ProductList** module, create a directory to store your custom module.
2. Open this directory and create a subdirectory to maintain your customizations.
Give this directory a name similar to the module being customized. For example:
`Modules/extensions/ProductListExtension@1.0.0`
3. In your new `ProductListExtension@1.0.0` module, create a subdirectory called `JavaScript`.
`Modules/extensions/ProductListExtension@1.0.0/JavaScript`
4. In your new `JavaScript` subdirectory, create a JavaScript file to extend `ProductList.DetailsLater.View.js`.
Name this file according to best practices. For example:
`ProductList.DetailsLater.View.Extension.js`
5. Open this file and extend the `deleteListItemHandler` method to include the following line:

```
1 | self.$('[data-action="pushable"]').scPush();
```


Assuming that you have followed best practices outlined in this procedure, your extension should look like this:

```

1  define('ProductList.DetailsLater.View.Extension'
2  , [
3    'ProductList.DetailsLater.View'
4    , 'underscore'
5    , 'jQuery'
6  ]
7  , function (
8    ProductListDetailsLaterView
9    ,
10   jQuery
11  )
12  {
13    'use strict';
14
15    _.extend(ProductListDetailsLaterView.prototype,
16    {
17      deleteListItemHandler: function (target)
18      {
19        var self = this
20        ,   itemid = jQuery(target).closest('article').data('id')
21        ,   product_list_item = this.model.get('items').findWhere({
22            internalid: itemid + ''
23          })
24        ,   success = function ()
25        {
26          if (self.application.getLayout().updateMenuItemsUI)
27          {
28            self.application.getLayout().updateMenuItemsUI();
29          }
30
31          self.deleteConfirmationView.$containerModal.modal('hide');
32          self.render();
33          self.$('[data-action="pushable"]').scPush();
34          self.showConfirmationMessage(_('The item was removed from your product list').translate(), true);
35        };
36
37        self.model.get('items').remove(product_list_item);
38        self.deleteListItem(product_list_item, success);
39      }
40    });
41  }
42  );

```

6. Save the file.

Step 2: Download the jQuery.scPush.js File

1. Download the following file:
[jQuery.scPush.zip](#)
2. Open the .zip file and extract jQuery.scPush.js to the JavaScript subdirectory of your custom module.
Modules/extensions/ProductListExtension@1.0.0/JavaScript/jQuery.scPush.js

Step 3: Prepare the Developer Tools for Your Customizations

1. Open the ProductListExtension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/ProductListExtension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```

1 {
2   "gulp": {
3     "javascript": [
4       "JavaScript/*.js"
5     ]
6   }
7   "overrides": {
8     "Modules/suitecommerce/jqueryExtras@x.y.z/JavaScript/jquery.scPush.js": "JavaScript/jquery.scPush.js"
9   }
10 }

```

Note: In the above code example, replace the string x.y.z with the current version of your source module.

4. Save the ns.package.json file.

5. Open the distro.json file.

This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the modules object to ensure that the Gulp tasks include your extension when you deploy.

In this example, the extensions/ProductListExtension module is added at the beginning of the list of modules. However, you can add the module anywhere in the modules object. The order of precedence in this list does not matter.

```

1 {
2   "name": "SuiteCommerce Advanced Elbrus",
3   "version": "2.0",
4   "buildToolsVersion": "1.3.0",
5   "folders": {
6     "modules": "Modules",
7     "suitecommerceModules": "Modules/suitecommerce",
8     "extensionsModules": "Modules/extensions",
9     "thirdPartyModules": "Modules/third_parties",
10    "distribution": "LocalDistribution",
11    "deploy": "DeployDistribution"
12  },
13  "modules": {
14    "extensions/ProductListExtension": "1.0.0",
15    "suitecommerce/Account": "2.3.0",
16    "suitecommerce/Address": "2.4.0",
17    ...

```

7. Include the file definition ("ProductList.DetailsLater.View.Extension") in the dependencies array of the Shopping and MyAccount applications of the JavaScript object.

Your distro.json file should look similar to the following:

```

1 "tasksConfig": {
2   //...
3   "javascript": [
4     //...
5     {
6       "entryPoint": "SC.Shopping.Starter",
7       "exportFile": "shopping.js",
8       "dependencies": [
9         //...
10        "Newsletter",
11        "ProductDetailToQuote",
12        "ProductList.DetailsLater.View.Extension"
13      ],
14      //...
15    }
16    {
17      "entryPoint": "SC.MyAccount.Starter",
18      "exportFile": "myaccount.js",

```

```

18         "dependencies": [
19             //...
20             "Location.Model",
21             "StoreLocator.Model",
22             "ProductList.DetailsLater.View.Extension"
23         ],
24         //...

```

Note: Best practice is to place any new modules at the bottom of the list in the dependencies array.

8. Save the distro.json file.

Step 4: Test and Deploy Your Extension

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. Confirm your results.

Upon successful deployment, you can remove items from the **Save for Later** list, return to the Cart, and scroll through your list of items using a mobile interface.

Error When Adding Items to Categories in Site Management Tools

Applies to: SuiteCommerce Advanced | Vinson

This patch contains a fix for an error on a category or subcategory that contains more than 10 items in Site Management Tools. SMT generates an error if you try to add more items to a category or subcategory that already contains more than 10 items. This error occurs because the default query sent to Items API to retrieve the items includes facets and the Items API returns no more than 10 results for queries that include facets.

This patch adds a Search API fieldset named **CmsAdapterSearch**. To install the patch, you must override files in the **Configuration**, **Facets**, and **CMSadapter** modules and add the new **CmsAdapterSearch** fieldset in the SuiteCommerce Advanced configuration. For an example of the changes needed for this patch, see [AddingItemstoCategoriesinSMT.zip](#).

Note: In general, NetSuite best practice is to extend JavaScript using the JavaScript prototype object. This improves the chances that your customizations continue to work when migrating to a newer version of SuiteCommerce Advanced. However, this patch requires you to modify files that you cannot extend, and therefore requires you to override the existing module files in a custom module. For more information, see the help topic [Develop Your SCA Customization](#).

Step 1. Override the ItemsSearchAPI.json File

1. To override the ItemsSearchAPI.json file, located in the **Configuration** module, create a directory to store your custom modules, for example, extensions.
For more information about module configuration with JSON, see [Modify JSON Configuration Files](#).
2. Open this directory and create a subdirectory to maintain your customizations.

Give this directory a name similar to the module being customized. For example:

Modules/extensions/ConfigurationExtension@1.0.0

3. In your new ConfigurationExtension@1.0.0 directory, create a subdirectory called Configuration.
4. Copy the following file into this directory:
Modules/suitecommerce/Configuration@1.0.0/Configuration/ItemsSearchAPI.json
5. Open ItemsSearchAPI.json and make the changes in steps 6 and 7.
6. In ItemsSearchAPI.json, replace the following line:

```
1 | "enum": ["Facets", "itemDetails", "relatedItems", "correlatedItems", "merchandisingZone", "typeAhead", "itemsSearcher"],
```

With the following JSON object:

```
1 | "enum": ["Facets", "itemDetails", "relatedItems", "correlatedItems", "merchandisingZone", "typeAhead", "itemsSearcher", "CmsAdapterSearch"],
```

7. In ItemsSearchAPI.json, add the following object to the default object:

```
1 | { "id": "CmsAdapterSearch", "fieldset": "search" }
```

The default object should look similar to the following code after this step:

```
1 |         "default": [
2 |             {
3 |                 "id": "Facets",
4 |                 "fieldset": "search",
5 |                 "include": "facets"
6 |             },
7 |             ...
8 |             {
9 |                 "id": "CmsAdapterSearch",
10 |                 "fieldset": "search"
11 |             }
12 |         ]
13 |
```

8. Save the file.

Step 2. Override the Facets.Model.js File

1. To override the Facets.Model.js file, located in the **Facets** module, create a directory in the extensions directory to store the custom module.

Give this directory a name similar to the module being customized. For example:

Modules/extensions/FacetsExtension@1.0.0

2. In the FacetsExtension@1.0.0 module, create a subdirectory called JavaScript.
3. Copy the following file into this directory:
Modules/suitecommerce/Facets@2.3.0/JavaScript/Facets.Model.js
4. Open Facets.Model.js and make the following change.

Replace the existing initialize function:

```
1 | initialize: function ()
2 | {
3 |     // Listen to the change event of the items and converts it to an ItemDetailsCollection
4 |     this.on('change:items', function (model, items)
5 |     {
6 |         if (!(items instanceof ItemDetailsCollection))
```

```

7         {
8             // NOTE: Compact is used to filter null values from response
9             model.set('items', new ItemDetailsCollection(_.compact(items)));
10        }
11    });
12 }

```

With the following JavaScript code:

```

1 initialize: function ()
2 {
3     if (options && options.searchApiMasterOptions)
4     {
5         this.searchApiMasterOptions = options.searchApiMasterOptions;
6     }
7
8     // Listen to the change event of the items and converts it to an ItemDetailsCollection
9     this.on('change:items', function (model, items)
10    {
11        if (!(items instanceof ItemDetailsCollection))
12        {
13            // NOTE: Compact is used to filter null values from response
14            model.set('items', new ItemDetailsCollection(_.compact(items)));
15        }
16    });
17 }

```

5. Save the file.

Step 3. Override the CMSadapterImpl.Categories.js File

1. To override the CMSadapterImpl.Categories.js file, located in the **CMSadapter** module, create a directory in the extensions directory to store the custom module.

Give this directory a name similar to the module being customized. For example:

Modules/extensions/CMSadapterExtension@1.0.0

2. In the CMSadapterExtension@1.0.0 module directory, create a subdirectory called JavaScript.
3. Copy the following file into this directory:
Modules/suitecommerce/CMSadapter@3.0.0/JavaScript/CMSadapterImpl.Categories.js
4. Open CMSadapterImpl.Categories.js and make the changes in steps 5 to 7.
5. In CMSadapterImpl.Categories.js, add 'SC.Configuration' to the list of dependencies. For example:

```

1 define('CMSadapterImpl.Categories'
2 , [
3     'Facets.Model'
4     , ...
5     , 'SC.Configuration'
6 ]

```

6. In CMSadapterImpl.Categories.js, add the Configuration function parameter. For example:

```

1 , function (
2     FacetsModel
3     , ...
4     , Configuration
5 )

```

7. In CMSadapterImpl.Categories.js, replace the following line:

```

1 var model = new FacetsModel();

```

With the following JavaScript code:

```
1 var model = new FacetsModel({ searchApiMasterOptions:
2 Configuration.get('searchApiMasterOptions.CmsAdapterSearch') });
```

8. Save the file.

Step 4. Prepare the Developer Tools for Your Overrides

1. To make sure that the Gulp tasks include your modules when you deploy, set up the `ns.package.json` files for your custom module and modify `distro.json`.

Create the `ns.package.json` file in the `ConfigurationExtension@X.X.X` directory. Add the following code to `ns.package.json` in the `Modules/extensions/ConfigurationExtension@X.X.X` directory:

```
1 {
2   "gulp": {
3     "configuration": [
4       "Configuration/*.json"
5     ]
6   }
7 }
```

2. Create the `ns.package.json` file in the `FacetsExtension@X.X.X` directory. Add the following code to `ns.package.json` in the `Modules/extensions/FacetsExtension@X.X.X` directory:

```
1 {
2   "gulp": {
3     "javascript": [
4       "JavaScript/*"
5     ]
6   }
7 }
```

3. Create the `ns.package.json` file in the `CMSadapterExtension@X.X.X` directory. Add the following code to `ns.package.json` in the `Modules/extensions/CMSadapterExtension@X.X.X` directory:

```
1 {
2   "gulp": {
3     "javascript": [
4       "JavaScript/*"
5     ]
6   }
7 }
```

4. In `distro.json`, add your custom modules to the `modules` object.

This ensures that the Gulp tasks include your module when you deploy. In this example, the custom modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the `modules` object. The order of precedence in this list does not matter.

```
1 {
2   "name": "SuiteCommerce Advanced Vinson Release",
3   "version": "2.0",
4   "buildToolsVersion": "1.2.1",
5   "folders": {
6     "modules": "Modules",
7     "suitecommerceModules": "Modules/suitecommerce",
8     "thirdPartyModules": "Modules/third_parties",
9     "distribution": "LocalDistribution",
10    "deploy": "DeployDistribution"
11  },
12  "modules": {
13    "extensions/ConfigurationExtension": "X.X.X",
```

```

14 | "extensions/FacetsExtension": "X.X.X",
15 | "extensions/CMSadapterExtension": "X.X.X",

```

Step 5. Deploy Your Override

1. To test this customization, you must first deploy it to your NetSuite account. You may need to modify the SuiteCommerce Advanced configuration, therefore you must deploy it first. See the help topic [Deploy SCA Customizations to NetSuite](#).
2. After you deploy the customization, log in to NetSuite and go to Commerce > Websites> Configuration, and select the appropriate web site and domain.
3. On the **Advanced > Search Results** tab, make sure that **CmsAdapterSearch** appears for the search fieldset. If it does not appear, you must add it and click **Save**.

You can now add more than 10 items to a category in Site Builder.

Item Search API Response Data not Cached

Applies to: SuiteCommerce Advanced | Vinson

This section applies to the **Vinson** release of SuiteCommerce Advanced.

In **Vinson** releases of SuiteCommerce Advanced, the Item Search API response data is not cached in the Content Delivery Network (CDN) by default. To enable caching of the Item Search API response, you must customize your implementation to include the `pricelevel` input parameter in the Item Search API query. Caching response data in the CDN is critical as it decreases Item Search API response time and improves application performance.

Note: Before proceeding, familiarize yourself with the [Best Practices for Customizing SCA](#).

You can download the code samples described in this procedure here: [SearchApiCdnCache.zip](#).

Step 1: Extend the Session.js File

1. To extend the Session.js file, which is located in the **Session** module, create a directory to store your custom module.
2. Open this directory and create a subdirectory to maintain your customizations.
Give this directory a name similar to the module being customized. For example:
`Modules/extensions/SessionExtension@1.0.0`
3. In your new SessionExtension@1.0.0 module, create a subdirectory called JavaScript.
`Modules/extensions/SessionExtension@1.0.0/JavaScript`
4. In your new JavaScript subdirectory, create a JavaScript file to extend Session.js.
Name this file according to best practices. For example:
`Session.Extension.js`
5. Open this file and extend the `getSearchApiParams()` method to add the `pricelevel` parameter to the Item Search API GET request.

```

1 | // Session.Fix.js
2 | // When Session.getSearchApiParams is called, this method adds the pricelevel param to the output
3 | define('Session.Extension'
4 | , [

```

```

5 |     'Session'
6 |   ]
7 |   , function (Session)
8 |   {
9 |     'use strict';
10 |    var original_getSearchApiParams = Session.getSearchApiParams;
11 |    Session.getSearchApiParams = function ()
12 |    {
13 |      var search_api_params = original_getSearchApiParams.apply(this, arguments);
14 |      search_api_params.pricelevel = this.get('priceLevel', '');
15 |      return search_api_params;
16 |    }
17 |  });

```

6. Save the file.

Step 2: Prepare the Developer Tools for Your Extension

1. Open the SessionExtension@1.0.0 module.
2. Create a file in this module and name it **ns.package.json**.
Modules/extensions/SessionExtension@1.0.0/ns.package.json
3. Build the ns.package.json file using the following code

```

1 | {
2 |   "gulp": {
3 |     "javascript": [
4 |       "JavaScript/*.js"
5 |     ]
6 |   }
7 | }

```

4. Save the ns.package.json file.
5. Open the distro.json file.
This file is located in the top-level directory of your SuiteCommerce Advanced source code.

6. Add your custom module to the modules object.

This ensures that the Gulp tasks include your extension when you deploy. In this example, the extensions/SessionExtension module is added at the beginning of the list of modules. However, you can add the module anywhere in the modules object. The order of precedence in this list does not matter.

```

1 | {
2 |   "name": "SuiteCommerce Advanced Elbrus",
3 |   "version": "2.0",
4 |   "buildToolsVersion": "1.3.0",
5 |   "folders": {
6 |     "modules": "Modules",
7 |     "suitecommerceModules": "Modules/suitecommerce",
8 |     "extensionsModules": "Modules/extensions",
9 |     "thirdPartyModules": "Modules/third_parties",
10 |    "distribution": "LocalDistribution",
11 |    "deploy": "DeployDistribution"
12 |  },
13 |   "modules": {
14 |     "extensions/SessionExtension": "1.0.0",
15 |     "suitecommerce/Account": "2.3.0",
16 |     "suitecommerce/Address": "2.4.0",
17 |     ...

```

7. Include the module definition ("Session.Extension") in the dependencies array of the Shopping application of the JavaScript object.

Your distro.json file should look similar to the following:


```

1  "tasksConfig": {
2    //...
3    "javascript": [
4      //...
5      {
6        "entryPoint": "SC.Shopping.Starter",
7        "exportFile": "shopping.js",
8        "dependencies": [
9          //...
10         "Newsletter",
11         "ProductDetailToQuote",
12         //...
13         "Session.Extension"
14       ],

```

Note: Best practice is to place any new modules at the bottom of the list in the dependencies array.

8. Save the distro.json file.

Step 3: Test and Deploy Your Extension

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)). If you are currently running SCA on a local server, your changes should appear on your local site immediately.
2. To verify that the Item Search API output response can now be cached in the CDN, navigate to the Product Display Page in your web store.
3. Open the browser's Developer Tools.
4. View the Network XHR response data.
5. Refresh your web store page.
6. Verify that the items query contains the `pricelevel` parameter.

Secure Shopping Domains (Elbrus, Vinson, Mont Blanc, and Denali)

Applies to: SuiteCommerce Advanced | Denali | Mont Blanc | Vinson | Elbrus

With the Elbrus release of SuiteCommerce Advanced, a single secure (HTTPS) domain for the checkout and shopping applications is supported. In order to provide this capability to prior releases of SuiteCommerce Advanced, you need to update your implementation by applying the patches provided here. For Elbrus implementations, you also need to apply the provided patch to fix a `ERR_WS_REQUIRE_CUSTOMER_LOGIN` error returned from the `setPurchaseNumber` method.

To update SuiteCommerce Advanced to use SSL correctly:

1. Create a backup of your existing bundle.
2. Download the patch according to your implementation:
 - [Elbrus-ssl.patch](#)
 - [Vinson-ssl.patch](#)
 - [MontBlanc-ssl.patch](#)

■ [Denali-ssl.patch](#)

3. Apply the patch. Details of what the patch is accomplishing are provided below.

Note: The original patches provided for Vinson, Mont Blanc, and Denali have been updated. If you have previously applied a patch for SSL, only the diff between the previous patch and the current one should be applied.

4. Switch the existing non-secure shopping domain to a secure shopping domain or set up a new secure shopping domain from scratch as described under [Set Up a Domain](#).

For each implementation patch, instead of using `url.indexOf('https')` to check the domain, multiple methods have been combined and centralized in the `Utils.js` file. The following properties are used to determine which domain is in use:

Utils Property	Domain In Use
<code>Utils.isShoppingDomain()</code>	Shopping domain
<code>Utils.isCheckoutDomain()</code>	Checkout domain
<code>Utils.isSingleDomain()</code>	Single domain
<code>Utils.isInShopping()</code>	shopping.ssp
<code>Utils.isInCheckout()</code>	checkout.ssp or myaccount.ssp

For the Denali implementation patch, NetSuite also maintains access control by replacing `session.isLoggedIn` with `session.IsLoggedIn2`. In addition, the Denali patch takes advantage of the Serversync touchpoint patch to maintain the identity of the user and cart contents.

Secure Shopping Domain (pre-Denali)

Applies to: SuiteCommerce Advanced | pre-Denali

This section applies to **pre-Denali** releases of SuiteCommerce Advanced.

A secure shopping domain creates an encrypted connection between your web server and your customer's web browser. You can use an SSL certificate to secure the shopping portion of your web store under an HTTPS domain. HTTPS is currently the industry standard for ecommerce services and consumers prefer seeing the secure icon in their browser address bar.

Including secure technology in your shopping area assures your customers that their activities on your site are safe. In addition, search engines tend to rank secure sites higher than non-secure sites.

There are two new SuiteCommerce API methods that help make HTTPS support possible. These methods distinguish between the checkout, shopping, and single domains.

- `isCheckoutSupported()`: returns true if we are in the checkout domain or in a single domain
- `isShoppingSupported()`: returns true if we are in the shopping domain or in a single domain

`!~request.getURL().indexOf('https')` is the legacy domain check and it will no longer work to distinguish between the domains.

The `session.isLoggedIn()` method is the legacy session check for a user's logged in status. This method has been replaced with the new `session.isLoggedIn2()` method in later versions of SuiteCommerce Advanced. The new session check method is required in this migration to help maintain the identity of the user.

To leverage a secure shopping domain in a pre-Denali version of SuiteCommerce Advanced, you need to:

- Replace all instances of the legacy domain check with the new domain check methods.
- Replace all instances of the legacy session check with the new session check method.

Migration Tasks

 **Applies to:** SuiteCommerce Advanced | pre-Denali

This migration is highly technical and involves customizing multiple files. Update the following files in each SSP Application to use the new domain check methods as well as the new session check method where applicable:

- [commons.js](#)
- [Utils.js](#)
- [Models.js](#)
- [sc.environment.ssp](#)
- [index.ssp](#) and [index-local.ssp](#)
- [jQuery.ajaxSetup.js](#)
- [live-order.ss](#) and [live-order-line.ss](#)
- [CheckoutSkipLogin.js](#)
- [NavigationHelper.js](#)

In addition, complete these tasks.

- [Update Legacy Checks and Methods](#)
- [Deploy the Migration](#)



Important: This document is intended to be a guide only. It is not a comprehensive procedure that guarantees a correct implementation of HTTPS support for your web store.

Before you begin making changes, back up any custom files that already exist. Be careful not to overwrite existing custom files.

For each file, go to the SSP Application under Documents > Files > File Cabinet > Web Site Hosting Files > Live Hosting Files > SSP Applications and copy the Reference files to the same location in the Custom folder.



Note: Checkout with Content Delivery Network (CDN) enabled is not supported by pre-Denali versions of SCA. As a workaround, see [Checkout With CDN](#) to enable CDN on a pre-Denali checkout flow.

To modify files for HTTPS:

1. Open each file at the specified location in the Custom folder.
2. Search each file for the appropriate location using the line number or surrounding text from the code snippet.

Use the following conventions to determine when you need to add or remove code.

```

1  --- = Remove this line of code.
2  +++ = Add this line of code.
3
4  Note: Surrounding text and line numbers vary depending on each SSP Application as well as existing customizations. They
   are included as reference points only.

```

3. Add or remove statements as indicated in the code snippet and save.

commons.js

File locations:

- ShopFlow > Custom ShopFlow > ssp_libraries > commons.js
- MyAccount > Custom MyAccount > ssp_libraries > commons.js
- Checkout > Custom Checkout > ssp_libraries > commons.js

Add the entire Utils variable to the Utilities section of each commons.js file. The Utils variable contains the new methods that distinguish between the domains.

```

1  ...
2
3  // Utilites
4  +++   var Utils = {
5  +++
6  +++       // @method isCheckoutDomain determines if we are in a secure checkout
7  +++       // domain or in a secure single domain environment
8  +++       // @return {Boolean} true if in checkout or in single domain
9  +++       isCheckoutDomain: function isCheckoutDomain()
10 +++       {
11 +++           return session.isCheckoutSupported();
12 +++       }
13 +++       // @method isShoppingDomain determines if we are in shopping domain (secure or non secure)
14 +++       // or in a secure single domain environment
15 +++       // @return {Boolean} true if in shopping or single domain
16 +++       , isShoppingDomain: function isShoppingDomain()
17 +++       {
18 +++           return session.isShoppingSupported();
19 +++       }
20 +++       // @method isSingleDomain determines if we are in a single domain environment
21 +++       // @return {Boolean} true if single domain
22 +++       , isSingleDomain: function isSingleDomain()
23 +++       {
24 +++           return this.isShoppingDomain() && this.isCheckoutDomain();
25 +++       }
26 +++       // @method isInShopping determines if we are in shopping ssp
27 +++       // @return {Boolean} true if in shopping domain, false if in checkout or myaccount
28 +++       , isInShopping: function isInShopping (request)
29 +++       {
30 +++           return this.isShoppingDomain() && (request.getHeader('X-SC-Touchpoint') === 'shopping' || request.getParame
31 +++           ter('X-SC-Touchpoint') === 'shopping');
32 +++       }
33 +++       // @method isInCheckout determines if we are in checkout ssp or my account ssp
34 +++       // @return {Boolean} true if in checkout domain
35 +++       , isInCheckout: function isInCheckout (request)
36 +++       {
37 +++           var self = this;
38 +++           if (!self.isSingleDomain())
39 +++           {
40 +++               return self.isCheckoutDomain();
41 +++           }
42 +++           else
43 +++           {
44 +++               var paypal_complete = ModelsInit.context.getSessionObject('paypal_complete') === 'T'

```

```

45  +++, is_in_checkout = request.getHeader('X-SC-Touchpoint') === 'checkout' ||
46  +++, request.getHeader('X-SC-Touchpoint') === 'myaccount' ||
47  +++, request.getParameter('X-SC-Touchpoint') === 'checkout' ||
48  +++, request.getParameter('X-SC-Touchpoint') === 'myaccount';
49  +++, return self.isCheckoutDomain() && (is_in_checkout || paypal_complete);
50  +++, }
51  +++, }
52  +++, }
53
54  ...

```

Call the new `session.isLoggedIn2()` method to maintain user identity.

```

1  ...
2  999: function isLoggedIn ()
3  {
4      'use strict';
5      --- // MyAccount (We need to make the following difference because isLoggedIn is always false in Shopping)
6      --- if (request.getURL().indexOf('https') === 0)
7      --- {
8      ---     return session.isLoggedIn();
9      --- }
10     --- else // Shopping
11     --- {
12     ---     return parseInt(nlapiGetUser() + '', 10) > 0 && !session.getCustomer().isGuest();
13     --- }
14
15     +++, return session.isLoggedIn2();
16 }
17
18 ...

```

Modify the `getEnvironment()` method to add the new domain check methods.

```

1  ...
2      , init: function () {}
3  220: , getEnvironment: function (session, request)
4      {
5          'use strict';
6          // Sets Default environment variables
7          var context = nlapiGetContext()
8          --- , isSecure = request.getURL().indexOf('https:') === 0;
9          +++, isSecure = Utils.isCheckoutDomain();
10
11          , siteSettings = session.getSiteSettings(['currencies', 'languages'])
12          , result = {
13          ---     baseUrl: session.getAbsoluteUrl(isSecure ? 'checkout' : 'shopping', '{{file}}')
14          +++,     baseUrl: session.getAbsoluteUrl(Utils.isInCheckout(request) ? 'checkout' : 'shopping', '{{file}}')
15          ,     currentHostString: request.getURL().match('http(s?):/(.*)/')[2]
16          ,     availableHosts: SC.Configuration.hosts || []
17          ,     availableLanguages: siteSettings.languages || []
18          ,     availableCurrencies: siteSettings.currencies || []
19          ,     companyId: context.getCompany()
20          ,     casesManagementEnabled: context.getSetting('FEATURE', 'SUPPORT') === 'T'
21          ,     giftCertificatesEnabled: context.getSetting('FEATURE', 'GIFTCERTIFICATES') === 'T'
22          };
23          // If there are hosts associated in the site we iterate them to check which we are in
24          // and which language and currency we are in
25          --- if (result.availableHosts.length && !isSecure)
26          +++, if (result.availableHosts.length && Utils.isShoppingDomain())
27          {
28              for (var i = 0; i < result.availableHosts.length; i++)
29
30  ...

```

Utils.js

File locations:

- ShopFlow > Custom ShopFlow > js > src > core > Utils.js
- MyAccount > Custom MyAccount > js > src > core > Utils.js
- Checkout > Custom Checkout > js > src > core > Utils.js

Update the `getDownloadPdfUrl()` method to add the new domain check methods.

```

1  ...
2  635: function getDownloadPdfUrl (params)
3      {
4      ---    params = params || {};
5      ---    params.n = SC.ENVIRONMENT.siteSettings.siteid;
6      ---    var origin = window.location.origin ? window.location.origin :
7      ---    (window.location.protocol + '//' + window.location.hostname + (window.location.port ? (':' + window.location.port) : ''));
8      ---    return __.addParamsToUrl(origin + __.getAbsolutePath('download.ssp'), params);
9
10     +++    params = params || {};
11     +++    params.n = SC && SC.ENVIRONMENT && SC.ENVIRONMENT.siteSettings && SC.ENVIRONMENT.siteSettings.siteid || '';
12     +++
13     +++    if(__.isSingleDomain())
14     +++    {
15     +++        return __.addParamsToUrl(__.getAbsolutePath('download.ssp'), params);
16     +++    }
17     +++    else
18     +++    {
19     +++        var origin = window.location.origin ? window.location.origin :
20     +++        (window.location.protocol + '//' + window.location.hostname + (window.location.port ? (':' + window.location.port) : ''));
21     +++
22     +++        return __.addParamsToUrl(origin + __.getAbsolutePath('download.ssp'), params);
23     +++    }
24
25    }
26
27    ...
28
29    797: function reorderUrlParams (url)
30    {
31        var params = []
32        ,   url_array = url.split('?');
33        if (url_array.length > 1)
34        {
35            params = url_array[1].split('&');
36            return url_array[0] + '?' + params.sort().join('&');
37        }
38        return url_array[0];
39    }
40
41
42    +++ // @method isShoppingDomain determines if we are in shopping domain (secure or non secure)
43    +++ // or single domain
44    +++ // @return {Boolean} true if in checkout or in single domain
45    +++ function isShoppingDomain ()
46    +++ {
47    +++     return SC.ENVIRONMENT.siteSettings.shoppingSupported;
48    +++ }
49    +++
50    +++ // @method isCheckoutDomain determines if we are in a secure checkout
51    +++ // domain or in a secure single domain environment
52    +++ // @return {Boolean} true if in checkout or in single domain
53    +++ function isCheckoutDomain ()
54    +++ {
55    +++     return SC.ENVIRONMENT.siteSettings.checkoutSupported;
56    +++ }
57    +++
58    +++ // @method isSingleDomain determines if we are in a single domain environment
59    +++ // @return {Boolean} true if single domain
60    +++ function isSingleDomain ()
61    +++ {
62    +++     return SC.ENVIRONMENT.siteSettings.isSingleDomain;
63    +++ }

```

```

64  +++
65  +++ // @method isInShopping determines if we are in shopping ssp
66  +++ // used when there are frontend features only shown in the shopping domain
67  +++ // @return {Boolean} true if in shopping domain, false if in checkout or myaccount
68  +++ function isInShopping ()
69  +++ {
70  +++     return !_isShoppingDomain() && (SC.ENVIRONMENT.SCTouchpoint === 'shopping' || SC.ENVIRONMENT.siteSettings.sitetype
71  +++     === 'STANDARD');
72  +++ }
73  +++
74  +++ // @method isInCheckout determines if we are in checkout or my account ssp
75  +++ // @return {Boolean} true if in checkout domain
76  +++ function isInCheckout ()
77  +++ {
78  +++     return !_isSingleDomain() ? !_isCheckoutDomain() : !_isCheckoutDomain() && (SC.ENVIRONMENT.SCTouchpoint === 'checkout'
79  +++     || SC.ENVIRONMENT.SCTouchpoint === 'myaccount');
80  +++ }
81  ...

```

Models.js

File locations:

- ShopFlow > Custom ShopFlow > ssp_libraries > Models.js
- MyAccount > Custom MyAccount > ssp_libraries > Models.js
- Checkout > Custom Checkout > ssp_libraries > Models.js

Verify the domain using the new domain check methods.

```

1  ...
2
3  19: Application.defineModel('SiteSettings', {
4
5  ...
6
7  121: settings.is_logged_in = session.isLoggedIn();
8      settings.touchpoints = basic_settings.touchpoints;
9      settings.shopperCurrency = session.getShopperCurrency();
10
11 ...
12
13 +++ settings.checkoutSupported = Utils.isCheckoutDomain();
14 +++ settings.shoppingSupported = Utils.isShoppingDomain();
15 +++ settings.isSingleDomain = settings.checkoutSupported && settings.shoppingSupported;
16
17 ...

```

Call the new session.isLoggedIn2() method and update the getFieldValues() method to use the new domain check methods.

```

1  ...
2
3  167: Application.defineModel('LiveOrder', {
4
5  ...
6
7  309: --- , isSecure: request.getURL().indexOf('https') === 0
8      --- , isLoggedIn: session.isLoggedIn()
9
10     +++ , isLoggedIn: session.isLoggedIn2()
11     +++ , isSecure: Utils.isCheckoutDomain()
12
13 ...
14
15  681: , getFieldValues: function ()

```

```

16         {
17             'use strict';
18             --- var order_field_keys = this.isSecure ? SC.Configuration.order_checkout_field_keys : SC.Configuration.or
der_shopping_field_keys;
19             +++ var order_field_keys = Utils.isInCheckout(request) && session.isLoggedIn2() ? SC.Configuration.order_check
out_field_keys : SC.Configuration.order_shopping_field_keys;
20         }
21     ...

```

Update the verifySession() method to remove the legacy domain check.

```

1  ...
2
3  1726:     Application.defineModel('ProductList', {
4
5  ...
6
7      // Returns a product list based on a given userId and id
8  1759:     , get: function (user, id)
9      {
10         // Verify session if and only if we are in My Account...
11         if (request.getURL().indexOf('https') === 0)
12         {
13             this.verifySession();
14         }
15         +++ this.verifySession();
16     }
17 ...
18
19     // Retrieves all Product Lists for a given user
20 1896:     , search: function (user, order)
21     {
22         // Verify session if and only if we are in My Account...
23         if (request.getURL().indexOf('https') === 0)
24         {
25             this.verifySession();
26         }
27
28         var filters = [new nlobjSearchFilter('isinactive', null, 'is', 'F')
29             , new nlobjSearchFilter('custrecord_ns_pl_pl_owner', null, 'is', user)]
30         , template_ids = []
31     }
32 ...

```

sc.environment.ssp

File locations:

- Checkout > Custom Checkout > sc.environment.ssp
- MyAccount > Custom MyAccount > sc.environment.ssp
- ShopFlow > Custom ShopFlow > sc.environment.ssp

Add a touchpoint check for Checkout and update the session.GetAbsoluteUrl() parameters to replace the legacy domain check.

```

1  ...
2
3  94:     <% if (SiteSettings) { %>
4         // Site Settings Info
5         <%
6         // under some wired cases the terms and conditions bring a script tag if there is a body tag present
7         // This code eliminates it in the case
8         var site_settings_json = JSON.stringify(SiteSettings).replace(/
9 >/ig, '').replace(/<\tbody*>/ig, '')
10         %>

```



```

11      SC.ENVIRONMENT.siteSettings = <%= site_settings_json %>;
12      // Site site (ADVANCED or STANDARD)
13      SC.ENVIRONMENT.siteType = '<%= SiteSettings.sitetype %>';
14
15      +++      // SCTouchpoint indicates the touchpoint the user is effectively in. We can only know with certain this in the proper
16      ssp
17      +++      // because there is still code that depends on the touchpoint
18      +++      // when in single ssp check if this it's necessary
19      +++      SC.ENVIRONMENT.SCTouchpoint = 'checkout';
20      <% } %>
21
22      ...
23
24      179:      if(!SC.ENVIRONMENT.baseUrl)
25      {
26      ---          SC.ENVIRONMENT.baseUrl = '<%= session.getAbsoluteUrl(request.getURL().indexOf('https:') === 0 ? 'checkout'
27      : 'shopping', '{{file}}') %>';
28      +++          SC.ENVIRONMENT.baseUrl = '<%= session.getAbsoluteUrl('checkout', '{{file}}') %>';
29      }
30
31      ...

```

Add a touchpoint check for MyAccount and update the session.GetAbsoluteUrl() parameters to replace the legacy domain check.

```

1      ...
2
3      106:      <% if (SiteSettings) { %>
4          // Site Settings Info
5          SC.ENVIRONMENT.siteSettings = <%= JSON.stringify(SiteSettings) %>;
6          // Site site (ADVANCED or STANDARD)
7          SC.ENVIRONMENT.siteType = '<%= SiteSettings.sitetype %>';
8
9      +++      // SCTouchpoint indicates the touchpoint the user is effectively in. We can only know with certain this in the proper
10      ssp
11      +++      // because there is still code that depends on the touchpoint
12      +++      // myaccount value is added just in case someone needs it
13      +++      // when in single ssp check if this it's necessary
14      +++      SC.ENVIRONMENT.SCTouchpoint = 'myaccount';
15
16      <% } %>
17
18      ...
19
20      205:      if(!SC.ENVIRONMENT.baseUrl)
21      {
22      ---          SC.ENVIRONMENT.baseUrl = '<%= session.getAbsoluteUrl(request.getURL().indexOf('https:') === 0 ? 'checkout'
23      : 'shopping', '{{file}}') %>';
24      +++          SC.ENVIRONMENT.baseUrl = '<%= session.getAbsoluteUrl('checkout', '{{file}}') %>';
25      }
26
27      ...

```

Add a touchpoint check for ShopFlow and update the session.GetAbsoluteUrl() parameters to replace the legacy domain check.

```

1      ...
2
3      73:      <% if (SiteSettings) { %>
4          // Site Settings Info
5          SC.ENVIRONMENT.siteSettings = <%= JSON.stringify(SiteSettings) %>;
6
7      +++      // SCTouchpoint indicates the touchpoint the user is effectively in. We can only know with certain this in the proper
8      ssp
9      +++      // because there is still code that depends on the touchpoint
10      +++      // when in single ssp check if this it's necessary
11      +++      SC.ENVIRONMENT.SCTouchpoint = 'shopping';
12
13      <% } %>

```

```

14 ...
15
16 115: if (!SC.ENVIRONMENT.baseUrl)
17     {
18     ---      SC.ENVIRONMENT.baseUrl = '<%= session.getAbsoluteUrl(request.getURL().indexOf('https:') === 0 ? 'checkout'
19     : 'shopping', '{{file}})' %>';
20     +++      SC.ENVIRONMENT.baseUrl = '<%= session.getAbsoluteUrl('shopping', '{{file}})' %>';
21     }
22 ...

```

index.ssp and index-local.ssp

File locations:

- ShopFlow > Custom ShopFlow > index.ssp
- MyAccount > Custom MyAccount > index.ssp
- Checkout > Custom Checkout > index.ssp
- ShopFlow > Custom ShopFlow > index-local.ssp
- MyAccount Premium > Custom MyAccount Premium > index-local.ssp

Modify each of these files to add the &X-SC-Touchpoint= query parameter when requesting `sc.environment.ssp` and `sc.user.environment.ssp`.

ShopFlow > Custom ShopFlow > index.ssp

```

1 ...
2
3 146: <script>
4     if (!SC.isCrossOrigin())
5     {
6         // Do we have SEO Support
7         if (SC.isPageGenerator())
8         {
9             document.body.className = document.body.className + ' seo-support';
10        }
11        SC.ENVIRONMENT.seoSupport = !!~document.body.className.indexOf("seo-support");
12        /* load language and sc.environment.ssp */
13        loadScript([
14        ---      '<%= session.getAbsoluteUrl("shopping", "sc.environment.ssp?lang=" + Language + "&cur=" + Currency) %>'
15        +++      '<%= session.getAbsoluteUrl("shopping", "sc.environment.ssp?lang=" + Language + "&cur=" + Currency + "&X-SC-
16        Touchpoint=shopping)" %>'
17        , '<%= session.getAbsoluteUrl("shopping", "languages/" + Language + ".js") %>'
18        ]);
19        if (SC.isPageGenerator())
20        {
21            SC.ENVIRONMENT.PROFILE = {};
22        }
23        // Loads the application files, if you need to have a less agresive cacheing you can move them
24        // to the sc.environment.ssp (Moderate cacheing) or to the sc.user.environment.ssp (No cache but less performant)
25        loadScript([
26        ---      '<%= session.getAbsoluteUrl("shopping", 'js/libs/Libraries-014c760ca5c2.js') %>'
27        , '<%= session.getAbsoluteUrl("shopping", 'templates/Templates-0152658bf631.js') %>'
28        , '<%= session.getAbsoluteUrl("shopping", 'js/Application-01526a8e6d58.js') %>'
29        // , ' /cms/1/cms.js'
30        ]);
31        if (SC.ENVIRONMENT.jsEnvironment == 'browser')
32        {
33            loadScript({
34            ---      url: '<%= session.getAbsoluteUrl("shopping", "sc.user.environment.ssp?lang=" + Language + "&cur=" +
35            Currency %>&t=' + new Date().getTime()
36            +++      url: '<%= session.getAbsoluteUrl("shopping", "sc.user.environment.ssp?lang=" + Language + "&cur=" +
37            Currency + "&X-SC-Touchpoint=shopping)" %>&t=' + new Date().getTime()
38            , async: true
39            });

```

```

37     }
38   }
39   </script>
40
41   ...

```

MyAccount > Custom MyAccount > index.ssp

```

1   ...
2
3   <script>
4   var SC = window.SC = {
5     ENVIRONMENT: {
6       jsEnvironment: (typeof nsglobal === 'undefined') ? 'browser' : 'server'
7     }
8     , isCrossOrigin: function() { return '<%= Environment.currentHostString %>' !== document.location.hostname; }
9     , isPageGenerator: function() { return typeof nsglobal !== 'undefined'; }
10    , getSessionInfo: function(key)
11    {
12      var session = SC.SESSION || SC.DEFAULT_SESSION || {};
13      return (key) ? session[key] : session;
14    }
15  };
16  </script>
17  65: --- <script> src="<%= session.getAbsoluteUrl('checkout', 'sc.environment.ssp?lang=' + Language + '&cur=' + Currency) %>"</
18  <script>
19  65: +++ <script> src="<%= session.getAbsoluteUrl('checkout', 'sc.environment.ssp?lang=' + Language + '&cur=' + Currency + "&X-
20  SC-Touchpoint=myaccount") %>"</script>
21
22  ...

```

Checkout > Custom Checkout > index.ssp

```

1   ...
2
3   139: <% if (login) { %>
4   --- <script src="<%= session.getAbsoluteUrl('checkout', 'sc.environment.ssp?lang=' + Language + '&cur=' + Currency +
5   (cart_bootstrap ? "&cart-bootstrap=T" : "") ) %>"></script>
6   +++ <script src="<%= session.getAbsoluteUrl('checkout', 'sc.environment.ssp?lang=' + Language + '&cur=' + Currency +
7   (cart_bootstrap ? "&cart-bootstrap=T" : "") + "&X-SC-Touchpoint=checkout" ) %>"></script>
8   <% } else { %>
9   <script>
10  loadScript({
11    url: '<%= session.getAbsoluteUrl("checkout", "sc.environment.ssp?lang=" + Language + "&cur=" + Currency +
12    (cart_bootstrap ? "&cart-bootstrap=T" : "") ) %>&t=' + (new Date().getTime())
13  +++ url: '<%= session.getAbsoluteUrl("checkout", "sc.environment.ssp?lang=" + Language + "&cur=" + Currency +
14    (cart_bootstrap ? "&cart-bootstrap=T" : "") + "&X-SC-Touchpoint=checkout" ) %>&t=' + (new Date().getTime())
15    });
16  </script>
17  <% } %>
18
19  ...

```

ShopFlow > Custom ShopFlow > index-local.ssp

```

1   ...
2
3   80: /* load language and sc.environment.ssp */
4   loadScript([
5   --- '<%= session.getAbsoluteUrl("shopping", "sc.environment.ssp?v=711&lang=" + Language + "&cur=" + Currency) %>'
6   +++ '<%= session.getAbsoluteUrl("shopping", "sc.environment.ssp?v=711&lang=" + Language + "&cur=" + Currency + "&X-SC-Touch
7   point=shopping") %>'
8   , '<%= session.getAbsoluteUrl("shopping", "languages/" + Language + ".js") %>'
9   ]);
10  if (SC.ENVIRONMENT.jsEnvironment === 'server')
11  {
12    SC.ENVIRONMENT.PROFILE = {};
13  }

```

```

13         loadScript([
14             '<%= root %>templates/Templates.php'
15             , '<%= root %>js/Application.php'
16         ]);
17
18         if (SC.ENVIRONMENT.jsEnvironment == 'browser')
19         {
20             loadScript({
21 ---         url: '<%= session.getAbsoluteUrl("shopping", "sc.user.environment.ssp?lang=" + Language + "&cur=" + Currency)
22             %>&t=' + new Date().getTime()
23 +++         url: '<%= session.getAbsoluteUrl("shopping", "sc.user.environment.ssp?lang=" + Language + "&cur=" + Currency + "&X-
24             SC-Touchpoint=shopping") %>&t=' + new Date().getTime()
25             , async: true
26             });
27     }

```

MyAccount Premium> Custom MyAccount Premium > index-local.ssp

```

1    ...
2
3        <script src="<%= root %>js/Utils/BootUtilities.js"></script>
4 70: ---    <script src="<%= session.getAbsoluteUrl('checkout', 'sc.environment.ssp?lang=' + Language + '&cur=' + Currency) %>"></
5          script>
6          +++    <script src="<%= session.getAbsoluteUrl('checkout', 'sc.environment.ssp?lang=' + Language + '&cur=' + Currency + '&X-
7              SC-Touchpoint=myaccount') %>"></script>
8
9    ...

```

jQuery.ajaxSetup.js

File locations:

- ShopFlow > Custom ShopFlow > js > src > core > extras > jQuery.ajaxSetup.js
- MyAccount > Custom MyAccount > js > src > core > extras > jQuery.ajaxSetup.js
- Checkout > Custom Checkout > js > src > core > extras > jQuery.ajaxSetup.js

Update the `beforeSend()` method to send the header to the backend for each request.

```

1    ...
2
3    // This registers an event listener to any ajax call
4 85:    jQuery(document)
5        // http://api.jquery.com/ajaxStart/
6        .ajaxStart(SC.LoadingIndicatorShow)
7        // http://api.jquery.com/ajaxStop/
8        .ajaxStop(SC.LoadingIndicatorHide);
9        // http://api.jquery.com/jQuery.ajaxSetup/
10
11    jQuery.ajaxSetup({
12 93:    beforeSend: function (jqXHR, options)
13        {
14        // BTW: "!~" means "==" -1"
15        if (!options.contentType.indexOf('charset'))
16        {
17            // If there's no charset, we set it to UTF-8
18            jqXHR.setRequestHeader('Content-Type', options.contentType + '; charset=UTF-8');
19        }
20
21        +++    // Add header so that suitescript code can know the current touchpoint
22        +++    jqXHR.setRequestHeader('X-SC-Touchpoint', SC.ENVIRONMENT.SCTouchpoint);
23        }
24
25    ...

```

live-order.ss and live-order-line.ss

File locations:

- ShopFlow > Custom ShopFlow > services > live-order.ss
- MyAccount > Custom MyAccount > > live-order.ss
- Checkout > Custom Checkout > services > live-order.ss
- ShopFlow > Custom ShopFlow > services > live-order-line.ss
- MyAccount > Custom MyAccount > services > live-order-line.ss
- Checkout > Custom Checkout > services > live-order-line.ss

In each file, add an if statement to verify if the user is in shopping or logged in.

```

1 | ...
2 |
3 |     // If we are not in the checkout OR we are logged in
4 |     // When on store, login is not required
5 |     // When on checkout, login is required
6 | --- if (!request.getURL().indexOf('https') || session.isLoggedIn())
7 | +++ if (Utils.isInShopping(request) || session.isLoggedIn2())
8 |
9 | ...

```

CheckoutSkipLogin.js

Customizing this file requires that you copy the **CheckoutSkipLogin** module folder from Reference Checkout to Custom Checkout because it is not created automatically.

Update Checkout > Custom Checkout > js > src > app > modules > CheckoutSkipLogin > CheckoutSkipLogin.js to use the new domain check method.

```

1 |         return promise;
2 |     };
3 |
4 |     --- // don't wrap on non-secure domains (Site Builder cart is in Checkout :)
5 | 89: --- if (window.location.protocol !== 'http:')
6 |     +++ // Site Builder cart is in Checkout :/ don't wrap if in shopping
7 |     +++ if (Utils.isInCheckout())
8 |     {
9 |         LiveOrderModel.prototype.save = _wrap(LiveOrderModel.prototype.save, wrapper);
10 |     }
11 |
12 | ...

```

NavigationHelper.js

Customizing this file requires that you copy the **NavigationHelper** module folder from Reference ShopFlow to Custom Shopflow because it is not created automatically.

File locations:

- ShopFlow > Custom ShopFlow > js > src > app > modules > NavigationHelper > NavigationHelper.js
- MyAccount > Custom MyAccount > js > src > app > modules > NavigationHelper > NavigationHelper.js
- Checkout > Custom Checkout > js > src > app > modules > NavigationHelper > NavigationHelper.js

Add the new domain check methods in each of the files.

```

1  ...
2  // if we are heading to a secure domain (myAccount or checkout), keep setting the language by url
299: --- if (target_touchpoint.indexOf('https:') >= 0)
    +++ if (Utils.isInCheckout())
3  {
4      var current_language = SC.ENVIRONMENT.currentLanguage;
5      if (current_language)
6      {
7          target_data.parameters = target_data.parameters ?
8              target_data.parameters + '&lang=' + current_language.locale :
9              'lang=' + current_language.locale;
10     }
11 }
12
13
14
15 ...

```

Update Legacy Checks and Methods

Previous sections have covered customizing multiple files. This procedure provides steps for finding and replacing any other instances of the legacy domain checks, the legacy `getAbsoluteUrl()` methods, and the legacy `session.isLoggedin()` methods.

To update the legacy checks and methods:

1. Search the files for remaining instances of these legacy domain checks.
 - `!~request.getURL().indexOf('https')`
 - `isSecure`
 - `isSecureDomain`
 - a. If instances are located in files that have not already been copied to the Custom folder, copy as needed.
 - b. For each instance, determine the reason for the domain check.
 - Was it to protect a resource from being accessed through the shopping domain? In this case, call the `Utils.isCheckoutDomain()` method.
 - Does the method return a different result depending on whether we are in the checkout domain or the shopping domain? In this case, call the `Utils.isInCheckout()` method.
 - Is there a visual change in the web store depending on whether we are in the checkout domain or the shopping domain? Call the `Utils.isInCheckout()` method or the `Utils.isInShopping()` method as appropriate for each domain.
 - c. Replace each instance with the appropriate new domain check method.
See the code snippet in [commons.js](#) for the method definitions.
 - `Utils.isCheckoutDomain()`
 - `Utils.isInCheckout()`
 - `Utils.isInShopping()`
 - `Utils.isSingleDomain()`
2. Search the files for remaining instances of:


```
session.getAbsoluteUrl(request.getURL().indexOf('https:') === 0 ? 'checkout' : 'shopping',
'/{file}')
```

 - a. If instances are located in files that have not already been copied to the Custom folder, copy as needed.
 - b. In Checkout or MyAccount, replace the instance with `session.getAbsoluteUrl( 'checkout', '/{file}')`.

- c. In Shopping, replace the instance with `session.getAbsoluteUrl( 'shopping', '{{file}}')`.
- d. You can also replace any instance with:

```
session.getAbsoluteUrl(Utils.isInCheckout(request) ? 'checkout' : 'shopping', '{{file}}')
```

This method call will check for either domain.
- 3. Search the files for remaining instances of the `session.isLoggedIn()` method.
 - a. If instances are located in files that have not already been copied to the Custom folder, copy as needed.
 - b. Replace each instance with the `session.isLoggedIn2()` method.

Deploy the Migration

When all customizations are complete, you need to configure NetSuite to use the custom files instead of the reference files. Complete the following tasks:

- Configure the secure domain.
- Add or update library scripts for each customized SSP Application.
- Add or update touch points for each customized SSP Application.
- Deploy your site to the secure domain.



Important: These customizations are extensive. You should test them in a sandbox account or other test site before deploying to a production account.

To deploy the migration:

1. If you have not already done so, complete the steps to configure a Secure Domain. For details, refer to [Set Up Domains for Web Stores](#).
2. Go to Commerce > Hosting > SSP Applications.
3. Click **Edit** to modify the custom SSP Application.
4. On the **Library** subtab under **Scripts**, add the library scripts as indicated:



Important: The order of these scripts must match exactly.

Web Site Hosting Files/Live Hosting Files/SSP Applications/<application>/...

- ... Reference <SSP Application>/ssp_libraries/underscore.js
- ... Reference <SSP Application>/ssp_libraries/backbone.validation.js
- ... Custom <SSP Application>/ssp_libraries/commons.js
- ... Reference <SSP Application>/ssp_libraries/backend.configuration.js
- ... Custom <SSP Application>/ssp_libraries/Models.js

For example, to update the commons.js file for ShopFlow 1.07.0, go to:

Web Site Hosting Files/Live Hosting Files/SSP Applications/NetSuite Inc. — ShopFlow 1.07.0/Custom ShopFlow /ssp_libraries/commons.js

5. Click on the **Supported Touch Points** subtab.
6. Add the appropriate touch points using the custom .ssp files that were modified as part of this migration. See the help topic [Select Supported Touch Points](#).

7. Click **Save**.
8. Deploy the Custom SSP Applications to your Secure Domain.
 - a. Select the **Save & Link to Domain** button (from the **Save** drop down menu).
 - b. Choose your secure domain.
 - c. Click **Save**.
9. Repeat the Scripts, Touch Points, and Deploy steps for each SSP Application you customized (Custom Checkout, Custom MyAccount, and Custom ShopFlow).

Checkout With CDN

Perform the following tasks to use CDN on the checkout flow for pre-Denali versions of SCA:

1. Copy the following files from the Reference Checkout folder to the Custom Checkout folder:
 - a. `index.ssp`
 - b. `cart.ssp`
 - c. `sc.environment.ssp`
2. Rename `sc.environment.ssp` to `checkout.environment.ssp`.
3. Search the following files for all instances of "`sc.environment.ssp`" and replace them with "`checkout.environment.ssp`":
 - a. `index.ssp`
 - b. `cart.ssp`
4. Modify the Custom Checkout SSP Application to point to the following files which you previously copied to the Custom Checkout folder. See [Deploy the Migration](#).
5. Modify the domain to point to the Custom Checkout SSP Application. See [Deploy the Migration](#).

PayPal Address not Retained in Customer Record

Applies to: SuiteCommerce Advanced | Vinson | Mont Blanc | Denali

For Vinson releases and earlier, when a PayPal address is submitted during a web store transaction, the NetSuite customer address is not updated to reflect the PayPal address. Instead, the original customer address within the NetSuite customer record is retained. Apply the patch described in this section to retain the PayPal address in the NetSuite customer record.



Important: Making changes to core JavaScript source files or changing any vital functionality of the application can make migrating to future releases difficult. Before making changes to SuiteCommerce Advanced, see the help topic [Develop Your SCA Customization](#).

Step 1: Create the `LiveOrder.Model.js` Override file

1. If you have not done so already, create a directory to store your custom module.
2. Open this directory and create a subdirectory to maintain your customizations.
Give this directory a name similar to the module being customized. For example:
`Modules/extensions/Live.Order.Override@1.0.0`
3. In the `Live.Order.Override@1.0.0` directory, create a subdirectory called `SuiteScript`.

4. Go to the SuiteScript directory located under the Modules > suitecommerce > LiveOrder@X.X.X directory.
5. Copy the LiveOrder.Model.js file from Modules > suitecommerce > LiveOrder@X.X.X into the Live.Order.Override@1.0.0 > SuiteScript directory.
6. Update the file to remove the following line from the submit() method.

```
1 | this.removePaypalAddress(paypal_address);
```

Note: This customization can also be applied to pre-Denali implementations of SuiteCommerce Advanced. Remove the line of code from the submit method located at ssp_libraries/Models.js

Step 2: Prepare the Developer Tools for Your Override

1. Create the ns.package.json file for the LiveOrderOverride@X.X.X directory. Add the following code:

```
1 | {
2 |   "gulp": {
3 |     "ssp-libraries":
4 |       "SuiteScript/*.js"
5 |   },
6 |   "overrides": {
7 |     "suitecommerce/LiveOrder@X.X.X/SuiteScript/LiveOrder.Model.js" :
8 |       SuiteScript/LiveOrder.Model.js"
9 |   }
}
```

2. Update the distro.json file.

```
1 | },
2 |   "modules": {
3 |     ...
4 |     "suitecommerce/LiveOrder": "X.X.X",
5 |     "extensions/Live.Order.Override": "X.X.X",
```

Step 3: Test and Deploy Your Override

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)).

Since this customization modifies a file that is stored as an SSP library, changes are not immediately visible in your local environment. You must first deploy your custom module directly to NetSuite. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

2. Confirm your results.

Login Email Address Appears in the Password Reset URL

Applies to: SuiteCommerce Advanced | Elbrus

This patch adds a method named `sendPasswordRetrievalEmail2()`, included in Kilimanjaro and later implementations of SuiteCommerce Advanced. Located in the Commerce API, `sendPasswordRetrievalEmail2()` generates a password reset email message. For added security, the original login email address for the customer does not appear in the password reset URL generated by this method. Other ecommerce solutions commonly use this secure solution.

To implement this patch, you extend JavaScript functions in the Account and LoginRegister modules and override the `login_register_reset_password.tpl` template file. For an example of the changes needed for this patch, see [EmailAddressPasswordResetURL.zip](#).

This method replaces the `sendPasswordRetrievalEmail()` method. However, the deprecated `sendPasswordRetrievalEmail()` method will continue to operate without change.

Note: Before proceeding, familiarize yourself with [Best Practices for Customizing SCA](#). The following sections show how to [Extend JavaScript](#) to implement the patch.

To implement this patch, follow each step to extend the functions in the `Account.Model.js` and `LoginRegister.ResetPassword.View.js` files, and override the `login_register_reset_password.tpl` template file.

Step 1: Create and Copy the Required Files

1. To extend the functions that require a code change for this patch, create a directory to store your custom modules, for example, create `Modules/extensions`.
2. Open this directory and create the following subdirectories to maintain your customizations.
Give this directory a name similar to the module being customized. For example, create the following directories:

`Modules/extensions/AccountExtension@1.0.0`

`Modules/extensions/LoginRegisterExtension@1.0.0`

3. In the `AccountExtension@X.X.X` directory, create a `SuiteScript` subdirectory. In the `SuiteScript` subdirectory, create a JavaScript file.

To follow best practices, name the JavaScript file `Account.Model.Extension.js`.

4. Open this file and extend the `forgotPassword` method as shown in the following code snippet:

```

1  define(
2    'Account.Model.Extension'
3    , [
4      'SC.Model'
5      , 'Application'
6      , 'Models.Init'
7
8      , 'underscore'
9    ]
10   , function (
11     SCModel
12     , Application
13     , ModelsInit
14
15     , -
16   )
17   {
18     'use strict';
19
20     _._extend(AccountModelExtension.prototype,
21     {
22       forgotPassword: function (email)
23       {
24         try
25         {
26           // this API method throws an exception if the email doesn't exist
27           // 'The supplied email has not been registered as a customer at our Web store.'
28           ModelsInit.session.sendPasswordRetrievalEmail2(email);
29         }
30         catch (e)
31         {
32           var error = Application.processError(e);

```

```

33         // if the customer failed to log in previously
34         // the password retrieval email is sent but an error is thrown
35         if (error.errorCode !== 'ERR_WS_CUSTOMER_LOGIN')
36         {
37             throw e;
38         }
39     }
40
41     return {
42         success: true
43     };
44 }
45
46 });
47 });

```

5. In the LoginRegisterExtension@X.X.X directory, create a JavaScript subdirectory. In the JavaScript subdirectory, create a JavaScript file named LoginRegister.ResetPassword.View.Extension.js.
6. Open this file and extend the function name method as shown in the following code snippet:

```

1  define('LoginRegister.ResetPassword.View.Extension'
2  , [ 'SC.Configuration'
3    , 'Account.ResetPassword.Model'
4    , 'Backbone.FormView'
5
6    , 'Backbone'
7    , 'underscore'
8  ]
9  , function (
10     Configuration
11     , AccountResetPasswordModel
12     , BackboneFormView
13
14     , Backbone
15     , _
16 )
17 {
18     'use strict';
19
20     _.extend(LoginRegisterResetPasswordViewExtension.prototype,
21     {
22         initialize: function ()
23         {
24             this.model = new AccountResetPasswordModel();
25             this.model.set('params', {'cb':_.parseUrlOptions(location.search).cb});
26             this.model.on('save', _.bind(this.showSuccess, this));
27
28             BackboneFormView.add(this);
29         }
30     });
31 });

```

7. In the LoginRegisterExtension@X.X.X directory, create a Templates subdirectory. Copy the Modules/suitecommerce/LoginRegister@2.3.0/Templates/login_register_reset_password.tpl template file into the Templates directory.
8. Open login_register_reset_password.tpl and make the following change.

Replace this HTML:

```

1  <p class="login-register-reset-password-description">
2    {{translate 'Enter a new password below for <b>${0}</b>' email}}
3  </p>

```

With this HTML:

```

1  <p class="login-register-reset-password-description">

```

```

2 | {{translate 'Enter a new password below'}}
3 | </p>

```

Step 2. Prepare the Developer Tools for Your Overrides

1. Create the `ns.package.json` file for the `AccountExtension@X.X.X` directory. Add the following code to `ns.package.json` in the `Modules/extensions/Account@X.X.X` directory:

```

1 | {
2 |   "gulp": {
3 |     "ssp-libraries":
4 |       "SuiteScript/*.js"
5 |   },
6 | }

```

2. Create the `ns.package.json` file for the `LoginRegisterExtension@X.X.X` directory. Add the following code to `ns.package.json` in the `Modules/extensions/LoginRegister@X.X.X` directory:

```

1 | {
2 |   "gulp": {
3 |     "javascript":
4 |       "JavaScript/*.js"
5 |     "templates": [
6 |       "JavaScript/*.js"
7 |     ]
8 |   },
9 |   "overrides": {
10 |     "suitecommerce/LoginRegister@X.X.X/Templates/login_register_reset_password.tpl" :
11 |       Templates/login_register_reset_password.tpl
12 |   }

```

3. In `distro.json`, add your custom modules to the `modules` object.

This ensures that the Gulp tasks include your extension when you deploy. In this example, the extension modules are added at the beginning of the list of modules. However, you can add the modules anywhere in the `modules` object. The order of precedence in this list does not matter.

```

1 | {
2 |   "name": "SuiteCommerce Advanced Elbrus",
3 |   "version": "2.0",
4 |   "buildToolsVersion": "1.3.0",
5 |   "folders": {
6 |     "modules": "Modules",
7 |     "suitecommerceModules": "Modules/suitecommerce",
8 |     "extensionsModules": "Modules/extensions",
9 |     "thirdPartyModules": "Modules/third_parties",
10 |    "distribution": "LocalDistribution",
11 |    "deploy": "DeployDistribution"
12 |   },
13 |   "modules": {
14 |     "extensions/AccountExtension": "X.X.X",
15 |     "extensions/LoginRegisterExtension": "X.X.X",
16 |     ...

```

Step 3. Test and Deploy Your Override

1. Test your source code customizations on a local server (see the help topic [Test SCA Customizations on a Local Server](#)) or deploy them to your NetSuite account (see the help topic [Deploy SCA Customizations to NetSuite](#)).

Since this customization modifies a file that is stored as an SSP library, changes are not immediately visible in your local environment. You must first deploy your custom module directly to NetSuite. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

2. Confirm your results.

How to Apply .patch Files

Some releases of SuiteCommerce Advanced include a series of .patch files, which contain the complete diffs between new and previous releases across all source files. In some cases, a patch might be available for specific purposes, such as for backporting code to previous releases to support new features. If you encounter a .patch file, refer to this section to learn how to interpret and apply it.

A patch file contains the differences between two identically titled files. The patch designates these folders as **a** and **b**. Distributing the patch to an identically titled path/folder allows the patch to transform changes in **b** (the source of the changes) to the matching file in **a** (the target of the changes).

Follow the appropriate procedure to apply the diff, depending on how you customized the source code in previous releases.

- If you have not altered any SuiteCommerce Advanced source files in Modules/suitecommerce/, you can use third-party software to run the patch.
- If you have modified any source files in Modules/suitecommerce/, you must implement changes manually.



Important: Applying a diff adds and removes lines of code. This can break your site if you are not careful. Do not apply any diffs using third-party software unless you have followed NetSuite's best practices for customizing JavaScript files as outlined in [Best Practices for Customizing SCA](#). Do not patch your live site directly. Create a test site or test files locally using the dev tools. Failure to test any changes thoroughly before deploying to your live site can result in data loss.

For a list of available patches, see [SCA Patches](#).

To apply a diff using third-party software:

1. Download the correct diff file to the target directory.
2. Make a backup of your original source code.
3. Review the diff as applicable. See the help topic [Diff File Structure](#) for details.
4. Ensure that you have not altered any source code provided by NetSuite and apply the diff using your preferred third-party method.

Various programs are available that can apply a diff file to your SCA source code. The following example applies a Mont Blanc release diff using Git.

```
cd montblanc
git init
git apply /path/montblanc-sensors.patch
rm -rf .git
```

To apply a diff manually:

1. Download the correct diff file.
2. Open the diff.
3. Make a backup of your local source code.
4. Make all changes manually in your local source code. For a detailed description of a diff file and its components, see the help topic [Diff File Structure](#)

Diff File Structure

This section explains how SuiteCommerce Advanced diff files are structured. The information in this section should help you understand any diff files provided by NetSuite for SuiteCommerce Advanced.

Note: Surrounding code and estimated line numbers vary depending on each file due to existing customizations. These instructions include them here as reference points only.

Example Diff File

The following example shows a simple diff (.patch file). This example only compares one file, although most diffs include more than one file. One diff file typically contains many comparisons, each with its own components, as described in this section.

```

1 | diff --git a/myDirectoryPath/myFile.js b/myDirectoryPath/myFile.js
2 | index 5d7cd7e..fca4091 100755
3 | --- a/myDirectoryPath/myFile.js
4 | +++ b/myDirectoryPath/myFile.js
5 | @@ -76,7 +77,7 @@ define('Sensors'
6 |
7 |     {
8 |         _.each(arguments, function (argument)
9 |         {
10 |             if(argument)
11 |             +   if (argument)
12 |                 {
13 |                     _.extend(data, argument);
14 |                 }

```

File Comparison Header

The File Comparison Header is a statement explaining two files being compared. The two files being compared include leading directory names **a** and **b** to distinguish the two files, which typically have identical names and locations.

This example shows changes between two versions of myFile.js. Note the file names and paths are identical. --git simply informs you that this diff is written in Git-specific diff format. Each comparison within the diff begins with a header.

```
diff --git a/myDirectoryPath/myFile.js b/myDirectoryPath/myFile.js
```

Index

Technically part of the Header, the index is a line of metadata describing technical information about the given files. The first two numbers (separated by . .) represent the IDs of the two files being compared. In this case, each version of the project file is an object at a specific revision.

The last number is a mode identifier for the file containing the changes:

- 100644 specifies a normal, non-executable file.
- 100755 specifies an executable file.
- 120000 specifies a symbolic link.

In this example, this part of the diff is comparing two different file IDs (5d7cd7e and fca4091) of myFile.js, which is an executable file.

```
index 5d7cd7e..fca4091 100755
```

Change Markers

The next two lines describe the two files (and paths) being compared, but with `+++` or `---` prefixes. The file with the minus symbols shows lines that exist within the **a** version but are missing from the **b** version. Likewise, the file with the plus symbols contains lines missing in the **a** version but are present in **b**.

In this example, **a** is the existing (old) file and **b** is the updated (new) file.

```
1 | --- a/myDirectoryPath/myFile.js
2 | +++ b/myDirectoryPath/myFile.js
```

Range Information

Diff files show only the portions of the file being changed. These portions are called hunks. One hunk of data containing changes is depicted within two sets of `@@` symbols. This information declares the lines within the hunk that are affected by the change. One index can contain multiple hunks, depicting the various changes throughout the file.

In this example, the diff contains one hunk of differences. Seven lines are extracted from the **a** file, beginning at line 76. Likewise, seven lines are displayed in the **b** file, beginning at line 77.

```
1 | @@ -76,7 +77,7 @@
```

Changes

The changes to each hunk immediately follow the range information. Each changed line is prepended with either a `+` or a `-` symbol. These symbols act in the same way as they do in the Change Markers section. The diff removes lines prepended with a `-` and adds lines prepended with a `+` sign. If making changes to your code manually, any lines bearing the `-` prefix need to be removed. Any lines bearing the `+` prefix need to be added.

Additionally, a hunk typically contains unchanged lines before and after the modification to provide context for the change. In this example, the diff removes `if (argument)` and adds the corrected code: `if (argument)`.

```
1 | define('Sensors'
2 |
3 |   {
4 |     _.each(arguments, function (argument)
5 |     {
6 | -     if(argument)
7 | +     if (argument)
8 |       {
9 |         _.extend(data, argument);
10 |      }
```



Note: In diffs provided by NetSuite, **a** files and lines marked with `-` are old, and **b** files and lines marked with `+` are updates.

Migrate SuiteCommerce Advanced

To ensure that you implement the latest features, bug fixes, and changes to SuiteCommerce Advanced (SCA), install the latest major or minor release of the active version, whichever is highest. See the help topic [SuiteCommerce, SuiteCommerce MyAccount, and SuiteCommerce Advanced Release Notes](#) for the list of changes and bundle ID of the latest version.

Because SuiteCommerce Advanced SuiteApps are **unmanaged** bundles, updates are not automatically pushed to your site. This structure is in place to ensure that existing customizations are not impacted unexpectedly with each update. Therefore, you must manually update the bundle to the latest major or minor release to receive the latest changes.

The correct procedures to follow depend on the current release of your SCA site. Follow the correct procedures, described below:

- [Migrate from Aconcagua and Later](#)
- [Migrate From Kilimanjaro and Earlier](#)



Important: Performing a bundle update causes all files in the existing SCA Source directory to be replaced with the updated set of files. The new source file changes and your existing customizations must be re-deployed from your local environment before changes take effect on a domain pointing to the Development SSP application. Even though both major and minor releases maintain the same Bundle ID, you must still re-link your site record and domain to the updated SSP application.

These topics also apply to administrators testing a new major or minor release by deploying to a development SSP within a site record.

Migrate from Aconcagua and Later

The Aconcagua release of SCA and later all support themes and extensions and the Extension Framework (also referred to as the Extensibility API). Follow the instructions in this section if you are currently implementing the Aconcagua release of SCA or higher.



Important: If you have modified any core SCA files in your existing implementation, you should recreate these changes as extensions instead of migrating your customizations manually. If your existing customizations modify objects not accessible through the Extensibility API, you must migrate this custom code manually to the new version. See the help topic [Overview](#) for more information.



Important: Before migrating to a new release, review the [SuiteCommerce, SuiteCommerce MyAccount, and SuiteCommerce Advanced Release Notes](#) for important notifications and details regarding changes included in the release. Each major release might include bug fixes and changes in the form of periodic minor releases. Make sure you implement the latest minor release of the active version.

To migrate to the latest release:

1. Install the new bundle.

1. Go to Customization > SuiteBundler > Search & Install Bundles.
2. Enter the bundle ID for the latest version of SCA you want to install. Bundle IDs are listed in the [SuiteCommerce, SuiteCommerce MyAccount, and SuiteCommerce Advanced Release Notes](#).
3. Select the bundle in the results list and then click **Install**. Follow any on-screen prompts to continue the install.

The bundle installation may take a while. A pending message displays in the status column of the Installed Bundles page. Refresh the screen during the process to see when the installation is complete.

 **Note:** Minor releases to the current SCA release do not receive new Bundle IDs. See the help topic [Choosing a Bundle to Update](#) for instructions on updating an installed bundle.

2. Download the new source code from the file cabinet to your local environment.

The source zip file is located at Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. — SCA [version] > Source > _Sources. After downloading is complete, unzip the file into a new working directory.

3. All customizations should be implemented using themes and extensions. However, if your existing implementation includes customizations not accessible using the Extensibility API, migrate these customizations to your new source code directory.
 - a. Create a duplicate of your custom module folder in the new source code directory.

 **Note:** This applies to custom modules that are not standard SCA source modules. Any source SCA modules that have been customized may require comparison against the new version plus any required manual edits to the code.

- b. Copy your existing custom modules into the new custom module folder.
 - c. Update the new distro.json file to include references to any custom modules and to define any custom application dependencies.

 **Important:** Do not copy the existing distro.json file contents directly into the new distro.json file. Doing this causes overwrites to changes made in the distro.json file to support the new release. For example, a new module may be referenced. NetSuite recommends evaluating the DIFF between files to ensure that you migrate all changes from your local distro.json file into the new distro.json file.

- d. Any module that changed in the new release bears a higher version number from its previous release. Compare module version numbers to assess which modules have been updated and migrate the changes to any existing customizations as necessary. Refer to the [SuiteCommerce, SuiteCommerce MyAccount, and SuiteCommerce Advanced Release Notes](#) for details on which modules were changed and why.

 **Important:** Migrating to a new release requires testing to ensure any existing customizations remain viable. Evaluate differences in code against your existing customizations and test thoroughly before deploying to your live site. See the help topic [Best Practices for Customizing SCA](#).

4. Link your site record and domain to an SSP application as required. This can be the SSP Application for the latest version of SCA or one specifically intended for development.
 - a. Go to Commerce > Hosting > SSP Applications and click **View** next to the SuiteCommerce Advanced Dev <version> application.

- b. Click **Link to Domain**.
 - c. Select the development domain for this SSP Application.
- 5. Examine all customizations to your existing source code of the previous implementation. If all customizations of your previous implementation were accomplished using themes and extensions, then reactivate them using the Extension Manager in NetSuite. See the help topic [Manage Themes and Extensions](#) for details.
- 6. If your previous implementation included customizations of SCA source code that were not part of the Extensibility API (implemented without extensions), then complete the following:
 - a. Create a duplicate of your custom module folder in the new source code directory.



Note: This applies to custom modules that are not standard SCA source modules. Any source SCA modules that have been customized may require comparison against the new version plus any required manual edits to the code.

- b. Copy your existing custom modules into the new custom module folder.
 - c. Update the new `distro.json` file to include references to any custom modules and to define any custom application dependencies.



Important: Do not copy the existing `distro.json` file contents directly into the new `distro.json` file. Doing this causes overwrites to changes made in the `distro.json` file to support the new release. For example, a new module may be referenced. NetSuite recommends evaluating the DIFF between files to ensure that you migrate all changes from your local `distro.json` file into the new `distro.json` file.

- d. Any module that changed in the new release bears a higher version number from its previous release. Compare module version numbers to assess which modules have been updated and migrate the changes to any existing customizations as necessary. Refer to the [SuiteCommerce, SuiteCommerce MyAccount, and SuiteCommerce Advanced Release Notes](#) for details on which modules were changed and why.



Important: Migrating to a new release requires testing to ensure any existing customizations remain viable. Evaluate differences in code against your existing customizations and test thoroughly before deploying to your live site. See the help topic [Best Practices for Customizing SCA](#).

- e. Deploy your local files and test appropriately.

Testing may consist of various scenarios depending on your setup. You can deploy your local files to any of the following locations:

- Core source code customizations require the core SCA developer tools. This requires setting up the appropriate developer environment. See [Set Up SCA Developer Tools](#) for details. Core SCA customizations can be deployed to:
 - A primary SSP Application linked to a primary domain (live site)
 - Development SSP Application linked to a development domain
 - Local server. See the help topic [Test SCA Customizations on a Local Server](#).
 - NetSuite sandbox account. See [Deploy to a NetSuite Sandbox](#).
 - Custom SSP Application. See [Deploy to a Custom SSP Application](#)

For example, you can deploy your customizations to a local server or to a NetSuite sandbox account for testing. If you do not have a Sandbox account, you can deploy to an SSP Application linked to a development domain in NetSuite. After testing, deploy your files to the SSP Application linked to the primary domain.

Migrate From Kilimanjaro and Earlier

Follow the instructions in this section if you are currently implementing any of the following releases of SCA. Migrating from these implementations requires adaptation of any current customizations into themes and extensions:

- Kilimanjaro
- Elbrus
- Vinson
- Mont Blanc
- Denali



Important: The best practice to customize SCA is through themes and extensions. Earlier versions of SCA (prior to Aconcagua) do not support themes and extensions, which do not require modification of source code. If you have modified any core SCA files in your existing implementation, best practice is to re-create these changes as themes or extensions. If your customizations modify objects not accessible through the Extensibility API, you must migrate this custom code manually to the new version. See the help topic [Overview](#) for more information.

To migrate to the latest release:

1. Install the new bundle.



Important: Before migrating to a new release, review the [SuiteCommerce](#), [SuiteCommerce MyAccount](#), and [SuiteCommerce Advanced Release Notes](#) for important notifications and details regarding changes included in the release. Each major release might include bug fixes and changes in the form of periodic minor releases. Make sure you implement the latest minor release of the active version.

1. Go to Customization > SuiteBundler > Search & Install Bundles.
2. Enter the bundle ID for the latest version of SCA. Bundle IDs are listed in the [SuiteCommerce](#), [SuiteCommerce MyAccount](#), and [SuiteCommerce Advanced Release Notes](#).
3. Select the bundle in the results list and then click **Install**. Follow any on-screen prompts to continue the install.

The bundle installation may take a while. A pending message displays in the status column of the Installed Bundles page. Refresh the screen during the process to see when the installation is complete.

2. Download the new source code from the file cabinet to your local environment.

The source zip file is located at Web Site Hosting Files > Live Hosting Files > SSP Applications > NetSuite Inc. — SCA [version] > Source > _Sources. After downloading is complete, unzip the file into a new working directory.

3. Examine all customizations to your existing source code of the previous implementation.



Important: The latest version of SuiteCommerce Advanced does not include Sass or HTML source and also utilizes the Extensibility API to customize most JavaScript, SuiteScript, and JSON. You must recreate existing customizations as themes or extensions to carry them over to your new implementation of SCA.

- a. Create one or more themes to implement any existing Sass and HTML template customizations. See [Develop Themes](#) for details.

- b. Create one or more extensions to implement any existing JavaScript, SuiteScript, or JSON customizations. See [Develop Extensions](#) and [Overview](#) for details.
- 4. All customizations should be implemented using themes and extensions. However, if your existing customizations include objects not accessible using the Extensibility API, migrate these customizations to the new source code directory.
 - a. Create a duplicate of your custom module folder in the new source code directory.



Note: This applies to custom modules that are not standard SCA source modules. Any source SCA modules that have been customized may require comparison against the new version plus any required manual edits to the code.

- b. Copy your existing custom modules into the new custom module folder.
 - c. Update the new distro.json file to include references to any custom modules and to define any custom application dependencies.



Important: Do not copy the existing distro.json file contents directly into the new distro.json file. Doing this causes overwrites to changes made in the distro.json file to support the new release. For example, a new module may be referenced. NetSuite recommends evaluating the DIFF between files to ensure that you migrate all changes from your local distro.json file into the new distro.json file.

- d. Any module that changed in the new release bears a higher version number from its previous release. Compare module version numbers to assess which modules have been updated and migrate the changes to any existing customizations as necessary. Refer to the [SuiteCommerce](#), [SuiteCommerce MyAccount](#), and [SuiteCommerce Advanced Release Notes](#) for details on which modules were changed and why.



Important: Migrating to a new release requires testing to ensure any existing customizations remain viable. Evaluate differences in code against your existing customizations and test thoroughly before deploying to your live site. See the help topic [Best Practices for Customizing SCA](#).

- 5. Link your site record and domain to an SSP application as required. This can be the SSP Application for the latest version of SCA or one specifically intended for development.
 - a. Go to Commerce > Hosting > SSP Applications and click **View** next to the SuiteCommerce Advanced Dev <version> application.
 - b. Click **Link to Domain**.
 - c. Select the development domain for this SSP Application.

- 6. Deploy your local files and test appropriately.

Testing may consist of various scenarios depending on your setup. You can deploy your local files to any of the following locations:

- Themes and extensions require the theme or extension developer tools. These are included with the SuiteCommerce Extension Management bundle. This requires setting up the appropriate developer environment. See [Develop Themes](#) and [Develop Extensions](#) for details.
- Core source code customizations require the core SCA developer tools. This requires setting up the appropriate developer environment. See [Set Up SCA Developer Tools](#) for details. Core SCA customizations can be deployed to:
 - A primary SSP Application linked to a primary domain (live site)
 - Development SSP Application linked to a development domain
 - Local server. See the help topic [Test SCA Customizations on a Local Server](#).

- NetSuite sandbox account. See [Deploy to a NetSuite Sandbox](#).
- Custom SSP Application. See [Deploy to a Custom SSP Application](#)

For example, you can deploy your customizations to a local server or to a NetSuite sandbox account for testing. If you do not have a Sandbox account, you can deploy to an SSP Application linked to a development domain in NetSuite. After testing, deploy your files to the SSP Application linked to the primary domain.

Upgrade SuiteCommerce to SuiteCommerce Advanced

Applies to: SuiteCommerce | SuiteCommerce Advanced

This topic explains how to upgrade from SuiteCommerce to SuiteCommerce Advanced (SCA).

To upgrade from SuiteCommerce to SCA, follow these steps:

1. [Test Your Site on a Development Domain](#)
2. [Set Up Your Production Domain for SCA](#)
3. [Verify Production Domain Operation](#)

Before you begin

Before you begin upgrading your site:

- Contact your account representative to provision the appropriate modules for your NetSuite account.
- In NetSuite, go to Commerce > Extensions > Extension Manager, select your web site and domain, and then click **Next**. Make a note of the active themes and extensions on your SuiteCommerce site.

Test Your Site on a Development Domain

It is a best practice to use a development domain to test your existing themes and extensions on an SCA implementation before upgrading your live production domain to SCA. Testing your existing NetSuite and third-party themes and extensions helps ensure that your SCA domain performs as expected when you upgrade your production domain.

To test SCA on your development domain:

1. If you have not already done so, create a development domain in your existing SuiteCommerce web site record.
See the help topic [Set Up Domains for Web Stores](#).
2. Go to Commerce > Websites > Website List.
3. Click **Edit** to access the web site record.
4. On the **Domains** tab, clear the **Primary Web Site URL** checkbox for all of the listed domains.
5. Copy the configuration record from the existing SuiteCommerce production domain to the development domain.
See the help topic [Copy a SuiteCommerce Configuration Record](#).
6. Install the Theme and Extension SuiteApp.
See the help topic [Installing Theme and Extension SuiteApps](#).
7. Link your development domain to the SuiteCommerce Advanced Dev application.
See the help topic [Link a Domain to an SSP Application](#). Use the latest release **SuiteCommerce Advanced — Dev** implementation for all touch points.

8. Activate the themes and extensions on the development domain.
See the help topic [Manage Themes and Extensions](#).
9. Clear (invalidate) the cache for the development **and** production domains.
See the help topic [Cache Invalidation Request](#).

Thoroughly test your themes and extensions on your development domain. When you are satisfied, set up your production domain for SCA.

Set Up Your Production Domain for SCA

After confirming that your site works as expected on the development domain, schedule the optimal time to upgrade your production domain to SCA. It is a best practice to schedule upgrades during a period of off-peak site usage.

To set up your production domain for SCA:

1. Assign the latest release **SuiteCommerce Advanced – Dev** implementation for all touch points (the same touch points you assigned on your development domain).
See the help topic [Select Supported Touch Points](#).

 **Note:** As an alternative, you can also assign touch points directly on the domain record. Using this method overrides the touch points assigned on the Touch Points subtab on the Web Site Setup record.

2. Link your production domain to the **SuiteCommerce Advanced Dev** SSP application.
See the help topic [Link a Domain to an SSP Application](#).
3. Activate the themes and extensions that were previously active in your production domain.
See the help topic [Manage Themes and Extensions](#).
4. Clear (invalidate) the cache for the development and production domains.
See the help topic [Cache Invalidation Request](#).

Verify Production Domain Operation

Go to your production site URL in your browser to test operation of your SCA site.

- Confirm that your touch points (listed below) are correct and accessible in your browser:
 - Log In
 - Log Out
 - Register
 - Customer Center (My Account)
 - Checkout
 - View Cart
 - Home Page
- Confirm that your theme and extension activations work as expected.